Databases

Relational and NoSQL, for Java Developers

Relational and non-relational data stores: modelling, SQL, indexes, transactions, internals, PostgreSQL, MySQL and MongoDB, with diagrams, real queries and execution plans, and production incidents shown from symptom to diagnosis, fix and prevention.

SQL · Normalisation · Indexes · Transactions · MVCC · Query Plans · Replication
MongoDB · Document Modelling · Sharding · CAP · Key-Value · Graph · Incident Playbooks

```
db=# EXPLAIN ANALYZE SELECT * FROM orders WHERE id = 42;
 Index Scan using orders_pkey on orders
   Index Cond: (id = 42)
db=# db.orders.find({_id: 42}).explain()
  winningPlan: { stage: 'IDHACK' }
  executionTimeMillis: 0
```


Maruthi Sreeram

Prepared for Java full stack interview preparation

Maruthi Sreeram

Version basis

Experiments were run on PostgreSQL 16 on a Linux virtual machine (2 CPUs, 8 GB); MySQL and MongoDB output is labelled
Output from other versions or production systems is labelled typical
Prepared for Maruthi Sreeram

How to use this guide

- **Who this is for.** A Java full stack developer who must explain, and diagnose, what the operating system does beneath a running service: process and thread management, memory, files, I/O, containers and synchronisation.
- **Every chapter follows the same steps:** (1) the concept and the problem it solves, (2) the mechanism, shown with diagrams, (3) a worked trace or calculation, (4) a hands-on experiment with real commands, C or Java code, (5) **real-world incidents**, each presented as symptoms, diagnosis, root cause, fix and prevention, and (6) the questions interviewers ask, with answers.
- **How to study.** Read each diagram before the prose and narrate it aloud. Reproduce every experiment on a Linux machine or virtual machine (never on a production host). Treat each incident as an exercise: cover the diagnosis, decide which command to run first, then compare with the text.
- **Running the code.** Save a Java listing as `Main.java` and run `java Main.java`; compile C with `gcc -O0 -o demo demo.c`. Terminal sessions begin with `$`. Output shown was produced on the build machine unless labelled typical.
- **Scope.** The focus is Linux, the platform on which most Java services run, together with the JVM and container behaviour that interacts with it. Windows and macOS are mentioned only where the difference matters.

Contents

Part 1: Relational Databases (SQL) 4

1.1 Data, Databases and Database Management Systems 4
1.2 The Relational Model 12
1.3 Entity-Relationship Modelling 21
1.4 Normalisation 28
1.5 Schema Design and DDL in PostgreSQL and MySQL 38
1.6 SQL Fundamentals 48
1.7 Joins 61
1.8 Aggregation and Grouping 70
1.9 Subqueries and Common Table Expressions 78
1.10 Window Functions 88
1.11 Set Operations, Views, Functions, Procedures and Triggers 95
1.12 NULL, Functions, CASE, Dates and JSON 105
1.13 Thirty Classic SQL Interview Problems 114
1.14 Storage and Pages 125
1.15 Indexes 134
1.16 Reading Query Plans 145
1.17 The Query Planner 153
1.18 Query Optimisation Techniques 159
1.19 Slow Query Diagnosis 168
1.20 Transactions and ACID 174
1.21 Concurrency Anomalies and Isolation Levels 180
1.22 MVCC Internals, VACUUM and Bloat 188
1.23 Locking, Deadlocks and Optimistic Concurrency 199
1.24 Durability: Write-Ahead Logging, Checkpoints and Crash Recovery 206
1.25 Transactions in Java and Spring 214
1.26 Connections and Connection Pooling 223

sreeram

1.27 Replication and High Availability . . . 232
1.28 Backup, Restore and Point-in-Time Recovery . . . 240
1.29 Partitioning and Sharding . . . 245
1.30 Caching and the Database . . . 251
1.31 Schema Migrations and Zero-Downtime Changes . . . 256
1.32 Database Security: Roles, Privileges and Row-Level Security . . . 261
1.33 Monitoring and Diagnosing a Slow Database . . . 265
1.34 Reference Architecture for Scaling and Operations . . . 268

Part 2: Non-Relational Databases (NoSQL) . . . 270

2.1 Why NoSQL . . . 270
2.2 Distributed data fundamentals . . . 276
2.3 Storage engines: B-tree and LSM-tree . . . 290
2.4 Key-value stores . . . 297
2.5 Wide-column stores . . . 301
2.6 Graph databases . . . 306
2.7 Search and time-series stores . . . 310
2.8 Choosing a database . . . 314
2.9 MongoDB Architecture and the Document Model . . . 319
2.10 CRUD in Depth . . . 326
2.11 Query Operators, Arrays, and Embedded Documents . . . 334
2.12 Schema Design for MongoDB . . . 341
2.13 The Aggregation Framework . . . 350
2.14 Indexes and explain in MongoDB . . . 359
2.15 MongoDB Transactions and Consistency . . . 370
2.16 Replication in MongoDB . . . 378
2.17 Sharding in MongoDB . . . 384
2.18 MongoDB Storage and Operations . . . 392
2.19 Spring Data MongoDB . . . 399
2.20 Relational versus Non-Relational: A Head-to-Head Comparison . . . 410
2.21 Interview Question Bank . . . 418
2.22 Cheat Sheets . . . 425
2.23 Six-Week Study Plan . . . 429
2.24 Putting It Together: One Order Through the Whole Data Layer . . . 431

sreeram

PART 1

Relational Databases (SQL)

From the relational model and schema design to indexes, transactions, replication and operations, with PostgreSQL 16 as the reference system and MySQL 8 as the comparison.

In this part

- **1.1** Data, Databases and Database Management Systems
- **1.2** The Relational Model
- **1.3** Entity-Relationship Modelling
- **1.4** Normalisation
- **1.5** Schema Design and DDL in PostgreSQL and MySQL
- **1.6** SQL Fundamentals
- **1.7** Joins
- **1.8** Aggregation and Grouping
- **1.9** Subqueries and Common Table Expressions
- **1.10** Window Functions
- **1.11** Set Operations, Views, Functions, Procedures and Triggers
- **1.12** NULL, Functions, CASE, Dates and JSON
- **1.13** Thirty Classic SQL Interview Problems
- **1.14** Storage and Pages
- **1.15** Indexes
- **1.16** Reading Query Plans
- **1.17** The Query Planner
- **1.18** Query Optimisation Techniques
- **1.19** Slow Query Diagnosis
- **1.20** Transactions and ACID
- **1.21** Concurrency Anomalies and Isolation Levels
- **1.22** MVCC Internals, VACUUM and Bloat
- **1.23** Locking, Deadlocks and Optimistic Concurrency
- **1.24** Durability: Write-Ahead Logging, Checkpoints and Crash Recovery
- **1.25** Transactions in Java and Spring
- **1.26** Connections and Connection Pooling
- **1.27** Replication and High Availability
- **1.28** Backup, Restore and Point-in-Time Recovery
- **1.29** Partitioning and Sharding
- **1.30** Caching and the Database
- **1.31** Schema Migrations and Zero-Downtime Changes
- **1.32** Database Security: Roles, Privileges and Row-Level Security
- **1.33** Monitoring and Diagnosing a Slow Database
- **1.34** Reference Architecture for Scaling and Operations

sreeram

1.1 Data, Databases and Database Management Systems

Every application that outlives a single process must store data somewhere. The first decision is whether the application manages that storage itself, in files of its own design, or delegates it to a **database management system** (DBMS). This chapter defines the vocabulary used throughout the book, explains what a DBMS provides that a directory of files does not, and locates the database within a typical Spring Boot service.

A **database** is an organised collection of related data that models some part of the real world and is stored persistently. A **DBMS** is the software that defines, creates, queries, updates and administers databases and that mediates every access to them. A **database system** is the combination of the DBMS, the database and the applications that use them. In everyday speech "database" often means all three; the distinction matters when diagnosing a problem, because a defect may lie in the data (wrong values), in the schema (missing constraint), in the DBMS configuration (memory, connection limits) or in the application (wrong transaction boundary).

1.1.1 Concept and mechanism

The limits of file-based storage

A file-based design stores each application's data in files whose format only that application understands. Five problems follow, and each corresponds to a service that a DBMS provides.

Problem with files	Consequence	DBMS service
Redundancy and inconsistency	The same customer address is copied into payroll, billing and shipping files and the copies diverge	One logical copy, referential integrity, constraints
Data dependence	Changing a record layout requires changing every program that parses the file	Catalog and schema, data independence
No declarative queries	Each new question requires a new program that scans and filters	Query language (SQL) and a query optimiser
No concurrency control	Two writers overwrite each other, or a reader sees a half-written record	Transactions, locking, MVCC
No atomicity or recovery	A crash in the middle of a multi-file update leaves the data in an unknown state	Write-ahead logging, crash recovery

Problems of file-based storage and the DBMS service that addresses each.

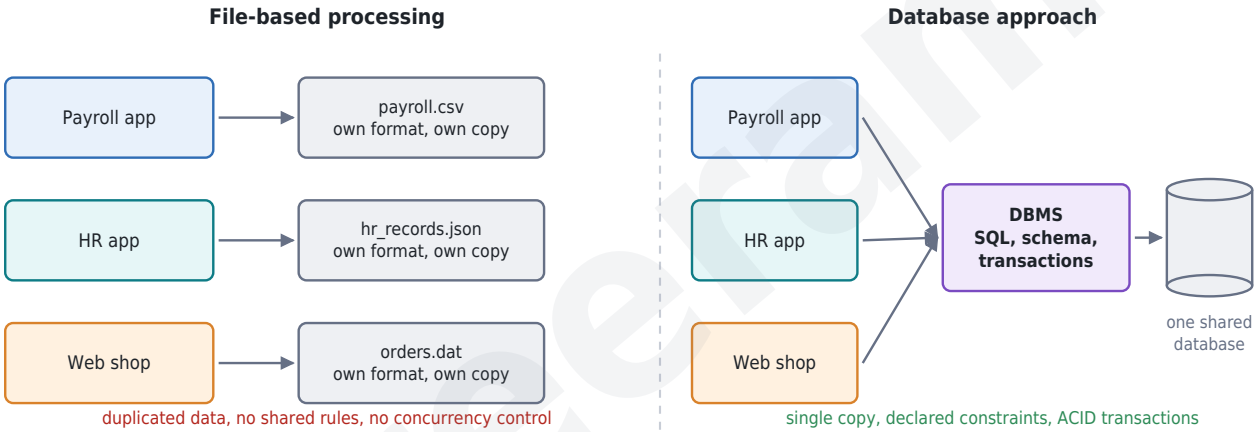


Figure 1.1. In the file-based design each application owns a private copy in its own format; with a DBMS all applications share one database through a single interface that enforces common rules.

The concurrency and atomicity problems are not theoretical. The following Python program is a minimal "bank" that stores a balance in a JSON file. Two processes each perform 2000 deposits of 1 by the sequence read, modify, write.

filebank.py PYTHON

```python
import json
from multiprocessing import Process

PATH = "balance.json"

def deposit(n):
    for _ in range(n):
        with open(PATH) as f:
            data = json.load(f)          # read
        data["balance"] += 1             # modify
        with open(PATH, "w") as f:
            json.dump(data, f)           # write

if __name__ == "__main__":
    with open(PATH, "w") as f:
        json.dump({"balance": 0}, f)
    ps = [Process(target=deposit, args=(2000,)) for _ in range(2)]
    [p.start() for p in ps]
    [p.join() for p in ps]
    with open(PATH) as f:
        print("expected 4000, found", json.load(f))
```

Two writers on one file (output of three consecutive runs was the same; traceback trimmed) TERMINAL

```
$ python3 filebank.py
Process Process-1:
Traceback (most recent call last):
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
expected 4000, found {'balance': 2000}
```

Two distinct failures occur. One process read the file while the other had truncated it for rewriting (a **torn read**) and crashed; and the deposits of the surviving process overwrote the other's progress (a **lost update**). Neither failure is detected by the operating system, because each individual file operation succeeded. A DBMS removes both failures by making "add 1 to the balance" a single atomic, isolated operation:

SQL

```sql
UPDATE acct SET balance = balance + 1 WHERE id = 1;
```

With two concurrent clients issuing 500 such statements each against PostgreSQL, the final balance was exactly 1000 on every run. The mechanism (row locks, transactions) is the subject of the transactions chapters later in this Part.

The three-schema architecture

The ANSI/SPARC architecture separates a database description into three levels so that changes at one level do not ripple through the others.

- The **external level** consists of the views seen by individual applications or user groups. A billing application sees invoices; a support application sees tickets and customer names but not payment data.
- The **conceptual level** describes the whole database logically: tables, columns, types, keys and constraints. The `CREATE TABLE` statements of a project are the conceptual schema.
- The **internal level** describes physical storage: files, pages, indexes, partitioning and compression.

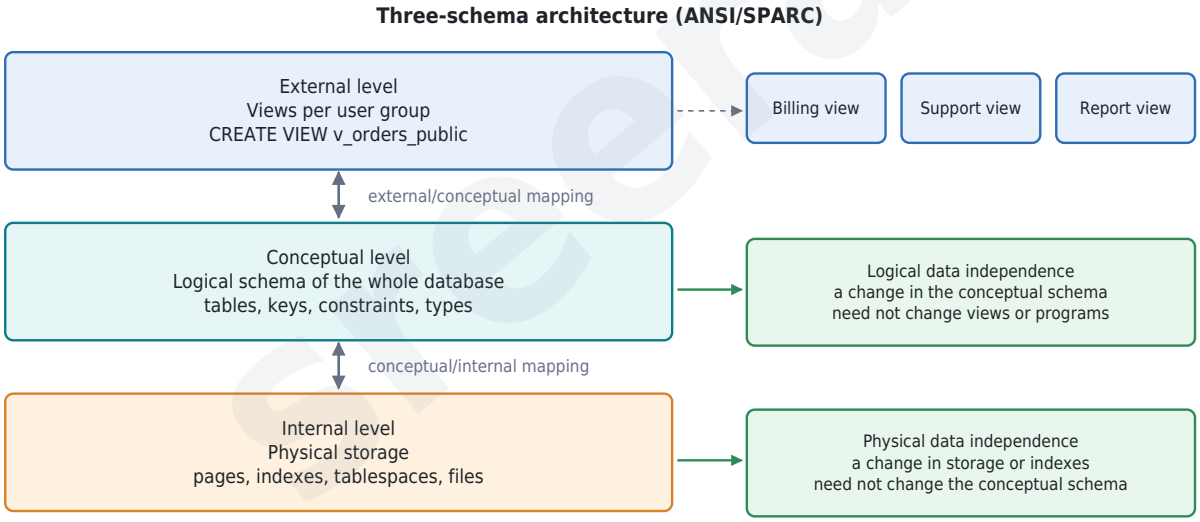


Figure 1.2. The two mappings between levels are what provide data independence: a physical change (a new index, a different storage layout) leaves the conceptual schema intact, and a logical change can often be hidden from programs by redefining a view.

Physical data independence means that adding an index or moving a table to another disk requires no change to queries. **Logical data independence** means that splitting a table into two can be hidden behind a view with the original name, so existing programs continue to work. Logical independence is harder to achieve in practice than physical independence, because views restrict what can be updated through them.

DBMS components

Although products differ in detail, every relational DBMS contains the same functional components. The names below follow PostgreSQL.

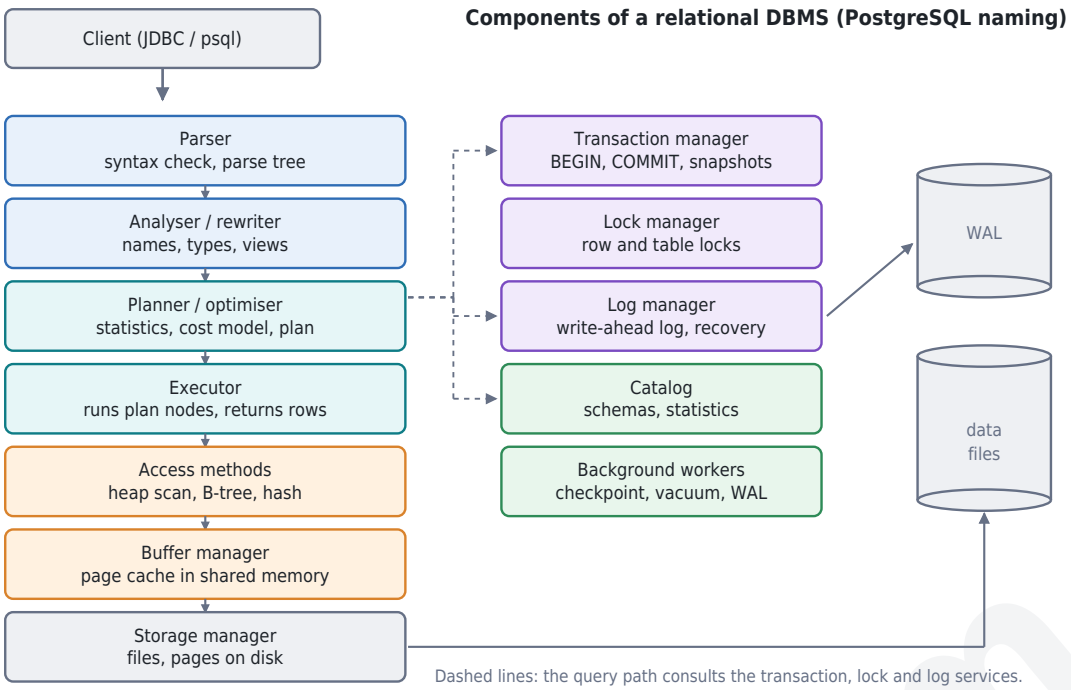


Figure 1.3. A query passes through the parser, rewriter, planner and executor; the executor calls access methods that read pages through the buffer manager; transaction, lock and log services cooperate at run time.

- The **parser** checks syntax and builds a parse tree. The **analyser** resolves table and column names against the **catalog** (the system tables that describe the schema) and checks types.
- The **planner** (query optimiser) enumerates equivalent execution strategies, estimates their cost from statistics stored in the catalog and selects the cheapest. SQL states what result is wanted; the plan states how to compute it.
- The **executor** runs the plan as a tree of operators (scan, join, sort, aggregate), pulling rows from the leaves upward.
- **Access methods** (heap scan, B-tree index scan and others) turn requests for rows into requests for pages.
- The **buffer manager** keeps recently used pages in shared memory; the **storage manager** reads and writes those pages in files.
- The **transaction manager** assigns transaction identifiers and snapshots; the **lock manager** arbitrates conflicting access; the **log manager** writes the write-ahead log (WAL) that makes committed changes durable and recoverable.
- **Background processes** perform checkpoints, clean up dead row versions (vacuum) and ship WAL.

The processes of a running PostgreSQL 16 server can be listed from within the server. The server on the build machine reports:

Background processes of a PostgreSQL 16 instance TERMINAL

```
$ psql -d shop -c "select pid, backend_type from pg_stat_activity order by backend_type"
 pid |         backend_type
-----+------------------------------
 576 | autovacuum launcher
 573 | background writer
 572 | checkpointer
 756 | client backend
 577 | logical replication launcher
 575 | walwriter
(6 rows)
```

One `client backend` exists per connected session; the others are the maintenance processes named in the list above. PostgreSQL uses one operating-system process per connection, which is why connection counts are a capacity concern (see the chapter on connection pooling later in this Part).

OLTP and OLAP workloads

Database workloads fall into two broad families with opposite requirements.

Property	OLTP	OLAP
Typical operation	Insert an order, read one account, update stock	Revenue by region and month over all orders
Rows touched per query	One to a few hundred	Millions to billions
Data scope	Current state	History
Latency target	Milliseconds	Seconds to minutes
Concurrency	Thousands of short transactions	Few long queries
Dominant design	Normalised schema, B-tree indexes, row storage	Star schema, columnar storage, scans

Online transaction processing compared with online analytical processing.

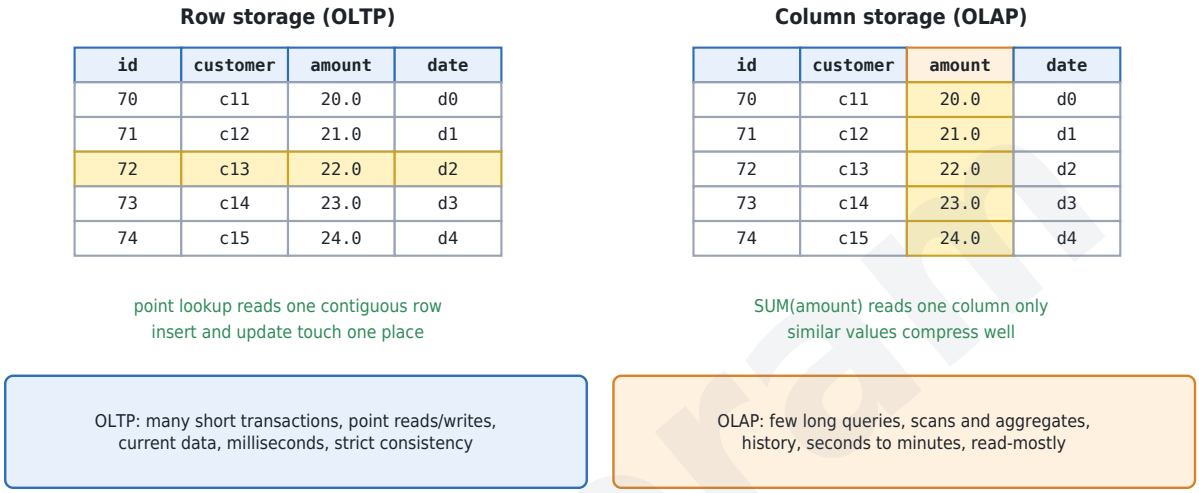


Figure 1.4. Row storage keeps all columns of one row together and favours point reads and writes; column storage keeps each column together and favours scans that need a few columns of many rows.

The difference is measurable even on one PostgreSQL table. A table `sale` with one million rows (87 MB with its primary key index) was queried in two ways.

The same table under an OLTP query and an OLAP query (typical timings, shared build machine) TERMINAL

```
$ psql -d shop
shop=# explain (analyze, buffers, costs off) select * from sale where sale_id = 424242;
 Index Scan using sale_pkey on sale (actual time=0.022..0.022 rows=1 loops=1)
   Index Cond: (sale_id = 424242)
   Buffers: shared hit=7
 Execution Time: 0.038 ms

shop=# explain (analyze, costs off) select product_id, sum(amount) as revenue
shop-#   from sale group by product_id order by revenue desc limit 5;
 Limit (actual time=465.860..465.994 rows=5 loops=1)
   ->  Sort (actual time=465.858..465.990 rows=5 loops=1)
         ->  Finalize HashAggregate (actual time=456.028..457.242 rows=2001 loops=1)
               ->  Gather (actual time=439.558..441.874 rows=6003 loops=1)
                     ->  Partial HashAggregate (actual time=435.114..436.269 rows=2001 loops=3)
                           ->  Parallel Seq Scan on sale (actual time=0.013..102.178 ...)
 Execution Time: 466.065 ms
```

The point lookup read 7 buffers and completed in 0.04 ms; the aggregate scanned all 8,334 pages with three processes and needed about 466 ms. Neither plan is wrong: each is the cheapest way to answer its question. The practical consequence is architectural. Running long analytical queries on the same primary server that handles checkout traffic makes the two workloads compete for CPU, memory and I/O; production systems therefore route analytics to a replica or a separate warehouse.

Where the database sits behind a Spring Boot service

In a typical Spring Boot application the database is a separate process, reached over TCP through a driver and a connection pool.

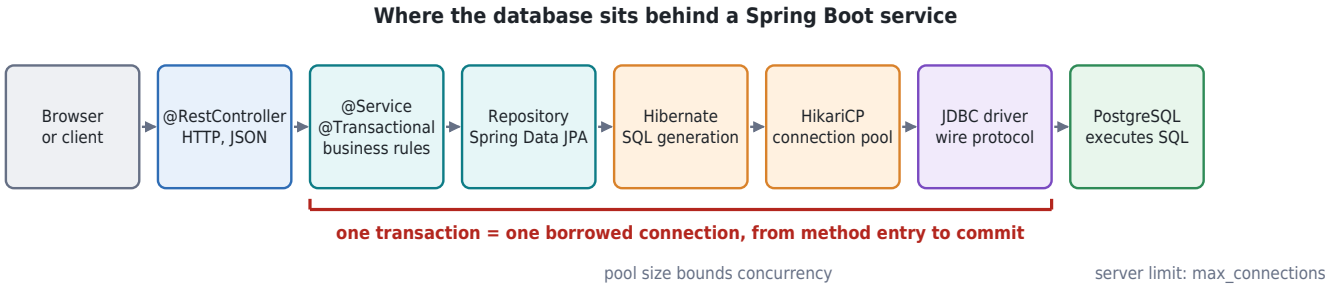


Figure 1.5. A request crosses the controller, service, repository and JPA provider before it reaches the pool and driver; a `@Transactional` service method holds one borrowed connection from entry to commit.

The layers have distinct responsibilities, and many production problems come from confusing them.

- The controller handles HTTP and contains no persistence logic.
- The service layer owns the **transaction boundary**. A method annotated `@Transactional` obtains one connection from the pool when it first touches the database and returns it at commit or rollback.
- The repository (Spring Data JPA) translates method calls into JPQL or SQL through Hibernate.
- **HikariCP** keeps a bounded number of open connections (default `maximumPoolSize` is 10). A thread that needs a connection when none is free waits, so the pool size is an upper limit on concurrent database work from one application instance.
- The **JDBC driver** implements the PostgreSQL or MySQL wire protocol. The database enforces its own limit on connections (`max_connections`, 100 by default in PostgreSQL).

The database is shared state. An application server can be restarted, scaled horizontally or replaced; the database must preserve correctness across all of those events, which is why constraints, transactions and backups belong in the database and not only in application code.

1.1.2 Worked example

The following exercise traces one SQL statement through the components above. Statement: `SELECT full_name FROM customer WHERE email = 'asha@example.com'`, with `customer` having a unique index on `email`.

1. The client sends the text over an existing connection. The backend process serving that connection receives it.
2. The parser produces a parse tree; the analyser looks up `customer` and `email` in the catalog, confirms that `email` is a text column and that the literal can be coerced to text.
3. The planner reads the statistics for `customer`. Because a unique index exists on `email` and the predicate is an equality, the estimated cost of an index scan (a few page reads) is far below that of a sequential scan; the index scan is chosen.
4. The executor asks the access method for tuples matching the key. The B-tree locates the entry; the heap access method fetches the row from its page.
5. Both pages are requested through the buffer manager. If they are in shared memory the request is satisfied without disk access (a buffer hit); otherwise the storage manager reads them from the data file.
6. Before returning the row, the executor checks that the row version is visible to the current transaction's snapshot (MVCC, covered in a later chapter).
7. The row is sent to the client. No WAL record is written because the statement changed nothing; an `UPDATE` would additionally append a WAL record before the data page is modified.

1.1.3 Hands-on

The catalog is itself a database that can be queried. The following session inspects the server and the cluster used for the experiments in this book (PostgreSQL 16.15, build machine).

Querying the system catalog TERMINAL

```
$ psql -d shop
shop=# select name, setting, unit from pg_settings
shop-#  where name in ('block_size','max_connections','shared_buffers',
shop-#                 'default_transaction_isolation');
             name              | setting        | unit
-------------------------------+----------------+------
 block_size                    | 8192           |
 default_transaction_isolation | read committed |
 max_connections               | 30             |
 shared_buffers                | 8192           | 8kB
(4 rows)
```

On this instance `shared_buffers` is 8192 pages of 8 kB (64 MB) and `max_connections` was lowered to 30 for the experiments; the PostgreSQL defaults are 128 MB and 100. The default isolation level `read committed` is explained in the chapter on isolation levels.

1.1.4 Real-world problems and solutions

Incident: lost updates in a shared file

Scenario. A small inventory service keeps stock counts in a JSON file on a shared volume. Two instances of the service run behind a load balancer. The file was adequate for one instance.

Symptoms. Stock counts drift from physical counts by a few units per day. Occasionally one instance fails with `JSONDecodeError` (or `EOFException` in a Java parser) and restarts.

Diagnosis.

1. Confirm that more than one writer exists: list the processes or pods that have the file open (`lsof` on the volume).
2. Reproduce with a stress script that performs a known number of increments from two processes and compares the final value with the expected value (the `filebank.py` program above).
3. Compare the file modification times with the application log: interleaved read-modify-write cycles appear as two writes within the same second carrying stale values.

Root cause. Read-modify-write on a file has no isolation. The operating system guarantees that single writes complete, not that a sequence of read and write is atomic with respect to other processes. Truncating and rewriting also exposes a window in which the file is empty.

Fix. Immediate: restrict to one instance and write through a temporary file followed by an atomic `rename`. Permanent: move the state to a DBMS and express the change as one statement (`UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty > 0`), which the engine executes atomically under a row lock.

Prevention. State shared between processes belongs in a system that provides transactions. A file is acceptable only for data owned by exactly one process.

Incident: an analytical report slows the checkout

Scenario. A nightly sales report is moved to run on demand. During business hours users start it from a dashboard.

Symptoms. Checkout latency rises from 30 ms to several seconds while reports run; `pg_stat_activity` shows many `active` backends.

Diagnosis.

1. List running statements by duration: `SELECT pid, now() - query_start AS runtime, state, left(query, 60) FROM pg_stat_activity WHERE state <> 'idle' ORDER BY runtime DESC;`
2. Identify the long-running aggregates scanning large tables.
3. Run `EXPLAIN (ANALYZE, BUFFERS)` on the report statement during a quiet period and note the number of buffers read: the experiment above shows 8,334 pages for one million rows.

Root cause. An OLAP query and OLTP transactions share one server's CPU, buffer cache and I/O bandwidth. Large scans also evict hot pages from the buffer cache.

Fix. Immediate: cancel the report (`SELECT pg_cancel_backend(pid)`) and schedule it off-peak. Permanent: run the report against a read replica or an extract table maintained by a batch job; add `statement_timeout` for the reporting role.

Prevention. Separate the workloads by role and by server, and give reporting users a role with a statement timeout and a connection limit.

1.1.5 Questions to expect

What is the difference between a database and a DBMS?

The database is the stored data together with its schema. The DBMS is the software that manages it: it parses and optimises queries, enforces constraints, coordinates concurrent access, and recovers after failures. PostgreSQL is a DBMS; the `shop` database it hosts is a database.

Why not store data in files?

Files provide no concurrency control, no atomic multi-record updates, no declarative queries, no constraint enforcement and no crash recovery. The application would have to implement each of these correctly, which is the work a DBMS has already done and tested.

Explain the three-schema architecture and data independence.

External level (views), conceptual level (logical schema) and internal level (physical storage). Mappings between the levels allow a change at one level without changing the level above it: physical independence for storage changes, logical independence for schema changes hidden behind views.

Distinguish OLTP from OLAP and say how it affects design.

OLTP serves many short transactions on current data and favours normalised schemas and B-tree indexes. OLAP serves few long queries over history and favours denormalised star schemas, columnar storage and scans. Mixing both on one server causes interference, so they are usually separated.

What are the main components of a DBMS?

Parser, analyser, planner/optimiser, executor, access methods, buffer manager, storage manager, transaction manager, lock manager, log manager and catalog, plus background workers for checkpoints and cleanup.

Where does the transaction boundary belong in a Spring application?

In the service layer, on the method that represents one business operation. The repository layer is too fine-grained (each call would commit separately) and the controller too coarse and too close to HTTP.

Why does connection pool size matter?

Each connection is a server process in PostgreSQL and consumes memory; the pool bounds concurrent database work per application instance, and the sum over all instances must stay below `max_connections`.

KEY POINTS

Key points

- A DBMS replaces per-application files with shared, declared structure, giving integrity, concurrent access, atomic updates and recovery.
- The three-schema architecture (external, conceptual, internal) provides physical and logical data independence.
- A query passes parser, analyser, planner, executor and access methods; the planner chooses a plan from statistics, and the plan, not the SQL text, determines performance.
- OLTP and OLAP workloads need different designs; separate them in production.
- In a Spring Boot service the transaction boundary sits on the service method and holds one pooled connection for its duration; pool size and `max_connections` bound concurrency.

1.2 The Relational Model

Before 1970 databases were navigated: programs followed pointers from record to record, and a change of storage layout broke the programs. E. F. Codd's relational model replaced pointers with a small mathematical structure, the **relation**, and replaced navigation with declarative operations on relations. Every mainstream SQL system, PostgreSQL and MySQL included, is an imperfect implementation of that model. Understanding the model explains why SQL behaves as it does, in particular why duplicates, ordering and NULL need separate attention.

1.2.1 Concept and mechanism

Relations, tuples and attributes

A **relation schema** is a name with a set of attributes, each attribute drawn from a **domain** (a set of permitted values). A **relation** is a finite set of **tuples**, each tuple assigning one value from the domain to every attribute. SQL calls these table, row and column. The terminology is summarised below.

Relational term	SQL term	Meaning
Relation	Table	A set of tuples over a schema
Tuple	Row	One fact: a value for each attribute
Attribute	Column	A named property with a domain
Domain	Data type plus constraints	The values an attribute may take
Degree	Number of columns	Arity of the relation
Cardinality	Number of rows	Size of the relation

Relational and SQL vocabulary.

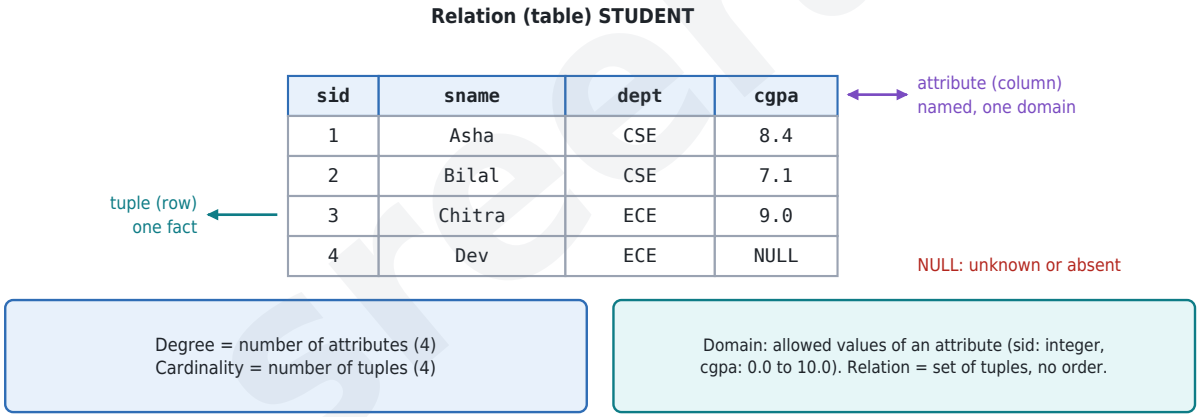


Figure 1.6. A relation is a set of tuples over named attributes; degree counts columns and cardinality counts rows.

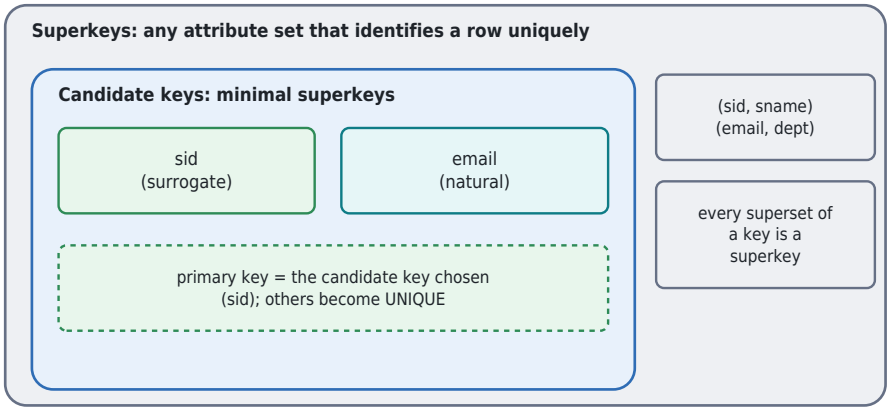
Two properties follow from the definition and differ from what a spreadsheet suggests. First, a relation is a **set**: tuples have no order and no duplicates. Second, every attribute value is atomic from the point of view of the model (first normal form, treated in the normalisation chapter). SQL deviates in two places: a table may contain duplicate rows unless a key prevents them (a table is a **bag**), and the query language has the special marker NULL. Both deviations are sources of defects covered below.

Keys

A **key** is a set of attributes that identifies tuples. The terms form a hierarchy.

- A **superkey** is any set of attributes whose values are unique across the relation. `{sid}`, `{sid, sname}` and `{email, dept}` are all superkeys of a student relation.
- A **candidate key** is a minimal superkey: removing any attribute destroys uniqueness. `{sid}` and `{email}` are candidate keys; `{sid, sname}` is not minimal.
- The **primary key** is the one candidate key chosen as the main identifier; it may not contain NULL. The remaining candidate keys are **alternate keys** and are declared `UNIQUE`.
- A **composite key** has more than one attribute, for example `(order_id, line_no)`.

- A **foreign key** is a set of attributes in one relation that references a candidate key (normally the primary key) of another relation (or the same one).
- A **surrogate key** is an identifier with no business meaning generated by the system (identity column, sequence, UUID). A **natural key** is formed from real-world data (email, ISBN, national identifier).



sid	email
1	a@x.in
2	b@x.in
3	c@x.in

Alternate key: email
Foreign key: a column that references a key of another table

Figure 1.7. Candidate keys are the minimal superkeys; one is chosen as primary key, the others become unique constraints, and a foreign key is a reference to a key of another table.

The choice between natural and surrogate primary keys is a design decision with consequences, summarised below.

Aspect	Natural key	Surrogate key
Stability	Can change (an email address changes; a name is corrected) and every referencing row must change with it	Never changes, so references stay valid
Size	Often wide text, repeated in every foreign key and index	Compact (4 or 8 bytes)
Meaning	Self-describing; usable in URLs and exports	None; requires a join to interpret
Uniqueness	Guaranteed only if the business rule is truly unique	Unique by construction, so the business rule must still be declared as a `UNIQUE` constraint on the natural attribute

Natural keys compared with surrogate keys.

The rule used in this book: use a surrogate primary key for entities whose natural identity may change, and always declare the natural candidate keys as `UNIQUE NOT NULL`. A surrogate key does not remove the need for the natural unique constraint; omitting it is the cause of the duplicate-rows incident later in this chapter.

Integrity constraints

An integrity constraint is a rule that every state of the database must satisfy; the DBMS rejects any change that would violate it.

Constraint	Rule	SQL
Domain	Value belongs to the type and range	Column type, `CHECK (rating BETWEEN 1 AND 5)`
Not null	Attribute must have a value	`NOT NULL`
Entity integrity	Primary key is unique and not NULL	`PRIMARY KEY`
Uniqueness	No two rows share the value	`UNIQUE`
Referential integrity	Each foreign key value matches a key value in the parent, or is NULL	`FOREIGN KEY ... REFERENCES`
Business rule	Any row-level predicate	`CHECK`

Integrity constraints in SQL.

Referential integrity governs what happens when a parent row is deleted or its key is updated. The options are `NO ACTION` or `RESTRICT` (refuse, the default), `CASCADE` (apply the change to children), `SET NULL` and `SET DEFAULT`. `CASCADE`

is appropriate for rows that cannot exist without the parent (order lines of an order) and dangerous elsewhere: deleting a customer must not silently delete the financial history.

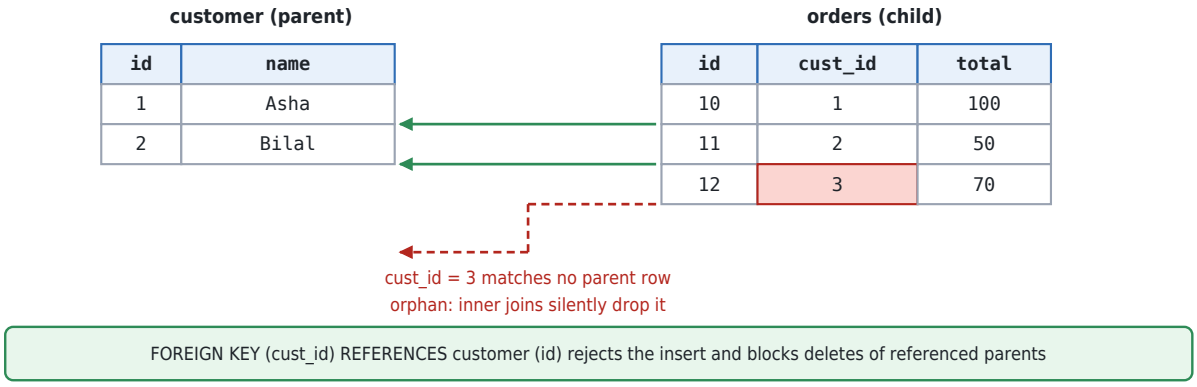


Figure 1.8. A foreign key rejects a child row whose value matches no parent row; without the constraint such an orphan stays in the table and disappears from inner joins.

Relational algebra

Relational algebra is the set of operations on relations that defines what a query means. Each operator takes one or two relations and returns a relation, so operators compose. SQL is a declarative language whose semantics are given by this algebra, and an optimiser rewrites SQL into equivalent algebra expressions.

Operator	Meaning	SQL
Selection (sigma)	Keep tuples satisfying a predicate	`WHERE`
Projection (pi)	Keep chosen attributes, remove duplicates	`SELECT DISTINCT cols`
Rename (rho)	Rename a relation or attribute	`AS`
Union	Tuples in either relation	`UNION`
Difference	Tuples in the first and not the second	`EXCEPT (MINUS in Oracle)`
Intersection	Tuples in both	`INTERSECT`
Cartesian product	All pairs of tuples	`CROSS JOIN`
Join	Product followed by selection on a condition	`JOIN ... ON`
Division	Tuples related to all tuples of another relation	`NOT EXISTS twice, or GROUP BY ... HAVING count`

Relational algebra operators and their SQL equivalents.

Union, intersection and difference require **union-compatible** operands: the same number of attributes with compatible types. The join is defined as a selection over a product, but an implementation never builds the product; join algorithms are the subject of the query processing chapters.

1.2.2 Worked example

The relations used below are `student(sid, sname, dept, cgpa)`, `course(cid, title, credits)` and `enrol(sid, cid, grade)` with primary key `(sid, cid)` and foreign keys to both parents. The data was loaded into PostgreSQL 16 and every result is real output.

university.sql — SQL

```sql
CREATE TABLE student (sid int PRIMARY KEY, sname text NOT NULL, dept text, cgpa numeric(3,1));
CREATE TABLE course  (cid text PRIMARY KEY, title text NOT NULL, credits int NOT NULL);
CREATE TABLE enrol   (sid int REFERENCES student, cid text REFERENCES course, grade text,
                      PRIMARY KEY (sid, cid));
INSERT INTO student VALUES (1,'Asha','CSE',8.4),(2,'Bilal','CSE',7.1),(3,'Chitra','ECE',9.0),
                           (4,'Dev','ECE',NULL),(5,'Esha','MECH',6.8);
INSERT INTO course VALUES ('DB101','Databases',4),('OS201','Operating Systems',4),
                          ('NW301','Networks',3);
INSERT INTO enrol VALUES (1,'DB101','A'),(1,'OS201','B'),(1,'NW301','A'),(2,'DB101','C'),
                         (3,'DB101','A'),(3,'OS201','A'),(3,'NW301','B'),(4,'OS201',NULL);
```

Selection, projection and join

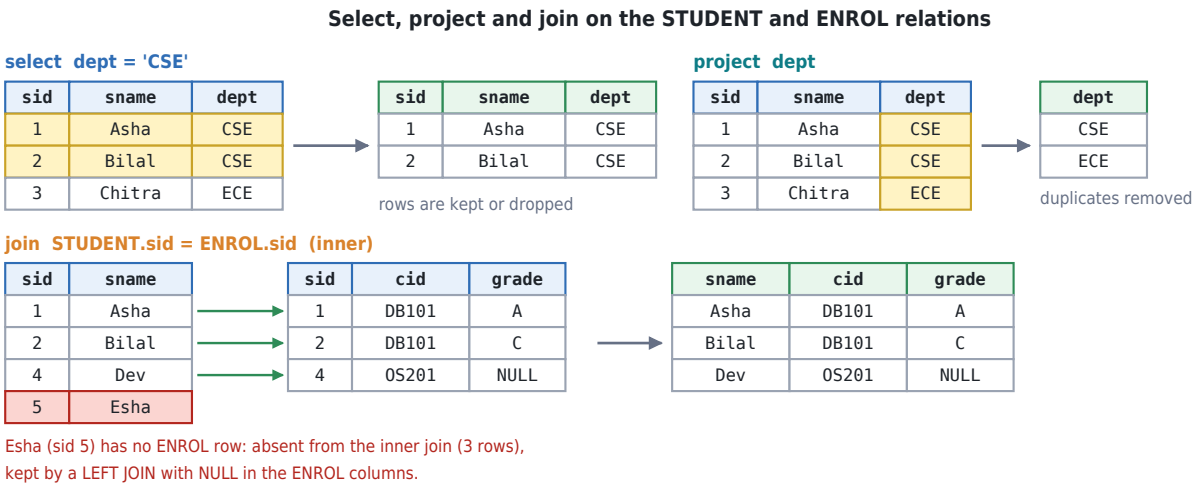


Figure 1.9. Selection removes rows, projection removes columns and duplicates, and an inner join pairs rows with equal key values; the student without enrolments disappears from the inner join.

Selection of CSE students and projection onto the department, with the set semantics of the algebra (`DISTINCT`) compared with the bag semantics of plain SQL:

Selection and projection TERMINAL

```
$ psql -d uni
uni=# select * from student where dept = 'CSE';
 sid | sname | dept | cgpa
-----+-------+------+------
   1 | Asha  | CSE  |  8.4
   2 | Bilal | CSE  |  7.1
(2 rows)

uni=# select dept from student;          -- bag: 5 rows, duplicates kept
 dept
------
 CSE
 CSE
 ECE
 ECE
 MECH
(5 rows)

uni=# select distinct dept from student; -- set: 3 rows
 dept
------
 CSE
 MECH
 ECE
(3 rows)
```

The order of the `DISTINCT` result is not defined: PostgreSQL chose a hash-based plan here. A relation has no order, and SQL guarantees an order only with `ORDER BY`.

The join of students with their enrolments:

Inner join and left join TERMINAL

```
uni=# select s.sname, e.cid, e.grade from student s join enrol e on e.sid = s.sid order by 1,2;
 sname  |  cid  | grade
--------+-------+--------
 Asha   | DB101 | A
 Asha   | NW301 | A
 Asha   | OS201 | B
 Bilal  | DB101 | C
 Chitra | DB101 | A
 Chitra | NW301 | B
 Chitra | OS201 | A
 Dev    | OS201 | (null)
```

Inner join and left join (continued) TERMINAL

```
(8 rows)

uni=# select s.sname from student s left join enrol e on e.sid = s.sid where e.cid is null;
 sname
-------
 Esha
(1 row)
```

Esha has no enrolment, so the inner join loses her; the left join with an `IS NULL` test on the child key finds exactly the unmatched parents (an **anti-join**).

Union, intersection and difference

Let `A` be the students enrolled in DB101 and `B` those enrolled in OS201:

Set operators on sid TERMINAL

```
uni=# select sid from enrol where cid='DB101' union select sid from enrol where cid='OS201';
 sid : 1, 2, 3, 4      (4 rows)
uni=# select sid from enrol where cid='DB101' intersect select sid from enrol where cid='OS201';
 sid : 1, 3            (2 rows)
uni=# select sid from enrol where cid='DB101' except select sid from enrol where cid='OS201';
 sid : 2               (1 row)
```

The listing above condenses the psql tables to one line each; the row values are the real results. `UNION`, `INTERSECT` and `EXCEPT` remove duplicates; the variants `UNION ALL`, `INTERSECT ALL` and `EXCEPT ALL` keep them and are cheaper because they skip the duplicate elimination.

Division

Division answers "for all" questions: which students are enrolled in every course? Let `E = enrol(sid, cid)` and `C = course(cid)`. Then `E / C` contains every `sid` such that, for each `cid` in `C`, the pair `(sid, cid)` occurs in `E`.

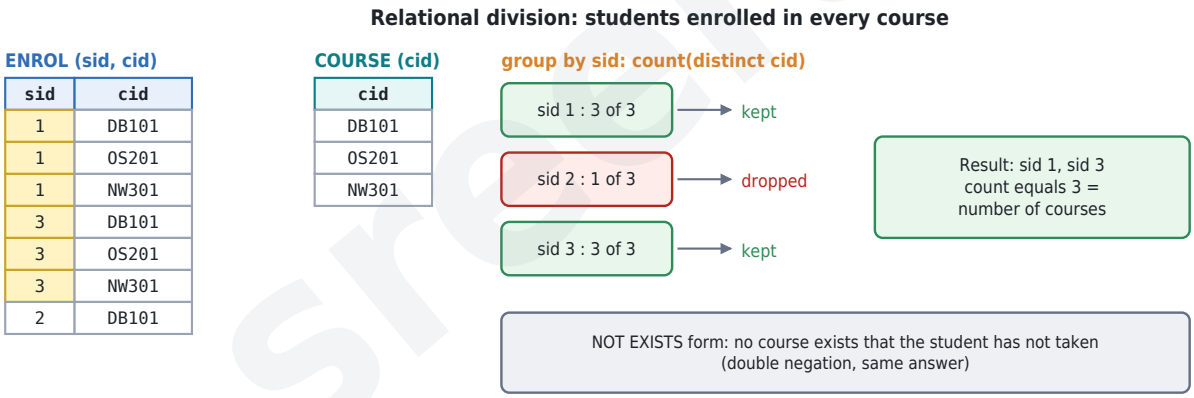


Figure 1.10. Division by counting: a student qualifies when the number of distinct courses taken equals the number of courses in COURSE; students 1 and 3 qualify.

SQL

```sql
-- counting form
SELECT sid FROM enrol
GROUP BY sid
HAVING count(DISTINCT cid) = (SELECT count(*) FROM course);

-- double-negation form: no course exists that the student has not taken
SELECT s.sid, s.sname FROM student s
WHERE NOT EXISTS (
  SELECT 1 FROM course c
  WHERE NOT EXISTS (
    SELECT 1 FROM enrol e WHERE e.sid = s.sid AND e.cid = c.cid));
```

Both forms returned `sid` 1 (Asha) and 3 (Chitra). The counting form is shorter and usually faster, but it is correct only when `enrol` is restricted to cids present in `course` (guaranteed here by the foreign key); the double-negation form is correct for any data and states the "for all" meaning directly.

NULL and three-valued logic

NULL is not a value. It is a marker meaning that the value is unknown or inapplicable, and any comparison with it yields a third truth value, **UNKNOWN**. A `WHERE` clause keeps a row only when the predicate evaluates to TRUE; UNKNOWN is discarded exactly like FALSE.

a	b	a AND b	a OR b	NOT a
T	U	U	T	F
F	U	F	U	T
U	U	U	U	U
U	T	U	T	U
U	F	F	U	U

Truth tables of three-valued logic (T true, F false, U unknown).

The results can be verified in PostgreSQL:

Three-valued logic and aggregates TERMINAL

```
uni=# select null = null as eq, null is null as isnull, (null::boolean or true) as or_true,
uni-#        (null::boolean and false) as and_false, (null::boolean and true) as and_true;
   eq   | isnull | or_true | and_false | and_true
--------+--------+---------+-----------+----------
 (null) | t      | t       | f         | (null)

uni=# select sname, cgpa from student where cgpa > 7;          -- Dev (NULL) is excluded
 Asha 8.4, Bilal 7.1, Chitra 9.0
uni=# select sname, cgpa from student where not (cgpa > 7);    -- Dev is excluded here too
 Esha 6.8

uni=# select count(*) as all_rows, count(cgpa) as non_null, avg(cgpa) from student;
 all_rows | non_null |        avg
----------+----------+--------------------
        5 |        4 | 7.8250000000000000
```

Four consequences must be remembered. `x = NULL` is never true; the test is `x IS NULL`. `cgpa > 7` and `NOT (cgpa > 7)` do not partition the table: Dev satisfies neither. `count(*)` counts rows while `count(column)` counts non-NULL values, and `avg` ignores NULL (7.825 is the mean of four values, not five). Finally, `x NOT IN (list)` returns no rows at all if the list contains a NULL, because for every `x` the comparison with that member is UNKNOWN:

The NOT IN trap TERMINAL

```
uni=# select sname from student where dept not in ('CSE', null);
 sname
-------
(0 rows)
```

Esha (MECH) is not in CSE, yet she is not returned. Use `NOT EXISTS`, or filter NULL out of the subquery, whenever the compared set may contain NULL. The operator `IS DISTINCT FROM` treats NULL as an ordinary comparable value: `cgpa IS DISTINCT FROM 9.0` returned Asha, Bilal, Dev and Esha (four rows), including Dev with NULL.

1.2.3 Hands-on

Constraints can be observed rejecting bad data. Using the `ecom` database defined in the next chapter, four invalid statements were issued and PostgreSQL returned the following errors (real output).

Constraint violations TERMINAL

```
ecom=# insert into order_item values (99,1,1,1,10);
ERROR:  insert or update on table "order_item" violates foreign key constraint
        "order_item_order_id_fkey"
DETAIL:  Key (order_id)=(99) is not present in table "orders".

ecom=# insert into product (sku,name,category_id,price) values ('PH-100','Dup',2,1);
ERROR:  duplicate key value violates unique constraint "product_sku_key"
DETAIL:  Key (sku)=(PH-100) already exists.

ecom=# insert into order_item values (2,2,3,0,10);
ERROR:  new row for relation "order_item" violates check constraint "order_item_quantity_check"
DETAIL:  Failing row contains (2, 2, 3, 0, 10.00).
```

Constraint violations (continued) TERMINAL

```
ecom=# delete from customer where customer_id=1;
ERROR:  update or delete on table "customer" violates foreign key constraint
        "orders_customer_id_fkey" on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

In Java these failures surface as `java.sql.SQLException` (Spring translates them to `DataIntegrityViolationException`); the SQLSTATE codes `23503` (foreign key), `23505` (unique), `23514` (check) and `23502` (not null) identify the class of violation without parsing the message.

1.2.4 Real-world problems and solutions

Incident: orphan rows because the foreign key was never declared

Scenario. A legacy order table was created without foreign keys "for faster inserts". Customers are deleted by a clean-up job.

Symptoms. The finance report totals are lower than the payment provider's totals. Support finds orders that open with an empty customer name.

Diagnosis. Compare the unfiltered total with the total after joining to the parent, then list the orphans with an anti-join. On the reproduction (`ord_nofk` without constraint, customer 2 deleted):

Orphans found by anti-join TERMINAL

```
ddl=# select o.id, o.cust_id from ord_nofk o
ddl-#   left join cust c on c.id = o.cust_id where c.id is null;
 id | cust_id
----+---------
 11 |       2
 12 |       3

ddl=# select count(*) as joined_orders, sum(o.total) as joined_total,
ddl-#        (select sum(total) from ord_nofk) as real_total
ddl-#   from ord_nofk o join cust c on c.id = o.cust_id;
 joined_orders | joined_total | real_total
---------------+--------------+------------
             1 |          100 |        220
```

Root cause. Nothing prevented deleting a parent that still had children, or inserting a child with a non-existent parent (order 12 references customer 3, who never existed). Inner joins then drop the orphans silently, so reports built on joins disagree with reports built on the single table.

Fix. Immediate: add the constraint without blocking on dirty data, repair, then validate. A plain `ADD FOREIGN KEY` on a table with orphans fails and leaves the table unprotected:

Adding a foreign key to dirty data TERMINAL

```
ddl=# alter table ord_nofk add constraint ord_cust_fk
ddl-#   foreign key (cust_id) references cust(id);
ERROR:  insert or update on table "ord_nofk" violates foreign key constraint "ord_cust_fk"
DETAIL:  Key (cust_id)=(2) is not present in table "cust".

ddl=# alter table ord_nofk add constraint ord_cust_fk
ddl-#   foreign key (cust_id) references cust(id) not valid;
ALTER TABLE
ddl=# delete from ord_nofk where id in (select o.id from ord_nofk o left join cust c
ddl-#                                   on c.id = o.cust_id where c.id is null);
DELETE 2
ddl=# alter table ord_nofk validate constraint ord_cust_fk;
ALTER TABLE
ddl=# insert into ord_nofk values (13,9,1);
ERROR:  insert or update on table "ord_nofk" violates foreign key constraint "ord_cust_fk"
```

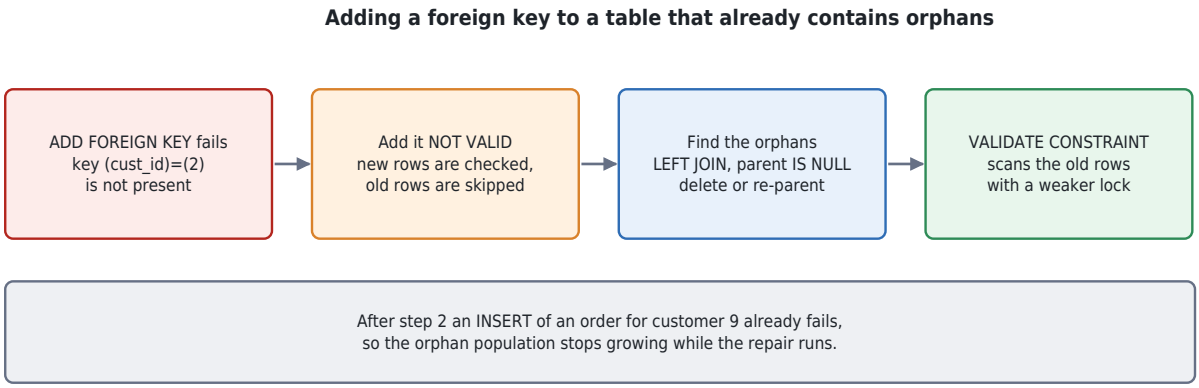


Figure 1.11. `NOT VALID` *makes the constraint effective for new rows immediately; the existing rows are checked later by* `VALIDATE CONSTRAINT`*, so orphan creation stops before the repair finishes. (In production the orphans are archived or re-parented rather than deleted.)*

Prevention. Declare every relationship as a foreign key and index the referencing column; choose `ON DELETE` behaviour deliberately; add a periodic orphan-detection query to the data quality checks of any legacy table that cannot be constrained.

Incident: duplicate rows because a natural key was not declared unique

Scenario. A subscription table has a surrogate `id` primary key and an `email` column. The sign-up endpoint checks `SELECT ... WHERE email = ?` before inserting.

Symptoms. Customers receive two copies of every newsletter and the unique-subscriber metric is inflated. A customer reports that "A@x.com" and "a@x.com" are both charged.

Diagnosis. Group by the normalised natural key and look for counts above one.

Finding duplicates TERMINAL

```
ddl=# select lower(email) as e, count(*) from subs group by 1 having count(*) > 1;
    e    | count
---------+-------
 a@x.com |     3
```

Root cause. The surrogate key is unique for every row by construction, so it never rejects a duplicate natural value. The check-then-insert pattern in the application is a race: two concurrent requests both find no row and both insert. The mixed-case variant escapes a case-sensitive comparison.

Fix. Remove the duplicates, keeping the oldest row, then create a unique index on the normalised value. Creating the index first fails and names the offending key:

Deduplicate, then enforce TERMINAL

```
ddl=# create unique index subs_email_lower_uq on subs (lower(email));
ERROR:  could not create unique index "subs_email_lower_uq"
DETAIL:  Key (lower(email))=(a@x.com) is duplicated.

ddl=# delete from subs where id in (
ddl(#   select id from (select id, row_number() over (partition by lower(email) order by id) rn
ddl(#                   from subs) t where rn > 1);
DELETE 2
ddl=# create unique index subs_email_lower_uq on subs (lower(email));
CREATE INDEX
ddl=# insert into subs (email) values ('B@X.COM');
ERROR:  duplicate key value violates unique constraint "subs_email_lower_uq"
DETAIL:  Key (lower(email))=(b@x.com) already exists.
```

The Spring application should catch `DataIntegrityViolationException` and treat it as "already subscribed"; the database is the arbiter, and the earlier existence check becomes an optimisation.

Prevention. Every natural candidate key receives a `UNIQUE` constraint. Never rely on application-level checks for uniqueness. In MySQL a unique index on a text column with a case-insensitive collation (the default `utf8mb4_0900_ai_ci`) is already case-insensitive; PostgreSQL needs the `lower()` expression index, the `citext` extension or a nondeterministic collation.

1.2.5 Questions to expect

What is the difference between a primary key, a candidate key and a unique key?

A candidate key is any minimal set of attributes that identifies rows. The primary key is the chosen candidate key and may not be NULL. Other candidate keys are declared `UNIQUE`; a unique column may contain NULL (PostgreSQL and MySQL allow several NULLs in a unique index, because NULL = NULL is not true).

Natural or surrogate primary key?

Surrogate keys are stable, compact and cheap to reference; natural keys can change and are wide. Use a surrogate primary key and declare the natural key as unique. Natural keys remain a good primary key for stable code tables (country code).

What is a foreign key and what does `ON DELETE CASCADE` do?

A foreign key is a column set that must match a key of the referenced table or be NULL. `ON DELETE CASCADE` deletes the child rows when the parent row is deleted. It suits dependent rows (order lines) and is unsafe for independent history.

Why does `NOT IN` sometimes return no rows?

If the subquery or list contains NULL, `x NOT IN (...)` evaluates to UNKNOWN for every `x` that is not matched, and UNKNOWN rows are filtered out. Use `NOT EXISTS`.

What is the difference between `count(*)` and `count(col)`?

`count(*)` counts rows. `count(col)` counts rows where `col` is not NULL. `count(DISTINCT col)` counts distinct non-NULL values.

How is relational division expressed in SQL?

With a double `NOT EXISTS` (no required item is missing) or with `GROUP BY ... HAVING count(DISTINCT item) = (SELECT count(*) FROM required)`.

Is a SQL table a relation?

Not exactly. A relation is a set (no duplicates, no order); a table is a bag unless a key prevents duplicates, and it may contain NULL, which has no counterpart in the pure model.

What is the difference between `UNION` and `UNION ALL`?

`UNION` removes duplicates (an extra sort or hash); `UNION ALL` concatenates. Use `UNION ALL` when duplicates are impossible or acceptable.

KEY POINTS

Key points

- A relation is a set of tuples over named, typed attributes; SQL tables add duplicates, ordering through `ORDER BY` only, and NULL.
- Superkey, candidate key, primary key, alternate key, composite key, surrogate key and foreign key form a hierarchy; every natural candidate key needs a `UNIQUE NOT NULL` constraint even when a surrogate primary key exists.
- Integrity constraints (not null, key, unique, foreign key, check) belong in the database; an undeclared foreign key produces orphans that inner joins hide.
- Relational algebra gives SQL its meaning: selection, projection, join, union, difference, intersection, division.
- NULL yields UNKNOWN: use `IS NULL`, beware `NOT IN`, and remember that aggregates ignore NULL.

sreeram

1.3 Entity-Relationship Modelling

A schema should be designed from the facts the business needs to record, not from the screens of the first application. **Entity-relationship (ER) modelling** is the technique for doing so: the designer identifies the things of interest (entities), their properties (attributes) and the associations between them (relationships), draws the result, and only then translates it to tables. Errors found on the diagram cost minutes; the same errors found after production data exists cost migrations.

1.3.1 Concept and mechanism

Entities, attributes and relationships

An **entity** is a distinguishable thing about which data is kept (a customer, a product). An **entity type** is the class of such things; an **entity set** is the collection of current instances. An **attribute** is a property of an entity. A **relationship** associates two or more entities (a customer places an order).

Attributes are classified as follows.

- **Simple** versus **composite**: `city` is simple; `address` is composite (street, city, postal code) and is normally split into columns.
- **Single-valued** versus **multivalued**: a customer may have several phone numbers. A multivalued attribute becomes a separate table (or, exceptionally, an array or JSON column).
- **Stored** versus **derived**: `age` can be computed from `birth_date` and is not stored; an order `total` derived from lines is a deliberate denormalisation if stored.
- **Key** attributes identify an entity instance.

Entity-relationship notation (Chen style) for STUDENT enrols in COURSE

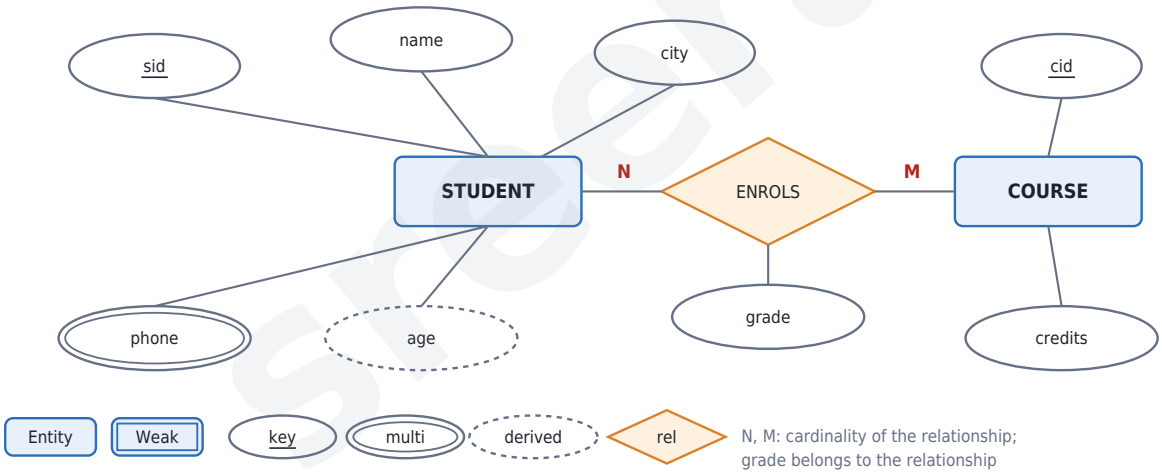


Figure 1.12. In the Chen notation entities are rectangles, attributes are ellipses (underlined for keys, double for multivalued, dashed for derived) and relationships are diamonds labelled with cardinalities; an attribute can belong to a relationship (grade).

Cardinality and participation

The **cardinality ratio** of a binary relationship states how many instances of one entity may relate to one instance of the other.

Ratio	Example	Meaning
One-to-one (1:1)	user and user_profile	each user has at most one profile and each profile belongs to one user
One-to-many (1:N)	customer and orders	a customer places many orders; an order has exactly one customer
Many-to-many (M:N)	customer and product (wishlist)	a customer lists many products; a product is listed by many customers

Cardinality ratios.

Participation states whether every instance must take part. Participation is **total** when each order must have a customer (the foreign key is `NOT NULL`) and **partial** when it need not (a product may have no review). Cardinality and participation together decide the column constraints: the foreign key side, its nullability and uniqueness.

A **weak entity** has no key of its own and is identified through an owner entity. An order line is identified by its order and a line number; it cannot exist without the order. The relationship to the owner is called the identifying relationship.

Mapping an ER diagram to tables

The translation is mechanical, which is why the diagram is worth drawing first.

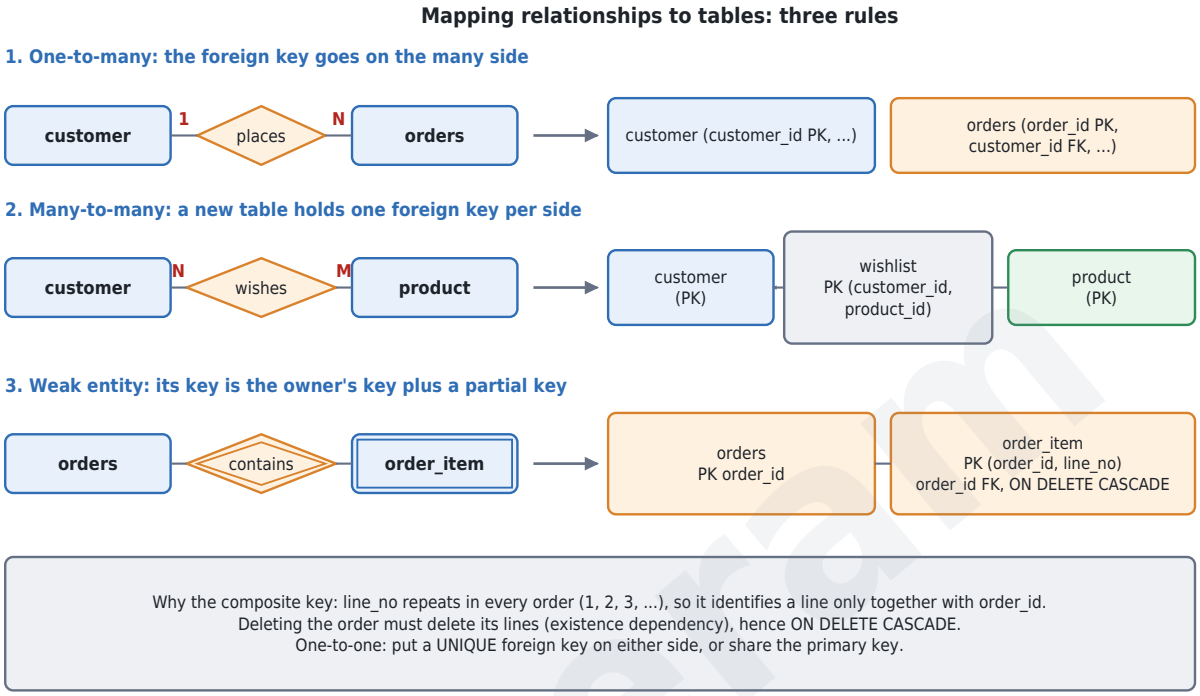


Figure 1.13. One-to-many places the foreign key on the many side, many-to-many creates a table holding two foreign keys, and a weak entity takes the owner key as part of its own composite key.

1. **Each strong entity** becomes a table; its key becomes the primary key; composite attributes become several columns; derived attributes are omitted.
2. **Each multivalued attribute** becomes a table with the owner key and the value, keyed by both.
3. **A 1:N relationship** adds a foreign key to the table on the N side. If participation of the N side is total, the foreign key is `NOT NULL`.
4. **A 1:1 relationship** adds a foreign key with a `UNIQUE` constraint to the side with total participation, or shares the primary key between the two tables.
5. **An M:N relationship** becomes a table whose primary key is the combination of the two foreign keys. Attributes of the relationship (quantity, grade, added date) become columns of that table.
6. **A weak entity** becomes a table whose primary key is the owner's key plus the partial key, with a foreign key to the owner.
7. **Hierarchies** (specialisation) have three mappings described next.

Mapping inheritance

When an entity type has specialised subtypes (a payment is a card payment, a UPI payment or a net-banking payment) there are three standard mappings, with the Hibernate names in parentheses.

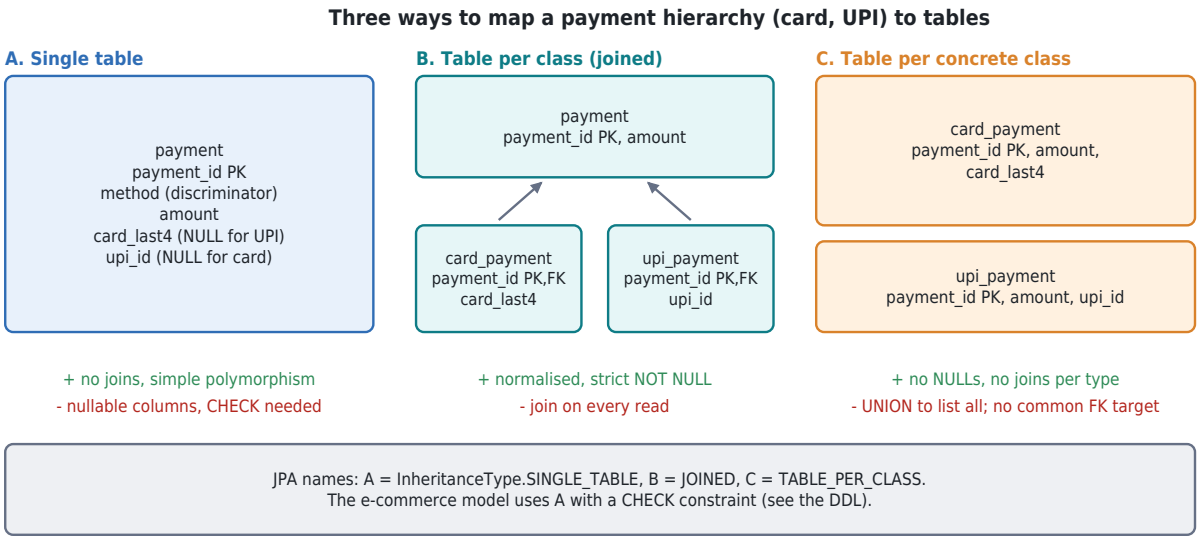


Figure 1.14. Single table is simplest and fastest to query but needs nullable columns and a CHECK; table per class normalises but joins on every read; table per concrete class avoids NULLs but cannot be the target of a single foreign key.

Criterion	Single table (SINGLE_TABLE)	Joined (JOINED)	Concrete tables (TABLE_PER_CLASS)
Polymorphic query	One scan	Join of parent and children	`UNION ALL` over all tables
NOT NULL on subtype columns	Not possible without CHECK	Natural	Natural
Foreign key to "any payment"	One FK	One FK	Not possible
Schema change for a new subtype	Add nullable columns	Add a table	Add a table
Typical use	Few subtypes, few columns each	Many subtype-specific columns	Rarely recommended

Inheritance mapping trade-offs.

1.3.2 Worked example

The running model for this book is an online shop. The requirements in words: customers register with a unique email and may keep several addresses. Products belong to a category; categories form a tree. A customer places orders shipped to one of their addresses; an order has one or more lines, each for a product at the price valid at the time of purchase. An order is paid by one or more payments, each by card, UPI or net banking. Customers may review a product once and keep a wishlist.

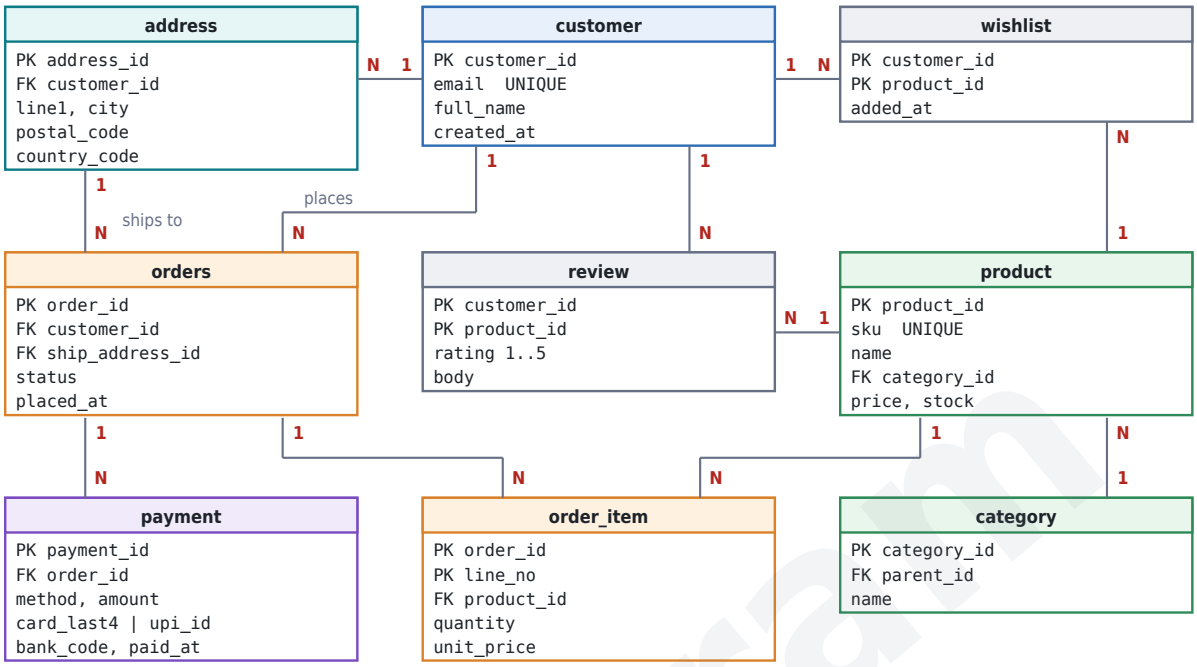
Step 1: entities. Customer, Address, Category, Product, Order, Payment (with subtypes), Review. Order line and wishlist entry are not independent things, which leads to the next steps.

Step 2: relationships and cardinality.

Relationship	Cardinality	Consequence in tables
Customer has Address	1:N, address total	`address.customer_id NOT NULL`
Customer places Order	1:N, order total	`orders.customer_id NOT NULL`
Order ships to Address	N:1	`orders.ship_address_id NOT NULL`
Order contains Product	M:N with attributes quantity, unit_price	table `order_item`, also weak (line number)
Product belongs to Category	N:1	`product.category_id NOT NULL`
Category has parent Category	N:1, partial (roots)	`category.parent_id` nullable self-reference
Customer reviews Product	M:N with rating, body; one review per pair	table `review`, key (customer, product)
Customer wishes Product	M:N	table `wishlist`
Order is paid by Payment	1:N	`payment.order_id NOT NULL`

Relationships of the shop model.

Step 3: decisions. `order_item.unit_price` copies the product price at purchase time. This is a deliberate duplication, justified because a later price change must not rewrite history; it is a different fact from `product.price`, not a redundant copy. Payment subtypes use the single-table mapping with a `CHECK` that makes exactly the columns of the chosen method mandatory. The table is named `orders` because `ORDER` is a reserved word.



category.parent_id references category: an optional self-reference (tree).
Every line is a foreign key; 1 marks the referenced (parent) side, N the referencing side.

Figure 1.15. The shop model: nine tables, each line a foreign key from the N side to the 1 side; `order_item` resolves the many-to-many between orders and products, and `category` references itself.

Step 4: DDL. The complete schema as created in PostgreSQL 16 (it ran without error, creating nine tables):

shop_schema.sql — SQL

```sql
CREATE TABLE customer (
    customer_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email        text        NOT NULL UNIQUE,
    full_name    text        NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE address (
    address_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id  bigint  NOT NULL REFERENCES customer (customer_id) ON DELETE CASCADE,
    line1        text    NOT NULL,
    city         text    NOT NULL,
    postal_code  text    NOT NULL,
    country_code char(2) NOT NULL,
    is_default   boolean NOT NULL DEFAULT false
);

CREATE TABLE category (
    category_id  int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent_id    int REFERENCES category (category_id),
    name         text NOT NULL,
    UNIQUE (parent_id, name)
);

CREATE TABLE product (
    product_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sku          text          NOT NULL UNIQUE,
    name         text          NOT NULL,
    category_id  int           NOT NULL REFERENCES category (category_id),
    price        numeric(12,2) NOT NULL CHECK (price >= 0),
    stock        int           NOT NULL DEFAULT 0 CHECK (stock >= 0)
```

shop_schema.sql (continued) SQL

```sql
);

CREATE TABLE orders (
    order_id        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id     bigint      NOT NULL REFERENCES customer (customer_id),
    ship_address_id bigint      NOT NULL REFERENCES address (address_id),
    status          text        NOT NULL DEFAULT 'NEW'
                    CHECK (status IN ('NEW','PAID','SHIPPED','DELIVERED','CANCELLED')),
    placed_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE order_item (
    order_id     bigint        NOT NULL REFERENCES orders (order_id) ON DELETE CASCADE,
    line_no      int           NOT NULL,
    product_id   bigint        NOT NULL REFERENCES product (product_id),
    quantity     int           NOT NULL CHECK (quantity > 0),
    unit_price   numeric(12,2) NOT NULL CHECK (unit_price >= 0),
    PRIMARY KEY (order_id, line_no)
);

CREATE TABLE payment (
    payment_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id     bigint        NOT NULL REFERENCES orders (order_id),
    method       text          NOT NULL CHECK (method IN ('CARD','UPI','NETBANKING')),
    amount       numeric(12,2) NOT NULL CHECK (amount > 0),
    card_last4   char(4),
    upi_id       text,
    bank_code    text,
    paid_at      timestamptz   NOT NULL DEFAULT now(),
    CHECK ((method = 'CARD'       AND card_last4 IS NOT NULL AND upi_id IS NULL
                                  AND bank_code IS NULL)
        OR (method = 'UPI'        AND upi_id IS NOT NULL AND card_last4 IS NULL
                                  AND bank_code IS NULL)
        OR (method = 'NETBANKING' AND bank_code IS NOT NULL AND card_last4 IS NULL
                                  AND upi_id IS NULL))
);

CREATE TABLE review (
    customer_id  bigint   NOT NULL REFERENCES customer (customer_id),
    product_id   bigint   NOT NULL REFERENCES product (product_id),
    rating       smallint NOT NULL CHECK (rating BETWEEN 1 AND 5),
    body         text,
    PRIMARY KEY (customer_id, product_id)
);

CREATE TABLE wishlist (
    customer_id  bigint NOT NULL REFERENCES customer (customer_id) ON DELETE CASCADE,
    product_id   bigint NOT NULL REFERENCES product (product_id)  ON DELETE CASCADE,
    added_at     timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (customer_id, product_id)
);
```

What to notice. The M:N tables `review` and `wishlist` have composite primary keys, which also enforce "one review per customer and product". `order_item` has a composite key `(order_id, line_no)` and `ON DELETE CASCADE` because the line cannot outlive the order. `orders.customer_id` has no cascade: deleting a customer who has orders must fail, as the constraint test in the previous chapter showed. Primary keys use `GENERATED ALWAYS AS IDENTITY`, the standard form that PostgreSQL recommends over `serial`; the foreign key columns have the same type (`bigint`) as the keys they reference.

1.3.3 Hands-on

A small data set (three customers, three products, two orders) was loaded, and the order totals were computed by joining four tables:

Order totals across four tables TERMINAL

```
ecom=# select o.order_id, c.full_name, o.status, count(*) as lines,
ecom-#        sum(i.quantity*i.unit_price) as total
ecom-#   from orders o join customer c using (customer_id)
ecom-#                 join order_item i using (order_id)
ecom-#  group by 1,2,3 order by 1;
 order_id | full_name  | status | lines |  total
----------+------------+--------+-------+----------
        1 | Asha Rao   | PAID   |     2 | 21597.00
        2 | Bilal Khan | NEW    |     1 | 54999.00
(2 rows)
```

The payment `CHECK` makes a subtype's columns mandatory and the others forbidden. Inserting a card payment that also carries a UPI id was rejected:

The subtype CHECK in action TERMINAL

```
ecom=# insert into payment (order_id,method,amount,card_last4,upi_id)
ecom-#   values (2,'CARD',10,'1234','x@upi');
ERROR:  new row for relation "payment" violates check constraint "payment_check"
DETAIL:  Failing row contains (2, 2, CARD, 10.00, 1234, x@upi, null, 2026-10-18 06:53:20.328782+00).
```

The corresponding JPA mapping of the weak entity uses an embedded composite key. This fragment is typical Spring/Hibernate code and was not compiled here (no Maven artifacts are available on the build machine).

OrderItem.java (fragment, not run) JAVA

```java
@Embeddable
public record OrderItemId(@Column(name = "order_id") Long orderId,
                          @Column(name = "line_no") Integer lineNo) implements Serializable { }

@Entity
@Table(name = "order_item")
public class OrderItem {
    @EmbeddedId
    private OrderItemId id;

    @MapsId("orderId")                 // part of the key is also the foreign key
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id")
    private Order order;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "product_id")
    private Product product;

    private int quantity;
    private BigDecimal unitPrice;      // BigDecimal, never double
}
```

1.3.4 Real-world problems and solutions

Incident: a many-to-many relationship stored in a delimited column

Scenario. To save a table, a product's categories are stored as the text `'2,12'` in a column `category_ids`. The catalogue page filters with `LIKE`.

Symptoms. The "Phones" category page (category 2) lists a cable that is only in category 12 (Accessories). Deleting a category leaves its id inside product rows. Reports by category require string parsing.

Diagnosis. Reproduce the filter and compare it with the intended answer:

LIKE on a delimited list returns false positives TERMINAL

```
er2=# select name from product_bad where category_ids like '%2%' order by product_id;
    name
-------------
 Phone Alpha
 Case
 Cable
(3 rows)

er2=# select name from product_bad
er2-#  where ',' || category_ids || ',' like '%,2,%' order by product_id;
```

sreram

LIKE on a delimited list returns false positives (continued) TERMINAL

```
    name
-------------
 Phone Alpha
 Case
(2 rows)
```

Root cause. The relationship is many-to-many and was stored in one column, violating first normal form. `'%2%'` matches `12` and `20`. No foreign key can verify that each listed id exists, and no index can serve the lookup.

Fix. Create the junction table and migrate the data (in PostgreSQL `string_to_table(category_ids, ',')` splits the list). The correct query is exact, indexable and verifiable:

The junction table TERMINAL

```
er2=# select p.name from product p join product_category pc using (product_id)
er2-#  where pc.category_id = 2 order by 1;
    name
-------------
 Case
 Phone Alpha
(2 rows)

er2=# insert into product_category values (1,99);
ERROR:  insert or update on table "product_category" violates foreign key constraint
        "product_category_category_id_fkey"
DETAIL:  Key (category_id)=(99) is not present in table "category".
```

Prevention. A column that holds a list is a sign of a missing table. Arrays and JSONB are legitimate for data that is read and written as a unit and never joined or constrained, not for relationships.

Incident: a one-to-one assumption that stopped being true

Scenario. The first release stores one shipping address directly in the customer table. Marketing then asks for separate billing and home addresses, and customers who move want history.

Symptoms. Every new address type requires new columns (`billing_line1`, `billing_city`), a migration of every query and a change in every client.

Diagnosis. Review the cardinality recorded at design time. "A customer has one address" was a 1:1 assumption that the business never guaranteed; the question that exposes it is "can the same customer ever have two?".

Root cause. The attribute set was modelled as part of the customer entity instead of as a separate entity with a 1:N relationship.

Fix. Create `address` with a foreign key to `customer` (as in the shop model), migrate each existing address into a row marked `is_default`, and replace the columns by a view during the transition to keep old queries working. **Prevention.** Model every group of attributes that has its own lifecycle (an address, a phone number) as an entity, and challenge every 1:1 and "exactly one" statement in the requirements.

1.3.5 Questions to expect

How is a many-to-many relationship implemented in a relational database?

With a junction (associative) table containing a foreign key to each side. Its primary key is usually the pair of foreign keys; relationship attributes (quantity, rating) are columns of the junction table.

What is a weak entity?

An entity that has no key of its own and depends on an owner entity for identification, for example an order line identified by `(order_id, line_no)`. Its foreign key to the owner is part of its primary key, and the relationship is usually deleted with `ON DELETE CASCADE`.

How are one-to-one relationships mapped?

A foreign key with a `UNIQUE` constraint on one side, normally the side that depends on the other; or both tables share the same primary key (the dependent table's key is also a foreign key to the parent).

Why is `unit_price` stored in `order_item` when `product.price` exists?

They are different facts: the price at the time of purchase is history; the current price is mutable. Reading the product price for an old order would give wrong totals.

What are the three ways to map inheritance and when is each appropriate?

Single table (few subtypes, queried together), joined tables (subtype-specific columns that need `NOT NULL`), table per concrete class (rarely, when subtypes are never queried together). The cost is respectively NULL columns, joins and UNIONs.

What is the difference between participation and cardinality?

Cardinality says how many instances relate (1, N). Participation says whether relating is mandatory (total) or optional (partial). Total participation on the many side maps to a `NOT NULL` foreign key.

How is a tree such as a category hierarchy stored?

By an adjacency list (a nullable `parent_id` self-reference, as above), which is simple and queried with recursive CTEs; other patterns are materialised path and nested sets, which speed up subtree reads and slow down moves.

KEY POINTS

Key points

- ER modelling records entities, attributes and relationships with cardinality and participation before any table is created.
- Mapping rules: 1:N puts the foreign key on the N side, M:N needs a junction table, weak entities take the owner key into their primary key, multivalued attributes become tables.
- Inheritance has three mappings (single table, joined, concrete tables); single table with a `CHECK` is the simplest and is used in the shop model.
- Store relationships as foreign keys, not as delimited lists; a list in a column signals a missing table.
- Challenge every "exactly one" requirement: an entity with its own lifecycle deserves its own table.

1.4 Normalisation

An ER-derived schema is usually close to normalised, but schemas also arise from spreadsheets, legacy exports and quick prototypes. **Normalisation** is the systematic procedure for removing redundancy from such a schema by decomposing tables, guided by **functional dependencies**. Its purpose is not tidiness: redundancy allows the same fact to be recorded twice, and two copies of a fact will eventually disagree. The procedure also gives a precise vocabulary (1NF to 4NF) that interviewers expect.

1.4.1 Concept and mechanism

The problem: anomalies

The following table `order_sheet` records orders as a spreadsheet would: one row per order line, with all customer and product details repeated. Its primary key is `(order_id, product_id)`.

The unnormalised table (PostgreSQL 16; columns abbreviated with aliases) TERMINAL

```
norm=# select order_id as oid, order_date as odate, customer_id as cust, customer_name as cname,
norm-#        customer_city as city, product_id as prod, product_name as pname,
norm-#        unit_price as price, qty from order_sheet order by 1,6;
  oid  |   odate    | cust |    cname    |    city    | prod |      pname       |  price   | qty
------+------------+------+-------------+------------+------+------------------+----------+-----
 1001 | 2026-03-01 |    1 | Asha Rao    | Vijayawada |   10 | Phone Alpha      | 19999.00 |   1
 1001 | 2026-03-01 |    1 | Asha Rao    | Vijayawada |   30 | Database Systems |   799.00 |   2
 1002 | 2026-03-02 |    2 | Bilal Khan  | Kolkata    |   20 | Laptop Beta      | 54999.00 |   1
 1003 | 2026-03-05 |    1 | Asha Rao    | Vijayawada |   30 | Database Systems |   799.00 |   1
 1004 | 2026-03-07 |    3 | Chitra Iyer | Chennai    |   10 | Phone Alpha      | 19999.00 |   2
```

order_sheet: one table, three anomalies

order	cust	customer_city	prod	product_name	price	qty
1001	1	Vijayawada	10	Phone Alpha	19999	1
1001	1	Vijayawada	30	Database Sys.	799	2
1002	2	Kolkata	20	Laptop Beta	54999	1
1003	1	Vijayawada	30	Database Sys.	799	1
1004	3	Chennai	10	Phone Alpha	19999	2

Highlighted cells repeat the same fact in several rows (city per customer, name and price per product).
Primary key: (order, prod). The facts about customer and product depend on only part of that key.

Update anomaly
Asha moves city: three rows hold the city; updating one leaves the other two stale

Insertion anomaly
a new product with no order cannot be stored: order and qty are part of the primary key

Deletion anomaly
cancelling order 1002 also erases the only record of Bilal and of Laptop Beta

Figure 1.16. Highlighted cells repeat the same fact in several rows; each repeated fact causes one of the three anomalies.

Each anomaly was reproduced on this table.

Update anomaly. Asha moves to Hyderabad. A program that updates only the row it has open changes one of her three rows:

Update anomaly TERMINAL

```
norm=# update order_sheet set customer_city = 'Hyderabad' where order_id = 1001 and product_id = 10;
UPDATE 1
norm=# select customer_id, customer_city, count(*) from order_sheet group by 1,2 order by 1,2;
 customer_id | customer_city | count
-------------+---------------+-------
           1 | Hyderabad     |     1
           1 | Vijayawada    |     2
           2 | Kolkata       |     1
           3 | Chennai       |     1
```

Customer 1 now lives in two cities. No constraint is violated, so the database cannot detect it.

Insertion anomaly. A new product, "Tablet Gamma", has not been ordered. It cannot be stored because `order_id` is part of the primary key:

Insertion anomaly TERMINAL

```
norm=# insert into order_sheet (product_id, product_name, unit_price)
norm-#   values (40,'Tablet Gamma',15999.00);
ERROR:  null value in column "order_id" of relation "order_sheet" violates not-null constraint
DETAIL:  Failing row contains (null, null, null, null, null, 40, Tablet Gamma, 15999.00, null).
```

Deletion anomaly. Cancelling order 1002, the only order of customer 2 and the only order of product 20, deletes the customer and the product from the database as well:

Deletion anomaly TERMINAL

```
norm=# delete from order_sheet where order_id = 1002;
DELETE 1
norm=# select distinct customer_id, customer_name from order_sheet order by 1;
 customer_id | customer_name
-------------+---------------
           1 | Asha Rao
           3 | Chitra Iyer
```

Bilal Khan and Laptop Beta are gone. All three anomalies have one cause: facts about different things (orders, customers, products) are stored in one table.

Functional dependencies

A **functional dependency** (FD) `X -> Y` holds in a relation when any two tuples that agree on the attributes `X` also agree on `Y`; equivalently, `X` determines `Y`. FDs are statements about the meaning of the data, declared by the designer from business rules, not discovered from a sample. In `order_sheet` the rules are:

- `order_id -> order_date, customer_id` (an order has one date and one customer)
- `customer_id -> customer_name, customer_city`

- `product_id -> product_name, unit_price`
- `order_id, product_id -> qty`

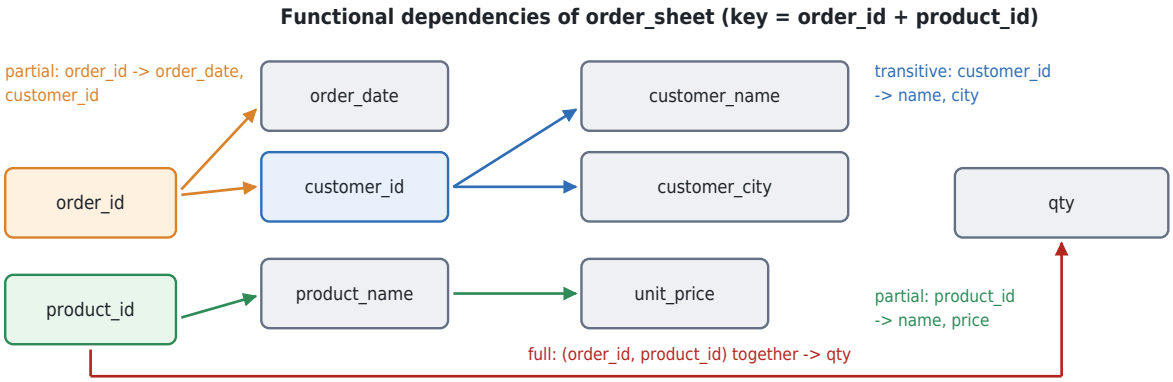


Figure 1.17. The dependencies of order_sheet: only qty depends on the whole key; order and customer facts depend on part of it (partial) and customer details depend on a non-key attribute (transitive).

A data sample can refute an FD but never prove one. A query for counterexamples is useful for testing a suspected FD against existing data:

Testing candidate FDs against the data TERMINAL

```
norm=# select customer_id from order_sheet group by customer_id
norm-#   having count(distinct customer_city) > 1;
(0 rows)
norm=# select product_id from order_sheet group by product_id
norm-#   having count(distinct unit_price) > 1;
(0 rows)
```

An empty result means that no counterexample exists in the current rows. It does not prove the rule: two products may merely happen to share a price today (the sample also satisfies the unintended `customer_city -> customer_id`, only because each city has one customer in six rows). The designer decides, from the business, which FDs are rules.

An FD `X -> Y` is **trivial** if `Y` is a subset of `X`. The inference rules for FDs are **Armstrong's axioms**: reflexivity (if `Y` is a subset of `X` then `X -> Y`), augmentation (if `X -> Y` then `XZ -> YZ`) and transitivity (if `X -> Y` and `Y -> Z` then `X -> Z`). From them follow union, decomposition and pseudotransitivity. The **closure** of an attribute set `X`, written `X+`, is the set of all attributes that `X` determines. It is computed by starting with `X` and repeatedly adding the right-hand side of any FD whose left-hand side is already contained in the result.

Worked closure. Compute `{order_id, product_id}+` for `order_sheet`:

1. Start: `{order_id, product_id}`.
2. `order_id -> order_date, customer_id` applies: add `order_date`, `customer_id`.
3. `customer_id -> customer_name, customer_city` applies: add both.
4. `product_id -> product_name, unit_price` applies: add both.
5. `order_id, product_id -> qty` applies: add `qty`.

The closure contains all nine attributes, so `{order_id, product_id}` is a superkey; no proper subset reaches all attributes (`{order_id}+` lacks `product_id` and `qty`), so it is a candidate key. A candidate key is exactly an attribute set whose closure is the whole relation and which is minimal. An attribute that appears in some candidate key is a **prime** attribute; the others are **non-prime**.

1.4.2 Worked example

First normal form

A relation is in **1NF** when every attribute value is atomic (one value per cell, no repeating groups or lists) and the relation has a key. Storing `category_ids = '2,12'` or one column per phone (`phone1`, `phone2`) violates 1NF. The remedy is a separate table with one row per value. In SQL practice 1NF also means: no arrays or JSON used to hide a relationship that is queried, joined or constrained.

Second normal form

A relation is in **2NF** when it is in 1NF and no non-prime attribute depends on a proper part of a candidate key. The condition matters only for composite keys. In `order_sheet`, `order_date` and `customer_id` depend on `order_id` alone, and `product_name` and `unit_price` on `product_id` alone: partial dependencies. The decomposition puts each partial dependency into its own relation together with its determinant:

- `orders_2(order_id, order_date, customer_id, customer_name, customer_city)`
- `product_n(product_id, product_name, unit_price)`
- `order_line_n(order_id, product_id, qty)`

The customer attributes travel with the order, because they depend on `order_id` (through `customer_id`), which is a complete key of `orders_2`.

Third normal form

A relation is in **3NF** when it is in 2NF and no non-prime attribute depends transitively on the key; formally, for every nontrivial FD `X -> A`, either `X` is a superkey or `A` is a prime attribute. In the 2NF result, `orders_2` still contains `customer_id -> customer_name, customer_city`: customer details depend on the key `order_id` through `customer_id`. Moving them out gives the final 3NF schema:

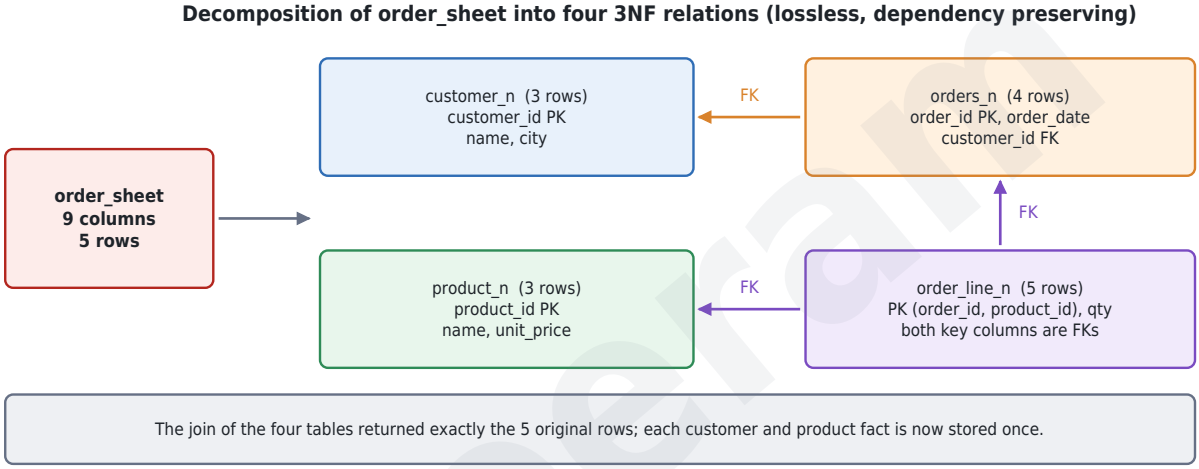


Figure 1.18. Four relations: every nonkey attribute depends on the key, the whole key and nothing but the key; foreign keys link the tables.

3NF decomposition SQL

```sql
CREATE TABLE customer_n (customer_id int PRIMARY KEY, customer_name text NOT NULL,
                         customer_city text NOT NULL);
CREATE TABLE product_n  (product_id int PRIMARY KEY, product_name text NOT NULL,
                         unit_price numeric(8,2) NOT NULL);
CREATE TABLE orders_n   (order_id int PRIMARY KEY, order_date date NOT NULL,
                         customer_id int NOT NULL REFERENCES customer_n);
CREATE TABLE order_line_n (order_id int REFERENCES orders_n, product_id int REFERENCES product_n,
                           qty int NOT NULL, PRIMARY KEY (order_id, product_id));
INSERT INTO customer_n  SELECT DISTINCT customer_id, customer_name, customer_city FROM order_sheet;
INSERT INTO product_n   SELECT DISTINCT product_id, product_name, unit_price FROM order_sheet;
INSERT INTO orders_n    SELECT DISTINCT order_id, order_date, customer_id FROM order_sheet;
INSERT INTO order_line_n SELECT order_id, product_id, qty FROM order_sheet;
```

The mnemonic for 3NF: every non-key attribute depends on **the key** (1NF), **the whole key** (2NF) and **nothing but the key** (3NF).

The decomposition must be checked. The four tables are joined back and compared with the original in both directions with `EXCEPT`; both differences must be empty for the join to be **lossless**:

Lossless-join check and the anomalies after decomposition TERMINAL

```
norm=# select * from rebuilt except select * from order_sheet;   -- (0 rows)
norm=# select * from order_sheet except select * from rebuilt;   -- (0 rows)

norm=# update customer_n set customer_city = 'Hyderabad' where customer_id = 1;
UPDATE 1
norm=# insert into product_n values (40,'Tablet Gamma',15999.00);
INSERT 0 1
norm=# delete from order_line_n where order_id = 1002;
```

Lossless-join check and the anomalies after decomposition (continued) TERMINAL

```
DELETE 1
norm=# select count(*) as products_kept from product_n;
 products_kept
---------------
             4
```

One UPDATE changes the city for all of Asha's orders; the unordered product is stored; deleting the order line removes neither customer nor product. (`rebuilt` is a view joining the four tables on their keys.)

Boyce-Codd normal form

A relation is in **BCNF** when for every nontrivial FD `X -> Y`, `X` is a superkey. BCNF is stricter than 3NF: 3NF tolerates a determinant that is not a superkey if the dependent attribute is prime. The classic case: students are taught a course by exactly one instructor each, an instructor teaches only one course, and a course has several instructors. The relation `tutoring(student, course, instructor)` has the FDs `student, course -> instructor` and `instructor -> course`. Candidate keys are `{student, course}` and `{student, instructor}`. The second FD has a determinant that is not a superkey, but its dependent `course` is prime, so the relation is in 3NF and not in BCNF. The redundancy is visible: the pair (Rao, DBMS) repeats in every row for Rao.

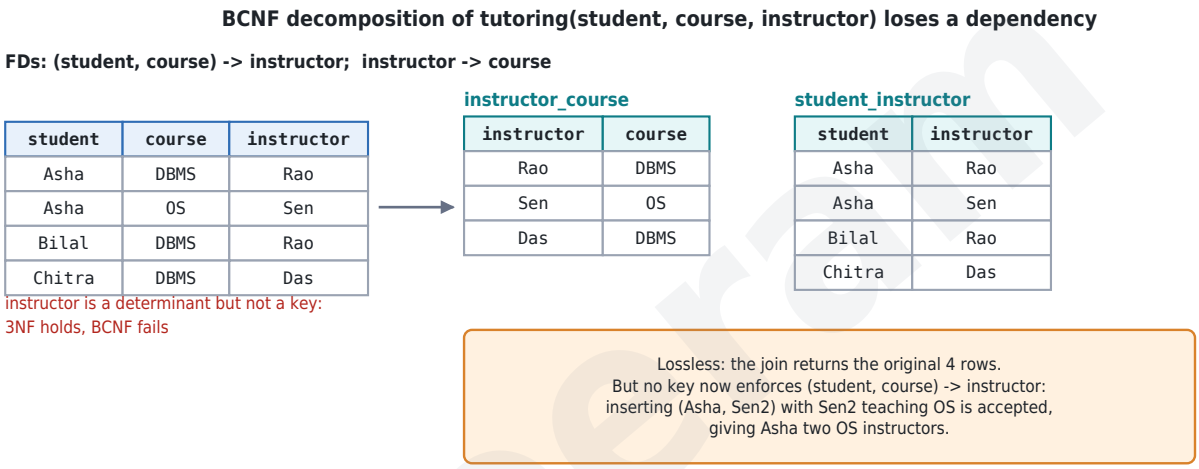


Figure 1.19. The BCNF decomposition is lossless but loses the dependency (student, course) -> instructor: no single table contains all its attributes, so no key can enforce it.

The decomposition into `instructor_course(instructor, course)` and `student_instructor(student, instructor)` was run:

BCNF decomposition: lossless, but one FD is no longer enforceable TERMINAL

```
norm=# select student, course, instructor from student_instructor join instructor_course
norm-#   using (instructor) except select student, course, instructor from tutoring;
(0 rows)
norm=# insert into instructor_course values ('Sen2','OS');
norm=# insert into student_instructor values ('Asha','Sen2');
norm=# select student, course, count(distinct instructor) as instructors
norm-#   from student_instructor join instructor_course using (instructor)
norm-#  group by 1,2 having count(distinct instructor) > 1;
 student | course | instructors
---------+--------+-------------
 Asha    | OS     |           2
```

The join reproduces the original rows (lossless), but the second instructor for Asha's OS course is accepted: the rule "one instructor per student and course" can no longer be declared as a key, because its attributes lie in different tables. It would need a trigger or an application check. This is the standard tension: **BCNF decomposition is always lossless but not always dependency-preserving; 3NF decomposition can always be made both lossless and dependency-preserving.** Practitioners therefore target 3NF and go to BCNF when the lost dependency is not important or can be enforced otherwise.

Lossless join and dependency preservation

A decomposition of `R` into `R1` and `R2` is **lossless** when `R1 JOIN R2` equals `R`. The test: it is lossless if the common attributes `R1 ∩ R2` form a key of `R1` or of `R2` (more generally, if `R1 ∩ R2 -> R1` or `R1 ∩ R2 -> R2` holds). A decomposition is **dependency-preserving** when every FD of `R` can be checked on a single table of the result, so that the key constraints alone enforce them.

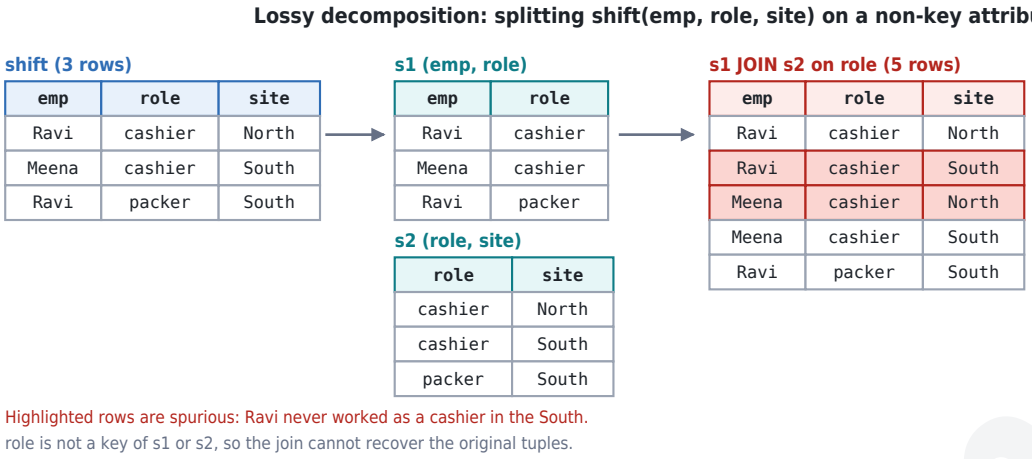


Figure 1.20. Splitting shift(emp, role, site) into (emp, role) and (role, site) on the non-key attribute role creates spurious tuples on the join.

A lossy decomposition was built for real. The relation `shift` has 3 rows; after projecting into `(emp, role)` and `(role, site)` the natural join returns 5:

A lossy decomposition TERMINAL

```
norm=# select count(*) as original from shift;          -- 3
norm=# select count(*) as rejoined from s1 natural join s2;   -- 5
norm=# select emp, role, site from s1 natural join s2 except select emp, role, site from shift;
  emp  |  role   | site
-------+---------+-------
 Meena | cashier | North
 Ravi  | cashier | South
```

The two extra rows claim that Meena worked at North and Ravi worked as a cashier at South; neither happened. The common attribute `role` determines neither `emp` nor `site`, so it is not a key of either part, and the information about which employee went where is destroyed. The word *lossy* is misleading: the decomposition loses no tuples; it gains false ones, which makes the information unrecoverable.

Fourth normal form

A **multivalued dependency** (MVD) `X ->> Y` holds when the set of `Y` values for a given `X` is independent of the other attributes. A relation is in **4NF** when every nontrivial MVD has a superkey as determinant. Typical case: an employee has a set of skills and a set of languages, unrelated to each other. Storing them in one table `emp_all(emp, skill, lang)` requires the cross product.

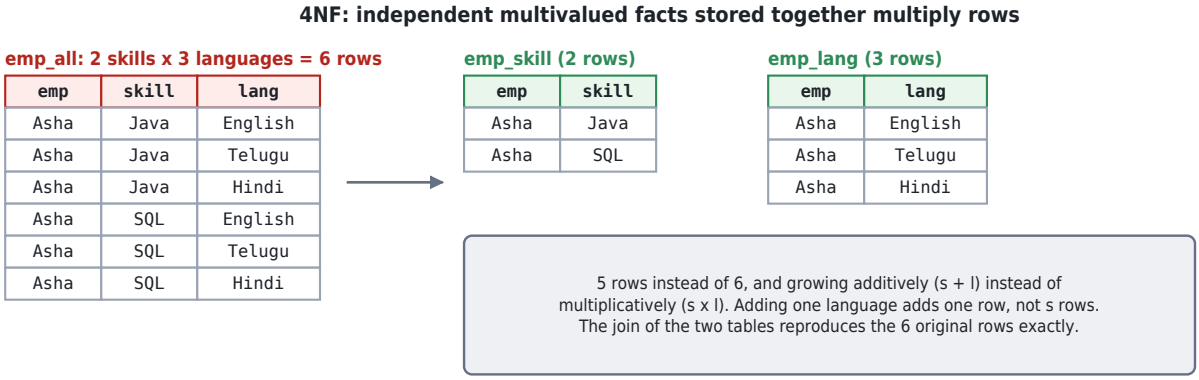


Figure 1.21. Two independent facts stored together force every combination; the 4NF decomposition stores each fact once and grows additively.

4NF decomposition TERMINAL

```
norm=# select count(*) as rows_for_one_employee from emp_all;      -- 2 skills x 3 languages
 rows_for_one_employee
-----------------------
                     6
norm=# select (select count(*) from emp_skill) + (select count(*) from emp_lang) as rows_after_4nf;
 rows_after_4nf
----------------
              5
norm=# select emp, skill, lang from emp_skill join emp_lang using (emp)
norm-#   except select emp, skill, lang from emp_all;
(0 rows)
```

With 5 skills and 8 languages the combined table needs 40 rows and the decomposed form 13. Adding one language to the combined table requires one new row per skill (and forgetting one produces a false statement that the employee lacks that skill-language combination).

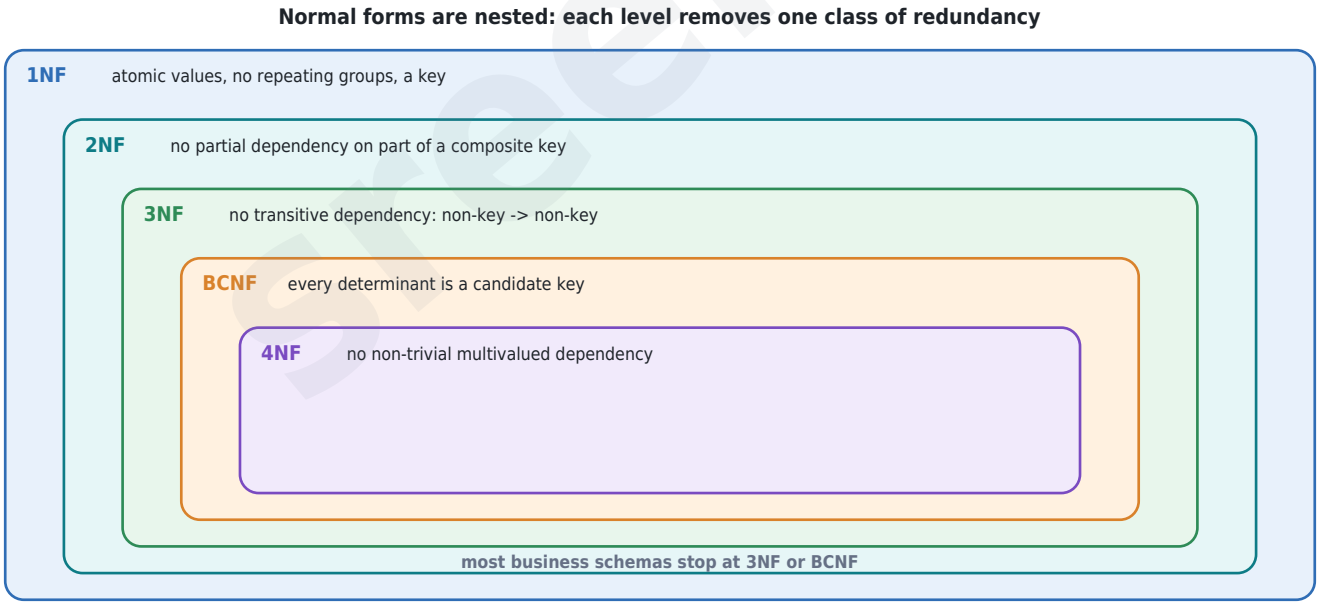


Figure 1.22. The normal forms are nested: each removes one class of dependency problem; 5NF (join dependencies) exists but rarely matters in practice.

Form	Condition	Typical violation
1NF	Atomic values, no repeating groups	Comma-separated list in a column
2NF	No partial dependency on a composite key	Product name stored in the order line
3NF	No transitive dependency; each determinant a superkey or dependent prime	Customer city stored in the order
BCNF	Every determinant is a superkey	Course determined by instructor in an enrolment table

Form	Condition	Typical violation
4NF	No nontrivial multivalued dependency except on a superkey	Skills and languages in one table

Summary of normal forms.

When to denormalise

Normalisation optimises writes and correctness; it costs joins on reads. **Denormalisation** is the deliberate introduction of redundancy to speed up a measured read path. It is a trade, and the rule is: normalise first, measure, then denormalise a specific query with a specific mechanism for keeping the copy correct.

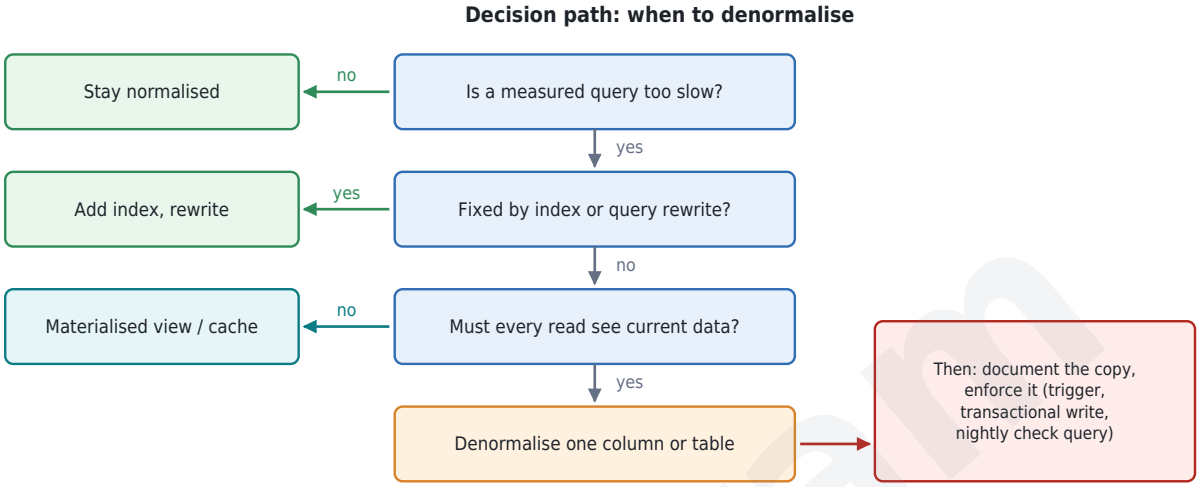


Figure 1.23. Denormalise only after a measured problem that indexes and query rewrites cannot solve, and only with a mechanism that keeps the copy correct.

The cost on both sides was measured on `orders` (300,000 rows), `order_item` (900,000 rows) and `customer` (20,000 rows). The query computes revenue per city. The normalised form joins three tables; the denormalised form reads one table `orders_d` that carries a copy of the city and the order total.

Revenue per city: normalised versus a denormalised copy (typical timings; shared build machine) TERMINAL

```
den=# explain (analyze, costs off, buffers off)
den-#   select c.city, sum(i.qty*i.unit_price)
den-#   from customer c join orders o using (customer_id) join order_item i using (order_id)
den-#   group by c.city;
 ...
 Execution Time: 968.176 ms

den=# explain (analyze, costs off, buffers off)
den-#   select customer_city, sum(total) from orders_d group by customer_city;
 ...
 Execution Time: 190.326 ms

den=# select pg_size_pretty(pg_total_relation_size('orders_d')) as denorm_size,
den-#        pg_size_pretty(pg_total_relation_size('orders')) as orders_size;
 denorm_size | orders_size
-------------+-------------
 18 MB       | 22 MB
```

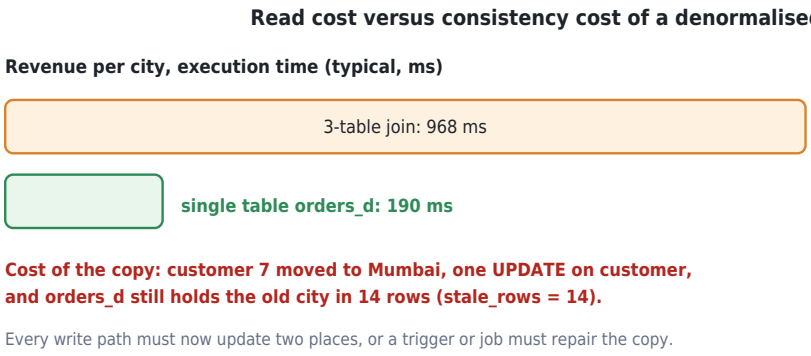


Figure 1.24. The denormalised read is about five times faster, but the copy becomes stale as soon as the source changes.

The read is five times faster. The consistency cost appears at the first update:

Drift after a customer moves TERMINAL

```
den=# update customer set city = 'Mumbai' where customer_id = 7;
UPDATE 1
den=# select count(*) as stale_rows from orders_d d join customer c using (customer_id)
den-#  where d.customer_city <> c.city;
 stale_rows
------------
         14
```

Fourteen order rows now carry a city that disagrees with the customer. Legitimate denormalisation techniques differ in how they keep the copy correct: a **materialised view** refreshed on a schedule (staleness is explicit and accepted), a **summary table** maintained in the same transaction as the source change, a **trigger**, or a **cache** with an expiry. Storing a historical fact (the price at purchase) is not denormalisation but the correct modelling of a different fact. Analytical stores such as star schemas denormalise by design because they are loaded in bulk and not updated by transactions.

1.4.3 Hands-on

A checklist for normalising an existing table, in the order that works in practice:

1. List the attributes and write the business rules as FDs in plain language ("an order belongs to one customer").
2. Find the candidate keys by computing closures; mark prime attributes.
3. Test each FD for counterexamples with the `GROUP BY ... HAVING count(DISTINCT ...) > 1` query; a hit means the rule is wrong or the data is already inconsistent.
4. Check 1NF (lists, repeating groups), then 2NF (partial dependencies, only if the key is composite), then 3NF and BCNF (every determinant).
5. Decompose, create the new tables with primary and foreign keys, copy the data with `INSERT ... SELECT DISTINCT`.
6. Verify losslessness with two `EXCEPT` queries in both directions and verify row counts.
7. Replace the old table by a view with its name if existing queries must continue to work (logical data independence).

1.4.4 Real-world problems and solutions

Incident: update anomaly from a denormalised table

Scenario. A reporting team added `customer_city` to the orders table "to avoid the join". The customer service tool updates the customer's city in the customer table.

Symptoms. The regional sales dashboard attributes revenue to cities where the customer no longer lives; two reports of the same period disagree. Support finds one customer with orders in two cities.

Diagnosis. Compare each copy with its source:

Detecting drift between a copy and its source TERMINAL

```
den=# select d.customer_id, d.customer_city as copied, c.city as current, count(*) as orders
den-#   from orders_d d join customer c using (customer_id)
den-#  where d.customer_city <> c.city
den-#  group by 1,2,3;
 customer_id | copied | current | orders
-------------+--------+---------+--------
           7 | Hyderabad | Mumbai |     14
```

(The result shows the shape of the output; in the reproduction the copied value of customer 7 was the original city of the generated data.)

Root cause. The same fact (a customer's city) is stored in 20,000 customer rows and 300,000 order rows; only one write path updates it. The functional dependency `customer_id -> city` is violated between tables, which the database cannot detect without a constraint.

Fix. Immediate: repair the copy with `UPDATE orders_d d SET customer_city = c.city FROM customer c WHERE c.customer_id = d.customer_id AND d.customer_city <> c.city;`. Permanent: remove the redundant column and let the report join (the measured cost was about 780 ms on 300,000 orders; an index and a narrower query are tried first). If the join is truly too slow, maintain the summary in a materialised view (`REFRESH MATERIALIZED VIEW CONCURRENTLY`) so that its staleness is explicit.

Prevention. Every denormalised column is recorded in the schema documentation with its source and its maintenance mechanism. A scheduled check query (the diagnosis query above) fails the build or alerts when drift is found.

Incident: a quick import into a wide table

Scenario. A CSV of orders was imported directly into one table with 40 columns, including customer and product details, and application code started to read from it.

Symptoms. A product price correction leaves different prices on different rows; reports that count distinct customers disagree depending on the spelling of the name.

Diagnosis. Run the FD test for each suspected rule (`product_id -> unit_price`); a returned `product_id` proves the table already holds contradictory facts.

Root cause. Facts about three entities share one table; the import preserved the spreadsheet structure.

Fix. Decompose as shown in the worked example, choosing for each conflicting value an authoritative source, and verify with the two `EXCEPT` queries. **Prevention.** Treat imported tables as staging only; load them into the normalised schema through `INSERT ... SELECT DISTINCT` statements.

1.4.5 Questions to expect

What is normalisation and why is it done?

It is the decomposition of relations, guided by functional dependencies, so that every fact is stored once. It prevents update, insertion and deletion anomalies and the inconsistencies they cause.

Define 1NF, 2NF and 3NF with an example.

1NF: atomic values. 2NF: 1NF plus no non-key attribute depends on part of a composite key (product name in an order-line table). 3NF: 2NF plus no non-key attribute depends on another non-key attribute (customer city in an order table). Mnemonic: the key, the whole key and nothing but the key.

What is the difference between 3NF and BCNF?

In BCNF every determinant must be a superkey. 3NF also allows a determinant that is not a superkey if the dependent attribute is prime. The two differ only when a relation has overlapping candidate keys.

What is a lossless decomposition and how is it tested?

Joining the parts reproduces the original relation exactly. For a binary decomposition the shared attributes must be a key of at least one part. A practical test compares the join with the original using `EXCEPT` in both directions.

Preview

What is dependency preservation and why can BCNF fail it?

Every original FD can be enforced within a single table. In `tutoring`, `student, course -> instructor` has attributes in two tables after the BCNF split, so no key enforces it.

When would you denormalise?

When a specific, measured read query is too slow after indexing and rewriting, the data changes rarely or tolerates staleness, and a mechanism (materialised view, trigger, transactional summary update) keeps the copy consistent.

What is a multivalued dependency?

`X ->> Y`: the set of `Y` values for an `X` is independent of the other attributes. Two independent multivalued facts in one table force a cross product; 4NF stores them in separate tables.

Does a surrogate key make a table 2NF?

A single-attribute key eliminates partial dependencies, so a table with a surrogate primary key is automatically in 2NF with respect to that key; transitive dependencies and the natural candidate keys still have to be examined.

KEY POINTS

Key points

- Redundancy causes update, insertion and deletion anomalies, reproduced above on one table; normalisation removes them.
- Functional dependencies come from business rules; closures find candidate keys; a data sample can refute an FD but not prove it.
- 1NF atomic values, 2NF no partial dependency, 3NF no transitive dependency, BCNF every determinant a key, 4NF no independent multivalued facts together.
- A decomposition must be lossless (verified with two `EXCEPT` queries) and should preserve dependencies; BCNF can lose a dependency, 3NF cannot.
- Denormalise only a measured query, with a stated mechanism to keep the copy correct; otherwise stay at 3NF.

1.5 Schema Design and DDL in PostgreSQL and MySQL

A logical design becomes a database through **data definition language** (DDL): `CREATE TABLE`, constraints, indexes. Most data defects that reach production originate in this step: a column type that cannot represent the value (floating point for money), a missing constraint (no unique key, no foreign key) or a type with hidden semantics (a timestamp without a zone). This chapter surveys the type systems of PostgreSQL 16 and MySQL 8, sets out naming and constraint conventions, and shows how to run a local PostgreSQL server with `initdb` and `psql`.

1.5.1 Concept and mechanism

Running PostgreSQL 16 locally

A PostgreSQL **cluster** is one data directory managed by one server process tree; it contains several databases. It is created once with `initdb` and then started with `pg_ctl`. The server refuses to run as root, so the commands below run as the unprivileged user that owns the directory (on a development machine simply the current user; on the build machine `runuser -u postgres --`).

Creating and starting a cluster TERMINAL

```
$ initdb -D /tmp/osw_d1/pgdata -A trust -E UTF8 --locale=C.UTF-8
The files belonging to this database system will be owned by user "postgres".
The database cluster will be initialized with locale "C.UTF-8".
The default text search configuration will be set to "english".
Data page checksums are disabled.
creating directory /tmp/osw_d1/pgdata ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
```

Creating and starting a cluster (continued) TERMINAL

```
Success. You can now start the database server using:
    pg_ctl -D /tmp/osw_d1/pgdata -l logfile start

$ pg_ctl -D /tmp/osw_d1/pgdata -o "-p 20100 -c shared_buffers=64MB -c max_connections=30" \
    -l /tmp/osw_d1/log -w start
server started
$ psql -p 20100 -d postgres -c "create database ecom"
CREATE DATABASE
```

The lines of `initdb` output are trimmed to the essential ones. Three choices deserve attention: the **encoding** (`UTF8` is correct for almost every application and cannot be changed for an existing database), the **locale** (it determines sort order and `LIKE` behaviour and cannot be changed after creation without reloading) and the authentication method (`-A trust` accepts any local connection and is only suitable for a throwaway development cluster; use `scram-sha-256` otherwise). The server time zone is taken from the operating system; an explicit `timezone = 'UTC'` in `postgresql.conf` avoids surprises.

The `psql` client has meta-commands (starting with a backslash) that read the catalog:

Command	Output
`\l`	list databases
`\c dbname`	connect to another database
`\dt`	list tables of the current schema
`\d table`	columns, indexes, constraints and referencing foreign keys of a table
`\d+ table`	additionally storage and statistics settings
`\di, \dv, \ds`	list indexes, views, sequences
`\du`	list roles
`\x`	toggle expanded (vertical) output
`\timing`	print the execution time of each statement
`\conninfo`	show the current connection

Frequently used psql meta-commands.

Data types compared

Need	PostgreSQL 16	MySQL 8 (InnoDB)
Surrogate key	`bigint GENERATED ALWAYS AS IDENTITY` (`serial` is an older shortcut)	`BIGINT UNSIGNED NOT NULL AUTO_INCREMENT`
Small integer / integer / big integer	`smallint` 2 B, `integer` 4 B (to 2,147,483,647), `bigint` 8 B	`SMALLINT`, `INT` (also `UNSIGNED`: to 4,294,967,295), `BIGINT`
Exact decimal	`numeric(p,s)`	`DECIMAL(p,s)` (same behaviour)
Approximate	`real` 4 B, `double precision` 8 B	`FLOAT`, `DOUBLE`
Variable text	`text` (unbounded) or `varchar(n)`; no performance difference	`VARCHAR(n)` (n counts characters; row limit 65,535 B), `TEXT` types
Fixed text	`char(n)`, blank-padded	`CHAR(n)`
Boolean	`boolean` (`true`/`false`)	`BOOLEAN` is an alias of `TINYINT(1)`
Enumeration	`CREATE TYPE ... AS ENUM` (reusable)	`ENUM('a','b')` per column
Unique identifier	`uuid` (16 B), `gen_random_uuid()`	`BINARY(16)` with `UUID_TO_BIN(UUID())`, or `CHAR(36)`
JSON	`json` (text copy) and `jsonb` (binary, indexable)	`JSON` (binary, validated)
Default character set	set per database (UTF8)	`utf8mb4` with collation `utf8mb4_0900_ai_ci` (case- and accent-insensitive)

Numeric and character types.

Preview

Need	PostgreSQL 16	MySQL 8
Calendar date	`date`	`DATE`
Time of day	`time`	`TIME`
Instant (absolute time)	`timestamptz` (stored as UTC, shown in the session zone)	`TIMESTAMP` (stored as UTC, converted by session `time_zone`; range 1970 to 2038-01-19)
Local date and time without zone	`timestamp` (stored as typed)	`DATETIME` (stored as typed; range year 1000 to 9999)
Interval	`interval`	no equivalent type
Current time	`now()`, `current_timestamp`	`NOW()`, `CURRENT_TIMESTAMP`

Date and time types.

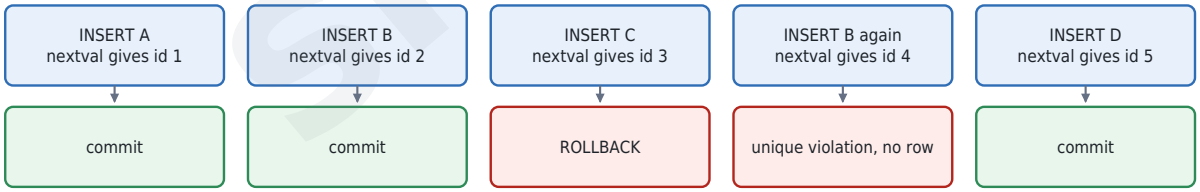
Seven rules follow from the type behaviour demonstrated later in the chapter.

1. Money is `numeric(p,2)` (or integer minor units), never `real` or `double precision`.
2. An instant in time (an event that happened) is `timestamptz` in PostgreSQL, and in MySQL `TIMESTAMP` with the 2038 limit in mind, or `DATETIME` that always holds UTC by convention. A `timestamp` without zone is for local wall-clock values that have no instant (a shop opening time, a birthday).
3. Use `text` in PostgreSQL; add `CHECK (length(code) <= 10)` only when the limit is a business rule. Avoid `char(n)`: it pads with blanks and offers no advantage.
4. Use `bigint` identity keys unless the table is provably small; changing `int` to `bigint` later rewrites the table and every referencing column.
5. Use `boolean`, not `char(1)` with `'Y'` and `'N'`.
6. Prefer a check constraint or a lookup table over an `ENUM` when the set of values changes often (details below).
7. Keep `jsonb` for genuinely schemaless payloads; relationships belong in columns and foreign keys.

Identity columns, sequences and AUTO_INCREMENT

PostgreSQL implements generated keys with a **sequence** object. `GENERATED ALWAYS AS IDENTITY` creates an owned sequence and refuses explicit values unless `OVERRIDING SYSTEM VALUE` is given; `GENERATED BY DEFAULT AS IDENTITY` permits explicit values. MySQL uses the `AUTO_INCREMENT` attribute, with a counter kept per table. In both systems the counter advances **outside the transaction**: a rolled back insert, a failed insert and an `INSERT ... ON CONFLICT DO NOTHING` (or `INSERT IGNORE`) all consume values. Gapless numbering therefore cannot be obtained from a sequence.

Identity values are consumed outside the transaction, so gaps are normal



Rows present: 1, 2, 5. The sequence is never rolled back: concurrent transactions may already hold the next values.

Gaps are harmless; a business rule requiring gapless numbers (invoices) needs a separate counter table updated in the same transaction.

Figure 1.25. The sequence hands out a value before the row is known to be valid, and it is never rolled back; rows 1, 2 and 5 remain from five attempts.

Feature	PostgreSQL 16	MySQL 8
Primary, unique, foreign key	Enforced	Enforced (InnoDB); MyISAM ignores foreign keys
`CHECK`	Enforced	Enforced since 8.0.16; ignored in earlier versions
Deferrable constraints	`DEFERRABLE INITIALLY DEFERRED`	Not supported
DDL inside a transaction	Transactional: a failed migration rolls back completely	Causes an implicit commit; a failed migration leaves partial changes

Feature	PostgreSQL 16	MySQL 8
Partial and expression indexes	Supported	Functional indexes since 8.0.13; no partial indexes
Identifier length	63 bytes	64 characters
Case of unquoted names	Folded to lower case	Depends on operating system (table names), case-insensitive for columns
Invalid data (overflow, wrong type)	Error	Error under the default strict `sql_mode`; older or relaxed modes truncate with a warning

Constraints and DDL behaviour compared.

The last row matters in practice: a MySQL server running with a relaxed `sql_mode` silently stores a truncated value where PostgreSQL always raises an error. Check `SELECT @@sql_mode;` and keep `STRICT_TRANS_TABLES` enabled.

Naming and design conventions

- Tables in singular or plural consistently (this book uses singular, except `orders`, a reserved word); snake_case lower-case identifiers avoid quoting. Quoted mixed-case identifiers are a standing source of errors in PostgreSQL.
- Name keys and constraints explicitly in migration scripts (`fk_orders_customer`, `uq_product_sku`), so that error messages and later `ALTER` statements can refer to them; the generated names (`orders_customer_id_fkey`) work but vary by version.
- Foreign key column named after the referenced key (`customer_id`). PostgreSQL does not index foreign key columns automatically; index every foreign key that is used in joins or in deletes of the parent.
- Every table has a primary key. Every natural candidate key has a `UNIQUE` constraint. Every column that must have a value is `NOT NULL`: nullable by default is the unsafe default.
- Store `created_at` and `updated_at` on mutable entities, as `timestamptz`.

1.5.2 Worked example

The constraint-based design of the shop schema (in the previous chapter) is translated to MySQL 8. The two dialects differ in the key syntax, the boolean and the timestamp. The MySQL text below is from documentation knowledge and was not run on the build machine (no MySQL server is installed).

Element	PostgreSQL (run)	MySQL 8 (typical, not run)
Key	`customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY`	`customer_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT, PRIMARY KEY (customer_id)`
Instant	`created_at timestamptz NOT NULL DEFAULT now()`	`created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`
Boolean	`is_default boolean NOT NULL DEFAULT false`	`is_default TINYINT(1) NOT NULL DEFAULT 0`
Check	`CHECK (price >= 0)`	`CHECK (price >= 0)` (8.0.16 or later)
Foreign key	`REFERENCES customer (customer_id) ON DELETE CASCADE`	same clause; the referencing column type must match exactly, including `UNSIGNED`
Engine	one storage engine	`ENGINE=InnoDB` (default)

The same two tables in both dialects.

mysql_customer.sql (MySQL 8, typical, not run) SQL

```sql
CREATE TABLE customer (
    customer_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    email       VARCHAR(254)    NOT NULL,
    full_name   VARCHAR(200)    NOT NULL,
    created_at  TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (customer_id),
    UNIQUE KEY uq_customer_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

In MySQL the unique constraint on `email` is already case-insensitive because of the default collation `utf8mb4_0900_ai_ci`; the equivalent in PostgreSQL requires `CREATE UNIQUE INDEX ... ON customer (lower(email))` or the `citext` type.

Types in action: PostgreSQL experiments

All outputs below are from PostgreSQL 16.15 on the build machine.

Exact versus approximate numbers.

float8 versus numeric — TERMINAL

```
ddl=# select 0.1::float8 + 0.2::float8 as float_sum,
ddl-#        (0.1::float8 + 0.2::float8) = 0.3::float8 as float_equal,
ddl-#        0.1::numeric + 0.2::numeric as numeric_sum;
      float_sum      | float_equal | numeric_sum
---------------------+-------------+-------------
 0.30000000000000004 | f           |         0.3

ddl=# select sum(0.10::float4) from generate_series(1,1000000);
    sum
-----------
 100958.34
ddl=# select sum(0.10::numeric) from generate_series(1,1000000);
     sum
-------------
 100000.00
```

Binary floating point cannot represent 0.1 exactly; each addition rounds, and the error accumulates (100,958.34 instead of 100,000 after a million additions in `float4`). `numeric` stores decimal digits and is exact. Constraints on `numeric(p,s)` are strict: `numeric(5,2)` rounds extra decimals and rejects values of 1000 or more:

numeric precision and integer range — TERMINAL

```
ddl=# select 123.456::numeric(5,2), 12345.678::numeric(7,2);
 numeric | numeric
---------+----------
  123.46 | 12345.68
ddl=# select 1234.5::numeric(5,2);
ERROR:  numeric field overflow
DETAIL:  A field with precision 5, scale 2 must round to an absolute value less than 10^3.
ddl=# select 2147483647::int + 1;
ERROR:  integer out of range
ddl=# select 2147483647::bigint + 1 as bigint_ok;
 bigint_ok
------------
 2147483648
```

Character types. `char(n)` pads with blanks and silently ignores the padding in comparisons, `varchar(n)` and `text` reject overlong values with an error:

char, varchar, text — TERMINAL

```
ddl=# create table t_str (a char(5), b varchar(5), c text);
ddl=# insert into t_str values ('ab','ab','ab');
ddl=# select length(a) la, octet_length(a) oa, length(b) lb, a = 'ab' as a_eq from t_str;
 la | oa | lb | a_eq
----+----+----+------
  2 |  5 |  2 | t
ddl=# insert into t_str values ('abcdefg','x','x');
ERROR:  value too long for type character(5)
```

The `char(5)` value occupies 5 bytes although its `length` is 2. PostgreSQL stores `text` and `varchar` identically; the length limit of `varchar(n)` is only a constraint.

Instants and wall clocks. One `timestamptz` and one `timestamp` column were filled from the same literal in a session whose zone was `Asia/Kolkata`, then read in three sessions:

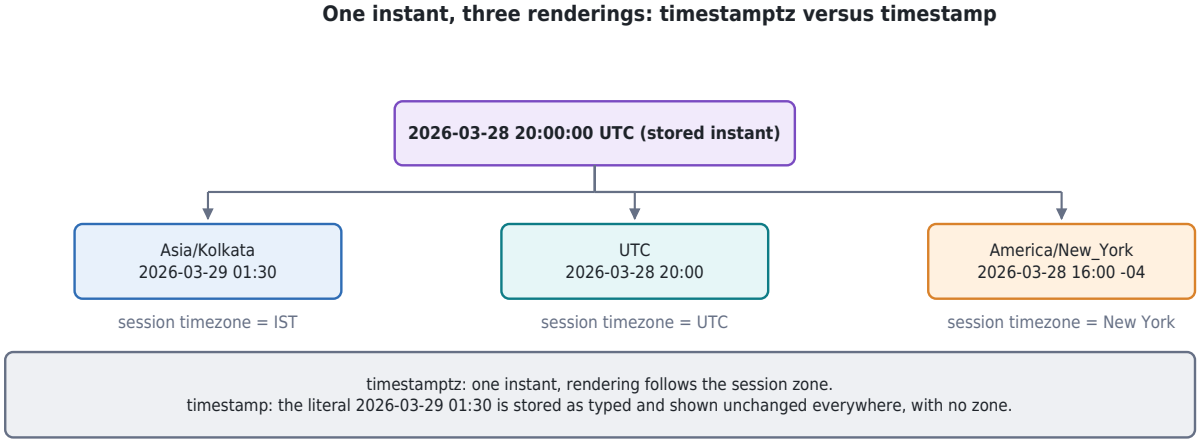


Figure 1.26. The timestamptz column holds one instant and is rendered per session zone; the timestamp column keeps the literal and cannot say which instant it denotes.

timestamp versus timestamptz TERMINAL

```
ddl=# create table ev (id int, naive timestamp, aware timestamptz);
ddl=# set timezone = 'Asia/Kolkata';
ddl=# insert into ev values (1, '2026-03-29 01:30', '2026-03-29 01:30');
ddl=# set timezone = 'UTC';
ddl=# select id, naive, aware from ev;
 id |        naive        |         aware
----+---------------------+-----------------------
  1 | 2026-03-29 01:30:00 | 2026-03-28 20:00:00+00
ddl=# set timezone = 'America/New_York';
ddl=# select id, naive, aware from ev;
 id |        naive        |         aware
----+---------------------+-----------------------
  1 | 2026-03-29 01:30:00 | 2026-03-28 16:00:00-04
```

The `aware` value is the instant 20:00 UTC and is displayed in each session's zone. The `naive` value stays `01:30` and no one can tell whether that was Kolkata or New York time. Note that `timestamptz` does not store a zone: it stores a UTC instant and converts on input and output. A "date boundary" also depends on the zone:

The calendar date of one instant depends on the zone TERMINAL

```
ddl=# select (timestamptz '2026-03-31 23:30+00') at time zone 'Asia/Kolkata' as ist,
ddl-#        ((timestamptz '2026-03-31 23:30+00') at time zone 'Asia/Kolkata')::date as ist_date,
ddl-#        (timestamptz '2026-03-31 23:30+00')::date as utc_date;
         ist         |  ist_date  |  utc_date
---------------------+------------+------------
 2026-04-01 05:00:00 | 2026-04-01 | 2026-03-31
```

An order placed at 23:30 UTC on 31 March is on 1 April in India. A monthly report that groups by `placed_at::date` in a UTC session assigns it to March; the business expected April.

Other types. Enumerations reject unknown values and can be extended, but values cannot be removed or reordered without recreating the type; `uuid` takes 16 bytes against 8 for `bigint`; `jsonb` normalises its input (it dropped the duplicate key and the extra whitespace):

enum, uuid, jsonb TERMINAL

```
ddl=# create type order_status as enum ('NEW','PAID','SHIPPED');
ddl=# insert into ord (st) values ('REFUNDED');
ERROR:  invalid input value for enum order_status: "REFUNDED"
ddl=# alter type order_status add value 'DELIVERED';
ddl=# select enum_range(null::order_status);
          enum_range
-----------------------------
 {NEW,PAID,SHIPPED,DELIVERED}
ddl=# select pg_column_size(gen_random_uuid()) as uuid_bytes,
ddl-#        pg_column_size(1::bigint) as bigint_bytes;
 uuid_bytes | bigint_bytes
------------+--------------
         16 |            8
ddl=# select '{"a": 1, "a": 2, "b":  [1,2]}'::jsonb as jsonb_out,
```

enum, uuid, jsonb (continued) TERMINAL

```
ddl-#         '{"a": 1, "a": 2, "b":  [1,2]}'::json as json_out;
       jsonb_out       |           json_out
-----------------------+------------------------------
 {"a": 2, "b": [1, 2]} | {"a": 1, "a": 2, "b":  [1,2]}
```

Random UUID primary keys (version 4) scatter inserts across the whole B-tree index, which hurts cache locality on large tables; time-ordered identifiers (UUIDv7, ULID) or `bigint` keys avoid that. Whether to use UUIDs is a design decision about exposure of identifiers and distributed generation, not a default.

Types in Java

The Java mapping must preserve what the column guarantees. The following program runs on the plain JDK and shows the two mistakes most often made at the boundary: `double` for money and `LocalDateTime` for an instant.

MoneyDemo.java JAVA

```java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;

public class MoneyDemo {
    public static void main(String[] args) {
        double d = 0.0;
        for (int i = 0; i < 10; i++) {
            d += 0.1;
        }
        System.out.println("double sum of ten times 0.1 = " + d);
        System.out.println("double 0.1 + 0.2 = " + (0.1 + 0.2));

        BigDecimal b = BigDecimal.ZERO;
        for (int i = 0; i < 10; i++) {
            b = b.add(new BigDecimal("0.1"));
        }
        System.out.println("BigDecimal sum = " + b);
        System.out.println("new BigDecimal(0.1) = " + new BigDecimal(0.1));
        BigDecimal one = new BigDecimal("1.0");
        BigDecimal oneTwo = new BigDecimal("1.00");
        System.out.println("equals 1.0 vs 1.00: " + one.equals(oneTwo)
                + ", compareTo: " + one.compareTo(oneTwo));
        BigDecimal x = new BigDecimal("2.345");
        System.out.println("2.345 HALF_UP = " + x.setScale(2, RoundingMode.HALF_UP)
                + ", HALF_EVEN = " + x.setScale(2, RoundingMode.HALF_EVEN));

        Instant when = Instant.parse("2026-03-31T23:30:00Z");
        System.out.println("Instant              = " + when);
        ZoneOffset ist = ZoneOffset.ofHoursMinutes(5, 30);
        System.out.println("as OffsetDateTime IST= " + when.atOffset(ist));
        LocalDateTime naive = LocalDateTime.ofInstant(when, ZoneId.of("UTC"));
        System.out.println("LocalDateTime (UTC)  = " + naive + "  (no zone information)");
        ZoneId kolkata = ZoneId.of("Asia/Kolkata");
        System.out.println("local date in Kolkata= " + when.atZone(kolkata).toLocalDate());
    }
}
```

Output of MoneyDemo (JDK 21) TERMINAL

```
$ java MoneyDemo.java
double sum of ten times 0.1 = 0.9999999999999999
double 0.1 + 0.2 = 0.30000000000000004
BigDecimal sum = 1.0
new BigDecimal(0.1) = 0.1000000000000000055511151231257827021181583404541015625
equals 1.0 vs 1.00: false, compareTo: 0
2.345 HALF_UP = 2.35, HALF_EVEN = 2.34
Instant              = 2026-03-31T23:30:00Z
as OffsetDateTime IST= 2026-04-01T05:00+05:30
LocalDateTime (UTC)  = 2026-03-31T23:30  (no zone information)
local date in Kolkata= 2026-04-01
```

What to notice. `BigDecimal.equals` compares scale as well as value (1.0 differs from 1.00), so compare with `compareTo`. Construct `BigDecimal` from a `String`, never from a `double`: `new BigDecimal(0.1)` exposes the binary error.

The rounding mode is a business decision: `HALF_EVEN` (banker's rounding) gave 2.34 where `HALF_UP` gave 2.35. In JPA map `numeric` columns to `BigDecimal` and `timestamptz` columns to `Instant` or `OffsetDateTime` (Hibernate 6 and the PostgreSQL JDBC driver support both); `LocalDateTime` corresponds to `timestamp` without zone. Configure the JVM and the connection to UTC (`hibernate.jdbc.time_zone=UTC`) so that conversion happens in one place. The Spring and JDBC snippets in this chapter are typical code and were not run against a driver on the build machine.

1.5.3 Hands-on

A DDL review checklist applied to every new table:

1. Does the table have a primary key, and is every natural key `UNIQUE NOT NULL`?
2. Is every relationship a foreign key with a deliberate `ON DELETE` action, and is the referencing column indexed?
3. Are amounts `numeric`, instants `timestamptz`, flags `boolean`, free text `text`?
4. Which business rules fit a `CHECK` (non-negative price, rating range, status set)?
5. Is every column that must always have a value `NOT NULL`?
6. Is the migration script repeatable and versioned (Flyway `V1__create_shop.sql`)?

In a Spring Boot project the schema lives in versioned Flyway migrations (`src/main/resources/db/migration/V1__create_shop.sql`) and Hibernate's `spring.jpa.hibernate.ddl-auto` is set to `validate` or `none`; letting Hibernate generate the production schema (`update`, `create`) produces types such as `float(53)` or `varchar(255)` chosen by the mapper, not by a designer.

1.5.4 Real-world problems and solutions

Incident: floating point used for money

Scenario. Prices and ledger amounts are stored in `real` (Java `float` or `double` columns generated by the ORM). The ledger has a million entries of 19.99.

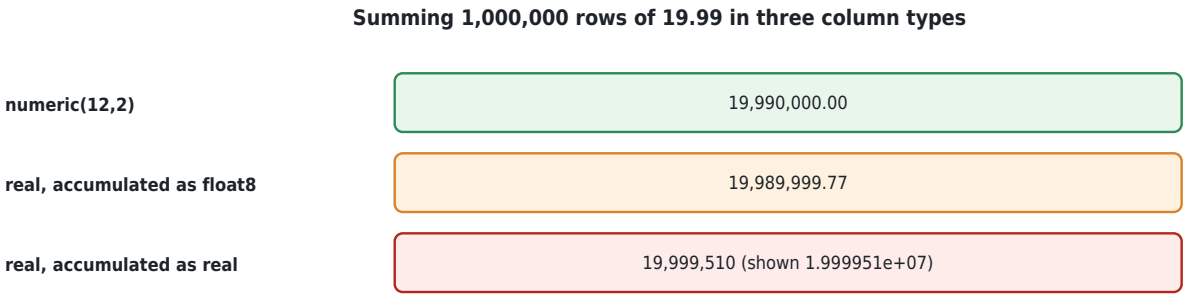
Symptoms. The finance system and the application disagree on the daily total by a few thousand units; an equality search for a price returns no rows although the row is visible.

Diagnosis. Compare the sum in both representations and the equality test.

Float ledger versus numeric ledger (PostgreSQL 16, 1,000,000 rows) TERMINAL

```
ddl=# select (select sum(amount) from ledger_f) as sum_real,
ddl-#        (select sum(amount::float8) from ledger_f) as sum_real_as_float8,
ddl-#        (select sum(amount) from ledger_n) as sum_numeric;
   sum_real   | sum_real_as_float8 | sum_numeric
--------------+--------------------+-------------
 1.999951e+07 |  19989999.771118164 | 19990000.00

ddl=# select count(*) from ledger_f where amount = 19.99;
 count
-------
     0
ddl=# select count(*) from ledger_f where amount = 19.99::real;
  count
---------
 1000000
ddl=# select amount::numeric, amount::float8 from ledger_f limit 1;
 amount  |       amount
---------+--------------------
   19.99 | 19.989999771118164
```



Also: SELECT count(*) FROM ledger_f WHERE amount = 19.99 returns 0 rows, because the literal is numeric and the comparison converts the real column to float8 (19.989999771118164).

Figure 1.27. The same column of 19.99 sums to three different values depending on the type and accumulator; only numeric is exact.

Root cause. `19.99` has no exact binary representation. The `real` column holds 19.989999771118164. The literal `19.99` is `numeric`, so the comparison converts the `real` column to `float8` and compares 19.989999771118164 with 19.99: no match. The sum of one million `real` values accumulates rounding error (19,999,510 instead of 19,990,000) because the accumulator is also `real`.

Fix. Immediate: report from a corrected query that casts to `numeric(12,2)` after rounding. Permanent: migrate the column in a transaction, `ALTER TABLE ledger_f ALTER COLUMN amount TYPE numeric(12,2) USING round(amount::numeric, 2);`; the rewrite takes an exclusive lock on the table for its duration, so schedule it or use a new column and a backfill. Change the Java type to `BigDecimal`.

Prevention. A schema review rule: no `real`, `double precision`, `float` in any column that holds an amount. Amounts carry currency: a `numeric(12,2)` column plus a `char(3)` currency code, or integer minor units.

Incident: timestamp without zone produces wrong day boundaries

Scenario. An orders table stores `placed_at timestamp` (no zone). The application server runs in UTC; a second reporting job runs in Asia/Kolkata. The business operates in India.

Symptoms. Daily totals are off by one day for orders placed between 18:30 and 24:00 UTC; after a daylight-saving change in another region's job the hourly chart has a missing or duplicated hour.

Diagnosis. Check the column type and the session zone of every writer, then compare the same row from two sessions.

Same row, different conclusions TERMINAL

```
ddl=# select column_name, data_type from information_schema.columns
ddl-#  where table_name = 'ev' and column_name in ('naive','aware');
 column_name |          data_type
-------------+----------------------------
 naive       | timestamp without time zone
 aware       | timestamp with time zone
```

Root cause. A `timestamp` stores the digits and has no instant. Writers in different zones wrote different instants as the same digits, and a reader cannot reconstruct which. Date-boundary grouping then uses whichever zone the reporting session happens to have.

Fix. Add a `timestamptz` column, backfill it by declaring the zone the digits were written in (`placed_at AT TIME ZONE 'UTC'`), switch readers and drop the old column. Group by day with an explicit zone: `date_trunc('day', placed_at AT TIME ZONE 'Asia/Kolkata')`. **Prevention.** Instants are `timestamptz`; the JVM and database sessions run in UTC; conversion to a zone happens only at the presentation or reporting boundary, with the business zone named explicitly.

Incident: auto-increment gaps and integer overflow

Scenario. An invoice table uses an identity `int` key. Auditors ask why invoice numbers 3 and 4 are missing, and a year later inserts begin to fail.

Symptoms. Gaps in ids after rollbacks and conflicts; then `ERROR: nextval: reached maximum value of sequence` in the application log, and all inserts fail.

Diagnosis. Reproduce the gaps, then inspect the sequence.

Gaps from rollback and conflicts, then the overflow TERMINAL

```
ddl=# insert into inv (ref) values ('A'),('B');
ddl=# begin; insert into inv (ref) values ('C'); rollback;
ddl=# insert into inv (ref) values ('B') on conflict do nothing;
ddl=# insert into inv (ref) values ('D');
ddl=# select * from inv order by id;
 id | ref
----+-----
  1 | A
  2 | B
  5 | D
ddl=# select sequencename, data_type, max_value, last_value from pg_sequences
ddl-#  where sequencename = 'inv_id_seq';
 sequencename | data_type | max_value  | last_value
--------------+-----------+------------+-----------
 inv_id_seq   | integer   | 2147483647 |          6
ddl=# insert into inv (ref) values ('G');      -- after the counter reached its maximum
ERROR:  nextval: reached maximum value of sequence "inv_id_seq" (2147483647)
```

Root cause. Ids 3 and 4 were consumed by the rolled-back insert and by the conflicting insert; sequences do not roll back because concurrent transactions may have already used later values. A signed 32-bit key holds 2,147,483,647 values; at 70 inserts per second it lasts about a year, and the failure is abrupt.

Fix. Immediate: widen the column and the sequence. In PostgreSQL altering the column type of an identity column also moves the sequence to `bigint`:

Widening the key TERMINAL

```
ddl=# alter table inv alter column id type bigint;
ALTER TABLE
ddl=# select sequencename, data_type, max_value from pg_sequences where sequencename = 'inv_id_seq';
 sequencename | data_type |      max_value
--------------+-----------+---------------------
 inv_id_seq   | bigint    | 9223372036854775807
ddl=# insert into inv (ref) values ('G') returning id;
     id
------------
 2147483648
```

The statement rewrites the table under an exclusive lock, so in production the change is done with a new column and a staged migration (see the schema migration chapter later in this Part); every foreign key column referencing the key must be widened as well, and Java entities must use `Long`. In MySQL the failure appears as `ERROR 1062 (23000): Duplicate entry '2147483647' for key 'PRIMARY'` because the counter sticks at the maximum (typical, not run).
Prevention. Use `bigint` for keys of growing tables; monitor `last_value / max_value` of every sequence and alert at 50 percent; if legal numbering must be gapless (tax invoices), generate it in a separate counter row updated in the same transaction, accepting that this serialises invoice creation.

1.5.5 Questions to expect

Why should money not be stored as float?

Binary floating point cannot represent most decimal fractions exactly, so sums accumulate error and equality comparisons fail. Use `numeric(p,s)` (or integer minor units) and `BigDecimal` in Java.

What is the difference between `timestamp` and `timestamptz` in PostgreSQL?

`timestamptz` stores an absolute instant (UTC) and converts to the session time zone on output; `timestamp` stores the literal date and time with no zone. Instants belong in `timestamptz`.

TIMESTAMP versus DATETIME in MySQL?

`TIMESTAMP` is stored in UTC, converted with the session `time_zone` and limited to 1970-2038; `DATETIME` stores the literal value, has no conversion and a range from year 1000 to 9999.

`serial`, identity column and `AUTO_INCREMENT`: what is the difference and why are there gaps?

`serial` is shorthand for an integer column with a default drawn from an owned sequence; `GENERATED ... AS IDENTITY` is the SQL-standard form; `AUTO_INCREMENT` is MySQL's per-table counter. All advance outside the transaction, so rollbacks and conflicts leave gaps.

`varchar(n)` or `text` in PostgreSQL?

They have the same storage and speed. Use `text` and add a check constraint only when the limit is a business rule; the length in `varchar(n)` is an error check, not an optimisation.

When is an enum appropriate?

For a small, stable set (order status) where the DB should reject unknown values. If values change often, use a lookup table with a foreign key or a text column with a check constraint: dropping an enum value requires recreating the type.

Why are UUID primary keys sometimes a problem?

Random v4 UUIDs are 16 bytes and insert at random positions in the index, increasing page splits and cache misses. Time-ordered identifiers or `bigint` keys avoid this; UUIDs are chosen for distributed generation or for hiding sequence information.

Does PostgreSQL support transactional DDL, and does it matter?

Yes: `CREATE`, `ALTER` and `DROP` roll back with the transaction, so a failed multi-statement migration leaves no partial change. MySQL commits implicitly at each DDL statement, so migrations must be written to be re-runnable.

How do you map money and instants in a Spring Data JPA entity?

`BigDecimal` for `numeric`, `Instant` or `OffsetDateTime` for `timestamptz`, `Long` for `bigint` identity keys, with `@Column(precision = 12, scale = 2)` for amounts and Flyway for the schema.

KEY POINTS

Key points

- Run PostgreSQL locally with `initdb`, `pg_ctl` and `psql`; encoding and locale are fixed at creation.
- Money is `numeric`; instants are `timestamptz`; keys are `bigint`; flags are `boolean`; free text is `text`.
- Identity and AUTO_INCREMENT counters are not transactional: gaps are normal, overflow is abrupt, gapless numbering needs a separate design.
- PostgreSQL and MySQL differ in DDL transactions, `CHECK` and strictness history, boolean and timestamp semantics, and identifier handling.
- Declare constraints in the schema (not null, unique, foreign key, check) and manage the schema with versioned migrations, not ORM auto-generation.

1.6 SQL Fundamentals

Structured Query Language (SQL) is the standard language for defining, populating, querying and controlling relational data. A Java service rarely sends anything else to its database: every `PreparedStatement`, every Spring Data derived query and every Hibernate-generated statement is SQL by the time it reaches the server. A developer who reads SQL fluently can predict what an ORM sends, can diagnose why a statement is slow or wrong, and can replace generated code with a hand-written statement when the generated code is inadequate.

SQL is declarative. A statement describes the result that is required, not the steps used to produce it. The database planner chooses the access paths, the join order and the join algorithms (the subject of the query-processing chapters). Two consequences follow. First, equivalent statements can run in very different times, so the structure of a statement matters. Second, the order in which clauses are written is not the order in which they are evaluated, which is the most common source of confusion for new users and is treated in detail below.

This chapter and the following ones use one sample schema throughout: `departments`, `employees`, `customers`, `products`, `orders` and `order_items`. All outputs are real results from PostgreSQL 16.15 running on the build machine. Statements that differ in MySQL 8.x are shown beside the PostgreSQL form; MySQL output was not produced on the build machine and is labelled accordingly.

1.6.1 The sample schema

The schema is small enough to be read in full and rich enough to expose every behaviour discussed in this part: a self-referencing foreign key (`employees.manager_id`), a nullable foreign key (`employees.dept_id`), a department without employees, customers without orders and a one-to-many-to-many chain from customers to orders to order lines.

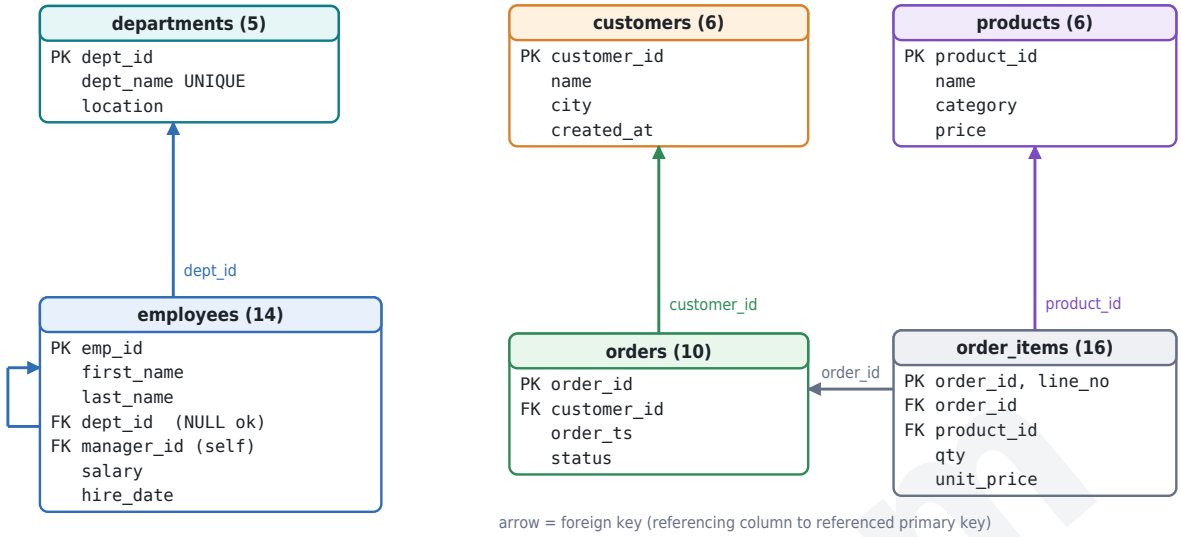


Figure 1.28. The sample schema. Arrows point from the referencing column to the referenced primary key. Row counts are those loaded in the examples; the self-reference on `employees` models the management hierarchy.

schema.sql (abridged) SQL

```sql
CREATE TABLE departments (
  dept_id    integer PRIMARY KEY,
  dept_name  text NOT NULL UNIQUE,
  location   text NOT NULL
);
CREATE TABLE employees (
  emp_id     integer PRIMARY KEY,
  first_name text NOT NULL,
  last_name  text NOT NULL,
  dept_id    integer CONSTRAINT emp_dept_fk REFERENCES departments(dept_id),
  manager_id integer CONSTRAINT emp_mgr_fk REFERENCES employees(emp_id),
  salary     numeric(10,2) NOT NULL CHECK (salary > 0),
  hire_date  date NOT NULL
);
CREATE TABLE orders (
  order_id    integer PRIMARY KEY,
  customer_id integer NOT NULL REFERENCES customers(customer_id),
  order_ts    timestamptz NOT NULL,
  status      text NOT NULL CHECK (status IN ('NEW','PAID','SHIPPED','CANCELLED'))
);
CREATE TABLE order_items (
  order_id   integer REFERENCES orders(order_id),
  line_no    integer,
  product_id integer NOT NULL REFERENCES products(product_id),
  qty        integer NOT NULL CHECK (qty > 0),
  unit_price numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, line_no)
);
```

The `employees` table holds fourteen rows. Employee 13 has no department (`dept_id` is NULL), employee 1 has no manager, and department 50 (`Legal`) has no employees. These deliberate irregularities produce the interesting behaviour in later chapters.

SQL

```sql
SELECT emp_id, first_name, last_name, dept_id, manager_id, salary, hire_date
FROM employees ORDER BY emp_id;
```

Result TERMINAL

```
 emp_id | first_name | last_name | dept_id | manager_id |  salary   | hire_date
--------+------------+-----------+---------+------------+-----------+------------
      1 | Asha       | Rao       |      10 |            | 250000.00 | 2015-03-01
      2 | Vikram     | Nair      |      10 |          1 | 180000.00 | 2016-07-15
      3 | Meera      | Iyer      |      10 |          2 | 150000.00 | 2018-01-10
      4 | Ravi       | Kumar     |      10 |          2 | 150000.00 | 2019-05-20
      5 | Sneha      | Reddy     |      10 |          3 | 120000.00 | 2021-09-01
      6 | Arjun      | Das       |      20 |          1 | 170000.00 | 2017-02-11
      7 | Priya      | Menon     |      20 |          6 | 110000.00 | 2020-06-30
      8 | Kiran      | Shah      |      20 |          6 |  95000.00 | 2022-03-14
      9 | Divya      | Pillai    |      30 |          1 | 140000.00 | 2016-11-05
     10 | Rahul      | Verma     |      30 |          9 |  80000.00 | 2023-01-09
     11 | Neha       | Gupta     |      40 |          1 | 160000.00 | 2018-08-21
     12 | Sanjay     | Bose      |      40 |         11 |  90000.00 | 2022-10-03
     13 | Tara       | Joshi     |         |          1 | 100000.00 | 2024-02-12
     14 | Omar       | Khan      |      20 |          6 | 110000.00 | 2024-04-01
(14 rows)
```

1.6.2 Concept and mechanism

Statement categories

SQL statements are conventionally grouped by purpose. The grouping matters in practice because it determines privileges (a role may be allowed to read but not to change data), transactional behaviour (PostgreSQL executes DDL inside transactions, MySQL does not) and which tool a Java application uses to issue the statement.

Category	Statements	Purpose
DDL (definition)	`CREATE, ALTER, DROP, TRUNCATE`	Create and change schema objects
DML (manipulation)	`INSERT, UPDATE, DELETE, MERGE`	Change rows
DQL (query)	`SELECT`	Read rows
DCL (control)	`GRANT, REVOKE`	Manage privileges
TCL (transaction)	`BEGIN, COMMIT, ROLLBACK, SAVEPOINT`	Group statements into units of work

Statement categories.

In PostgreSQL every statement, including DDL, is transactional: `BEGIN; DROP TABLE t; ROLLBACK;` leaves `t` intact. In MySQL (InnoDB) DDL statements cause an implicit commit and cannot be rolled back; a failed multi-step migration therefore leaves partial changes. This difference is significant for the migration chapter later in the book.

Logical processing order of SELECT

A SELECT statement is written in the order SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, but it is evaluated logically in a different order: FROM (and joins) first, then WHERE, GROUP BY, HAVING, the SELECT list, DISTINCT, ORDER BY and finally LIMIT/OFFSET. The planner is free to execute the physical steps in any order that yields the same result, so this is a rule about meaning, not about performance. The meaning explains which names are visible where.

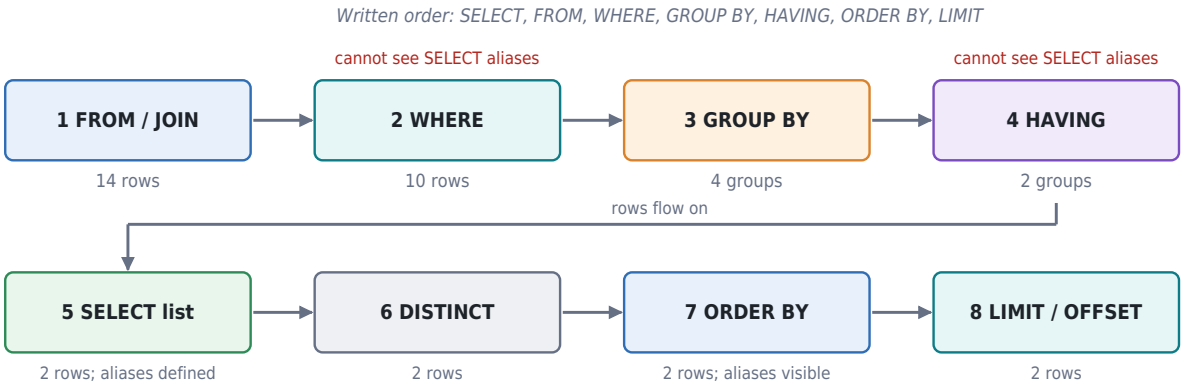


Figure 1.29. Logical evaluation order of a SELECT statement, with the intermediate row count for the query shown below it. The alias `annual` exists only after the SELECT step, so WHERE and HAVING cannot reference it, while ORDER BY can.

The following statement exercises every stage. On the sample data FROM yields 14 rows; WHERE removes employee 13 (no department, so the predicate `dept_id IS NOT NULL` is false) and the three employees with salary below 100000, leaving 10 rows; GROUP BY forms four groups; HAVING keeps the two groups with at least two members.

SQL

```sql
SELECT dept_id, count(*) AS headcount, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IS NOT NULL AND salary >= 100000
GROUP BY dept_id
HAVING count(*) >= 2
ORDER BY avg_salary DESC
LIMIT 3;
```

Result TERMINAL

```
 dept_id | headcount | avg_salary
---------+-----------+------------
      10 |         5 |     170000
      20 |         3 |     130000
(2 rows)
```

The statement filters rows first (WHERE), groups the survivors, discards groups with fewer than two members (HAVING), sorts the groups and returns at most three of them (two exist). The alias `avg_salary` is usable in ORDER BY because ORDER BY runs after the SELECT list. Using an alias in WHERE fails:

SQL

```sql
SELECT salary * 12 AS annual FROM employees WHERE annual > 2000000;
```

Result TERMINAL

```
ERROR:  column "annual" does not exist
LINE 1: SELECT salary * 12 AS annual FROM employees WHERE annual > 2...
                                                          ^
```

PostgreSQL reports that the column does not exist. MySQL differs here: it also rejects an alias in WHERE but accepts an alias in GROUP BY and HAVING, an extension that PostgreSQL permits only in GROUP BY and ORDER BY (not HAVING). Portable SQL repeats the expression or wraps the query in a subquery or common table expression.

1.6.3 Reading data: SELECT

Projection, expressions and aliases

The SELECT list may contain columns, constants, arithmetic, function calls and subqueries. `AS` introduces an alias; the keyword is optional but recommended because a missing comma between two column names silently turns the second into an alias of the first.

SQL

```sql
SELECT first_name || ' ' || last_name AS full_name,
       salary,
       salary * 12 AS annual,
       salary / 3 AS int_or_numeric
FROM employees WHERE emp_id IN (3, 10);
```

Result TERMINAL

```
  full_name  |  salary   |   annual   |   int_or_numeric
-------------+-----------+------------+--------------------
 Meera Iyer  | 150000.00 | 1800000.00 | 50000.000000000000
 Rahul Verma |  80000.00 |  960000.00 | 26666.666666666667
(2 rows)
```

String concatenation uses `||` in standard SQL and PostgreSQL. In MySQL `||` is by default a logical OR; the portable function is `CONCAT(a, b)`, which PostgreSQL also provides. Division of two integers truncates in PostgreSQL (`7 / 2` is 3), while the MySQL `/` operator always returns a decimal (`7 / 2` is 3.5000) and `DIV` performs integer division. The query above divides a `numeric` value, so the fraction is kept.

Filtering with WHERE

WHERE accepts a boolean expression. A row is kept only when the expression is TRUE; both FALSE and NULL (unknown) discard the row. This three-valued behaviour is detailed in the NULL chapter later in this part; the practical consequence is that `col = NULL` never matches and `IS NULL` must be used.

Predicate	Meaning	Example
`=`, `<>`, `<`, `<=`, `>`, `>=`	Comparison (`!=` accepted by both engines)	`salary >= 100000`
BETWEEN a AND b	Inclusive range, equivalent to `>= a AND <= b`	`hire_date BETWEEN '2018-01-01' AND '2018-12-31'`
IN (list)	Membership	`dept_id IN (10, 20)`
LIKE / ILIKE	Pattern match: `%` any string, `_` one character; ILIKE is case-insensitive (PostgreSQL only)	`last_name LIKE 'R%'`
IS [NOT] NULL	Null test	`dept_id IS NULL`
IS [NOT] DISTINCT FROM	Null-safe equality	`dept_id IS NOT DISTINCT FROM 10`

Common predicates.

SQL

```sql
SELECT emp_id, last_name, hire_date
FROM employees
WHERE hire_date BETWEEN '2018-01-01' AND '2019-12-31'
   OR last_name LIKE 'R%'
ORDER BY hire_date;
```

Result TERMINAL

```
 emp_id | last_name | hire_date
--------+-----------+------------
      1 | Rao       | 2015-03-01
      3 | Iyer      | 2018-01-10
     11 | Gupta     | 2018-08-21
      4 | Kumar     | 2019-05-20
      5 | Reddy     | 2021-09-01
(5 rows)
```

DISTINCT and DISTINCT ON

`SELECT DISTINCT` removes duplicate result rows, comparing all selected columns together and treating NULLs as equal to each other for this purpose. It is frequently used to hide a join that produced duplicates, which treats a symptom and not the cause (see the join chapter). PostgreSQL additionally offers `DISTINCT ON (expr)`, which keeps the first row of each group of rows that share the expression, where "first" is defined by ORDER BY. It has no MySQL equivalent; a window function is used instead.

SQL

```sql
SELECT DISTINCT location FROM departments ORDER BY location;

SELECT DISTINCT ON (dept_id) dept_id, first_name, salary
FROM employees WHERE dept_id IS NOT NULL
ORDER BY dept_id, salary DESC;
```

Result TERMINAL

```
 location
-----------
 Delhi
 Hyderabad
 Mumbai
(3 rows)

 dept_id | first_name |  salary
---------+------------+-----------
      10 | Asha       | 250000.00
      20 | Arjun      | 170000.00
      30 | Divya      | 140000.00
      40 | Neha       | 160000.00
(4 rows)
```

The second statement returns the highest-paid employee of each department. The ORDER BY must begin with the DISTINCT ON expression; otherwise PostgreSQL rejects the statement.

ORDER BY, LIMIT, OFFSET and FETCH

Without ORDER BY the order of result rows is undefined: the engine returns rows in whatever order its chosen plan produces them, and that order can change after an UPDATE, a VACUUM, a new index or a version upgrade. Any statement that uses LIMIT, a cursor or pagination must have an ORDER BY that is a total order, that is, one that never leaves two rows tied.

PostgreSQL and MySQL both accept `LIMIT n OFFSET m`. The standard form is `OFFSET m ROWS FETCH FIRST n ROWS ONLY`, supported by PostgreSQL (since 8.4), Oracle and SQL Server but not by MySQL. MySQL additionally accepts `LIMIT m, n` with the arguments in the opposite order, a frequent source of mistakes.

SQL

```sql
SELECT emp_id, last_name, salary FROM employees
ORDER BY salary DESC, emp_id
OFFSET 2 ROWS FETCH FIRST 3 ROWS ONLY;

SELECT emp_id, last_name, salary FROM employees
ORDER BY salary DESC
FETCH FIRST 5 ROWS WITH TIES;
```

Result TERMINAL

```
 emp_id | last_name |  salary
--------+-----------+-----------
      6 | Das       | 170000.00
     11 | Gupta     | 160000.00
      3 | Iyer      | 150000.00
(3 rows)

 emp_id | last_name |  salary
--------+-----------+-----------
      1 | Rao       | 250000.00
      2 | Nair      | 180000.00
      6 | Das       | 170000.00
     11 | Gupta     | 160000.00
      3 | Iyer      | 150000.00
      4 | Kumar     | 150000.00
(6 rows)
```

`WITH TIES` (PostgreSQL 13 and later) returns additional rows that tie with the last row on the ORDER BY key; the fifth row has salary 150000 and the sixth row ties with it, so six rows are returned by the second statement although five were requested. MySQL has no equivalent and uses a window function (RANK) instead.

NULLs sort after all non-null values in ascending order in PostgreSQL (and first in MySQL). `NULLS FIRST` and `NULLS LAST` control this in PostgreSQL; MySQL requires `ORDER BY col IS NULL, col`.

sreeram

SQL

```sql
SELECT emp_id, dept_id FROM employees ORDER BY dept_id DESC NULLS LAST, emp_id LIMIT 4;
```

Result TERMINAL

```
 emp_id | dept_id
--------+---------
     11 |      40
     12 |      40
      9 |      30
     10 |      30
(4 rows)
```

1.6.4 Changing data: INSERT, UPDATE, DELETE, UPSERT

INSERT

`INSERT` adds rows. Listing the target columns is mandatory in application code: a statement without a column list depends on the physical column order and breaks when a column is added. Multiple rows can be inserted with one statement; a batch of N single-row statements costs N network round trips, while one multi-row statement costs one.

SQL

```sql
INSERT INTO departments (dept_id, dept_name, location)
VALUES (60, 'Support', 'Pune'), (70, 'Research', 'Pune');

INSERT INTO departments (dept_id, dept_name, location)
VALUES (10, 'Duplicate', 'Nowhere');
```

Result TERMINAL

```
INSERT 0 2
ERROR:  duplicate key value violates unique constraint "departments_pkey"
DETAIL:  Key (dept_id)=(10) already exists.
```

The second statement violates the primary key. PostgreSQL aborts the statement and, inside a transaction, marks the whole transaction as failed (hence the `rb` wrapper used when producing this output). MySQL reports error 1062 "Duplicate entry '10' for key 'departments.PRIMARY'".

UPDATE and DELETE

`UPDATE` and `DELETE` without a WHERE clause affect every row. This is legal, silent and the cause of a well-known class of production incidents; the PITR section of the backup chapter later in the book recovers from one. Habit and tooling help: run the statement as a SELECT first, wrap it in a transaction and check the reported row count before committing. In MySQL the client option `--safe-updates` rejects UPDATE and DELETE without a key in WHERE.

SQL

```sql
UPDATE employees SET salary = salary * 1.10
WHERE dept_id = 30 AND hire_date >= '2023-01-01'
RETURNING emp_id, first_name, salary;

DELETE FROM order_items WHERE order_id = 104;
```

Result TERMINAL

```
 emp_id | first_name |  salary
--------+------------+----------
     10 | Rahul      | 88000.00
(1 row)

UPDATE 1
DELETE 1
```

PostgreSQL also supports `UPDATE ... FROM` and `DELETE ... USING` to modify rows by joining another table, which standard SQL expresses with subqueries; MySQL writes `UPDATE t JOIN u ON ... SET ...`.

RETURNING

`RETURNING` (PostgreSQL, also MariaDB 10.5 and later for some statements, but not MySQL 8.x) returns columns of the affected rows from INSERT, UPDATE and DELETE in the same round trip. It is the reliable way to obtain generated keys and defaults and the basis of the "claim a row" queue pattern. In JDBC the portable route is `Statement.RETURN_GENERATED_KEYS`; the PostgreSQL driver implements it by appending `RETURNING` to the statement.

SQL

```sql
INSERT INTO customers (customer_id, name, city, created_at)
VALUES (7, 'Gopal Electronics', 'Pune', now())
RETURNING customer_id, name, created_at IS NOT NULL AS has_ts;
```

Result TERMINAL

```
 customer_id |       name        | has_ts
-------------+-------------------+--------
           7 | Gopal Electronics | t
(1 row)

INSERT 0 1
```

OrderDao.java (fragment; not run) JAVA

```java
// Spring JdbcTemplate: obtain the generated key of an insert.
KeyHolder keys = new GeneratedKeyHolder();
jdbc.update(con -> {
    PreparedStatement ps = con.prepareStatement(
        "INSERT INTO orders (customer_id, order_ts, status) VALUES (?, now(), 'NEW')",
        new String[] {"order_id"});          // request only this column
    ps.setInt(1, customerId);
    return ps;
}, keys);
long orderId = keys.getKey().longValue();
```

UPSERT: ON CONFLICT and ON DUPLICATE KEY

An upsert inserts a row, or updates the existing row when a uniqueness constraint would be violated, as one atomic statement. The alternative written in application code (SELECT, then INSERT or UPDATE) has a race: two sessions can both find the row missing and both attempt the insert. PostgreSQL uses `INSERT ... ON CONFLICT (columns) DO UPDATE | DO NOTHING`; MySQL uses `INSERT ... ON DUPLICATE KEY UPDATE`. The conflict target in PostgreSQL names the unique index that acts as the arbiter; the pseudo-table `EXCLUDED` holds the row that was proposed.

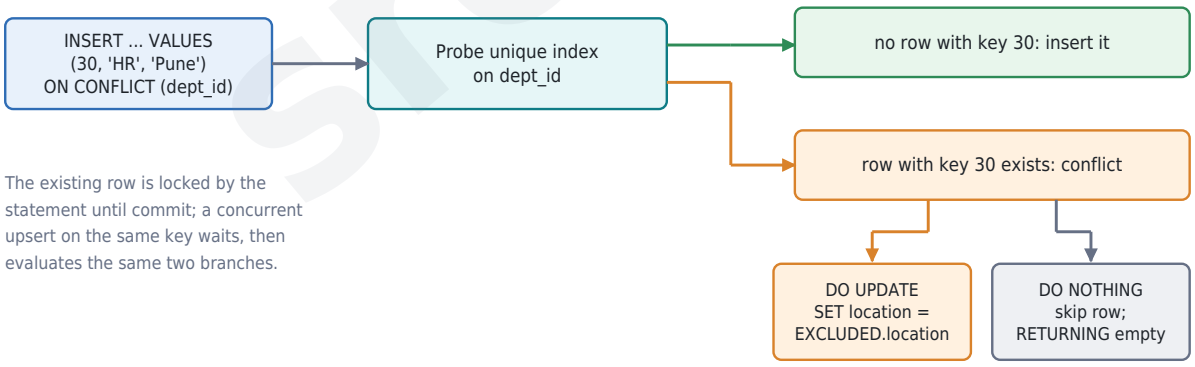


Figure 1.30. Decision path of INSERT ... ON CONFLICT. The unique index is checked first; only when it reports a conflicting live row does the DO UPDATE branch run. Both branches are atomic with respect to concurrent sessions.

Sreeram

SQL

```sql
-- Existing key: update the location. Proposed row is available as EXCLUDED.
INSERT INTO departments (dept_id, dept_name, location)
VALUES (30, 'HR', 'Bengaluru')
ON CONFLICT (dept_id) DO UPDATE SET location = EXCLUDED.location
RETURNING *;

-- DO NOTHING: skip silently; RETURNING yields no row for the skipped input.
INSERT INTO departments (dept_id, dept_name, location)
VALUES (30, 'HR', 'Pune')
ON CONFLICT DO NOTHING
RETURNING *;
```

Result TERMINAL

```
 dept_id | dept_name | location
---------+-----------+-----------
      30 | HR        | Bengaluru
(1 row)

INSERT 0 1
 dept_id | dept_name | location
---------+-----------+----------
(0 rows)

INSERT 0 0
```

PostgreSQL 16	MySQL 8.x
`INSERT INTO departments (dept_id, dept_name, location) VALUES (30, 'HR', 'Pune') ON CONFLICT (dept_id) DO UPDATE SET location = EXCLUDED.location;`	`INSERT INTO departments (dept_id, dept_name, location) VALUES (30, 'HR', 'Pune') AS new ON DUPLICATE KEY UPDATE location = new.location;`
`ON CONFLICT DO NOTHING`	`INSERT IGNORE` (also hides other errors, such as truncation)
Names the arbiter unique index or constraint	Any unique key may trigger the update; ambiguous with several unique keys
`MERGE` (PostgreSQL 15 and later) for multi-branch logic	No `MERGE`

The same upsert in PostgreSQL and MySQL.

The MySQL `AS new` row alias requires 8.0.19 or later; older code uses the deprecated `VALUES(location)` function. When a table has several unique keys, MySQL fires the clause on whichever key conflicts first, which can update an unexpected row. PostgreSQL requires the conflict target to be named explicitly, which removes the ambiguity.

TRUNCATE

`TRUNCATE TABLE t` removes all rows by discarding the table files, which is far faster than `DELETE FROM t` for large tables and reclaims disk space immediately. It acquires an ACCESS EXCLUSIVE lock in PostgreSQL, does not fire row-level triggers, and cannot be used on a table referenced by a foreign key unless the referencing tables are truncated in the same statement (`TRUNCATE a, b` or `CASCADE`). In PostgreSQL TRUNCATE is transactional and MVCC-safe; in MySQL it is DDL, commits implicitly and resets `AUTO_INCREMENT`. PostgreSQL resets sequences only with `RESTART IDENTITY`.

SQL

```sql
BEGIN;
TRUNCATE tickets;
SELECT count(*) AS inside_tx FROM tickets;
ROLLBACK;
SELECT count(*) AS after_rollback FROM tickets;
```

Result TERMINAL

```
BEGIN
TRUNCATE TABLE
 inside_tx
-----------
         0
(1 row)

ROLLBACK
 after_rollback
----------------
          20000
(1 row)
```

1.6.5 Worked example: tracing one query by hand

The statement below lists, per department, the number of employees hired since 2019 whose salary exceeds 100000, ordered by that number. Evaluating it in logical order on the sample data gives the following trace.

The statement and its result are printed after the trace.

1. FROM and JOIN: 13 of the 14 employees match a department (employee 13 has a NULL `dept_id` and is discarded by the inner join); 13 joined rows exist.
2. WHERE: keeps employees hired on or after 2019-01-01 with salary above 100000. These are employees 4 (150000, 2019), 5 (120000, 2021), 7 (110000, 2020) and 14 (110000, 2024).
3. GROUP BY `dept_name`: Engineering receives employees 4 and 5, Sales receives 7 and 14. Two groups exist.
4. SELECT: `count(*)` is evaluated per group: 2 and 2.
5. ORDER BY: both groups have `n = 2`, so the tie is broken by `dept_name`.
6. The result is therefore (Engineering, 2) followed by (Sales, 2).

SQL

```sql
SELECT d.dept_name, count(*) AS n
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
WHERE e.hire_date >= '2019-01-01' AND e.salary > 100000
GROUP BY d.dept_name
ORDER BY n DESC, d.dept_name;
```

Result TERMINAL

```
  dept_name  | n
-------------+---
 Engineering | 2
 Sales       | 2
(2 rows)
```

1.6.6 Hands-on

The PostgreSQL command-line client `psql` is the primary tool for the examples. A local cluster is created as in the schema-design chapter; the following session shows the commands that matter most.

psql essentials TERMINAL

```
$ psql -h /tmp/osw_d2 -p 20200 shop
# inside psql:
\dt          list tables
\d employees describe a table (columns, indexes, foreign keys)
\x auto      expanded display when rows are wide
\timing on   print execution time of each statement
\i file.sql  run a script
```

SQL

```sql
\d employees
```

sreram

Result TERMINAL

```
                Table "public.employees"
   Column   |     Type      | Collation | Nullable | Default
------------+---------------+-----------+----------+---------
 emp_id     | integer       |           | not null |
 first_name | text          |           | not null |
 last_name  | text          |           | not null |
 dept_id    | integer       |           |          |
 manager_id | integer       |           |          |
 salary     | numeric(10,2) |           | not null |
 hire_date  | date          |           | not null |
Indexes:
    "employees_pkey" PRIMARY KEY, btree (emp_id)
Check constraints:
    "employees_salary_check" CHECK (salary > 0::numeric)
Foreign-key constraints:
    "emp_dept_fk" FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
    "emp_mgr_fk" FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
Referenced by:
    TABLE "employees" CONSTRAINT "emp_mgr_fk" FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
```

The `\d` output lists the foreign keys in both directions (`Foreign-key constraints` and `Referenced by`), which makes it a quick way to learn an unfamiliar schema.

1.6.7 Real-world problems and solutions

Incident: pagination returns duplicate rows and skips others

Scenario. A support dashboard lists tickets sorted by priority, twenty per page, using `ORDER BY priority LIMIT 20 OFFSET :page * 20`. Users report that some tickets appear on two pages and others never appear at all. The table `tickets` has 20,000 rows and only five distinct priority values.

Symptoms. The same `ticket_id` is visible on consecutive pages; a ticket known to exist cannot be found by paging; the effect is not reproducible after a restart and differs between environments. Page size 5 is used below for readability.

Diagnosis. The first step is to compare consecutive pages directly.

SQL

```sql
SELECT ticket_id, priority FROM tickets ORDER BY priority LIMIT 5 OFFSET 0;
SELECT ticket_id, priority FROM tickets ORDER BY priority LIMIT 5 OFFSET 5;
```

Result TERMINAL

```
 ticket_id | priority
-----------+----------
        20 |        1
        10 |        1
        15 |        1
         5 |        1
        25 |        1
(5 rows)

 ticket_id | priority
-----------+----------
        20 |        1
        15 |        1
        25 |        1
        10 |        1
        50 |        1
(5 rows)
```

Ticket 20 appears at the top of both pages, and tickets 15, 25 and 10 are split inconsistently. All ten rows have priority 1, so ORDER BY gives no ordering among them. The second step is to confirm that the plan, not the data, differs between executions.

SQL

```sql
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT ticket_id, priority FROM tickets ORDER BY priority LIMIT 5 OFFSET 0;
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT ticket_id, priority FROM tickets ORDER BY priority LIMIT 5 OFFSET 10;
```

Result TERMINAL

```
                        QUERY PLAN
-----------------------------------------------------------
 Limit (actual rows=5 loops=1)
   ->  Sort (actual rows=5 loops=1)
         Sort Key: priority
         Sort Method: top-N heapsort  Memory: 25kB
         ->  Seq Scan on tickets (actual rows=20000 loops=1)
(5 rows)

                        QUERY PLAN
-----------------------------------------------------------
 Limit (actual rows=5 loops=1)
   ->  Sort (actual rows=15 loops=1)
         Sort Key: priority
         Sort Method: top-N heapsort  Memory: 25kB
         ->  Seq Scan on tickets (actual rows=20000 loops=1)
(5 rows)
```

Both plans use a top-N heapsort, but the heap holds `OFFSET + LIMIT` rows (5 versus 15, visible as the row count of the Sort node), and a heap does not keep tied rows in a stable order. A different `OFFSET` therefore evaluates a differently sized heap and the tie order changes.

Root cause. `ORDER BY priority` is not a total order. SQL guarantees nothing about the relative order of rows that compare equal, so each execution may pick a different arrangement.

Fix. Add a unique tie-breaker, typically the primary key, so that the order is total. Then verify that consecutive pages are disjoint.

SQL

```sql
SELECT ticket_id, priority FROM tickets ORDER BY priority, ticket_id LIMIT 5 OFFSET 0;
SELECT ticket_id, priority FROM tickets ORDER BY priority, ticket_id LIMIT 5 OFFSET 5;
```

Result TERMINAL

```
 ticket_id | priority
-----------+----------
         5 |        1
        10 |        1
        15 |        1
        20 |        1
        25 |        1
(5 rows)

 ticket_id | priority
-----------+----------
        30 |        1
        35 |        1
        40 |        1
        45 |        1
        50 |        1
(5 rows)
```

The permanent fix has two parts. In the repository layer, every pageable query gets a unique final sort key. In Spring Data this means `PageRequest.of(page, size, Sort.by("priority").and(Sort.by("id")))`; a `Pageable` that sorts by a non-unique property alone reproduces the defect. For deep pages OFFSET itself becomes the problem, because the server still produces and discards all skipped rows; keyset pagination (`WHERE (priority, ticket_id) > (:p, :id) ORDER BY priority, ticket_id LIMIT 20`) avoids this and is developed in the query-optimisation chapters.

Prevention. A code review rule: any `LIMIT` must be accompanied by an `ORDER BY` whose last element is unique. A test that pages through a table containing duplicate sort values and asserts that the union of the pages equals the table detects the defect.

sreram

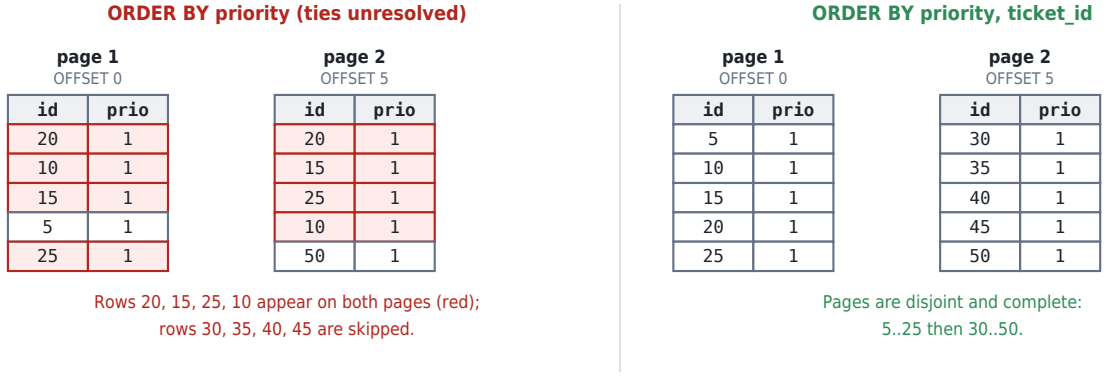


Figure 1.31. Pages produced by ORDER BY priority (ties) compared with ORDER BY priority, ticket_id. With the non-unique key the same rows appear on both pages and others are never returned; with the tie-breaker the pages are disjoint and complete.

Incident: UPDATE without WHERE (summary)

Scenario. A one-line salary correction is run without its WHERE clause in an interactive session with autocommit on. **Immediate action.** Stop writes if possible; autocommit means the change is already durable. **Fix.** Restore from a backup or by point-in-time recovery to a moment before the statement (see the backup chapter), or, when the previous values are known from an audit table, rewrite them from it. **Prevention.** Perform manual changes inside `BEGIN`, inspect the row count reported by the command (`UPDATE 14` when 3 were expected is the alarm), and only then `COMMIT`; give humans a role without direct write access to production tables.

1.6.8 Questions to expect

INTERVIEW ANGLE
What is the difference between WHERE and HAVING? WHERE filters individual rows before grouping and cannot reference aggregates; HAVING filters groups after aggregation and can. A condition that does not involve an aggregate belongs in WHERE, because it reduces the rows before the aggregation work is done.

INTERVIEW ANGLE
In what order is a SELECT statement evaluated? FROM and joins, WHERE, GROUP BY, HAVING, SELECT list (including window functions), DISTINCT, ORDER BY, LIMIT/OFFSET. This is the logical order that defines visibility of names; the physical execution order is chosen by the planner. It explains why an alias defined in SELECT can be used in ORDER BY but not in WHERE.

INTERVIEW ANGLE
Why can two executions of the same query return rows in different orders? Without ORDER BY the order is unspecified, and with an ORDER BY on non-unique columns the order among ties is unspecified. Plans, parallelism, vacuum activity and caching change it. Pagination therefore requires a total order, normally the sort key followed by the primary key.

INTERVIEW ANGLE
What is the difference between DELETE, TRUNCATE and DROP? DELETE removes selected rows, fires row triggers, writes one WAL record per row and can be filtered. TRUNCATE removes all rows by discarding storage, takes an exclusive lock and is much faster. DROP removes the table definition as well. In PostgreSQL all three are transactional; in MySQL TRUNCATE and DROP are DDL and commit implicitly.

INTERVIEW ANGLE
How is an upsert written in PostgreSQL and in MySQL, and why is it preferable to SELECT-then-INSERT? PostgreSQL: INSERT ... ON CONFLICT (key) DO UPDATE SET ...; MySQL: INSERT ... ON DUPLICATE KEY UPDATE .... The single statement is atomic and relies on the unique index, whereas SELECT followed by INSERT lets two sessions both observe the row as missing and one of them then fails or creates a duplicate.

INTERVIEW ANGLE
What does RETURNING do and what is the JDBC equivalent where it is unavailable? RETURNING (PostgreSQL) returns columns of rows written by INSERT, UPDATE or DELETE in the same round trip. Where it is not supported, as in MySQL 8, the application uses `Statement.RETURN_GENERATED_KEYS` for generated keys, or a follow-up SELECT inside the same transaction.

INTERVIEW ANGLE
Why is OFFSET pagination slow for deep pages, and what is the alternative? The server must produce and discard all skipped rows, so the cost grows with the offset. Keyset (seek) pagination filters with `WHERE (sort_key, id) > (last_key, last_id)` and uses an index, so the cost is independent of the page number.

KEY POINTS
SQL fundamentals

- SQL is declarative; a statement describes a result, and the planner chooses the execution steps.
- The logical order is FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT; it determines which names are visible in each clause.
- Result order is guaranteed only by an ORDER BY that is a total order; pagination without one duplicates and skips rows.
- UPDATE and DELETE without WHERE are legal; run them in a transaction and check the reported row count.
- Upserts (`ON CONFLICT`, `ON DUPLICATE KEY UPDATE`) and `RETURNING` replace racy read-then-write code and extra round trips.
- PostgreSQL DDL is transactional; MySQL DDL commits implicitly.

1.7 Joins

Relational design stores each fact once: a department name lives in `departments`, not in every employee row. A query that needs both facts must recombine the tables, and the JOIN operation does this. Most SQL defects found in production code are join defects: a missing row because an outer join was silently turned into an inner join, an inflated total because a join multiplied rows, or a slow report because the join condition could not use an index. This chapter defines each join type by the rows it produces, demonstrates the behaviour on the sample data and then treats the pitfalls.

1.7.1 Concept and mechanism

Definition: a filtered Cartesian product

Conceptually every join starts from the Cartesian product of its two inputs (every row of the left table paired with every row of the right table) and keeps the pairs for which the join condition is true. The `ON` clause holds that condition. The join types differ only in what happens to the rows that have no partner.

Join	Result	Unmatched rows
`INNER JOIN`	Pairs satisfying the condition	Dropped from both sides
`LEFT [OUTER] JOIN`	Inner result plus unmatched left rows	Left rows kept, right columns NULL
`RIGHT [OUTER] JOIN`	Inner result plus unmatched right rows	Right rows kept, left columns NULL
`FULL [OUTER] JOIN`	Inner result plus unmatched rows of both sides	Both kept, other side NULL (not in MySQL)
`CROSS JOIN`	Every pairing, no condition	Not applicable

Join types.

The following diagram uses real rows from the sample schema (five employees and the five departments) to show which pairs each join produces.

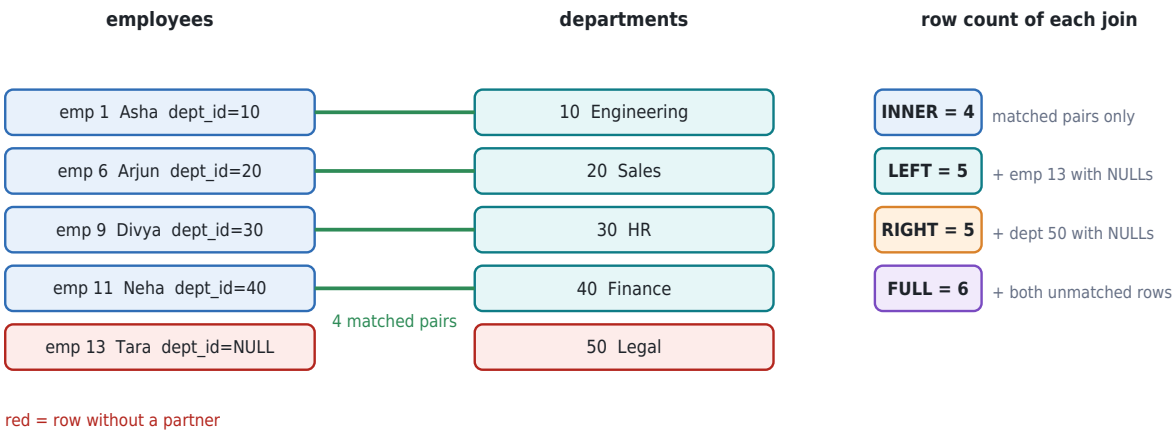


Figure 1.32. Row matching between employees (left) and departments (right) on `dept_id`. Lines are matched pairs. Employee 13 has a NULL `dept_id` and department 50 has no employees; they are the only rows that differ between INNER, LEFT and FULL results.

The four joins on real data

SQL

```sql
SELECT e.emp_id, e.first_name, d.dept_id, d.dept_name
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IN (1, 6, 9, 11, 13)
ORDER BY e.emp_id;

SELECT e.emp_id, e.first_name, d.dept_id, d.dept_name
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IN (1, 6, 9, 11, 13)
ORDER BY e.emp_id;
```

Result — TERMINAL

```
 emp_id | first_name | dept_id |  dept_name
--------+------------+---------+-------------
      1 | Asha       |      10 | Engineering
      6 | Arjun      |      20 | Sales
      9 | Divya      |      30 | HR
     11 | Neha       |      40 | Finance
(4 rows)

 emp_id | first_name | dept_id |  dept_name
--------+------------+---------+-------------
      1 | Asha       |      10 | Engineering
      6 | Arjun      |      20 | Sales
      9 | Divya      |      30 | HR
     11 | Neha       |      40 | Finance
     13 | Tara       |         |
(5 rows)
```

The inner join returns four rows; employee 13 disappears. The left join returns all five, with NULL in the department columns for employee 13. A RIGHT JOIN is a LEFT JOIN with the tables exchanged and is rarely written; reading queries is easier when one convention (left joins only) is used throughout a code base.

SQL

```sql
SELECT e.emp_id, e.first_name, d.dept_id, d.dept_name
FROM employees e
FULL JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL OR e.emp_id = 1
ORDER BY e.emp_id NULLS LAST;
```

sreeram

Result TERMINAL

```
 emp_id | first_name | dept_id |  dept_name
--------+------------+---------+-------------
      1 | Asha       |      10 | Engineering
     13 | Tara       |         |
        |            |      50 | Legal
(3 rows)
```

The full join exposes both kinds of orphan: employee 13 without a department and department 50 without an employee. MySQL has no FULL JOIN; the equivalent is `LEFT JOIN ... UNION ... RIGHT JOIN ...`, or a `UNION ALL` of a left join and an anti-join.

USING, NATURAL JOIN and CROSS JOIN

`USING (col)` is shorthand for an equality on identically named columns and outputs the column once. `NATURAL JOIN` joins on every column name the two tables share; it silently changes meaning when a column is added or renamed, and should not appear in production code. `CROSS JOIN` has legitimate uses: generating combinations (a calendar of dates times a list of stores) and expanding data. An unintended cross join (a missing ON clause, or a missing predicate between three tables) multiplies row counts and is a typical cause of a query that never returns.

SQL

```sql
SELECT count(*) AS cross_rows FROM employees CROSS JOIN departments;
SELECT count(*) AS accidental FROM employees e, departments d;   -- comma join with no WHERE
```

Result TERMINAL

```
 cross_rows
------------
         70
(1 row)

 accidental
------------
         70
(1 row)
```

Self-join

A self-join joins a table to itself under two aliases and is the standard way to follow a one-level relationship such as employee to manager. The manager is optional, so a left join keeps the root employee.

SQL

```sql
SELECT e.emp_id, e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
WHERE e.emp_id <= 6
ORDER BY e.emp_id;
```

Result TERMINAL

```
 emp_id | employee | manager
--------+----------+---------
      1 | Asha     |
      2 | Vikram   | Asha
      3 | Meera    | Vikram
      4 | Ravi     | Vikram
      5 | Sneha    | Meera
      6 | Arjun    | Asha
(6 rows)
```

Hierarchies of unknown depth need a recursive common table expression, treated in the subqueries chapter.

Semi-join and anti-join

A semi-join returns the rows of one table that have at least one match in another, without duplicating them and without exposing the other table's columns. An anti-join returns the rows that have no match. SQL has no dedicated keyword; both are written with `EXISTS`, `NOT EXISTS`, `IN` or a left join. The planner recognises these patterns and executes them with dedicated join strategies (Hash Semi Join, Hash Anti Join, Nested Loop Anti Join).

sreeram

SQL

```sql
-- semi-join: customers that have placed at least one order
SELECT c.customer_id, c.name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;

-- anti-join: customers that have never ordered
SELECT c.customer_id, c.name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;

-- equivalent anti-join with LEFT JOIN ... IS NULL
SELECT c.customer_id, c.name
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY 1;
```

Result TERMINAL

```
 customer_id |      name
-------------+----------------
           1 | Anil Traders
           2 | Bharat Stores
           3 | Chitra Boutique
           5 | Eshwar Mart
(4 rows)

 customer_id |      name
-------------+---------------
           4 | Deepak & Sons
           6 | Farah Fashions
(2 rows)

 customer_id |      name
-------------+---------------
           4 | Deepak & Sons
           6 | Farah Fashions
(2 rows)
```

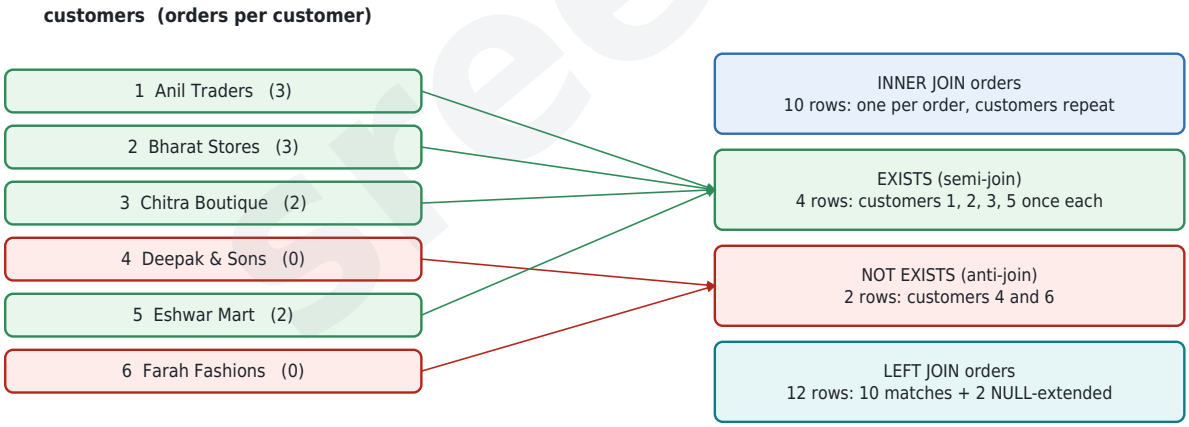


Figure 1.33. Semi-join and anti-join on customers and orders. A plain join returns one row per matching order (10 rows) and repeats customers; the semi-join returns each matching customer once (4 rows) and the anti-join returns the two customers with no order.

The IS NULL test in the last statement must reference a column that is never NULL in the right table (the primary key); testing a nullable column would also match real rows whose value is NULL. `NOT IN` is a third spelling that is dangerous when the subquery can return NULL; this is the subject of an incident in the subqueries chapter.

1.7.2 Join algorithms in one page

The logical join type does not fix how the server computes it. PostgreSQL chooses between three physical algorithms for each join, based on estimated row counts and available indexes; the query-processing chapters later in this part show them in EXPLAIN output.

Algorithm	How it works	Best when
`Nested loop`	For each outer row, probe the inner side (usually through an index)	Few outer rows and an index on the inner join column
`Hash join`	Build an in-memory hash table on the smaller input, stream the larger input through it	Large unsorted inputs, equality conditions only
`Merge join`	Sort both inputs on the join key (or read them from indexes) and merge	Inputs already sorted; large inputs; range-free equality

Physical join algorithms.

MySQL 8.0.18 introduced the hash join for equi-joins without usable indexes; before that, MySQL had only nested-loop variants (including block nested loop). Since MySQL has no merge join, a join of two large unindexed tables was historically much slower there than in PostgreSQL.

1.7.3 Worked example: LEFT JOIN with aggregation, step by step

Requirement: for every customer, the number of orders and the total quantity ordered; customers without orders must appear with zeros. The statement joins three tables and aggregates.

The statement and its result are printed after the trace.

1. `customers LEFT JOIN orders`: customer 1 yields three rows (orders 101, 102, 106), customer 2 yields three (103, 104, 109), customer 3 two (105, 110), customer 5 two (107, 108), and customers 4 and 6 one row each with NULL order columns: 3 + 3 + 2 + 2 + 1 + 1 = 12 rows.
2. `LEFT JOIN order_items`: every order row is expanded by its line count (order 101 has 2 lines, 102 has 1, and so on). Rows with a NULL `order_id` find no match and remain, with NULL item columns.
3. GROUP BY collapses the expanded rows per customer. Because step 2 repeats each order once per line, `count(*)` would overstate the orders; `count(DISTINCT o.order_id)` removes the repetition, and for customers 4 and 6 it ignores the NULL.
4. `sum(oi.qty)` is NULL for customers without orders; `coalesce` substitutes 0.

SQL

```sql
SELECT c.customer_id, c.name,
       count(DISTINCT o.order_id) AS orders,
       coalesce(sum(oi.qty), 0)   AS units
FROM customers c
LEFT JOIN orders o       ON o.customer_id = c.customer_id
LEFT JOIN order_items oi ON oi.order_id   = o.order_id
GROUP BY c.customer_id, c.name
ORDER BY c.customer_id;
```

Result TERMINAL

```
 customer_id |      name       | orders | units
-------------+-----------------+--------+-------
           1 | Anil Traders    |      3 |     8
           2 | Bharat Stores   |      3 |    10
           3 | Chitra Boutique |      2 |     9
           4 | Deepak & Sons   |      0 |     0
           5 | Eshwar Mart     |      2 |    11
           6 | Farah Fashions  |      0 |     0
(6 rows)
```

1.7.4 Hands-on

Where the filter goes: ON versus WHERE

For an inner join a predicate in ON and the same predicate in WHERE are equivalent. For an outer join they are not. The ON clause decides which rows match; WHERE then filters the already joined result, including the NULL-extended rows. A WHERE predicate on the nullable side therefore discards the unmatched rows and converts the outer join back into an inner join.

SQL

```sql
-- Intent: all customers, with their SHIPPED orders where they exist.
-- Filter in ON: outer join preserved.
SELECT c.customer_id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status = 'SHIPPED'
ORDER BY c.customer_id;

-- Filter in WHERE: unmatched customers are removed.
SELECT c.customer_id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'SHIPPED'
ORDER BY c.customer_id;
```

Result TERMINAL

```
 customer_id |      name       | order_id
-------------+-----------------+----------
           1 | Anil Traders    |      102
           2 | Bharat Stores   |
           3 | Chitra Boutique |
           4 | Deepak & Sons   |
           5 | Eshwar Mart     |      107
           6 | Farah Fashions  |
(6 rows)

 customer_id |     name     | order_id
-------------+--------------+----------
           1 | Anil Traders |      102
           5 | Eshwar Mart  |      107
(2 rows)
```

ON o.status = 'SHIPPED'

customer_id	order_id
1	102
2	NULL
3	NULL
4	NULL
5	107
6	NULL

6 rows: unmatched customers kept with NULL order columns (grey)

WHERE o.status = 'SHIPPED'

customer_id	order_id
1	102
5	107

4 rows lost: NULL = 'SHIPPED' is unknown, so WHERE drops them

Figure 1.34. The same predicate placed in ON and in WHERE of a LEFT JOIN. In ON it only restricts which orders match, so all six customers survive; in WHERE it is applied after the join to rows whose `status` is NULL for unmatched customers, so four customers are lost.

When the intent is "customers that have no shipped order", the correct form is an anti-join, not `WHERE o.status <> 'SHIPPED'`, which is also false for NULL.

Joins in Java: JDBC and JPA

A join result is flat, while a Java domain model is a graph. Two designs are common. With JDBC or `JdbcTemplate` a `RowMapper` maps one flat row to one DTO, and a one-to-many join needs a `ResultSetExtractor` that groups consecutive rows by parent key. With JPA, an association such as `Customer.orders` is loaded lazily by default and produces one additional query per customer (the N+1 problem, treated in the query-optimisation chapters); `JOIN FETCH` loads parents and children in a single joined query.

CustomerRepository.java (fragment; not run) JAVA

```java
public interface CustomerRepository extends JpaRepository<Customer, Integer> {

    // One SQL statement: customers LEFT JOIN orders. DISTINCT removes the
    // duplicate Customer instances that the join produces in the result set.
    @Query("select distinct c from Customer c left join fetch c.orders")
    List<Customer> findAllWithOrders();

    // Anti-join in JPQL: customers without orders.
    @Query("select c from Customer c where not exists "
         + "(select 1 from Order o where o.customer.id = c.id)")
    List<Customer> findWithoutOrders();
}
```

Fetch-joining a collection together with pagination (`Pageable`) is a known defect: Hibernate cannot apply LIMIT to the joined rows, so it fetches everything and paginates in memory, emitting the warning `HHH000104: firstResult/maxResults specified with collection fetch; applying in memory`. The remedy is to page the parent ids first and then fetch the children for those ids.

1.7.5 Real-world problems and solutions

Incident: revenue totals inflated by a join fan-out

Scenario. A finance report computes revenue per customer from order lines and also reports the amount received from the `payments` table, in a single query joining `orders`, `order_items` and `payments`. The revenue column exceeds the real revenue for some customers, and the discrepancy changes when a customer pays in instalments.

Symptoms. The report restricts itself to paid and shipped orders. For customer 1 it shows revenue 80,700 and received 120,500, while the ledger shows 77,800 for both; for customer 2 it shows 75,750 and 50,500 against a ledger value of 25,250. Customer 5, whose orders each have one line and one payment, is correct, which hides the defect in tests that use simple data.

Diagnosis. Step 1: reproduce the report query.

SQL

```sql
SELECT o.customer_id,
       sum(oi.qty * oi.unit_price) AS revenue,
       sum(p.amount)               AS received
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN payments p     ON p.order_id  = o.order_id
WHERE o.status IN ('PAID','SHIPPED')
GROUP BY o.customer_id
ORDER BY o.customer_id;
```

Result TERMINAL

```
 customer_id | revenue  | received
-------------+----------+-----------
           1 | 80700.00 | 120500.00
           2 | 75750.00 |  50500.00
           3 |  6000.00 |  12000.00
           5 | 57500.00 |  57500.00
(4 rows)
```

Step 2: compute each measure separately, with no join between the two child tables.

SQL

```sql
SELECT o.customer_id,
       (SELECT sum(oi.qty * oi.unit_price) FROM order_items oi
        JOIN orders o2 ON o2.order_id = oi.order_id
        WHERE o2.customer_id = o.customer_id
          AND o2.status IN ('PAID','SHIPPED')) AS revenue,
       (SELECT sum(p.amount) FROM payments p
        JOIN orders o3 ON o3.order_id = p.order_id
        WHERE o3.customer_id = o.customer_id) AS received
FROM (SELECT DISTINCT customer_id FROM orders) o
ORDER BY o.customer_id;
```

Result TERMINAL

```
 customer_id | revenue  | received
-------------+----------+----------
           1 | 77800.00 | 77800.00
           2 | 25250.00 | 25250.00
           3 |  6000.00 |  6000.00
           5 | 57500.00 | 57500.00
(4 rows)
```

Revenue and receipts agree with each other and with the ledger (77,800 and 25,250); the report query is therefore wrong, not the data. Step 3: inspect the rows of one order that has several lines and several payments.

SQL

```sql
SELECT o.order_id, oi.line_no, oi.qty * oi.unit_price AS line_amount, p.payment_id, p.amount
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN payments p     ON p.order_id  = o.order_id
WHERE o.order_id = 101
ORDER BY oi.line_no, p.payment_id;
```

Result TERMINAL

```
 order_id | line_no | line_amount | payment_id | amount
----------+---------+-------------+------------+--------
      101 |       1 |     1500.00 |          1 | 1000.00
      101 |       1 |     1500.00 |          2 | 1900.00
      101 |       2 |     1400.00 |          1 | 1000.00
      101 |       2 |     1400.00 |          2 | 1900.00
(4 rows)
```

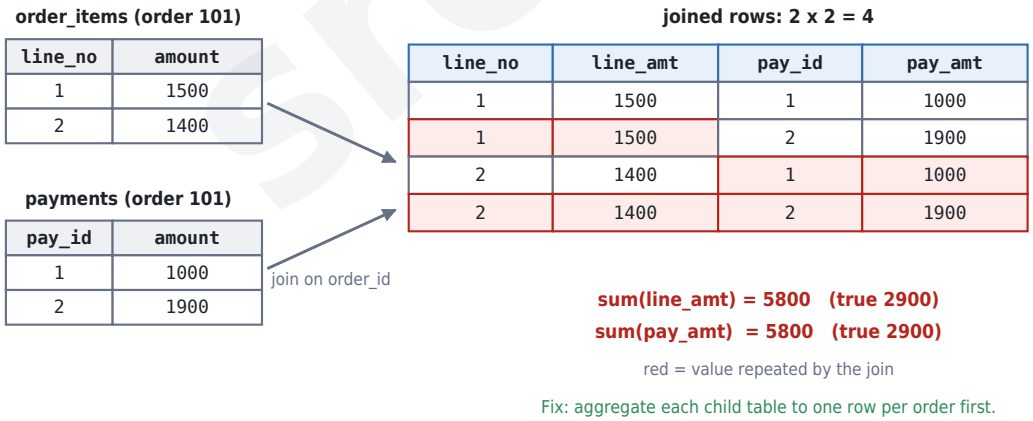


Figure 1.35. Order 101 has 2 lines and 2 payments. Joining both child tables to the order yields 2 x 2 = 4 rows, so each line amount is counted twice and each payment twice; summing after the join doubles both totals (2900 becomes 5800 and the payments 2900 becomes 5800).

Root cause. The two child tables are both one-to-many from `orders` and are unrelated to each other. Joining them to the same parent produces the product of their row counts per parent (here 2 x 2 for order 101 and 2 x 3 for order 103), and the aggregate then sums every repeated value. Row multiplication after a join is called fan-out. The error does not raise an exception and grows with the data.

Fix. Immediate: aggregate each child table to one row per order before joining, so that every input to the join is unique on the join key.

SQL

```sql
SELECT o.customer_id,
       sum(i.revenue)  AS revenue,
       sum(p.received) AS received
FROM orders o
JOIN (SELECT order_id, sum(qty * unit_price) AS revenue FROM order_items GROUP BY order_id) i
  ON i.order_id = o.order_id
LEFT JOIN (SELECT order_id, sum(amount) AS received FROM payments GROUP BY order_id) p
  ON p.order_id = o.order_id
WHERE o.status IN ('PAID','SHIPPED')
GROUP BY o.customer_id
ORDER BY o.customer_id;
```

Result TERMINAL

```
 customer_id | revenue  | received
-------------+----------+----------
           1 | 77800.00 | 77800.00
           2 | 25250.00 | 25250.00
           3 |  6000.00 |  6000.00
           5 | 57500.00 | 57500.00
(4 rows)
```

Every customer now shows equal revenue and receipts, matching step 2. The permanent fix is to make the grain of each aggregate explicit in the query: each child table is reduced in its own subquery or common table expression, so that every join input is unique on the join key. The outer join to payments keeps orders whose payment has not yet arrived.

Prevention. Before joining a table, state its grain (the columns that identify one row) and the grain of the result. If a join target is not unique on the join key, aggregate it first. A reconciliation test that compares the report total with `SELECT sum(...)` on the base table, using data with several lines and several payments per order, detects fan-out. Treat `SELECT DISTINCT` or `count(DISTINCT ...)` added "to fix duplicates" as a defect marker: it hides the multiplication without correcting sums.

Incident: customers disappear from a report after a filter is added

Scenario. A report lists every customer with the count of shipped orders. A developer adds `AND status = 'SHIPPED'` as a WHERE predicate to an existing LEFT JOIN query. **Symptoms.** Customers with no shipped orders vanish from the report, and the report now shows two customers instead of six. **Diagnosis.** The predicate references the nullable side of an outer join, so rows extended with NULL are discarded (see the hands-on section above: the WHERE form returns two rows, the ON form six). **Fix.** Move the predicate into the ON clause, or compute the count with `count(*) FILTER (WHERE o.status = 'SHIPPED')` over an unfiltered left join. **Prevention.** Review rule: in a query with an outer join, every predicate on the optional table belongs in ON; a WHERE predicate on that table is almost always a defect unless it is `IS NULL` (anti-join).

1.7.6 Questions to expect

INTERVIEW ANGLE
What is the difference between INNER JOIN and LEFT JOIN? INNER JOIN returns only pairs that satisfy the condition. LEFT JOIN returns all rows of the left table, with NULL in the right-hand columns where no match exists. Choosing the wrong one silently drops or keeps rows; an outer join is required whenever the report must show entities with no related rows.

INTERVIEW ANGLE
Why does a WHERE condition on the right table change the meaning of a LEFT JOIN? WHERE is evaluated after the join. Unmatched left rows carry NULL in right-table columns, so a predicate such as `o.status = 'SHIPPED'` is not true for them and they are removed. The join then behaves as an inner join. The predicate belongs in ON when it should only restrict the matching.

INTERVIEW ANGLE
How is a semi-join different from an inner join? A semi-join (EXISTS or IN) returns each left row at most once if any match exists and returns no right-hand columns. An inner join returns one row per matching pair, so a left row with several matches is repeated. Semi-joins are the correct tool for "has at least one".

INTERVIEW ANGLE
Name three ways to find rows with no match and compare them. NOT EXISTS, LEFT JOIN with IS NULL on the right primary key, and NOT IN. NOT EXISTS and the left join are null-safe and usually produce the same anti-join plan in PostgreSQL. NOT IN returns no rows at all when the subquery contains a NULL, and is also harder for the planner to optimise.

INTERVIEW ANGLE
Describe join fan-out and how it is prevented. When two unrelated one-to-many tables are joined to the same parent, rows multiply per parent and sums are inflated. Prevention is to aggregate each many-side table to the grain of the join key before joining, or to compute each measure in a separate subquery.

INTERVIEW ANGLE
Which join algorithms exist and when does the planner choose each? Nested loop (few outer rows, indexed inner side), hash join (large inputs, equality), merge join (sorted inputs or large inputs with equality). The choice is cost-based from the statistics; `EXPLAIN` shows it.

INTERVIEW ANGLE
How is a FULL OUTER JOIN written in MySQL? MySQL has no FULL JOIN. A left join combined with a right join through `UNION` gives the same result; `UNION ALL` of a left join and an anti-join avoids the duplicate-removal step.

KEY POINTS
Joins

- A join is a filtered Cartesian product; the join type decides the fate of unmatched rows.
- Predicates on the optional side of an outer join belong in ON; in WHERE they turn the join into an inner join.
- Semi-joins (EXISTS) and anti-joins (NOT EXISTS) answer "has any" and "has none" without duplicating rows.
- Joining two unrelated one-to-many tables to one parent multiplies rows; aggregate each child table first.
- Know the grain of every input: a non-unique join key is the root of duplicates.
- JPA lazy associations cause N+1 queries; JOIN FETCH fixes it but must not be combined with pagination of the parent.

1.8 Aggregation and Grouping

An aggregate function reduces a set of rows to one value: a count, a sum, an average, a minimum. Combined with GROUP BY, the reduction is performed once per group, which turns a table of events into a table of metrics. Nearly every dashboard, report and reconciliation query is an aggregation. The defects found in them have a small number of causes: NULL values that aggregates skip silently, groups formed on the wrong columns, filters applied at the wrong stage and rows that were multiplied before aggregation (see the join chapter).

1.8.1 Concept and mechanism

Aggregate functions and their treatment of NULL

An aggregate ignores NULL inputs, with one exception: `count(*)` counts rows and does not look at any column. If every input is NULL, or the group has no rows, the result is NULL for every aggregate except `count`, which returns 0.

Function	Result	NULL handling
`count(*)`	Number of rows in the group	Counts rows with NULLs
`count(expr)`	Number of rows where expr is not NULL	Skips NULL
`count(DISTINCT expr)`	Number of distinct non-NULL values	Skips NULL
`sum(expr), avg(expr)`	Total and mean of the non-NULL values	Skips NULL; NULL if no input
`min(expr), max(expr)`	Smallest and largest value, any ordered type	Skips NULL
`bool_and, bool_or`	Logical AND / OR over booleans (PostgreSQL; MySQL uses MIN/MAX of 0/1)	Skips NULL
`string_agg(expr, sep)`	Concatenation of the values (MySQL: GROUP_CONCAT)	Skips NULL

Function	Result	NULL handling
`array_agg(expr)`	Array of the values (PostgreSQL only; MySQL: JSON_ARRAYAGG)	Keeps NULL

Common aggregate functions.

The result types matter in Java. In PostgreSQL `count` returns `bigint` (a Java `long`), `sum(integer)` returns `bigint`, `sum(bigint)` and `sum(numeric)` return `numeric` (a `BigDecimal`), and `avg` of an integer type returns `numeric`. Reading `count(*)` into an `int` field with an unchecked cast, or comparing a `Long` with `==`, are common faults.

SQL

```sql
SELECT count(*)                AS all_rows,
       count(dept_id)          AS with_dept,
       count(DISTINCT dept_id) AS distinct_depts,
       sum(salary)             AS payroll,
       avg(salary)::numeric(10,2) AS mean_salary,
       min(hire_date)          AS first_hire
FROM employees;
```

Result TERMINAL

```
 all_rows | with_dept | distinct_depts |  payroll   | mean_salary | first_hire
----------+-----------+----------------+------------+-------------+------------
       14 |        13 |              4 | 1905000.00 |   136071.43 | 2015-03-01
(1 row)
```

Fourteen rows exist, but `count(dept_id)` is 13 because employee 13 has no department, and `count(DISTINCT dept_id)` is 4 because the employees reference four departments (department 50, Legal, has no employee).

How GROUP BY works

GROUP BY partitions the rows that survive WHERE into groups of rows with equal values in the grouping expressions, then evaluates each aggregate once per group. For grouping purposes NULLs are treated as equal to each other, so all NULLs of a grouping column form one group (unlike in a comparison, where NULL is never equal to NULL). Every column in the SELECT list must be a grouping expression, an aggregate, or functionally dependent on the grouping columns (a column of a table whose primary key is grouped).

The server forms the groups in one of two ways. A hash aggregate builds an in-memory hash table keyed by the grouping columns and updates the aggregate state as rows stream past; it needs no sorted input but needs memory proportional to the number of groups. A group aggregate requires the input sorted on the grouping columns (by an explicit sort or an index) and emits a group whenever the key changes; it needs almost no memory. EXPLAIN shows `HashAggregate` or `GroupAggregate`.

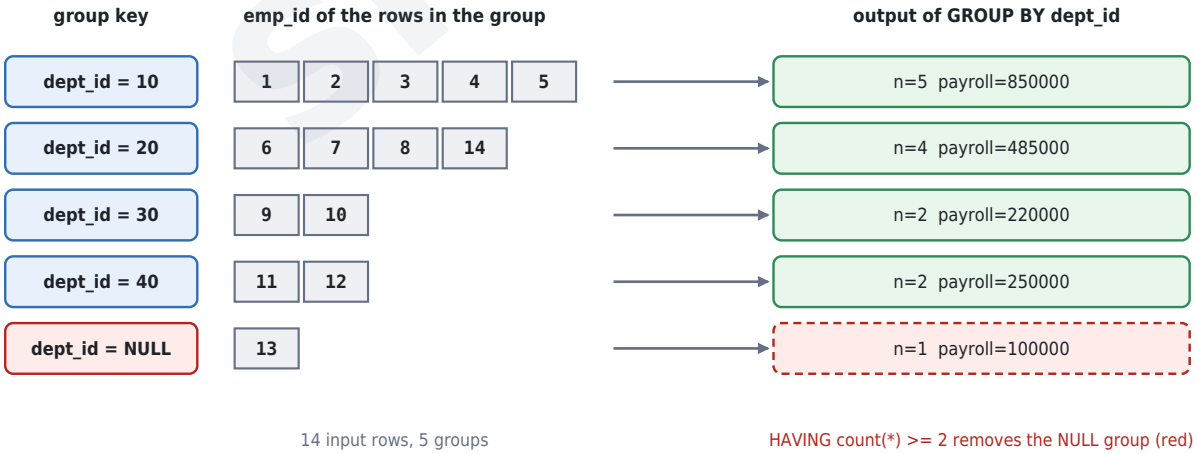


Figure 1.36. GROUP BY on `employees`. The 14 input rows are routed to groups by `dept_id` (NULL forms its own group); each group is reduced to one output row. HAVING then removes groups after aggregation, so it can test the aggregate values.

SQL

```sql
SELECT dept_id, count(*) AS n, sum(salary) AS payroll, round(avg(salary)) AS mean
FROM employees
GROUP BY dept_id
ORDER BY dept_id NULLS LAST;
```

Result TERMINAL

```
 dept_id | n |  payroll   |  mean
---------+---+------------+--------
      10 | 5 | 850000.00 | 170000
      20 | 4 | 485000.00 | 121250
      30 | 2 | 220000.00 | 110000
      40 | 2 | 250000.00 | 125000
         | 1 | 100000.00 | 100000
(5 rows)
```

The NULL group is shown last: employee 13 forms a group of its own. A frequent request, "employees per department including empty departments", cannot be answered from `employees` alone; it needs a left join from `departments` and `count(e.emp_id)`, not `count(*)`, because an empty department still yields one joined row.

SQL

```sql
SELECT d.dept_name, count(*) AS wrong, count(e.emp_id) AS right
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

Result TERMINAL

```
  dept_name  | wrong | right
-------------+-------+-------
 Engineering |     5 |     5
 Finance     |     2 |     2
 HR          |     2 |     2
 Legal       |     1 |     0
 Sales       |     4 |     4
(5 rows)
```

Legal shows 1 with `count(*)` and 0 with `count(e.emp_id)`. The NULL-extended row produced by the outer join is a row, and `count(*)` counts it.

The standard rule and MySQL's ONLY_FULL_GROUP_BY

A SELECT list item that is neither grouped nor aggregated is an error in PostgreSQL. MySQL has the same rule under the SQL mode `ONLY_FULL_GROUP_BY` (enabled by default since 5.7.5) and detects functional dependence on the primary key; with the mode off it returns an arbitrary row's value, which produces plausible but wrong results.

SQL

```sql
SELECT dept_id, first_name, max(salary) FROM employees GROUP BY dept_id;
```

Result TERMINAL

```
ERROR:  column "employees.first_name" must appear in the GROUP BY clause or be used in an aggregate
        function
LINE 1: SELECT dept_id, first_name, max(salary) FROM employees GROUP...
                        ^
```

PostgreSQL does accept `SELECT d.dept_name, count(*) FROM departments d JOIN employees e USING (dept_id) GROUP BY d.dept_id` because `dept_name` is functionally dependent on the grouped primary key `d.dept_id`.

1.8.2 Filtering aggregates: HAVING and FILTER

HAVING filters groups, WHERE filters rows. A condition that does not involve an aggregate must be placed in WHERE, because it then reduces the work done before grouping. The conditional aggregate clause `FILTER (WHERE ...)` (PostgreSQL 9.4 and later, SQLite 3.30 and later, not in MySQL) restricts the rows that feed one aggregate and allows several differently filtered aggregates in a single pass; the portable form is `sum(CASE WHEN cond THEN x END)`, which works because CASE without ELSE yields NULL and aggregates skip NULL.

```sql
SELECT c.name,
       count(*)                                          AS orders,
       count(*) FILTER (WHERE o.status = 'PAID')         AS paid,
       count(CASE WHEN o.status = 'NEW' THEN 1 END)      AS new_portable
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name
HAVING count(*) >= 2
ORDER BY c.name;
```

Result

```
      name       | orders | paid | new_portable
-----------------+--------+------+--------------
 Anil Traders    |      3 |    2 |            0
 Bharat Stores   |      3 |    1 |            1
 Chitra Boutique |      2 |    1 |            1
 Eshwar Mart     |      2 |    1 |            0
(4 rows)
```

The two conditional forms are equivalent; the FILTER form reads more clearly and cannot be broken by a missing ELSE. This technique is also the standard way to pivot rows into columns, used in the interview problems at the end of this part.

1.8.3 Multiple levels of aggregation: GROUPING SETS, ROLLUP, CUBE

Reports often require totals at several levels in one result: per department, per location and overall. Three clauses generate them from a single scan. `GROUPING SETS ((a, b), (a), ())` lists the groupings explicitly. `ROLLUP (a, b)` is shorthand for `((a, b), (a), ())`, the hierarchy of subtotals. `CUBE (a, b)` is shorthand for every subset: `((a, b), (a), (b), ())`. In rows produced for a coarser grouping the omitted columns are NULL, which is indistinguishable from a real NULL value; the function `GROUPING(col)` returns 1 when the column was rolled up and 0 otherwise.

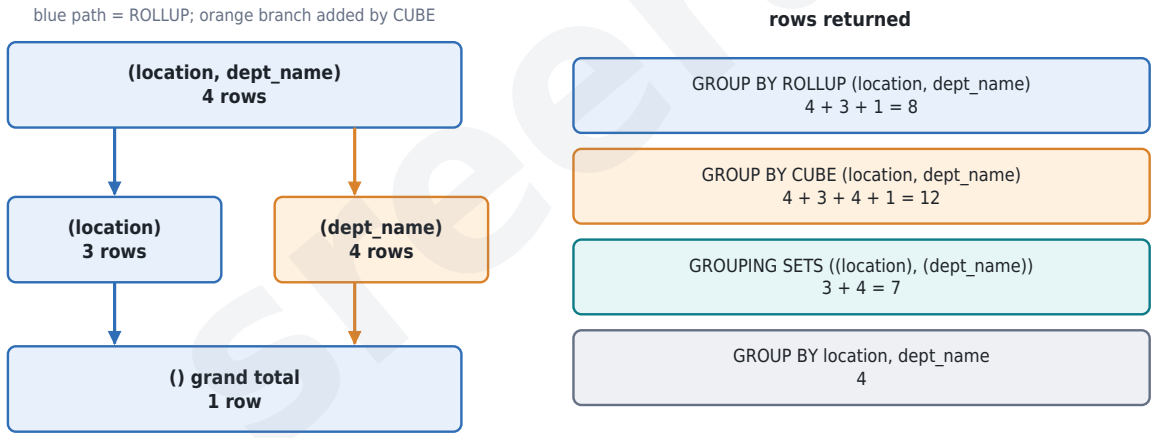


Figure 1.37. The grouping lattice for the columns (location, dept_name) of the sample data. ROLLUP emits the left path (8 rows); CUBE adds the right branch (12 rows). Each node is one GROUP BY evaluated in a single pass over the input.

```sql
SELECT d.location, d.dept_name, count(*) AS n, sum(e.salary) AS payroll,
       GROUPING(d.location) AS g_loc, GROUPING(d.dept_name) AS g_dept
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY ROLLUP (d.location, d.dept_name)
ORDER BY d.location NULLS LAST, d.dept_name NULLS LAST;
```

Result TERMINAL

```
 location  |  dept_name  | n  |  payroll   | g_loc | g_dept
-----------+-------------+----+------------+-------+--------
 Delhi     | Finance     |  2 |  250000.00 |     0 |      0
 Delhi     |             |  2 |  250000.00 |     0 |      1
 Hyderabad | Engineering |  5 |  850000.00 |     0 |      0
 Hyderabad | HR          |  2 |  220000.00 |     0 |      0
 Hyderabad |             |  7 | 1070000.00 |     0 |      1
 Mumbai    | Sales       |  4 |  485000.00 |     0 |      0
 Mumbai    |             |  4 |  485000.00 |     0 |      1
           |             | 13 | 1805000.00 |     1 |      1
(8 rows)
```

Rows with `g_dept = 1` are the location subtotals and the row with both flags set is the grand total. Without `GROUPING` the application cannot tell a subtotal row from a department with a NULL name. MySQL supports only `GROUP BY a, b WITH ROLLUP`, with `GROUPING()` available since 8.0.1; it has no `GROUPING SETS` or `CUBE`, so those require `UNION ALL` of several GROUP BY queries, scanning the table once per grouping.

SQL

```sql
SELECT count(*) AS cube_rows FROM (
  SELECT d.location, d.dept_name FROM employees e JOIN departments d ON d.dept_id = e.dept_id
  GROUP BY CUBE (d.location, d.dept_name)) t;
```

Result TERMINAL

```
 cube_rows
-----------
        12
(1 row)
```

1.8.4 String and array aggregation

`string_agg(expr, delimiter ORDER BY ...)` concatenates the values of a group. MySQL's equivalent is `GROUP_CONCAT(expr ORDER BY ... SEPARATOR ',')`. PostgreSQL places the ordering inside the aggregate call; without it the order of concatenation is undefined, as for any unordered set of rows.

SQL

```sql
SELECT d.dept_name,
       string_agg(e.first_name, ', ' ORDER BY e.first_name) AS members,
       array_agg(e.emp_id ORDER BY e.emp_id) AS ids
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name ORDER BY d.dept_name;
```

Result TERMINAL

```
  dept_name  |            members             |     ids
-------------+--------------------------------+-------------
 Engineering | Asha, Meera, Ravi, Sneha, Vikram | {1,2,3,4,5}
 Finance     | Neha, Sanjay                   | {11,12}
 HR          | Divya, Rahul                   | {9,10}
 Legal       |                                | {NULL}
 Sales       | Arjun, Kiran, Omar, Priya      | {6,7,8,14}
(5 rows)
```

The Legal row shows the difference in NULL handling: `string_agg` skips the NULL produced by the outer join and returns NULL, whereas `array_agg` keeps it and returns `{NULL}`, an array with one element.

PostgreSQL 16	MySQL 8.x (not run on the build machine)
`string_agg(first_name, ', ' ORDER BY first_name)`	`GROUP_CONCAT(first_name ORDER BY first_name SEPARATOR ', ')`
`No length limit (1 GB value limit)`	`Truncated silently to `group_concat_max_len` (default 1024 bytes)`
`array_agg(x), jsonb_agg(x)`	`JSON_ARRAYAGG(x)`

Concatenating aggregates.

The MySQL truncation produces a result that looks valid and is incomplete; it generates a warning (code 1260) that most clients and drivers do not surface.

1.8.5 Worked example: a per-department salary report

Requirement: for each department with at least two employees, the payroll, the highest salary, and the share of the company payroll, ordered by share.

The statement and its result are printed after the trace.

1. FROM and JOIN produce 13 rows (employee 13 has no department).
2. GROUP BY `dept_name` forms four groups: Engineering (5 rows, payroll 850000), Sales (4, 485000), HR (2, 220000) and Finance (2, 250000).
3. HAVING keeps groups with two or more rows; all four qualify.
4. The select-list expression `sum(sum(e.salary)) OVER ()` first computes `sum(e.salary)` per group, then applies a window function over the grouped result: the grand total of the four payrolls, 1,805,000. A window function operates after aggregation; the window chapter explains the mechanics.
5. The percentages are 850000 / 1805000 = 47.1 for Engineering, and so on.

SQL

```sql
SELECT d.dept_name,
       count(*)                                              AS headcount,
       sum(e.salary)                                         AS payroll,
       max(e.salary)                                         AS top_salary,
       round(100 * sum(e.salary) / sum(sum(e.salary)) OVER (), 1) AS pct
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING count(*) >= 2
ORDER BY pct DESC;
```

Result TERMINAL

```
  dept_name  | headcount |  payroll  | top_salary | pct
-------------+-----------+-----------+------------+------
 Engineering |         5 | 850000.00 |  250000.00 | 47.1
 Sales       |         4 | 485000.00 |  170000.00 | 26.9
 Finance     |         2 | 250000.00 |  160000.00 | 13.9
 HR          |         2 | 220000.00 |  140000.00 | 12.2
(4 rows)
```

1.8.6 Hands-on

Aggregation over Java data access typically returns either an array row or a projection. Spring Data JPA maps a grouped query to an interface projection:

DeptStats.java (fragment; not run) JAVA

```java
public interface DeptStats {            // interface-based projection
    String getDeptName();
    Long   getHeadcount();              // count() is bigint -> Long, never Integer
    BigDecimal getPayroll();            // sum(numeric) -> BigDecimal
}

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
    @Query("select d.name as deptName, count(e) as headcount, sum(e.salary) as payroll "
         + "from Employee e join e.department d "
         + "group by d.name having count(e) >= :min order by sum(e.salary) desc")
    List<DeptStats> payrollByDepartment(@Param("min") long min);
}
```

Performance of aggregation is governed by the amount of data scanned, not by the result size: `count(*)` on a large table reads the whole table (or a whole index) in PostgreSQL, because row visibility must be checked under MVCC. The query-optimisation chapters treat this and the alternatives (approximate counts, summary tables, partial indexes).

Preview

1.8.7 Real-world problems and solutions

Incident: customers-per-city dashboard does not add up

Scenario. A dashboard shows the number of customers per city using `SELECT city, count(city) FROM customers GROUP BY city`. The management total of customers is 6, but the dashboard figures add up to 5, and one row shows a city with an empty label and a count of 0.

Symptoms. The dashboard has a row without a name and a zero count, and the column sums disagree with `SELECT count(*) FROM customers`.

Diagnosis. Step 1: compare the grouped counts with the total.

SQL

```sql
SELECT city, count(city) AS counted, count(*) AS rows_in_group
FROM customers GROUP BY city ORDER BY city NULLS LAST;

SELECT count(*) AS total, count(city) AS with_city FROM customers;
```

Result TERMINAL

```
   city    | counted | rows_in_group
-----------+---------+--------------
 Chennai   |       1 |             1
 Delhi     |       1 |             1
 Hyderabad |       2 |             2
 Mumbai    |       1 |             1
           |       0 |             1
(5 rows)

 total | with_city
-------+-----------
     6 |         5
(1 row)
```

The first query shows a group with `city` NULL: the grouping step keeps customer 4 (city unknown) as a group, but `count(city)` counts only non-NULL values of the grouped column, which for that group is none. The second query confirms the discrepancy: 6 rows against 5 non-NULL cities.

Root cause. `count(column)` skips NULL by definition. Applied to the grouping column itself it returns 0 for the NULL group. The same rule underlies other silently wrong metrics: `avg(col)` divides by the number of non-NULL values, so averages over a sparse column describe only the rows that have a value.

Fix. Immediate: use `count(*)` for "number of rows" and label the NULL group explicitly with `coalesce(city, 'Unknown')`. Permanent: decide per metric what a missing value means (excluded, zero, or its own category) and encode that decision in the query: `avg(coalesce(x, 0))` for "missing means zero", `avg(x)` for "average of known values".

SQL

```sql
SELECT coalesce(city, 'Unknown') AS city, count(*) AS customers
FROM customers GROUP BY coalesce(city, 'Unknown') ORDER BY 1;
```

Result TERMINAL

```
   city    | customers
-----------+-----------
 Chennai   |         1
 Delhi     |         1
 Hyderabad |         2
 Mumbai    |         1
 Unknown   |         1
(5 rows)
```

Prevention. Reconcile every grouped report against an ungrouped total (`sum` of the groups equals the total). Add NOT NULL constraints to columns whose absence is not meaningful so that the question does not arise.

Sreeram

Incident: truncated group_concat in MySQL (typical)

Scenario. A MySQL-backed export builds a comma-separated list of tag identifiers per article with `GROUP_CONCAT(tag_id)`. Articles with many tags are exported with a cut-off list. **Diagnosis.** `SHOW VARIABLES LIKE 'group_concat_max_len'` returns 1024 (the default), and `SHOW WARNINGS` after the query contains `Warning 1260 Row 3 was cut by GROUP_CONCAT()`. **Fix.** `SET SESSION group_concat_max_len = 1048576;` before the query or set it in the server configuration; prefer `JSON_ARRAYAGG` and let the application decode the array. **Prevention.** Treat warnings from the driver as errors in tests, and avoid delivering data as one concatenated string.

1.8.8 Questions to expect

INTERVIEW ANGLE
What is the difference between `count(*)`, `count(column)` and `count(DISTINCT column)`? `count(*)` counts rows. `count(column)` counts rows where the column is not NULL. `count(DISTINCT column)` counts distinct non-NULL values. A table with 14 rows, 13 non-NULL department ids of which 4 distinct values are used returns 14, 13 and 4.

INTERVIEW ANGLE
Why can a SELECT list not contain a column that is absent from GROUP BY? Each group yields one output row, so a non-grouped column would have several candidate values. The standard requires the column to be grouped, aggregated, or functionally dependent on grouped columns (such as a column of a table whose primary key is grouped). MySQL with ONLY_FULL_GROUP_BY applies the same rule; without that mode it returns an arbitrary value.

INTERVIEW ANGLE
Where does a filter belong, in WHERE or in HAVING? A condition on individual rows belongs in WHERE: it reduces the rows before grouping. A condition on an aggregate belongs in HAVING because the aggregate exists only after grouping. Putting a row condition in HAVING gives the same result but does unnecessary work.

INTERVIEW ANGLE
How can several differently filtered counts be computed in one query? With conditional aggregation: `count(*) FILTER (WHERE status = 'PAID')` in PostgreSQL or `sum(CASE WHEN status = 'PAID' THEN 1 ELSE 0 END)` in any engine. One pass over the data computes all of them.

INTERVIEW ANGLE
What do ROLLUP, CUBE and GROUPING SETS do? They compute aggregates at several grouping levels in one statement. ROLLUP produces the hierarchy of subtotals ending in a grand total, CUBE every combination of the columns and GROUPING SETS exactly the listed combinations. `GROUPING()` distinguishes a subtotal NULL from a data NULL. MySQL supports only `WITH ROLLUP`.

INTERVIEW ANGLE
How do NULLs behave in GROUP BY and in aggregates? NULLs form a single group in GROUP BY, although NULL = NULL is not true in comparisons. Aggregates other than `count(*)` skip NULL inputs, and return NULL when no non-NULL input exists (`count` returns 0).

INTERVIEW ANGLE
What do HashAggregate and GroupAggregate mean in a plan? HashAggregate builds a hash table of groups and needs memory proportional to the number of groups. GroupAggregate consumes input sorted on the grouping keys and needs little memory. The planner chooses by estimated group count and by whether sorted input (an index) is available.

KEY POINTS
Aggregation and grouping

- Aggregates skip NULL; only `count(*)` counts rows. Choose count variants deliberately.
- GROUP BY forms one group for all NULLs; non-grouped columns in SELECT are an error unless functionally dependent.
- WHERE reduces rows before grouping, HAVING filters groups after aggregation; FILTER or CASE gives several conditional aggregates in one pass.
- ROLLUP, CUBE and GROUPING SETS replace several UNION ALL queries; use GROUPING() to identify subtotal rows.
- The grain of the input decides the correctness of every sum; reconcile each report against an ungrouped total.
- Java side: `count` maps to `long`, `sum(numeric)` to `BigDecimal`.

1.9 Subqueries and Common Table Expressions

A subquery is a SELECT nested inside another statement. A common table expression (CTE), introduced with `WITH`, gives a name to a subquery so that it can be referenced like a table, can be referenced more than once, and, in its recursive form, can walk hierarchies and graphs. Both constructs express logic that a single flat join cannot: "employees paid above their department average", "customers with no orders", "the latest order of each customer", "all subordinates of a manager at any depth". They are also the source of two of the most common production defects, the NOT IN trap and the correlated subquery executed once per outer row.

1.9.1 Concept and mechanism

Where a subquery may appear

Kind	Result shape and position	Example use
`Scalar subquery`	One row, one column; usable wherever an expression is	`SELECT (SELECT max(salary) FROM employees)`
`Row or column subquery`	Several rows, one column; used with IN, ANY, ALL, EXISTS	`WHERE dept_id IN (SELECT dept_id FROM departments)`
`Table (derived) subquery`	Any shape; used in FROM with an alias	`FROM (SELECT ... GROUP BY ...) t`
`Correlated subquery`	References a column of the outer query; logically executed once per outer row	`WHERE salary > (SELECT avg(salary) FROM employees x WHERE x.dept_id = e.dept_id)`

Kinds of subquery.

A scalar subquery that returns more than one row is a run-time error in PostgreSQL ("more than one row returned by a subquery used as an expression"). MySQL raises error 1242 ("Subquery returns more than 1 row"). A scalar subquery that returns no row yields NULL.

SQL

```sql
SELECT emp_id, first_name,
       (SELECT dept_name FROM departments d WHERE d.dept_id = e.dept_id) AS dept
FROM employees e WHERE emp_id IN (1, 13);

SELECT (SELECT dept_name FROM departments) AS bad;
```

Result TERMINAL

```
 emp_id | first_name |    dept
--------+------------+-------------
      1 | Asha       | Engineering
     13 | Tara       |
(2 rows)

ERROR:  more than one row returned by a subquery used as an expression
```

Correlated subqueries

An uncorrelated subquery is independent of the outer query and is evaluated once. A correlated subquery refers to the outer row, so its result changes per row. The standard example compares each employee with the average of the employee's own department.

SQL

```sql
SELECT e.emp_id, e.first_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT avg(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id, e.salary DESC;
```

sreeram

Result TERMINAL

```
 emp_id | first_name | dept_id |  salary
--------+------------+---------+-----------
      1 | Asha       |      10 | 250000.00
      2 | Vikram     |      10 | 180000.00
      6 | Arjun      |      20 | 170000.00
      9 | Divya      |      30 | 140000.00
     11 | Neha       |      40 | 160000.00
(5 rows)
```

Logically the inner query runs once for each of the 14 outer rows. Physically the planner may execute it exactly like that (a `SubPlan` node evaluated per row), or, for the forms it can transform (EXISTS, IN, NOT EXISTS, ANY), convert it into a join. Scalar correlated subqueries in the SELECT list or with an aggregate are normally not transformed by PostgreSQL, which is the source of the performance incident described below. The equivalent rewrite computes each department average once and joins to it:

SQL

```sql
SELECT e.emp_id, e.first_name, e.dept_id, e.salary
FROM employees e
JOIN (SELECT dept_id, avg(salary) AS avg_sal FROM employees GROUP BY dept_id) a
  ON a.dept_id = e.dept_id
WHERE e.salary > a.avg_sal
ORDER BY e.dept_id, e.salary DESC;
```

Result TERMINAL

```
 emp_id | first_name | dept_id |  salary
--------+------------+---------+-----------
      1 | Asha       |      10 | 250000.00
      2 | Vikram     |      10 | 180000.00
      6 | Arjun      |      20 | 170000.00
      9 | Divya      |      30 | 140000.00
     11 | Neha       |      40 | 160000.00
(5 rows)
```

Employee 13 (no department) never appears in either form: the comparison with the average of an empty or NULL-keyed group is not true.

EXISTS, IN and JOIN

Three constructs answer "rows of A that have a matching row in B". They differ in duplicate handling and in NULL behaviour.

Form	Duplicates from B	NULL in B's column	Notes
`` `WHERE EXISTS (SELECT 1 FROM b WHERE b.k = a.k)` ``	None: stops at first match	Not affected	Correct for any "has a match" test
`` `WHERE a.k IN (SELECT k FROM b)` ``	None	Not matching; harmless	Same plan as EXISTS in PostgreSQL
`` `JOIN b ON b.k = a.k` ``	One output row per match	Not matching	Must be de-duplicated if only A is wanted
`` `WHERE a.k NOT IN (SELECT k FROM b)` ``	None	One NULL makes the result empty	Avoid; use NOT EXISTS
`` `WHERE NOT EXISTS (...)` ``	None	Not affected	Correct anti-join

EXISTS, IN and JOIN compared.

In current PostgreSQL versions `EXISTS` and `IN` over an uncorrelated or simply correlated subquery are planned as the same semi-join, so the choice is one of readability. This was not true historically for MySQL, where `IN (subquery)` was executed as a dependent subquery before 5.6 and is optimised by semi-join strategies from 5.6 onwards. `NOT IN` is not equivalent to `NOT EXISTS`, as the first incident below shows.

sreeram

Common table expressions

`WITH name AS (subquery) SELECT ...` defines one or more named result sets visible in the main statement. A CTE improves readability, lets a result be used twice and structures multi-step logic. Since PostgreSQL 12 a non-recursive CTE that is referenced once and has no side effects is inlined like a subquery, so the planner can push predicates into it; one referenced several times is computed once and materialised. The keywords `MATERIALIZED` and `NOT MATERIALIZED` override the choice. Before version 12 every CTE was an optimisation fence, and many old queries were written (or rewritten) with that behaviour in mind. MySQL 8.0 merges CTEs into the outer query when possible and materialises them otherwise.

SQL

```sql
WITH dept_pay AS (
  SELECT dept_id, sum(salary) AS payroll, count(*) AS n FROM employees GROUP BY dept_id
), company AS (
  SELECT sum(payroll) AS total FROM dept_pay WHERE dept_id IS NOT NULL
)
SELECT d.dept_name, p.n, p.payroll, round(100 * p.payroll / c.total, 1) AS pct
FROM dept_pay p
JOIN departments d ON d.dept_id = p.dept_id
CROSS JOIN company c
ORDER BY pct DESC;
```

Result TERMINAL

```
  dept_name  | n |  payroll  | pct
-------------+---+-----------+------
 Engineering | 5 | 850000.00 | 47.1
 Sales       | 4 | 485000.00 | 26.9
 Finance     | 2 | 250000.00 | 13.9
 HR          | 2 | 220000.00 | 12.2
(4 rows)
```

CTEs may also contain data-modifying statements (PostgreSQL), which allows delete-and-archive in one atomic statement:

SQL

```sql
WITH moved AS (
  DELETE FROM payments WHERE order_id = 101 RETURNING *
)
INSERT INTO payments_archive SELECT * FROM moved;

SELECT (SELECT count(*) FROM payments) AS payments_left,
       (SELECT count(*) FROM payments_archive) AS archived;
```

Result TERMINAL

```
INSERT 0 2
 payments_left | archived
---------------+----------
             8 |        2
(1 row)
```

1.9.2 Recursive CTEs and hierarchies

A recursive CTE has two parts joined by `UNION ALL`: an anchor query that produces the starting rows and a recursive query that references the CTE itself. The engine evaluates it iteratively: the anchor result becomes the working table; the recursive query is run against the working table; its output is appended to the result and becomes the next working table; evaluation stops when an iteration produces no rows. The algorithm terminates only when the data is finite and acyclic, or when the query contains its own stopping condition.

SQL

```sql
WITH RECURSIVE org AS (
  SELECT emp_id, first_name, manager_id, 0 AS depth, first_name::text AS path
  FROM employees WHERE manager_id IS NULL                    -- anchor
  UNION ALL
  SELECT e.emp_id, e.first_name, e.manager_id, o.depth + 1,
         o.path || ' > ' || e.first_name
  FROM employees e JOIN org o ON e.manager_id = o.emp_id     -- recursive step
)
SELECT emp_id, depth, path FROM org ORDER BY path;
```

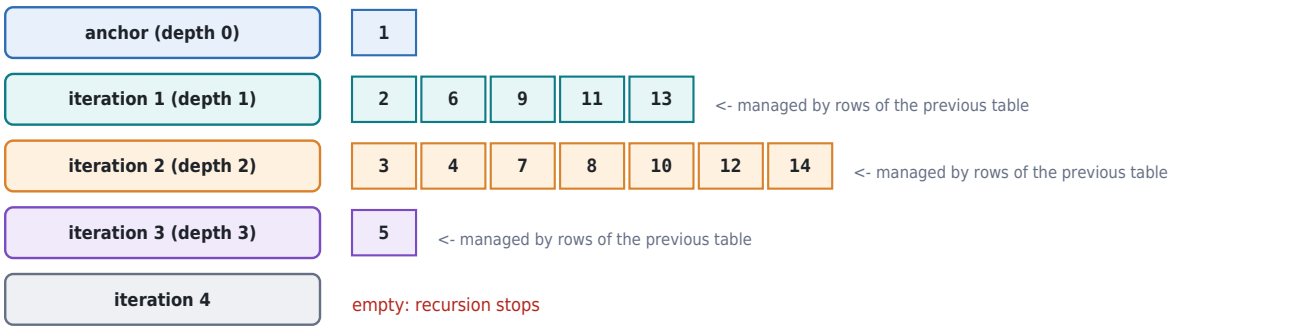


Figure 1.38. The working table of the recursive CTE over `employees`, iteration by iteration. Each iteration finds the employees whose manager is in the previous working table; the iteration after depth 3 returns no row and evaluation stops after producing 14 rows in total.

SQL

```sql
WITH RECURSIVE org AS (
  SELECT emp_id, first_name, manager_id, 0 AS depth, first_name::text AS path
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.first_name, e.manager_id, o.depth + 1,
         o.path || ' > ' || e.first_name
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT emp_id, depth, path FROM org ORDER BY path;
```

Result TERMINAL

```
 emp_id | depth |             path
--------+-------+-------------------------------
      1 |     0 | Asha
      6 |     1 | Asha > Arjun
      8 |     2 | Asha > Arjun > Kiran
     14 |     2 | Asha > Arjun > Omar
      7 |     2 | Asha > Arjun > Priya
      9 |     1 | Asha > Divya
     10 |     2 | Asha > Divya > Rahul
     11 |     1 | Asha > Neha
     12 |     2 | Asha > Neha > Sanjay
     13 |     1 | Asha > Tara
      2 |     1 | Asha > Vikram
      3 |     2 | Asha > Vikram > Meera
      5 |     3 | Asha > Vikram > Meera > Sneha
      4 |     2 | Asha > Vikram > Ravi
(14 rows)
```

The recursion supports "all subordinates of manager 6" (anchor `WHERE emp_id = 6`), "the chain of managers above an employee" (follow `manager_id` upward) and bill-of-materials explosions. Cycles in the data (employee A manages B and B manages A) make the recursion run forever. Defences are a depth limit in the recursive step (`WHERE o.depth < 20`), `UNION` instead of `UNION ALL` (which removes already-seen rows, though only when entire rows match, so a changing `depth` or `path` column defeats it) and the PostgreSQL 14 clause `CYCLE emp_id SET is_cycle USING cycle_path`. MySQL 8 supports `WITH RECURSIVE`, limits recursion by `cte_max_recursion_depth` (default 1000) and has no `CYCLE` or `SEARCH` clause.

1.9.3 LATERAL subqueries

A `LATERAL` subquery in FROM may reference columns of the tables that precede it, which makes it a per-row function returning a table. The classic use is "top N rows of a child table for each parent", which a plain join cannot express because LIMIT applies to the whole result. PostgreSQL has supported it since 9.3, MySQL since 8.0.14 (as `JOIN LATERAL`), and SQL Server offers `CROSS APPLY`.

Sriram

```sql
SELECT c.customer_id, c.name, r.order_id, r.order_ts::date AS day, r.status
FROM customers c
LEFT JOIN LATERAL (
    SELECT o.order_id, o.order_ts, o.status
    FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY o.order_ts DESC
    LIMIT 2
) r ON true
ORDER BY c.customer_id, r.order_ts DESC;
```

Result

```
 customer_id |      name       | order_id |    day     |  status
-------------+-----------------+----------+------------+-----------
           1 | Anil Traders    |      106 | 2025-01-06 | PAID
           1 | Anil Traders    |      102 | 2025-01-04 | SHIPPED
           2 | Bharat Stores   |      109 | 2025-01-09 | NEW
           2 | Bharat Stores   |      104 | 2025-01-05 | CANCELLED
           3 | Chitra Boutique |      110 | 2025-01-10 | PAID
           3 | Chitra Boutique |      105 | 2025-01-05 | NEW
           4 | Deepak & Sons   |          |            |
           5 | Eshwar Mart     |      108 | 2025-01-08 | PAID
           5 | Eshwar Mart     |      107 | 2025-01-07 | SHIPPED
           6 | Farah Fashions  |          |            |
(10 rows)
```

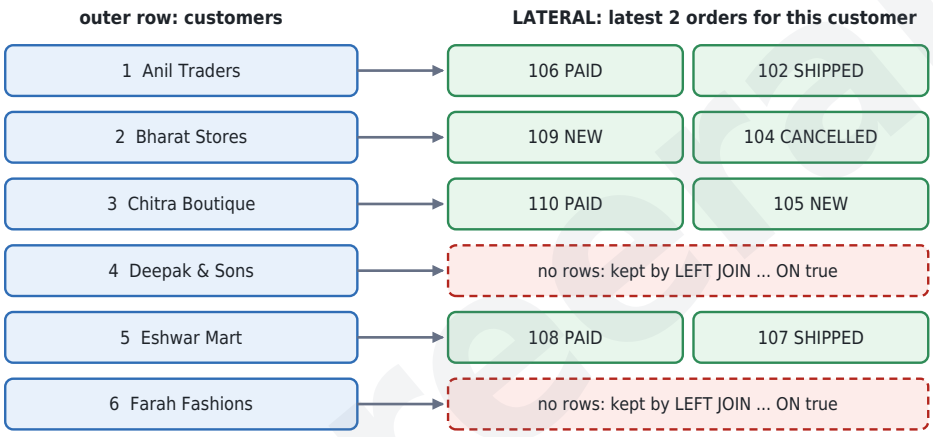


Figure 1.39. LATERAL evaluation for the latest two orders of each customer. The subquery is re-evaluated for each outer row with that row's `customer_id` bound; customers 4 and 6 return no rows and are kept by the LEFT JOIN.

With an index on `(customer_id, order_ts DESC)` each execution is a short index range scan, so the cost is proportional to the number of customers times N, independent of the total number of orders.

1.9.4 Worked example: employees above their department average

The statement `SELECT ... WHERE salary > (SELECT avg(...) ... WHERE x.dept_id = e.dept_id)` is traced on the sample data.

1. For employee 1 the inner query binds `e.dept_id = 10` and computes the average of the Engineering salaries 250000, 180000, 150000, 150000 and 120000, which is 170000. The comparison 250000 > 170000 is true; the row is kept.
2. Employee 2 (180000) is compared with the same average of 170000: true.
3. Employees 3 and 4 (150000) and employee 5 (120000) are below 170000: false.
4. For Sales (employees 6, 7, 8 and 14; salaries 170000, 110000, 95000 and 110000) the average is 121250. Only employee 6 (170000) exceeds it.
5. For employee 13 the inner query binds `e.dept_id = NULL`; `x.dept_id = NULL` is never true, so the inner query sees no rows, `avg` returns NULL, and `salary > NULL` is unknown: the row is dropped.

The result shown earlier confirms the trace: Engineering contributes employees 1 and 2, Sales contributes employee 6, and employees 9 and 11 qualify because each is the better paid of two in a department whose average lies between the two salaries.

1.9.5 Hands-on

The three formulations of an anti-join (`NOT EXISTS`, `LEFT JOIN ... IS NULL`, `NOT IN`) produce visibly different plans. The experiment uses `big_customers` (2,000 rows) and `big_orders` (200,000 rows) loaded for the examples in this chapter.

SQL

```sql
EXPLAIN (COSTS OFF)
SELECT count(*) FROM big_customers c
WHERE NOT EXISTS (SELECT 1 FROM big_orders o WHERE o.customer_id = c.customer_id);

EXPLAIN (COSTS OFF)
SELECT count(*) FROM big_customers c
WHERE c.customer_id NOT IN (SELECT o.customer_id FROM big_orders o);
```

Result TERMINAL

```
                     QUERY PLAN
-----------------------------------------------------
 Aggregate
   ->  Hash Right Anti Join
         Hash Cond: (o.customer_id = c.customer_id)
         ->  Seq Scan on big_orders o
         ->  Hash
               ->  Seq Scan on big_customers c
(6 rows)

                QUERY PLAN
----------------------------------------
 Aggregate
   ->  Seq Scan on big_customers c
         Filter: (NOT (hashed SubPlan 1))
         SubPlan 1
           ->  Seq Scan on big_orders o
(5 rows)
```

`NOT EXISTS` becomes a hash anti join (`Hash Right Anti Join`, with the larger table as the probe side), an efficient single pass. `NOT IN` becomes a filter on a hashed `SubPlan` because the planner must preserve the NULL semantics: it cannot turn the construct into an anti-join unless it can prove the subquery column is never NULL. The first plan is therefore preferable on large tables, in addition to being semantically safer.

1.9.6 Real-world problems and solutions

Incident: NOT IN returns no rows because of a NULL

Scenario. A clean-up job lists departments without employees with `SELECT dept_name FROM departments WHERE dept_id NOT IN (SELECT dept_id FROM employees)`. Legal has no employees, yet the job reports that all departments have staff.

Symptoms. The query returns zero rows although `departments` contains a department that no employee references. The result was correct in the development database and became empty after a contractor without a department was added in production.

Diagnosis. Step 1: run the query and the same query with the subquery restricted to non-NULL values.

SQL

```sql
SELECT dept_name FROM departments
WHERE dept_id NOT IN (SELECT dept_id FROM employees);

SELECT dept_name FROM departments
WHERE dept_id NOT IN (SELECT dept_id FROM employees WHERE dept_id IS NOT NULL);
```

Result TERMINAL

```
 dept_name
-----------
(0 rows)

 dept_name
-----------
 Legal
(1 row)
```

Step 2: confirm that the subquery returns a NULL.

SQL

```sql
SELECT count(*) FILTER (WHERE dept_id IS NULL) AS null_rows,
       count(DISTINCT dept_id) AS distinct_values
FROM employees;
```

Result TERMINAL

```
 null_rows | distinct_values
-----------+-----------------
         1 |               4
(1 row)
```

Root cause. `x NOT IN (a, b, NULL)` is evaluated as `x <> a AND x <> b AND x <> NULL`. The last term is unknown for every `x`, so the whole conjunction can be at best unknown and never true. A WHERE clause keeps only true rows, so none survives. The behaviour is correct three-valued logic, not a defect of the database.

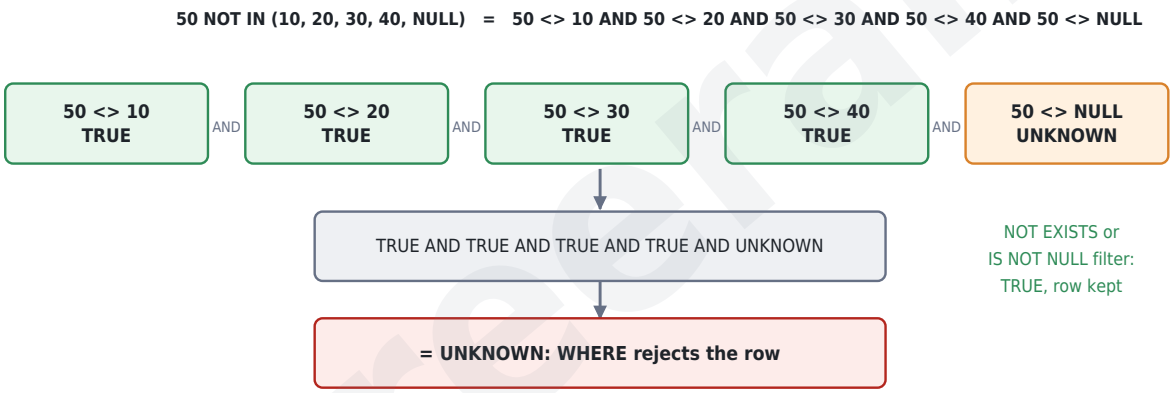


Figure 1.40. Evaluation of `50 NOT IN (10, 20, 30, 40, NULL)`. Each comparison with a real value is true, but the comparison with NULL is unknown and the AND of true and unknown is unknown, so the row is rejected. With the NULL removed, or with NOT EXISTS, the result is true.

Fix. Immediate: rewrite as an anti-join that is not sensitive to NULL.

SQL

```sql
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Result TERMINAL

```
 dept_name
-----------
 Legal
(1 row)
```

Permanent: use `NOT EXISTS` for every "has no matching row" test, in SQL, JPQL and Criteria queries; make columns that must be present `NOT NULL`, so that the question cannot arise. **Prevention.** A test with a NULL in the subquery column is part of the standard test data; static review flags every `NOT IN (SELECT ...)`. Note that `NOT IN` over a literal list from application code has the same behaviour when the list contains a null element.

Incident: slow report caused by a correlated subquery

Scenario. A nightly report lists every customer with total spend on orders that are not in status NEW. The query uses a scalar subquery in the SELECT list. It ran in well under a second with 20 test customers and takes minutes in production, where `big_customers` has 2,000 rows and `big_orders` 200,000.

Symptoms. The query's run time grows with the product of customers and orders; CPU is saturated and `pg_stat_activity` shows one backend running for minutes; adding more memory does not help.

Diagnosis. Step 1: run EXPLAIN ANALYZE on a sample of 100 customers.

SQL

```sql
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY ON)
SELECT c.customer_id,
       (SELECT sum(o.amount) FROM big_orders o
        WHERE o.customer_id = c.customer_id AND o.status <> 'NEW') AS spend
FROM big_customers c WHERE c.customer_id <= 100;
```

Result TERMINAL

```
                                  QUERY PLAN
-------------------------------------------------------------------------------------
 Index Only Scan using big_customers_pkey on big_customers c (actual rows=100 loops=1)
   Index Cond: (customer_id <= 100)
   Heap Fetches: 100
   SubPlan 1
     ->  Aggregate (actual rows=1 loops=100)
           ->  Seq Scan on big_orders o (actual rows=80 loops=100)
                 Filter: ((status <> 'NEW'::text) AND (customer_id = c.customer_id))
                 Rows Removed by Filter: 199920
 Planning Time: 0.889 ms
 Execution Time: 1350.645 ms
(10 rows)
```

The plan shows a `SubPlan` with `loops=100`: the inner `Seq Scan on big_orders` is executed once per customer and removes 199,920 rows each time (reading all 200,000 rows to keep 80). The whole statement costs 100 full scans (about 1.5 s in the run shown; runs on the same machine varied between 1.5 and 4 s, because other processes share it); the 2,000-customer report therefore needs 20 times as many scans, and the time grows linearly with both table sizes. All timings are measured on the build machine and are typical, not exact.

Root cause. A correlated scalar subquery with an aggregate is executed per outer row, and no index exists on `big_orders(customer_id)` to make each execution cheap. The planner cannot unnest this form into a join.

Fix. Immediate: aggregate once and join.

SQL

```sql
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY ON)
SELECT c.customer_id, s.spend
FROM big_customers c
LEFT JOIN (SELECT customer_id, sum(amount) AS spend
           FROM big_orders WHERE status <> 'NEW'
           GROUP BY customer_id) s ON s.customer_id = c.customer_id
WHERE c.customer_id <= 100;
```

Result TERMINAL

```
                                        QUERY PLAN
-------------------------------------------------------------------------------------------------------
 Hash Right Join (actual rows=100 loops=1)
   Hash Cond: (big_orders.customer_id = c.customer_id)
   ->  Finalize HashAggregate (actual rows=1600 loops=1)
         Group Key: big_orders.customer_id
         Batches: 1  Memory Usage: 881kB
         ->  Gather (actual rows=3200 loops=1)
               Workers Planned: 1
               Workers Launched: 1
               ->  Partial HashAggregate (actual rows=1600 loops=2)
                     Group Key: big_orders.customer_id
                     Batches: 1  Memory Usage: 881kB
                     Worker 0:  Batches: 1  Memory Usage: 881kB
                     ->  Parallel Seq Scan on big_orders (actual rows=80000 loops=2)
                           Filter: (status <> 'NEW'::text)
                           Rows Removed by Filter: 20000
   ->  Hash (actual rows=100 loops=1)
```

Result (continued) TERMINAL

```
        Buckets: 1024  Batches: 1  Memory Usage: 12kB
        ->  Index Only Scan using big_customers_pkey on big_customers c (actual rows=100 loops=1)
              Index Cond: (customer_id <= 100)
              Heap Fetches: 100
 Planning Time: 0.487 ms
 Execution Time: 43.549 ms
(22 rows)
```

The aggregation runs once over the 200,000 rows (about 50 ms instead of about 1.5 s), whatever the number of customers. Permanent: where the correlated form is required (a per-customer top-N or a lateral query), create the supporting index so that each execution is an index scan.

SQL

```sql
CREATE INDEX big_orders_cust_idx ON big_orders (customer_id);
ANALYZE big_orders;
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY ON)
SELECT c.customer_id,
       (SELECT sum(o.amount) FROM big_orders o
        WHERE o.customer_id = c.customer_id AND o.status <> 'NEW') AS spend
FROM big_customers c WHERE c.customer_id <= 100;
DROP INDEX big_orders_cust_idx;
```

Result TERMINAL

```
CREATE INDEX
ANALYZE
                                      QUERY PLAN
---------------------------------------------------------------------------------------
 Index Only Scan using big_customers_pkey on big_customers c (actual rows=100 loops=1)
   Index Cond: (customer_id <= 100)
   Heap Fetches: 100
   SubPlan 1
     ->  Aggregate (actual rows=1 loops=100)
           ->  Bitmap Heap Scan on big_orders o (actual rows=80 loops=100)
                 Recheck Cond: (customer_id = c.customer_id)
                 Filter: (status <> 'NEW'::text)
                 Rows Removed by Filter: 20
                 Heap Blocks: exact=10000
                 ->  Bitmap Index Scan on big_orders_cust_idx (actual rows=100 loops=100)
                       Index Cond: (customer_id = c.customer_id)
 Planning Time: 0.315 ms
 Execution Time: 11.613 ms
(14 rows)

DROP INDEX
```

With the index each of the 100 executions reads only the 100 rows of one customer through a bitmap index scan, and the statement finishes in about 20 ms. **Prevention.** Review every scalar subquery in a SELECT list as a loop; check foreign-key columns for supporting indexes (PostgreSQL does not create them automatically); test reports at production data volume.

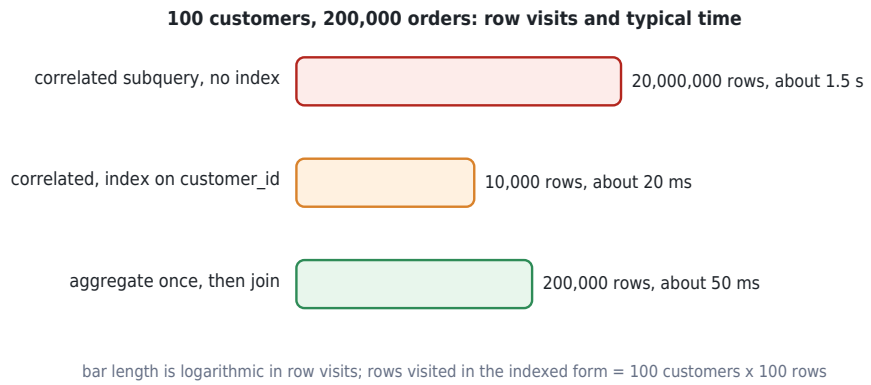


Figure 1.41. Work done by the correlated subquery compared with the aggregate-and-join rewrite for 100 customers over 200,000 orders. The correlated form reads the orders table once per customer (20,000,000 row visits); the rewrite reads it once (200,000 row visits).

1.9.7 Questions to expect

INTERVIEW ANGLE
What is a correlated subquery and why can it be slow? It references a column of the outer query, so it is logically evaluated once per outer row. If the planner cannot convert it to a join and the inner table has no supporting index, each evaluation scans the table, giving a cost proportional to rows(outer) x rows(inner). The fix is an index on the correlation column or a rewrite as an aggregate joined once.

INTERVIEW ANGLE
Why does NOT IN return nothing when the subquery contains NULL? `x NOT IN (a, NULL)` expands to `x <> a AND x <> NULL`; the second term is unknown, so the conjunction is never true and WHERE rejects every row. `NOT EXISTS` does not have the problem because it tests for the existence of a row, not a comparison with each value.

INTERVIEW ANGLE
When should EXISTS be used instead of IN or a JOIN? EXISTS for a pure existence test: it yields each outer row at most once and can stop at the first match. A JOIN multiplies the outer rows when several matches exist. IN gives the same plan as EXISTS in PostgreSQL but has the NOT IN hazard when negated.

INTERVIEW ANGLE
What is a CTE and how does it differ from a subquery or a view? A CTE is a named subquery scoped to one statement. It improves structure and supports recursion and data-modifying statements. A view is a stored definition reused across statements. Since PostgreSQL 12 a CTE referenced once is inlined; before, it was always materialised and blocked predicate push-down.

INTERVIEW ANGLE
How does a recursive CTE execute, and how is an infinite loop avoided? The anchor produces the first working table; the recursive part repeatedly joins the current working table with the base table and appends results until an iteration returns nothing. Loops in cyclic data are avoided with a depth limit, with UNION (duplicate elimination), or with the CYCLE clause.

INTERVIEW ANGLE
How is the latest row per group obtained? With a window function (`row_number() ... = 1`), with `DISTINCT ON` (PostgreSQL), with a correlated subquery on `max(...)` or with a LATERAL subquery limited to one row. The lateral form with a matching index is the fastest for few parents and many children.

INTERVIEW ANGLE
What does a scalar subquery return when it matches no row, and when it matches several? NULL for no row; a run-time error for several rows. A subquery that must return a single row should be backed by a unique constraint.

KEY POINTS
Subqueries and CTEs

- Subqueries return scalars, columns or tables; correlated subqueries are logically executed per outer row.
- `NOT IN` over a nullable column returns no rows; use `NOT EXISTS` for every anti-join.
- A correlated scalar subquery in a SELECT list is a hidden loop; rewrite it as an aggregate join or add the supporting index.
- CTEs structure queries; since PostgreSQL 12 a single-use CTE is inlined. `WITH RECURSIVE` walks hierarchies and needs a stopping condition for cyclic data.
- LATERAL gives top-N per parent; it needs a matching index to be fast.
- Verify claims about plans with EXPLAIN ANALYZE at production data volume.

sreeram

1.10 Window Functions

Aggregation with GROUP BY collapses rows: eight employees in a department become one output row. Many questions need the aggregate and the individual rows at the same time: each employee next to the department average, each order next to the running total of the customer, each salary together with its rank. A window function computes a value for every row from a set of related rows (its window) without collapsing them. Window functions replace self-joins and correlated subqueries for ranking, running totals, period-over-period comparison and top-N per group, and they are the most frequently tested advanced SQL topic in interviews.

1.10.1 Concept and mechanism

Anatomy of a window call

A window function call has the form `function(args) OVER (PARTITION BY ... ORDER BY ... frame)`. The three parts of the window specification are independent.

- `PARTITION BY` divides the rows into independent groups (partitions); the function never looks across a partition boundary. Omitting it makes the whole result one partition.
- `ORDER BY` orders the rows inside each partition. Ranking and offset functions require it; for aggregates it also defines the default frame.
- The frame clause selects, for each row, which rows of the partition the function sees (the current row, the preceding rows, a sliding range).

Window functions are evaluated after WHERE, GROUP BY and HAVING and before the final DISTINCT, ORDER BY and LIMIT. They can therefore be used only in the SELECT list and in ORDER BY. A filter on a window result (for example `rank() = 1`) requires wrapping the query in a subquery or CTE, because WHERE is evaluated before the window function exists.

employees (selected rows), PARTITION BY dept_id ORDER BY salary DESC

emp_id	first_name	dept_id	salary	sum() over frame	
9	Divya	30	140000	140000	
10	Rahul	30	80000	220000	partition boundary
6	Arjun	20	170000	170000	
7	Priya	20	110000	390000 <- current	frame for Priya: rows 3 to 5 (peer Omar included)
14	Omar	20	110000	390000	
8	Kiran	20	95000	485000	

Figure 1.42. A window `PARTITION BY dept_id ORDER BY salary DESC` over the Sales department. For the current row (Priya) the default frame ends at the last peer, so `sum` sees Arjun, Priya and Omar (blue); Kiran is outside the frame. The result is attached to every row; no row disappears.

SQL

```sql
SELECT emp_id, first_name, dept_id, salary,
       avg(salary) OVER (PARTITION BY dept_id) AS dept_avg,
       salary - avg(salary) OVER (PARTITION BY dept_id) AS diff
FROM employees
WHERE dept_id IN (20, 30)
ORDER BY dept_id, salary DESC;
```

Result TERMINAL

```
 emp_id | first_name | dept_id |  salary   |      dept_avg       |        diff
--------+------------+---------+-----------+---------------------+---------------------
      6 | Arjun      |      20 | 170000.00 | 121250.000000000000 |  48750.000000000000
      7 | Priya      |      20 | 110000.00 | 121250.000000000000 | -11250.000000000000
     14 | Omar       |      20 | 110000.00 | 121250.000000000000 | -11250.000000000000
      8 | Kiran      |      20 |  95000.00 | 121250.000000000000 | -26250.000000000000
      9 | Divya      |      30 | 140000.00 | 110000.000000000000 |  30000.000000000000
     10 | Rahul      |      30 |  80000.00 | 110000.000000000000 | -30000.000000000000
(6 rows)
```

Ranking functions

`row_number()` numbers the rows of a partition 1, 2, 3, ... with no ties: rows with equal sort keys receive different numbers in an arbitrary order unless the ORDER BY is total. `rank()` gives equal rows the same number and leaves gaps after them (1, 2, 2, 4). `dense_rank()` gives equal rows the same number without gaps (1, 2, 2, 3). `ntile(n)` splits the partition into n buckets of nearly equal size, and `percent_rank()` and `cume_dist()` give relative positions.

SQL

```sql
SELECT first_name, salary,
       row_number() OVER w AS row_number,
       rank()       OVER w AS rank,
       dense_rank() OVER w AS dense_rank,
       ntile(3)     OVER w AS tile
FROM employees
WHERE dept_id IN (10, 20)
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC, first_name;
```

Result TERMINAL

```
 first_name |  salary   | row_number | rank | dense_rank | tile
------------+-----------+------------+------+------------+------
 Asha       | 250000.00 |          1 |    1 |          1 |    1
 Vikram     | 180000.00 |          2 |    2 |          2 |    1
 Arjun      | 170000.00 |          3 |    3 |          3 |    1
 Meera      | 150000.00 |          4 |    4 |          4 |    2
 Ravi       | 150000.00 |          5 |    4 |          4 |    2
 Sneha      | 120000.00 |          6 |    6 |          5 |    2
 Omar       | 110000.00 |          8 |    7 |          6 |    3
 Priya      | 110000.00 |          7 |    7 |          6 |    3
 Kiran      |  95000.00 |          9 |    9 |          7 |    3
(9 rows)
```

The named window (`WINDOW w AS (...)`) avoids repeating the specification. Meera and Ravi both earn 150000 and share rank 4 (the three higher salaries occupy ranks 1 to 3); the next salary, 120000 (Sneha), receives rank 6 with `rank()` and 5 with `dense_rank()`; Priya and Omar (110000) tie again. `row_number()` still distinguishes tied rows, but which of the two receives the lower number is not defined by the query: in this run Omar received 8 and Priya 7, and another plan could reverse them.

first_name	salary	row_number	rank	dense_rank
Asha	250000	1	1	1
Vikram	180000	2	2	2
Arjun	170000	3	3	3
Meera	150000	4	4	4
Ravi	150000	5	4	4
Sneha	120000	6	6	5
Priya	110000	7	7	6
Omar	110000	8	7	6
Kiran	95000	9	9	7

tie: same salary

rank: 4, 4, then 6

dense_rank: 4, 4, then 5

row_number differs for tied rows (7, 8 for Priya, Omar: order not defined)

Figure 1.43. Ranking functions on the salaries of Engineering and Sales. Tied salaries share a value in rank and dense_rank; rank skips the numbers consumed by the tie, dense_rank does not, and row_number always differs.

Offset functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE

`lag(x, n, default)` returns the value of x from n rows before the current row in the partition order (default 1), `lead` from n rows after. They need no self-join and make period-over-period comparisons direct. `first_value`, `last_value` and `nth_value` read values at fixed positions of the frame. `last_value` surprises beginners: with the default frame the last row of the frame is the current row (or its last peer), so `last_value` returns the current value; an explicit frame ending at `UNBOUNDED FOLLOWING` is needed.

```sql
SELECT order_id, customer_id, order_ts::date AS day,
       lag(order_ts::date)  OVER (PARTITION BY customer_id ORDER BY order_ts) AS prev_day,
       order_ts::date - lag(order_ts::date) OVER (PARTITION BY customer_id ORDER BY order_ts)
                                                                  AS days_since_prev,
       lead(order_id) OVER (PARTITION BY customer_id ORDER BY order_ts) AS next_order
FROM orders
WHERE customer_id IN (1, 2)
ORDER BY customer_id, order_ts;
```

Result

```
 order_id | customer_id |    day     |  prev_day  | days_since_prev | next_order
----------+-------------+------------+------------+-----------------+------------
      101 |           1 | 2025-01-03 |            |                 |        102
      102 |           1 | 2025-01-04 | 2025-01-03 |               1 |        106
      106 |           1 | 2025-01-06 | 2025-01-04 |               2 |
      103 |           2 | 2025-01-04 |            |                 |        104
      104 |           2 | 2025-01-05 | 2025-01-04 |               1 |        109
      109 |           2 | 2025-01-09 | 2025-01-05 |               4 |
(6 rows)
```

The first row of each partition has no predecessor, so `lag` returns NULL there (or the default, when one is supplied as the third argument).

1.10.2 Frames: ROWS, RANGE and GROUPS

When an aggregate is used with OVER and an ORDER BY, it is evaluated over a frame: a sliding subset of the partition. The frame clause has the form `{ROWS | RANGE | GROUPS} BETWEEN start AND end`, where start and end are `UNBOUNDED PRECEDING`, `n PRECEDING`, `CURRENT ROW`, `n FOLLOWING` or `UNBOUNDED FOLLOWING`.

- `ROWS` counts physical rows relative to the current row.
- `RANGE` works on the value of the ORDER BY expression: the frame includes all rows whose sort value lies within the stated distance, and the current row's peers (rows with an equal sort value) are always included together.
- `GROUPS` counts peer groups (PostgreSQL 11 and later; not in MySQL).

When ORDER BY is present and no frame is given, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. This default includes all peers of the current row, so two rows with the same sort value receive the same running total. This is the cause of the incident described below. Without ORDER BY the default frame is the whole partition.

```sql
WITH t(id, v) AS (VALUES (1, 10), (2, 20), (3, 20), (4, 40))
SELECT id, v,
       sum(v) OVER (ORDER BY v)                                          AS default_range,
       sum(v) OVER (ORDER BY v ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame,
       sum(v) OVER (ORDER BY v ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS moving2
FROM t ORDER BY id;
```

Result

```
 id | v  | default_range | rows_frame | moving2
----+----+---------------+------------+---------
  1 | 10 |            10 |         10 |      10
  2 | 20 |            50 |         30 |      30
  3 | 20 |            50 |         50 |      40
  4 | 40 |            90 |         90 |      60
(4 rows)
```

sum(v) OVER (ORDER BY v ...) over the rows (1,10), (2,20), (3,20), (4,40)

id	v	RANGE (default)	ROWS unbounded	ROWS 1 PRECEDING
1	10	10	10	10
2	20	50	30	30
3	20	50	50	40
4	40	90	90	60

Rows 2 and 3 are peers (equal v). RANGE adds all peers at once: both show 50.

ROWS adds one physical row at a time: 30, then 50.

ROWS 1 PRECEDING sums the current and previous row only: 20+10, 20+20, 40+20.

Figure 1.44. The same four rows evaluated with the default RANGE frame and with an explicit ROWS frame. Rows 2 and 3 are peers (v = 20): RANGE includes both for each of them, so both show 50; ROWS adds the rows one at a time (30, then 50).

MySQL 8 supports `ROWS` and `RANGE` frames but not `GROUPS`, and `RANGE` with an offset only on numeric and temporal ORDER BY expressions. MySQL 8.0 does not support `FILTER` on window aggregates.

1.10.3 Top-N per group and de-duplication

The standard pattern ranks rows inside each partition in a subquery and filters on the rank in the outer query. The choice of ranking function decides how ties are treated: `row_number()` returns exactly N rows per group, `rank()` and `dense_rank()` may return more when ties straddle the boundary.

SQL

```sql
SELECT dept_id, first_name, salary, rn
FROM (SELECT dept_id, first_name, salary,
             row_number() OVER (PARTITION BY dept_id ORDER BY salary DESC, emp_id) AS rn
      FROM employees WHERE dept_id IS NOT NULL) t
WHERE rn <= 2
ORDER BY dept_id, rn;
```

Result TERMINAL

```
 dept_id | first_name |  salary   | rn
---------+------------+-----------+----
      10 | Asha       | 250000.00 |  1
      10 | Vikram     | 180000.00 |  2
      20 | Arjun      | 170000.00 |  1
      20 | Priya      | 110000.00 |  2
      30 | Divya      | 140000.00 |  1
      30 | Rahul      |  80000.00 |  2
      40 | Neha       | 160000.00 |  1
      40 | Sanjay     |  90000.00 |  2
(8 rows)
```

PostgreSQL also offers `DISTINCT ON` for N = 1 and `LATERAL` for general N (see the subqueries chapter). The same `row_number() ... WHERE rn > 1` pattern identifies duplicates for deletion; the interview problems at the end of this part use it.

1.10.4 Gaps and islands

Consecutive sequences ("islands") in a sorted series are found by subtracting a row number from the value: within an island of consecutive values both increase by one per row, so their difference is constant; a gap changes the difference. Grouping by that difference isolates the islands. The technique applies to consecutive login days, consecutive ids, contiguous time slots and streaks.

day	rn	day - rn
03-01	1	02-28
03-02	2	02-28
03-03	3	02-28
03-05	4	03-01
03-06	5	03-01
03-09	6	03-03

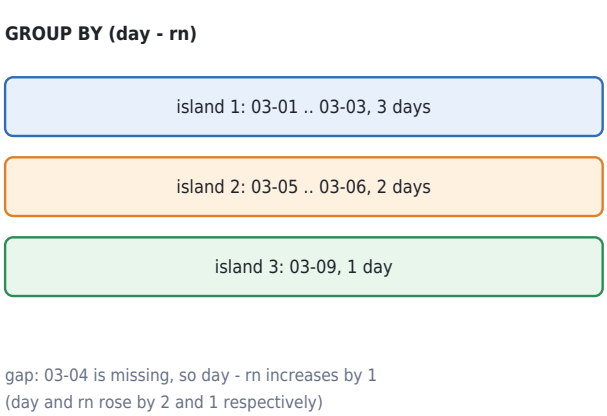


Figure 1.45. Gaps and islands on the login days of one user. The difference between the day number and the row number is constant inside a run of consecutive days; each distinct value is one island. The data is the `logins` table created for this example.

SQL

```
CREATE TEMP TABLE logins (user_id int, day date);
INSERT INTO logins VALUES (1,'2025-03-01'),(1,'2025-03-02'),(1,'2025-03-03'),
  (1,'2025-03-05'),(1,'2025-03-06'),(1,'2025-03-09');

SELECT day, row_number() OVER (ORDER BY day) AS rn,
       day - (row_number() OVER (ORDER BY day))::int AS grp
FROM logins ORDER BY day;

SELECT min(day) AS island_start, max(day) AS island_end, count(*) AS days
FROM (SELECT day, day - (row_number() OVER (ORDER BY day))::int AS grp FROM logins) t
GROUP BY grp ORDER BY island_start;
```

Result TERMINAL

```
CREATE TABLE
INSERT 0 6
    day     | rn |    grp
------------+----+------------
 2025-03-01 |  1 | 2025-02-28
 2025-03-02 |  2 | 2025-02-28
 2025-03-03 |  3 | 2025-02-28
 2025-03-05 |  4 | 2025-03-01
 2025-03-06 |  5 | 2025-03-01
 2025-03-09 |  6 | 2025-03-03
(6 rows)

 island_start | island_end | days
--------------+------------+------
 2025-03-01   | 2025-03-03 |    3
 2025-03-05   | 2025-03-06 |    2
 2025-03-09   | 2025-03-09 |    1
(3 rows)
```

1.10.5 Worked example: running total and share of the customer total

Requirement: for every paid or shipped order, show its amount, the running total of the customer's orders in time order, and the order's share of the customer's total.

The statement and its result are printed after the trace.

1. The CTE collapses each order to one row with its amount (GROUP BY). Window functions then operate on that grouped result.
2. The partitions are customers: customer 1 has orders 101 (2900), 102 (55000) and 106 (19900); customer 2 has 103 (25250); customer 3 has 110 (6000); customer 5 has 107 (55000) and 108 (2500).
3. For customer 1 the running total is 2900, then 2900 + 55000 = 57900, then 57900 + 19900 = 77800. The second window, without ORDER BY, spans the whole partition: 77800 for every row of the customer.
4. The share of order 102 is 55000 / 77800 = 70.7 percent.

```sql
WITH order_totals AS (
  SELECT o.order_id, o.customer_id, o.order_ts, sum(oi.qty * oi.unit_price) AS amount
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status IN ('PAID', 'SHIPPED')
  GROUP BY o.order_id, o.customer_id, o.order_ts
)
SELECT order_id, customer_id, amount,
       sum(amount) OVER (PARTITION BY customer_id ORDER BY order_ts, order_id) AS running,
       round(100 * amount / sum(amount) OVER (PARTITION BY customer_id), 1)  AS pct_of_customer
FROM order_totals
ORDER BY customer_id, order_ts;
```

Result

```
 order_id | customer_id |  amount  | running  | pct_of_customer
----------+-------------+----------+----------+-----------------
      101 |           1 |  2900.00 |  2900.00 |             3.7
      102 |           1 | 55000.00 | 57900.00 |            70.7
      106 |           1 | 19900.00 | 77800.00 |            25.6
      103 |           2 | 25250.00 | 25250.00 |           100.0
      110 |           3 |  6000.00 |  6000.00 |           100.0
      107 |           5 | 55000.00 | 55000.00 |            95.7
      108 |           5 |  2500.00 | 57500.00 |             4.3
(7 rows)
```

1.10.6 Hands-on

A window query needs a sort per distinct window specification, unless an index supplies the order. EXPLAIN shows a `WindowAgg` node above a `Sort`. Two functions with the same PARTITION BY and ORDER BY share one sort; a different ordering adds another.

```sql
EXPLAIN (COSTS OFF)
SELECT order_id, customer_id,
       sum(amount) OVER (PARTITION BY customer_id ORDER BY order_ts) AS running,
       rank() OVER (ORDER BY amount DESC) AS amount_rank
FROM big_orders WHERE order_id <= 1000;
```

Result

```
                            QUERY PLAN
------------------------------------------------------------------------
 WindowAgg
   ->  Sort
         Sort Key: customer_id, order_ts
         ->  WindowAgg
               ->  Sort
                     Sort Key: amount DESC
                     ->  Index Scan using big_orders_pkey on big_orders
                           Index Cond: (order_id <= 1000)
(8 rows)
```

The plan contains two WindowAgg nodes and two Sort nodes because the two windows have different orderings. Spring Data JPA does not support window functions in JPQL (Hibernate 6 adds some HQL support); they are normally issued through `@Query(nativeQuery = true)`, `JdbcTemplate` or a SQL-based tool such as jOOQ.

1.10.7 Real-world problems and solutions

Incident: running total repeats and jumps for same-day rows

Scenario. A cash-flow report shows the cumulative sum of daily payment postings per customer. Several orders are posted at the same timestamp (a batch job writes them in one transaction). The cumulative column shows the same value on several consecutive rows and then jumps.

Symptoms. Running totals do not increase row by row; reconciliation by row fails although the final total is right. The issue appears only where the sort column has duplicates.

Diagnosis. Step 1: reproduce on a small sample with a duplicate in the ordering column.

SQL

```sql
WITH p(id, posted, amount) AS (VALUES (1,'2025-01-01',100),(2,'2025-01-02',50),
                                      (3,'2025-01-02',70),(4,'2025-01-03',30))
SELECT id, posted, amount,
       sum(amount) OVER (ORDER BY posted) AS running_default,
       sum(amount) OVER (ORDER BY posted, id) AS running_explicit
FROM p ORDER BY id;
```

Result TERMINAL

```
 id |   posted   | amount | running_default | running_explicit
----+------------+--------+-----------------+------------------
  1 | 2025-01-01 |    100 |             100 |              100
  2 | 2025-01-02 |     50 |             220 |              150
  3 | 2025-01-02 |     70 |             220 |              220
  4 | 2025-01-03 |     30 |             250 |              250
(4 rows)
```

Step 2: identify the frame. With `ORDER BY posted` and no frame clause the frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`; rows 2 and 3 are peers because their `posted` values are equal, so each sees both of them and shows 220.

Root cause. The default frame in the presence of ORDER BY is RANGE, which treats rows with equal sort values as one unit. The result is correct by definition, but not what a per-row running total requires.

Fix. Make the order total by appending a unique column (`ORDER BY posted, id`), or state the frame explicitly with `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Both give 100, 150, 220, 250 for the example. **Prevention.** Write the frame explicitly in every running aggregate (`ROWS UNBOUNDED PRECEDING`), so that the intent is visible and the engine default is irrelevant.

Incident: top-3 per department returns four rows

Scenario. A report lists the three best paid employees of each department using `rank() <= 3`. Engineering shows four rows. **Diagnosis.** Ranking Engineering by salary gives Asha 1, Vikram 2, Meera 3 and Ravi 3 (the query below); `rank() <= 3` returns every row tied at the boundary, here four. **Fix.** Use `row_number()` with a total order when exactly N rows are needed, or keep `rank()` deliberately when ties must all be shown and state that in the report definition. **Prevention.** Decide, and document, how ties are treated before writing a top-N query; include a tie in the test data.

SQL

```sql
SELECT first_name, salary, rank() OVER (ORDER BY salary DESC) AS rnk
FROM employees WHERE dept_id = 10 ORDER BY salary DESC, first_name;
```

Result TERMINAL

```
 first_name |  salary   | rnk
------------+-----------+-----
 Asha       | 250000.00 |   1
 Vikram     | 180000.00 |   2
 Meera      | 150000.00 |   3
 Ravi       | 150000.00 |   3
 Sneha      | 120000.00 |   5
(5 rows)
```

1.10.8 Questions to expect

INTERVIEW ANGLE

What is the difference between GROUP BY and a window function? GROUP BY collapses each group to one row. A window function computes a value for each row from a related set of rows and keeps every row. Window functions are evaluated after GROUP BY and HAVING and can be applied to the grouped result.

INTERVIEW ANGLE

How do ROW_NUMBER, RANK and DENSE_RANK differ? ROW_NUMBER assigns unique consecutive numbers, breaking ties arbitrarily. RANK gives ties the same number and skips the following numbers (1, 2, 2, 4). DENSE_RANK gives ties the same number without gaps (1, 2, 2, 3).

INTERVIEW ANGLE
What is the default window frame, and why does it matter? With ORDER BY and no frame clause it is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which includes all peers of the current row. Rows with equal sort values therefore get the same running total, and `last_value` returns the current value. An explicit ROWS frame or a unique ordering gives per-row behaviour.

INTERVIEW ANGLE
How is the second highest salary found with a window function? `SELECT DISTINCT salary FROM (SELECT salary, dense_rank() OVER (ORDER BY salary DESC) AS r FROM employees) t WHERE r = 2`. DENSE_RANK is used so that tied top salaries count as one level; the subquery is needed because a window function cannot appear in WHERE.

INTERVIEW ANGLE
How is the top N per group written? Rank inside each partition in a subquery (`row_number() OVER (PARTITION BY g ORDER BY x DESC)`) and filter `rn <= N` outside. Use rank() when ties at the boundary must be included.

INTERVIEW ANGLE
What are LAG and LEAD used for? They read a column from a preceding or following row of the partition without a self-join; typical uses are the difference to the previous period, detection of changes and computing time between events.

INTERVIEW ANGLE
Why can a window function not be used in WHERE? WHERE is evaluated before window functions exist in the logical processing order. The filter must be applied in an enclosing query or CTE, or with the QUALIFY clause on engines that offer it (not PostgreSQL or MySQL).

KEY POINTS
Window functions

- OVER (PARTITION BY ... ORDER BY ... frame) computes per-row values from a window and keeps every row.
- Window functions run after GROUP BY and HAVING; filtering on them needs a subquery or CTE.
- ROW_NUMBER is unique, RANK leaves gaps after ties, DENSE_RANK does not.
- The default frame with ORDER BY is RANGE up to the current row including peers; state ROWS explicitly for running totals.
- Top-N per group, de-duplication, running totals, period comparison and gaps-and-islands are the standard patterns.
- Windows with different orderings cost separate sorts; check the plan for WindowAgg and Sort nodes.

1.11 Set Operations, Views, Functions, Procedures and Triggers

The preceding chapters treat SQL as a query language. This chapter covers the constructs that package and automate it: set operations that combine whole result sets, views and materialised views that give a query a name, and server-side code (functions, procedures and triggers) that runs inside the database. Each of them solves a real problem and each, used without discipline, creates behaviour that is invisible to the application developer. The closing sections state when not to use them.

1.11.1 Set operations

A set operation combines the results of two queries that have the same number of columns with compatible types. The column names come from the first query.

Operator	Result	Duplicates
`UNION`	Rows in either input	Removed (requires a sort or hash)
`UNION ALL`	Rows of both inputs appended	Kept; no extra work
`INTERSECT`	Rows present in both inputs	Removed (`INTERSECT ALL` keeps multiplicities)

Operator	Result	Duplicates
EXCEPT	Rows of the first input absent from the second	Removed (`EXCEPT ALL`; Oracle: MINUS)

Set operators.

Duplicate elimination is the hidden cost: `UNION` must compare whole rows, so when the inputs are known to be disjoint, or duplicates are acceptable, `UNION ALL` is cheaper and returns the same rows. MySQL supports UNION and UNION ALL in all versions and `INTERSECT` and `EXCEPT` from 8.0.31. For set operations NULLs are treated as equal to each other, unlike in a join.

SQL

```sql
SELECT city AS place FROM customers WHERE city IS NOT NULL
UNION
SELECT location FROM departments
ORDER BY place;

SELECT city FROM customers WHERE city IS NOT NULL
INTERSECT
SELECT location FROM departments;

SELECT city FROM customers WHERE city IS NOT NULL
EXCEPT
SELECT location FROM departments
ORDER BY 1;
```

Result TERMINAL

```
   place
-----------
 Chennai
 Delhi
 Hyderabad
 Mumbai
(4 rows)

   city
-----------
 Mumbai
 Delhi
 Hyderabad
(3 rows)

  city
---------
 Chennai
(1 row)
```

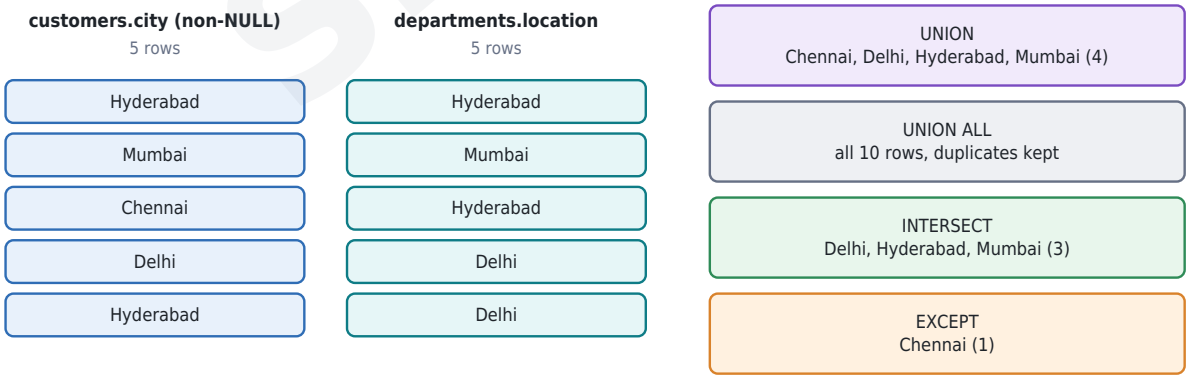


Figure 1.46. Set operations on customer cities (left set) and department locations (right set). UNION returns 4 distinct places, INTERSECT the 3 places that occur in both sets, and EXCEPT the one customer city (Chennai) without a department. UNION ALL would return 10 rows, keeping every duplicate.

Set operations also solve problems that joins handle badly. `EXCEPT` expresses "present in A and missing in B" over several columns at once without NULL traps, and `UNION ALL` combines tables with identical structure (monthly partitions, archive and live tables). A `UNION ALL` of two aggregations replaces `GROUPING SETS` in MySQL.

sreeram

1.11.2 Views

A view stores a query under a name. It stores no data: every reference to the view is replaced by its definition (view expansion), and the planner optimises the combined statement, so predicates on the view are pushed into the underlying query. A view therefore adds no performance cost of its own, and equally gives no speed-up.

SQL

```sql
CREATE VIEW order_totals AS
SELECT o.order_id, o.customer_id, o.order_ts, o.status,
       sum(oi.qty * oi.unit_price) AS amount,
       count(*)                    AS line_count
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.customer_id, o.order_ts, o.status;
```

SQL

```sql
CREATE VIEW order_totals AS
SELECT o.order_id, o.customer_id, o.order_ts, o.status,
       sum(oi.qty * oi.unit_price) AS amount,
       count(*)                    AS line_count
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.customer_id, o.order_ts, o.status;

SELECT customer_id, count(*) AS orders, sum(amount) AS spend
FROM order_totals WHERE status <> 'CANCELLED' GROUP BY customer_id ORDER BY 1;
```

Result TERMINAL

```
CREATE VIEW
 customer_id | orders |  spend
-------------+--------+----------
           1 |      3 | 77800.00
           2 |      2 | 28250.00
           3 |      2 |  7450.00
           5 |      2 | 57500.00
(4 rows)
```

The predicate `status <> 'CANCELLED'` is applied by the planner to the base table `orders` before aggregation because `status` is a grouping column of the view. A predicate on `amount` cannot be pushed below the aggregation. This is the typical limit of view expansion: filters on computed columns act on the view's output.

Uses of views: a stable interface over a changing schema (the application reads the view, while the tables are restructured), a single definition of a business rule (the meaning of "amount"), and security (a role is granted access to a view that exposes only some columns or rows).

Updatable views

A simple view over one table (no aggregation, DISTINCT, LIMIT or set operations) is automatically updatable in PostgreSQL and MySQL: INSERT, UPDATE and DELETE on the view are rewritten to the base table. `WITH CHECK OPTION` rejects changes that would make a row invisible through the view.

SQL

```sql
CREATE VIEW sales_staff AS
  SELECT emp_id, first_name, salary, dept_id FROM employees WHERE dept_id = 20
  WITH CHECK OPTION;

UPDATE sales_staff SET salary = salary + 1000 WHERE emp_id = 7;
UPDATE sales_staff SET dept_id = 30 WHERE emp_id = 7;
```

Result TERMINAL

```
CREATE VIEW
UPDATE 1
ERROR:  new row violates check option for view "sales_staff"
DETAIL:  Failing row contains (7, Priya, Menon, 30, 6, 111000.00, 2020-06-30).
```

SQL

```sql
UPDATE order_totals SET amount = 0 WHERE order_id = 101;
```

Result TERMINAL

```
ERROR:  cannot update view "order_totals"
DETAIL:  Views containing GROUP BY are not automatically updatable.
HINT:  To enable updating the view, provide an INSTEAD OF UPDATE trigger or an unconditional ON
       UPDATE DO INSTEAD rule.
```

The second statement fails because the changed row would leave the view; the separate third statement fails because an aggregating view is not updatable. Such views need `INSTEAD OF` triggers (PostgreSQL; MySQL has no such triggers) or must be bypassed by writing to the base tables.

1.11.3 Materialised views

A materialised view stores the result of its query as a table. Reads are as fast as reads from a table (and can be indexed), but the content is a snapshot that becomes stale until it is refreshed. It trades freshness for speed and is the standard remedy for an expensive aggregation that many users read.

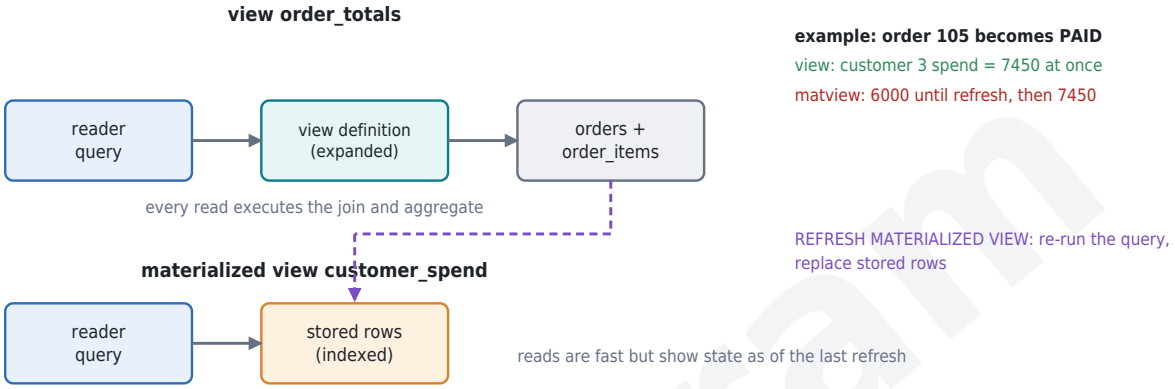


Figure 1.47. A view re-executes its query on every read; a materialised view executes it at refresh time and serves reads from stored rows. Between refreshes the stored rows lag behind the base tables.

SQL

```sql
CREATE MATERIALIZED VIEW customer_spend AS
SELECT customer_id, count(*) AS orders, sum(amount) AS spend
FROM order_totals WHERE status IN ('PAID','SHIPPED') GROUP BY customer_id;

CREATE UNIQUE INDEX customer_spend_pk ON customer_spend (customer_id);
SELECT * FROM customer_spend ORDER BY customer_id;
```

Result TERMINAL

```
SELECT 4
CREATE INDEX
 customer_id | orders |  spend
-------------+--------+----------
           1 |      3 | 77800.00
           2 |      1 | 25250.00
           3 |      1 |  6000.00
           5 |      2 | 57500.00
(4 rows)
```

A refresh re-runs the query. `REFRESH MATERIALIZED VIEW` takes an exclusive lock and blocks readers while it runs; `REFRESH MATERIALIZED VIEW CONCURRENTLY` builds the new content beside the old one and swaps differences in, so readers continue, but it requires a unique index on the materialised view and is slower.

SQL

```sql
UPDATE orders SET status = 'PAID' WHERE order_id = 105;
SELECT customer_id, spend FROM customer_spend WHERE customer_id = 3;
REFRESH MATERIALIZED VIEW CONCURRENTLY customer_spend;
SELECT customer_id, spend FROM customer_spend WHERE customer_id = 3;
```

Result TERMINAL

```
UPDATE 1
 customer_id |  spend
-------------+--------
           3 | 6000.00
(1 row)

REFRESH MATERIALIZED VIEW
 customer_id |  spend
-------------+--------
           3 | 7450.00
(1 row)
```

Before the refresh customer 3 shows 6000, the value computed before order 105 was paid; after it the view shows 7450. MySQL has no materialised views; the usual substitutes are a summary table maintained by the application or by triggers, a scheduled `EVENT` that rebuilds it, or a stored procedure run by a job scheduler. PostgreSQL does not refresh materialised views automatically; the schedule (pg_cron, an application job, a Spring `@Scheduled` method) is part of the design, and a refresh failure must be monitored.

1.11.4 Functions and stored procedures

PostgreSQL distinguishes functions, which return a value or a set of rows and are called inside queries, from procedures (PostgreSQL 11 and later), which are invoked with `CALL`, return no value (apart from `INOUT` parameters) and may commit or roll back transactions. MySQL has the same two objects, with a different dialect. Languages in PostgreSQL include SQL, PL/pgSQL, and extensions such as PL/Python.

SQL and PL/pgSQL functions

A SQL-language function is a parameterised query, which the planner can inline into the calling statement. A PL/pgSQL function adds variables, control flow, loops and exception handling, and is opaque to the planner.

SQL

```sql
CREATE OR REPLACE FUNCTION dept_payroll(p_dept integer) RETURNS numeric
LANGUAGE sql STABLE AS $$
  SELECT coalesce(sum(salary), 0) FROM employees WHERE dept_id = p_dept
$$;

CREATE OR REPLACE FUNCTION raise_salary(p_emp integer, p_pct numeric) RETURNS numeric
LANGUAGE plpgsql AS $$
DECLARE v_new numeric;
BEGIN
  IF p_pct <= 0 OR p_pct > 50 THEN
    RAISE EXCEPTION 'invalid percentage: %', p_pct USING ERRCODE = '22023';
  END IF;
  UPDATE employees SET salary = round(salary * (1 + p_pct / 100), 2)
  WHERE emp_id = p_emp RETURNING salary INTO v_new;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'employee % not found', p_emp;
  END IF;
  RETURN v_new;
END $$;

SELECT dept_payroll(20) AS sales_payroll, dept_payroll(50) AS legal_payroll;
BEGIN;
SELECT raise_salary(7, 10) AS new_salary;
SELECT raise_salary(7, 90) AS bad;
ROLLBACK;
```

Result TERMINAL

```
CREATE FUNCTION
CREATE FUNCTION
 sales_payroll | legal_payroll
---------------+---------------
     485000.00 |             0
(1 row)

BEGIN
 new_salary
------------
  121000.00
```

Result (continued) TERMINAL

```
(1 row)

ERROR:  invalid percentage: 90
CONTEXT:  PL/pgSQL function raise_salary(integer,numeric) line 5 at RAISE
ROLLBACK
```

The volatility label is a contract with the planner. `IMMUTABLE` promises the same result for the same arguments forever (usable in index expressions), `STABLE` the same result within one statement (no data modification), and `VOLATILE` (the default) anything. A function wrongly declared `IMMUTABLE` that reads a table returns stale results from cached plans and expression indexes; a function that is `VOLATILE` and could have been `STABLE` cannot be used for index scans in some forms and is called once per row.

Procedures with transaction control

A procedure can process a large batch in chunks and commit between chunks, which a function cannot (a function always runs inside the caller's transaction). The following procedure archives paid orders' payments in a loop; the control flow and the `COMMIT` are what matter.

SQL

```sql
CREATE OR REPLACE PROCEDURE archive_payments(p_batch integer)
LANGUAGE plpgsql AS $$
DECLARE v_moved integer;
BEGIN
  LOOP
    WITH moved AS (
      DELETE FROM payments WHERE payment_id IN
        (SELECT payment_id FROM payments ORDER BY payment_id LIMIT p_batch)
      RETURNING *)
    INSERT INTO payments_archive SELECT * FROM moved;
    GET DIAGNOSTICS v_moved = ROW_COUNT;
    EXIT WHEN v_moved = 0;
    COMMIT;                                   -- release locks, make progress durable
  END LOOP;
END $$;

CALL archive_payments(4);
SELECT (SELECT count(*) FROM payments) AS left_in_payments,
       (SELECT count(*) FROM payments_archive) AS archived;
```

Result TERMINAL

```
CREATE PROCEDURE
CALL
 left_in_payments | archived
------------------+----------
                0 |       10
(1 row)
```

PostgreSQL 16	MySQL 8.x
`CREATE FUNCTION f(a int) RETURNS int LANGUAGE sql AS $$ SELECT a + 1 $$;`	`DELIMITER // CREATE FUNCTION f(a INT) RETURNS INT DETERMINISTIC BEGIN RETURN a + 1; END// DELIMITER ;`
`CALL p(1);`	`CALL p(1);`
Dollar quoting `$$ ... $$` for the body	The client needs a temporary delimiter (DELIMITER) because the body contains semicolons
Functions may return tables (RETURNS TABLE, SETOF)	Functions return scalars; result sets only from procedures
Volatility: IMMUTABLE, STABLE, VOLATILE	DETERMINISTIC / NOT DETERMINISTIC, READS SQL DATA

The same server-side routine in PostgreSQL and MySQL (MySQL not run on the build machine).

sreeram

1.11.5 Triggers

A trigger runs a function automatically when a row is inserted, updated or deleted. Timing is BEFORE, AFTER or INSTEAD OF (views); granularity is FOR EACH ROW or FOR EACH STATEMENT (PostgreSQL; MySQL has row-level triggers only). A BEFORE row trigger may modify the new row (`NEW`) or cancel the operation by returning NULL; an AFTER row trigger sees the final row and is the right place for side effects such as writing an audit record. Inside the function `OLD` and `NEW` hold the old and new row.

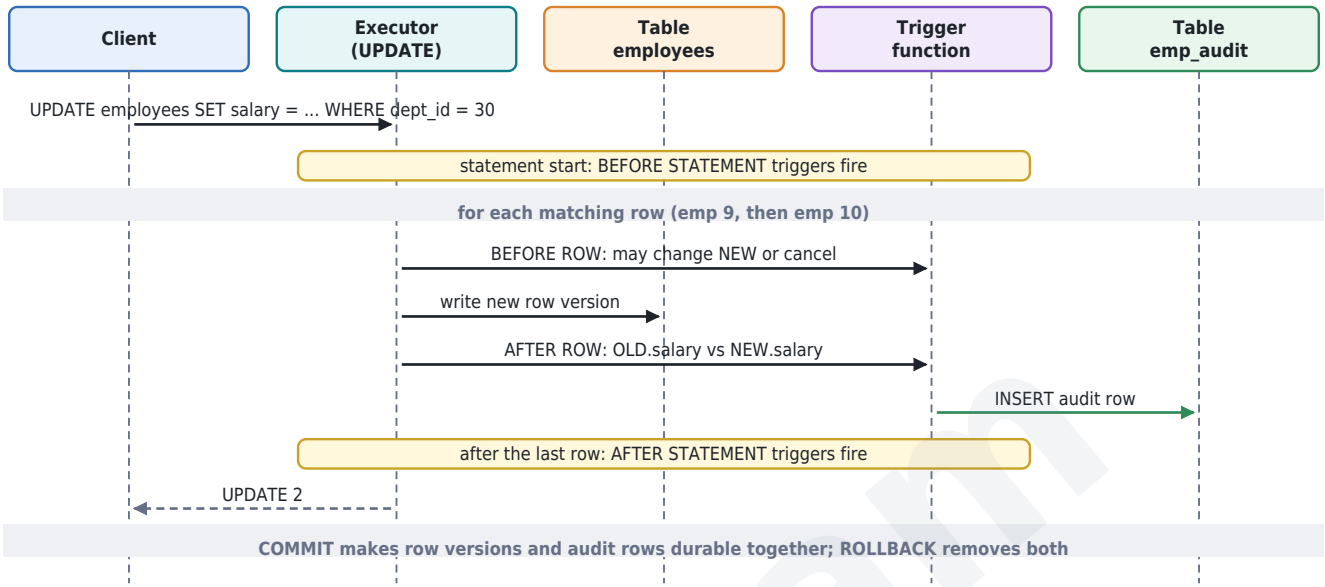


Figure 1.48. Order of events for `UPDATE employees SET salary = ...` on one row with a BEFORE row trigger, an AFTER row trigger writing to `emp_audit`, and a statement-level AFTER trigger. All run in the caller's transaction; an error in any of them rolls the whole statement back.

SQL

```sql
CREATE TABLE emp_audit (
  audit_id   bigserial PRIMARY KEY,
  emp_id     integer, old_salary numeric, new_salary numeric,
  changed_at timestamptz DEFAULT now()
);

CREATE OR REPLACE FUNCTION trg_salary_audit() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.salary IS DISTINCT FROM OLD.salary THEN
    INSERT INTO emp_audit (emp_id, old_salary, new_salary)
    VALUES (OLD.emp_id, OLD.salary, NEW.salary);
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER employees_salary_audit
AFTER UPDATE OF salary ON employees
FOR EACH ROW EXECUTE FUNCTION trg_salary_audit();

UPDATE employees SET salary = salary * 1.05 WHERE dept_id = 30;
SELECT audit_id, emp_id, old_salary, new_salary FROM emp_audit ORDER BY audit_id;
```

Result TERMINAL

```
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
UPDATE 2
 audit_id | emp_id | old_salary | new_salary
----------+--------+------------+------------
        1 |      9 |  140000.00 |  147000.00
        2 |     10 |   80000.00 |   84000.00
(2 rows)
```

The trigger recorded both changes without any change to the application. The same code written in MySQL is a `CREATE TRIGGER ... AFTER UPDATE ON employees FOR EACH ROW BEGIN ... END` with the `DELIMITER` command and no `WHEN`, `OF column` or statement-level forms.

When not to use triggers, functions and procedures

Server-side logic is hidden from the application's code, tests and review: a developer reading a Spring service sees an `UPDATE`, not the five rows the update also writes. It also executes inside the caller's transaction, so its cost, its locks and its failures are added to every statement. Guidance for choosing:

- Use a trigger for invariants that must hold for every writer regardless of application (audit rows, maintaining `updated_at`, enforcing a rule that a CHECK constraint cannot express), keeping the body short and free of external calls.
- Do not put business workflows, network calls, emails or queue publishing in triggers; use an outbox table written by the application in the same transaction and a separate publisher.
- Prefer declarative constraints (NOT NULL, CHECK, UNIQUE, FOREIGN KEY) to triggers wherever they suffice; they are optimised, visible in the schema and cheaper.
- Version and test server-side code like application code: store it in migration files (Flyway) and run it in tests; unversioned functions edited on the production server are a frequent source of unexplained behaviour.
- Procedures with business logic make horizontal scaling and database portability harder, since the database becomes the place where application code runs and where the CPU cost lands.

1.11.6 Worked example: a view, a function and a trigger together

Requirement: keep `emp_audit` complete, expose the payroll of a department through a function, and report salary changes through a view. The sequence below is the exact order of evaluation for a raise of 5 percent to department 30.

1. `UPDATE employees ... WHERE dept_id = 30` finds employees 9 (140000) and 10 (80000).
2. For each row, the new salary is computed (147000 and 84000) and the row version is written (PostgreSQL writes a new tuple version).
3. After each row the AFTER ROW trigger runs, compares `OLD.salary` and `NEW.salary`, and inserts one audit row, so two audit rows are created.
4. The statement ends; the transaction commits and the audit rows commit with it. If the transaction rolled back, the audit rows would disappear, which is the desired consistency but also means that an audit trigger cannot record failed attempts.
5. A view over `emp_audit` (`SELECT emp_id, new_salary - old_salary AS raise FROM emp_audit`) shows the changes; `dept_payroll(30)` returns the new payroll of 231000.

SQL

```sql
SELECT emp_id, new_salary - old_salary AS raise FROM emp_audit ORDER BY emp_id;
SELECT dept_payroll(30) AS payroll_30;
```

Result TERMINAL

```
 emp_id |  raise
--------+---------
      9 | 7000.00
     10 | 4000.00
(2 rows)

 payroll_30
------------
  231000.00
(1 row)
```

1.11.7 Hands-on

The cost of a row-level trigger is the cost of its body once per row. The experiment inserts 100,000 rows into two identical tables, one with an AFTER INSERT row trigger that writes one audit row, and compares the times (measured on the build machine, typical).

SQL

```
CREATE TABLE t_plain (id int, v int);
CREATE TABLE t_trig  (id int, v int);
CREATE TABLE t_log   (id int, v int);
CREATE FUNCTION t_trig_fn() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN INSERT INTO t_log VALUES (NEW.id, NEW.v); RETURN NEW; END $$;
CREATE TRIGGER t_trig_ai AFTER INSERT ON t_trig FOR EACH ROW EXECUTE FUNCTION t_trig_fn();
\timing on
INSERT INTO t_plain SELECT g, g FROM generate_series(1, 100000) g;
INSERT INTO t_trig  SELECT g, g FROM generate_series(1, 100000) g;
\timing off
DROP TABLE t_plain, t_trig, t_log;
DROP FUNCTION t_trig_fn();
```

Result — TERMINAL

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
Timing is on.
INSERT 0 100000
Time: 87.045 ms
INSERT 0 100000
Time: 417.528 ms
Timing is off.
DROP TABLE
DROP FUNCTION
```

The insert with the trigger takes several times longer than the plain insert (67 ms against 429 ms in the run shown), because PL/pgSQL is entered 100,000 times and 100,000 additional rows are written. A set-based statement (`INSERT ... SELECT` into the log table after the main insert, or a statement-level trigger with a transition table) does the same work in one pass.

1.11.8 Real-world problems and solutions

Incident: bulk load becomes several times slower after an audit trigger is added

Scenario. A nightly import loads 2 million rows into `employees`-like tables. After a compliance change added a row-level audit trigger, the import grows from minutes to hours and the transaction log grows by gigabytes.

Symptoms. Import throughput drops; `pg_stat_activity` shows the backend running `INSERT` with CPU saturated; WAL volume and table size of the audit table grow as fast as the main table.

Diagnosis. (1) List the triggers on the table with `\d table` or `SELECT tgname, tgenabled FROM pg_trigger WHERE tgrelid = 'employees'::regclass AND NOT tgisinternal`. (2) Time the load with and without the trigger as in the hands-on experiment: the trigger run is several times slower. (3) Check `pg_stat_user_functions` (requires `track_functions = pl`) for the call count and total time of the trigger function.

Root cause. A row-level trigger executes its function once per row, each time with plan lookup and a separate insert into the audit table. The cost is linear in rows and the constant is large.

Fix. Immediate: disable the trigger for the import session where policy allows (`ALTER TABLE ... DISABLE TRIGGER name`, or `SET session_replication_role = replica` for superusers) and write the audit rows with one set-based `INSERT ... SELECT` after the load. Permanent: change the trigger to statement level with a transition table (`REFERENCING NEW TABLE AS new_rows`) so that a statement writes all audit rows in one INSERT.

Prevention. Measure bulk-load behaviour whenever a trigger is added; document triggers in the schema repository; keep trigger bodies minimal.

Sreeram

Incident: stale management report from a materialised view

Scenario. A dashboard reads `customer_spend`, a materialised view refreshed nightly. At midday the sales team sees a customer's order missing. **Diagnosis.** `SELECT relname, pg_stat_get_last_vacuum_time(oid)` does not show refresh times, since PostgreSQL does not record them; the evidence is the view's content compared with the base query (`SELECT ... FROM order_totals` returns the new row, `SELECT ... FROM customer_spend` does not). **Root cause.** The view content is a snapshot as of the last `REFRESH`. **Fix.** Refresh immediately with `REFRESH MATERIALIZED VIEW CONCURRENTLY` (a unique index exists, so readers are not blocked), and move the schedule to a frequency matching the business need. **Prevention.** Store the refresh time in a one-row table updated by the refresh job and show it on the dashboard; alert when it is older than the agreed limit.

1.11.9 Questions to expect

INTERVIEW ANGLE
What is the difference between UNION and UNION ALL? UNION removes duplicate rows, which requires sorting or hashing the combined result; UNION ALL appends the inputs without any duplicate check and is cheaper. UNION ALL is correct when the inputs cannot overlap or when duplicates are wanted.

INTERVIEW ANGLE
What is a view and does it improve performance? A view is a stored query; the engine substitutes its definition into the calling statement. It stores no data and gives no speed-up. It provides abstraction, a single definition of logic and a security boundary.

INTERVIEW ANGLE
What is a materialised view and when is it appropriate? A view whose result is stored as a table and refreshed on demand. It suits expensive aggregations read often where bounded staleness is acceptable. The costs are refresh time, refresh scheduling and stale reads between refreshes. `REFRESH ... CONCURRENTLY` avoids blocking readers but needs a unique index.

INTERVIEW ANGLE
What is the difference between a function and a stored procedure in PostgreSQL? A function returns a value or a set and is used inside queries; it runs within the caller's transaction. A procedure is called with CALL, returns no value and may commit or roll back transactions, which makes it suitable for batch processing in chunks.

INTERVIEW ANGLE
What are the types of triggers, and what are their drawbacks? BEFORE, AFTER and INSTEAD OF, per row or per statement. They enforce invariants regardless of the client but hide logic, add cost to every write, complicate bulk loads, make debugging harder and run inside the caller's transaction. Declarative constraints and application-level outboxes are preferred where possible.

INTERVIEW ANGLE
What do IMMUTABLE, STABLE and VOLATILE mean for a PostgreSQL function? They declare how the result depends on state: IMMUTABLE never changes for the same arguments, STABLE is constant within a statement, VOLATILE may change at any call. The planner uses the label for caching, constant folding and index usage, so an incorrect label produces wrong results.

INTERVIEW ANGLE
Why is `WITH CHECK OPTION` used on a view? It rejects INSERT or UPDATE through the view that would produce a row the view cannot show, so a view that restricts rows (for example one department) also restricts what can be written through it.

SreGram

KEY POINTS

Set operations, views, routines and triggers

- UNION removes duplicates and costs a sort or hash; UNION ALL does not. INTERSECT and EXCEPT compare whole rows with NULLs equal.
- A view is a stored query expanded into the calling statement; a materialised view is a stored snapshot that must be refreshed on a schedule.
- PostgreSQL functions run inside queries, procedures can commit; volatility labels are contracts with the planner.
- Triggers enforce invariants for every writer but run per row inside the transaction; keep them short and avoid side effects outside the database.
- Prefer declarative constraints; put server-side code under version control and test it.

1.12 NULL, Functions, CASE, Dates and JSON

This chapter collects the everyday building blocks of SQL expressions: the semantics of NULL, which is the source of more incorrect results than any other single feature; string and date functions; conditional expressions; and the querying of JSON documents stored in relational columns. For each topic the PostgreSQL behaviour is shown with real output and the MySQL 8.x equivalent is given in a comparison table.

1.12.1 NULL and three-valued logic

NULL is not a value; it is a marker meaning "unknown or not applicable". Any comparison with NULL yields a third truth value, UNKNOWN. SQL therefore uses three-valued logic: TRUE, FALSE and UNKNOWN. A WHERE clause, a JOIN condition and a HAVING clause keep a row only when the condition is TRUE; a CHECK constraint, in contrast, rejects a row only when the condition is FALSE, so UNKNOWN passes.

AND

AND	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

OR

OR	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

NOT

x	NOT x
T	F
F	T
U	U

T = TRUE, F = FALSE, U = UNKNOWN (orange). WHERE keeps a row only when the condition is T.

Figure 1.49. Truth tables of three-valued logic. NOT UNKNOWN is UNKNOWN; FALSE AND UNKNOWN is FALSE; TRUE OR UNKNOWN is TRUE. Because an UNKNOWN condition does not keep a row, both `x = NULL` and `x <> NULL` match nothing.

SQL

```sql
SELECT NULL = NULL            AS eq,
       NULL IS NULL           AS is_null,
       NULL IS NOT DISTINCT FROM NULL AS nd,
       1 <> NULL              AS neq,
       (NULL AND FALSE)       AS and_f,
       (NULL OR TRUE)         AS or_t,
       coalesce(NULL, 0)      AS coalesced,
       nullif(5, 5)           AS nullif_eq,
       NULL || 'abc'          AS cat,
       5 + NULL               AS arith;
```

Result TERMINAL

```
 eq | is_null | nd | neq | and_f | or_t | coalesced | nullif_eq | cat | arith
----+---------+----+-----+-------+------+-----------+-----------+-----+-------
    | t       | t  |     | f     | t    |         0 |           |     |
(1 row)
```

Empty cells are NULL in `psql` output. The rules that matter in practice:

- Test with `IS NULL` and `IS NOT NULL`, or with `IS [NOT] DISTINCT FROM` for null-safe equality. MySQL offers the operator `<=>` for the same purpose.
- `coalesce(a, b, ...)` returns the first non-NULL argument; `nullif(a, b)` returns NULL when the arguments are equal, which prevents division by zero: `x / nullif(y, 0)`.

- Arithmetic and string concatenation with NULL give NULL. In PostgreSQL `concat()` and `concat_ws()` skip NULL arguments, whereas `||` does not.
- Aggregates skip NULL (see the aggregation chapter); `DISTINCT`, `GROUP BY` and set operations treat NULLs as equal to each other, while `=` does not.
- A UNIQUE constraint admits several NULLs in PostgreSQL and MySQL. PostgreSQL 15 adds `UNIQUE NULLS NOT DISTINCT` to forbid them.
- `NOT IN` with a NULL in the list never succeeds (see the subqueries chapter).
- Sorting places NULLs last in ascending order in PostgreSQL and first in MySQL.

1.12.2 CASE and conditional logic

`CASE` is the SQL expression for branching. The searched form evaluates conditions in order and returns the result of the first true one; the simple form compares one expression with values. A missing ELSE yields NULL. `CASE` can appear in SELECT, WHERE, ORDER BY, GROUP BY and inside aggregates, which makes it the tool for classification, custom ordering and pivoting.

SQL

```sql
SELECT first_name, salary,
       CASE WHEN salary >= 170000 THEN 'senior'
            WHEN salary >= 110000 THEN 'mid'
            ELSE 'junior' END AS band,
       CASE dept_id WHEN 10 THEN 'ENG' WHEN 20 THEN 'SAL' ELSE 'OTH' END AS code
FROM employees
ORDER BY CASE WHEN dept_id IS NULL THEN 0 ELSE 1 END, salary DESC
LIMIT 6;
```

Result TERMINAL

```
 first_name |  salary   |  band  | code
------------+-----------+--------+------
 Tara       | 100000.00 | junior | OTH
 Asha       | 250000.00 | senior | ENG
 Vikram     | 180000.00 | senior | ENG
 Arjun      | 170000.00 | senior | SAL
 Neha       | 160000.00 | mid    | OTH
 Ravi       | 150000.00 | mid    | ENG
(6 rows)
```

Conditional aggregation pivots rows into columns: each `CASE` selects the rows that contribute to one output column.

SQL

```sql
SELECT dept_id,
       count(*) FILTER (WHERE salary >= 110000) AS high,
       count(*) FILTER (WHERE salary <  110000) AS low,
       sum(CASE WHEN hire_date >= '2022-01-01' THEN 1 ELSE 0 END) AS recent_hires
FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id ORDER BY dept_id;
```

Result TERMINAL

```
 dept_id | high | low | recent_hires
---------+------+-----+--------------
      10 |    5 |   0 |            0
      20 |    3 |   1 |            2
      30 |    1 |   1 |            1
      40 |    1 |   1 |            1
(4 rows)
```

The portable CASE-inside-aggregate form is the one that also works in MySQL. In a `CASE`, the branches must have compatible types, and the conditions are evaluated lazily in order, but the PostgreSQL planner may evaluate constant subexpressions early; relying on `CASE` to prevent a division by zero in a branch is safe only for non-constant operands.

1.12.3 String functions

Task	PostgreSQL 16	MySQL 8.x
Concatenate	`a || b`, `concat(a, b)`, `concat_ws(',', a, b)`	`CONCAT(a, b)`, `CONCAT_WS(',', a, b)`
Length in characters	`length(s)`, `char_length(s)`	`CHAR_LENGTH(s)` (`LENGTH` counts bytes)
Substring	`substr(s, 2, 3)`, `substring(s from 2 for 3)`	`SUBSTRING(s, 2, 3)`
Position	`strpos(s, 'x')`, `position('x' in s)`	`LOCATE('x', s)`, `INSTR(s, 'x')`
Split part	`split_part(s, ',', 2)`	`SUBSTRING_INDEX(SUBSTRING_INDEX(s, ',', 2), ',', -1)`
Trim and pad	`trim(s)`, `lpad(s, 5, '0')`	`TRIM(s)`, `LPAD(s, 5, '0')`
Case-insensitive match	`ILIKE`, `lower(a) = lower(b)`	Default collations are case-insensitive: `a = b`, `LIKE`
Regular expression	`s ~ '^R'`, `regexp_replace(s, 'a', 'b')`	`s REGEXP '^R'`, `REGEXP_REPLACE(s, 'a', 'b')`
Format	`format('%s-%s', a, b)`, `to_char(n, 'FM0000')`	`FORMAT(n, 2)`, `LPAD`, `DATE_FORMAT`

Frequently used string functions.

SQL

```sql
SELECT upper(last_name)                          AS up,
       length(first_name)                        AS len,
       substr(first_name, 1, 3) || '...'         AS short,
       lpad(emp_id::text, 5, '0')                AS code,
       split_part('a,b,c', ',', 2)               AS second,
       regexp_replace('Hyd-500001', '[^0-9]', '', 'g') AS digits,
       'Kumar' ILIKE 'k%'                        AS ilike_match
FROM employees WHERE emp_id = 4;
```

Result TERMINAL

```
  up   | len | short  | code  | second | digits | ilike_match
-------+-----+--------+-------+--------+--------+-------------
 KUMAR |   4 | Rav... | 00004 | b      | 500001 | t
(1 row)
```

String comparison depends on collation. In PostgreSQL the default collation comes from the database (often `en_US.UTF-8`, where `'a' < 'B'`), while the `C` collation compares bytes (`'B' < 'a'`). MySQL's default collation `utf8mb4_0900_ai_ci` (8.0) is accent-insensitive and case-insensitive, so `'a' = 'A'` is true and a UNIQUE index on such a column rejects `'abc'` after `'ABC'`. Both facts matter when migrating data between the engines and when an application assumes case-sensitive keys.

1.12.4 Date and time functions

Date handling is a major source of defects; the types themselves are treated in the schema-design chapter. The operations below use `timestamptz` columns.

Task	PostgreSQL 16	MySQL 8.x
Current time	`now()`, `current_date`, `current_timestamp`	`NOW()`, `CURDATE()`
Add an interval	`ts + interval '7 days'`	`DATE_ADD(ts, INTERVAL 7 DAY)`
Difference	`d2 - d1` (days, integer), `age(ts2, ts1)`	`DATEDIFF(d2, d1)`, `TIMESTAMPDIFF(HOUR, a, b)`
Truncate	`date_trunc('month', ts)`	`DATE_FORMAT(ts, '%Y-%m-01')` or `DATE(ts)`
Extract a part	`extract(year from ts)`, `extract(isodow from ts)`	`EXTRACT(YEAR FROM ts)`, `DAYOFWEEK(ts)`

Task	PostgreSQL 16	MySQL 8.x
Format	`to_char(ts, 'YYYY-MM-DD HH24:MI')`	`DATE_FORMAT(ts, '%Y-%m-%d %H:%i')`
Time zone conversion	`ts AT TIME ZONE 'Asia/Kolkata'`	`CONVERT_TZ(ts, 'UTC', 'Asia/Kolkata')` (needs time zone tables)
Series of dates	`generate_series(d1, d2, interval '1 day')`	Recursive CTE

Date and time operations.

SQL

```sql
SELECT date_trunc('week', order_ts)::date AS week_start,
       count(*) AS orders,
       min(order_ts)::date AS first_day, max(order_ts)::date AS last_day
FROM orders GROUP BY 1 ORDER BY 1;

SELECT d::date AS day, count(o.order_id) AS orders
FROM generate_series('2025-01-03'::date, '2025-01-10'::date, interval '1 day') d
LEFT JOIN orders o ON o.order_ts::date = d::date
GROUP BY d ORDER BY d;
```

Result TERMINAL

```
 week_start | orders | first_day  |  last_day
------------+--------+------------+------------
 2024-12-30 |      5 | 2025-01-03 | 2025-01-05
 2025-01-06 |      5 | 2025-01-06 | 2025-01-10
(2 rows)

    day     | orders
------------+--------
 2025-01-03 |      1
 2025-01-04 |      2
 2025-01-05 |      2
 2025-01-06 |      1
 2025-01-07 |      1
 2025-01-08 |      1
 2025-01-09 |      1
 2025-01-10 |      1
(8 rows)
```

The second query shows the calendar-table pattern: `generate_series` produces every day, and a left join keeps days without orders (every day from January 3 to 10 has at least one order in the sample data; a day without orders would appear with a count of 0). A report that groups only the rows that exist silently omits empty days.

1.12.5 JSON in relational databases

PostgreSQL stores JSON as `jsonb` (a decomposed binary format that supports indexing, with key order and duplicate keys not preserved) or as `json` (the original text). MySQL has a single `JSON` type, stored in a binary format as well. JSON columns are useful for attributes that vary by row (the specification of a product) and are not a replacement for relational columns that are joined, filtered or constrained.

Task	PostgreSQL 16 (jsonb)	MySQL 8.x
Field as JSON	`specs -> 'brand'`	`specs -> '$.brand'`
Field as text	`specs ->> 'brand'`	`specs ->> '$.brand'` or `JSON_UNQUOTE(JSON_EXTRACT(...))`
Nested path	`specs #>> '{cpu,model}'`	`specs ->> '$.cpu.model'`
Containment	`specs @> '{"panel":"IPS"}'`	`JSON_CONTAINS(specs, '"IPS"', '$.panel')`
Key exists	`specs ? 'wireless'`	`JSON_CONTAINS_PATH(specs, 'one', '$.wireless')`
Array elements as rows	`jsonb_array_elements_text(specs -> 'tags')`	`JSON_TABLE(specs, '$.tags[*]' COLUMNS (t VARCHAR(20) PATH '$'))`

Task	PostgreSQL 16 (jsonb)	MySQL 8.x
Index	GIN on the column (`@>`, `?`) or B-tree on an expression	Functional index or generated column index

JSON access operations.

SQL

```sql
SELECT p.name,
       s.specs ->> 'brand'               AS brand,
       (s.specs ->> 'ram_gb')::int       AS ram_gb,
       s.specs #>> '{cpu,model}'         AS cpu_model,
       s.specs -> 'tags'                 AS tags
FROM products p JOIN product_specs s USING (product_id)
ORDER BY p.product_id;

SELECT p.name FROM products p JOIN product_specs s USING (product_id)
WHERE s.specs @> '{"tags": ["usb"]}' ORDER BY 1;

SELECT t.tag, count(*) AS products
FROM product_specs s, jsonb_array_elements_text(s.specs -> 'tags') AS t(tag)
GROUP BY t.tag ORDER BY products DESC, t.tag;
```

Result — TERMINAL

```
   name   |  brand   | ram_gb | cpu_model |         tags
----------+----------+--------+-----------+----------------------
 Keyboard | KeyCo    |        |           | ["office", "usb"]
 Monitor  | ViewMax  |        |           | ["office", "4k"]
 Laptop   | CompuOne |     16 | X8        | ["portable", "work"]
 Webcam   | SeeIt    |        |           | ["usb", "video"]
(4 rows)

   name
----------
 Keyboard
 Webcam
(2 rows)

   tag    | products
----------+----------
 office   |        2
 usb      |        2
 4k       |        1
 portable |        1
 video    |        1
 work     |        1
(6 rows)
```

A missing key returns NULL, not an error, which makes typos silent: `specs ->> 'brnad'` is NULL for every row. For queries that filter on a JSON field often, an index is required; the index chapters describe GIN indexes, and an expression index `CREATE INDEX ON product_specs ((specs ->> 'brand'))` serves equality on one key. In Java, the `jsonb` column maps to a `String` or to a type provided by a library (for Hibernate 6, `@JdbcTypeCode(SqlTypes.JSON)`).

1.12.6 PostgreSQL and MySQL: a difference table

The engines share most of the standard but differ in behaviours that cause defects during porting or when a developer moves between them.

Area	PostgreSQL 16	MySQL 8.x (InnoDB)
Identifier quoting	Double quotes `"name"`; unquoted names fold to lower case	Backticks; case sensitivity of table names depends on the OS
String concatenation	`||`	`CONCAT()`; `||` is OR unless `PIPES_AS_CONCAT`
Auto-generated keys	`GENERATED ... AS IDENTITY`, `serial`; sequences	`AUTO_INCREMENT`
LIMIT syntax	`LIMIT n OFFSET m`, `FETCH FIRST`	`LIMIT n OFFSET m`, `LIMIT m, n`
Upsert	`ON CONFLICT ... DO UPDATE`, `MERGE` (15+)	`ON DUPLICATE KEY UPDATE`
RETURNING	Yes	No

Area	PostgreSQL 16	MySQL 8.x (InnoDB)
FULL OUTER JOIN	Yes	No
INTERSECT and EXCEPT	Yes	Since 8.0.31
CTE and window functions	Yes, since 8.4	Yes, since 8.0
LATERAL	Yes, since 9.3	Since 8.0.14
FILTER on aggregates	Yes	No (use CASE)
DISTINCT ON	Yes	No
Boolean type	`boolean` (true, false)	`TINYINT(1)` alias; `BOOL`
Case sensitivity of = on text	Case-sensitive by default collations	Case-insensitive by default collations
Integer division	`7 / 2 = 3`	`7 / 2 = 3.5000`, `DIV` for integers
Implicit casts	Strict: `'abc' = 1` is an error	Lenient: `'abc' = 0` is true with a warning
GROUP BY non-grouped columns	Error (except functional dependence)	Error with ONLY_FULL_GROUP_BY (default)
Invalid data in INSERT	Error	Error in strict mode (default in 8.0); silently truncated when disabled
DDL in transactions	Transactional	Implicit commit
Default isolation level	READ COMMITTED	REPEATABLE READ
Default string encoding	Database encoding, UTF8 usual	`utf8mb4` (8.0)
JSON type	`json`, `jsonb`	`JSON`
Materialised views	Yes	No
Partial and expression indexes	Yes	Functional indexes (8.0.13+), no partial indexes
Statement-level triggers	Yes	No (row-level only)

PostgreSQL 16 and MySQL 8.x differences relevant to SQL code.

Implicit casts and silent truncation produce the most expensive surprises when moving code from MySQL to PostgreSQL: statements that worked with `WHERE varchar_col = 123` or with an over-long string now fail, and the failure is the correct behaviour. In the opposite direction, PostgreSQL code that relies on `RETURNING`, `DISTINCT ON`, `FILTER` or `FULL JOIN` needs rewriting.

1.12.7 Worked example: classifying and bucketing with CASE, dates and NULL

Requirement: a monthly report of order counts and value by status class (open, closed) including a "no date" bucket for rows with a missing timestamp. The statement combines `CASE`, `date_trunc` and `coalesce`.

SQL

```sql
SELECT coalesce(to_char(date_trunc('month', o.order_ts), 'YYYY-MM'), 'no date') AS month,
       CASE WHEN o.status IN ('NEW','PAID') THEN 'open' ELSE 'closed' END AS state,
       count(DISTINCT o.order_id) AS orders,
       sum(oi.qty * oi.unit_price) AS value
FROM orders o JOIN order_items oi USING (order_id)
GROUP BY 1, 2
ORDER BY 1, 2;
```

Result TERMINAL

```
  month   | state  | orders |   value
----------+--------+--------+-----------
 2025-01 | closed |      3 | 113200.00
 2025-01 | open   |      7 |  61000.00
(2 rows)
```

Trace: CASE maps NEW and PAID (orders 101, 103, 105, 106, 108, 109, 110) to open and SHIPPED and CANCELLED (102, 104, 107) to closed; `date_trunc` maps every January 2025 timestamp to `2025-01-01 00:00`, and `to_char` formats it; `coalesce` would substitute the label only for NULL timestamps, of which the sample data has none. `count(DISTINCT order_id)` is needed because the join to order lines repeats each order once per line.

1.12.8 Hands-on

NULL-handling and date logic are easily tested in `psql` before they enter application code. A useful habit is to test every report query with a row that has NULL in each nullable column and with timestamps just before and after a day boundary; those two data sets expose most expression defects.

SQL

```sql
SELECT 'x' AS v, NULL::int AS n
UNION ALL SELECT 'y', 3
UNION ALL SELECT 'z', NULL;

WITH t(v, n) AS (VALUES ('x', NULL::int), ('y', 3), ('z', NULL))
SELECT count(*) AS rows, count(n) AS non_null, sum(n) AS total, avg(n) AS mean,
       count(*) FILTER (WHERE n IS NULL) AS nulls,
       count(*) FILTER (WHERE n <> 3) AS not_three,
       count(*) FILTER (WHERE n IS DISTINCT FROM 3) AS distinct_from_three
FROM t;
```

Result TERMINAL

```
 v | n
---+---
 x |
 y | 3
 z |
(3 rows)

 rows | non_null | total |        mean        | nulls | not_three | distinct_from_three
------+----------+-------+--------------------+-------+-----------+---------------------
    3 |        1 |     3 | 3.0000000000000000 |     2 |         0 |                   2
(1 row)
```

`n <> 3` matches no row: the row with n = 3 is FALSE and the NULL rows are UNKNOWN. `IS DISTINCT FROM 3` matches the two NULL rows, which is usually what "not equal to 3" is meant to include.

1.12.9 Real-world problems and solutions

Incident: daily report is off by a day for evening orders

Scenario. A business in India (UTC+5:30) reports orders per day. The report uses `order_ts::date` on a `timestamptz` column. Orders placed late in the evening, local time, are counted on the following day, and daily totals differ between the server report and a report built in the application.

Symptoms. Counts per day disagree with the sales team's records by one or two orders; the disagreement is largest near the end of the local day; the same query gives different results when run from `psql` on the server and from a tool configured with a different time zone.

Diagnosis. Step 1: compare the daily counts under two session time zones.

SQL

```sql
SET timezone = 'UTC';
SELECT order_ts::date AS day, count(*) AS orders FROM orders GROUP BY 1 ORDER BY 1;

SET timezone = 'Asia/Kolkata';
SELECT order_ts::date AS day, count(*) AS orders FROM orders GROUP BY 1 ORDER BY 1;
```

Result TERMINAL

```
SET
    day     | orders
------------+--------
 2025-01-03 |      1
 2025-01-04 |      2
 2025-01-05 |      2
 2025-01-06 |      1
 2025-01-07 |      1
 2025-01-08 |      1
 2025-01-09 |      1
 2025-01-10 |      1
(8 rows)

SET
```

Result (continued) TERMINAL

```
    day     | orders
------------+--------
 2025-01-03 |      1
 2025-01-04 |      1
 2025-01-05 |      2
 2025-01-06 |      2
 2025-01-07 |      1
 2025-01-09 |      2
 2025-01-11 |      1
(7 rows)
```

The two reports assign different orders to different days. Step 2: list the orders whose date changes.

SQL

```sql
SELECT order_id,
       order_ts AT TIME ZONE 'UTC'           AS utc_clock,
       order_ts AT TIME ZONE 'Asia/Kolkata'  AS ist_clock,
       (order_ts AT TIME ZONE 'UTC')::date
         <> (order_ts AT TIME ZONE 'Asia/Kolkata')::date AS shifts_day
FROM orders ORDER BY order_id;
```

Result TERMINAL

```
 order_id |      utc_clock      |      ist_clock      | shifts_day
----------+---------------------+---------------------+------------
      101 | 2025-01-03 10:00:00 | 2025-01-03 15:30:00 | f
      102 | 2025-01-04 11:30:00 | 2025-01-04 17:00:00 | f
      103 | 2025-01-04 18:45:00 | 2025-01-05 00:15:00 | t
      104 | 2025-01-05 09:10:00 | 2025-01-05 14:40:00 | f
      105 | 2025-01-05 20:00:00 | 2025-01-06 01:30:00 | t
      106 | 2025-01-06 13:25:00 | 2025-01-06 18:55:00 | f
      107 | 2025-01-07 05:55:00 | 2025-01-07 11:25:00 | f
      108 | 2025-01-08 22:40:00 | 2025-01-09 04:10:00 | t
      109 | 2025-01-09 14:00:00 | 2025-01-09 19:30:00 | f
      110 | 2025-01-10 23:30:00 | 2025-01-11 05:00:00 | t
(10 rows)
```

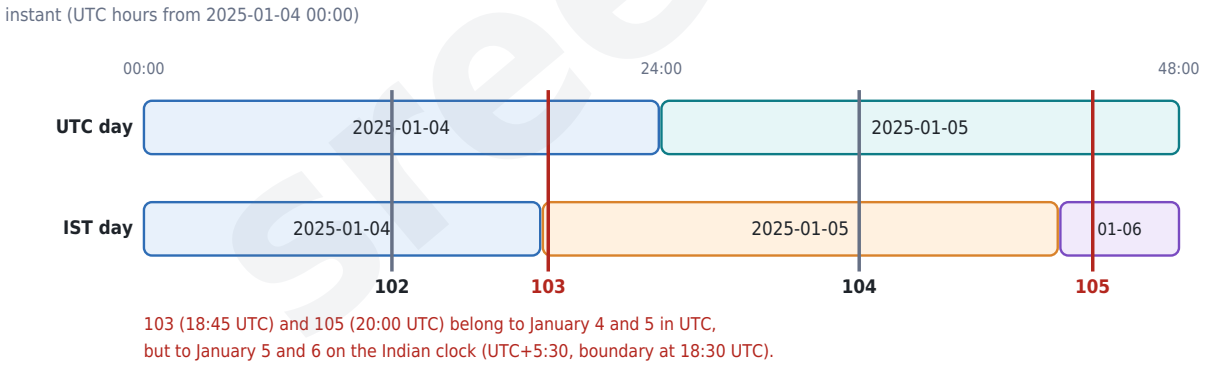


Figure 1.50. One day boundary on two clocks. An order at 18:45 UTC on January 4 is 00:15 on January 5 in India. A cast to `date` uses the session time zone, so UTC and IST sessions put the order on different days; orders 103, 105, 108 and 110 cross the boundary.

Root cause. A `timestamptz` holds one instant (stored as UTC). Converting it to a calendar date requires a time zone; `::date` silently uses the session's `TimeZone` setting, which comes from the server configuration, the role, the connection pool or the JDBC driver (the PostgreSQL JDBC driver sets the session zone from the JVM default). Two components with different settings compute different dates from the same instant.

Fix. Immediate: state the business time zone in the query: `(order_ts AT TIME ZONE 'Asia/Kolkata')::date` or `date_trunc('day', order_ts AT TIME ZONE 'Asia/Kolkata')`. Permanent: store instants in `timestamptz` (MySQL: `TIMESTAMP`, with the session `time_zone` set deliberately, or `DATETIME` holding UTC by convention), set the JVM and the pool to UTC (`-Duser.timezone=UTC`; Hibernate property `hibernate.jdbc.time_zone=UTC`), map to `Instant` or `OffsetDateTime` in Java, and convert to the business zone only at the edges, in queries or in presentation. For frequently queried business dates, store a derived `business_date date` column computed once at write time.
Prevention. Tests with timestamps at 23:59 and 00:01 local time, executed under a non-UTC session zone; a database default `ALTER DATABASE ... SET timezone = 'UTC'` so that the server configuration is explicit.

Incident: rows missing from a filter on a nullable column

Scenario. A job selects customers to mail: `WHERE city <> 'Hyderabad'`. The customer with no recorded city is never mailed. **Diagnosis.** `SELECT count(*) FROM customers WHERE city <> 'Hyderabad'` returns 3 although 4 customers are outside Hyderabad: the row with `city` NULL evaluates to UNKNOWN. **Fix.** `WHERE city IS DISTINCT FROM 'Hyderabad'`, or `coalesce(city, '') <> 'Hyderabad'` where an empty default is acceptable. **Prevention.** Define for each nullable column what its absence means; prefer NOT NULL with a real default where possible.

SQL

```sql
SELECT count(*) FILTER (WHERE city <> 'Hyderabad')                AS neq,
       count(*) FILTER (WHERE city IS DISTINCT FROM 'Hyderabad')  AS distinct_from
FROM customers;
```

Result TERMINAL

```
 neq | distinct_from
-----+---------------
   3 |             4
(1 row)
```

1.12.10 Questions to expect

INTERVIEW ANGLE
What is NULL, and why does `WHERE col = NULL` return no rows? NULL means unknown or inapplicable. Comparing anything with NULL, including NULL itself, yields UNKNOWN, and WHERE keeps only rows whose condition is TRUE. The correct tests are `IS NULL`, `IS NOT NULL` and `IS DISTINCT FROM`.

INTERVIEW ANGLE
How do COALESCE and NULLIF differ? COALESCE returns the first non-NULL argument and supplies defaults. NULLIF returns NULL when its two arguments are equal and the first argument otherwise; its common use is turning a sentinel such as 0 into NULL to avoid division by zero.

INTERVIEW ANGLE
What is the difference between timestamp with time zone and without, and how does a date boundary bug arise? `timestamptz` stores an instant and converts it to the session zone on output; `timestamp` stores a wall-clock reading with no zone. Casting a `timestamptz` to `date` uses the session time zone, so components with different zone settings assign the same instant to different days. The business zone must be stated explicitly in the query.

INTERVIEW ANGLE
How is a pivot (rows to columns) written in SQL? With conditional aggregation: one aggregate per output column, each restricted by CASE or FILTER, grouped by the row key: `sum(CASE WHEN status = 'PAID' THEN 1 ELSE 0 END)`. PostgreSQL's `crosstab` extension function and SQL Server's PIVOT are alternatives.

INTERVIEW ANGLE
When is a JSON column appropriate in a relational schema? For attributes that vary per row and are read as a whole or filtered rarely (product specifications, settings, event payloads). Data that is joined, constrained or filtered frequently belongs in columns. `jsonb` with a GIN index supports containment queries in PostgreSQL.

INTERVIEW ANGLE
Name differences between PostgreSQL and MySQL that affect application SQL. Case-sensitivity of string comparison, implicit casts and silent truncation in MySQL, `||` semantics, RETURNING, FULL JOIN, DISTINCT ON and FILTER in PostgreSQL only, transactional DDL, default isolation level (READ COMMITTED versus REPEATABLE READ) and integer division.

INTERVIEW ANGLE
Does a UNIQUE constraint allow several NULLs? Yes, in PostgreSQL and MySQL, because NULLs are not equal to each other. PostgreSQL 15 adds `NULLS NOT DISTINCT` to treat them as equal. SQL Server allows only one NULL.

KEY POINTS

NULL, functions, CASE, dates and JSON

- NULL gives three-valued logic; use IS NULL, IS DISTINCT FROM, COALESCE and NULLIF deliberately.
- CASE with aggregates (or FILTER) pivots and classifies in one pass; missing ELSE yields NULL.
- A cast from `timestamptz` to `date` depends on the session zone; always state the business zone and keep servers, pools and JVMs on UTC.
- `jsonb` operators (`->`, `->>`, `@>`, `?`) query documents; index the keys that are filtered.
- Collation, implicit casts, concatenation and integer division are the main PostgreSQL and MySQL traps.

1.13 Thirty Classic SQL Interview Problems

The problems below recur, in one form or another, in interviews for backend and data roles. Each is stated, solved on the sample schema and verified by running it on PostgreSQL 16; the output shown is the real output. Where a MySQL 8 solution differs materially, the difference is stated. A reader should attempt each problem before reading the solution, since the patterns (anti-join, top-N per group, running aggregates, gaps and islands, recursion, conditional aggregation) are what the interviewer looks for. Tables used besides the sample schema: `dup_emails`, `user_logins`, `seq_ids`, `daily_sales`, `month_sales`, `product_sales`, `bookings` and `seats`, each shown with its problem.

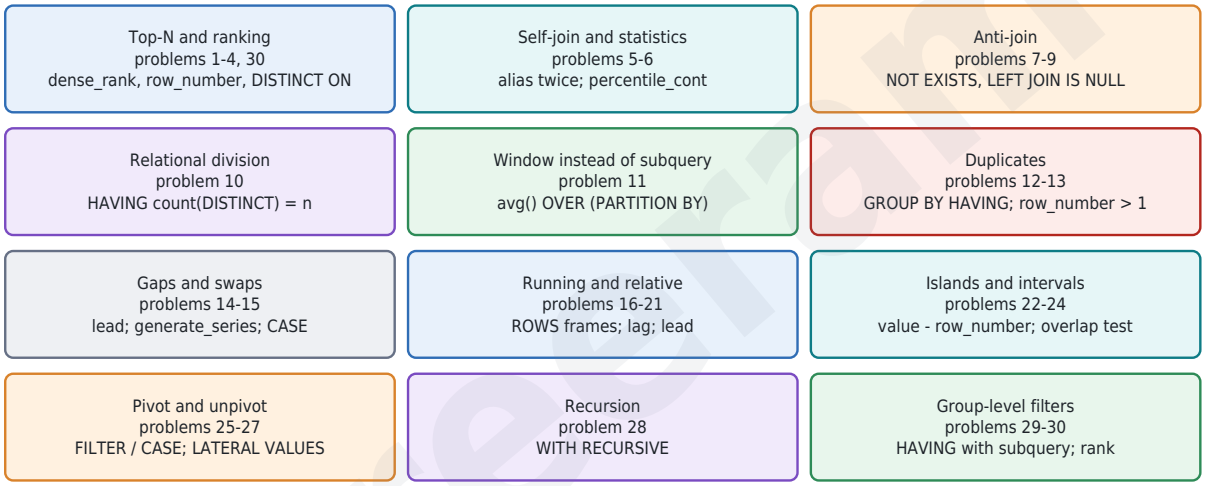


Figure 1.51. The patterns behind the thirty problems. Recognising the pattern is most of the solution; the problems that use each pattern are listed with it.

1.13.1 Ranking and top-N problems

Problem 1: second highest salary

Return the second highest distinct salary. The answer must be NULL (not an error or an empty set that crashes the caller) when it does not exist. A scalar subquery wrapped around `OFFSET` supplies the NULL.

SQL

```sql
SELECT (SELECT DISTINCT salary FROM employees
        ORDER BY salary DESC OFFSET 1 LIMIT 1) AS second_highest;

SELECT max(salary) AS second_highest
FROM employees WHERE salary < (SELECT max(salary) FROM employees);
```

Result TERMINAL

```
 second_highest
----------------
      180000.00
(1 row)

 second_highest
----------------
      180000.00
(1 row)
```

The second form uses only `max` and an uncorrelated subquery, works in every engine, and returns NULL on its own when there is no second salary.

Problem 2: Nth highest salary with ties handled

Return the 3rd highest distinct salary. `dense_rank()` numbers distinct values without gaps, so the Nth rank is the Nth highest distinct value.

SQL

```sql
SELECT DISTINCT salary AS third_highest
FROM (SELECT salary, dense_rank() OVER (ORDER BY salary DESC) AS r FROM employees) t
WHERE r = 3;
```

Result TERMINAL

```
 third_highest
---------------
     170000.00
(1 row)
```

Problem 3: highest paid employee in each department

Return the name and salary of the best paid employee of every department. `DISTINCT ON` is the PostgreSQL idiom; the portable form joins to a per-department maximum, and returns every tied employee.

SQL

```sql
SELECT d.dept_name, e.first_name, e.salary
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE e.salary = (SELECT max(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY d.dept_name;
```

Result TERMINAL

```
  dept_name  | first_name |  salary
-------------+------------+-----------
 Engineering | Asha       | 250000.00
 Finance     | Neha       | 160000.00
 HR          | Divya      | 140000.00
 Sales       | Arjun      | 170000.00
(4 rows)
```

Problem 4: top two salaries in each department

Use `row_number()` for exactly two rows per department, or `dense_rank() <= 2` for the two highest distinct salaries including ties.

SQL

```sql
SELECT dept_id, first_name, salary
FROM (SELECT dept_id, first_name, salary,
             dense_rank() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
      FROM employees WHERE dept_id IS NOT NULL) t
WHERE r <= 2
ORDER BY dept_id, salary DESC, first_name;
```

Result TERMINAL

```
 dept_id | first_name |  salary
---------+------------+-----------
      10 | Asha       | 250000.00
      10 | Vikram     | 180000.00
      20 | Arjun      | 170000.00
      20 | Omar       | 110000.00
      20 | Priya      | 110000.00
      30 | Divya      | 140000.00
      30 | Rahul      |  80000.00
      40 | Neha       | 160000.00
      40 | Sanjay     |  90000.00
(9 rows)
```

Sales returns three employees, because Priya and Omar share the second distinct salary, 110000. With `row_number()` the result would have exactly two per department and an arbitrary choice between the tied employees.

Sriram

Problem 5: employees earning more than their manager

A self-join compares the employee row with the manager row.

SQL

```sql
SELECT e.first_name AS employee, e.salary, m.first_name AS manager, m.salary AS manager_salary
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

Result TERMINAL

```
 employee | salary | manager | manager_salary
----------+--------+---------+----------------
(0 rows)
```

No row is returned on the sample data, which is itself the correct answer. To test a solution, change a row (`UPDATE ... SET salary = 190000 WHERE emp_id = 3` makes Meera earn more than her manager Vikram) and run the query again.

Problem 6: median salary

PostgreSQL provides ordered-set aggregates; MySQL has no median function and uses a window function with row counts.

SQL

```sql
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median,
       avg(salary)::numeric(10,0)                          AS mean
FROM employees;
```

Result TERMINAL

```
 median |  mean
--------+--------
 130000 | 136071
(1 row)
```

The median (average of the two middle values for an even count) is 130000 against a mean of 136071; the difference shows the skew caused by the highest salary.

1.13.2 Anti-join, semi-join and relational division

Problem 7: departments with no employees

SQL

```sql
SELECT d.dept_name FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Result TERMINAL

```
 dept_name
-----------
 Legal
(1 row)
```

Problem 8: employees without a department

SQL

```sql
SELECT emp_id, first_name FROM employees WHERE dept_id IS NULL;
```

Result TERMINAL

```
 emp_id | first_name
--------+------------
     13 | Tara
(1 row)
```

sreeram

Problem 9: customers who never ordered

```sql
SELECT c.customer_id, c.name
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL ORDER BY 1;
```

Result

```
 customer_id |      name
-------------+----------------
           4 | Deepak & Sons
           6 | Farah Fashions
(2 rows)
```

Problem 10: customers who bought both a keyboard and a webcam

This is relational division: customers related to every member of a set. Count the distinct required products per customer and compare with the size of the set.

```sql
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p     ON p.product_id = oi.product_id
WHERE p.name IN ('Keyboard', 'Webcam') AND o.status <> 'CANCELLED'
GROUP BY o.customer_id
HAVING count(DISTINCT p.name) = 2;
```

Result

```
 customer_id
-------------
           1
(1 row)
```

Customer 2 also ordered a webcam, but in the cancelled order 104; excluding cancelled orders leaves only customer 1.

Problem 11: orders above the customer's own average

A window function replaces the correlated subquery.

```sql
SELECT order_id, customer_id, amount, round(avg_amt, 2) AS customer_avg
FROM (SELECT order_id, customer_id, amount,
             avg(amount) OVER (PARTITION BY customer_id) AS avg_amt
      FROM order_totals) t
WHERE amount > avg_amt ORDER BY order_id;
```

Result

```
 order_id | customer_id |  amount  | customer_avg
----------+-------------+----------+--------------
      102 |           1 | 55000.00 |     25933.33
      103 |           2 | 25250.00 |     10483.33
      107 |           5 | 55000.00 |     28750.00
      110 |           3 |  6000.00 |      3725.00
(4 rows)
```

1.13.3 Duplicates and data cleaning

Problem 12: find duplicate values

Table `dup_emails` holds `(1,a@x.com) (2,b@x.com) (3,a@x.com) (4,c@x.com) (5,b@x.com) (6,a@x.com)`.

```sql
SELECT email, count(*) AS copies, array_agg(id ORDER BY id) AS ids
FROM dup_emails GROUP BY email HAVING count(*) > 1 ORDER BY email;
```

Result TERMINAL

```
  email  | copies |   ids
---------+--------+---------
 a@x.com |      3 | {1,3,6}
 b@x.com |      2 | {2,5}
(2 rows)
```

Problem 13: delete duplicates, keeping the lowest id

The `row_number()` form numbers the copies of each value; every row numbered above 1 is deleted. The statement is shown inside a rolled-back transaction.

SQL

```sql
DELETE FROM dup_emails
WHERE id IN (SELECT id FROM (SELECT id, row_number() OVER (PARTITION BY email ORDER BY id) AS rn
                             FROM dup_emails) t WHERE rn > 1);
SELECT * FROM dup_emails ORDER BY id;
```

Result TERMINAL

```
DELETE 3
 id |  email
----+---------
  1 | a@x.com
  2 | b@x.com
  4 | c@x.com
(3 rows)
```

An equivalent self-join form is `DELETE FROM dup_emails a USING dup_emails b WHERE a.email = b.email AND a.id > b.id`. MySQL cannot delete from a table that the subquery reads in the same statement without wrapping it in a derived table (error 1093); it uses `DELETE a FROM dup_emails a JOIN dup_emails b ON ... AND a.id > b.id`. The lasting fix is a unique constraint: `ALTER TABLE dup_emails ADD CONSTRAINT uq_email UNIQUE (email)` after the clean-up.

Problem 14: swap adjacent seat ids

Table `seats` holds students with ids 1 to 5. Swap the seats of consecutive pairs (1 with 2, 3 with 4); an odd last student stays.

SQL

```sql
SELECT CASE WHEN id % 2 = 1 AND id = (SELECT max(id) FROM seats) THEN id
            WHEN id % 2 = 1 THEN id + 1
            ELSE id - 1 END AS id,
       student
FROM seats ORDER BY 1;
```

Result TERMINAL

```
 id | student
----+---------
  1 | Doris
  2 | Abbot
  3 | Green
  4 | Emerson
  5 | Jeames
(5 rows)
```

Problem 15: find missing ids in a sequence

Table `seq_ids` holds 1, 2, 3, 5, 6, 9, 10. Two solutions: compare each id with the next one using `lead`, or generate the full series and anti-join.

SQL

```sql
SELECT id + 1 AS gap_start, next_id - 1 AS gap_end
FROM (SELECT id, lead(id) OVER (ORDER BY id) AS next_id FROM seq_ids) t
WHERE next_id - id > 1;

SELECT g AS missing_id
FROM generate_series((SELECT min(id) FROM seq_ids), (SELECT max(id) FROM seq_ids)) g
WHERE NOT EXISTS (SELECT 1 FROM seq_ids s WHERE s.id = g);
```

sreram

Result TERMINAL

```
 gap_start | gap_end
-----------+---------
         4 |       4
         7 |       8
(2 rows)

 missing_id
------------
          4
          7
          8
(3 rows)
```

1.13.4 Running totals, comparisons and time series

Table `daily_sales(day, amount)` holds seven days of February 2025 with amounts 100, 120, 90, 150, 130, 170, 160; `month_sales` holds four monthly totals.

Problem 16: running total

SQL

```sql
SELECT day, amount, sum(amount) OVER (ORDER BY day ROWS UNBOUNDED PRECEDING) AS running
FROM daily_sales ORDER BY day;
```

Result TERMINAL

```
    day     | amount | running
------------+--------+---------
 2025-02-01 |    100 |     100
 2025-02-02 |    120 |     220
 2025-02-03 |     90 |     310
 2025-02-04 |    150 |     460
 2025-02-05 |    130 |     590
 2025-02-06 |    170 |     760
 2025-02-07 |    160 |     920
(7 rows)
```

Problem 17: three-day moving average

SQL

```sql
SELECT day, amount,
       round(avg(amount) OVER (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS avg3,
       count(*) OVER (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS window_rows
FROM daily_sales ORDER BY day;
```

Result TERMINAL

```
    day     | amount | avg3  | window_rows
------------+--------+-------+-------------
 2025-02-01 |    100 | 100.0 |           1
 2025-02-02 |    120 | 110.0 |           2
 2025-02-03 |     90 | 103.3 |           3
 2025-02-04 |    150 | 120.0 |           3
 2025-02-05 |    130 | 123.3 |           3
 2025-02-06 |    170 | 150.0 |           3
 2025-02-07 |    160 | 153.3 |           3
(7 rows)
```

The first two rows average fewer than three values (`window_rows` shows 1 and 2); a report that requires full windows filters on `window_rows = 3`.

Problem 18: month-over-month growth

SQL

```sql
SELECT to_char(month, 'YYYY-MM') AS month, amount,
       lag(amount) OVER (ORDER BY month) AS previous,
       round(100.0 * (amount - lag(amount) OVER (ORDER BY month))
             / nullif(lag(amount) OVER (ORDER BY month), 0), 1) AS growth_pct
FROM month_sales ORDER BY month;
```

Result TERMINAL

```
  month  | amount | previous | growth_pct
---------+--------+----------+------------
 2025-01 |   1000 |          |
 2025-02 |   1200 |     1000 |       20.0
 2025-03 |    900 |     1200 |      -25.0
 2025-04 |   1500 |      900 |       66.7
(4 rows)
```

Problem 19: days with sales above the previous day

SQL

```sql
SELECT day, amount
FROM (SELECT day, amount, lag(amount) OVER (ORDER BY day) AS prev
      FROM daily_sales) t
WHERE amount > prev ORDER BY day;
```

Result TERMINAL

```
    day     | amount
------------+--------
 2025-02-02 |    120
 2025-02-04 |    150
 2025-02-06 |    170
(3 rows)
```

Problem 20: first and last order of each customer

`DISTINCT ON` or window functions give the whole row; for two values per customer plain aggregates are enough.

SQL

```sql
SELECT customer_id, min(order_ts)::date AS first_order, max(order_ts)::date AS last_order,
       (array_agg(order_id ORDER BY order_ts DESC))[1] AS last_order_id
FROM orders GROUP BY customer_id ORDER BY customer_id;
```

Result TERMINAL

```
 customer_id | first_order | last_order | last_order_id
-------------+-------------+------------+---------------
           1 | 2025-01-03  | 2025-01-06 |           106
           2 | 2025-01-04  | 2025-01-09 |           109
           3 | 2025-01-05  | 2025-01-10 |           110
           5 | 2025-01-07  | 2025-01-08 |           108
(4 rows)
```

Problem 21: cumulative share of payroll

SQL

```sql
SELECT first_name, salary,
       round(100 * sum(salary) OVER (ORDER BY salary DESC, emp_id)
                 / sum(salary) OVER (), 1) AS cumulative_pct
FROM employees ORDER BY salary DESC, emp_id LIMIT 5;
```

Result TERMINAL

```
 first_name |  salary   | cumulative_pct
------------+-----------+----------------
 Asha       | 250000.00 |           13.1
 Vikram     | 180000.00 |           22.6
 Arjun      | 170000.00 |           31.5
 Neha       | 160000.00 |           39.9
 Meera      | 150000.00 |           47.8
(5 rows)
```

1.13.5 Gaps, islands and intervals

Problem 22: users with three or more consecutive login days

`user_logins` holds the days of three users. The difference between the date and a row number is constant inside a run of consecutive days.

SQL

```sql
SELECT user_id, min(login_date) AS from_day, max(login_date) AS to_day, count(*) AS streak
FROM (SELECT user_id, login_date,
             login_date - (row_number() OVER (PARTITION BY user_id
                                              ORDER BY login_date))::int AS grp
      FROM user_logins) t
GROUP BY user_id, grp
HAVING count(*) >= 3
ORDER BY user_id, from_day;
```

Result TERMINAL

```
 user_id |  from_day  |   to_day   | streak
---------+------------+------------+--------
       1 | 2025-03-01 | 2025-03-03 |      3
       2 | 2025-03-01 | 2025-03-05 |      5
(2 rows)
```

Problem 23: longest streak per user

SQL

```sql
SELECT DISTINCT ON (user_id) user_id, streak, from_day
FROM (SELECT user_id, min(login_date) AS from_day, count(*) AS streak
      FROM (SELECT user_id, login_date,
                   login_date - (row_number() OVER (PARTITION BY user_id
                                                    ORDER BY login_date))::int AS grp
            FROM user_logins) a
      GROUP BY user_id, grp) b
ORDER BY user_id, streak DESC, from_day;
```

Result TERMINAL

```
 user_id | streak |  from_day
---------+--------+------------
       1 |      3 | 2025-03-01
       2 |      5 | 2025-03-01
       3 |      1 | 2025-03-02
(3 rows)
```

Problem 24: overlapping bookings

Two bookings in the same room overlap when each starts before the other ends. The condition `a.starts < b.ends AND b.starts < a.ends` is the standard test; `a.booking_id < b.booking_id` reports each pair once. Back-to-back bookings (ending at 10:00 and starting at 10:00) do not overlap.

SQL

```sql
SELECT a.room, a.booking_id AS first, b.booking_id AS second
FROM bookings a JOIN bookings b
  ON a.room = b.room AND a.booking_id < b.booking_id
 AND a.starts < b.ends AND b.starts < a.ends;
```

Result TERMINAL

```
 room | first | second
------+-------+--------
 A    |     1 |      2
(1 row)
```

1.13.6 Pivoting and conditional aggregation

Problem 25: pivot rows into columns

`product_sales(product, quarter, amount)` has one row per product and quarter, with missing quarters. Conditional aggregation produces one column per quarter; `coalesce` turns the missing values into 0.

SQL

```sql
SELECT product,
       coalesce(sum(amount) FILTER (WHERE quarter = 'Q1'), 0) AS q1,
       coalesce(sum(amount) FILTER (WHERE quarter = 'Q2'), 0) AS q2,
       coalesce(sum(amount) FILTER (WHERE quarter = 'Q3'), 0) AS q3,
       coalesce(sum(amount) FILTER (WHERE quarter = 'Q4'), 0) AS q4,
       sum(amount) AS total
FROM product_sales GROUP BY product ORDER BY product;
```

Result TERMINAL

```
 product | q1  | q2  | q3  | q4 | total
---------+-----+-----+-----+----+-------
 Laptop  | 100 | 150 | 120 |  0 |   370
 Mouse   |  30 |  25 |   0 | 40 |    95
(2 rows)
```

Problem 26: unpivot columns into rows

The reverse operation turns the pivoted columns back into rows with a `LATERAL VALUES` list (MySQL: `UNION ALL` of one SELECT per column).

SQL

```sql
WITH pivoted AS (
  SELECT product,
         sum(amount) FILTER (WHERE quarter = 'Q1') AS q1,
         sum(amount) FILTER (WHERE quarter = 'Q2') AS q2
  FROM product_sales GROUP BY product)
SELECT p.product, v.quarter, v.amount
FROM pivoted p CROSS JOIN LATERAL (VALUES ('Q1', p.q1), ('Q2', p.q2)) AS v(quarter, amount)
ORDER BY p.product, v.quarter;
```

Result TERMINAL

```
 product | quarter | amount
---------+---------+--------
 Laptop  | Q1      |    100
 Laptop  | Q2      |    150
 Mouse   | Q1      |     30
 Mouse   | Q2      |     25
(4 rows)
```

Problem 27: count by status in one row per customer

SQL

```sql
SELECT customer_id,
       count(*) FILTER (WHERE status = 'PAID')      AS paid,
       count(*) FILTER (WHERE status = 'NEW')       AS new,
       count(*) FILTER (WHERE status = 'SHIPPED')   AS shipped,
       count(*) FILTER (WHERE status = 'CANCELLED') AS cancelled
FROM orders GROUP BY customer_id ORDER BY customer_id;
```

Result TERMINAL

```
 customer_id | paid | new | shipped | cancelled
-------------+------+-----+---------+-----------
           1 |    2 |   0 |       1 |         0
           2 |    1 |   1 |       0 |         1
           3 |    1 |   1 |       0 |         0
           5 |    1 |   0 |       1 |         0
(4 rows)
```

sreeram

1.13.7 Hierarchies and aggregates over groups

Problem 28: all subordinates of a manager

SQL

```sql
WITH RECURSIVE sub AS (
  SELECT emp_id, first_name, manager_id, 1 AS level FROM employees WHERE manager_id = 6
  UNION ALL
  SELECT e.emp_id, e.first_name, e.manager_id, s.level + 1
  FROM employees e JOIN sub s ON e.manager_id = s.emp_id)
SELECT emp_id, first_name, level FROM sub ORDER BY level, emp_id;

WITH RECURSIVE sub AS (
  SELECT emp_id, manager_id FROM employees WHERE manager_id IS NOT NULL
), chain(root, member) AS (
  SELECT manager_id, emp_id FROM sub
  UNION ALL
  SELECT c.root, s.emp_id FROM chain c JOIN sub s ON s.manager_id = c.member)
SELECT m.first_name AS manager, count(*) AS total_reports
FROM chain c JOIN employees m ON m.emp_id = c.root
GROUP BY m.first_name ORDER BY total_reports DESC, manager LIMIT 4;
```

Result TERMINAL

```
 emp_id | first_name | level
--------+------------+-------
      7 | Priya      |     1
      8 | Kiran      |     1
     14 | Omar       |     1
(3 rows)

 manager | total_reports
---------+---------------
 Asha    |            13
 Arjun   |             3
 Vikram  |             3
 Divya   |             1
(4 rows)
```

The second query counts direct and indirect reports of every manager: Asha has 13, Arjun 3 (Priya, Kiran, Omar) and Vikram 3 (Meera, Ravi and Meera's report Sneha).

Problem 29: departments whose average salary exceeds the company average

SQL

```sql
SELECT d.dept_name, round(avg(e.salary)) AS dept_avg
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING avg(e.salary) > (SELECT avg(salary) FROM employees)
ORDER BY dept_avg DESC;
```

Result TERMINAL

```
  dept_name  | dept_avg
-------------+----------
 Engineering |   170000
(1 row)
```

Problem 30: the department with the highest payroll, including ties

`rank() = 1` returns every department tied for the maximum; `ORDER BY ... LIMIT 1` would return an arbitrary one of them.

SQL

```sql
SELECT dept_name, payroll FROM (
  SELECT d.dept_name, sum(e.salary) AS payroll, rank() OVER (ORDER BY sum(e.salary) DESC) AS r
  FROM employees e JOIN departments d ON d.dept_id = e.dept_id GROUP BY d.dept_name) t
WHERE r = 1;
```

Preeram

Result TERMINAL

```
  dept_name  |  payroll
-------------+-----------
 Engineering | 850000.00
(1 row)
```

1.13.8 Solving an unfamiliar SQL problem

A method that applies to the problems above and to new ones, in order:

1. State the grain of the result: one row per what? Employee, department, customer, day, streak?
2. Identify which tables supply the facts and whether any join can multiply rows; aggregate many-sides to the grain first.
3. Identify the pattern (see the figure at the start of the chapter) and write the smallest query that produces it on three or four rows.
4. Test with the awkward data: NULLs, ties, empty groups, duplicates, boundaries.
5. State the tie and NULL semantics in words, and mention the index that would support the query at production size.

1.13.9 Diagnosing a wrong result

Wrong results are more dangerous than slow ones, because nothing signals them. The decision path below maps the observable symptom to the defects documented in this part of the book and to the check that confirms each.

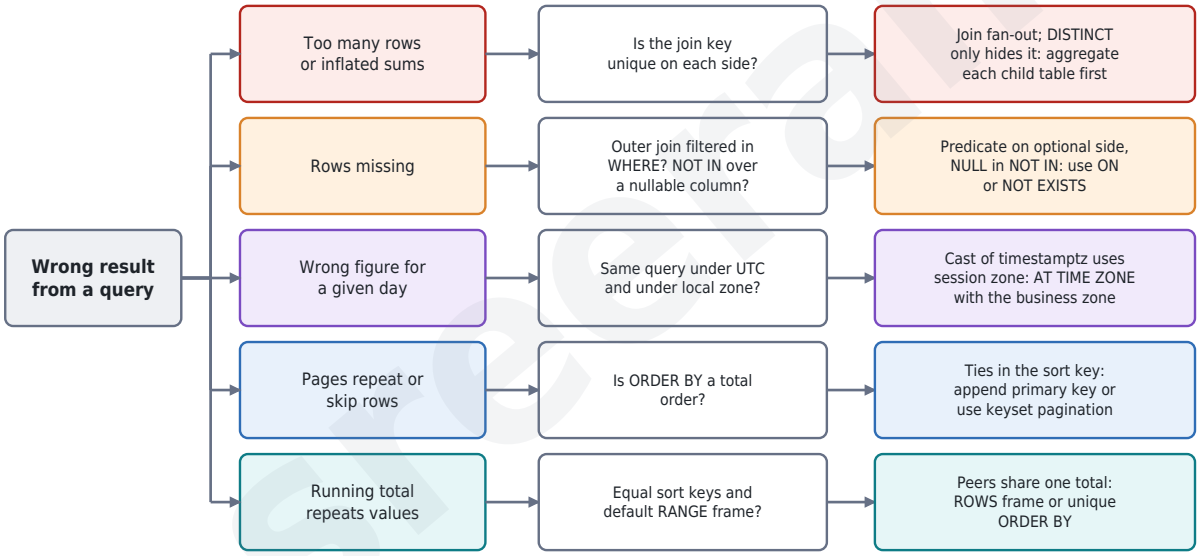


Figure 1.52. Decision path for a query that returns wrong data. Start from the symptom (too many rows, too few rows, wrong totals, wrong day, unstable order), apply the check on the arrow and arrive at the usual root cause and fix.

1.13.10 Questions to expect

INTERVIEW ANGLE

How would the interviewer's "second highest salary" question be answered in MySQL? The same two forms work: `SELECT max(salary) FROM employees WHERE salary < (SELECT max(salary) FROM employees)` and the `dense_rank()` window function (MySQL 8.0 and later). `LIMIT 1 OFFSET 1` over `SELECT DISTINCT salary ... ORDER BY salary DESC` also works, wrapped in a scalar subquery so that the result is NULL when absent.

INTERVIEW ANGLE

How are duplicate rows deleted while keeping one copy? Number the copies with `row_number() OVER (PARTITION BY key ORDER BY id)` and delete those numbered above 1, or self-join on equal key and a greater id. Then add a UNIQUE constraint so that the data cannot degrade again.

INTERVIEW ANGLE
How is a running total computed? `sum(x) OVER (ORDER BY key ROWS UNBOUNDED PRECEDING)`. The explicit ROWS frame avoids the default RANGE frame, which gives equal totals for rows with the same ordering key.

INTERVIEW ANGLE
How are consecutive sequences detected? Subtract a row number from the value (or date); the result is constant within a run of consecutive values, so grouping by it isolates each run. Then min, max and count describe the run.

INTERVIEW ANGLE
How is a pivot written without a PIVOT keyword? One conditional aggregate per output column: `sum(CASE WHEN quarter = 'Q1' THEN amount END) AS q1`, or `FILTER` in PostgreSQL, grouped by the row key.

KEY POINTS
Interview problems

- Most problems are instances of a few patterns: anti-join, top-N per group, running aggregate, gaps and islands, pivot, recursion, division.
- State the grain, the tie semantics and the NULL semantics before writing the query.
- Verify each solution on data that contains ties, NULLs, empty groups and duplicates.
- Prefer window functions to correlated subqueries, and `NOT EXISTS` to `NOT IN`.
- Follow a delete-duplicates or fix-up statement with a constraint that prevents the problem from returning.

1.14 Storage and Pages

Every performance property of a relational database follows from one physical fact: data lives on disk in fixed-size blocks, and the cost of a query is dominated by the number of blocks that must be moved into memory. A query that reads 20 834 blocks and a query that reads 4 blocks may return the same row; the first takes about 120 ms on the build machine used for this book and the second about 0.1 ms. This chapter describes how rows are packed into blocks, how large values are stored out of line, how blocks are cached in memory, and how PostgreSQL and MySQL InnoDB differ in organising a table. The following chapters on indexes and query plans build directly on this model.

All experiments in this part use PostgreSQL 16.15 with the default 8 kB block size, `shared_buffers` set to 64 MB, and a database named `shop` that contains three tables: `customers` (100 000 rows), `orders` (1 500 000 rows) and `order_items` (900 000 rows). Timings are measured on a two-CPU virtual machine and are typical, not exact. MySQL behaviour is taken from the MySQL 8.x documentation and is labelled as not run on the build machine.

1.14.1 Concept and mechanism

Heap files and pages

A PostgreSQL table is stored as a **heap**: a file (split into 1 GB segments) that is an array of **pages**, also called blocks. A page is 8192 bytes by default (`SHOW block_size`). Rows are appended wherever free space exists; there is no ordering. A row version stored in a page is called a **tuple**. A tuple is addressed by its **ctid**, the pair (page number, line pointer number), for example `(0,1)` for the first tuple on the first page.

The page uses a *slotted* layout. A 24-byte header is followed by an array of 4-byte **line pointers** that grows toward the end of the page, while tuple data is written from the end of the page toward the beginning. The gap between the two is free space. A line pointer records the offset and length of its tuple, so a tuple can be moved within the page (compaction) without changing its ctid, and an index entry that holds a ctid stays valid.

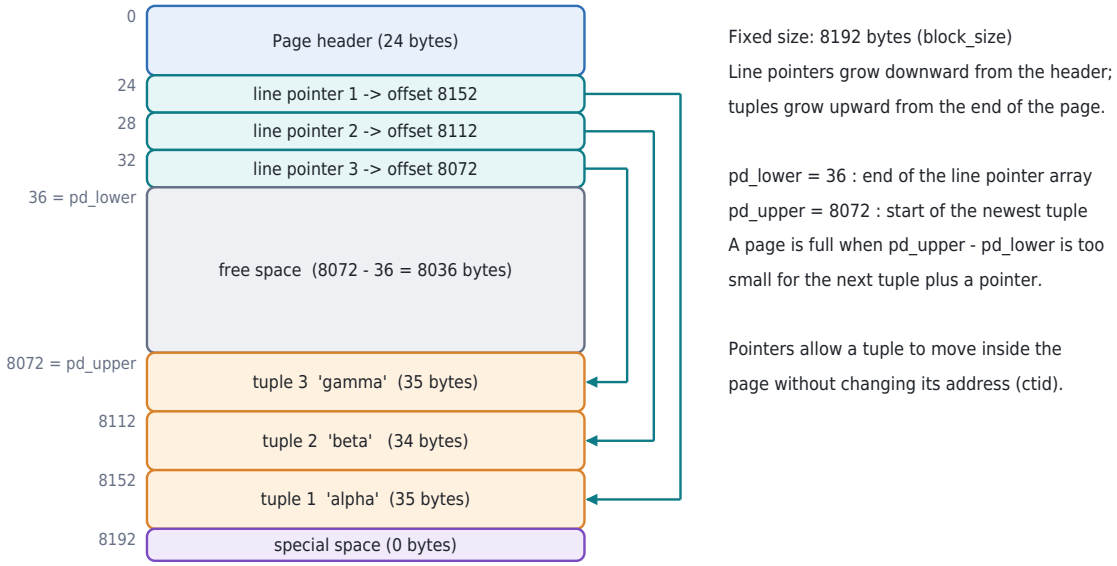


Figure 1.53. Layout of an 8 kB heap page holding three rows of a small table: line pointers grow downward from the header, tuples grow upward from the end, and each line pointer stores the offset of its tuple.

The numbers in the figure are real. They were read with the `pageinspect` extension from a table of three rows:

SQL

```sql
CREATE EXTENSION pageinspect;
CREATE TABLE t_small(id int, name text, flag boolean);
INSERT INTO t_small VALUES (1,'alpha',true),(2,'beta',false),(3,'gamma',true);

SELECT lp, lp_off, lp_flags, lp_len, t_xmin, t_xmax, t_ctid, t_hoff
FROM heap_page_items(get_raw_page('t_small', 0));

SELECT lower, upper, special, pagesize FROM page_header(get_raw_page('t_small', 0));
```

Output (PostgreSQL 16.15, real) TERMINAL

```
$ psql -d shop
 lp | lp_off | lp_flags | lp_len | t_xmin | t_xmax | t_ctid | t_hoff
----+--------+----------+--------+--------+--------+--------+--------
  1 |   8152 |        1 |     35 |    834 |      0 | (0,1)  |     24
  2 |   8112 |        1 |     34 |    834 |      0 | (0,2)  |     24
  3 |   8072 |        1 |     35 |    834 |      0 | (0,3)  |     24

 lower | upper | special | pagesize
-------+-------+---------+----------
    36 |  8072 |    8192 |     8192
```

`lower = 36` is 24 header bytes plus three 4-byte pointers; `upper = 8072` is the start of the most recently written tuple. Free space is `upper - lower = 8036` bytes. `lp_flags = 1` means the pointer is in use (`LP_NORMAL`).

Tuple layout

Each tuple begins with a 23-byte header (padded to 24 bytes, the value of `t_hoff`), followed by an optional null bitmap and then the column values in table order. The header fields that matter for later chapters are `t_xmin` (the transaction that created this version), `t_xmax` (the transaction that deleted or locked it, 0 if none) and `t_ctid` (the location of the newest version of the row). The multi-version concurrency control chapters in the part on transactions explain how these fields decide which sessions see a tuple; here they are only a storage detail.

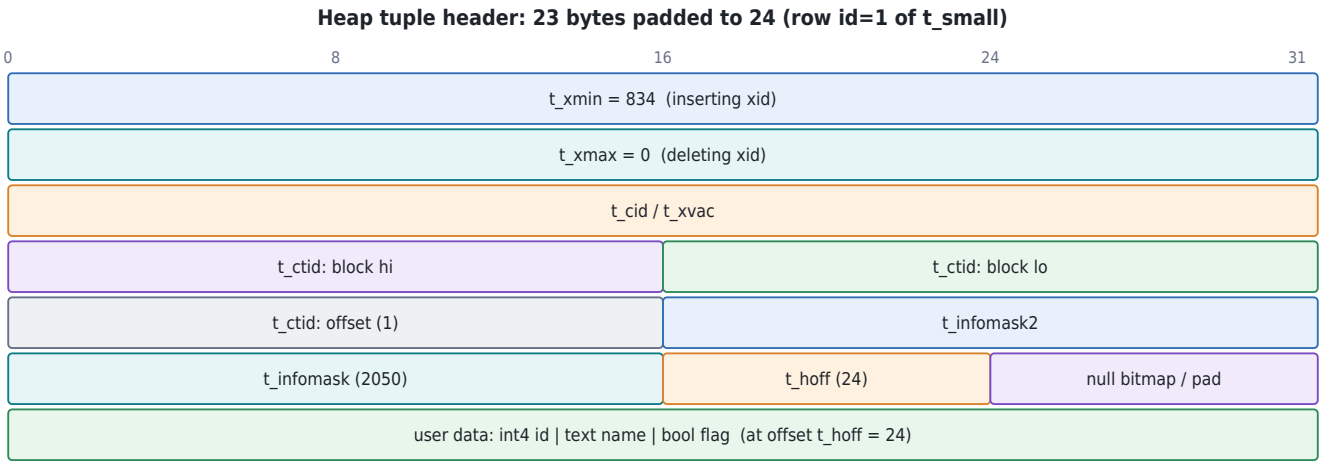


Figure 1.54. The 23-byte heap tuple header of row id=1: the transaction ids (834, 0), the self-referencing ctid (0,1), the flag words and the offset of the user data (t_hoff = 24).

The user data of the first row, `\x010000000d616c70686101`, decodes as follows: `01000000` is the 4-byte integer 1; `0d` is a one-byte variable-length header (length 6 including itself, stored as (6 shifted left by one) plus one); `616c706861` is the text `alpha`; `01` is the boolean true. Two properties follow: a short `text` value carries 1 byte of overhead, and every row carries at least 24 bytes of header, so a table of a single `int` column still costs roughly 28 bytes per row including the line pointer.

Updates create new tuples

PostgreSQL never overwrites a tuple in place. An `UPDATE` marks the old version with the updating transaction id in `t_xmax` and writes a complete new version, usually on the same page when space is available.

SQL

```sql
UPDATE t_small SET name = 'ALPHA' WHERE id = 1;
SELECT lp, lp_off, lp_len, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('t_small', 0));
```

Output (real) TERMINAL

```
$ psql -d shop
 lp | lp_off | lp_len | t_xmin | t_xmax | t_ctid
----+--------+--------+--------+--------+--------
  1 |   8152 |     35 |    834 |    835 | (0,4)
  2 |   8112 |     34 |    834 |      0 | (0,2)
  3 |   8072 |     35 |    834 |      0 | (0,3)
  4 |   8032 |     35 |    835 |      0 | (0,4)
```

Before UPDATE ... SET name='ALPHA' WHERE id=1

lp	xmin	xmax	ctid	name
1	834	0	(0,1)	alpha
2	834	0	(0,2)	beta
3	834	0	(0,3)	gamma

pg_relation_size unchanged: 1 page (8192 bytes)

Every UPDATE writes a complete new tuple version;
the old one stays until VACUUM removes it.

After the UPDATE (same page, before VACUUM)

lp	xmin	xmax	ctid	name
1	834	835	(0,4)	alpha
2	834	0	(0,2)	beta
3	834	0	(0,3)	gamma
4	835	0	(0,4)	ALPHA

pd_lower 36 -> 40, pd_upper 8072 -> 8032

Old version: xmax = 835 marks it deleted by xid 835.

New version: xmin = 835, reached through ctid (0,4).

Figure 1.55. The same page before and after an UPDATE: line pointer 1 now carries xmax = 835 and a ctid pointing to the new version in line pointer 4; the old version remains until VACUUM reclaims it.

Two consequences recur throughout this part. First, a table that receives many updates grows beyond its live data until `VACUUM` reclaims dead versions (table bloat). Second, an index entry points to a ctid, so a new tuple version normally requires new entries in every index of the table, unless the update qualifies as a heap-only tuple (HOT) update: the changed columns are not indexed and the new version fits on the same page.

Large values: TOAST

A tuple must fit on one page, and PostgreSQL tries to keep at least four tuples per page. When a row exceeds about 2 kB (the TOAST threshold), the largest variable-length columns are first compressed and, if the row is still too wide, moved out of line into a companion table named `pg_toast.pg_toast_<oid>` (TOAST, The Oversized-Attribute Storage Technique). The main row keeps an 18-byte pointer. The stored value is cut into chunks of at most 1996 bytes; one chunk is one row of the toast table.

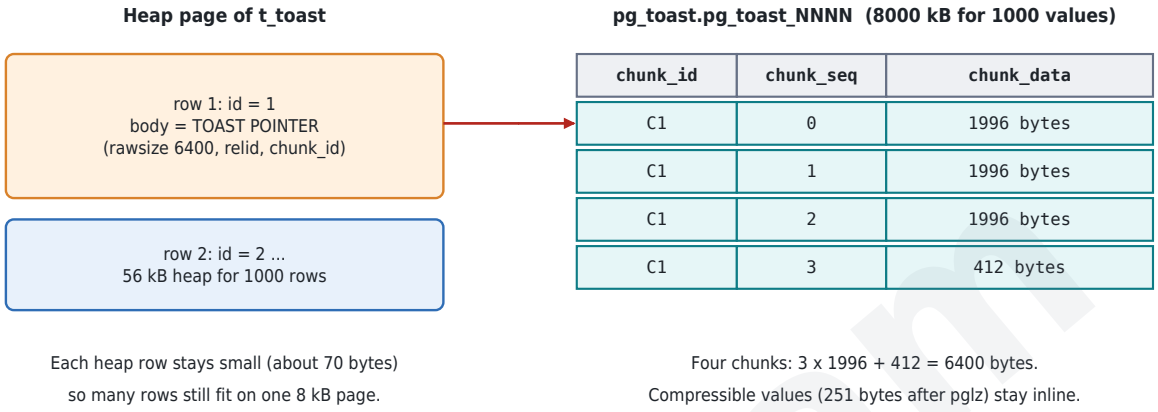


Figure 1.56. A 6400-character incompressible text value occupies four rows in the toast table (3 x 1996 bytes + 412 bytes), while the heap row keeps only a pointer.

SQL

```sql
CREATE TABLE t_toast(id int, body text);
INSERT INTO t_toast
SELECT g, (SELECT string_agg(md5(random()::text || i::text), '') FROM generate_series(1,200) i)
FROM generate_series(1,1000) g;

SELECT pg_size_pretty(pg_relation_size('t_toast')) AS heap,
       pg_size_pretty(pg_relation_size(reltoastrelid)) AS toast
FROM pg_class WHERE relname = 't_toast';
```

Output (real) TERMINAL

```
$ psql -d shop
  heap  |  toast
--------+---------
 56 kB  | 8000 kB
```

One thousand rows of 6400 characters occupy only 56 kB of heap, because every row holds a pointer, and 8000 kB of toast data. A highly compressible value of 20 000 characters (`repeat('abcdefghij', 2000)`) was stored in only 251 bytes inline and needed no toast rows at all. The practical consequence is that a query that does not mention a large column never reads its toast data, and a query that does must fetch and decompress every chunk (see the first incident at the end of this chapter).

The per-column strategy can be changed with `ALTER TABLE ... ALTER COLUMN ... SET STORAGE`: `plain` (never out of line), `main` (compress, avoid out of line), `external` (out of line without compression, which accelerates substring operations on already-compressed data such as images) and `extended` (the default for most variable-length types).

Row stores and column stores

PostgreSQL and MySQL are **row stores**: all columns of one row are adjacent. Fetching or modifying a whole row touches one place, which suits OLTP. An aggregate over one column of a wide table still reads every page, including the columns it does not use. A **column store** keeps each column in its own contiguous run and compresses it, which suits OLAP scans. The experiment below measures the effect of width on a row store: the same 1.5 million amounts are summed from the full `orders` table and from a two-column copy.

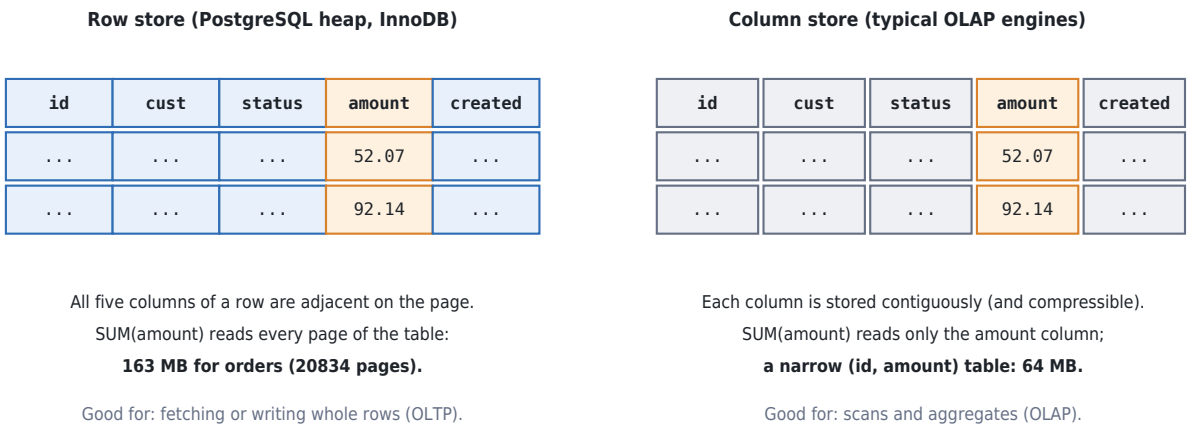


Figure 1.57. A row store keeps all five columns of each row together, so summing one column reads the whole table; a column store reads only the amount column.

SQL

```sql
CREATE TABLE orders_amount AS SELECT id, amount FROM orders;
SELECT pg_size_pretty(pg_relation_size('orders'))        AS wide,
       pg_size_pretty(pg_relation_size('orders_amount')) AS narrow;
```

Output (real) TERMINAL

```
$ psql -d shop
  wide  | narrow
--------+--------
 163 MB | 64 MB
```

The wide table has 20 834 pages and 72 rows per page; the narrow table has about 8 200 pages. Row width is therefore a performance property: removing unused or rarely read columns into a side table, or storing large rarely used values separately, directly reduces the I/O of every scan.

Column order and alignment padding

Fixed-width types are aligned to their natural boundary (`bigint` to 8 bytes, `int` to 4). A narrow type placed before a wide one forces padding bytes. Two tables with the same five columns, filled with one million identical rows, differ in size only by column order:

SQL

```sql
CREATE TABLE pad_bad (a boolean, b bigint, c boolean, d bigint, e boolean);
CREATE TABLE pad_good(b bigint, d bigint, a boolean, c boolean, e boolean);
-- insert 1 000 000 rows into each
SELECT pg_size_pretty(pg_relation_size('pad_bad')) AS bad,
       pg_size_pretty(pg_relation_size('pad_good')) AS good;
```

Output (real) TERMINAL

```
$ psql -d shop
  bad  | good
-------+-------
 65 MB | 50 MB
```

Ordering columns from widest fixed-width type to narrowest saved 23 percent here. The effect is real but secondary; it matters for very large narrow tables.

The buffer pool

Pages are not read from disk for every access. PostgreSQL keeps a shared cache, `shared_buffers`, divided into 8 kB buffers; the default is 128 MB and the build machine uses 64 MB (8192 buffers). A hash table maps (relation, fork, block number) to a buffer. When an executor node requests a block it looks it up in the hash table: a **hit** returns the buffer, a **miss** asks the operating system for the block (the OS page cache may satisfy it without touching the disk) and evicts a victim buffer chosen by a clock-sweep algorithm. `EXPLAIN (ANALYZE, BUFFERS)` reports `shared hit` and `shared read`; a read means the block was not in `shared_buffers`.

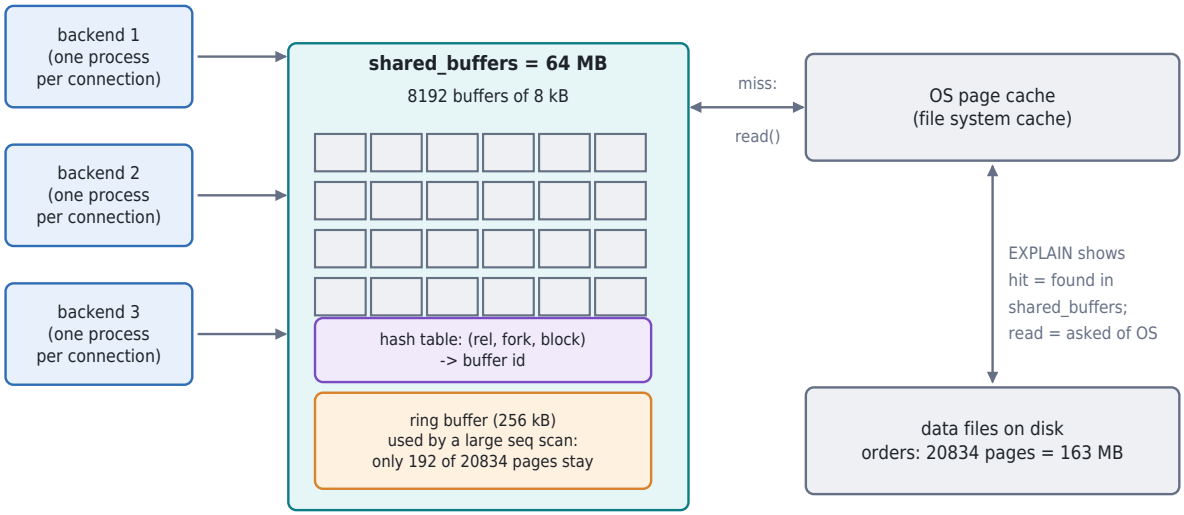


Figure 1.58. Backends share one buffer pool; a hash table finds a block, a miss reads it through the operating system cache, and a large sequential scan cycles through a small ring buffer instead of evicting the whole pool.

A sequential scan of a table larger than a quarter of `shared_buffers` uses a small ring buffer (256 kB) so that one big scan does not flush the useful working set. This was observable on the build machine. After scanning `orders` (20 834 pages, 163 MB, three times the pool) the pool still held only 192 buffers of that table:

SQL

```
CREATE EXTENSION pg_buffercache;
SELECT c.relname, count(*) AS buffers
FROM pg_buffercache b JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE b.reldatabase = (SELECT oid FROM pg_database WHERE datname = 'shop')
GROUP BY 1 ORDER BY 2 DESC LIMIT 4;
```

Output after the scans (real) TERMINAL

```
$ psql -d shop
 relname  | buffers
----------+---------
 pad_good |    6373
 pad_bad  |    1396
 orders   |     192
 pg_class |      18
```

The tables read most recently by index or by a small scan (`pad_good`) stay cached; the large scanned table does not. The cache-hit ratio of the whole database is available from the statistics views:

Cache hit ratio (real) TERMINAL

```
$ psql -d shop
 datname | blks_hit | blks_read | hit_pct
---------+----------+-----------+---------
 shop    |   451081 |     33014 |   93.18
```

For an OLTP system a ratio below roughly 99 percent on the hot tables usually means that the working set exceeds memory or that queries scan more than they should. A rule of thumb sets `shared_buffers` to about 25 percent of RAM and relies on the OS cache for the rest; `effective_cache_size` informs the planner about the total cache and does not allocate anything.

InnoDB: the table is the primary key index

MySQL's default engine, InnoDB, stores a table as a **clustered index**: a B+tree ordered by the primary key whose leaf pages contain the complete rows. Its page size is 16 KB by default (`innodb_page_size`), and the buffer pool (`innodb_buffer_pool_size`, default 128 MB, commonly 50 to 75 percent of RAM on a dedicated server) is its equivalent of `shared_buffers`. Secondary indexes store the indexed columns plus the **primary key value**, not a physical address. A lookup through a secondary index therefore descends the secondary tree, obtains the primary key, and descends the clustered tree a second time. Values that do not fit the page (large `TEXT` and `BLOB` columns in the `DYNAMIC` row format) are stored on overflow pages, which corresponds to TOAST.

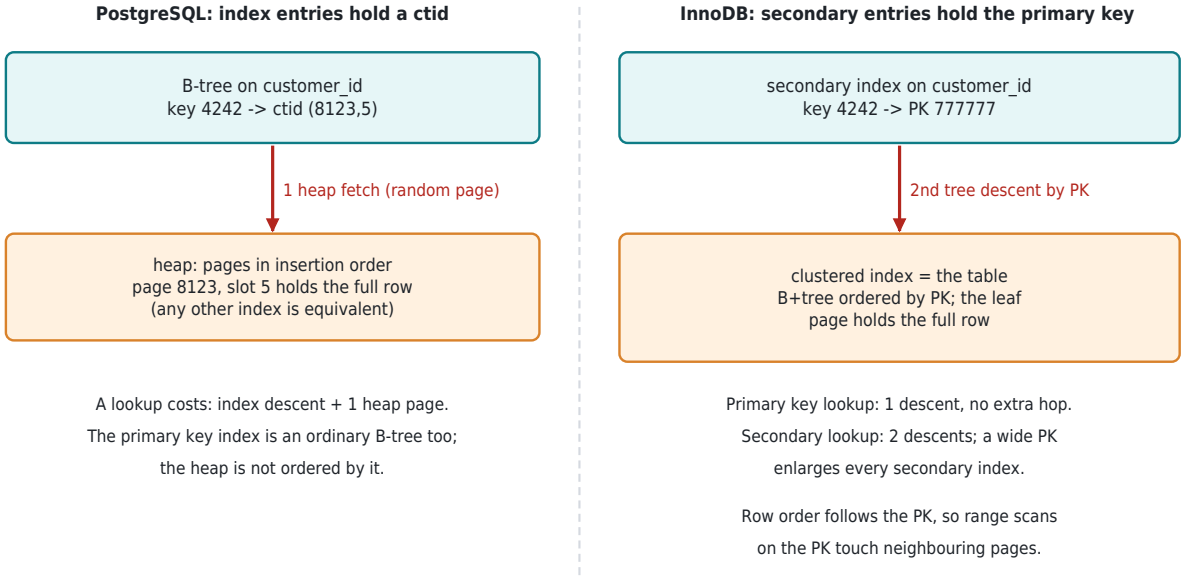


Figure 1.59. PostgreSQL indexes point to a physical ctid in an unordered heap; InnoDB secondary indexes point to the primary key and need a second descent through the clustered index.

Aspect	PostgreSQL 16	MySQL 8.x InnoDB
Table organisation	Unordered heap; every index is secondary	Rows stored in primary key order (clustered index)
Index entry points to	ctid (page, slot)	Primary key value
Page size	8 kB (compile-time)	16 KB (default)
Lookup by primary key	Index descent plus one heap fetch	One descent, row in the leaf
Lookup by secondary index	Index descent plus one heap fetch	Two descents (secondary, then clustered)
Update of a non-indexed column	New tuple version; HOT if it fits on the page	In-place change; old version in the undo log
Cost of a wide primary key	Only the primary key index grows	Every secondary index grows
Random primary keys (UUID v4)	Heap unaffected; index pages split	Clustered pages split and fragment
Large values	TOAST table	Off-page overflow pages

PostgreSQL heap compared with InnoDB clustered storage.

The difference can be measured in PostgreSQL by clustering a table. Physical order is irrelevant to InnoDB because the primary key defines it, but in PostgreSQL the heap order of rows with equal keys decides how many pages a range scan touches. On a 600 000-row copy of `orders`, the range `customer_id BETWEEN 5000 AND 5100` (606 rows) needed 606 heap pages while the data was in insertion order (correlation 0.002) and 13 buffers after `CLUSTER oc USING oc_cust` (correlation 1). `CLUSTER` is a one-time rewrite under an exclusive lock; PostgreSQL does not maintain the order afterwards.

1.14.2 Worked example

How many pages does a query touch?

Given the facts above, the page count of simple queries can be predicted without running them. The table `orders` has 1 500 000 rows on 20 834 pages (72 rows per page); the index `orders_pkey` has 4099 leaf pages and three levels.

1. `SELECT * FROM orders WHERE customer_id = 4242` without an index: the sequential scan reads all 20 834 pages (the plan reports `shared hit=352 read=20482`, the sum being 20 834). The 15 matching rows are found by examining 1.5 million tuples.
2. `SELECT * FROM orders WHERE id = 777777`: three index pages (root, internal, leaf) and one heap page, four buffers in total. The warm run reports `shared hit=4`.

3. `SELECT * FROM orders WHERE customer_id = 4242` with an index on `customer_id`: three index pages and up to 15 heap pages, because the 15 orders of one customer were inserted at different times and lie on 15 different pages. The plan reports `Heap Blocks: exact=15`.

Query 3 shows that an index lowers the page count from 20 834 to about 18, a factor of more than 1000, but it does not make the result free: each additional matching row on a distinct page costs one more block. This is the reason that the planner prefers a sequential scan once a predicate selects a large part of the table (see the chapters on indexes and the planner).

1.14.3 Hands-on

The following commands reproduce the figures used in this chapter. Run them in `psql` connected to a scratch database.

SQL

```sql
-- size of the table, its pages, and its indexes
SELECT pg_size_pretty(pg_relation_size('orders'))        AS heap,
       pg_relation_size('orders') / 8192                 AS pages,
       pg_size_pretty(pg_relation_size('orders_pkey'))   AS pk,
       pg_size_pretty(pg_total_relation_size('orders'))  AS total;

-- rows per page, measured from the ctid
SELECT count(*) AS rows,
       count(DISTINCT (ctid::text::point)[0]) AS pages
FROM orders;
```

Output (real) TERMINAL

```
$ psql -d shop
  heap  | pages |  pk   | total
--------+-------+-------+--------
 163 MB | 20834 | 32 MB | 195 MB

  rows   | pages
---------+-------
 1500000 | 20834
```

`pg_total_relation_size` includes indexes and the toast table. Other useful catalogue functions are `pg_table_size` (heap plus toast, no indexes), `pg_indexes_size`, and `pgstattuple('orders')`, which reports dead tuples and free space. The statement `SELECT ctid, * FROM t LIMIT 5` shows the physical address of rows and is a quick way to see that deletes and updates leave gaps.

Task	MySQL 8.x statement	Note
Table and index size	`SELECT table_name, data_length, index_length FROM information_schema.tables`	Bytes; data_length is the clustered index
Page size	`SHOW VARIABLES LIKE 'innodb_page_size'`	16384 by default
Buffer pool	`SHOW VARIABLES LIKE 'innodb_buffer_pool_size'`	134217728 (128 MB) by default
Hit ratio	`SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%'`	read_requests versus reads
Row format	`SHOW TABLE STATUS LIKE 'orders'`	Row_format is Dynamic by default

MySQL counterparts (typical, not run on the build machine).

1.14.4 Real-world problems and solutions

Incident 1: SELECT * makes a harmless table slow

Scenario. A document service stores 20 000 articles in a table `docs(id, title, body)`. The list endpoint of a Spring Boot application, `findAll()` on a Spring Data JPA repository, became progressively slower as editors pasted longer articles, although the table has only 1328 kB of heap.

Symptoms. The list page takes several hundred milliseconds and moves megabytes over the network. The query plan looks innocent: a sequential scan of 166 pages.

Diagnosis. First compare the size of the heap with the size of its toast table:

```sql
SELECT pg_size_pretty(pg_relation_size('docs')) AS heap,
       pg_size_pretty(pg_relation_size(reltoastrelid)) AS toast
FROM pg_class WHERE relname = 'docs';
```

Output (real)

```
$ psql -d shop
  heap   | toast
---------+--------
 1328 kB | 156 MB
```

The data is 100 times larger than the heap suggests. `EXPLAIN` does not show it, because toast values are fetched when the output column is computed, not by the scan node. Timing two statements reveals it:

Timing with psql \timing (real, build machine)

```
$ psql -d shop
shop=# SELECT count(*) FROM (SELECT id, title FROM docs) s;
Time: 2.987 ms
shop=# SELECT sum(length(body)) FROM docs;
Time: 342.517 ms
```

Root cause. Reading `body` requires fetching and decompressing roughly 156 MB of toast chunks. The entity maps every column, so `findAll()` selects them all even when the page shows only titles.

Fix. Immediate: select only the needed columns in the list query. In Spring Data JPA use an interface-based projection (`interface ArticleSummary { Long getId(); String getTitle(); }`) or a constructor expression in JPQL. Permanent: move the large text to a separate entity or table that is joined only on the detail page, or mark the column lazy with bytecode enhancement.

Prevention. Review the width of every table that is listed or scanned; track the ratio of toast size to heap size; never expose `SELECT *` on tables with large columns.

Incident 2: the working set no longer fits in memory

Scenario. (Illustrative figures.) An order-history service ran with a 99.6 percent cache hit ratio for a year. After the order table doubled, latency of the history endpoint rose from 5 ms to 60 ms with no code change and no change in query plans.

Symptoms. Plans are unchanged (index scans), but `EXPLAIN (ANALYZE, BUFFERS)` now shows mostly `shared read` instead of `shared hit`; disk read I/O on the host is elevated.

Diagnosis.

```sql
SELECT round(100.0 * blks_hit / nullif(blks_hit + blks_read, 0), 2) AS hit_pct
FROM pg_stat_database WHERE datname = current_database();

SELECT relname, heap_blks_read, heap_blks_hit, idx_blks_read, idx_blks_hit
FROM pg_statio_user_tables ORDER BY heap_blks_read DESC LIMIT 5;

SELECT pg_size_pretty(pg_database_size(current_database()));
SHOW shared_buffers;
```

A hit ratio that fell from about 99.6 to 93 percent (the value measured above for the experiment database) identifies which tables read from disk. Compare the size of the hot tables and indexes with the memory available.

Root cause. The working set (the pages touched by the daily workload) grew beyond `shared_buffers` plus the OS cache, so blocks are evicted and re-read.

Fix. Immediate: add RAM or raise `shared_buffers` (restart required) within about 25 percent of memory. Permanent: reduce the working set by dropping unused indexes, trimming wide columns, archiving or partitioning old orders so that cold rows are never read, and adding a covering index so that an index-only scan avoids heap pages.

Prevention. Alert on the hit ratio and on the growth of the hottest tables; load-test with data volume comparable to production, because a query that is fast on a table that fits in cache can be slow when it does not.

1.14.5 Questions to expect

INTERVIEW ANGLE
How is a row physically stored in PostgreSQL? In a heap table made of 8 kB pages. A page has a header, an array of line pointers growing down, and tuples growing up from the end. Each tuple has a 23-byte header with xmin, xmax and ctid, then the column values. A row is addressed by its ctid (page, slot), which is what index entries store.

1. What is TOAST and when does it apply? Values that make a row exceed about 2 kB are compressed and then moved to a companion table in chunks of at most 1996 bytes, leaving a pointer in the heap row. Queries that do not read the column never touch the toast table.
2. Why does an UPDATE in PostgreSQL create a new tuple? Old versions must remain visible to concurrent transactions that started earlier (MVCC), so the update writes a new version and marks the old one with xmax. VACUUM removes versions that no transaction can see.
3. What is the difference between a clustered and a non-clustered index? In a clustered index the rows are stored in key order in the leaf pages (InnoDB primary key); a non-clustered index is a separate structure pointing to the row (PostgreSQL: ctid; InnoDB secondary: primary key).
4. Why can a wide primary key hurt in InnoDB but not as much in PostgreSQL? InnoDB secondary indexes contain the primary key in every entry, so the key width multiplies across all indexes.
5. What does `shared read` mean in EXPLAIN BUFFERS? The page was not in shared_buffers and was requested from the operating system, which may still serve it from its cache.
6. Why is a column store faster for analytics? A query reads only the columns it needs, stored contiguously and compressed, whereas a row store reads whole rows.
7. How does column order affect table size? Alignment padding: placing 8-byte columns before 1-byte columns removed 15 MB of 65 MB in the measured example.
8. What fraction of the cache does a large sequential scan consume? Only a small ring buffer (256 kB) for tables larger than a quarter of shared_buffers, so it does not evict the working set.

KEY POINTS
- A heap page is 8 kB with line pointers at the front and tuples at the back; a row is addressed by its ctid.
- UPDATE writes a new tuple version; old versions remain until VACUUM, and every extra index must be maintained.
- Values over about 2 kB move to a toast table; avoid reading wide columns that are not needed.
- Row stores read whole rows; width and column order determine scan cost.
- The buffer pool decides hit or read; a large sequential scan uses a ring buffer and leaves the working set intact.
- InnoDB clusters the table on the primary key and secondary indexes store the primary key; PostgreSQL uses an unordered heap and ctid pointers.

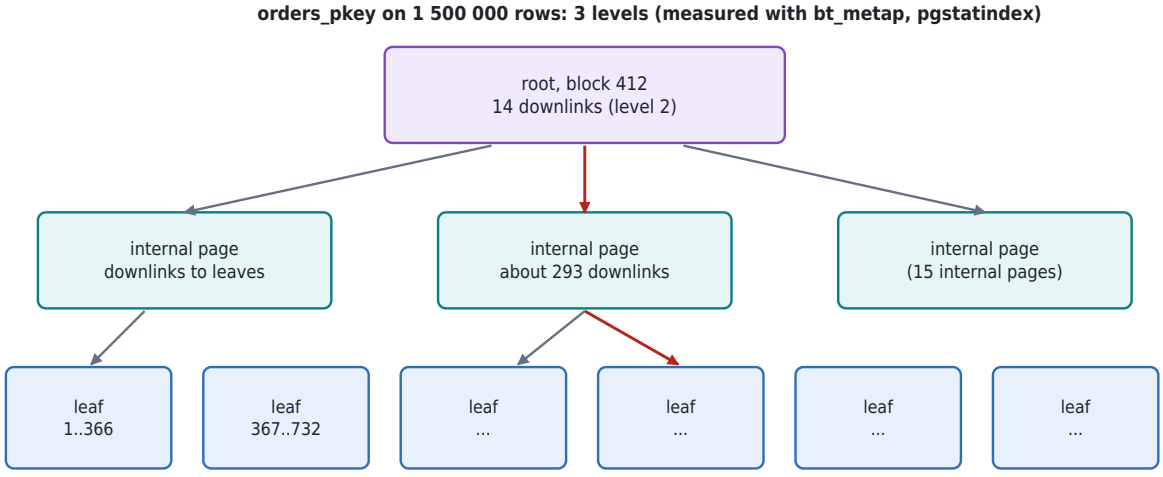
1.15 Indexes

Without an index, finding the orders of one customer in a table of 1.5 million rows requires reading every page: 20 834 pages and about 120 ms on the build machine. With an index on `customer_id` the same query reads about 18 pages and finishes in 0.13 ms. An **index** is a separate, sorted data structure that maps column values to the location of the rows that contain them, trading extra storage and slower writes for faster reads. Choosing the right indexes is the single most effective tool of database performance work, and choosing them badly is among the most common causes of slow systems. This chapter explains the B+tree in detail, shows a real node split, and then covers composite, covering, partial, expression and specialised indexes, their cost, and the differences in MySQL InnoDB.

1.15.1 Concept and mechanism

The B+tree

The default index type in PostgreSQL (`btree`) and the only general-purpose type in InnoDB is the **B+tree**: a balanced, wide, shallow tree of pages. Every page is one disk block (8 kB in PostgreSQL, 16 KB in InnoDB). The lowest level, the **leaf** pages, holds all index entries in sorted order, and each leaf has links to its left and right neighbours. The upper levels hold only separator keys and pointers to child pages. All leaves are at the same depth, which is what makes lookup cost predictable.



Sibling links (btpo_prev / btpo_next) connect leaves left-to-right: a range scan walks them.
Warm lookup of id = 777777: Buffers: shared hit=4 (root, internal, leaf, heap page).

Figure 1.60. The three-level B-tree of the primary key of the 1.5 million row orders table: one root, 15 internal pages and 4099 leaf pages, so a lookup reads three index pages plus the heap page.

The shape in the figure was read from the index itself with the `pageinspect` and `pgstattuple` extensions:

SQL

```sql
SELECT level, root FROM bt_metap('orders_pkey');
SELECT internal_pages, leaf_pages, avg_leaf_density FROM pgstatindex('orders_pkey');
SELECT type, live_items, avg_item_size FROM bt_page_stats('orders_pkey', 412);
```

Output (real) TERMINAL

```
$ psql -d shop
 level | root
-------+------
     2 |  412

 internal_pages | leaf_pages | avg_leaf_density
----------------+------------+------------------
             15 |       4099 |            90.07

 type | live_items | avg_item_size
------+------------+---------------
 r    |         14 |            15
```

`level = 2` means two levels above the leaves, so a search visits three pages. The fan-out is high: a leaf with an 8-byte key (a 4-byte integer padded to 8, plus an 8-byte item header, 16 bytes per entry) holds 407 entries, and an internal page holds several hundred downlinks. The capacity of a tree of height *h* is roughly (fan-out) raised to the power *h*, so three levels cover hundreds of millions of rows, and a lookup in a table of one billion rows needs four pages. This logarithmic behaviour is the reason an index lookup is nearly independent of table size, whereas a sequential scan grows linearly.

A search for key *k* starts at the root, finds the last separator not greater than *k*, follows the downlink, and repeats until the leaf; a binary search inside each page finds the position. A range search finds the first matching leaf entry and then follows the sibling links, reading entries in sorted order without returning to the root. Two properties are essential in practice:

- The index is **ordered**, so it serves equality (`=`), ranges (`<`, `BETWEEN`), `ORDER BY` without a sort, `MIN` and `MAX`, and prefix matches (`LIKE 'abc%'` under a suitable operator class).
- The index is **sorted by the full key, left to right**, which produces the leftmost-prefix rule for composite indexes described below.

Insertion and page splits

New entries are inserted into the leaf that covers their key. When the leaf has no room the page is **split**: a new page is allocated, about half of the entries move to it, a new separator key is inserted into the parent, and a parent that overflows splits in turn. If the root splits, a new root is created and the tree grows by one level. Trees grow upward, never unbalanced.

A reduced example with at most three keys per node shows the mechanics before the real data. Keys 10, 20 and 30 fit in the root leaf; inserting 40 forces a split:

```
insert 10, 20, 30     leaf [10 20 30]                      (the root is a leaf)

insert 40             no room: split into two leaves, promote the first key of the right one

                               root [ 30 ]
                              /           \
                   leaf [10 20]  <->  leaf [30 40]

insert 25             25 < 30, goes left: leaf [10 20 25]
insert 22             left leaf full: split, promote 22

                               root [ 22 | 30 ]
                              /      |        \
                   [10 20]  <->  [22 25]  <->  [30 40]
```

The real index behaves the same way. A table `bt(k int primary key)` holds 407 rows, exactly the capacity of one leaf page; the 408th insertion forces the first split. The state before and after was read page by page:

SQL

```sql
CREATE TABLE bt(k int PRIMARY KEY);
INSERT INTO bt SELECT generate_series(1, 407);
SELECT level, root FROM bt_metap('bt_pkey');                    -- level 0, root 1
SELECT live_items, free_size FROM bt_page_stats('bt_pkey', 1);   -- 407 items, 8 bytes free
INSERT INTO bt VALUES (408);
SELECT level, root FROM bt_metap('bt_pkey');                    -- level 1, root 3
SELECT s.b AS blkno, p.type, p.live_items, p.free_size, p.btpo_prev, p.btpo_next
FROM generate_series(1, 3) s(b), LATERAL bt_page_stats('bt_pkey', s.b) p;
```

Output after the 408th insert (real) TERMINAL

```
$ psql -d shop
 blkno | type | live_items | free_size | btpo_prev | btpo_next
-------+------+------------+-----------+-----------+-----------
     1 | l    |        367 |       808 |         0 |         2
     2 | l    |         42 |      7308 |         1 |         0
     3 | r    |          2 |      8116 |         0 |         0
```

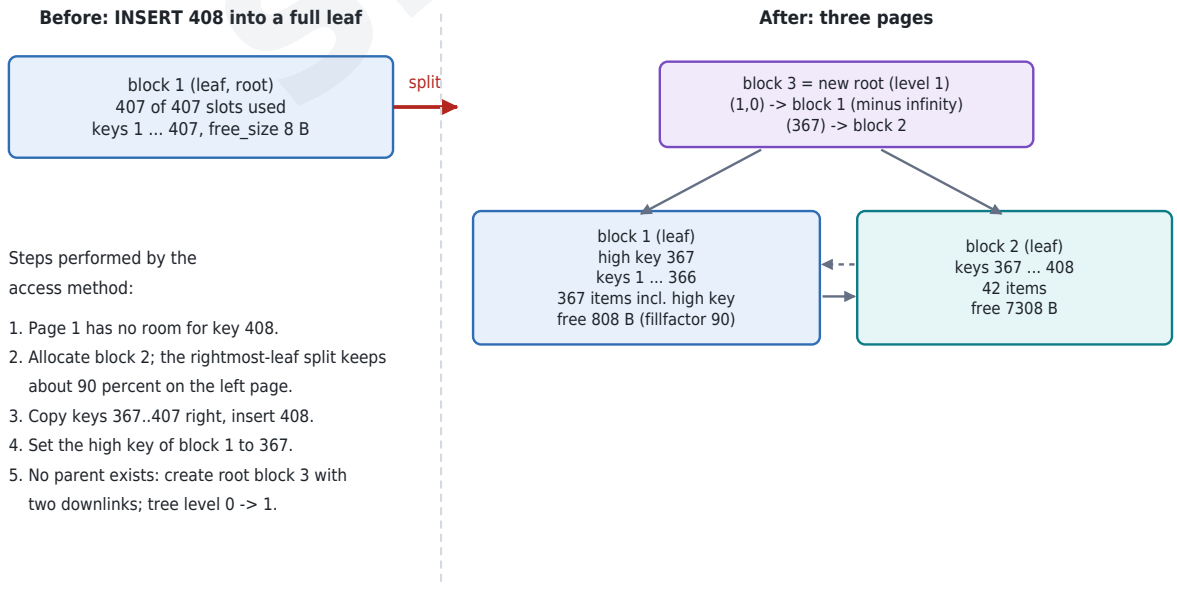


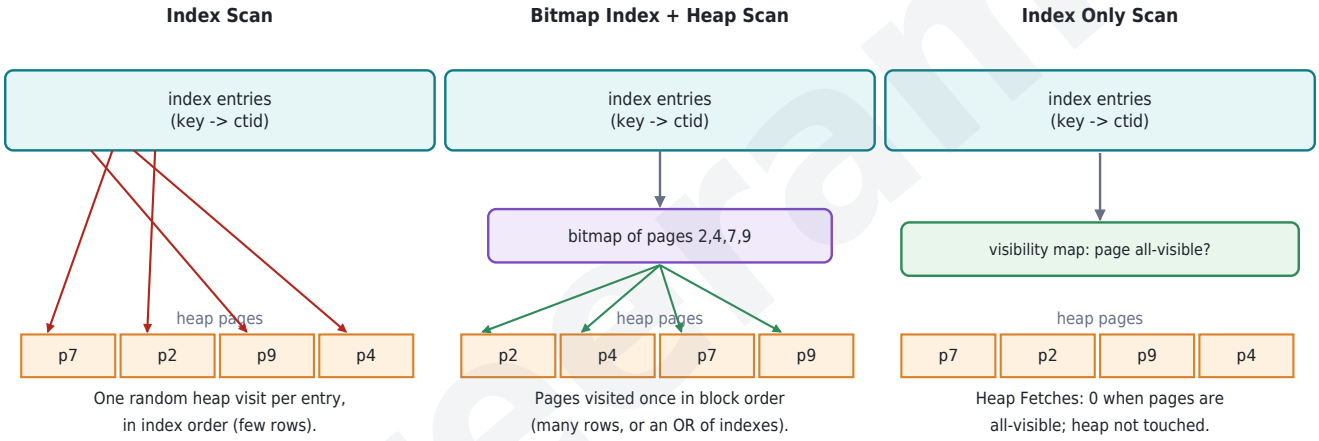
Figure 1.61. The 408th insertion splits the full leaf of a one-page index: keys 1 to 366 stay in block 1 (with high key 367), keys 367 to 408 move to the new block 2, and a new root block 3 is created.

`bt_page_items` exposes the separator entries. The root (block 3) holds two items: a "minus infinity" downlink to block 1 and the key 367 with a downlink to block 2. Block 1 ends with the key 366, and its first item is the **high key** 367, an upper bound for the keys it may hold. Two details differ from textbook trees. First, the split is asymmetric, 366 versus 42: when the insertion happens at the right edge (monotonically increasing keys, such as a sequence), PostgreSQL leaves the left page about 90 percent full (`fillfactor` 90 for the rightmost page) because that page will never receive another insertion; this produces the high leaf density of 90.07 percent measured above. Random insertions split in the middle and settle at an average density near 70 percent. Second, PostgreSQL 13 and later **deduplicate** equal keys in a leaf, which is why the index on the low-cardinality `customer_id` (100 000 distinct values among 1.5 million rows) takes only 12 MB, against 32 MB for the unique primary key.

Index scan, bitmap scan and index-only scan

The planner can use an index in three ways, shown in the figure below.

- **Index scan.** Walk the index in key order and fetch the heap page for each entry immediately. It returns rows in index order, so a plain `ORDER BY` is free, but every entry costs a random heap access.
- **Bitmap scan.** Collect all matching entries into a bitmap of heap pages, then read the pages in physical order, each once. It is chosen when a moderate number of rows is expected, and it can combine several indexes with AND/OR (`BitmapOr`). The result is unordered.
- **Index-only scan.** If every column the query needs is stored in the index, the heap is not read at all, except for pages that the **visibility map** does not mark as all-visible.



Measured: customer_id = 4242 (15 rows) -> bitmap scan, 0.13 ms; 15 heap pages touched.

The planner chooses the shape from estimated row count and heap correlation.

Figure 1.62. Three ways of using an index: an index scan visits the heap per entry, a bitmap scan visits each heap page once in block order, and an index-only scan avoids the heap when the visibility map allows.

Composite indexes and the leftmost-prefix rule

An index on several columns, `(a, b, c)`, sorts entries by `a`, then by `b` within equal `a`, then by `c`. A search can use the index efficiently for a condition on `a`, on `a` and `b`, or on `a`, `b` and `c`, with equality on all leading columns and at most a range on the last column used. A condition on `b` alone cannot seek, because entries with a given `b` are scattered through the whole index.

sreram

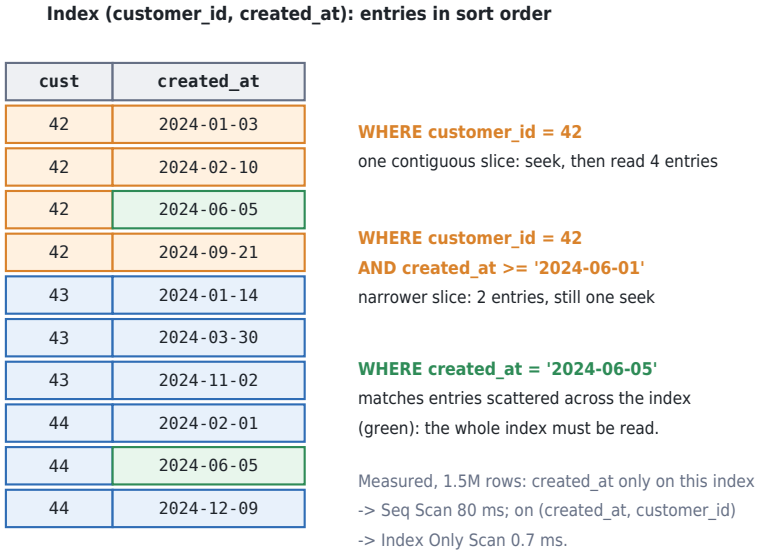


Figure 1.63. Entries of an index on (customer_id, created_at) in sort order: a condition on customer_id selects one contiguous slice, an additional range on created_at narrows it, and a condition on created_at alone selects scattered entries.

The effect was measured on `orders` (1.5 million rows). With an index on `(customer_id, created_at)`, a query on `customer_id` and a month of `created_at` read 5 buffers and finished in 0.11 ms. A query on a single day of `created_at` alone could not use the index and required a parallel sequential scan of 80 ms; after creating the index with the reverse column order `(created_at, customer_id)` the same query became an index-only scan with 0.73 ms. Design rules that follow:

1. Put columns compared with **equality** first and the column compared with a **range** or used for ordering last.
2. Among equality columns, put the one that appears in the most queries first, because its prefix can be reused by other queries; selectivity matters less for ordering than for deciding whether the index is worth having at all.
3. A separate index on `(a)` is redundant when `(a, b)` exists. Dropping the shorter one saves write cost.
4. `ORDER BY a, b` can use an index on `(a, b)` and avoid a sort; `ORDER BY a, b DESC` requires matching directions, which can be declared (`(a, b DESC)`).

PostgreSQL 16 requires the leading column in the condition. PostgreSQL 18 (released in 2025) added B-tree **skip scan**, which can use an index when a leading column has no restriction and few distinct values; this book observes PostgreSQL 16, where the rule is strict. MySQL 8.0.13 added a comparable *skip scan range access* optimisation for specific query shapes.

Covering indexes and the visibility map

A **covering index** contains all the columns a query needs. PostgreSQL 11 added `INCLUDE`: payload columns are stored in the leaf entries but are not part of the search key, so they do not need to be orderable and do not enlarge the upper levels.

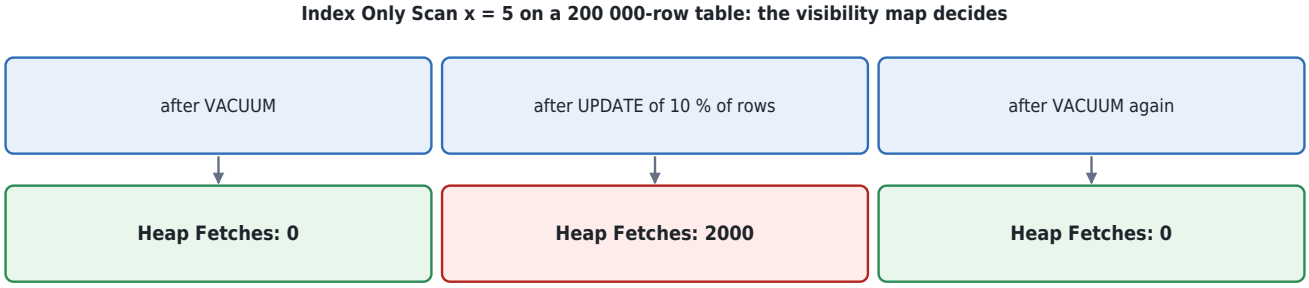
SQL

```sql
CREATE INDEX idx_cust_incl ON orders (customer_id) INCLUDE (amount);
VACUUM orders;
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF, TIMING OFF)
SELECT customer_id, sum(amount) FROM orders WHERE customer_id = 4242 GROUP BY customer_id;
```

Plan (real, trimmed) TERMINAL

```
$ psql -d shop
 GroupAggregate (actual rows=1 loops=1)
   ->  Index Only Scan using idx_cust_incl on orders (actual rows=15 loops=1)
         Index Cond: (customer_id = 4242)
         Heap Fetches: 0
         Buffers: shared hit=4
```

An index-only scan is only free when the visibility map says that all tuples on a heap page are visible to every transaction. Updates clear the bit; `VACUUM` sets it. A table with a 200 000-row test index showed `Heap Fetches: 0` after `VACUUM`, `Heap Fetches: 2000` after updating 10 percent of the rows, and `0` again after the next `VACUUM`. Index-only scans therefore need regular vacuuming (autovacuum provides it).



Pages touched by the UPDATE lose their all-visible bit; the scan must visit the heap to check each entry until VACUUM sets the bit again (relallvisible = relpages = 1978).

Figure 1.64. An index-only scan on a 200 000-row table: the number of heap fetches follows the state of the visibility map, which is cleared by UPDATE and restored by VACUUM.

In MySQL a covering index is created by adding the columns to the key (`INDEX (customer_id, amount)`); because every InnoDB secondary index already carries the primary key, a query that needs only the indexed columns and the primary key is covered automatically, and `EXPLAIN` shows `Using index` in the `Extra` column.

Partial, expression and unique indexes

A **partial index** indexes only the rows that satisfy a predicate. On `orders`, only 20 percent of rows have `status = 'NEW'` and a worker queries only those; a partial index on `created_at WHERE status = 'NEW'` took 6.6 MB, against 50 MB for the full index on `(status, created_at)`, and the query that matched its predicate used an index-only scan of 1.4 ms. The query predicate must imply the index predicate. MySQL has no partial indexes.

An **expression index** indexes the value of an expression, which makes a predicate on that expression searchable. Searching `lower(email) = 'user777@example.com'` on 100 000 customers took a 35 ms sequential scan even with an index on `email`, and 0.036 ms after `CREATE INDEX ON customers (lower(email))`. MySQL 8.0.13 supports functional key parts with a similar syntax (`INDEX ((LOWER(email)))`). The expression must be **immutable**: the attempt to index `date(created_at)` for a `timestamptz` column fails with `functions in index expression must be marked IMMUTABLE`, because the date depends on the session time zone. The immutable form fixes the zone: `((created_at AT TIME ZONE 'UTC')::date)`.

A **unique index** enforces a constraint and is the implementation of `PRIMARY KEY` and `UNIQUE`. PostgreSQL treats NULLs as distinct unless `NULLS NOT DISTINCT` (PostgreSQL 15) is specified; unique constraints are checked at the moment of insert, and a violation raises `duplicate key value violates unique constraint`.

Other index types

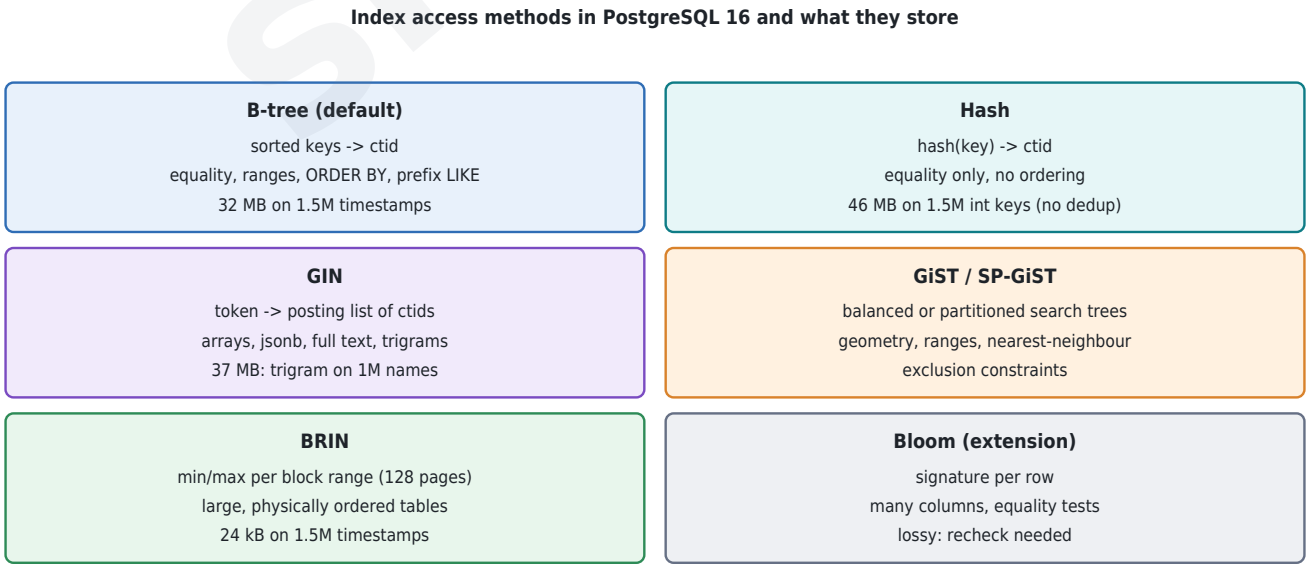
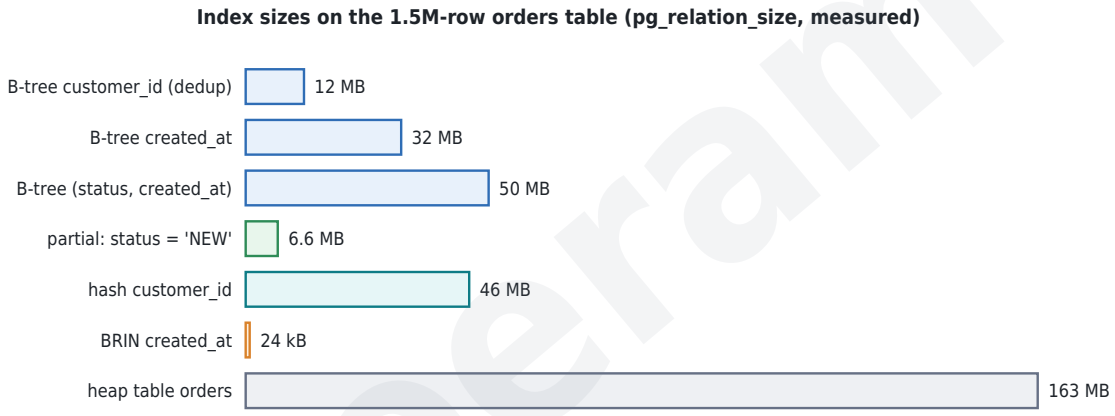


Figure 1.65. The access methods of PostgreSQL 16 with the structure each stores and a measured size where available.

Type	Stores	Use when
B-tree	Sorted keys with ctids	Equality, range, ordering, prefix pattern; default choice
Hash	Hash of the key	Equality only; in measured data larger than a B-tree (46 MB against 12 MB on `customer_id`), so rarely preferable
GIN	Token to list of ctids (inverted index)	Arrays, `jsonb` containment, full-text search, trigram `LIKE '%x%'`
GiST / SP-GiST	Generalised search trees	Geometry, ranges, nearest-neighbour search, exclusion constraints
BRIN	Min and max per block range	Very large append-only tables whose column correlates with physical order; 24 kB for 1.5 million timestamps
Bloom	Per-row signature	Equality tests on many columns when a single multi-column B-tree is impractical

Choosing an access method.

BRIN deserves a measurement because its size is surprising. A BRIN index on the monotonically increasing `created_at` of `orders` occupies 24 kB, against 32 MB for a B-tree, and answered a one-day range query with a lossy bitmap scan of 128 heap pages in 11.5 ms, against 0.6 ms for the B-tree. It is useful on time-series tables of billions of rows where a B-tree no longer fits in memory, and useless when the physical order does not follow the column.



Same table, same data; only the access method and the indexed predicate differ.

Figure 1.66. Sizes of several indexes on the same 1.5 million row table: the partial index is a fraction of the full one, a hash index exceeds a deduplicated B-tree, and BRIN is orders of magnitude smaller.

Selectivity: when the planner does not use an index

Selectivity is the fraction of rows a predicate returns. An index pays off when few rows match, because each match costs a random page access. The measurements below use the single-column index on `customer_id` and one predicate on the unindexed-in-effect column `status`:

Predicate	Rows returned	Plan and time
customer_id = 4242	15	Bitmap Heap Scan, 0.13 ms
customer_id between 4242 and 4400	2385	Bitmap Heap Scan, 11.9 ms
status = 'NEW' (5 distinct values)	300 000 (20 %)	Seq Scan, 173 ms
customer_id < 40000	599 985 (40 %)	Bitmap Heap Scan, 176 ms
No predicate	1 500 000	Parallel Seq Scan, 250 to 440 ms

Same table, growing result set (measured, typical).

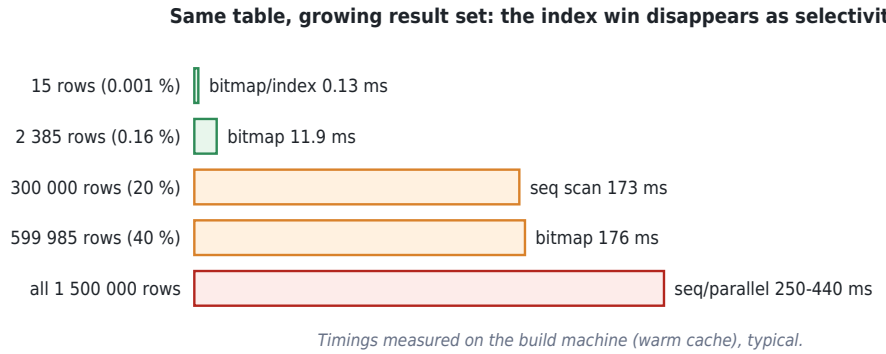


Figure 1.67. The index advantage shrinks as the result set grows: past a few percent of the table the planner reads sequentially.

An index on a column with few distinct values, such as a boolean or a five-value status, is almost never used by itself. It becomes valuable as the **leading column of a composite index**, as a **partial index** for the rare value, or when the data is skewed and the query asks for the rare value. The statistics that let the planner recognise skew are described in the chapter on the query planner.

The cost of an index

Every index must be updated by every `INSERT`, by every `DELETE` (at vacuum time) and by every `UPDATE` that changes an indexed column or cannot use a HOT update. The cost was measured by inserting 500 000 rows into otherwise identical unlogged tables:

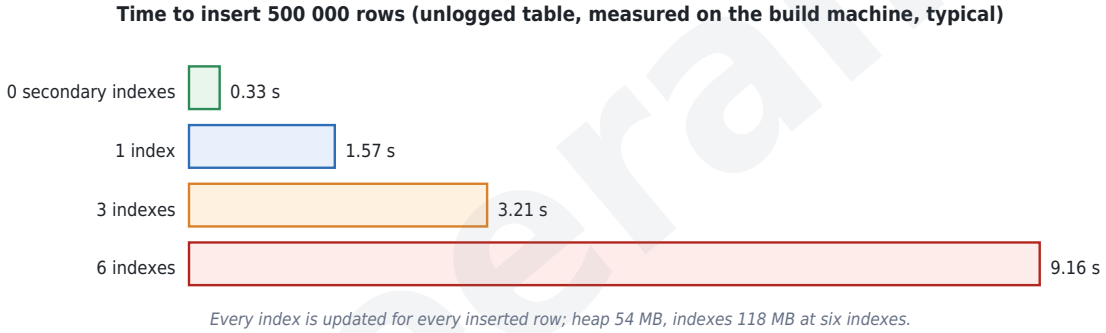


Figure 1.68. Inserting 500 000 rows grows from 0.33 s with no secondary index to 9.16 s with six indexes, and the six indexes occupy 118 MB against a 54 MB table.

An index also consumes memory (its hot pages compete with table pages in the buffer pool), disk and vacuum time. Updates of an indexed column create a second effect, **index bloat**. A table of 300 000 rows with an index on `amount` was updated three times, each time changing `amount` for all rows. The index grew from 6.6 MB to 39 MB; `VACUUM` removed the dead entries but left the size at 39 MB and reduced the leaf density to 15 percent; only `REINDEX` restored 6.6 MB.

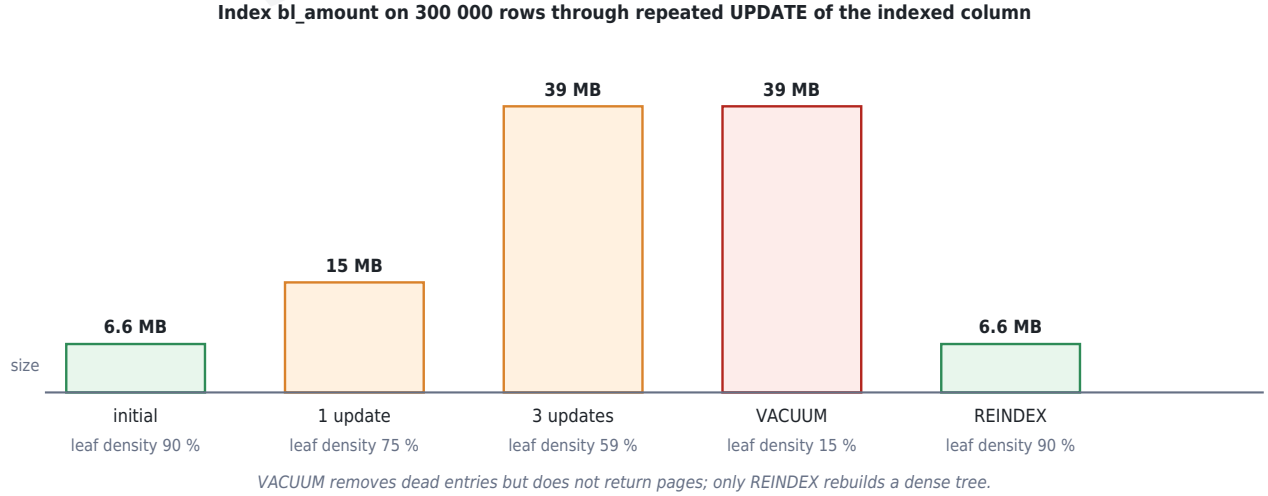


Figure 1.69. Repeated updates of an indexed column grow the index from 6.6 MB to 39 MB; VACUUM frees entries but not pages, and REINDEX rebuilds a dense index.

Sreeram

```sql
SELECT pg_size_pretty(pg_relation_size('bl_amount')) AS idx,
       (pgstatindex('bl_amount')).avg_leaf_density AS density;
REINDEX INDEX CONCURRENTLY bl_amount;   -- PostgreSQL 12 or later; no exclusive lock
```

An index that exists but is never used still costs every write. PostgreSQL reports usage in `pg_stat_user_indexes.idx_scan`; an index whose `idx_scan` stays 0 over a representative period (and which does not enforce a constraint) is a candidate for removal.

```sql
SELECT relname, indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE idx_scan = 0 AND indexrelname NOT LIKE '%pkey'
ORDER BY pg_relation_size(indexrelid) DESC;
```

MySQL InnoDB differences

Topic	PostgreSQL 16	MySQL 8.x InnoDB
Default structure	B-tree on a heap	B+tree; table itself is the primary-key B+tree
Secondary entry	Key plus ctid	Key plus primary key value
Covering index	`INCLUDE (cols)`	Add columns to the key; primary key is implicit
Partial index	Yes (`WHERE`)	No
Expression index	Yes	Functional key part (8.0.13 and later)
Descending key parts	Yes	Yes (8.0 and later; earlier versions ignored `DESC`)
Index hidden from planner	Not built in (use a test transaction or `hypopg`)	`ALTER TABLE ... ALTER INDEX x INVISIBLE`
Hash index	Yes, WAL-logged since 10	Only in the MEMORY and NDB engines; InnoDB adaptive hash is internal
Create without blocking writes	`CREATE INDEX CONCURRENTLY`	Online DDL (`ALGORITHM=INPLACE, LOCK=NONE`) is the default for most index builds
Maximum key length	About one third of a page (btree row version limit 2704 bytes)	3072 bytes (DYNAMIC row format)
Full-text and `LIKE '%x%'`	GIN with `tsvector` or `pg_trgm`	`FULLTEXT` index; `LIKE '%x%'` cannot use B+tree

Index behaviour in PostgreSQL 16 and MySQL 8.x (MySQL column documented, not run on the build machine).

1.15.2 Worked example

Choosing indexes for a real workload

Three queries dominate the order service:

1. `SELECT * FROM orders WHERE customer_id = ? ORDER BY created_at DESC LIMIT 20` (customer order history).
2. `SELECT count(*), sum(amount) FROM orders WHERE created_at >= ? AND created_at < ?` (daily report).
3. `SELECT id FROM orders WHERE status = 'NEW' AND created_at < now() - interval '1 hour'` (worker queue; 20 percent of rows are `NEW`, but a worker should see only a fraction).

Step 1: query 1 compares `customer_id` for equality and orders by `created_at`, so `(customer_id, created_at)` delivers rows already sorted, and `LIMIT 20` stops after 20 entries. The index is declared `(customer_id, created_at DESC)` only when the mixed direction is needed; PostgreSQL scans a plain ascending index backward at no cost.

Step 2: query 2 needs `created_at` first, and `amount` is read from each row. A covering index `(created_at) INCLUDE (amount)` yields an index-only scan; without it the plan reads the range from the index and fetches heap pages. Because the workload appends in time order the heap correlation is 1.0, so even the non-covering variant reads consecutive pages.

Step 3: the predicate selects a small, changing subset. A partial index `ON orders (created_at) WHERE status = 'NEW'` is small (6.6 MB), always hot in memory, and cheap to maintain because rows leave it once their status changes.

Step 4: remove the duplicates. The single-column index on `customer_id` is a prefix of the index of step 1 and is dropped; `(created_at)` is covered by the index of step 2.

The resulting set has three indexes plus the primary key instead of the six that each query would have suggested alone, and each exists for a named query.

1.15.3 Hands-on

SQL

```sql
-- 1. baseline: no index on customer_id
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 4242;
-- 2. create it, and compare
CREATE INDEX idx_orders_customer ON orders (customer_id);
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 4242;
```

Before and after (real, trimmed) TERMINAL

```
$ psql -d shop
 Gather  (cost=1000.00..29648.00 rows=15 width=77)
   (actual time=10.290..120.229 rows=15 loops=1)
   ->  Parallel Seq Scan on orders  (cost=0.00..28646.50 rows=6 width=77)
         Filter: (customer_id = 4242)
         Rows Removed by Filter: 499995
 Execution Time: 120.264 ms

 Bitmap Heap Scan on orders  (cost=4.54..63.52 rows=15 width=77)
   (actual time=0.029..0.116 rows=15 loops=1)
   Recheck Cond: (customer_id = 4242)
   Heap Blocks: exact=15
   ->  Bitmap Index Scan on idx_orders_customer  (cost=0.00..4.54 rows=15 width=0)
 Execution Time: 0.128 ms
```

The query changed from 120 ms to 0.13 ms, and the index occupies 12 MB. The next listing shows a partial and an expression index, with the corresponding plans.

SQL

```sql
CREATE INDEX idx_orders_new_created ON orders (created_at) WHERE status = 'NEW';
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF)
SELECT count(*) FROM orders WHERE status = 'NEW' AND created_at >= '2024-12-01';

CREATE INDEX idx_cust_lower_email ON customers (lower(email));
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF)
SELECT * FROM customers WHERE lower(email) = 'user777@example.com';
```

Plans (real) TERMINAL

```
$ psql -d shop
 Aggregate (actual rows=1 loops=1)
   ->  Index Only Scan using idx_orders_new_created on orders (actual rows=10561 loops=1)
         Index Cond: (created_at >= '2024-12-01 00:00:00+00'::timestamp with time zone)
         Heap Fetches: 0
 Execution Time: 1.420 ms

 Bitmap Heap Scan on customers (actual rows=1 loops=1)
   Recheck Cond: (lower(email) = 'user777@example.com'::text)
   ->  Bitmap Index Scan on idx_cust_lower_email (actual rows=1 loops=1)
 Execution Time: 0.036 ms
```

1.15.4 Real-world problems and solutions

Incident 3: a composite index in the wrong column order

Scenario. The reporting team of an order system created an index `(customer_id, created_at)` to speed up a monthly customer report, and the report became fast. A second report, "all orders of a given day for any customer", remained slow, although `created_at` is "in an index".

Symptoms. The daily report takes 80 ms on 1.5 million rows and grows linearly with the table; CPU on the database rises when it runs in parallel with the day-end batch.

Diagnosis.

```sql
EXPLAIN (ANALYZE, COSTS OFF)
SELECT count(*) FROM orders
WHERE created_at >= '2024-06-01' AND created_at < '2024-06-02';
```

Plan (real, trimmed) TERMINAL

```
$ psql -d shop
 Finalize Aggregate (actual time=79.186..80.055 rows=1 loops=1)
   ->  Gather
         ->  Partial Aggregate
               ->  Parallel Seq Scan on orders (actual time=35.062..71.522 rows=1440 loops=3)
                     Filter: ((created_at >= ...) AND (created_at < ...))
                     Rows Removed by Filter: 498560
 Execution Time: 80.076 ms
```

The plan shows a sequential scan: the index `(customer_id, created_at)` cannot seek on `created_at` because it is sorted by `customer_id` first.

Root cause. Violation of the leftmost-prefix rule. The matching entries are scattered across the index and the planner correctly rejects a full index scan.

Fix. Create an index with `created_at` as the leading column. `CREATE INDEX CONCURRENTLY idx_created_cust ON orders (created_at, customer_id);` produced an `Index Only Scan` with `Heap Fetches: 0` in 0.73 ms (a 110-fold improvement). Keep both indexes only if both access paths are needed; otherwise a single index on `(created_at)` suffices for the day report and the customer report keeps `(customer_id, created_at)`.

Prevention. For each index, write down the query it serves and its leading column. Review composite indexes with `\d table` and `pg_stat_user_indexes`. Test every new query with `EXPLAIN` against production-sized data.

Incident 4: too many indexes and index bloat slow the writes

Scenario. Over two years developers added an index for each slow ticket. The `orders` table has nine indexes (an illustrative scenario). The nightly import of 500 000 rows, which took 40 seconds, now takes 8 minutes, and the largest index is 2.5 times the size of its table.

Symptoms. `INSERT` and `UPDATE` latency is high and rises with data volume while reads are fast; disk usage grows faster than row count; autovacuum runs constantly.

Diagnosis. Measure write amplification and look for unused or oversized indexes.

```sql
SELECT indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid)) AS size,
       (pgstatindex(indexrelid::regclass)).avg_leaf_density AS density
FROM pg_stat_user_indexes
WHERE relname = 'orders' AND indexrelname NOT LIKE '%pkey'
ORDER BY pg_relation_size(indexrelid) DESC;
```

The reference experiment (500 000 rows into an unlogged table) showed time rising with the index count: 0.33 s with none, 1.57 s with one, 3.21 s with three and 9.16 s with six. An index with `idx_scan = 0` and a density below 60 percent is a removal or rebuild candidate.

Root cause. Every index adds a B-tree insertion per row, plus WAL volume and cache pressure; updates of an indexed column, as in the bloat experiment above (6.6 MB to 39 MB in three updates), leave dead entries that `VACUUM` marks reusable but does not return.

Fix. Immediate: `DROP INDEX CONCURRENTLY` on unused and duplicate indexes (after confirming that no constraint or foreign key depends on them) and `REINDEX INDEX CONCURRENTLY` on bloated ones. For bulk loads drop secondary indexes, load, then create them (a single sort-based build is much faster than millions of insertions). Permanent: lower `fillfactor` on frequently updated tables (for example 90) so that HOT updates fit on the page, avoid indexing frequently updated columns, and consolidate indexes with `INCLUDE` or partial predicates.

Prevention. Require a named query for every index in code review; schedule a quarterly review of unused indexes; monitor index size against table size.

Sreeram

1.15.5 Questions to expect

INTERVIEW ANGLE
How does a B+tree index find a row, and why is the cost logarithmic? The search descends from the root through internal pages by comparing separator keys to a leaf page that holds the key and a row pointer, one page read per level. Because pages are wide (hundreds of entries) the height is 3 or 4 for hundreds of millions of rows. Range scans follow leaf-to-leaf links.

1. Why is a B+tree preferred over a binary tree on disk? A binary tree has a height of about log2(n) with one key per node, so each level costs a disk read; a B+tree packs hundreds of keys per page, cutting the height to 3 or 4.
2. What is the leftmost-prefix rule? An index on (a, b, c) can seek only if the condition constrains a, then optionally b, then c; a condition on b alone cannot seek because entries with the same b are scattered.
3. When does the optimiser ignore an index? When it expects to read a large fraction of the table (low selectivity), when the predicate is not sargable (function on the column, type mismatch), when statistics are stale, or when the table is small.
4. What is a covering index? An index that contains all columns a query needs so that the heap is not read (index-only scan; PostgreSQL needs a current visibility map).
5. What is a partial index and when is it useful? An index on a subset of rows defined by a WHERE clause; it is small and cheap, for example only unprocessed rows of a queue table.
6. What happens in a clustered index on a secondary lookup? The secondary entry yields the primary key and a second descent of the clustered index fetches the row, so a short primary key keeps secondary indexes small.
7. How do indexes affect write performance? Each insert, delete and key-changing update modifies every index of the table, adds WAL and defeats HOT updates; measured here as 0.33 s versus 9.16 s for 500 000 rows with zero versus six indexes.
8. Why is a hash index rarely better than a B-tree? It supports equality only and had no WAL logging until PostgreSQL 10; in a measured case it was 46 MB against a 12 MB deduplicated B-tree.
9. Explain a page split. A full page cannot take an entry; a new page is allocated, entries are redistributed, a separator is added to the parent, and the parent splits recursively; if the root splits, the tree gains a level.
10. How would you add an index to a production table without blocking writes? In PostgreSQL `CREATE INDEX CONCURRENTLY` (two table scans, does not run inside a transaction block); in MySQL online DDL with `LOCK=NONE`.

KEY POINTS
- A B+tree has wide pages, three or four levels for very large tables, and linked leaves; lookups cost about one page per level.
- Leaf splits grow the tree upward; right-edge inserts fill leaves to 90 percent, random inserts to about 70 percent.
- Composite indexes sort by the first column first: equality columns first, range column last, and no seek without the leading column.
- Covering, partial and expression indexes reduce I/O and size, but index-only scans depend on the visibility map and expressions must be immutable.
- Low-selectivity predicates make the planner read the table; skewed or composite use is needed to benefit.
- Every index costs write time, memory and bloat; measure with `pg_stat_user_indexes` and remove what no query uses.

1.16 Reading Query Plans

A SQL statement states what result is wanted, not how to compute it. The database converts the statement into a **query plan**, a tree of operators that fetch, join, sort and aggregate rows, and then executes that tree. When a query is slow, the plan is the evidence: it shows which access path was chosen, how many rows the planner expected and how many occurred, how much time and how many pages each step consumed, and where the time went. Reading plans fluently is the central diagnostic skill of this part. This chapter defines the notation of PostgreSQL's `EXPLAIN`, describes every common node type with real output from the 1.5 million row `orders` table, and lists the differences in MySQL's `EXPLAIN`.

1.16.1 Concept and mechanism

EXPLAIN, EXPLAIN ANALYZE and BUFFERS

`EXPLAIN` shows the plan the planner would choose and its estimates, without running the statement. `EXPLAIN ANALYZE` executes the statement and adds actual rows, actual time and loop counts. Because it really runs, a data-modifying statement is changed by it; wrap it in a transaction and roll back:

```sql
BEGIN;
EXPLAIN (ANALYZE, BUFFERS) UPDATE orders SET status = 'PAID' WHERE id = 1;
ROLLBACK;
```

The option `BUFFERS` adds page counts: `shared hit` (found in `shared_buffers`), `shared read` (requested from the operating system), `dirtied` and `written`, and `temp read` and `temp written` for spills to disk. Pages are the most reliable cost measure, because they do not vary with machine load. PostgreSQL 18 prints buffers by default with `ANALYZE`; in PostgreSQL 16 the option must be given. Other useful options are `VERBOSE` (output columns), `SETTINGS` (non-default parameters), `WAL` (log volume), `COSTS OFF` and `TIMING OFF` (stable, shorter output, and cheaper measurement of many-loop plans), and `FORMAT JSON` (for tools such as plan visualisers).

Anatomy of a plan node

Every line of the plan is a node. Children are indented below the parent, and rows flow from the leaves upward to the root. The notation is the same for all nodes:

```
Index Scan using orders_pkey on orders o  (cost=0.43..8.45 rows=1 width=77)
     (actual time=0.046..0.046 rows=1 loops=1)
  Index Cond: (id = 777777)
  Buffers: shared hit=3 read=4
```

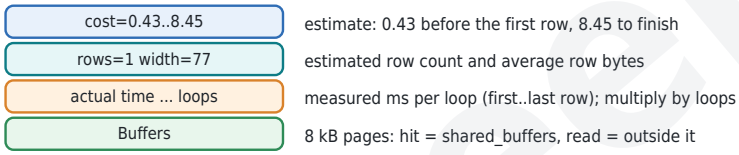


Figure 1.70. The parts of one plan node: estimated cost range, estimated rows and width, measured time per loop, loop count, conditions and buffer counts.

- `cost=0.43..8.45` gives **startup cost** (spent before the first row can be returned) and **total cost**, in abstract planner units where one sequential page read costs 1.0. They are not milliseconds. A node's cost includes its children.
- `rows=1` is the estimated number of rows that the node returns *per execution*; `width` is the average row size in bytes.
- `actual time=0.046..0.046` are milliseconds to the first and to the last row, *per loop*; `rows=1` is the actual rows *per loop*; `loops=1` is how many times the node ran. The total time of a node is `actual time` (last row) multiplied by `loops`, and the total rows are `rows` times `loops`. Parallel workers are reported as loops as well.
- Condition lines name the work: `Index Cond` (used to search the index), `Filter` (evaluated on each row after fetching; `Rows Removed by Filter` counts the discards), `Recheck Cond` (re-evaluation after a bitmap scan), `Join Filter`, `Hash Cond`, `Merge Cond`.

The first thing to compare in any plan is **estimated rows against actual rows**. A difference of ten times or more at any node means that the planner chose the plan from wrong information and is the most common cause of a bad plan (see the chapter on the planner).

Scan nodes

A scan node reads a base table. The same predicate `customer_id = 4242` on `orders` gives different nodes depending on what exists.

sreeram

Seq Scan: no index (real) TERMINAL

```
$ psql -d shop -f by_customer.sql
 Gather  (cost=1000.00..29648.00 rows=15 width=77) (actual time=10.290..120.229 rows=15 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=352 read=20482
   ->  Parallel Seq Scan on orders  (cost=0.00..28646.50 rows=6 width=77)
         (actual time=18.898..114.796 rows=5 loops=3)
         Filter: (customer_id = 4242)
         Rows Removed by Filter: 499995
         Buffers: shared hit=352 read=20482
 Execution Time: 120.264 ms
```

A **Seq Scan** reads the whole heap. `Rows Removed by Filter: 499995` per worker (three workers including the leader, `loops=3`) shows how many rows were read and discarded. A **Parallel Seq Scan** under a **Gather** divides the pages among workers; `rows=5 loops=3` is 15 rows in total.

Index Scan on the primary key (real) TERMINAL

```
$ psql -d shop -f by_id.sql
 Index Scan using orders_pkey on orders  (cost=0.43..8.45 rows=1 width=77)
   (actual time=0.046..0.046 rows=1 loops=1)
   Index Cond: (id = 777777)
   Buffers: shared hit=4
```

An **Index Scan** traverses the index and fetches each matching heap tuple (here 4 buffers: three index levels and one heap page, in the warm run). Cost 8.45 against 29 648 for the scan above is the planner's expression of the same fact.

Bitmap Heap Scan (real) TERMINAL

```
$ psql -d shop -f by_customer.sql
 Bitmap Heap Scan on orders  (cost=4.54..63.52 rows=15 width=77)
   (actual time=0.029..0.116 rows=15 loops=1)
   Recheck Cond: (customer_id = 4242)
   Heap Blocks: exact=15
   Buffers: shared read=18
   ->  Bitmap Index Scan on idx_orders_customer  (cost=0.00..4.54 rows=15 width=0)
         (actual time=0.020..0.020 rows=15 loops=1)
         Index Cond: (customer_id = 4242)
         Buffers: shared read=3
 Execution Time: 0.128 ms
```

The **Bitmap Index Scan** builds a bitmap of qualifying heap pages (and tuples); the **Bitmap Heap Scan** above it visits those pages once in physical order. `Heap Blocks: exact=15` means the bitmap kept one bit per tuple; `lossy=N` means that `work_mem` was too small to keep tuple precision, so whole pages were rechecked (`Rows Removed by Index Recheck` counts the false candidates). Bitmaps can be combined with `BitmapAnd` and `BitmapOr`, which is how `WHERE a = 1 OR b = 2` uses two separate indexes.

An **Index Only Scan** shows `Heap Fetches: N`; the closer to 0, the better.

Join nodes

PostgreSQL has three join algorithms. The planner chooses by estimated sizes, available indexes, sort order and memory.

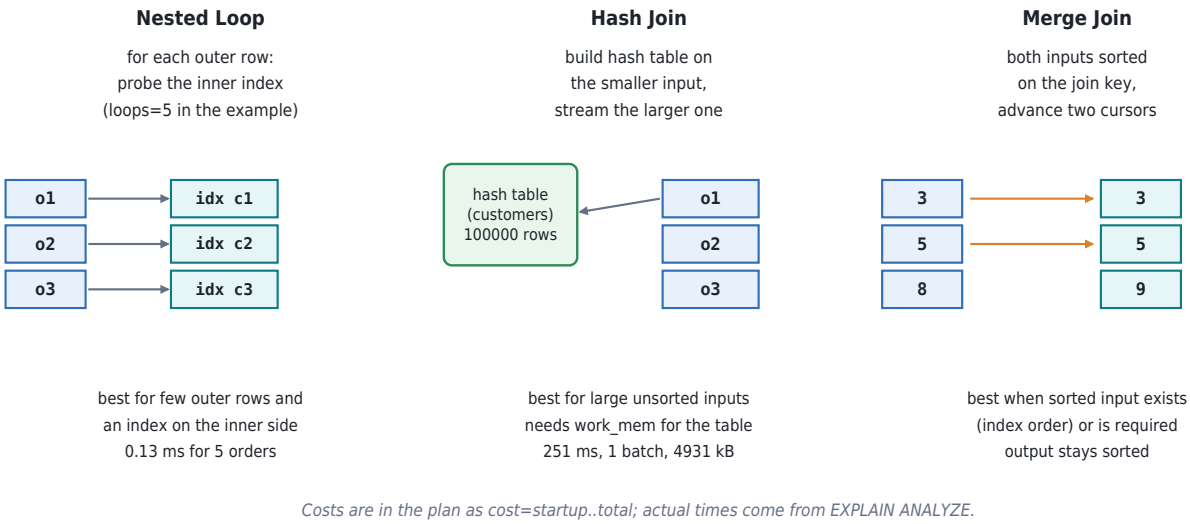


Figure 1.71. Nested loop probes an inner index once per outer row, hash join builds a hash table on one input and streams the other, and merge join walks two sorted inputs in step.

Nested Loop. For each row of the outer input, the inner input is executed (usually an index lookup). Cost is outer rows times inner cost; excellent when the outer side is small, catastrophic when it is large and the inner side is not indexed.

SQL

```sql
EXPLAIN (ANALYZE)
SELECT o.id, c.name FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.id BETWEEN 1000 AND 1004;
```

Nested Loop (real) TERMINAL

```
$ psql -d shop -f nested.sql
 Nested Loop  (cost=0.72..50.08 rows=5 width=22) (actual time=0.039..0.112 rows=5 loops=1)
   ->  Index Scan using orders_pkey on orders o  (cost=0.43..8.53 rows=5 width=12)
         (actual time=0.022..0.024 rows=5 loops=1)
         Index Cond: ((id >= 1000) AND (id <= 1004))
   ->  Index Scan using customers_pkey on customers c  (cost=0.29..8.31 rows=1 width=18)
         (actual time=0.016..0.016 rows=1 loops=5)
         Index Cond: (id = o.customer_id)
 Execution Time: 0.132 ms
```

The inner scan shows `loops=5` and `rows=1`: it ran five times returning one row each time, five rows in all.

Hash Join. The second (inner) input, preferably the smaller one, is read completely into an in-memory hash table keyed by the join column (the `Hash` node), then the outer input is streamed and probed. It needs no index and no sort and handles large inputs, but it is limited by `work_mem`: if the hash table exceeds it, the join runs in several **batches** spilled to temporary files (`Batches: 4`, with `temp read` and `temp written` buffers).

Merge Join. Both inputs must be sorted on the join key (by an index or an explicit `Sort`). Two cursors advance together. It is attractive when sorted inputs are free (index order) and for very large joins, and its output is sorted on the join key.

SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.country, count(*), sum(o.amount)
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.created_at >= '2024-12-01'
GROUP BY c.country ORDER BY 3 DESC;
```

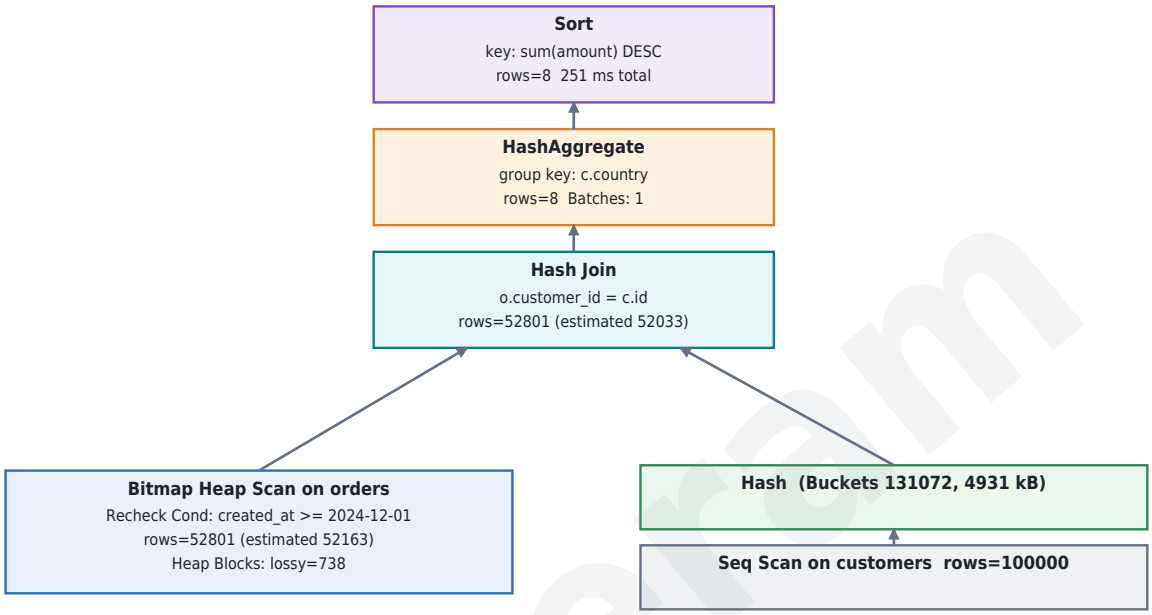
Hash Join under an aggregate (real, trimmed) TERMINAL

```
$ psql -d shop -f revenue.sql
 Sort  (cost=25260.83..25260.85 rows=8 width=43) (actual time=250.982..250.986 rows=8 loops=1)
   Sort Key: (sum(o.amount)) DESC
   Sort Method: quicksort  Memory: 25kB
   ->  HashAggregate  (cost=25260.61..25260.71 rows=8 width=43) (actual rows=8 loops=1)
         Group Key: c.country
         Batches: 1  Memory Usage: 24kB
```

Hash Join under an aggregate (real, trimmed) (continued) TERMINAL

```
        ->  Hash Join  (cost=3209.23..24870.36 rows=52033 width=9) (actual rows=52801 loops=1)
              Hash Cond: (o.customer_id = c.id)
              ->  Bitmap Heap Scan on orders o  (cost=25.23..21549.42 rows=52163 width=10)
                    Recheck Cond: (created_at >= '2024-12-01 00:00:00+00'::timestamptz)
                    Heap Blocks: lossy=738
                    ->  Bitmap Index Scan on idx_created_brin  (actual rows=7380 loops=1)
              ->  Hash  (cost=1934.00..1934.00 rows=100000 width=7) (actual rows=100000 loops=1)
                    Buckets: 131072  Batches: 1  Memory Usage: 4931kB
                    ->  Seq Scan on customers c  (actual rows=100000 loops=1)
Execution Time: 251.043 ms
```

Plan tree: December revenue by country (rows flow from the leaves up to the root)



Probe side (left child) streams; build side (right child) is hashed first.

Figure 1.72. The plan above as a tree: the orders scan is the probe side and the hashed customers table the build side; estimates (52 033, 52 163) are close to the actual rows (52 801).

To read a plan, find the **innermost nodes** and the most expensive subtree. Here the estimates are close to reality at every node, the hash table needs 4931 kB and fits in `work_mem`, and the sorted result has 8 rows. The time is spent in building the hash table of all 100 000 customers and in the join itself.

Sort, aggregate, limit

A **Sort** node reports its method: `quicksort Memory: 25kB` (in memory), `top-N heapsort` (a `LIMIT` keeps only the best N rows, so memory stays small) or `external merge Disk: 133304kB` (spilled to temporary files because the data exceeded `work_mem`). On `orders`, `ORDER BY amount DESC OFFSET 1499990` with the default `work_mem = 4MB` produced `external merge Disk: 133304kB` and `temp read=33323 written=33372`; with `work_mem = 256MB` the same sort ran as `quicksort Memory: 206751kB` without temporary I/O.

ORDER BY amount over 1 500 000 rows: sort memory vs spill (EXPLAIN ANALYZE)



Plan cost 326 759 (spill) vs 193 458 (memory); the planner also prices spills.

Figure 1.73. The same sort of 1.5 million rows spills 133 MB to disk with a 4 MB work_mem and runs in 207 MB of memory with 256 MB; the planner prices the spill (cost 326 759 against 193 458).

An **Aggregate** is `Aggregate` (single group), `HashAggregate` (hash table per group; `Batches` above 1 means spilling since PostgreSQL 13) or `GroupAggregate` (requires sorted input, memory-free). A **Limit** node stops pulling rows from its child as soon as it has enough, so the plan `Limit -> Index Scan` for `ORDER BY created_at LIMIT 20` read 4 buffers

and ran in 0.047 ms while the estimated total cost of its child (93 764) was never paid. When a `Limit` node's child shows a large `actual rows` (for example 1 000 020 for an `OFFSET 1000000`) the rows were generated and discarded.

Parallel query

Large scans and aggregates can be divided among background workers. A `Gather` (or `Gather Merge`) node collects their output. Summing `amount` over 1.5 million rows took 442 ms with `max_parallel_workers_per_gather = 0` and 252 ms with 2 workers on the build machine's two CPUs. Parallelism is only planned when the table is larger than `min_parallel_table_scan_size` (8 MB) and the estimated gain exceeds `parallel_setup_cost`; it multiplies CPU and I/O use per query and so is not a substitute for an index in a high-concurrency OLTP system. The text of this part sometimes sets `max_parallel_workers_per_gather = 0` for clarity and states so.

MySQL EXPLAIN

MySQL prints a table with one row per table accessed in the join order, rather than a tree. `EXPLAIN FORMAT=TREE` (8.0.16 and later) and `EXPLAIN ANALYZE` (8.0.18 and later) give a tree with actual timings and loops similar to PostgreSQL's.

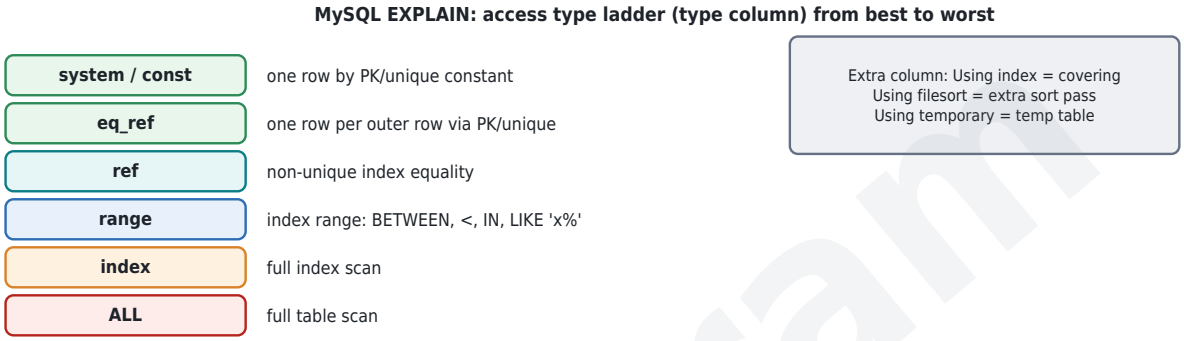


Figure 1.74. The access type column of MySQL EXPLAIN, ordered from the best (const) to the worst (ALL), and the three most important Extra flags.

MySQL 8.x EXPLAIN (typical output, not run on the build machine) TERMINAL

```
$ psql -d shop
mysql> EXPLAIN SELECT * FROM orders WHERE customer_id = 4242\G
           id: 1
  select_type: SIMPLE
        table: orders
   partitions: NULL
         type: ref
possible_keys: idx_orders_customer
          key: idx_orders_customer
      key_len: 4
          ref: const
         rows: 15
     filtered: 100.00
        Extra: NULL
```

Question	PostgreSQL 16	MySQL 8.x
Is an index used?	Node name: Index Scan, Bitmap, Index Only Scan	`type` not ALL, `key` not NULL
Full table scan	Seq Scan	`type = ALL`
Covering index	Index Only Scan with Heap Fetches near 0	`Extra: Using index`
Extra sort	Sort node (see Sort Method)	`Extra: Using filesort`
Temporary table	Materialize, HashAggregate on disk	`Extra: Using temporary`
Rows examined	Rows Removed by Filter plus rows	`rows` times `filtered` (estimate); `EXPLAIN ANALYZE` actual
Actual execution	EXPLAIN ANALYZE	EXPLAIN ANALYZE (8.0.18 and later)
Join algorithms	Nested loop, hash, merge	Nested loop (index or block), hash join (8.0.18 and later); no merge join

Reading a plan in PostgreSQL and MySQL.

1.16.2 Worked example

Reading a plan line by line

Consider the query of the previous section, now with real counts, and trace its execution in order.

1. `Seq Scan on customers` returns 100 000 rows (estimate 100 000). The scan reads 934 pages of customers, taking 50 ms in the cold run.
2. `Hash` builds a table of 100 000 entries in 131 072 buckets using 4931 kB, in one batch. Its time (131 ms) includes the scan below it: a node's time includes its children.
3. `Bitmap Index Scan on idx_created_brin` identifies 738 candidate pages (a BRIN index stores page ranges, so the bitmap is lossy). Estimate 55 215 rows, actual 7380 entries of the bitmap.
4. `Bitmap Heap Scan` reads the 738 pages (about 5.7 MB), rechecks `created_at >= '2024-12-01'` and returns 52 801 rows (estimate 52 163).
5. `Hash Join` probes each of the 52 801 order rows against the hash table and returns 52 801 joined rows (estimate 52 033) in 216 ms total.
6. `HashAggregate` reduces them to 8 groups (the countries) using 24 kB.
7. `Sort` orders the 8 groups by the sum; the total is 251 ms.

Where was the time spent? Subtracting children: the aggregation adds about 35 ms, the join itself about 59 ms beyond its two inputs, the hash build 131 ms (of which 50 ms is the customers scan and the remainder loading the table and I/O waits) and the heap scan 26 ms. A possible improvement is to aggregate before joining, because only 8 countries are needed, which removes the 52 801 probes; the optimisation chapter returns to this style of rewrite.

1.16.3 Hands-on

Plans change with data, so experiment on a table of realistic size. Create the objects used in this part with a script similar to the following; every plan shown in this part was produced from it.

SQL

```
CREATE TABLE customers(id int PRIMARY KEY, name text NOT NULL, email text NOT NULL,
  country text NOT NULL);
INSERT INTO customers
SELECT g, 'Customer ' || g, 'user' || g || '@example.com',
       (ARRAY['IN','US','DE','UK','FR','BR','JP','AU'])[1 + (g % 8)]
FROM generate_series(1, 100000) g;

CREATE TABLE orders(
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id int NOT NULL, status text NOT NULL, amount numeric(10,2) NOT NULL,
  created_at timestamptz NOT NULL, note text);
INSERT INTO orders(customer_id, status, amount, created_at, note)
SELECT 1 + ((g::bigint * 7919) % 100000),
       (ARRAY['NEW','PAID','SHIPPED','DELIVERED','CANCELLED'])[1 + (g % 5)],
       round((random() * 500)::numeric, 2),
       timestamptz '2024-01-01' + (g * interval '20 seconds'),
       'order note ' || md5(g::text)
FROM generate_series(1, 1500000) g;
ANALYZE;
```

Then try the three views of the same query and compare:

SQL

```
EXPLAIN SELECT * FROM orders WHERE id = 777777;                    -- estimate only
EXPLAIN ANALYZE SELECT * FROM orders WHERE id = 777777;            -- executed
EXPLAIN (ANALYZE, BUFFERS, SETTINGS) SELECT * FROM orders WHERE id = 777777;
```

Run the second and third a few times: the first run reports `shared read` and the later runs `shared hit`, which is the difference between a cold and a warm cache and the reason a single timing proves little.

1.16.4 Real-world problems and solutions

Incident 5: the plan hides a spill to disk

Scenario. A monthly export sorts all orders by amount. After the table grew to 1.5 million rows it takes about three seconds (3.1 s measured on the build machine), although the plan shape did not change.

Symptoms. CPU is low, disk write traffic is high during the query, and `/var/lib/postgresql` temporary space grows by hundreds of megabytes.

Diagnosis.

SQL

```sql
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders ORDER BY amount DESC;
```

Plan (real, trimmed) TERMINAL

```
$ psql -d shop
 Gather Merge  (cost=143796.20..289639.72 rows=1250000 width=77)
   (actual time=1281.600..3056.729 rows=1500000 loops=1)
   Buffers: shared hit=947 read=19999, temp read=29218 written=29265
   ->  Sort  (cost=142796.18..144358.68 rows=625000 width=77) (actual rows=500000 loops=3)
         Sort Key: amount DESC
         Sort Method: external merge  Disk: 43480kB
         Worker 0:  Sort Method: external merge  Disk: 56920kB
         Worker 1:  Sort Method: external merge  Disk: 33104kB
 Execution Time: 3117.212 ms
```

`Sort Method: external merge Disk: ...` and `temp read/written` in the buffer line identify the spill.

Root cause. Each sort exceeds `work_mem` (4 MB by default) and sorts in runs written to temporary files, then merges them.

Fix. Immediate: raise `work_mem` for this session or transaction only (`SET LOCAL work_mem = '256MB'`). With 256 MB the single-process sort ran as `quicksort Memory: 206751kB`. Permanent: avoid sorting the entire table (export in key order from an index, stream with a cursor, or use `LIMIT` with keyset pagination); size `work_mem` globally with care, because it is allocated per sort or hash node *per query*, so a global value of 256 MB with 200 connections can consume the whole memory of the machine.

Prevention. Monitor `temp_files` and `temp_bytes` in `pg_stat_database`, and set `log_temp_files = 0` (or a threshold) to log every spill with the statement that caused it.

1.16.5 Questions to expect

INTERVIEW ANGLE

What is the difference between EXPLAIN and EXPLAIN ANALYZE? EXPLAIN shows the planned tree with estimated cost and rows without running the statement. EXPLAIN ANALYZE executes it and adds actual time, actual rows and loops per node; with BUFFERS it adds page counts. Because it executes, wrap data-changing statements in a transaction that is rolled back.

1. What does `cost=0.43..8.45` mean? Estimated startup cost and total cost in planner units (one sequential page fetch is 1.0), not milliseconds.
2. How do you compute the total time of a node with `loops=5`? Multiply the per-loop actual time and rows by the loop count.
3. When is a nested loop the right join? When the outer input is small and the inner input has an index on the join column; it is a poor choice for large unsorted inputs.
4. What does `Batches: 4` in a Hash node indicate? The hash table did not fit in `work_mem` and the join was split into four batches spooled to temporary files.
5. What is the difference between `Filter` and `Index Cond`? `Index Cond` restricts the entries read from the index; `Filter` is checked on rows already fetched, so many `Rows Removed by Filter` means wasted reads.
6. How can an estimate error show up in a plan? The `rows=` estimate differs greatly from `actual rows=` at some node, often causing a wrong join algorithm or order.
7. What is an Index Only Scan with `Heap Fetches` greater than 0? A scan that still visited the heap for pages not marked all-visible; running VACUUM brings it toward 0.

8. How does MySQL show a full table scan? `type = ALL` in EXPLAIN; `Using filesort` marks an extra sort and `Using index` a covering index.

KEY POINTS

- EXPLAIN ANALYZE with BUFFERS gives estimates, actual rows and times, and page counts; pages are the stable measure.
- Read plans from the innermost nodes up, multiply per-loop figures by `loops`, and compare estimated with actual rows at every node.
- Scans: Seq, Index, Bitmap, Index Only; joins: Nested Loop, Hash, Merge; each has a situation where it wins.
- `external merge Disk`, `Batches` above 1 and `temp read` mean a spill past `work_mem`.
- MySQL EXPLAIN lists tables with `type`, `key`, `rows` and `Extra`; `EXPLAIN ANALYZE` gives a tree with timings.

1.17 The Query Planner

The planner (optimiser) turns a parse tree into an execution plan by enumerating alternative plans, estimating the cost of each and choosing the cheapest. Its estimates are only as good as its **statistics**, the compact summary of each column that the database keeps. Nearly every case of "the database chose a stupid plan" is one of three things: the statistics are stale, the statistics cannot express the data (correlated columns, skew), or the query hides the information from the planner (a function on the column, a type mismatch). This chapter explains what the planner knows, how it converts that knowledge to numbers, and how to make it choose better.

1.17.1 Concept and mechanism

From SQL to plan

A statement passes through four stages. The **parser** checks syntax and builds a parse tree. The **rewriter** expands views and rules. The **planner** generates paths for each table (sequential scan, each usable index, bitmap), generates join orders and algorithms, costs every combination (an exhaustive search for fewer than `geqo_threshold` = 12 relations, a genetic heuristic beyond) and keeps the cheapest. The **executor** runs the chosen tree.

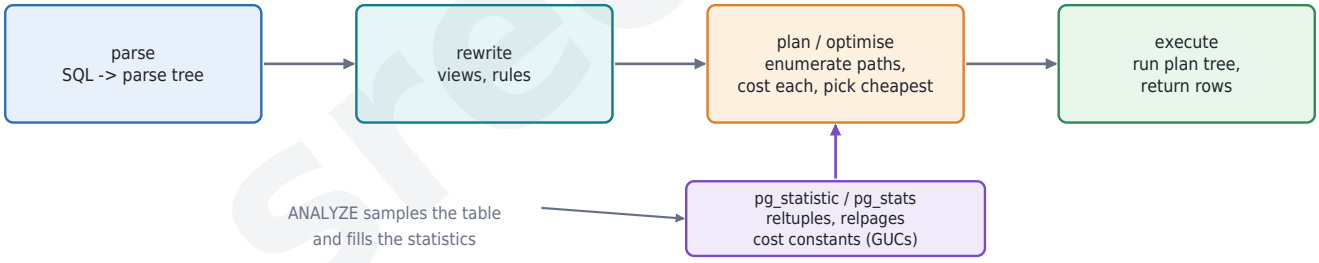


Figure 1.75. The four stages of statement processing; the planner reads table statistics and cost parameters to compare candidate plans.

A **prepared statement** is parsed once; PostgreSQL then plans it either per execution with the actual parameter values (a *custom plan*) or once with generic estimates (a *generic plan*), choosing the generic plan after five executions if its estimated cost is not worse than the average custom plan (`plan_cache_mode`, default `auto`). The JDBC driver switches to server-side prepared statements after `prepareThreshold` executions (default 5), so an application can change from custom to generic plans mid-run, which matters for columns with skewed data (see the incident on skewed parameters below).

Statistics: what the planner knows

`ANALYZE` reads a random sample of rows (about 300 times the *statistics target*, 100 by default, that is 30 000 rows) and stores per-column summaries in `pg_statistic`, readable as the view `pg_stats`. Table-level counts, `reltuples` and `relpages`, are kept in `pg_class`.

```sql
SELECT attname, null_frac, n_distinct, most_common_vals, most_common_freqs, correlation
FROM pg_stats WHERE tablename = 'orders' AND attname = 'status';
```

Output (real) TERMINAL

```
$ psql -d shop
 attname | null_frac | n_distinct |           most_common_vals           |  most_common_freqs
---------+-----------+------------+--------------------------------------+--------------------
 status  |         0 |          5 | {DELIVERED,SHIPPED,PAID,NEW,CANCELLED}
   | {0.2047,0.2018,0.1988,0.1981,0.1965}
(correlation of status with physical order: 0.1986)
```

Field	Meaning and use
`null_frac`	Fraction of NULL values; used for `IS NULL` and to scale other fractions
`n_distinct`	Number of distinct values; negative means a fraction of the row count (-1: all unique). Drives `GROUP BY` and equality on non-frequent values
`most_common_vals`, `most_common_freqs`	The most frequent values and their frequencies (up to 100); an equality on one of them uses its exact frequency
`histogram_bounds`	Bounds of 100 equal-population buckets over the remaining values; used for `<`, `>` and `BETWEEN`
`correlation`	Statistical correlation between value order and physical row order, from -1 to 1; near 1 makes index range scans cheap, near 0 makes them random I/O
`avg_width`	Average size in bytes; feeds row width and memory estimates

The statistics kept per column in pg_stats.

The values are estimates from a sample. ANALYZE sampled the status frequency as 0.2035 for the plan that produced the estimate 305 300 rows in an earlier listing, and 0.1981 for `NEW` later; the true value is 0.2. This variation is normal and is the reason that estimates are never exact.

Selectivity and row estimates

The estimate of a node is the number of input rows multiplied by the **selectivity** of its condition. For equality on a frequent value the selectivity is its listed frequency; for a range it is derived from the histogram, assuming a uniform distribution inside a bucket; for a value that is not in the list it is the share of the rest divided by the remaining distinct values. For several conditions the planner multiplies selectivities, assuming the columns are **independent**.

The numbers can be checked by hand. For `amount > 499` the histogram bounds of the last two buckets are 494.81 and 499.97. Each bucket holds 1 percent of the rows, and 499 lies (499.97 - 499) / (499.97 - 494.81) = 0.188 of the way below the top of the last bucket, so the selectivity is about 0.188 x 0.01 = 0.00188 and the estimate is 1 500 000 x 0.00188 = 2820 rows. The planner printed `rows=2818`; the actual count was 3016. For `status = 'PAID'` the frequency 0.1988 gives 298 200 estimated rows (actual 300 000). For the two conditions together the planner multiplies: 1 500 000 x 0.1988 x 0.001879 = 560 rows (printed `rows=560`; actual 623). The columns are independent in this table, so the product is accurate.

Cost model

Each plan node has a cost expressed in units where reading one page sequentially costs `seq_page_cost = 1.0`. The default parameters are:

Parameter	Default	Meaning
`seq_page_cost`	1.0	One page read in a sequential scan
`random_page_cost`	4.0	One non-sequential page read; 1.1 to 1.5 is typical for SSD storage
`cpu_tuple_cost`	0.01	Processing one row
`cpu_index_tuple_cost`	0.005	Processing one index entry
`cpu_operator_cost`	0.0025	Evaluating one operator or function
`effective_cache_size`	4 GB	Planner's assumption about OS plus database cache; lowers estimated index I/O. Allocates nothing
`work_mem`	4 MB	Memory per sort or hash node; determines spill estimates
`parallel_setup_cost`	1000	Cost of launching parallel workers

Planner cost parameters (PostgreSQL 16 defaults).

The sequential scan of `orders` shows the model at work. The plan `Seq Scan on orders (cost=0.00..39584.00)` is exactly the sum 20 834 x 1.0 for the pages, 1 500 000 x 0.01 for the rows and 1 500 000 x 0.0025 for the filter on each row: 20 834 + 15 000 + 3750 = 39 584.



Figure 1.76. The cost of a sequential scan reproduced from the formula: pages, rows and filter evaluations add to the 39 584 shown in the plan.

An index scan is costed from the number of index pages and the number of heap pages it expects to fetch at `random_page_cost`, discounted for correlation and for the cache. For `customer_id < 20000` (301 700 estimated rows, 20 percent of the table) the planner compared three plans: a sequential scan (39 584), a bitmap scan (28 232) and a plain index scan (89 903, obtained by forcing it with `enable_bitmapscan = off, enable_seqscan = off`). It chose the bitmap scan; the plain index scan was the most expensive because 300 000 entries at random heap positions require up to 300 000 random page visits. At 40 percent selectivity (`customer_id < 60000`, 904 476 estimated rows) the cost of the bitmap plan exceeded 39 584 and the sequential scan won.

The `enable_*` settings (`enable_seqscan`, `enable_nestloop` and so on) do not forbid a plan type; they add a huge cost to it. They are diagnostic tools to learn what the alternative costs and how long it takes, not production settings.

Keeping statistics fresh

Autovacuum runs `ANALYZE` on a table when the number of modified rows exceeds `autovacuum_analyze_threshold` (50) plus `autovacuum_analyze_scale_factor` (0.1) times the row count: for a 1.5 million row table, 150 050 changed rows. That is correct for slow-changing tables and too late for a table whose *shape* changes abruptly, such as a queue table that was empty at the last analyze and is full now. Statistics also do not exist for data loaded in the same transaction until `ANALYZE` runs, and a freshly restored or bulk-loaded database must be analysed before it is used.

SQL

```sql
SELECT relname, last_analyze, last_autoanalyze, n_live_tup, n_mod_since_analyze
FROM pg_stat_user_tables WHERE relname = 'orders';

ANALYZE orders;                                  -- refresh now
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;   -- finer histogram
ALTER TABLE orders SET (autovacuum_analyze_scale_factor = 0.02);  -- analyse more often
```

Extended statistics

Independence fails when columns are related. In a table `addr(city, state)` of 500 000 rows each city belongs to one state, so `city = 'City42' AND state = 'State4'` returns 5000 rows, but multiplying the selectivities 0.01 and 0.1 predicts 476. `CREATE STATISTICS` teaches the planner the dependency:

SQL

```sql
CREATE STATISTICS st_addr (dependencies) ON city, state FROM addr;
ANALYZE addr;
EXPLAIN (ANALYZE, TIMING OFF, SUMMARY OFF)
SELECT * FROM addr WHERE city = 'City42' AND state = 'State4';
```

Estimate before and after (real) — TERMINAL

```
$ psql -d shop
 Seq Scan on addr  (cost=0.00..10685.00 rows=476 width=17) (actual rows=5000 loops=1)
 Seq Scan on addr  (cost=0.00..10685.00 rows=4983 width=17) (actual rows=5000 loops=1)
```

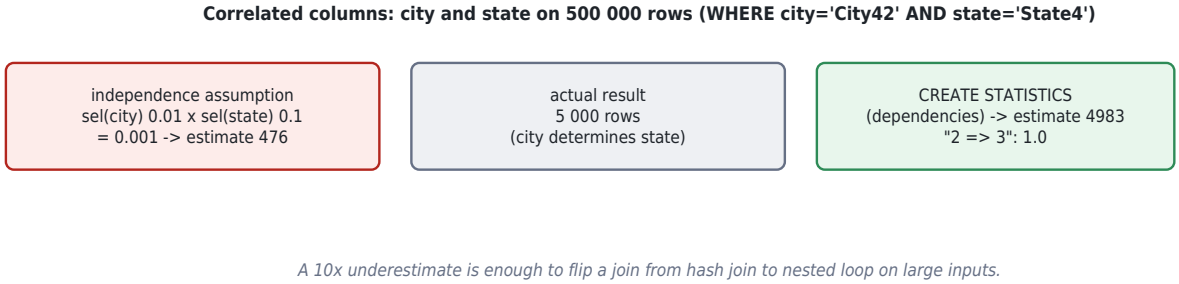


Figure 1.77. A functional dependency invisible to per-column statistics causes a tenfold underestimate; extended statistics restore the estimate to 4983 against 5000 actual rows.

The kinds are `dependencies` (functional dependencies), `ndistinct` (combined distinct counts, for `GROUP BY a, b`) and `mcv` (multi-column most-common combinations). An estimate that is wrong by ten times is enough to change a join from hash join to nested loop, as the next section shows.

Why the planner ignores an index

When an expected index is not used, work through this list in order:

1. **The predicate is not sargable** (a function or cast is applied to the indexed column). The index cannot be searched. See the optimisation chapter.
2. **Low selectivity.** The predicate matches a large fraction of the table, so reading the heap sequentially is cheaper (measured at 20 to 40 percent in the examples above).
3. **Leading column absent.** A composite index whose first column is not constrained cannot be searched in PostgreSQL 16.
4. **Stale statistics.** The planner believes the table is small or the predicate unselective.
5. **Small table.** For a table of a few pages a sequential scan is cheaper than any index descent. This is correct behaviour and not a defect.
6. **Cost parameters.** `random_page_cost = 4.0` suits spinning disks; on SSD or fully cached data it overstates the price of index access. Lowering it to 1.1 moved the bitmap-versus-sequential crossover only slightly in the experiment above, so measure before changing it.
7. **The index is invalid or not yet built** (`indisvalid = false` after a failed `CREATE INDEX CONCURRENTLY`).

1.17.2 Worked example

One estimate error changes the join algorithm

A table `jobs(id, state, cust)` receives 1 000 000 `DONE` rows and is analysed; the application then inserts 400 000 `PENDING` rows before the next analyze (autovacuum was disabled on the table to freeze the situation). A report joins pending jobs to `customers` to count them per country.

Step 1: examine the statistics. `pg_stats` contains `most_common_vals {DONE}` and `most_common_freqs {1}`: the planner believes every row is `DONE`. For `state = 'PENDING'` it estimates the minimum, one row.

Step 2: the plan follows from that estimate. For one expected outer row, a nested loop with an index lookup is the cheapest, so the planner chose `Nested Loop` with `Index Scan using customers_pkey ... loops=400000`:

Plan with stale statistics (real, trimmed) TERMINAL

```
$ psql -d shop
 GroupAggregate  (cost=12.77..12.79 rows=1 width=11) (actual time=2131.277..2324.393 rows=8 loops=1)
   ->  Sort  (cost=12.77..12.77 rows=1 width=3) (actual rows=400000 loops=1)
         Sort Method: external merge  Disk: 2768kB
         ->  Nested Loop  (cost=0.72..12.76 rows=1 width=3) (actual rows=400000 loops=1)
               ->  Index Scan using idx_jobs_state on jobs j  (cost=0.43..4.45 rows=1 width=4)
                     (actual rows=400000 loops=1)
               ->  Index Scan using customers_pkey on customers c  (cost=0.29..8.31 rows=1 width=7)
                     (actual rows=1 loops=400000)
 Execution Time: 2324.729 ms
```

Step 3: after `ANALYZE jobs`, `most_common_freqs` becomes `{0.712, 0.288}` and the estimate becomes 397 367 rows. The cheapest plan for 400 000 outer rows is a hash join:

Plan after ANALYZE (real, trimmed) TERMINAL

```
$ psql -d shop
 HashAggregate  (cost=16619.56..16619.64 rows=8 width=11)
   (actual time=827.051..827.057 rows=8 loops=1)
   ->  Hash Join  (cost=3184.43..14657.48 rows=392415 width=3) (actual rows=400000 loops=1)
         Hash Cond: (j.cust = c.id)
         ->  Index Scan using idx_jobs_state on jobs j  (cost=0.43..10430.35 rows=397367 width=4)
         ->  Hash  (cost=1934.00..1934.00 rows=100000 width=7)
               ->  Seq Scan on customers c  (actual rows=100000 loops=1)
 Execution Time: 827.277 ms
```

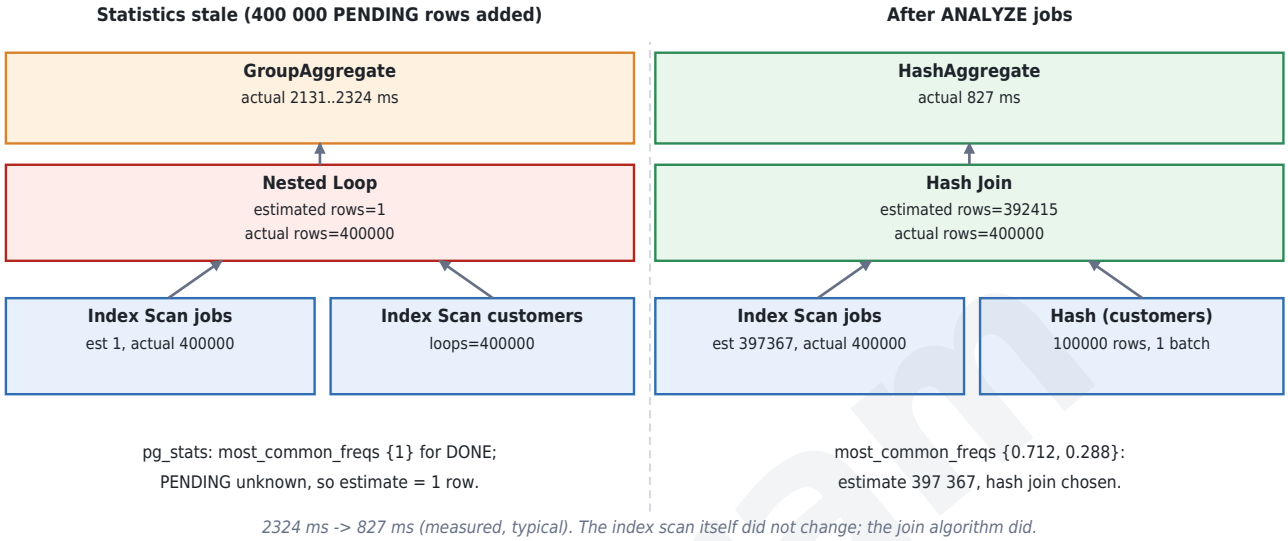


Figure 1.78. The same query before and after ANALYZE: a one-row estimate selects a nested loop executed 400 000 times (2324 ms); the correct estimate selects a hash join (827 ms).

Observe that the index scan of `jobs` is identical in both plans, so the access path was not the problem: the join method was. The first sign of trouble in the first plan is visible without running anything else: `rows=1` on a node whose actual rows are 400 000.

1.17.3 Hands-on

SQL

```sql
-- Reproduce stale statistics
CREATE TABLE jobs(id serial PRIMARY KEY, state text NOT NULL, cust int NOT NULL)
  WITH (autovacuum_enabled = false);
INSERT INTO jobs(state, cust) SELECT 'DONE', 1 + (g % 100000) FROM generate_series(1,1000000) g;
CREATE INDEX idx_jobs_state ON jobs(state);
ANALYZE jobs;
INSERT INTO jobs(state, cust) SELECT 'PENDING', 1 + (g % 100000) FROM generate_series(1,400000) g;

SET max_parallel_workers_per_gather = 0;
EXPLAIN (ANALYZE, COSTS ON) SELECT c.country, count(*)
FROM jobs j JOIN customers c ON c.id = j.cust WHERE j.state = 'PENDING' GROUP BY c.country;

ANALYZE jobs;
-- run the same EXPLAIN again and compare estimated and actual rows
```

To see statistics directly, inspect `pg_stats` for the column. To see what the planner would have done differently, set `enable_nestloop = off` and re-run `EXPLAIN` (for diagnosis only), then `RESET enable_nestloop`.

1.17.4 Real-world problems and solutions

Incident 6: the planner chooses a sequential scan because of stale statistics

Scenario. A job-processing table is emptied every night and filled during the day. A dashboard query joins pending jobs to customers. It is fast in the morning and slow in the afternoon, and it became worse after a data-volume increase (an illustrative scenario). No index or code change occurred.

Symptoms. The slow plan uses nested loops with an inner index scan and `loops` in the hundreds of thousands; CPU is high; running `ANALYZE` by hand makes the query fast again.

Diagnosis.

```sql
EXPLAIN (ANALYZE) SELECT ... ;   -- find a node with rows=1 and actual rows=400000
SELECT relname, last_autoanalyze, n_live_tup, n_mod_since_analyze
FROM pg_stat_user_tables WHERE relname = 'jobs';
SELECT most_common_vals, most_common_freqs FROM pg_stats
WHERE tablename = 'jobs' AND attname = 'state';
```

A large `n_mod_since_analyze` and a most-common-values list that lacks the predicate value confirm the cause.

Root cause. The statistics describe the table as it was when last analysed. Autovacuum waits for 10 percent of the table to change, and on a table where the *distribution* changes (a new status value appears) the planner must work with wrong numbers until then.

Fix. Immediate: `ANALYZE jobs;` (a plain ANALYZE only takes a share-update-exclusive lock and does not block reads or writes). Permanent: run `ANALYZE` explicitly at the end of the data load or batch step; set a lower `autovacuum_analyze_scale_factor` on the table; for skewed columns raise `SET STATISTICS`; for a hot-spot queue table that is meant to be small, keep it small (delete finished rows promptly or partition by status).

Prevention. Alert on tables with high `n_mod_since_analyze` and on queries whose estimated rows differ from actual by more than ten times; `auto_explain` with `log_analyze` records both numbers (see the diagnosis chapter).

Incident: a generic plan chosen for a skewed parameter

Scenario. A Spring Boot service runs `SELECT ... FROM jobs WHERE state = ?` through a pooled prepared statement. The query is fast for the rare value `FAILED` and for the frequent value `DONE` at first; after a while one of them becomes slow on connections that have run it several times.

Symptoms. Same SQL text, different latencies per parameter after the fifth execution on a connection.

Diagnosis. `EXPLAIN (ANALYZE)` with the literal value shows a good plan; `PREPARE q(text) AS ...; EXPLAIN EXECUTE q('DONE');` repeated six times shows `Index Scan` with a generic plan. In PostgreSQL 12 and later, `plan_cache_mode` can be inspected, and the plan type can be forced for the session.

Root cause. After five executions the server may switch to a generic plan, which uses average selectivity and ignores the actual parameter. For skewed data one generic plan cannot be right for both a rare and a very frequent value.

Fix. Set `plan_cache_mode = force_custom_plan` for the affected role or connection (`ALTER ROLE app SET plan_cache_mode = force_custom_plan`), or add `prepareThreshold=0` to the JDBC URL to avoid server-side preparation (at a small planning cost per call). For a genuinely different access pattern per value, use separate queries.

Prevention. Test queries with both the rare and the frequent parameter values and after the sixth execution.

1.17.5 Questions to expect

INTERVIEW ANGLE

How does the optimiser decide between an index scan and a full table scan? It estimates the number of matching rows from column statistics (frequencies and histograms) and costs each path: a sequential scan reads every page cheaply, an index scan pays a random page cost per fetched row. When the estimated fraction is large, or the table is small, the sequential scan is cheaper. Wrong statistics therefore lead to wrong choices.

1. What is stored in `pg_stats`? Per column: null fraction, distinct count, most-common values and frequencies, histogram bounds, correlation, average width.
2. What does ANALYZE do and when does it run? It samples rows (about 30 000 at the default target) to refresh statistics; autovacuum triggers it after about 10 percent of rows change; it should also be run after bulk loads.
3. What is the independence assumption and how do you override it? The planner multiplies selectivities of different columns; `CREATE STATISTICS` with dependencies, ndistinct or mcv corrects it for correlated columns.
4. What are `random_page_cost` and `seq_page_cost`? Relative costs of non-sequential and sequential page reads (4.0 and 1.0 by default); lowering the first favours index scans on SSD.
5. What does `correlation` in pg_stats tell you? How well the column order follows the physical row order; near 1 makes range index scans touch few pages, near 0 makes them random.

Stefan

6. Why can the same query be fast with one parameter and slow with another? Skewed data: different parameters have different selectivities, and a generic prepared plan fits only the average.
7. Should you use `enable_seqscan = off` in production? No, it is a diagnostic tool that only penalises a plan type; fix the cause (statistics, index, query).
8. What is the effect of `work_mem` on plans? It bounds sort and hash memory; the planner estimates spills against it and prefers other plans (for example avoiding a large hash join) when it is small.

KEY POINTS

- The planner multiplies row counts by selectivities taken from `pg_stats`; any error in them propagates through the plan.
- Cost is a sum of page, row and operator costs; the sequential scan of 1.5 million rows costs exactly 20 834 + 15 000 + 3750.
- Stale statistics, correlated columns and skewed parameters are the typical causes of bad plans; ANALYZE, extended statistics and plan_cache_mode are the tools.
- Compare estimated and actual rows at every node before changing anything else.
- `enable_*` settings are for experiments, not for production.

1.18 Query Optimisation Techniques

Once a slow statement is found and its plan read, the remedy usually belongs to one of a few families: let the optimiser use an existing index (sargable predicates, matching types), give it a better index, reduce the work (fetch fewer rows and columns, batch round trips, paginate by key instead of by offset), or rewrite the query so that it does less (aggregate before joining, avoid correlated subqueries). Each technique in this chapter is demonstrated with a before and after `EXPLAIN ANALYZE` on the 1.5 million row `orders` table; timings are measured on the build machine and are typical rather than exact.

1.18.1 Concept and mechanism

Sargable predicates

A predicate is **sargable** (search-argument-able) when the database can use it as an index condition. The indexed column must stand alone on one side of a comparison. Applying a function, an arithmetic expression or a cast to the column turns the predicate into a per-row computation that no ordinary index can answer.

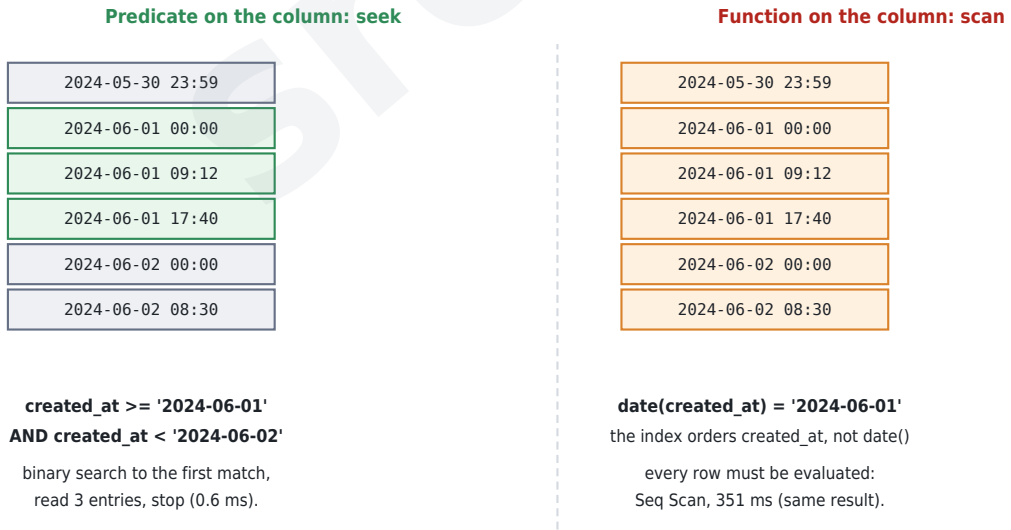


Figure 1.79. A range on the bare column maps to a contiguous slice of the sorted index, while date(created_at) can only be evaluated row by row.

Non-sargable	Sargable equivalent	Note
`date(created_at) = '2024-06-01'`	`created_at >= '2024-06-01' AND created_at < '2024-06-02'`	Half-open range; also avoids the time zone ambiguity

Non-sargable	Sargable equivalent	Note
`extract(year FROM created_at) = 2024`	`created_at >= '2024-01-01' AND created_at < '2025-01-01'`	Same pattern for any extracted part
`lower(email) = 'a@b.c'`	Same predicate with an index on `lower(email)`	Expression index
`amount * 1.2 > 100`	`amount > 100 / 1.2`	Move arithmetic to the constant
`customer_id::text = '4242'`	`customer_id = 4242`	Compare in the column's own type
`name LIKE '%son'`	trigram GIN index, or reverse-string expression index	Leading wildcard defeats B-tree
`coalesce(status, 'NEW') = 'NEW'`	`status = 'NEW' OR status IS NULL`	Or a partial or expression index

Rewriting non-sargable predicates.

Implicit conversions

When the two sides of a comparison have different types, the database inserts a cast. If the cast lands on the **column** side the index is useless. PostgreSQL resolves `integer = numeric` by converting the integer column to numeric for every row (`(customer_id)::numeric = '4242'`). In MySQL, comparing a string column to a number converts the column, and comparing columns of different character sets or collations disables the index in the same way. The same issue arises from Java code: a `BigDecimal` or `double` parameter bound against an `int` or `varchar` column, or a `setString` against a numeric column, changes the plan without any visible change to the SQL text.

Pagination: OFFSET against keyset

`LIMIT 20 OFFSET 1000000` is executed by producing the first 1 000 020 rows in order and discarding the first million; the cost grows linearly with the page number. **Keyset pagination** (also called seek) remembers the sort key of the last row returned and asks for rows after it, which the index answers by a descent to that key. It needs a deterministic, unique, indexed sort key (a tie-breaker such as `id` must be part of the key), and it cannot jump to an arbitrary page number.

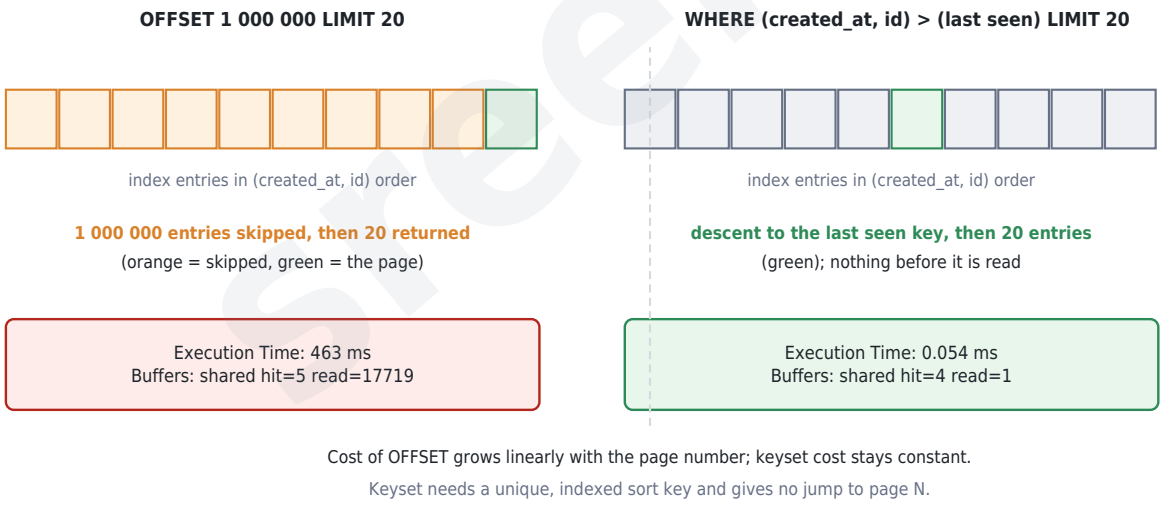


Figure 1.80. The cost of OFFSET pagination grows with the page number because skipped rows are still read; keyset pagination descends the index directly to the last seen key.

Reducing work: columns, counts, batching

- **Select only needed columns.** Every column read widens rows; large values cost toast reads; an index-only scan becomes possible only if the select list is covered.
- `COUNT(*)` on a large table is a scan: PostgreSQL must check visibility of every row. With an index-only scan it read 1590 index pages in 248 ms for 1.5 million rows; the planner estimate in `pg_class.reltuples` is free and sufficient for "about 1.5 million" displays. MySQL InnoDB also scans (an index scan). Maintain a counter table or use an estimate when exact counts are not needed.
- **Batching.** Each round trip costs network latency plus parse and plan time. Loading 50 related rows with 51 statements instead of 2 is the N+1 pattern below.
- **Aggregate before joining** and avoid correlated subqueries that execute once per outer row.

The N+1 query problem

The N+1 problem appears when code loads a list of N entities with one query and then, for each, triggers a second query to load a related entity. In Spring Data JPA with Hibernate it arises from lazy associations: `orderRepository.findAll()` issues `SELECT * FROM orders`, and the first call to `order.getCustomer().getName()` for each order issues `SELECT * FROM customers WHERE id = ?`, once per order.

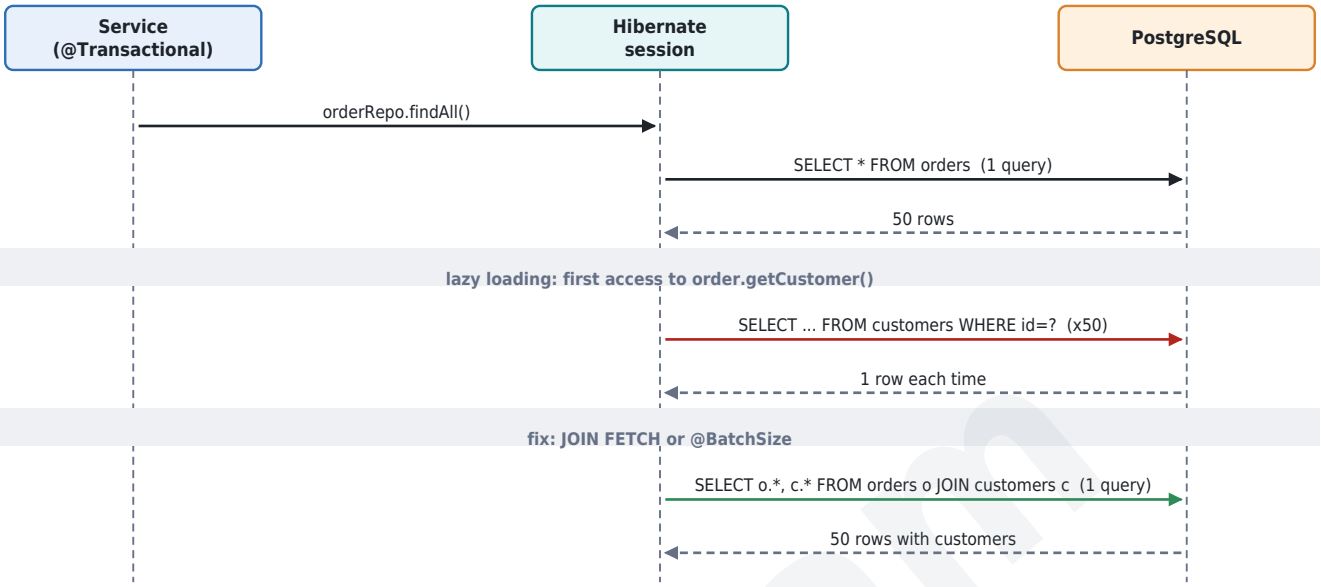


Figure 1.81. The N+1 exchange: one query for the orders, then one query per order for its customer; a fetch join returns everything in one statement.

Order.java (fragment, unverified: no JDBC driver available on the build machine) JAVA

```java
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)       // default for @ManyToOne is EAGER; LAZY is a choice
    @JoinColumn(name = "customer_id")
    private Customer customer;

    private BigDecimal amount;
    // getters omitted
}
```

OrderService.java (fragment, unverified) JAVA

```java
@Transactional(readOnly = true)
public List<OrderDto> recent() {
    List<Order> orders = orderRepository.findTop50ByOrderByCreatedAtDesc();   // 1 query
    return orders.stream()
        .map(o -> new OrderDto(o.getId(), o.getCustomer().getName()))          // N queries
        .toList();
}
```

With `spring.jpa.show-sql=true` (or the `org.hibernate.SQL` logger at DEBUG) the log shows the signature:

Application log (typical output, not run on the build machine) TERMINAL

```
$ psql -d shop
Hibernate: select o.id, o.customer_id, o.amount from orders o order by o.created_at desc limit ?
Hibernate: select c.id, c.name, c.email from customers c where c.id=?
Hibernate: select c.id, c.name, c.email from customers c where c.id=?
Hibernate: select c.id, c.name, c.email from customers c where c.id=?
(... 47 more identical statements ...)
```

The same signature is visible on the server. A load of the pattern (51 statements per request, 100 requests) was recorded in `pg_stat_statements`:

pg_stat_statements (real; the single-row lookup stands for the repeated lazy load) TERMINAL

```
$ psql -d shop
 calls | total_ms | mean_ms | rows  | query
-------+----------+---------+-------+-----------------------------------------------------------
  5000 |    309.7 |   0.062 | 75000 | select id, amount from orders where customer_id = $1
   100 |      1.2 |   0.012 |  5000 | select id from customers where id <= $1
```

One statement was executed 5000 times with the same shape, each returning about 15 rows; the database cost per call is tiny (0.062 ms), but the application pays network latency 5000 times. At a typical 0.5 ms round trip within a data centre, 50 extra statements add 25 ms to each request, and under concurrency the pool is held for the whole time.

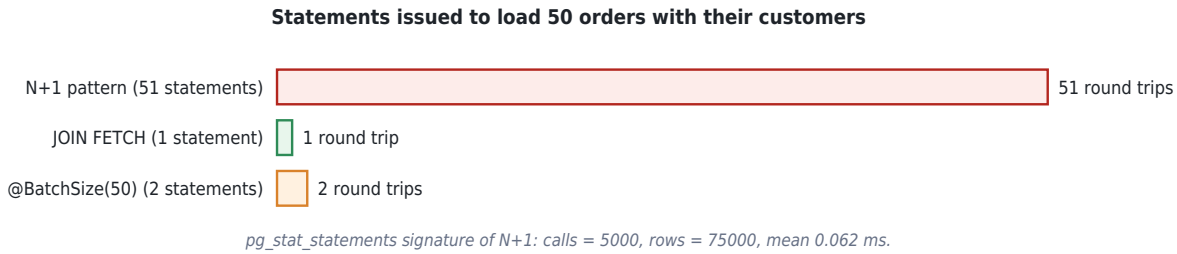


Figure 1.82. Statement counts for loading 50 orders with their customers by N+1, fetch join and batch fetching.

Three remedies exist, and the right one depends on the use case:

OrderRepository.java (fragment, unverified) JAVA

```java
public interface OrderRepository extends JpaRepository<Order, Long> {

    // 1. Fetch join: one statement with a join; use for a bounded list that needs the association.
    @Query("select o from Order o join fetch o.customer order by o.createdAt desc")
    List<Order> findRecentWithCustomer(Pageable page);

    // 2. Entity graph: the same, declared without JPQL.
    @EntityGraph(attributePaths = "customer")
    List<Order> findTop50ByOrderByCreatedAtDesc();
}
```

The third is batch fetching: `spring.jpa.properties.hibernate.default_batch_fetch_size=50` (or `@BatchSize(size = 50)` on the association) makes Hibernate load the lazy customers of 50 orders with a single `where c.id in (?, ?, ...)`, reducing 51 statements to 2 without changing the queries. A fetch join combined with a `Pageable` on a *collection* association (one-to-many) produces the warning `firstResult/maxResults specified with collection fetch; applying in memory`, because the join multiplies rows; use batch fetching or a two-step query for collections. A projection that selects only the needed columns (`select o.id, c.name from Order o join o.customer c`) avoids loading entities at all.

1.18.2 Worked example

Rewriting a correlated subquery

A report lists every customer with the number of December orders. The first version uses a correlated subquery:

SQL

```sql
SELECT c.id,
       (SELECT count(*) FROM orders o
         WHERE o.customer_id = c.id AND o.created_at >= '2024-12-01') AS dec_orders
FROM customers c;
```

Step 1: the plan executes the subquery once for each of the 100 000 customers (`loops=100000`) as a bitmap scan of `idx_orders_customer`, which reads about 15 rows per call and discards 14 of them by the date filter.

Plan of the correlated version (real, trimmed) TERMINAL

```
$ psql -d shop
 Seq Scan on customers c  (actual rows=100000 loops=1)
   Buffers: shared hit=300196 read=1500738 written=16
   SubPlan 1
     ->  Aggregate  (actual rows=1 loops=100000)
           ->  Bitmap Heap Scan on orders o  (actual rows=1 loops=100000)
                 Recheck Cond: (customer_id = c.id)
                 Filter: (created_at >= '2024-12-01 00:00:00+00'::timestamp with time zone)
                 Rows Removed by Filter: 14
                 Heap Blocks: exact=1500000
 Execution Time: 14732.559 ms
```

Step 2: count the work: 1.8 million buffers (the sum of hit and read) to produce a result that needs only the 52 801 December orders. Step 3: invert the logic. Aggregate the December orders once, then join:

SQL

```sql
SELECT c.id, COALESCE(x.n, 0) AS dec_orders
FROM customers c
LEFT JOIN (SELECT customer_id, count(*) AS n
           FROM orders WHERE created_at >= '2024-12-01'
           GROUP BY customer_id) x ON x.customer_id = c.id;
```

Plan of the rewritten version (real, trimmed) TERMINAL

```
$ psql -d shop
 Hash Left Join  (actual rows=100000 loops=1)
   Hash Cond: (c.id = x.customer_id)
   ->  Seq Scan on customers c  (actual rows=100000 loops=1)
   ->  Hash  (actual rows=52801 loops=1)
         ->  HashAggregate  (actual rows=52801 loops=1)
               ->  Index Scan using idx_orders_created on orders  (actual rows=52801 loops=1)
                     Index Cond: (created_at >= '2024-12-01 00:00:00+00'::timestamp with time zone)
 Execution Time: 63.894 ms
```

The execution time dropped from 14 733 ms to 64 ms (230 times) and the buffers from 1.8 million to about 1600. The change is not magic: the first plan touched each of the 1.5 million orders through a per-customer index lookup; the second touched only the 52 801 relevant orders, once, in index order. The general principle is to make the filter that removes most data run first and once.

1.18.3 Hands-on

Each technique below has a before and after that can be repeated on the `orders` table.

1. Function on an indexed column

SQL

```sql
CREATE INDEX idx_orders_created ON orders (created_at);
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF)
SELECT count(*) FROM orders WHERE date(created_at) = '2024-06-01';
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF) SELECT count(*) FROM orders
WHERE created_at >= '2024-06-01' AND created_at < '2024-06-02';
```

Before and after (real, trimmed) TERMINAL

```
$ psql -d shop
 Aggregate  (actual rows=1 loops=1)
   ->  Seq Scan on orders  (cost=0.00..43334.00 rows=7500 width=0) (actual rows=4320 loops=1)
         Filter: (date(created_at) = '2024-06-01'::date)
         Rows Removed by Filter: 1495680
         Buffers: shared hit=416 read=20418
 Execution Time: 351.006 ms

 Aggregate  (actual rows=1 loops=1)
   ->  Index Only Scan using idx_orders_created on orders  (actual rows=4320 loops=1)
         Heap Fetches: 0
         Buffers: shared hit=1 read=14
 Execution Time: 0.602 ms
```

If the function must be kept, index the expression. For a `timestamptz` column `date(created_at)` is not immutable (it depends on the session time zone), and `CREATE INDEX ... ((date(created_at)))` fails with `functions in index`

`expression must be marked IMMUTABLE`. Fix the zone: `CREATE INDEX ON orders (((created_at AT TIME ZONE 'UTC')::date))` and query with the identical expression, which produced a bitmap scan of 67 buffers.

2. Implicit cast

SQL

```sql
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF) SELECT * FROM orders WHERE customer_id = 4242::numeric;
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF) SELECT * FROM orders WHERE customer_id = 4242::bigint;
```

Before and after (real, trimmed) TERMINAL

```
$ psql -d shop
 Seq Scan on orders  (cost=0.00..43334.00 rows=7500 width=77) (actual rows=15 loops=1)
   Filter: ((customer_id)::numeric = '4242'::numeric)
   Rows Removed by Filter: 1499985
 Execution Time: 548.670 ms

 Bitmap Heap Scan on orders  (cost=4.54..63.52 rows=15 width=77) (actual rows=15 loops=1)
   Recheck Cond: (customer_id = '4242'::bigint)
   ->  Bitmap Index Scan on idx_orders_customer  (actual rows=15 loops=1)
 Execution Time: 0.191 ms
```

The visible clue is `(customer_id)::numeric` in the `Filter` line: a cast on the column side. `integer = bigint` is resolved inside the same operator family, so it keeps the index; `integer = numeric` and `integer = text` do not.

3. Pattern matching

SQL

```sql
-- Database collation C.UTF-8: a plain B-tree is not usable for LIKE 'prefix%'
CREATE INDEX idx_cust_email_pat ON customers (email text_pattern_ops);
EXPLAIN (ANALYZE, TIMING OFF, SUMMARY OFF) SELECT * FROM customers WHERE email LIKE 'user77777@%';
```

Plan (real) TERMINAL

```
$ psql -d shop
 Index Scan using idx_cust_email_pat on customers  (cost=0.42..8.44 rows=10 width=42)
   (actual rows=1 loops=1)
   Index Cond: ((email ~>=~ 'user77777@'::text) AND (email ~<~ 'user77777A'::text))
   Filter: (email ~~ 'user77777@%'::text)
```

A B-tree can serve `LIKE 'abc%'` only if the index sorts by character values, which requires the `C` collation or the operator class `text_pattern_ops` (or `varchar_pattern_ops`); in the experiment the database collation was `C.UTF-8`, and a plain index on `email` still produced a `Seq Scan` for `LIKE 'user77777@%'`; only the `text_pattern_ops` index was used. A pattern with a leading wildcard needs a trigram index, shown in the incident below. MySQL can use a standard B+tree index for `LIKE 'abc%'` under the usual collations.

4. OR and UNION

`WHERE customer_id = 4242 OR created_at < '2024-01-01 00:10'` was answered by a `BitmapOr` of two indexes (0.1 ms). The same query with `OR note = 'order note xyz'`, where `note` has no index, falls back to a sequential scan of the whole table, because one unindexed branch forces the scan. Either add the missing index or rewrite as `UNION ALL` of two queries when the branches are disjoint (use `UNION` otherwise to remove duplicates).

5. Pagination

SQL

```sql
CREATE INDEX idx_orders_created_id ON orders (created_at, id);
-- OFFSET: reads and discards 1 000 000 rows
SELECT id, created_at, amount FROM orders ORDER BY created_at, id LIMIT 20 OFFSET 1000000;
-- keyset: remember (created_at, id) of the last row of the previous page
SELECT id, created_at, amount FROM orders
WHERE (created_at, id) > ('2024-08-19 11:33:20+00', 1000000)
ORDER BY created_at, id LIMIT 20;
```

Sheram

Plans (real, trimmed) TERMINAL

```
$ psql -d shop
 Limit  (actual rows=20 loops=1)
   ->  Index Scan using idx_orders_created_id on orders  (actual rows=1000020 loops=1)
         Buffers: shared hit=5 read=17719 written=5257
 Execution Time: 463.003 ms

 Limit  (actual rows=20 loops=1)
   ->  Index Scan using idx_orders_created_id on orders  (actual rows=20 loops=1)
         Index Cond: (ROW(created_at, id) > ROW('2024-08-19 11:33:20+00'::timestamptz, 1000000))
         Buffers: shared hit=4 read=1
 Execution Time: 0.054 ms
```

The row-value comparison `(created_at, id) > (a, b)` is the SQL-standard way to express a lexicographic condition; PostgreSQL turns it into an index condition on both columns. MySQL 8.0 supports row constructors but may not use the index for them as effectively, so the equivalent expanded form `created_at > ? OR (created_at = ? AND id > ?)` is used there.

6. COUNT(*)

SQL

```sql
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF) SELECT count(*) FROM orders;
SELECT reltuples::bigint AS estimate FROM pg_class WHERE relname = 'orders';
```

Output (real) TERMINAL

```
$ psql -d shop
 Aggregate  (actual rows=1 loops=1)
   ->  Index Only Scan using idx_orders_customer on orders  (actual rows=1500000 loops=1)
         Heap Fetches: 0
         Buffers: shared hit=1 read=1590
 Execution Time: 248.396 ms

 estimate
----------
  1500000
```

Counting is linear in table size. When an approximate figure suffices (for example "about 1.5 million results" in a search page), read `reltuples`; for an exact count that is requested often, maintain it in a summary table.

1.18.4 Real-world problems and solutions

Incident 7: a function on an indexed column causes a full table scan

Scenario. The daily revenue endpoint of an order service filters with `WHERE date(created_at) = :day`. It was written when the table had 50 000 rows and has an index on `created_at`. At 1.5 million rows it takes 350 ms and the database CPU rises with concurrent users.

Symptoms. The query is slow although `\d orders` shows an index on `created_at`; `pg_stat_user_indexes.idx_scan` for that index does not increase.

Diagnosis.

SQL

```sql
EXPLAIN (ANALYZE, BUFFERS) SELECT count(*) FROM orders WHERE date(created_at) = '2024-06-01';
```

The plan (shown above) is a `Seq Scan` with `Filter: (date(created_at) = '2024-06-01'::date)` and `Rows Removed by Filter: 1495680`: 1 495 680 rows read to find 4320.

Root cause. The function is applied to the column, so the planner cannot map the condition onto the index order. The estimate (7500 rows) also shows that it used a default guess for the function result.

Fix. Rewrite as a half-open range on the bare column (`created_at >= :day AND created_at < :day + 1`): 0.6 ms with an index-only scan, 580 times faster. In Spring Data use `findByCreatedAtGreaterThanEqualAndCreatedAtLessThan(start, end)` with `Instant` or `OffsetDateTime` bounds computed in the application's business time zone. If rewriting is impossible, create the matching expression index with an immutable expression.

Prevention. Code review rule: no function or cast on a column in a WHERE clause on a large table; check the `Filter` line of every new query plan.

Incident 8: an implicit cast disables the index

Scenario. A support tool looks up orders by customer id using a `BigDecimal` parameter, bound through JDBC `setBigDecimal`. A migration changed the column type of the search field from `varchar` to `numeric` in the DTO, and the lookup became slow; the SQL text and the indexes did not change.

Symptoms. A lookup that took 0.2 ms takes 550 ms; the slow query appears in `pg_stat_statements` with `customer_id = $1` and a mean time of hundreds of milliseconds.

Diagnosis. Run the query with the same parameter type, for example `customer_id = 4242::numeric`: the plan is `Seq Scan` with `Filter: ((customer_id)::numeric = '4242'::numeric)`. The cast appears around the column. Comparing with `4242::bigint` returns the index plan at 0.19 ms.

Root cause. PostgreSQL has no `integer = numeric` operator inside the integer operator family; to compare, it converts the column of each row to numeric and then compares, which cannot use an index on the integer value. (MySQL behaves similarly when a string column is compared with a number: the column is converted for each row.)

Fix. Immediate: cast the parameter, not the column (`customer_id = CAST(? AS integer)`), or pass an `Integer` or `Long`. Permanent: keep parameter types aligned with column types in DTOs and repository method signatures (`findByCustomerId(Long id)`); in PostgreSQL JDBC avoid `setObject` with a type that does not match the column.

Prevention. Look for `::numeric`, `::text` or `CAST` around column names in the `Filter` line of plans; add a performance test that asserts an index scan for the key lookups.

Incident 9: deep OFFSET pagination gets slower on every page

Scenario. An admin screen pages through orders with `ORDER BY created_at, id LIMIT 20 OFFSET :page * 20`. Early pages respond instantly; the export job that walks to the last page takes minutes and the database load rises as it proceeds.

Symptoms. Latency grows linearly with the page number; the plan of a deep page shows `actual rows=1000020` inside a `Limit` that returns 20.

Diagnosis.

```sql
EXPLAIN (ANALYZE, BUFFERS) SELECT id, created_at, amount FROM orders
ORDER BY created_at, id LIMIT 20 OFFSET 1000000;
```

`Index Scan ... actual rows=1000020` and `Buffers: shared hit=5 read=17719` (463 ms) against 4 buffers (0.05 ms) for the first page.

Root cause. The database cannot jump to the millionth row; it reads and discards everything before it. Concurrent inserts also make offset pages unstable, so rows may be skipped or repeated between requests.

Fix. Switch to keyset pagination: return the last `(created_at, id)` with each page and send it back as the cursor (`WHERE (created_at, id) > (?, ?) ORDER BY created_at, id LIMIT 20`): 0.054 ms for any page. In Spring Data, `Window`/`ScrollPosition` with `KeysetScrollPosition` (Spring Data 3.1 and later) implements this; otherwise write the query with `@Query`.

Prevention. Use keyset pagination for any list that can grow beyond a few thousand rows, and for batch exports; reserve OFFSET for small, bounded result sets.

Incident 10: LIKE with a leading wildcard scans the whole table

Scenario. A product search runs `WHERE name LIKE '%M123456%'` on a table of one million products. A regular index on `name` exists, and the search is slow.

Symptoms. 220 ms per search and a plan with `Seq Scan` and `Filter: (name ~~ '%M123456%'::text)`.

Diagnosis. A B-tree orders by the beginning of the string; a pattern that starts with `%` has no usable prefix. Check the available extensions with `SELECT * FROM pg_available_extensions WHERE name = 'pg_trgm'`.

Root cause. The condition is not an index condition for a B-tree. The sequential scan read 8384 pages (`shared hit=1153 read=7231`) and removed 999 999 rows.

Fix. Create a trigram GIN index:

```sql
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_products_trgm ON products USING gin (name gin_trgm_ops);
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF) SELECT * FROM products WHERE name LIKE '%M123456%';
```

Plan after the fix (real, trimmed)

```
$ psql -d shop
 Bitmap Heap Scan on products  (cost=114.06..482.55 rows=100 width=34) (actual rows=1 loops=1)
   Recheck Cond: (name ~~ '%M123456%'::text)
   Heap Blocks: exact=1
   ->  Bitmap Index Scan on idx_products_trgm  (actual rows=1 loops=1)
         Buffers: shared hit=22
 Execution Time: 0.728 ms
```

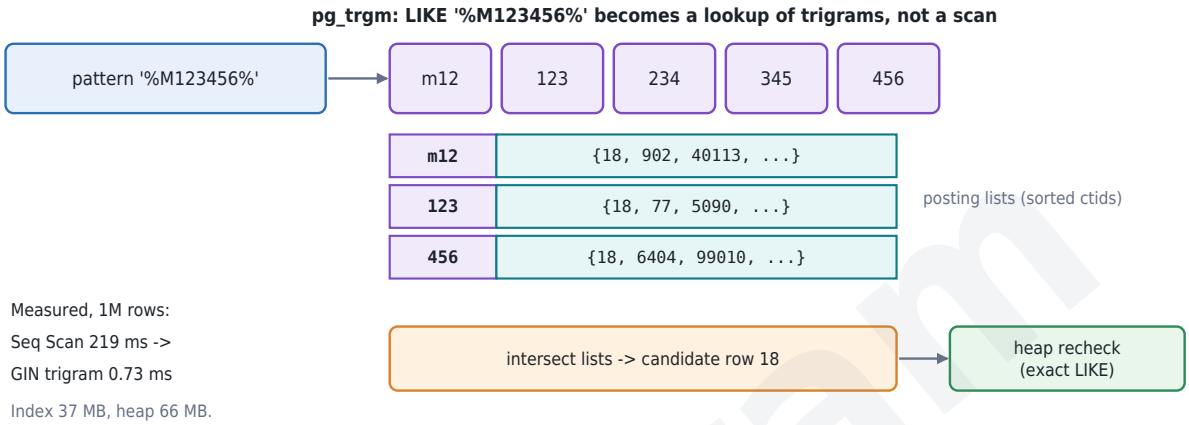


Figure 1.83. The pattern is split into three-character fragments, each fragment is looked up in the GIN index to obtain row lists, the lists are intersected, and the heap rows are rechecked with the exact pattern.

The search dropped from 219 ms to 0.73 ms (300 times). The trigram index took 37 MB against a 66 MB table. Very short patterns (fewer than three characters) and very common fragments still yield many candidates; for ranked word search `tsvector` full-text search is the better tool, and for `ILIKE '%gaming mouse%'` the planner may still choose a sequential scan when it expects many matches (here it did, with a `Limit` that stopped after 10 rows). MySQL offers `FULLTEXT` indexes with `MATCH ... AGAINST` instead of trigrams.

Prevention. Decide early whether the feature is "search" (use full text or an external search engine) or "lookup" (use exact or prefix match), and do not use `LIKE '%x%'` on large tables for interactive features without a trigram index.

1.18.5 Questions to expect

INTERVIEW ANGLE

Explain the N+1 problem and how to fix it in Spring Data JPA. One query loads N parent entities and then one query per parent loads a lazy association, producing N+1 statements. Fix by fetching the association in the same statement (JOIN FETCH or @EntityGraph), by batch fetching (default_batch_fetch_size or @BatchSize, which turns N lookups into N/size IN queries), or by using a DTO projection that selects exactly the needed columns.

1. What does sargable mean? A predicate whose form lets the engine use an index to locate rows: the indexed column stands alone on one side of the comparison, with no function, arithmetic or cast applied.
2. Why does WHERE `YEAR(order_date) = 2024` hurt performance in MySQL? The function is evaluated for every row, so the index on `order_date` cannot be used; rewrite as a range `order_date >= '2024-01-01' AND order_date < '2025-01-01'`.
3. Why is OFFSET pagination slow and what is the alternative? The server must generate and discard all skipped rows; keyset pagination filters by the last seen key and uses the index.
4. Is `COUNT(*)` faster than `COUNT(id)`? Not materially; both count rows (`COUNT(col)` skips NULLs). In PostgreSQL both scan the table or an index; for big tables use an estimate or a maintained counter.
5. How do you make `LIKE '%term%'` fast? A trigram GIN index (PostgreSQL), full-text search, or an external search engine; a B-tree cannot help.
6. When is `SELECT *` harmful? It reads every column, may include large TOAST values, prevents index-only scans, and breaks when columns are added; select the needed columns or use projections.

7. How do you rewrite a correlated subquery? Aggregate the inner table once in a derived table or CTE and join it, or use a window function or lateral join; the measured example went from 14.7 s to 64 ms.
8. How would you speed up `WHERE a = 1 OR b = 2`? Index both columns so the planner can use a bitmap OR, or rewrite as UNION ALL (disjoint) or UNION.
9. What is the cost of fetching a list with JPA entities when only two columns are displayed? Full entity hydration, extra associations and dirty-checking snapshots; a projection avoids all of it.

KEY POINTS

- Keep the indexed column bare in the predicate: ranges instead of functions, matching types instead of casts.
- Fewer rows and fewer columns: keyset pagination, projections, estimates instead of exact counts.
- Fewer round trips: fetch joins, entity graphs and batch fetching eliminate N+1.
- Rewrite correlated subqueries as a single aggregation and join; the work should happen once.
- Every claim should be checked with EXPLAIN (ANALYZE, BUFFERS) before and after.

1.19 Slow Query Diagnosis

A production complaint rarely arrives as "the plan of statement 4711 has a wrong estimate". It arrives as "the checkout page is slow since Tuesday". Diagnosis therefore has two stages: **finding** which statement is responsible, and **explaining** why it is slow. The first stage uses aggregated statistics and logs; the second uses `EXPLAIN (ANALYZE, BUFFERS)`, which earlier chapters of this part describe. This chapter sets up the instruments in PostgreSQL (`pg_stat_statements`, the slow-statement log, `auto_explain`, `pg_stat_activity`) and in MySQL (the slow query log, `performance_schema`, the `sys` schema), and ends with a repeatable procedure.

1.19.1 Concept and mechanism

Three sources of evidence

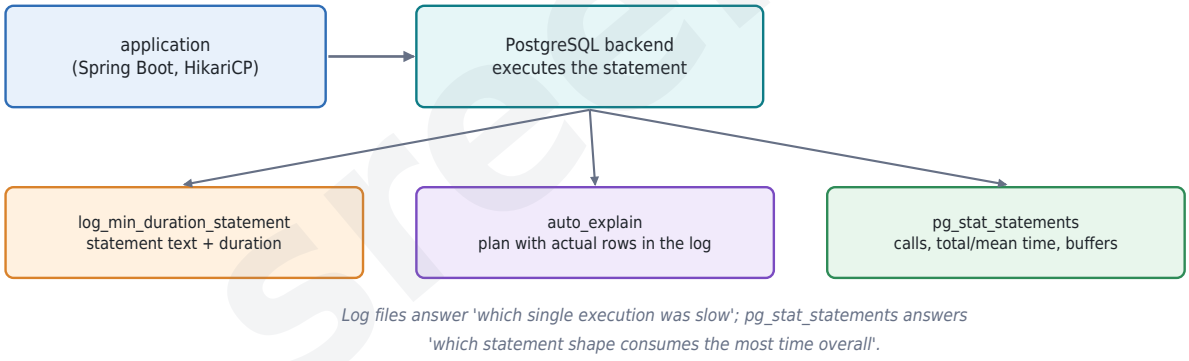


Figure 1.84. Three complementary instruments: the statement log records single slow executions, auto_explain adds the plan of those executions, and pg_stat_statements aggregates all executions by statement shape.

Aggregated statistics (`pg_stat_statements`) answer "which kind of statement consumes the most time in total". The extension normalises each statement (constants become `$1`, `$2`), and keeps cumulative counters: calls, total and mean time, rows, buffer hits and reads, temporary file use, WAL volume. It finds the death by a thousand cuts (a fast statement run millions of times, including the N+1 pattern) that no slow log would show.

The slow-statement log (`log_min_duration_statement`) answers "which individual execution was slow, and when". It records the statement text with its parameters and duration, so an incident can be tied to a time and a client.

`auto_explain` logs the *plan with actual row counts* of slow executions as they happen in production, which removes the guesswork of reproducing a problem that depends on parameter values, cache state or concurrent load.

pg_stat_statements

The module must be preloaded, which needs a server restart, and the extension created in each database:

postgresql.conf PROPERTIES

```
shared_preload_libraries = 'pg_stat_statements'
pg_stat_statements.track = all          # include statements nested in functions
compute_query_id = auto                 # on by default when the module is loaded
```

SQL

```
CREATE EXTENSION pg_stat_statements;
SELECT pg_stat_statements_reset();      -- start a measurement window
```

After a workload, rank the statements. Total time identifies where the server spends its effort; mean time identifies slow individual statements; calls identify chatty code.

SQL

```
SELECT calls,
       round(total_exec_time::numeric, 1) AS total_ms,
       round(mean_exec_time::numeric, 3)  AS mean_ms,
       rows,
       left(regexp_replace(query, '\s+', ' ', 'g'), 60) AS query
FROM pg_stat_statements
WHERE dbid = (SELECT oid FROM pg_database WHERE datname = current_database())
ORDER BY total_exec_time DESC LIMIT 6;
```

Output after a mixed workload (real) TERMINAL

```
$ psql -d shop
 calls | total_ms | mean_ms | rows  | query
-------+----------+---------+-------+------------------------------------------------------------
  5000 |    309.7 |   0.062 | 75000 | select id, amount from orders where customer_id = $1
     1 |    186.8 | 186.815 |    15 | select * from orders where customer_id=$1::numeric
     1 |    184.6 | 184.601 |     1 | select count(*) from orders where date(created_at)=$1
     1 |    176.2 | 176.185 |     1 | select count(*) from orders where status=$1
   100 |      1.2 |   0.012 |  5000 | select id from customers where id <= $1
```

Reading this output yields four findings without any plan: the first row is a statement called 5000 times, a repeated lookup (N+1 or a loop in application code); the second shows a cast in the query text; the third a function on a column; the fourth an aggregate that scans. Useful columns beyond these are `shared_blks_read` (disk-bound statements), `temp_blks_written` (spills), `rows / calls` (rows per call, which exposes unbounded results) and `stddev_exec_time` (unstable plans or parameter skew).

Column	Use
`calls`, `rows`	Frequency and rows per call; high calls with few rows suggests N+1
`total_exec_time`, `mean_exec_time`	Total cost to the server and typical latency (milliseconds)
`stddev_exec_time`, `min_exec_time`, `max_exec_time`	Variability; a large spread suggests plan or parameter dependence
`shared_blks_hit`, `shared_blks_read`	Pages from cache and from outside it; the stable measure of work
`temp_blks_read`, `temp_blks_written`	Spills past `work_mem`
`wal_bytes`	Log volume produced by write statements
`queryid`	Stable identifier to join with logs and other views

Columns of pg_stat_statements (PostgreSQL 16) used in diagnosis.

The slow-statement log and auto_explain

`log_min_duration_statement` logs every statement that runs longer than the threshold; `0` logs all statements and `-1` disables it. A threshold of 100 to 500 ms is a common production starting point. `log_line_prefix` should include timestamp, process id, user, database and application name (`'%m [%p] %u@%d %a '`), so that entries can be correlated with application logs. Statements with parameter values are logged in full, which makes the log sensitive data; protect it accordingly.

SQL

```
ALTER SYSTEM SET log_min_duration_statement = '100ms';
SELECT pg_reload_conf();            -- no restart needed
```

Server log (real) TERMINAL

```
$ psql -d shop
2026-10-18 06:57:40.931 ASIA [5046] LOG:  duration: 122.278 ms  statement: select count(*)
   from orders where status='CANCELLED'
```

`auto_explain` is a loadable module. It can be loaded per session for a test (`LOAD 'auto_explain'`, superuser) or for all sessions with `session_preload_libraries` or `shared_preload_libraries`. With `log_analyze = on` it measures every statement, which has overhead, so use `auto_explain.sample_rate` or a high `log_min_duration` in production.

SQL

```
LOAD 'auto_explain';
SET auto_explain.log_min_duration = '100ms';
SET auto_explain.log_analyze = on;
SET auto_explain.log_buffers = on;
SELECT count(*) FROM orders WHERE status = 'PAID' AND amount > 499;
```

Server log (real) TERMINAL

```
$ psql -d shop
2026-10-18 06:57:41.274 ASIA [5050] LOG:  duration: 283.924 ms  plan:
        Query Text: select count(*) from orders where status='PAID' and amount > 499;
        Aggregate  (cost=43335.40..43335.41 rows=1 width=8)
          (actual time=283.909..283.911 rows=1 loops=1)
          Buffers: shared hit=7845 read=12989
          ->  Seq Scan on orders  (cost=0.00..43334.00 rows=560 width=0)
                (actual time=0.079..283.686 rows=623 loops=1)
                Filter: ((amount > '499'::numeric) AND (status = 'PAID'::text))
                Rows Removed by Filter: 1499377
                Buffers: shared hit=7845 read=12989
```

The log entry is a complete diagnosis: a sequential scan that removes 1 499 377 rows to return 623, with estimate (560) close to actual (623). The estimate is right, so the fix is an index, not statistics.

What is running right now

`pg_stat_activity` lists sessions with their current statement, state and wait event. Long-running statements, sessions that are `idle in transaction`, and sessions waiting on locks are visible here.

SQL

```
SELECT pid, state, wait_event_type, wait_event,
       date_trunc('second', now() - query_start) AS running, left(query, 50) AS q
FROM pg_stat_activity
WHERE datname = 'shop' AND state <> 'idle' AND pid <> pg_backend_pid();
```

Output (real; the statement includes pg_sleep(4) to stay running) TERMINAL

```
$ psql -d shop
-[ RECORD 1 ]---+--------------------------------------------------
pid             | 9107
state           | active
wait_event_type | Timeout
wait_event      | PgSleep
running         | 00:00:01
q               | select count(*), pg_sleep(4) from orders where sta
```

A statement that is slow because it is **waiting** shows a `wait_event_type` of `Lock`, `IO` or `LWLock`; a statement that is slow because it is **working** shows `active` with no wait event. This distinction decides whether to look at the plan (working) or at locks, I/O and connections (waiting; see the part on transactions for locks). To stop a runaway statement use `pg_cancel_backend(pid)` (cancels the query) and, if needed, `pg_terminate_backend(pid)` (closes the connection).

MySQL instruments

Need	PostgreSQL 16	MySQL 8.x
Aggregated by statement shape	`pg_stat_statements`	`performance_schema.events_statements_summary_by_digest` and the `sys` views
Slow-statement log	`log_min_duration_statement`	`slow_query_log = ON`, `long_query_time` (default 10 s; set to 0.1 to 1 s)
Statements without index use	Inspect plans; `pg_stat_user_tables.seq_scan`	`log_queries_not_using_indexes`; `sys.statements_with_full_table_scans`
Plan of a slow execution	`auto_explain`	`EXPLAIN ANALYZE` after the fact (8.0.18 and later); optimizer trace
Digest of the slow log	`pgBadger` on the log	`mysqldumpslow`, `pt-query-digest`
Unused indexes	`pg_stat_user_indexes.idx_scan = 0`	`sys.schema_unused_indexes`
Current activity	`pg_stat_activity`	`SHOW FULL PROCESSLIST`, `performance_schema.threads`
Lock waits	`pg_locks`, `wait_event`	`performance_schema.data_lock_waits`, `sys.innodb_lock_waits`

Slow-query tooling (MySQL 8.x documented; not run on the build machine).

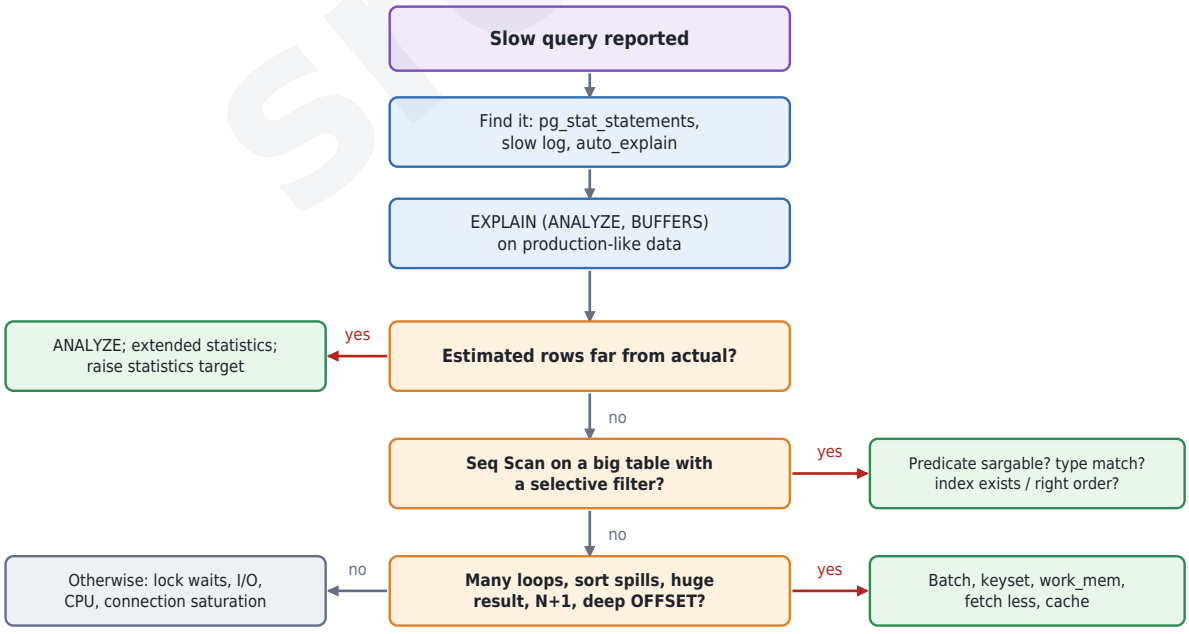
SQL

```sql
-- MySQL 8.x (typical, not run on the build machine)
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 0.5;
SET GLOBAL log_queries_not_using_indexes = ON;

SELECT digest_text, count_star, ROUND(sum_timer_wait / 1e12, 2) AS total_s,
       ROUND(avg_timer_wait / 1e9, 2) AS avg_ms, sum_rows_examined, sum_rows_sent
FROM performance_schema.events_statements_summary_by_digest
ORDER BY sum_timer_wait DESC LIMIT 5;
```

The ratio `sum_rows_examined / sum_rows_sent` is the MySQL counterpart of `Rows Removed by Filter`: a statement that examines 100 000 rows to send 10 is scanning. The slow log lines carry `Rows_examined` and `Rows_sent` for each execution in the same way.

A procedure



After each change: re-run EXPLAIN ANALYZE and compare buffers, not only milliseconds.

Figure 1.85. A decision path from complaint to fix: locate the statement, obtain the plan, compare estimates with actuals, look for scans and spills, and verify after each change.

1. **Scope.** Which endpoint, since when, for which users or parameters? Compare with deployments, data growth and batch schedules.
2. **Locate.** Rank `pg_stat_statements` by `total_exec_time`, by `mean_exec_time` and by `calls`; check `pg_stat_activity` for current waits.
3. **Reproduce.** Run the statement with the real parameter values on production-like data, with `EXPLAIN (ANALYZE, BUFFERS)`. A statement that is fast on a developer database with a thousand rows proves nothing.
4. **Compare estimates with actuals** at every node; a tenfold difference means statistics or correlation (planner chapter).
5. **Find the dominant cost**: a scan that discards almost everything (`Rows Removed by Filter`), a loop count in the hundreds of thousands, a spill (`temp`), a join that multiplies rows, a sort of the entire result.
6. **Change one thing** (index, rewrite, statistics, setting) and run EXPLAIN again; compare buffers, then time.
7. **Verify in production** with `pg_stat_statements` after resetting the window, and keep the evidence (before and after plans) with the change.

1.19.2 Worked example

From a complaint to a fix in five steps

Complaint. "The back-office order screens are slow in the afternoon."

Step 1: reset the window at the start of the afternoon peak and rank by total time after thirty minutes. The first row of the listing in this chapter is a statement called 5000 times with a mean of 0.062 ms, total 310 ms: its total is not the largest in the sample, but its call count is 50 times that of any other statement.

Step 2: the second and third rows are single executions at 185 ms. The query texts are `customer_id=$1::numeric` and `date(created_at)=$1`. Neither needs a plan to be suspected; each plan confirms it with a `Seq Scan` and a cast or function in the `Filter` line.

Step 3: fix each cause in the order of benefit.

Finding	Evidence	Fix	Result
Cast on column side	`Filter: ((customer_id)::numeric = ...)`	Bind a `Long`	548 ms to 0.19 ms
Function on column	`Filter: (date(created_at) = ...)`, 1 495 680 rows removed	Range on the bare column	351 ms to 0.6 ms
Repeated per-row lookup	calls = 5000, 15 rows each	Fetch join or batch fetch	51 statements to 2 per request
Unselective filter scan	`status = 'NEW'` returns 20 percent	Accept the scan or restructure (partial index for a queue)	no change needed

Findings, evidence and fixes in this order of impact (measured before and after on the 1.5 million row table).

Step 4: the fourth row is not a defect. `status = $1` selects a fifth of the table, and a scan is the correct plan for that selectivity. Not every full scan is a problem; the diagnosis must separate scans that discard rows from scans that need most of them.

Step 5: reset `pg_stat_statements` after the deployment and confirm that the totals of the affected statements fell. The reset is part of the method, because cumulative counters carry the history.

1.19.3 Hands-on

Enable the slow log, run a workload, then rank it.

```bash
$ psql -d shop -c "SELECT pg_stat_statements_reset()"
$ pgbench -n -f nplus1.sql -t 100 -c 1 shop          # a workload script of your own
$ psql -d shop -c "SELECT calls, round(total_exec_time::numeric,1) total_ms, \
                   left(query,60) FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 5"
```

`pgbench` is the benchmarking tool shipped with PostgreSQL. A script file of SQL (with `\set` lines for random parameters) is replayed by `-c` clients for `-t` transactions or `-T` seconds, and it reports latency average and transactions per second. The N+1 and batched variants in this chapter were driven this way. On the build machine the 51-statement script

averaged 7 to 11 ms per transaction and the equivalent 2-statement script 4 to 7 ms over a Unix-domain socket (measurements varied between runs because the machine is shared; the difference is dominated by statement count, and over a network with 0.5 ms round trip it grows by 0.5 ms per extra statement).

1.19.4 Real-world problems and solutions

Incident 11: fast in psql, slow in the application

Scenario. A query runs in 2 ms in `psql` with literal values but takes 800 ms from the Spring Boot service for some customers. Developers cannot reproduce it.

Symptoms. Latency distribution is bimodal; `pg_stat_statements` shows a large `stddev_exec_time` and `max_exec_time` for the statement; the slow executions occur on pooled connections that have run the statement several times.

Diagnosis. Enable `auto_explain` for the application role with a threshold above the normal latency, and read the plan of a slow execution from the log:

```SQL
ALTER ROLE app SET auto_explain.log_min_duration = '300ms';
ALTER ROLE app SET auto_explain.log_analyze = on;
ALTER ROLE app SET session_preload_libraries = 'auto_explain';
```

The log shows the generic plan (parameters `$1`) with a nested loop for a customer who has 200 000 rows, estimated from the average. Compare with `EXPLAIN (ANALYZE)` using the literal for the heavy customer.

Root cause. Server-side prepared statements switch to a generic plan after five executions, and the generic plan fits an average customer, not a very large one (data skew).

Fix. Immediate: `ALTER ROLE app SET plan_cache_mode = force_custom_plan`. Permanent: keep the statistics current for the skewed column (`SET STATISTICS`), or route large customers through a different query.

Prevention. Test with the largest and the smallest tenant; keep `auto_explain` enabled with a conservative threshold so that such cases are caught the first time.

Incident 12: the slow query is found, but the table scan is the right plan

Scenario. A report on an audit table with 50 million rows scans the whole table daily. A developer adds an index on the filtered column to "fix" it; the planner ignores it, and the new index slows the writes of the audit table by 30 percent.

Symptoms. The new index has `idx_scan = 0`; the plan is unchanged; insert latency is higher.

Diagnosis. `EXPLAIN` shows `Seq Scan` with an estimate of 40 percent of rows returned, and the actual rows agree. The index cannot help a query that needs 40 percent of the table.

Root cause. The problem is the access pattern, not the plan: the report reads a large share of the data every day.

Fix. Drop the index (`DROP INDEX CONCURRENTLY`). Reduce the work: precompute a daily summary table, use a materialised view refreshed after the load, partition the table by date so that each report reads one partition, or move the report to a replica. A covering index may help only if it makes the scan narrow enough for an index-only scan.

Prevention. Before adding an index, estimate selectivity from `pg_stats` and check that the predicate returns a small fraction; record the expected plan in the change request and confirm it afterwards.

1.19.5 Questions to expect

INTERVIEW ANGLE
A query is slow in production. How do you investigate it? Locate the statement with pg_stat_statements or the slow log, reproduce it with the real parameters on realistic data, run EXPLAIN (ANALYZE, BUFFERS), compare estimated with actual rows, look for scans that discard rows, large loop counts, spills and sorts, change one thing at a time, and verify in production by comparing statistics before and after.

1. What does pg_stat_statements show that the slow query log does not? Aggregates by statement shape over all executions, so many fast executions (N+1) and total cost are visible; the log shows only single executions above a threshold.
2. How do you capture the plan of a query that is only slow sometimes? auto_explain with a duration threshold, which logs the actual plan of slow executions including parameters-dependent plans.

3. How do you tell whether a slow statement is working or waiting? pg_stat_activity: an active session with no wait event is using CPU or I/O in its plan; a wait_event_type of Lock, IO or LWLock indicates waiting.
4. What are Rows_examined and Rows_sent in MySQL? The rows the engine read and the rows returned to the client; a large ratio indicates a scan or a weak index.
5. Why reset pg_stat_statements? The counters are cumulative since the last reset, so a measurement window around a change gives a clean before and after.
6. Why is average latency a poor metric? A few very slow executions hide behind a low mean; use the maximum, the standard deviation and percentiles from application metrics.
7. Is a sequential scan always a problem? No; it is correct for small tables and for predicates that return a large fraction of the rows.
8. What is the first thing to check in a plan? Estimated versus actual row counts at each node, then the node with the largest actual time or buffer count.

KEY POINTS

- Find first, explain second: pg_stat_statements for totals and call counts, the slow log for single executions, auto_explain for plans in production.
- Distinguish working from waiting with pg_stat_activity before reading any plan.
- A repeated cheap statement (high `calls`) is as costly to the application as a single slow statement is to the database.
- Change one thing at a time and compare buffers and time before and after; reset statistics to measure the effect in production.
- MySQL offers the same tools under different names: slow query log, performance_schema digests and the sys schema.

1.20 Transactions and ACID

A database that stores data but cannot protect it during change is of little use for an application that moves money, reserves seats, or decrements stock. A single business operation usually requires several SQL statements: a debit and a credit, an order row and its order lines, a stock decrement and a reservation. Between any two of these statements a process may crash, a constraint may fail, a network connection may drop, or another client may read or modify the same rows. A **transaction** is the mechanism that groups statements into one logical unit of work so that the database, and not the application, enforces correct behaviour in each of those situations.

This chapter defines the transaction abstraction, states the four ACID properties together with the specific failure each one prevents, and shows the behaviour on a real PostgreSQL 16 server. The remaining chapters of this Part examine how isolation, multiversion concurrency control, locking, write-ahead logging and the Java transaction API realise these guarantees.

1.20.1 Concept and mechanism

DEFINITION

A transaction is a sequence of database operations that the DBMS treats as a single unit with respect to failure and concurrency. It ends either with `COMMIT`, which makes all of its effects permanent and visible, or with `ROLLBACK`, which discards all of them.

The four ACID properties

Atomicity states that all operations of a transaction take effect, or none of them does. The failure prevented is a partially applied operation: a debit recorded without the matching credit. The DBMS implements atomicity by recording, for every row version a transaction creates, which transaction created it, and by consulting a durable record of each transaction outcome. A row version written by an aborted transaction is never returned to a reader. In PostgreSQL the abort path costs almost nothing because the old row versions remain in place; in InnoDB the old versions are restored from the undo log.

Consistency states that a transaction moves the database from one valid state to another valid state, where valid means that every declared integrity constraint holds: `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK`, `NOT NULL`. The failure prevented is the visibility of data that violates a business rule. Consistency in the ACID sense is a joint responsibility. The DBMS enforces the declared constraints; the application is responsible for declaring all rules that matter. A rule that is not declared (for example "the sum of order line amounts equals the order total") is not protected by ACID.

Isolation states that concurrently executing transactions do not observe each other's intermediate states. The failure prevented is a family of anomalies (dirty reads, non-repeatable reads, phantoms, lost updates, write skew) analysed in the next chapter. Isolation is configurable because the strongest form, serial execution, is also the most expensive.

Durability states that once the DBMS has acknowledged `COMMIT`, the effects survive a crash of the process, the operating system or the machine. The failure prevented is the loss of a confirmed commit. The mechanism is the write-ahead log: the log record describing the commit is forced to stable storage before the acknowledgement is sent.

	Property	Guarantee	Failure prevented	Mechanism
A	Atomicity	all statements apply or none	crash or error after debit, before credit	undo / abort marking
C	Consistency	constraints hold at commit	balance < 0 or orphan row becomes visible	CHECK, FK, UNIQUE
I	Isolation	concurrent transactions do not see partial effects	two sessions overwrite each other's update	MVCC snapshots, locks
D	Durability	committed data survives a crash	power loss loses a confirmed commit	WAL flushed before ack

Figure 1.86. Each ACID property is paired with the failure it prevents and the mechanism that provides it; atomicity and durability rely on the log, isolation on MVCC and locks, consistency on declared constraints.

Transaction boundaries and states

A transaction begins explicitly with `BEGIN` (or `START TRANSACTION`) or implicitly with the first statement when the client library has disabled autocommit. It ends with `COMMIT` or `ROLLBACK`. A transaction passes through the states shown below. The state "partially committed" exists between the moment the client issues `COMMIT` and the moment the commit record is durable; a failure to flush the log in that interval moves the transaction to the failed state.

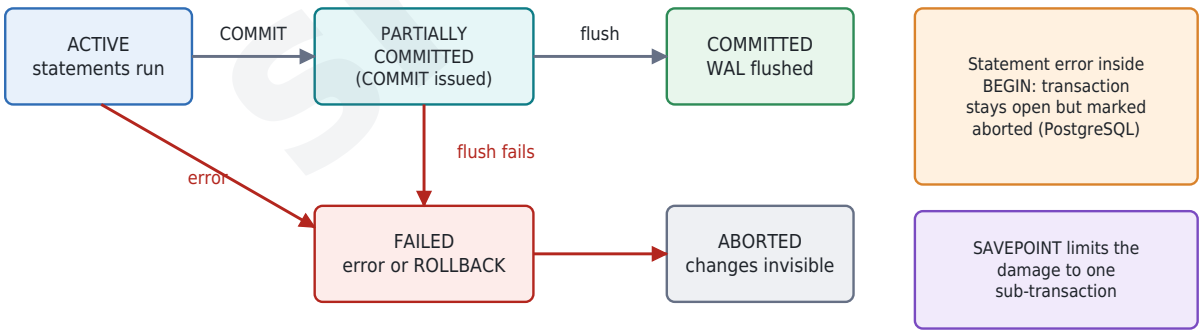


Figure 1.87. State machine of a transaction: any error leads to failed and then aborted, and only a successful log flush leads to committed.

PostgreSQL has one notable behaviour. When a statement fails inside an explicit transaction block, the whole transaction enters an aborted state. Every following statement is rejected with `current transaction is aborted, commands ignored until end of transaction block` until `ROLLBACK` is issued. A `COMMIT` issued in this state is silently converted to a `ROLLBACK`, which the server reports through the command tag. MySQL InnoDB behaves differently: a failed statement rolls back only that statement, and the transaction stays usable, with the exceptions of deadlock and lock wait timeout in some configurations.

Autocommit

Under **autocommit**, every statement is its own transaction, committed as soon as it completes. PostgreSQL (psql and the JDBC driver by default) and MySQL both start in autocommit mode. Two consequences matter to application developers. First, a business operation consisting of two statements issued under autocommit is two transactions, and no atomicity exists between them. Second, JDBC disables autocommit when `Connection.setAutoCommit(false)` is called; Spring does this on behalf of the application when a `@Transactional` method starts.

Aspect	PostgreSQL 16	MySQL 8.x (InnoDB)
Start	`BEGIN` or `START TRANSACTION`	`START TRANSACTION` or `BEGIN`
Default mode	autocommit on	autocommit on (`SET autocommit=0` to disable)
Statement error in a transaction	whole transaction aborted	only the statement is rolled back
DDL inside a transaction	transactional; `CREATE TABLE` can be rolled back	implicit commit before and after most DDL
Savepoints	`SAVEPOINT`, `ROLLBACK TO`, `RELEASE`	same syntax
Non-transactional engine	none	MyISAM (not crash safe; avoid)

Transaction control differences between PostgreSQL 16 and MySQL 8.x (InnoDB).

The fourth row has operational consequences. A schema migration that contains five DDL statements runs in a single atomic unit in PostgreSQL, so a failure at statement four leaves the schema unchanged. In MySQL the first three statements have already been committed, and the migration tool must be prepared for a half-migrated schema.

Savepoints

A **savepoint** marks a position inside a transaction to which the transaction can return without abandoning the work done before it. `ROLLBACK TO SAVEPOINT name` undoes the statements issued after the savepoint and also clears the aborted state caused by an error in that region. Internally PostgreSQL implements a savepoint as a subtransaction with its own transaction identifier. JDBC exposes the feature through `Connection.setSavepoint()` and `rollback(Savepoint)`; Spring uses it for `Propagation.NESTED`.

1.20.2 Worked example

The scenario is a transfer of 600 from alice to bob, where the table carries `CHECK (balance >= 0)` and alice holds 500. The expected result is that no balance changes, because the debit would violate the constraint.

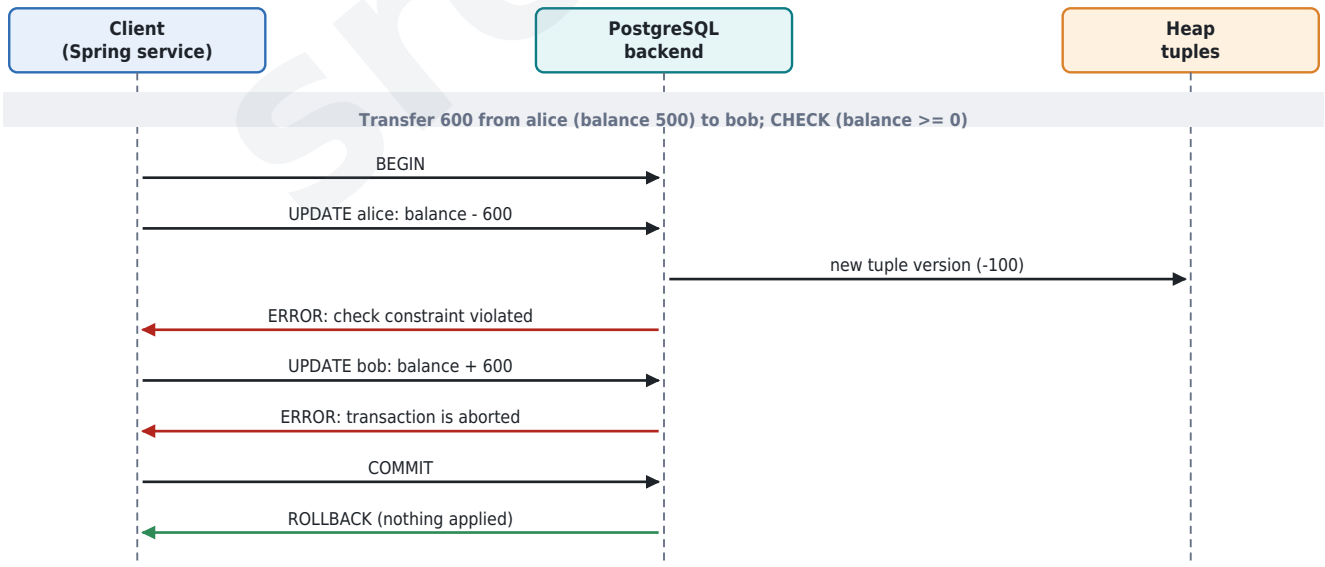


Figure 1.88. The debit fails at the CHECK constraint, the credit statement is then rejected because the transaction is aborted, and COMMIT turns into a ROLLBACK, so neither row changes.

Trace of the steps:

1. `BEGIN` opens the transaction and assigns no transaction identifier yet; an identifier is allocated lazily at the first write.

2. `UPDATE ... balance = balance - 600 WHERE id = 1` creates a new row version with balance -100. The `CHECK` constraint is evaluated before the statement completes and rejects the row. The error aborts the transaction.
3. The credit to bob is refused without being executed, because the transaction is already aborted.
4. `COMMIT` has nothing to commit. The server answers `ROLLBACK`, and a later `SELECT` shows both balances unchanged.

1.20.3 Hands-on

The following session ran on the build machine against PostgreSQL 16.15. The table is created and filled first.

setup.sql — SQL

```
CREATE TABLE account (
    id      int PRIMARY KEY,
    owner   text,
    balance numeric(12,2) CHECK (balance >= 0)
);
INSERT INTO account VALUES (1, 'alice', 500), (2, 'bob', 100);
```

Rollback discards all changes

psql session (real output)

```
t=# BEGIN;
BEGIN
t=*# UPDATE account SET balance = balance - 200 WHERE id = 1;
UPDATE 1
t=*# UPDATE account SET balance = balance + 200 WHERE id = 2;
UPDATE 1
t=*# SELECT * FROM account ORDER BY id;
 id | owner | balance
----+-------+---------
  1 | alice |  300.00
  2 | bob   |  300.00
(2 rows)

t=*# ROLLBACK;
ROLLBACK
t=# SELECT * FROM account ORDER BY id;
 id | owner | balance
----+-------+---------
  1 | alice |  500.00
  2 | bob   |  100.00
(2 rows)
```

The asterisk in the prompt `t=*#` indicates an open transaction. Inside the transaction the session sees its own uncommitted changes; after `ROLLBACK` they are gone.

Constraint failure aborts the transaction

psql session (real output)

```
t=# BEGIN;
BEGIN
t=*# UPDATE account SET balance = balance - 600 WHERE id = 1;
ERROR:  new row for relation "account" violates check constraint "account_balance_check"
DETAIL:  Failing row contains (1, alice, -100.00).
t=!# UPDATE account SET balance = balance + 600 WHERE id = 2;
ERROR:  current transaction is aborted, commands ignored until end of transaction block
t=!# COMMIT;
ROLLBACK
t=# SELECT * FROM account ORDER BY id;
 id | owner | balance
----+-------+---------
  1 | alice |  500.00
  2 | bob   |  100.00
(2 rows)
```

The prompt marker changes to `!` in the aborted state, and `COMMIT` returns the tag `ROLLBACK`. An application must therefore treat the exception of the failing statement as the end of the transaction and not rely on the outcome of a later `COMMIT`.

sreeram

Savepoints isolate a failing statement

psql session (real output)

```
t=# BEGIN;
BEGIN
t=*# INSERT INTO account VALUES (3, 'carol', 10);
INSERT 0 1
t=*# SAVEPOINT s1;
SAVEPOINT
t=*# INSERT INTO account VALUES (1, 'dup', 0);
ERROR:  duplicate key value violates unique constraint "account_pkey"
DETAIL:  Key (id)=(1) already exists.
t=!# ROLLBACK TO SAVEPOINT s1;
ROLLBACK
t=*# INSERT INTO account VALUES (4, 'dave', 20);
INSERT 0 1
t=*# COMMIT;
COMMIT
```

The duplicate insert failed, but the rollback to `s1` restored a usable transaction, and the rows for carol and dave were committed together. Rows 3 and 4 exist; the failing row never did.

Autocommit off in psql

psql session (real output)

```
t=# \set AUTOCOMMIT off
t=# UPDATE account SET balance = 0 WHERE id = 4;
UPDATE 1
t=*# ROLLBACK;
ROLLBACK
t=# SELECT balance FROM account WHERE id = 4;
 balance
---------
   20.00
(1 row)
```

With `AUTOCOMMIT off` psql opens a transaction implicitly before the first statement, which is what the JDBC call `setAutoCommit(false)` achieves for Java code.

1.20.4 Java view of the same transaction

The following fragment shows the plain JDBC form of the transfer. It is a fragment, because no JDBC driver is available on the build machine; the SQL semantics were verified with psql above.

TransferDao.java (fragment, not run) JAVA

```
public void transfer(DataSource ds, int from, int to, BigDecimal amount) throws SQLException {
    try (Connection con = ds.getConnection()) {
        con.setAutoCommit(false);                       // BEGIN is implicit
        try (PreparedStatement debit = con.prepareStatement(
                 "UPDATE account SET balance = balance - ? WHERE id = ?");
             PreparedStatement credit = con.prepareStatement(
                 "UPDATE account SET balance = balance + ? WHERE id = ?")) {
            debit.setBigDecimal(1, amount);
            debit.setInt(2, from);
            debit.executeUpdate();
            credit.setBigDecimal(1, amount);
            credit.setInt(2, to);
            credit.executeUpdate();
            con.commit();                               // all or nothing
        } catch (SQLException e) {
            con.rollback();                             // discard both updates
            throw e;
        }
    }
}
```

The `try` block pairs every `setAutoCommit(false)` with either `commit()` or `rollback()`. A path that returns the connection to the pool without either leaves the transaction open on the server, a defect examined in the chapter on Java and Spring transactions.

1.20.5 Real-world problems and solutions

Incident: half a transfer after an application crash

Scenario. A legacy service executes the debit and the credit of a transfer as two separate JDBC calls on a connection in autocommit mode. The service host is restarted during a deployment.

Symptoms. A reconciliation job reports a total across all accounts that is lower than the total of the previous day by 600. One customer reports a debit with no corresponding credit.

Diagnosis.

1. Compare the sum of balances with the ledger of transfers.
2. Search the transfer log for transfers that have a debit entry without a credit entry.
3. Check the connection setting in the service: `SHOW AUTOCOMMIT` in MySQL, or in PostgreSQL the log with `log_statement = 'mod'`, which shows `BEGIN`/`COMMIT` around statements, or their absence.

reconcile.sql — SQL

```sql
SELECT t.id, t.amount
FROM transfer_log t
LEFT JOIN ledger_entry c ON c.transfer_id = t.id AND c.side = 'CREDIT'
WHERE c.id IS NULL;
```

Root cause. The two statements were separate transactions. The first committed immediately; the process terminated before the second one was sent, so atomicity was never available.

Fix. Immediate: for every orphaned debit, apply the missing credit inside a transaction that also marks the transfer as repaired. Permanent: execute both statements within one transaction, with `@Transactional` on the service method or the explicit `setAutoCommit(false)` pattern above.

Prevention. Enable an integration test that terminates the connection between the two statements (for example through `pg_terminate_backend`) and asserts that the balances are unchanged. Declare double-entry invariants through a deferred constraint trigger or by periodic reconciliation.

Incident: migration half applied on MySQL

Scenario. A release script adds a column, back-fills it and adds a `NOT NULL` constraint. The back-fill statement fails with a data error.

Symptoms. The application fails at startup because the schema is in an intermediate state: the column exists but is nullable. Re-running the migration fails with `Duplicate column name`.

Diagnosis. `SHOW CREATE TABLE` reveals the intermediate schema. The migration tool history table (for example `flyway_schema_history`) marks the migration as failed.

Root cause. MySQL commits implicitly before and after each DDL statement, so the first `ALTER TABLE` could not be rolled back together with the failing statement. In PostgreSQL the same migration, run inside a single transaction, would leave no trace.

Fix. Immediate: manually drop the column or complete the migration, then repair the migration history. Permanent: split the migration into idempotent steps (`ADD COLUMN IF NOT EXISTS` is available in MariaDB and PostgreSQL but not in MySQL 8.0, which requires a guard through `information_schema`).

Prevention. Keep each migration to one DDL statement on MySQL; on PostgreSQL rely on transactional DDL, with the exceptions of statements such as `CREATE INDEX CONCURRENTLY`, which cannot run inside a transaction block.

1.20.6 Questions to expect

INTERVIEW ANGLE
What does ACID stand for, and which property does a write-ahead log mainly provide? Atomicity, Consistency, Isolation, Durability. The write-ahead log provides durability, because the commit record is flushed before the acknowledgement, and it also supports atomicity, because recovery can identify and ignore the effects of transactions without a commit record.

1. Which ACID property is the responsibility of the application rather than the DBMS? Consistency. The DBMS enforces declared constraints only; business rules that are not declared as constraints are not protected.

2. What happens in PostgreSQL after an error inside a transaction block? The transaction is marked aborted, and every statement fails until `ROLLBACK` or `ROLLBACK TO SAVEPOINT`. `COMMIT` performs a rollback.
3. Does DDL participate in transactions? In PostgreSQL, yes, with a few exceptions such as `CREATE INDEX CONCURRENTLY`. In MySQL, DDL causes an implicit commit.
4. What is autocommit and why does it matter for atomicity? Every statement is its own transaction. Two statements executed in autocommit mode are not atomic as a pair.
5. What is a savepoint used for? It allows the rollback of part of a transaction, for example to continue after a failing insert. Spring `NESTED` propagation maps to it.
6. Can a committed transaction be lost? Only if durability is relaxed (`synchronous_commit = off`, `fsync = off`, a volatile disk cache) or storage is damaged. The next chapters quantify this.
7. Does a read-only query need a transaction? Every statement runs in a transaction. Explicit read-only transactions (`BEGIN READ ONLY`) additionally provide a stable snapshot at higher isolation levels.

KEY POINTS

Key points

- A transaction groups statements into an all-or-nothing unit; `COMMIT` publishes, `ROLLBACK` discards.
- Atomicity and durability depend on the log; isolation depends on MVCC and locks; consistency depends on declared constraints.
- PostgreSQL aborts the whole transaction on a statement error; savepoints contain the damage. MySQL rolls back only the failing statement.
- PostgreSQL DDL is transactional; MySQL DDL commits implicitly.
- Autocommit makes every statement its own transaction, so multi-statement business operations need an explicit transaction.

1.21 Concurrency Anomalies and Isolation Levels

Isolation answers the question of what one transaction may observe while others execute at the same time. The ideal, called **serializability**, states that the outcome of any concurrent execution equals the outcome of some serial execution of the same transactions. Enforcing it for every workload is expensive, so the SQL standard defines weaker **isolation levels** that tolerate named anomalies in exchange for higher throughput. The choice of level is a correctness decision, and the defaults differ between engines: PostgreSQL defaults to `READ COMMITTED`, MySQL InnoDB to `REPEATABLE READ`.

This chapter defines each anomaly, reproduces it with two real sessions on PostgreSQL 16.15, and then states what each level in PostgreSQL and in MySQL InnoDB permits. All outputs labelled real were produced on the build machine with two `psql` sessions, A and B, executing the listed statements in the order given.

1.21.1 Concept and mechanism

Anomalies

- **Dirty read.** Transaction A reads a change made by transaction B that has not committed. If B rolls back, A has used data that never existed.
- **Non-repeatable read.** A reads a row twice and obtains different values because B committed an update between the two reads.
- **Phantom read.** A executes the same predicate query twice and obtains different sets of rows because B committed an insert or delete that matches the predicate.
- **Lost update.** A and B both read a value, compute a new value in the application, and write it back; the second write overwrites the first, which is lost.
- **Write skew.** A and B each read an overlapping set of rows, then write to different rows in a way that together violates an invariant that each transaction preserved on its own.

The standard levels and what the engines implement

The SQL standard defines the four levels `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ` and `SERIALIZABLE` through the anomalies each one forbids. The definitions are loose, and real engines implement the levels with snapshots or locks in ways that do not match the standard table exactly. The matrix below describes the actual behaviour of the two engines.

Level	Dirty read	Non-repeatable read	Phantom	Lost update (RMW)	Write skew
READ COMMITTED PostgreSQL default	no	possible	possible	possible	possible
REPEATABLE READ PostgreSQL	no	no	no	error 40001	possible
SERIALIZABLE PostgreSQL (SSI)	no	no	no	error 40001	error 40001
REPEATABLE READ InnoDB default	no	no	mostly no (gap locks)	possible (plain SELECT)	possible
SERIALIZABLE InnoDB	no	no	no	no (locks)	no (locks)

Figure 1.89. Actual anomaly behaviour per engine and level: PostgreSQL REPEATABLE READ already forbids phantoms but not write skew, and only SERIALIZABLE closes write skew by aborting one transaction with error 40001.

Level	PostgreSQL 16	MySQL 8.x InnoDB
READ UNCOMMITTED	accepted as a synonym of READ COMMITTED; dirty reads never occur	dirty reads possible
READ COMMITTED	new snapshot per statement; default	new read view per statement; gap locks mostly off
REPEATABLE READ	one snapshot at first statement; write conflicts raise 40001	default; consistent read from the first read view; next-key locks for locking reads
SERIALIZABLE	Serializable Snapshot Isolation (SSI): snapshot plus conflict detection, no blocking	plain SELECT turns into `SELECT ... FOR SHARE`; locks and deadlocks

Implementation of the levels in PostgreSQL 16 and MySQL 8.x InnoDB.

Snapshot isolation versus locking

PostgreSQL READ COMMITTED and REPEATABLE READ are built on MVCC snapshots, described in the next chapter: a reader never blocks a writer and a writer never blocks a reader. The difference between the two levels is the lifetime of the snapshot. In READ COMMITTED every statement obtains a new snapshot, so two statements of one transaction can disagree. In REPEATABLE READ one snapshot is taken at the first statement and kept until the end, so all statements agree.

A write-write conflict is handled by waiting for the other transaction. When the other transaction commits, behaviour depends on the level. In READ COMMITTED the waiting statement re-reads the newest committed version of the row and re-evaluates its `WHERE` clause (the **EvalPlanQual** step), and then proceeds. In REPEATABLE READ and SERIALIZABLE the waiting statement cannot proceed against a row that changed after its snapshot, so it fails with `ERROR: could not serialize access due to concurrent update` (SQLSTATE 40001). The application must retry the whole transaction.

SSI extends snapshot isolation with detection of dangerous read-write dependency patterns, as shown below. Transactions record what they read using predicate-like **SIREAD locks**, which never block. When two concurrent transactions each read data the other writes (two consecutive read-write antidependencies in a cycle), one of them is aborted with SQLSTATE 40001.

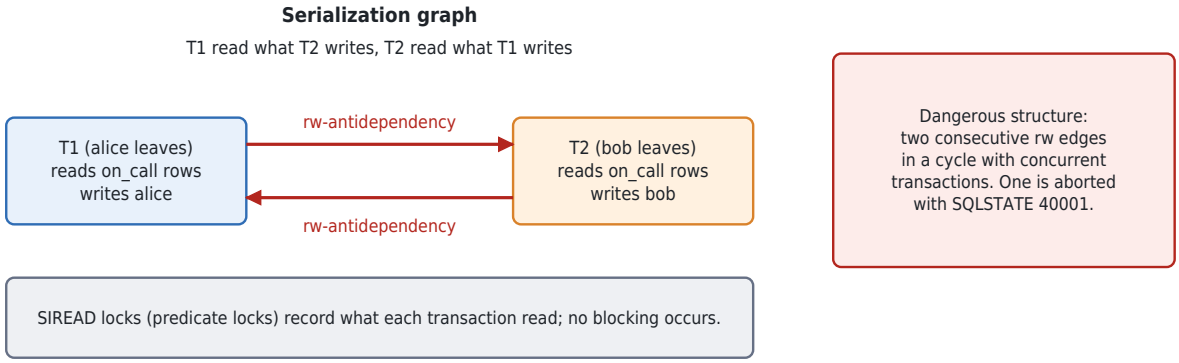


Figure 1.90. In the on-call example each transaction reads rows that the other updates, which forms a cycle of read-write antidependencies; SSI aborts one participant instead of blocking.

InnoDB uses an alternative design. A plain `SELECT` under REPEATABLE READ reads from a read view (a snapshot built from undo logs). Locking reads (`SELECT ... FOR UPDATE`, `UPDATE`, `DELETE`) and writes always operate on the latest committed row, not the snapshot, and use **next-key locks**, a combination of a record lock and a gap lock on the index range scanned, which prevents other transactions from inserting into that range and thereby prevents phantoms for locking statements. Because `UPDATE` reads the latest version, an increment such as `SET balance = balance - 50` does not lose updates under any InnoDB level; a read-modify-write cycle with the value computed in the application can lose updates under REPEATABLE READ, because the plain `SELECT` reads the snapshot.

1.21.2 Worked example

The following step-by-step trace uses the lost update scenario. The account balance is 500. Session A withdraws 100 and session B withdraws 50. The application reads the balance, subtracts the amount, and writes the result.

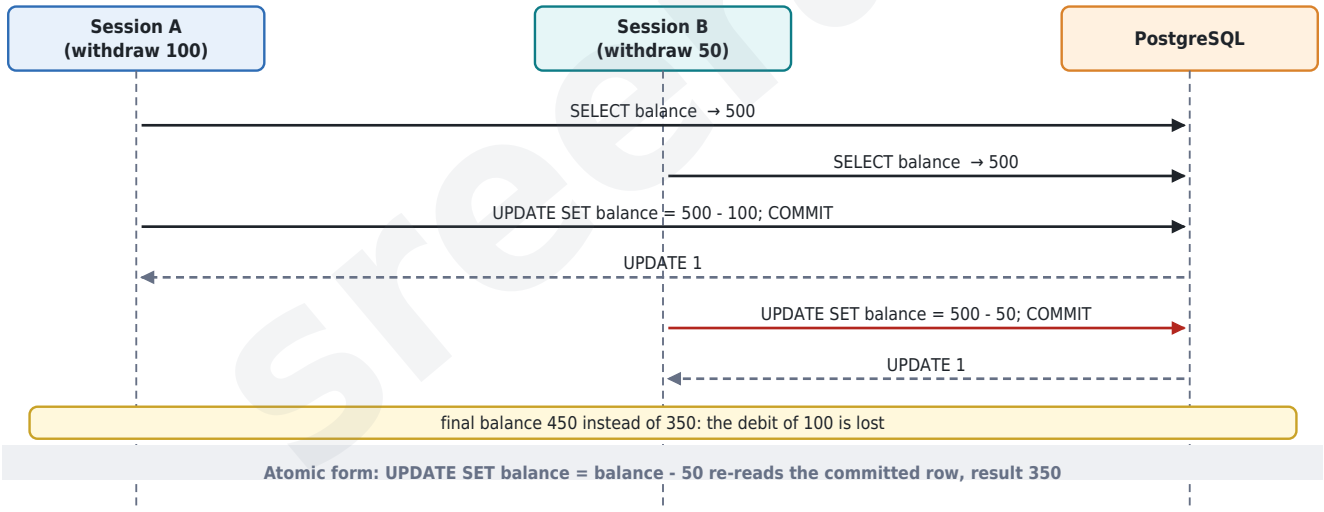


Figure 1.91. Both sessions read 500; each writes a value computed from the stale read, so the second commit silently overwrites the first and the final balance is 450 instead of 350.

1. A and B start at READ COMMITTED. Both `SELECT balance` and receive 500.00.
2. A writes `balance = 500 - 100` and commits. The row holds 400.
3. B writes `balance = 500 - 50` (the 500 is a Java variable) and commits. The row holds 450. The first write is lost.

Replacing the write by `UPDATE account SET balance = balance - 50 WHERE id = 1` changes step 3: B waits for A to commit, re-reads the row (400), and computes 350. The result is correct without any change of isolation level.

1.21.3 Hands-on

The table used throughout is `account(id, owner, balance)` with rows (1, alice, 500.00) and (2, bob, 100.00). Each experiment starts from this state.

Dirty read is impossible in PostgreSQL

two sessions, real output

```
A: BEGIN; UPDATE account SET balance = 0 WHERE id = 1;
B: BEGIN TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
B: SELECT balance FROM account WHERE id = 1;
 balance
---------
  500.00
B: COMMIT;
A: ROLLBACK;
```

B requested `READ UNCOMMITTED` and still saw the committed value 500.00. PostgreSQL treats the level as `READ COMMITTED`.

Non-repeatable read at READ COMMITTED

two sessions, real output

```
A: BEGIN;
A: SELECT balance FROM account WHERE id = 1;      -- 500.00
B: UPDATE account SET balance = 450 WHERE id = 1; -- autocommit
A: SELECT balance FROM account WHERE id = 1;      -- 450.00
A: COMMIT;
```

The same query inside one transaction returned 500.00 and then 450.00. See the sequence diagram that follows for the interleaving.

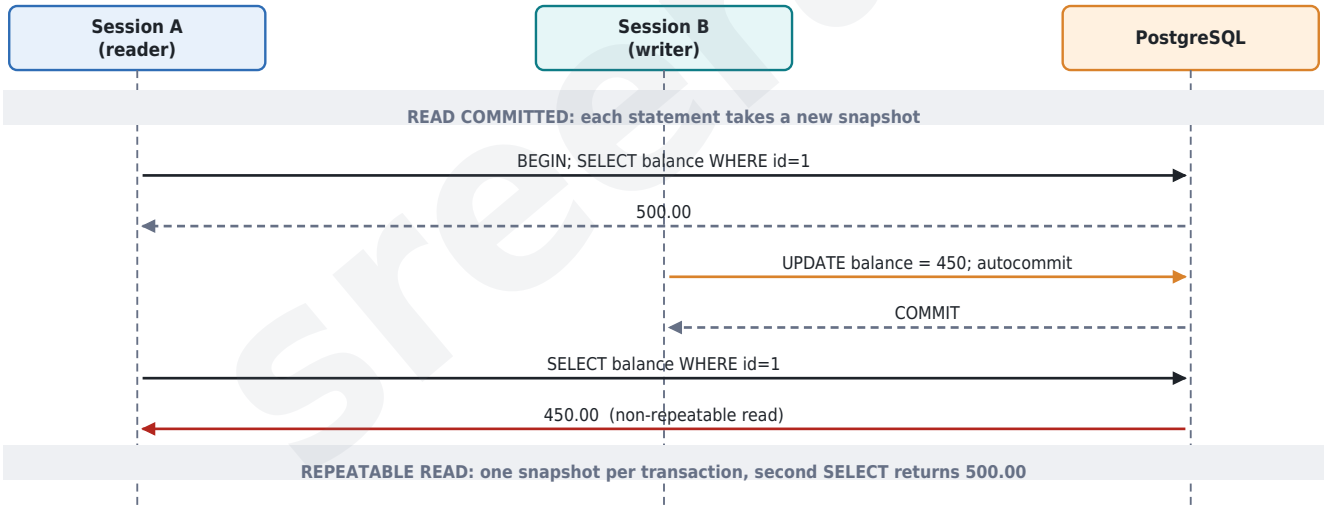


Figure 1.92. Under READ COMMITTED the second SELECT sees the update committed by B; under REPEATABLE READ it would still return 500.00.

sreeram

Phantom read at READ COMMITTED, none at REPEATABLE READ

two sessions, real output

```
-- READ COMMITTED
A: BEGIN; SELECT count(*), sum(balance) FROM account;
 count |  sum
-------+--------
     2 | 600.00
B: INSERT INTO account VALUES (3, 'carol', 50);
A: SELECT count(*), sum(balance) FROM account; COMMIT;
 count |  sum
-------+--------
     3 | 650.00

-- REPEATABLE READ (table now holds three rows)
A: BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT count(*), sum(balance) FROM account;
     3 | 650.00
B: INSERT INTO account VALUES (4, 'dave', 70);
A: SELECT count(*), sum(balance) FROM account; COMMIT;
     3 | 650.00
```

At READ COMMITTED the aggregate changed from 2 rows to 3 within one transaction. At REPEATABLE READ the inserted row 4 stayed invisible, so PostgreSQL REPEATABLE READ also prevents phantoms, which is stronger than the standard demands.

Lost update by read-modify-write

two sessions, real output

```
A: BEGIN; SELECT balance FROM account WHERE id = 1;       -- 500.00
B: BEGIN; SELECT balance FROM account WHERE id = 1;       -- 500.00
A: UPDATE account SET balance = 500 - 100 WHERE id = 1; COMMIT;
B: UPDATE account SET balance = 500 - 50  WHERE id = 1; COMMIT;
A: SELECT balance FROM account WHERE id = 1;
 balance
---------
  450.00
```

The correct final balance is 350.00. The debit of 100 vanished without any error.

The atomic update avoids the anomaly

two sessions, real output

```
A: BEGIN; UPDATE account SET balance = balance - 100 WHERE id = 1;
B: BEGIN; UPDATE account SET balance = balance - 50  WHERE id = 1;   -- blocks
A: COMMIT;
B: COMMIT; SELECT balance FROM account WHERE id = 1;
 balance
---------
  350.00
```

B blocked until A committed, then re-evaluated against the committed row 400.00 and wrote 350.00.

REPEATABLE READ detects the conflict instead

two sessions, real output

```
A: BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT balance FROM account WHERE id = 1;  -- 500.00
B: BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT balance FROM account WHERE id = 1;  -- 500.00
A: UPDATE account SET balance = 400 WHERE id = 1; COMMIT;
B: UPDATE account SET balance = 450 WHERE id = 1;
ERROR:  could not serialize access due to concurrent update
B: ROLLBACK;
```

The error is a signal to retry the transaction from the beginning; the lost update is converted into a visible failure.

Write skew with doctors on call

The business rule is that at least one doctor must stay on call. The table is `doctor(name, on_call)` with alice and bob both on call.

sreram

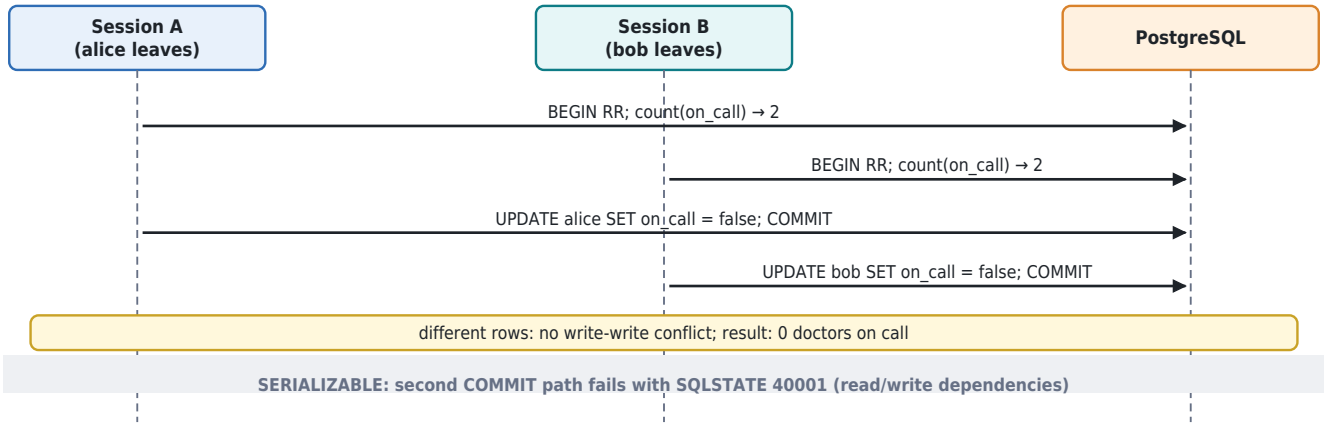


Figure 1.93. At REPEATABLE READ both transactions see two doctors on call, each removes a different doctor, and no row-level conflict is detected.

two sessions, real output

```
-- REPEATABLE READ: invariant broken
A: BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT count(*) FROM doctor WHERE on_call;   -- 2
B: BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT count(*) FROM doctor WHERE on_call;   -- 2
A: UPDATE doctor SET on_call = false WHERE name = 'alice'; COMMIT;
B: UPDATE doctor SET on_call = false WHERE name = 'bob';   COMMIT;
A: SELECT * FROM doctor ORDER BY name;
 name  | on_call
-------+---------
 alice | f
 bob   | f

-- SERIALIZABLE: one transaction is aborted
B: UPDATE doctor SET on_call = false WHERE name = 'bob';
ERROR:  could not serialize access due to read/write dependencies among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during write.
HINT:  The transaction might succeed if retried.
A: SELECT * FROM doctor ORDER BY name;
 name  | on_call
-------+---------
 alice | f
 bob   | t
```

At REPEATABLE READ no doctor remained on call. At SERIALIZABLE the second transaction failed with SQLSTATE 40001 and the invariant held.

Pessimistic fix at READ COMMITTED

two sessions, real output

```
A: BEGIN; SELECT count(*) FROM (SELECT 1 FROM doctor WHERE on_call FOR UPDATE) x;     -- 2
B: BEGIN; SELECT count(*) FROM (SELECT 1 FROM doctor WHERE on_call FOR UPDATE) x;     -- blocks
A: UPDATE doctor SET on_call = false WHERE name = 'alice'; COMMIT;
B:  count
   -------
       1
B: ROLLBACK;     -- the application sees 1 and refuses to remove the last doctor
```

`FOR UPDATE` on all rows that the decision depends on serialises the two transactions. B re-evaluated the predicate after A committed and obtained 1, so its application logic can refuse the change.

1.21.4 Real-world problems and solutions

Incident: lost update on a balance

Scenario. A Spring Data JPA service loads an `Account` entity, calls `account.setBalance(account.getBalance().subtract(amount))` and lets the persistence context flush at commit. Two requests for the same account overlap.

Symptoms. The ledger shows two withdrawals but the balance has been reduced by only one. No exception is logged.

Diagnosis.

sreeram

1. Identify the pattern in the SQL log (`spring.jpa.show-sql=true` or `log_statement = 'mod'`): two `SELECT` statements followed by two `UPDATE account SET balance=? WHERE id=?` with literal values rather than an expression.
2. Compare the sum of ledger entries with the stored balance for the affected account.

diagnose_lost_update.sql SQL

```sql
SELECT a.id, a.balance, COALESCE(SUM(l.amount), 0) AS ledger_sum
FROM account a LEFT JOIN ledger l ON l.account_id = a.id
GROUP BY a.id, a.balance
HAVING a.balance <> 1000 + COALESCE(SUM(l.amount), 0);
```

Root cause. The read-modify-write cycle computed the new balance from a stale value at READ COMMITTED, as in the worked example.

Fix. Immediate: repair affected balances from the ledger. Permanent, in order of preference: (1) an atomic statement `UPDATE account SET balance = balance - :amount WHERE id = :id AND balance >= :amount`, checking the update count; (2) optimistic locking with `@Version`, where the update carries `WHERE version = ?` and a zero update count raises `OptimisticLockingFailureException`; (3) `SELECT ... FOR UPDATE` (`@Lock(PESSIMISTIC_WRITE)`) before the read.

AccountRepository.java (fragment, not run) JAVA

```java
public interface AccountRepository extends JpaRepository<Account, Long> {
    @Modifying
    @Query("update Account a set a.balance = a.balance - :amt "
         + "where a.id = :id and a.balance >= :amt")
    int debit(@Param("id") long id, @Param("amt") BigDecimal amt);   // 0 rows = insufficient funds
}
```

Prevention. Treat any read-modify-write on a shared row as a defect during review; add a concurrency test that runs the debit from several threads and checks the final balance.

Incident: overselling and double booking

Scenario. A shop sells the last unit of an item. Two checkout requests check `stock > 0`, both see 1, and both proceed. The same pattern applies to seat reservations where the check and the insert are separate statements.

Symptoms. Two orders exist for one unit; stock is zero or, without a constraint, negative.

Diagnosis. Compare ordered quantities with stock, and look for two committed transactions whose reads preceded each other's writes.

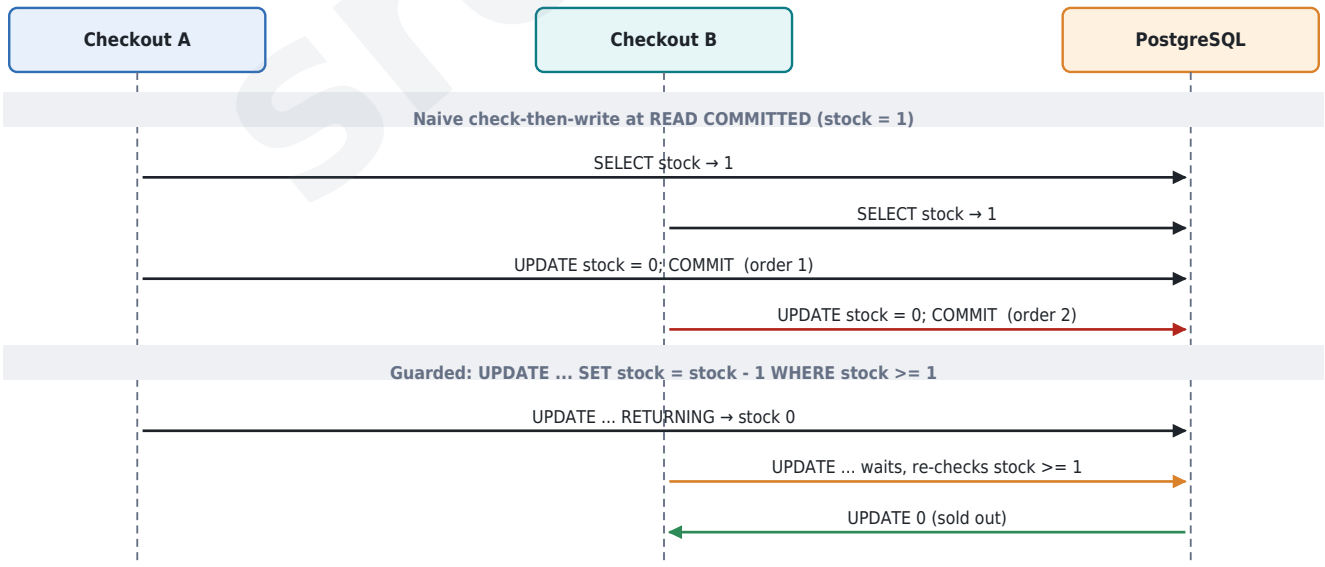


Figure 1.94. Both checkouts read stock 1 before either writes; the guarded UPDATE moves the check into the write, so the second request updates zero rows.

two sessions, real output (stock = 1)

```
-- naive check-then-write at READ COMMITTED
A: BEGIN; SELECT stock FROM item WHERE id = 1;         -- 1
B: BEGIN; SELECT stock FROM item WHERE id = 1;         -- 1
A: UPDATE item SET stock = 0 WHERE id = 1; COMMIT;
B: UPDATE item SET stock = 0 WHERE id = 1; COMMIT;     -- both orders accepted

-- guarded update
A: BEGIN; UPDATE item SET stock = stock - 1 WHERE id = 1 AND stock >= 1 RETURNING stock;
 stock
-------
     0
B: BEGIN; UPDATE item SET stock = stock - 1 WHERE id = 1 AND stock >= 1 RETURNING stock;  -- blocks
A: COMMIT;
B:  stock
   -------
   (0 rows)
UPDATE 0
```

Root cause. The condition was evaluated in a read that was separate from the write, so the check-then-act sequence was not atomic. For seats, a missing `UNIQUE (event_id, seat_no)` constraint removes the last defence.

Fix. Use the guarded `UPDATE` and treat `UPDATE 0` as "sold out". Back it with `CHECK (stock >= 0)`. For seats and other inserts, declare `UNIQUE (event_id, seat_no)` and handle the unique violation (SQLSTATE 23505).

Prevention. Encode every invariant as a constraint, and write the application code so that the write statement carries the condition.

Incident: write skew on on-call doctors

Scenario. Two doctors request leave at the same time. The service counts doctors on call and permits the request if more than one remains.

Symptoms. A morning audit finds zero doctors on call. Neither transaction raised an error.

Diagnosis. The two update transactions overlap in time and touch different rows, which is the signature of write skew. Confirm with `SHOW transaction_isolation;` and the application log of both requests.

Root cause. At REPEATABLE READ (and READ COMMITTED) the decision is based on a snapshot that the other transaction invalidates by writing a different row.

Fix. Immediate: restore the on-call roster manually. Permanent, one of: run the transaction at `SERIALIZABLE` and retry on SQLSTATE 40001; lock the rows read with `SELECT ... FOR UPDATE`; or materialise the conflict by locking a single parent row (for example the ward) that every leave request must update.

LeaveService.java (fragment, not run) JAVA

```java
@Retryable(retryFor = {CannotAcquireLockException.class, SerializationFailureException.class},
           maxAttempts = 3, backoff = @Backoff(delay = 50))
@Transactional(isolation = Isolation.SERIALIZABLE)
public void requestLeave(String doctor) {
    if (doctors.countOnCall() < 2) throw new IllegalStateException("last doctor on call");
    doctors.setOnCall(doctor, false);
}
```

Prevention. List multi-row invariants during design and decide for each one between SERIALIZABLE with retry, explicit row locks, or a materialised conflict row. Serialization failures are expected under SERIALIZABLE and must not reach the user as errors.

1.21.5 Questions to expect

INTERVIEW ANGLE

What is the default isolation level in PostgreSQL and in MySQL InnoDB, and what follows from the difference? PostgreSQL uses READ COMMITTED and InnoDB uses REPEATABLE READ. Code that is correct under one default can break under the other: a read-modify-write that raises no error in READ COMMITTED may raise a serialization failure in PostgreSQL REPEATABLE READ, while InnoDB REPEATABLE READ silently permits it for plain selects.

1. Define a phantom read. A repeated predicate query returns a different set of rows because another transaction committed a matching insert or delete. PostgreSQL REPEATABLE READ prevents it through the snapshot; InnoDB prevents it for locking reads through next-key locks.

2. How is a lost update prevented? By an atomic `UPDATE ... SET x = x - ?`, by `SELECT ... FOR UPDATE`, by a version column checked in the `WHERE` clause, or by REPEATABLE READ/SERIALIZABLE in PostgreSQL with retry.
3. What is write skew and which level prevents it in PostgreSQL? Two transactions read overlapping data and update disjoint rows, violating an invariant. Only SERIALIZABLE (SSI) prevents it automatically; RR and RC do not.
4. What does SQLSTATE 40001 mean and what must the application do? A serialization failure or deadlock. Retry the entire transaction, not only the failed statement.
5. Does `READ UNCOMMITTED` in PostgreSQL permit dirty reads? No. It behaves as READ COMMITTED.
6. How does PostgreSQL SERIALIZABLE differ from MySQL SERIALIZABLE? PostgreSQL uses non-blocking SSI with aborts; InnoDB turns plain reads into shared locking reads and blocks, with more deadlocks.
7. Why does a higher level not always mean a safer application? The application must handle aborts and retries; without retries a higher level converts silent anomalies into visible errors, which is an improvement but still a defect.
8. When is READ COMMITTED sufficient? When each business operation is expressed through atomic statements or explicit row locks, and no multi-statement decision depends on a consistent view.

KEY POINTS

Key points

- An anomaly is defined by what a transaction can observe; each isolation level forbids a subset of anomalies.
- PostgreSQL: RC takes a snapshot per statement, RR per transaction, SERIALIZABLE adds SSI conflict detection; no dirty reads at any level.
- InnoDB: default REPEATABLE READ with a read view for plain reads and next-key locks for locking reads.
- Lost updates are fixed by atomic updates, row locks or version columns; write skew needs SERIALIZABLE with retry or explicit locking.
- Serialization failures (SQLSTATE 40001) require a retry of the whole transaction.

1.22 MVCC Internals, VACUUM and Bloat

Lock-based concurrency control makes readers wait for writers and writers wait for readers. **Multiversion concurrency control (MVCC)** removes most of this waiting by keeping several versions of a row at the same time: a writer creates a new version and leaves the old version in place, and every reader selects the version that was current at the moment of its snapshot. The price is that obsolete versions accumulate and must be removed by a background process, which in PostgreSQL is called **VACUUM**. Most operational problems of PostgreSQL, from table bloat to transaction identifier wraparound, are consequences of this design, so a working model of it is required for diagnosis.

This chapter examines the PostgreSQL implementation with real output from `pageinspect`, then summarises the InnoDB approach for comparison.

1.22.1 Concept and mechanism

Row versions and system columns

Every table in PostgreSQL is a heap of 8 kB pages. Each row version, a **tuple**, starts with a 23-byte header whose first fields are the identifier of the transaction that created it (`xmin`), the identifier of the transaction that deleted or locked it (`xmax`, zero while the tuple is live) and the tuple's own location or the location of its successor (`ctid`, a pair of block number and line pointer offset). These fields are visible to SQL as system columns.

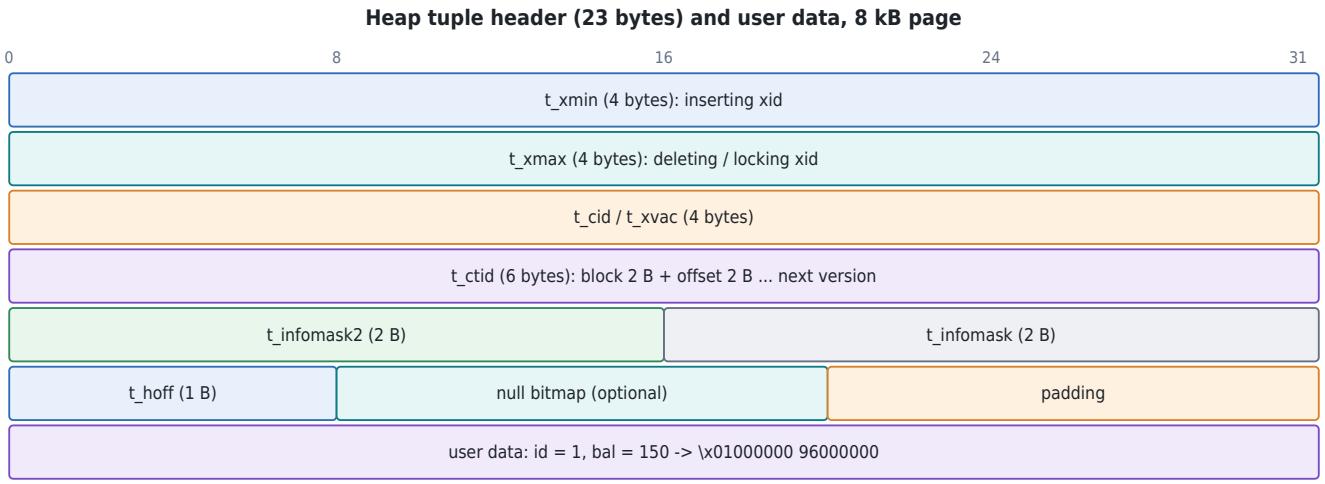


Figure 1.95. The heap tuple header begins with xmin, xmax and the command id, followed by t_ctid and the two flag words; the user data follows after t_hoff.

An `UPDATE` never modifies a tuple in place. It marks the old tuple with its own transaction identifier in `xmax` and writes a new tuple with the same identifier in `xmin`; the old tuple's `t_ctid` points to the new one. A `DELETE` only sets `xmax`. Rolled-back changes leave tuples whose creating transaction aborted; they are invisible to every snapshot.

Snapshots

A **snapshot** is a compact description of which transactions count as committed at a point in time. It consists of `xmin` (the lowest transaction identifier still in progress), `xmax` (the first identifier not yet assigned) and the list `xip` of identifiers in progress between them. `pg_current_snapshot()` prints it as `xmin:xmax:xip_list`. READ COMMITTED takes a new snapshot per statement; REPEATABLE READ and SERIALIZABLE take one at the first statement.

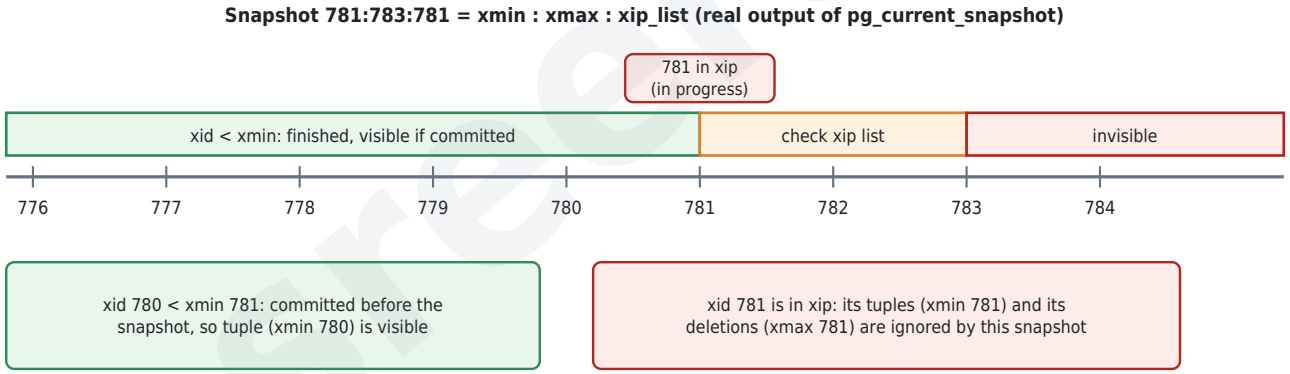


Figure 1.96. A snapshot divides the identifier line into finished transactions below xmin, an uncertain band resolved through the in-progress list, and transactions at or above xmax that are invisible.

Visibility rules

A tuple is visible to a snapshot when its inserter is visible and no visible transaction has deleted it. The decision sequence is shown in the figure. In practice PostgreSQL consults the commit log (`pg_xact`) to learn whether a transaction committed, and caches the answer in **hint bits** in the tuple's `t_infomask` so that later readers skip the lookup. A tuple whose xmax is only a row lock (as set by `SELECT ... FOR UPDATE`) remains visible.

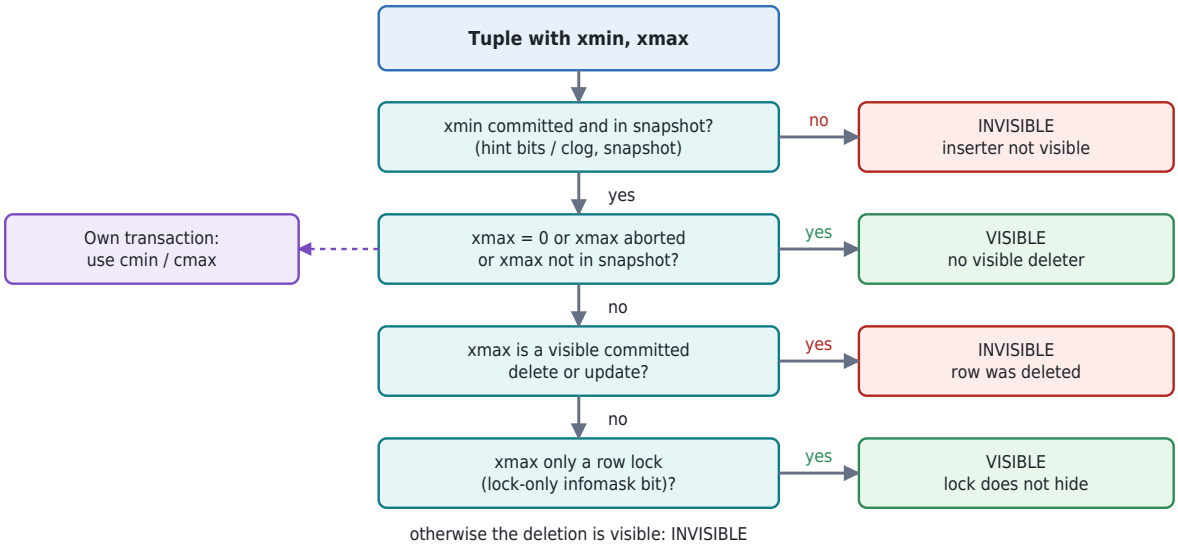


Figure 1.97. Decision path for one tuple: the inserter must be visible, and a deleter that is committed and visible hides the tuple; a lock-only xmax does not.

HOT updates and line pointers

Each page begins with an array of line pointers (`lp`) that point to tuples inside the page. Indexes store the page number and line pointer offset. When an update does not change any indexed column and the new version fits in the same page, PostgreSQL creates a **heap-only tuple (HOT)**: the old tuple points to the new one through `t_ctid`, and no index entry is added. Pruning later turns the first line pointer into a redirect, so index entries remain valid. This is why updates of non-indexed columns are much cheaper than updates of indexed ones.

VACUUM and autovacuum

VACUUM scans a table, finds tuples that are dead to every current and future snapshot, removes them (and their index entries), and records the freed space in the free space map so later inserts can reuse it. Plain VACUUM does not return space to the operating system, except for empty pages at the end of the file. `VACUUM FULL` rewrites the table into a new file, which reclaims space but holds an `ACCESS EXCLUSIVE` lock for the duration.

Autovacuum launches VACUUM and ANALYZE automatically. A table is vacuumed when the number of dead tuples exceeds `autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples`. The defaults observed on the build cluster are 50 and 0.2, so a table of 10 million rows is vacuumed only after about 2 million dead tuples. Large hot tables therefore commonly receive per-table settings.

The removable horizon

VACUUM may remove a dead tuple only when no snapshot can still need it. The oldest transaction identifier that any session might still need is the **horizon**. A session that holds an open transaction holds back the horizon: its transaction identifier, or its snapshot if it has read data, defines the oldest tuple version that must be kept. A transaction that is idle but open therefore blocks cleanup for the entire database cluster, not just for the tables it touched.

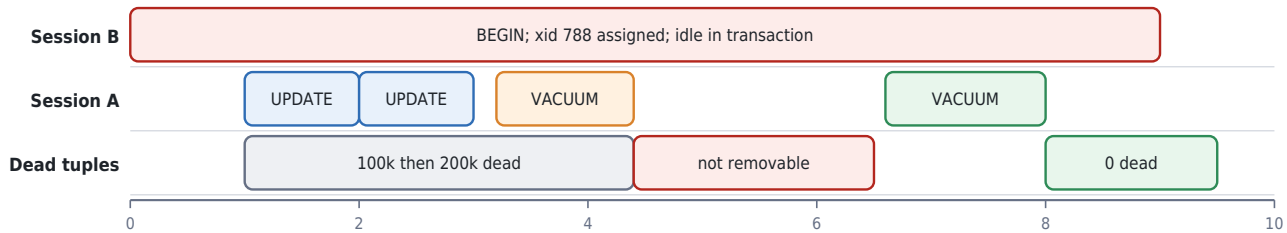


Figure 1.98. While session B holds an open transaction, two successive updates by session A create dead tuples that VACUUM reports as not yet removable; after B commits, the next VACUUM removes them.

Transaction identifier wraparound

Transaction identifiers are 32-bit counters that wrap around. PostgreSQL compares two identifiers modulo 2^32: an identifier is in the past when it is less than 2^31 behind the current one. A tuple that stays unvacuumed for 2^31 transactions would appear to come from the future and vanish from every query. To prevent this, VACUUM **freezes** old tuples, replacing the inserter's identifier with a special state that is visible to all snapshots.
`autovacuum_freeze_max_age` (default 200 000 000) forces an anti-wraparound autovacuum on tables whose `relfrozenxid` age exceeds it, even if autovacuum is otherwise disabled. If the age reaches within 40 million of the limit, the server warns; within three million of the limit, it stops accepting commands that assign new transaction identifiers.

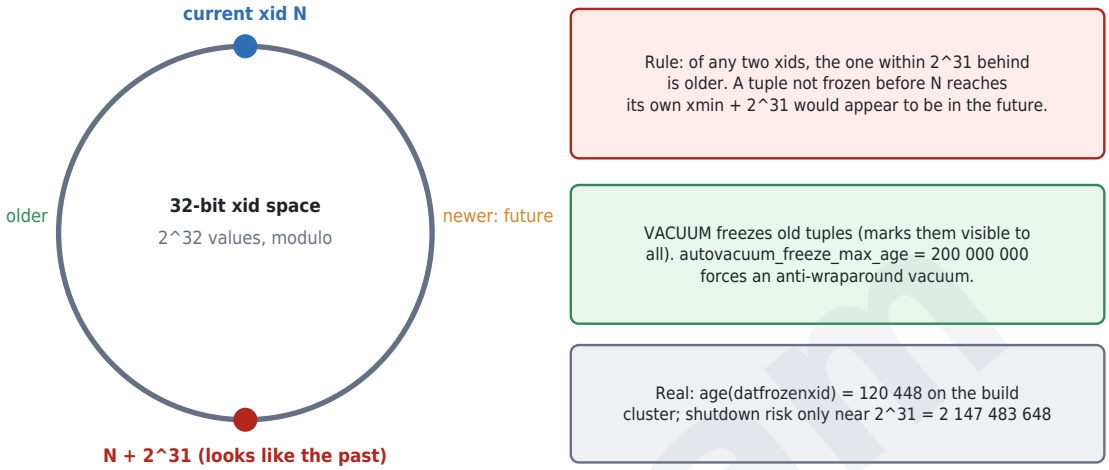


Figure 1.99. The identifier space is a ring; xids within 2^31 behind the current value are in the past, so unfrozen tuples approaching 2^31 would flip into the future.

InnoDB for comparison

InnoDB stores the newest version of a row in the clustered index and keeps previous versions in **undo logs**. Each row carries a transaction identifier (`DB_TRX_ID`) and a roll pointer (`DB_ROLL_PTR`) to the undo record that can reconstruct the preceding version. A consistent read builds a **read view** (list of active transaction identifiers) and walks the undo chain until it reaches a version visible to that view. A background **purge** thread removes undo records that no read view requires. A long-running transaction therefore causes growth of the undo tablespace and of the history list length, the InnoDB counterpart of PostgreSQL bloat.

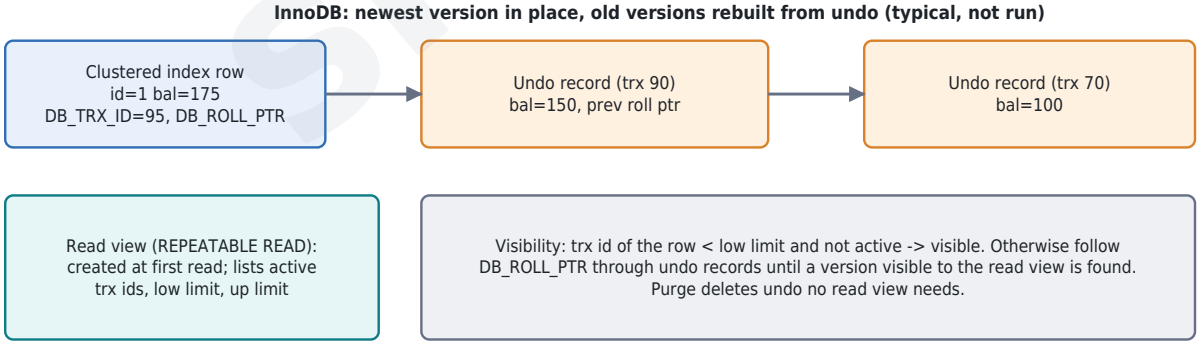


Figure 1.100. InnoDB keeps the newest row in place and rebuilds older versions through the roll pointer into the undo log (typical behaviour, not run on the build machine).

Aspect	PostgreSQL	InnoDB
Old versions kept in	the table heap itself	undo logs
Update cost	new tuple and possibly new index entries	in-place change plus undo record
Rollback cost	immediate (mark aborted)	undo applied, proportional to the work done
Cleanup	VACUUM / autovacuum	purge threads

Aspect	PostgreSQL	InnoDB
Long transaction effect	table and index bloat; horizon blocked	undo growth; slower reads through long chains
Wraparound concern	32-bit xid, freezing required	48-bit row transaction ids, no wraparound vacuum

MVCC storage compared (PostgreSQL 16 and InnoDB, MySQL 8.x).

1.22.2 Worked example

The following steps use the real data of the next section. A table `acct(id, bal)` holds (1, 100) and (2, 200), inserted by transaction 778. Autovacuum is disabled for the table so that the versions stay visible.

1. Transaction 780 runs `UPDATE acct SET bal = 150 WHERE id = 1`. Tuple lp 1 gets `xmax = 780`; a new tuple lp 3 is written with `xmin = 780`, and lp 1 points to it through `t_ctid = (0,3)`.
2. Session A opens a transaction, `txid_current()` returns 781, and it runs `UPDATE acct SET bal = 175 WHERE id = 1` without committing. Tuple lp 3 gets `xmax = 781`, and lp 4 is created with `xmin = 781`.
3. Session B takes a snapshot, `pg_current_snapshot()` returning `781:783:781`: the lowest active identifier is 781, the next unassigned is 783, and 781 is in progress.
4. Visibility check by session B for each tuple of row 1:
   - lp 1: `xmin 778` is committed and below 781. `xmax 780` is committed and below 781, so the deletion is visible. The tuple is invisible.
   - lp 3: `xmin 780` is visible. `xmax 781` is in the in-progress list, so that deletion is not visible to this snapshot. The tuple is visible, with `bal = 150`.
   - lp 4: `xmin 781` is in progress, so the insertion is not visible. The tuple is invisible.
1. Session B therefore reads `bal = 150` with `xmax = 781`, exactly as the real output below shows. Once transaction 781 commits, a new snapshot sees lp 4 and not lp 3.

1.22.3 Hands-on

The cluster was PostgreSQL 16.15 with the `pageinspect` extension. Transaction identifiers are typical values of this run.

System columns after an update

psql session (real output)

```
t=# SELECT xmin, xmax, ctid, * FROM acct ORDER BY id;
 xmin | xmax | ctid  | id | bal
------+------+-------+----+-----
  778 |    0 | (0,1) |  1 | 100
  778 |    0 | (0,2) |  2 | 200

t=# UPDATE acct SET bal = 150 WHERE id = 1;
t=# SELECT xmin, xmax, ctid, * FROM acct ORDER BY id;
 xmin | xmax | ctid  | id | bal
------+------+-------+----+-----
  780 |    0 | (0,3) |  1 | 150
  778 |    0 | (0,2) |  2 | 200
```

The updated row moved from `ctid (0,1)` to `(0,3)`, and its `xmin` changed from 778 to 780. The row with id 2 is untouched.

Reading the page with pageinspect

heap_page_items.sql — SQL

```sql
SELECT lp, t_xmin, t_xmax, t_ctid,
       (t_infomask & 256) > 0   AS xmin_commit,
       (t_infomask & 2048) > 0  AS xmax_invalid,
       (t_infomask2 & 16384) > 0 AS hot_updated,
       t_data
FROM heap_page_items(get_raw_page('acct', 0));
```

result (real output)

```
 lp | t_xmin | t_xmax | t_ctid | xmin_commit | xmax_invalid | hot_updated |       t_data
----+--------+--------+--------+-------------+--------------+-------------+--------------------
  1 |    778 |    780 | (0,3)  | t           | f            | t           | \x0100000064000000
  2 |    778 |      0 | (0,2)  | t           | t            | f           | \x02000000c8000000
  3 |    780 |      0 | (0,3)  | t           | t            | f           | \x0100000096000000
```

Tuple 1 is the old version: `t_xmax = 780`, `t_ctid` pointing to tuple 3, and the HOT flag. `t_data` shows the integers in little-endian form: `0x64` is 100, `0x96` is 150.

An uncommitted update seen from another session

two sessions, real output

```
A: BEGIN; SELECT txid_current();      -- 781
A: UPDATE acct SET bal = 175 WHERE id = 1;
B: SELECT xmin, xmax, ctid, * FROM acct WHERE id = 1;
 xmin | xmax | ctid  | id | bal
------+------+-------+----+-----
  780 |  781 | (0,3) |  1 | 150

B: SELECT lp, t_xmin, t_xmax, t_ctid FROM heap_page_items(get_raw_page('acct',0)) ORDER BY lp;
 lp | t_xmin | t_xmax | t_ctid
----+--------+--------+--------
  1 |    778 |    780 | (0,3)
  2 |    778 |      0 | (0,2)
  3 |    780 |    781 | (0,4)
  4 |    781 |      0 | (0,4)

B: SELECT pg_current_snapshot();
 781:783:781
```

The page contains both versions, and session B reads the old one. The `xmax` of 781 in the result of B's query is the pending update, not yet effective.

DELETE, VACUUM and line pointers

psql session (real output)

```
t=# DELETE FROM acct WHERE id = 2;
t=# SELECT lp, lp_flags, t_xmin, t_xmax, t_ctid FROM heap_page_items(get_raw_page('acct',0));
 lp | lp_flags | t_xmin | t_xmax | t_ctid
----+----------+--------+--------+--------
  1 |        1 |    778 |    780 | (0,3)
  2 |        1 |    778 |    783 | (0,2)
  3 |        1 |    780 |    781 | (0,4)
  4 |        1 |    781 |      0 | (0,4)

t=# VACUUM acct;
t=# SELECT lp, lp_flags, lp_off, t_xmin, t_xmax FROM heap_page_items(get_raw_page('acct',0));
 lp | lp_flags | lp_off | t_xmin | t_xmax
----+----------+--------+--------+--------
  1 |        2 |      4 |        |
  2 |        0 |      0 |        |
  3 |        0 |      0 |        |
  4 |        1 |   8160 | 781    | 0
```

After VACUUM only lp 4 holds data. `lp_flags = 2` marks lp 1 as a **redirect** to lp 4 (the `lp_off` column then holds the target offset), and `lp_flags = 0` marks unused line pointers that new tuples may reuse.

Heap page of acct (block 0) after two updates of id=1 (real values, xids typical)

lp	t_xmin	t_xmax	t_ctid	row (id, bal)	state
1	778	780	(0,3)	(1, 100)	dead: replaced by lp 3 (HOT)
2	778	783	(0,2)	(2, 200)	dead: deleted by 783
3	780	781	(0,4)	(1, 150)	dead: replaced by lp 4
4	781	0	(0,4)	(1, 175)	live: newest version

ctid chain: lp1 -> lp3 -> lp4 (each t_ctid names the successor; newest points to itself)

After VACUUM: lp 1 becomes a redirect to lp 4 (lp_flags 2), lp 2 and lp 3 become unused (flags 0), lp 4 stays

Index entry for id=1 still points to lp 1. The line pointer redirect keeps the index valid without index changes (heap-only tuple)

Figure 1.101. The page after two updates and a delete holds four tuples linked by t_ctid; only the newest is live, and VACUUM turns lp 1 into a redirect and frees lp 2 and lp 3.

Row locks live in xmax too

psql session (real output)

```
t=# BEGIN; SELECT * FROM acct WHERE id = 1 FOR UPDATE;
t=*# SELECT lp, t_xmin, t_xmax,
       (t_infomask & 128) > 0 AS lock_only, (t_infomask & 64) > 0 AS excl_lock
     FROM heap_page_items(get_raw_page('acct',0)) WHERE lp = 4;
 lp | t_xmin | t_xmax | lock_only | excl_lock
----+--------+--------+-----------+-----------
  4 |    781 |    784 | t         | t
```

`SELECT ... FOR UPDATE` writes the locking transaction identifier into `xmax` and sets the lock-only and exclusive-lock bits; no new tuple version is created. This is the reason a long `FOR UPDATE` workload still causes page writes.

Bloat reproduced: an open transaction versus VACUUM

The table `orders_b` had 100 000 rows and autovacuum disabled. Session B opened a transaction and requested its identifier (`txid_current()`) without further activity; session A then updated every row twice.

psql session (real output)

```
t=# SELECT pg_size_pretty(pg_relation_size('orders_b')) AS heap,
          pg_size_pretty(pg_relation_size('orders_b_pkey')) AS pk;
  heap   |   pk
---------+---------
 4328 kB | 2208 kB

t=# SELECT pid, state, backend_xid, now() - xact_start AS xact_age FROM pg_stat_activity
    WHERE datname = 't' AND pid <> pg_backend_pid();
 pid  |        state        | backend_xid |    xact_age
------+---------------------+-------------+-----------------
 3342 | idle in transaction |         788 | 00:00:00.601725

t=# UPDATE orders_b SET status = 'PAID';  UPDATE orders_b SET status = 'SHIPPED';
UPDATE 100000
UPDATE 100000
t=# SELECT pg_size_pretty(pg_relation_size('orders_b')) AS heap, n_live_tup, n_dead_tup
    FROM pg_stat_user_tables WHERE relname = 'orders_b';
 heap  | n_live_tup | n_dead_tup
-------+------------+------------
 13 MB |     200000 |     100000
```

The heap tripled although the logical data did not grow. The counter `n_dead_tup` is an estimate maintained by the statistics system.

VACUUM while session B is still open (real output, excerpt)

```
t=# VACUUM VERBOSE orders_b;
INFO:  finished vacuuming "t.public.orders_b": index scans: 0
pages: 0 removed, 1622 remain, 1622 scanned (100.00% of total)
tuples: 0 removed, 300000 remain, 200000 are dead but not yet removable
removable cutoff: 788, which was 14 XIDs old when operation ended
```

`200000 are dead but not yet removable` and `removable cutoff: 788` identify the cause: the horizon is 788, the identifier of session B, so no tuple deleted after that identifier can be removed.

VACUUM after session B committed (real output, excerpt)

```
t=# VACUUM VERBOSE orders_b;
INFO:  finished vacuuming "t.public.orders_b": index scans: 1
pages: 0 removed, 1622 remain, 1622 scanned (100.00% of total)
tuples: 200000 removed, 100000 remain, 0 are dead but not yet removable
removable cutoff: 802, which was 0 XIDs old when operation ended
index scan needed: 1082 pages from table (66.71% of total) had 200000 dead item identifiers removed
index "orders_b_pkey": pages: 825 in total, 0 newly deleted, 0 currently deleted, 0 reusable
WAL usage: 4608 records, 1 full page images, 1566512 bytes

t=# SELECT pg_size_pretty(pg_relation_size('orders_b')) AS heap, n_live_tup, n_dead_tup
    FROM pg_stat_user_tables WHERE relname = 'orders_b';
 heap  | n_live_tup | n_dead_tup
-------+------------+------------
 13 MB |     100000 |          0
```

All 200 000 dead tuples were removed, but the file kept its size of 13 MB. The freed space is reusable by later updates; it was not returned to the operating system. A further full-table update followed by plain VACUUM leaves the size at 13 MB (real result), whereas `VACUUM FULL` rewrites the table:

psql session (real output)

```
t=# VACUUM FULL orders_b;
t=# SELECT pg_size_pretty(pg_relation_size('orders_b'));
 pg_size_pretty
----------------
 4328 kB
```

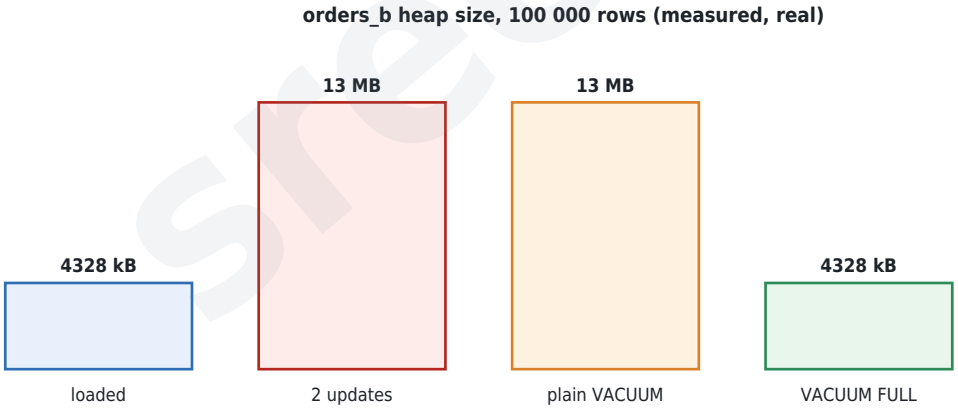


Figure 1.102. Heap size of the same 100 000 rows: 4328 kB after load, 13 MB after two full-table updates under a blocked horizon, unchanged after plain VACUUM, and 4328 kB after VACUUM FULL.

Wraparound monitoring queries

psql session (real output on a young cluster)

```
t=# SELECT datname, age(datfrozenxid) AS xid_age FROM pg_database ORDER BY 1;
  datname  | xid_age
-----------+---------
 postgres  |  120448
 t         |  120448
 template0 |  120448
 template1 |  120448

t=# SELECT name, setting FROM pg_settings WHERE name IN
    ('autovacuum_freeze_max_age', 'vacuum_failsafe_age',
     'vacuum_freeze_table_age', 'vacuum_freeze_min_age');
           name            |  setting
---------------------------+------------
 autovacuum_freeze_max_age | 200000000
 vacuum_failsafe_age       | 1600000000
 vacuum_freeze_min_age     | 50000000
 vacuum_freeze_table_age   | 150000000
```

1.22.4 Real-world problems and solutions

Incident: idle in transaction blocks VACUUM and bloats tables

Scenario. A Spring service opens a transaction through `@Transactional`, calls a slow remote payment API inside the method, and in some failure paths never finishes because of a missing client timeout. Alternatively a developer leaves `BEGIN` open in a SQL console.

Symptoms. Table sizes grow week after week without data growth; queries on hot tables slow down; autovacuum runs repeatedly yet `n_dead_tup` remains high; disk usage alarms fire.

Diagnosis. Follow the decision path below.

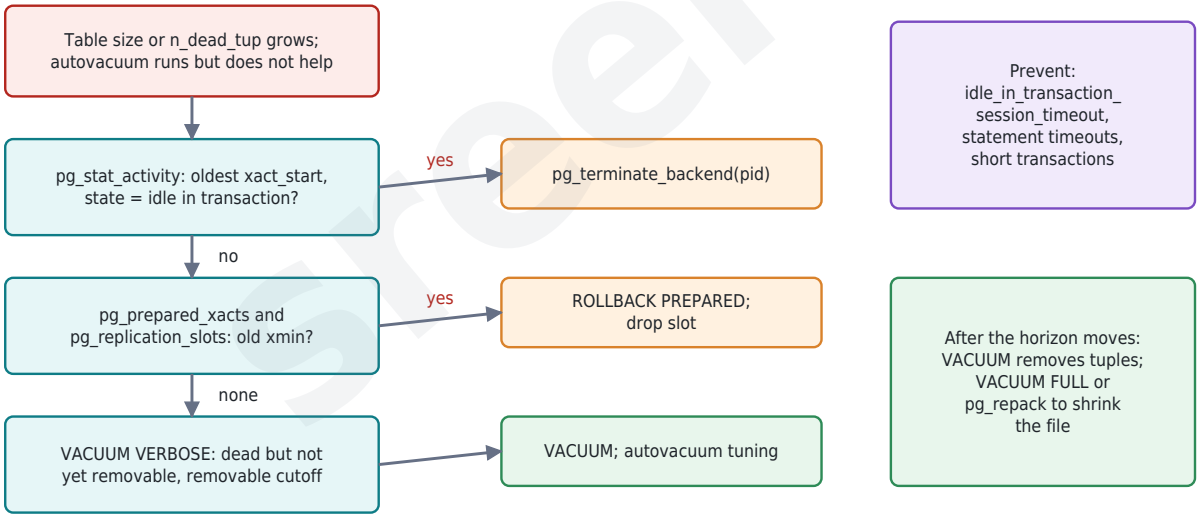


Figure 1.103. Diagnosis path: find sessions that hold the horizon, confirm with VACUUM VERBOSE, then decide between terminating the session and tuning timeouts.

1. Find the oldest transactions and their states.

oldest_transactions.sql SQL

```sql
SELECT pid, usename, state, now() - xact_start AS xact_age,
       backend_xid, backend_xmin, left(query, 40) AS last_query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start
LIMIT 5;
```

1. Include prepared transactions and replication slots, which also hold the horizon.

other_horizon_holders.sql SQL

```sql
SELECT gid, prepared, transaction FROM pg_prepared_xacts;
SELECT slot_name, active, xmin, catalog_xmin FROM pg_replication_slots;
```

1. Confirm with `VACUUM VERBOSE table` and look for `dead but not yet removable` and the `removable cutoff`.
2. Quantify with `pg_stat_user_tables`: `n_dead_tup`, `last_autovacuum`, `autovacuum_count`.

Root cause. An open transaction fixed the horizon, so dead tuples created after it could not be removed. Every table in the cluster is affected, not only the tables the session used.

Fix. Immediate: end the offending session with `SELECT pg_terminate_backend(pid);` (kill only the process that was identified), then run `VACUUM` on the most affected tables. If space must be returned to the operating system, use `VACUUM FULL` in a maintenance window, or `pg_repack` online, understanding that both rewrite the table. Permanent: set `idle_in_transaction_session_timeout` and `transaction_timeout` (the latter exists from PostgreSQL 17) and client-side timeouts, and move remote calls outside of transactions.

session timeout, real output

```
t=# SET idle_in_transaction_session_timeout = '1s'; BEGIN; SELECT 1;
 ?column?
----------
        1
-- two and a half seconds later:
t=# SELECT 1;
FATAL:  terminating connection due to idle-in-transaction timeout
server closed the connection unexpectedly
```

Prevention. Alert on `max(now() - xact_start)` for non-idle transactions and on `idle in transaction` sessions older than a minute. Set per-table autovacuum parameters on large churn tables, for example `ALTER TABLE orders SET (autovacuum_vacuum_scale_factor = 0.02);`. Keep transactions short, and avoid network calls inside them.

Incident: transaction ID wraparound warning

Scenario. A database of several terabytes has autovacuum throttled by the default cost limits, and a single long-running anti-wraparound vacuum of a large table has been cancelled repeatedly by `ALTER TABLE` statements of a migration job.

Symptoms. The log contains `WARNING: database "app" must be vacuumed within 39985967 transactions` and `HINT: To avoid a database shutdown, execute a database-wide VACUUM in that database.` (typical output, number varies). Autovacuum workers appear in `pg_stat_activity` with the query suffix `(to prevent wraparound)`. If ignored, PostgreSQL refuses new transaction identifiers with `database is not accepting commands that assign new transaction IDs to avoid wraparound data loss`.

Diagnosis.

1. Measure the age per database and per table; values approaching 2 billion are critical, and values above `autovacuum_freeze_max_age` mean a forced vacuum is due.

xid_age.sql SQL

```sql
SELECT datname, age(datfrozenxid) AS xid_age FROM pg_database ORDER BY 2 DESC;

SELECT c.oid::regclass AS table_name, age(c.relfrozenxid) AS xid_age,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS size
FROM pg_class c
WHERE c.relkind IN ('r', 'm', 't')
ORDER BY age(c.relfrozenxid) DESC
LIMIT 10;
```

1. Check for holders of the horizon (previous incident, step 2), because they also prevent freezing.
2. Check why the anti-wraparound vacuum did not complete: cancelled by lock conflicts, throttled by `autovacuum_vacuum_cost_delay`, or blocked by an old prepared transaction.

Root cause. Freezing did not keep pace with identifier consumption; a high transaction rate, throttled or cancelled vacuum, and an old horizon holder combined.

Fix. Immediate: terminate the horizon holder, drop abandoned prepared transactions (`ROLLBACK PREPARED 'gid'`) and inactive replication slots, then run `VACUUM (FREEZE, VERBOSE)` manually on the oldest tables, starting with the largest age. Run it without throttling for the duration: `SET vacuum_cost_delay = 0;`. When the server has stopped accepting

commands, start it in single-user mode and run `VACUUM`. Permanent: raise `autovacuum_vacuum_cost_limit`, lower `autovacuum_naptime` where useful, increase `autovacuum_max_workers`, and avoid migrations that cancel autovacuum.

Prevention. Monitor `age(datfrozenxid)` against a threshold of about half of `autovacuum_freeze_max_age` growth, and monitor `mxid_age` for multixact identifiers. Failsafe mode (`vacuum_failsafe_age`, default 1 600 000 000) lets VACUUM skip nonessential work, but it must not be relied on.

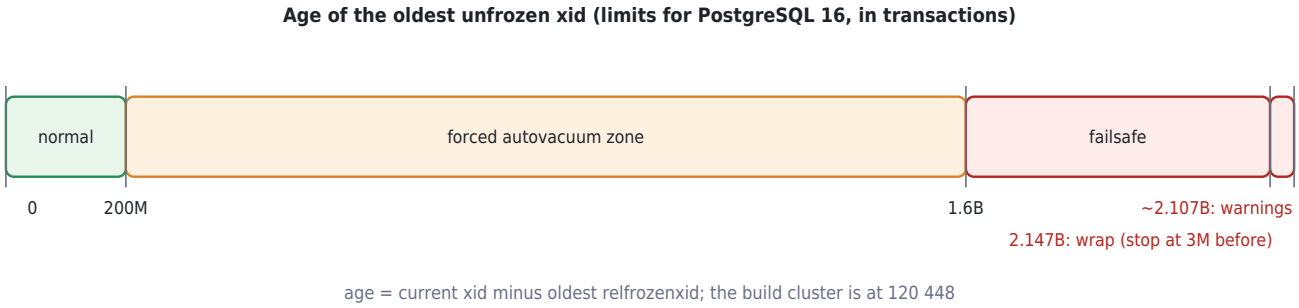


Figure 1.104. The risk grows through four stages: normal, forced anti-wraparound autovacuum at 200 million, warnings 40 million before the limit, and refusal of new transaction ids close to 2 147 483 648 (limits as documented for PostgreSQL 16).

1.22.5 Questions to expect

INTERVIEW ANGLE

How does PostgreSQL implement MVCC and what is the cost of this design? Every update writes a new tuple version, with `xmin` and `xmax` recording the creating and deleting transactions; readers choose the version that fits their snapshot, so readers and writers do not block each other. The cost is dead tuples, which VACUUM must remove, and table and index bloat when cleanup lags.

1. What are `xmin`, `xmax` and `ctid`? The inserting transaction id, the deleting or locking transaction id (0 if live), and the physical location of the tuple or its successor.
2. What does a snapshot contain? `xmin`, `xmax` and the list of in-progress transaction ids; a tuple is visible when its inserter is committed before the snapshot and its deleter is not.
3. Why does a long open transaction cause bloat in unrelated tables? It holds the cleanup horizon back for the whole cluster; VACUUM cannot remove any tuple that became dead after that horizon.
4. What is the difference between `VACUUM` and `VACUUM FULL`? VACUUM marks space reusable without an exclusive lock; VACUUM FULL rewrites the table, returns space to the OS and takes an `ACCESS EXCLUSIVE` lock.
5. What is a HOT update? An update that changes no indexed column and fits on the same page; it adds no index entries and is cleaned by page pruning.
6. What is transaction ID wraparound? The 32-bit transaction counter wraps; unfrozen old tuples would appear to be in the future. Freezing by VACUUM prevents it.
7. How does InnoDB differ? The newest version lives in the clustered index; old versions are rebuilt from undo logs; purge threads clean up; long transactions grow the undo history.
8. When is autovacuum triggered for a table? When dead tuples exceed the threshold plus scale factor times the row estimate (default 50 + 20 percent).

KEY POINTS

Key points

- A PostgreSQL update creates a new tuple; `xmin`, `xmax` and `ctid` link and date the versions, and snapshots decide visibility.
- VACUUM removes dead tuples only behind the horizon; an open transaction anywhere holds the horizon for the cluster.
- Plain VACUUM makes space reusable; only VACUUM FULL or a rewrite shrinks the file.
- Freezing prevents transaction id wraparound; monitor `age(datfrozenxid)` and keep autovacuum healthy.
- InnoDB reconstructs old versions from undo; the equivalent operational risk is a long transaction and a growing history list.

1.23 Locking, Deadlocks and Optimistic Concurrency

MVCC lets readers proceed without blocking, but two transactions that write the same row must still be ordered, and DDL must be kept apart from queries on the same table. **Locks** provide this ordering. A lock is a promise, held by a transaction until it ends, that other transactions may not perform conflicting operations on a resource. Locks cause waiting, waiting causes queues, and queues cause the production incidents examined in this chapter: lock pile-ups behind DDL, long blocking chains, and deadlocks.

PostgreSQL 16.15 supplied all real output. MySQL InnoDB behaviour is summarised from the MySQL 8.x documentation and is labelled as such.

1.23.1 Concept and mechanism

Lock granularity and where locks live

PostgreSQL uses several kinds of lock for different purposes.

- **Table-level locks** protect a relation against conflicting commands. Every statement takes one automatically: `SELECT` takes `ACCESS SHARE`, `INSERT`, `UPDATE` and `DELETE` take `ROW EXCLUSIVE`, most `ALTER TABLE` forms take `ACCESS EXCLUSIVE`. They live in shared memory and are visible in `pg_locks`.
- **Row-level locks** protect individual rows. They are not stored in a lock table: a transaction that updates, deletes or explicitly locks a row writes its transaction identifier into the tuple's `xmax`, as the previous chapter showed. A second transaction that wants the same row waits on the **transaction identifier lock** of the first one, so the wait ends exactly when that transaction commits or rolls back. This design allows locking millions of rows without exhausting memory.
- **Advisory locks** are application-defined locks on arbitrary numbers, taken explicitly and not tied to any table.

InnoDB differs: it stores row locks in a lock table in memory, attached to index records. Besides record locks it uses **gap locks** (the gap before an index record) and **next-key locks** (record plus preceding gap) to prevent phantoms at REPEATABLE READ, and **insert intention locks** to coordinate concurrent inserts into the same gap.

Table-level lock modes

PostgreSQL defines eight table-level modes. Two modes conflict when both cannot be held by different transactions at the same time. A transaction never conflicts with itself.

	ACCESS SHARE	ROW SHARE	ROW EXCLUSIVE	SHARE UPDATE EXCL.	SHARE	SHARE ROW EXCL.	EXCLUSIVE	ACCESS EXCLUSIVE
ACCESS SHARE								X
ROW SHARE							X	X
ROW EXCLUSIVE					X	X	X	X
SHARE UPDATE EXCL.				X	X	X	X	X
SHARE			X	X		X	X	X
SHARE ROW EXCL.			X	X	X	X	X	X
EXCLUSIVE		X	X	X	X	X	X	X
ACCESS EXCLUSIVE	X	X	X	X	X	X	X	X

Statement examples: SELECT = ACCESS SHARE; INSERT/UPDATE/DELETE = ROW EXCLUSIVE; CREATE INDEX = SHARE; ALTER TABLE = ACCESS EXCLUSIVE

Figure 1.105. Conflict matrix of the eight table lock modes (X marks a conflict): ACCESS SHARE conflicts only with ACCESS EXCLUSIVE, and ACCESS EXCLUSIVE conflicts with every mode, including plain SELECT.

The practical consequences are:

- Ordinary queries (`ACCESS SHARE`) and DML (`ROW EXCLUSIVE`) never block each other at table level.
- `CREATE INDEX` without `CONCURRENTLY` takes `SHARE`, which blocks writers but not readers.
- `ALTER TABLE ... ADD COLUMN`, `DROP COLUMN`, most constraint changes, `TRUNCATE`, `VACUUM FULL` and `LOCK TABLE` take `ACCESS EXCLUSIVE`, which blocks everything, including reads.

Row-level lock modes

Four modes exist: `FOR KEY SHARE`, `FOR SHARE`, `FOR NO KEY UPDATE` and `FOR UPDATE`. A plain `UPDATE` takes `FOR NO KEY UPDATE` when it does not change key columns (this is what lets foreign key checks, which take `FOR KEY SHARE` on the parent row, proceed in parallel with updates of non-key columns) and `FOR UPDATE` otherwise; `DELETE` takes `FOR UPDATE`.

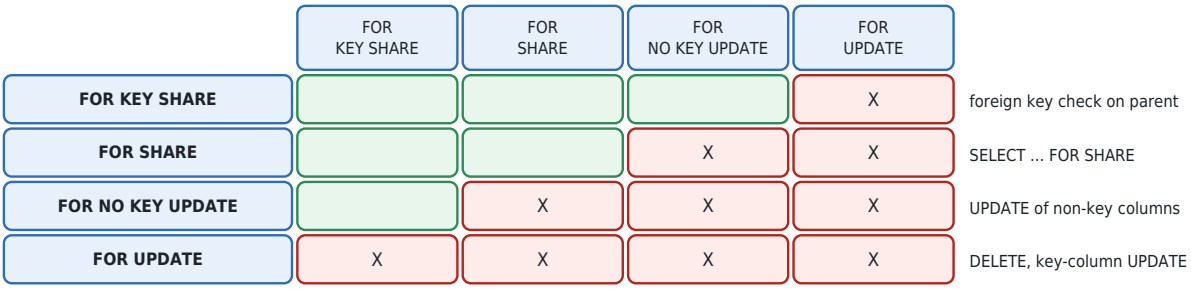


Figure 1.106. Row-level conflicts: FOR UPDATE conflicts with all four modes, while FOR KEY SHARE conflicts only with FOR UPDATE, so foreign key checks do not block ordinary updates.

Explicit row locking clauses

`SELECT ... FOR UPDATE` locks the selected rows for modification. Two modifiers change the waiting behaviour.

- `NOWAIT` fails at once with an error if a selected row is locked.
- `SKIP LOCKED` silently skips rows that are locked. This turns a table into a work queue: several workers each take different rows without blocking each other. The returned set is intentionally inconsistent, which is acceptable for queues but wrong for general queries.

PostgreSQL has no direct equivalent of the MySQL "gap lock", and its `SERIALIZABLE` level detects instead of locking ranges.

Waiting, timeouts and deadlocks

A waiting statement waits indefinitely unless a timeout is set. `lock_timeout` aborts a statement that waits longer than the limit for any lock; `statement_timeout` limits the total statement time; `deadlock_timeout` (default one second) is the delay before the server checks for a deadlock.

A **deadlock** is a cycle of transactions in which each waits for a lock held by the next. PostgreSQL detects the cycle after `deadlock_timeout` has elapsed, aborts one participant with SQLSTATE 40P01, and lets the others continue. InnoDB detects deadlocks immediately by default (`innodb_deadlock_detect = ON`) and rolls back the transaction that has changed the least, and its lock wait timeout (`innodb_lock_wait_timeout`) defaults to 50 seconds, whereas PostgreSQL has no default timeout.

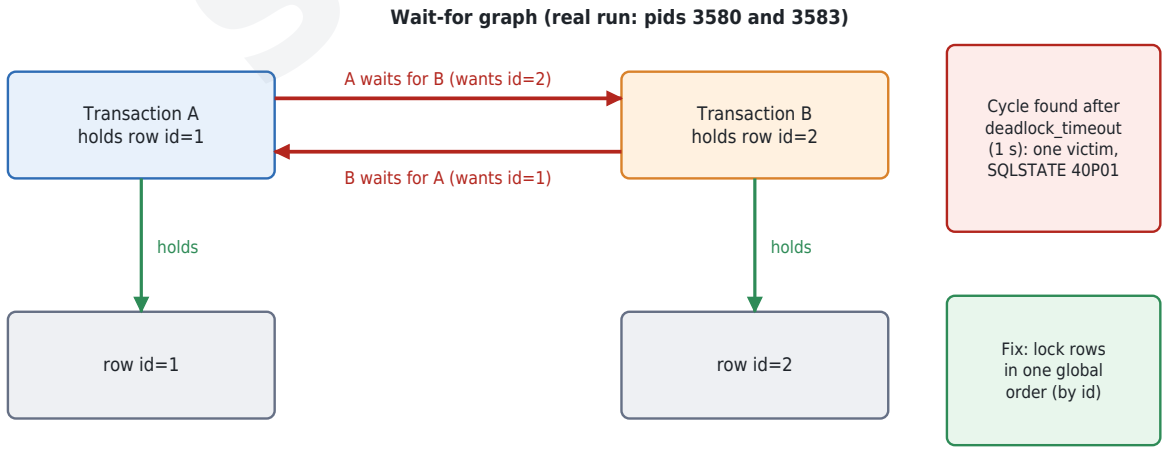


Figure 1.107. Two transactions update the same two rows in opposite order; each waits for the other, the wait-for graph contains a cycle, and the detector aborts one of them.

Aspect	PostgreSQL 16	MySQL 8.x InnoDB
Row lock storage	`xmax` in the tuple	lock table in memory, attached to index records
Range protection	none (SSI at SERIALIZABLE)	gap and next-key locks at REPEATABLE READ
Lock wait default	unlimited (`lock_timeout = 0`)	50 s (`innodb_lock_wait_timeout`)
Deadlock detection	after `deadlock_timeout` (1 s)	immediate; victim is the smaller transaction
Queue syntax	`FOR UPDATE SKIP LOCKED`, `NOWAIT`	same, since MySQL 8.0
Inspecting locks	`pg_locks`, `pg_blocking_pids()`	`performance_schema.data_locks`, `data_lock_waits`, `sys.innodb_lock_waits`
Last deadlock	server log	`SHOW ENGINE INNODB STATUS`, section LATEST DETECTED DEADLOCK

Locking behaviour compared (PostgreSQL 16 and MySQL 8.x InnoDB).

Optimistic versus pessimistic concurrency

Pessimistic locking assumes conflicts are likely: it locks first (`SELECT ... FOR UPDATE`) and then works. It is simple to reason about and correct under contention, at the cost of holding locks across the application think time and of deadlock risk.

Optimistic locking assumes conflicts are rare: it reads without locking, remembers a version number, and issues the write conditioned on that version, `UPDATE ... SET ..., version = version + 1 WHERE id = ? AND version = ?`. A zero update count means another transaction changed the row, and the application re-reads and retries or reports a conflict. JPA implements this with `@Version`. It holds no locks during the think time, which suits web requests, but it wastes work under high contention.

1.23.2 Worked example

The scenario is a row lock wait. Session A (pid 3480) updates seat 1 and keeps the transaction open. Session B (pid 3479) tries to update the same seat.

1. Session A runs `UPDATE seat SET holder = 'ann' WHERE id = 1`. It takes `ROW EXCLUSIVE` on the table and writes its transaction identifier into the tuple's `xmax`. It also holds an exclusive lock on its own transaction identifier.
2. Session B runs the same row update. It finds the row's `xmax` set to a live transaction. It takes a `tuple` lock to queue in order and requests a `ShareLock` on A's transaction identifier, which A holds exclusively, so B waits.
3. The wait ends when A commits or rolls back, because the transaction identifier lock is released. B then re-evaluates the row, which at READ COMMITTED means re-reading the newest version.

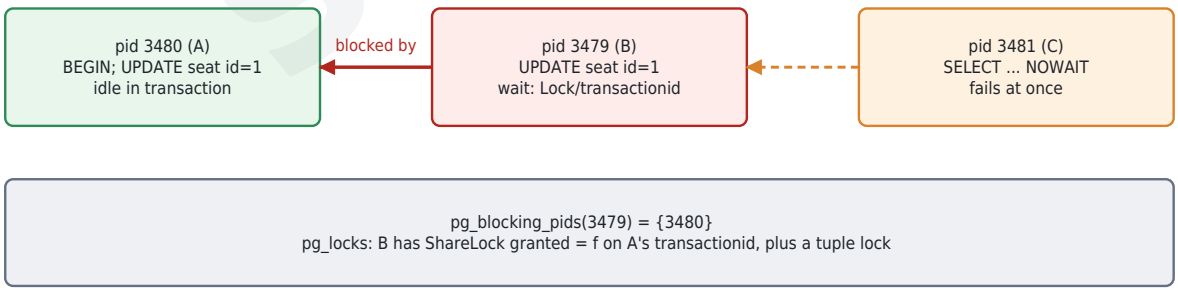


Figure 1.108. The waiting session B points to the holder A through a transaction-id lock; pg_blocking_pids returns the holder for the waiting backend.

1.23.3 Hands-on

Observing a blocked update

Table `seat(id, holder)` holds five rows. Session A updates row 1 inside an open transaction; B issues the same update and blocks.

lock_inspection.sql SQL

```sql
SELECT l.locktype, l.relation::regclass, l.mode, l.granted, l.pid
FROM pg_locks l
WHERE l.pid IN (SELECT pid FROM pg_stat_activity WHERE datname = 't')
  AND l.locktype IN ('relation', 'transactionid', 'tuple')
  AND (l.relation IS NULL OR l.relation::regclass::text = 'seat')
ORDER BY l.pid, l.locktype;
```

result (real output)

```
   locktype    | relation |       mode       | granted | pid
---------------+----------+------------------+---------+------
 relation      | seat     | RowExclusiveLock | t       | 3479
 transactionid |          | ShareLock        | f       | 3479
 transactionid |          | ExclusiveLock    | t       | 3479
 tuple         | seat     | ExclusiveLock    | t       | 3479
 relation      | seat     | RowExclusiveLock | t       | 3480
 transactionid |          | ExclusiveLock    | t       | 3480
```

Pid 3479 (the waiter) holds a `tuple` lock and has an ungranted `ShareLock` on a transaction identifier, which is the one of pid 3480. The holder owns only its table lock and its own transaction identifier lock. The simpler way to read the same situation is `pg_blocking_pids`:

blockers.sql SQL

```sql
SELECT pid, pg_blocking_pids(pid) AS blocked_by, wait_event_type, wait_event, state,
       left(query, 40) AS q
FROM pg_stat_activity
WHERE datname = 't' AND pid <> pg_backend_pid()
ORDER BY pid;
```

result (real output)

```
 pid  | blocked_by | wait_event_type |  wait_event   | state  |               q
------+------------+-----------------+---------------+--------+------------------------------
 3479 | {3480}     | Lock            | transactionid | active | update seat set holder='bob'..
 3481 | {}         | Client          | ClientRead    | idle   |
```

NOWAIT, SKIP LOCKED and lock_timeout

third session, real output

```
t=# BEGIN; SELECT * FROM seat WHERE id = 1 FOR UPDATE NOWAIT;
ERROR:  could not obtain lock on row in relation "seat"

t=# ROLLBACK; BEGIN; SELECT * FROM seat ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 2;
 id | holder
----+--------
  2 |
  3 |

t=# ROLLBACK; SET lock_timeout = '500ms'; UPDATE seat SET holder = 'cat' WHERE id = 1;
ERROR:  canceling statement due to lock timeout
```

`NOWAIT` failed immediately, `SKIP LOCKED` returned the next two free rows (2 and 3, skipping the locked row 1), and `lock_timeout` aborted the wait after 500 ms.

Advisory locks

two sessions, real output

```
A: SELECT pg_advisory_lock(42);
B: SELECT pg_try_advisory_lock(42);
 f
A: SELECT pg_advisory_unlock(42);
 t
B: SELECT pg_try_advisory_lock(42);
 t
```

Session-level advisory locks persist until released or until the session ends, which makes them dangerous with connection pools: the lock stays with the physical connection. Transaction-level variants (`pg_advisory_xact_lock`) release at commit and are safe in pooled environments.

Optimistic locking with a version column

two sessions, real output (product stock = 10, version = 0)

```
A: SELECT id, stock, version FROM product WHERE id = 1;       -- 1 | 10 | 0
B: SELECT id, stock, version FROM product WHERE id = 1;       -- 1 | 10 | 0
A: UPDATE product SET stock = 9, version = version + 1 WHERE id = 1 AND version = 0;
UPDATE 1
B: UPDATE product SET stock = 9, version = version + 1 WHERE id = 1 AND version = 0;
UPDATE 0
```

B updated zero rows, which is the signal that the row changed since B read it. In Spring Data JPA the field annotated `@Version` produces the same condition, and a zero count is raised as `ObjectOptimisticLockingFailureException`.

Deadlock reproduced

Two transactions transfer between accounts 1 and 2 in opposite order.

two sessions, real output

```
A: BEGIN; UPDATE acc2 SET bal = bal - 10 WHERE id = 1;
B: BEGIN; UPDATE acc2 SET bal = bal - 10 WHERE id = 2;
A: UPDATE acc2 SET bal = bal + 10 WHERE id = 2;      -- waits for B
B: UPDATE acc2 SET bal = bal + 10 WHERE id = 1;      -- waits for A: cycle
A: ERROR:  deadlock detected
   DETAIL:  Process 3580 waits for ShareLock on transaction 818; blocked by process 3583.
   Process 3583 waits for ShareLock on transaction 815; blocked by process 3580.
   HINT:  See server log for query details.
   CONTEXT:  while updating tuple (0,2) in relation "acc2"
```

The server log contains the complete picture:

server log (real output, timestamps shortened)

```
LOG:  process 3580 detected deadlock while waiting for ShareLock on transaction 818
      after 1000.105 ms
DETAIL:  Process holding the lock: 3583. Wait queue: .
CONTEXT:  while updating tuple (0,2) in relation "acc2"
STATEMENT:  update acc2 set bal=bal+10 where id=2;
ERROR:  deadlock detected
DETAIL:  Process 3580 waits for ShareLock on transaction 818; blocked by process 3583.
        Process 3583 waits for ShareLock on transaction 815; blocked by process 3580.
        Process 3580: update acc2 set bal=bal+10 where id=2;
        Process 3583: update acc2 set bal=bal+10 where id=1;
```

Anatomy of the message: the first `LOG` line says which backend (3580) ran the detector and after how long (`deadlock_timeout`, 1000 ms). The `DETAIL` lists, for every participant, the lock it waits for, the transaction it waits for and the process that holds it, and finally the statement each process was executing. Together they identify the cycle, the statements, and the table and tuple, which is sufficient to locate the conflicting code paths. The victim (3580) received `40P01`; the other transaction proceeded.

1.23.4 Real-world problems and solutions

Incident: deadlock between two updates

Scenario. A funds transfer service debits the source account and then credits the target account. Two simultaneous transfers (A to B and B to A) arrive.

Symptoms. Intermittent errors `deadlock detected` (SQLSTATE 40P01; MySQL error 1213) in the application log, correlated with peak load. Retried requests succeed.

Diagnosis.

1. Locate the report in the server log (`log_lock_waits = on` also records waits longer than `deadlock_timeout`). In MySQL read the `LATEST DETECTED DEADLOCK` section of `SHOW ENGINE INNODB STATUS` or enable `innodb_print_all_deadlocks`.
2. Extract the two statements and the relation from the `DETAIL` lines (shown above).
3. Map each statement to the code path and note the order in which each path touches rows.

Root cause. The code paths locked rows in different orders (source first, target second), so opposite transfers formed a cycle.

Fix. Immediate: retry on SQLSTATE 40P01 with a short randomised backoff. Permanent: acquire locks in a global order, for example by ascending account id.

ordered_locking.sql — SQL

```sql
BEGIN;
SELECT id FROM account WHERE id IN (:from, :to) ORDER BY id FOR UPDATE;
UPDATE account SET balance = balance - :amt WHERE id = :from;
UPDATE account SET balance = balance + :amt WHERE id = :to;
COMMIT;
```

Prevention. Document the lock order for each table set; avoid mixing row locks from user interaction; keep transactions short; add a deadlock test that runs opposite transfers in parallel.

Incident: lock queue pile-up behind a DDL statement

Scenario. A migration executes `ALTER TABLE acc2 ADD COLUMN note text` during business hours. The table is busy but the DDL itself is metadata-only and would take milliseconds. At the moment it starts, a reporting session holds an open transaction that has read the table.

Symptoms. Within seconds every query on the table hangs. Connection pool utilisation rises to 100 percent and the application times out, although CPU is idle.

Diagnosis. In `pg_stat_activity` look for the chain of sessions waiting on `Lock`.

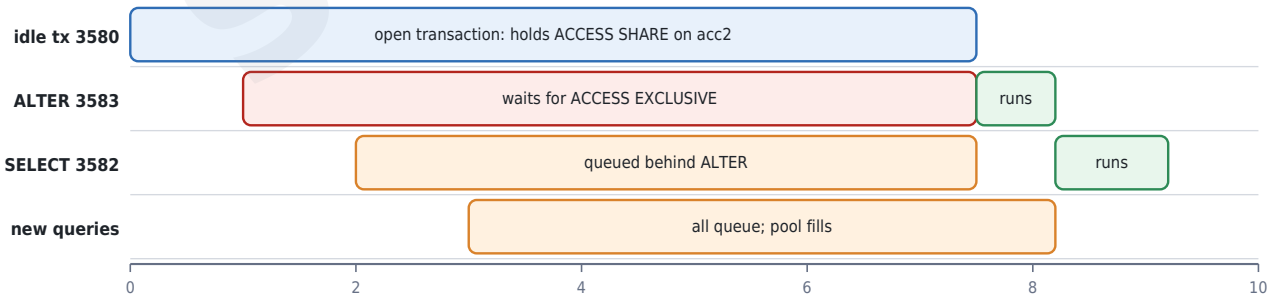


Figure 1.109. The DDL waits for an ACCESS EXCLUSIVE lock behind a reader that has not finished; every later query queues behind the DDL, so one idle transaction stalls the whole table.

real output

```
 pid  | blocked_by |        state        | wait_event_type |                  q
------+------------+---------------------+-----------------+-------------------------------------
 3580 | {}         | idle in transaction | Client          | select txid_current() > 0 as open_tx;
 3582 | {3583}     | active              | Lock            | select * from acc2 where id=2;
 3583 | {3580}     | active              | Lock            | alter table acc2 add column note text;
```

The ALTER (3583) is blocked by the idle transaction (3580), and the plain SELECT (3582) is blocked by the ALTER, even though a SELECT does not conflict with the idle transaction. The log with `log_lock_waits` shows the queue:

server log (real output)

```
LOG:  process 3583 still waiting for AccessExclusiveLock on relation 16522
      of database 16384 after 1000.196 ms
DETAIL:  Process holding the lock: 3580. Wait queue: 3583, 3582.
LOG:  process 3582 still waiting for AccessShareLock on relation 16522
      of database 16384 after 1000.149 ms at character 15
DETAIL:  Process holding the lock: 3580. Wait queue: 3583, 3582.
```

Root cause. Lock requests are served in queue order. The `ACCESS EXCLUSIVE` request waits for the old reader, and every compatible request that arrives later queues behind it, to avoid starving the DDL.

Fix. Immediate: terminate the idle transaction (`pg_terminate_backend(3580)`) or cancel the DDL (`pg_cancel_backend(3583)`); either one releases the queue. Permanent: run every DDL on a hot table with a short `lock_timeout` and a retry loop.

safe_ddl.sql SQL

```sql
SET lock_timeout = '2s';
ALTER TABLE acc2 ADD COLUMN note text;   -- fails fast instead of stalling the table
```

Prevention. Set `idle_in_transaction_session_timeout`, alert on sessions older than a few seconds waiting on `Lock`, and use `CREATE INDEX CONCURRENTLY`. In MySQL 8.x, `ALTER TABLE` waits on metadata locks (`Waiting for table metadata lock` in `SHOW PROCESSLIST`) with identical pile-up behaviour; `lock_wait_timeout` limits it, and `ALGORITHM=INSTANT` or `INPLACE` reduces the lock hold time.

Incident: lock waits caused by long transactions with user interaction

Scenario. A back-office screen loads an order with `SELECT ... FOR UPDATE` when the operator opens it and commits when the operator presses Save, up to several minutes later.

Symptoms. Other operators and batch jobs touching the same orders wait; `lock_timeout` errors or hangs appear.

Diagnosis. `pg_blocking_pids` of the waiting sessions leads to a session `idle in transaction` with an old `xact_start`.

Root cause. A pessimistic lock was held across user think time.

Fix. Replace the lock by optimistic concurrency: load the order with its version, and on save issue `UPDATE ... WHERE id = ? AND version = ?`, reporting a conflict dialog when the count is zero. Use `FOR UPDATE SKIP LOCKED` for worker queues.

Prevention. Never keep a database transaction open while waiting for a person or a remote service.

1.23.5 Questions to expect

INTERVIEW ANGLE

How does PostgreSQL lock a row, and why does a blocked session wait on a transaction id? Row locks are recorded in the tuple header (`xmax` and flag bits), not in a lock table. A second writer finds the live locker's transaction id there and waits for that transaction's id lock, which is released at commit or rollback. This avoids lock-table memory costs for large updates.

1. What conflicts with `ACCESS EXCLUSIVE`? Every other table lock mode, including `ACCESS SHARE` taken by `SELECT`.
2. What does `SKIP LOCKED` do and when is it appropriate? It skips rows locked by other transactions, so concurrent workers can claim different rows of a queue table; it is not appropriate for queries that need a complete result.
3. How are deadlocks detected and resolved? The wait-for graph is checked after `deadlock_timeout`; one transaction is aborted with 40P01. The application must retry. Consistent lock ordering removes most deadlocks.
4. Pessimistic versus optimistic locking? Pessimistic locks first and blocks others; optimistic checks a version at write time and fails on conflict. Optimistic suits low contention and interactive flows.
5. What does a gap lock protect against in InnoDB? Insertion into an index range, which prevents phantoms for locking reads at REPEATABLE READ. It can also cause surprising waits and deadlocks on inserts.
6. Why can a harmless `ALTER TABLE` freeze an application? It queues for `ACCESS EXCLUSIVE` behind running transactions, and all new queries queue behind it. Use `lock_timeout` and short transactions.
7. When are advisory locks useful, and what is the pooling risk? They serialise application-level jobs, for example a scheduled task on several instances; session-level locks outlive the logical request when the connection returns to the pool, so prefer transaction-level locks.

8. How does `@Version` work in JPA? Hibernate appends `AND version = ?` to the update and increments the version; zero rows updated produces an optimistic locking exception.

KEY POINTS

Key points

- Row locks in PostgreSQL are stored in the tuple (`xmax`) and waited for through the holder's transaction id; table locks are visible in `pg_locks`.
- `pg_blocking_pids` and `pg_stat_activity` locate blockers quickly; `log_lock_waits` records the queue.
- A DDL statement that waits for `ACCESS EXCLUSIVE` blocks all later queries; use `lock_timeout` and short transactions.
- Deadlocks follow from inconsistent lock order; the fix is a global order plus retry on 40P01.
- Optimistic locking with a version column avoids holding locks across think time; a zero update count is the conflict signal.

1.24 Durability: Write-Ahead Logging, Checkpoints and Crash Recovery

A database must acknowledge a commit only when the change can no longer be lost, yet writing every modified 8 kB data page to disk at each commit would be slow, because the pages are scattered across files and the writes are random. The solution used by PostgreSQL, InnoDB and nearly every other transactional engine is **write-ahead logging (WAL)**: the description of every change is first appended to a sequential log, the log is forced to stable storage at commit, and the modified data pages are written lazily later. After a crash the engine replays the log and reconstructs every committed change that had not reached the data files.

This chapter describes the WAL protocol, the role of checkpoints, the durability settings and their trade-offs, the recovery procedure, and the real behaviour observed on PostgreSQL 16.15 using `pg_waldump` and a deliberately killed server process.

1.24.1 Concept and mechanism

The write-ahead rule

The WAL protocol rests on two rules.

1. **Write-ahead:** a modified data page may be written to disk only after all log records describing changes to that page have been flushed to the log file.
2. **Force-log-at-commit:** a transaction is committed only after its commit record, and therefore every earlier log record, has been flushed to the log file.

Under these rules the log alone suffices to restore the committed state. The data pages in the buffer pool (`shared_buffers` in PostgreSQL) may be written at any time (**steal**) and need not be written at commit (**no-force**), which allows the engine to optimise data I/O freely.

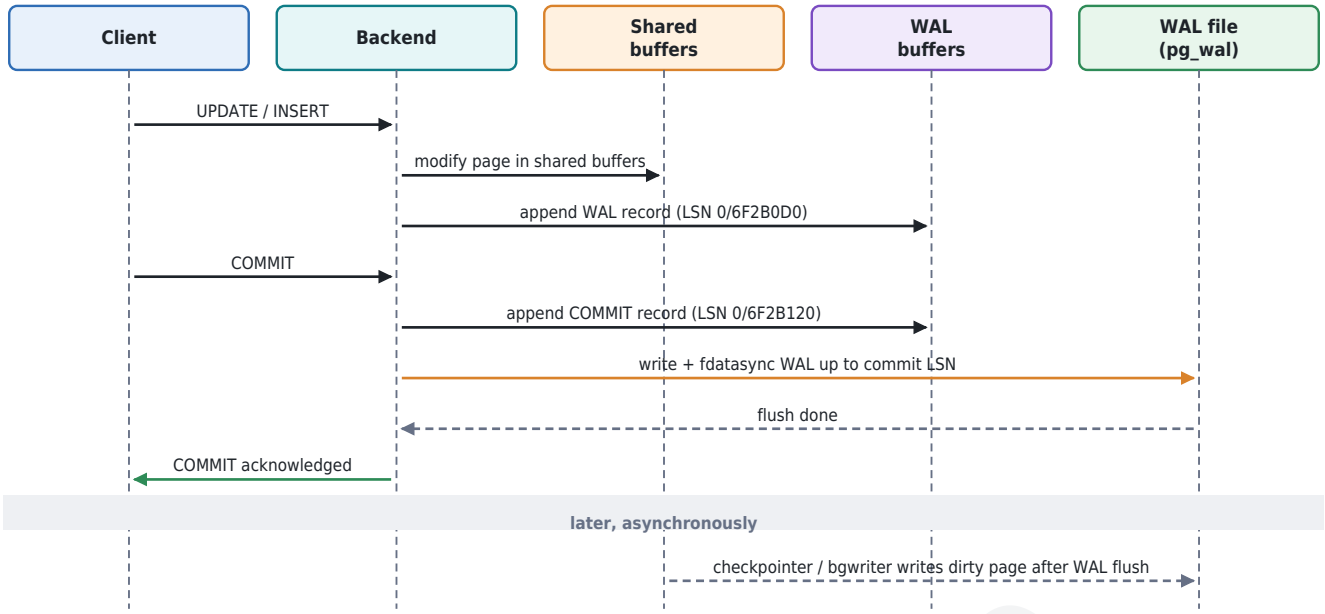


Figure 1.110. A commit appends records to the WAL buffer, flushes the log with fsync and then acknowledges the client; the dirty data page is written later by the background writer or the checkpointer.

Where the pieces live in PostgreSQL

An `UPDATE` modifies a page in `shared_buffers` and generates a WAL record in the WAL buffers (`wal_buffers`, 2 MB on the build cluster). Each record has a **log sequence number (LSN)**, a 64-bit byte position in the log stream. Every page stores the LSN of the last record that modified it (`pd_lsn`), which allows the write-ahead rule to be enforced: before a page is written, the log is flushed up to that LSN. The WAL is stored in 16 MB segment files in `pg_wal`, named after timeline and LSN, for example `000000010000000000000006`. At commit a WAL writer or the committing backend flushes the log with `fdatasync` (`wal_sync_method`, default `fdatasync` on Linux), and only then reports success.

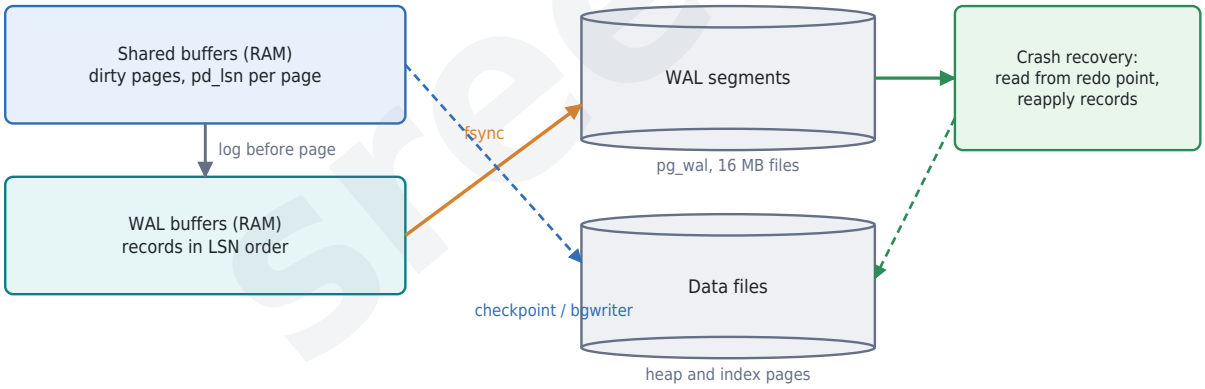


Figure 1.111. Dirty pages stay in shared buffers while the log is flushed sequentially at commit; checkpointed pages reach the data files later, and recovery replays the log from the last checkpoint.

Checkpoints

If the log grew forever, recovery would take forever as well. A **checkpoint** writes all dirty pages to the data files and records the LSN at which this was completed. Recovery has to replay only the records after the last checkpoint's **redo point**, and older WAL files can be recycled. PostgreSQL starts a checkpoint every `checkpoint_timeout` (default 5 minutes) or when the WAL volume since the last one approaches `max_wal_size` (default 1 GB), and spreads the writes over `checkpoint_completion_target` (0.9) of the interval to avoid I/O bursts.

Full-page writes. If the operating system crashes while an 8 kB page is being written, the file may contain half of the old page and half of the new one (a **torn page**). Row-level WAL records cannot repair a torn page. PostgreSQL therefore writes the entire page image into the WAL the first time the page is modified after a checkpoint (`full_page_writes = on`). This is why the first changes after a checkpoint generate much more WAL than later ones.

InnoDB solves the same problem with a **doublewrite buffer**: each page is written first to a sequential doublewrite area and then to its final location, and recovery restores torn pages from the doublewrite copy.

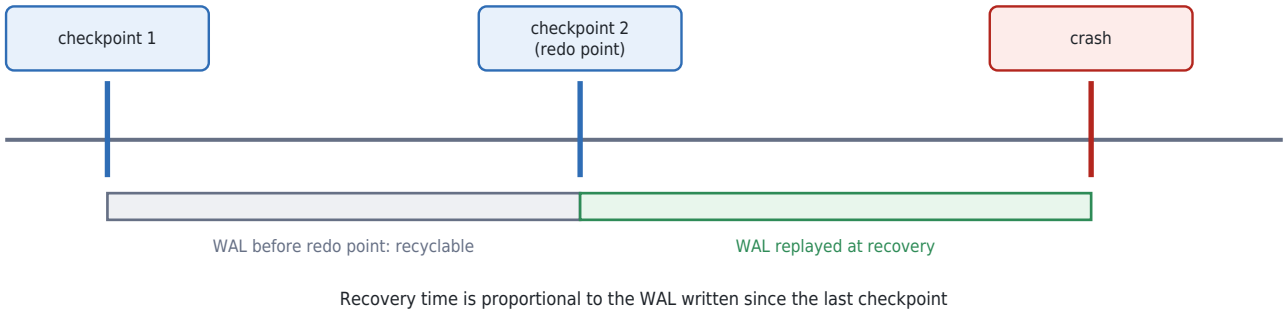


Figure 1.112. Between checkpoints the WAL grows; a crash after the second checkpoint requires redo only from that checkpoint's redo point, not from the start.

Recovery: redo, undo and ARIES

The classical recovery algorithm **ARIES** (Algorithms for Recovery and Isolation Exploiting Semantics) has three passes.

1. **Analysis.** Starting at the last checkpoint, scan the log forward to find which transactions were active at the crash and which pages were dirty.
2. **Redo.** Repeat history: reapply every logged change from the redo point onward, including the changes of transactions that will later be rolled back. A page is changed only if its stored LSN is lower than the record's LSN, which makes redo idempotent.
3. **Undo.** Roll back the transactions that had no commit record, writing compensation records so that a crash during recovery can itself be recovered.

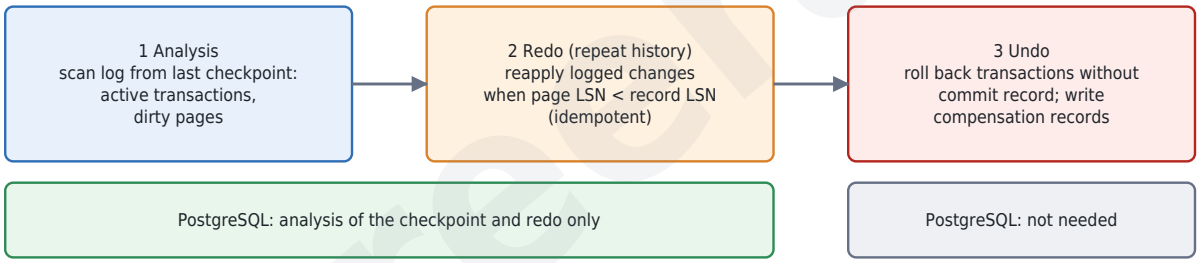


Figure 1.113. ARIES recovery runs analysis, redo and undo; PostgreSQL performs the first two and replaces the third with visibility rules, because tuples of uncommitted transactions are simply invisible.

PostgreSQL implements the redo pass and has no undo pass. Changes made by a transaction without a commit record leave tuple versions with an `xmin` that is never marked committed. Visibility rules ignore them, and VACUUM removes them later. This is why PostgreSQL crash recovery time depends on the WAL volume since the last checkpoint, not on the size of in-flight transactions. InnoDB uses the redo log (`ib_logfile` files, or `#innodb_redo` directory in MySQL 8.0.30 and later, sized by `innodb_redo_log_capacity`) for the redo pass and the undo log for rolling back incomplete transactions, so it has both phases.

Durability settings

Setting	Meaning	Risk when relaxed
`fsync`	flush WAL and data files to stable storage; default on	`off` can corrupt the cluster after a crash; never use in production
`synchronous_commit`	wait for WAL flush before acknowledging; default on	`off`: the most recent commits (up to about three times `wal_writer_delay`, 200 ms by default) can be lost on crash, but no corruption
`full_page_writes`	write full page images after a checkpoint; default on	`off` risks torn pages and corruption
`wal_sync_method`	system call used to flush; `fdatasync` default	wrong choice or a disk that lies about flushes defeats durability
MySQL `innodb_flush_log_at_trx_commit`	`1` flush at every commit (default); `2` write at commit, flush each second; `0` write and flush each second	`2` loses up to about one second on an OS crash; `0` also on a mysqld crash
MySQL `sync_binlog`	`1` flush the binary log at every commit (default)	`0` risks replicas and point-in-time recovery data

Durability-related settings and their effect (PostgreSQL 16; MySQL equivalent as documented).

`synchronous_commit = off` is a transaction-level setting and can be applied to selected workloads, such as bulk event ingestion, while financial transactions keep the default. With synchronous replication, the same setting also has `remote_write`, `remote_apply` and `local` values, covered in the chapter on replication.

1.24.2 Worked example

The scenario follows one small transaction through the WAL. The statements are `INSERT INTO wal_t VALUES (1, 'hello')` and `UPDATE wal_t SET v = 'world' WHERE id = 1` inside one transaction, executed after a checkpoint.

1. `BEGIN` writes nothing. The first modification allocates transaction id 822.
2. The `INSERT` places the new tuple into a new heap page. The backend appends a `Heap INSERT+INIT` record (65 bytes) and, because the table's primary key index is empty, creates the index root: `Btree NEWROOT` (90 bytes) and `Btree INSERT_LEAF` (64 bytes). The modified pages stay in `shared_buffers` with their `pd_lsn` set to the LSN of the last record.
3. The `UPDATE` changes a non-indexed column on the same page, so it is a HOT update: one `Heap HOT_UPDATE` record (73 bytes) with the new tuple, and no index record.
4. `COMMIT` appends a `Transaction COMMIT` record (34 bytes) and flushes the log up to its LSN with `fdatasync`. Only then does the client receive `COMMIT`.
5. The three dirty pages are still only in memory. If the server crashes now, recovery replays these five records from the log and rebuilds the pages; this is the durability guarantee at work.

1.24.3 Hands-on

Reading the log with pg_waldump

Before the transaction, a checkpoint was forced and the current LSN was saved.

wal_probe.sql SQL

```sql
CHECKPOINT;
SELECT pg_current_wal_lsn() AS before_lsn \gset
BEGIN;
INSERT INTO wal_t VALUES (1, 'hello');
UPDATE wal_t SET v = 'world' WHERE id = 1;
COMMIT;
SELECT pg_current_wal_lsn() AS after_lsn,
       pg_walfile_name(pg_current_wal_lsn()) AS walfile,
       pg_wal_lsn_diff(pg_current_wal_lsn(), :'before_lsn') AS bytes;
```

result (real output)

```
 after_lsn |         walfile          | bytes
-----------+--------------------------+-------
 0/6F2B148 | 000000010000000000000006 |  7808
```

The 7808 bytes include a 7376-byte record produced by page pruning of a catalog page in the same interval (a full-page image); the transaction itself produced five records. `pg_waldump` decodes the range:

pg_waldump BASH

```
$ pg_waldump -p pgdata/pg_wal -s 0/6F292C8 -e 0/6F2B148 00000001000000000000006
Heap2  rec/tot 60/7376 tx 0   lsn 0/06F292C8 PRUNE ... blkref rel 1663/16384/1255 blk 45 FPW
Heap   rec/tot 65/65   tx 822 lsn 0/06F2AFB0 INSERT+INIT off: 1 ...
Btree  rec/tot 90/90   tx 822 lsn 0/06F2AFF8 NEWROOT level: 0 ...
Btree  rec/tot 64/64   tx 822 lsn 0/06F2B058 INSERT_LEAF off: 1 ...
Standby rec/tot 54/54  tx 0   lsn 0/06F2B098 RUNNING_XACTS nextXid 823 ...
Heap   rec/tot 73/73   tx 822 lsn 0/06F2B0D0 HOT_UPDATE old_xmax: 822 ...
Transaction rec/tot 34/34 tx 822 lsn 0/06F2B120 COMMIT 2026-10-18 06:53:43
```

The columns are condensed and shortened where marked by `...`; the real lines contain the labels `rmgr`, `len` and `desc` and more fields. Each record names its resource manager (`rmgr`), length, transaction, LSN and a description of the physical change. The records of transaction 822 follow the plan of the worked example. The statistics view confirms the proportions:

pg_waldump --stats BASH

```
$ pg_waldump -p pgdata/pg_wal -s 0/6F292C8 -e 0/6F2B148 --stats=record 00000001000000000000006
Type                   N   Record size   FPI size   Combined size
Transaction/COMMIT     1            34          0              34
Standby/RUNNING_XACTS  1            54          0              54
Heap2/PRUNE            1            60       7316            7376
Heap/HOT_UPDATE        1            73          0              73
Heap/INSERT+INIT       1            65          0              65
Btree/INSERT_LEAF      1            64          0              64
Btree/NEWROOT          1            90          0              90
Total                  7           440       7316            7756
```

The logical work of the transaction cost under 400 bytes of WAL. The full-page image (`FPI size`) dominates the total, which illustrates why the period after each checkpoint is more expensive.

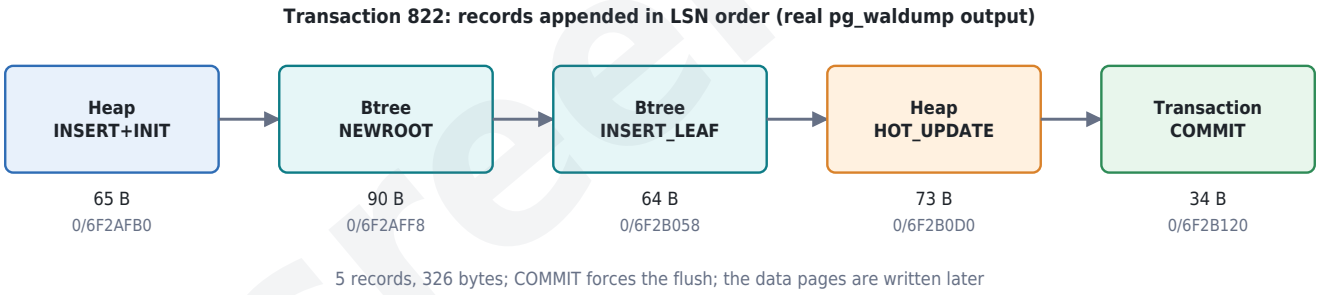


Figure 1.114. The real records of transaction 822 in LSN order: the insert, the index root creation, the leaf insert, the HOT update and the commit record that triggers the flush.

The price of the flush

`pgbench` with one client executing `INSERT INTO sc VALUES (1)` as a separate transaction measured the commit cost on the build machine (typical, a shared virtual disk).

pgbench results (measured on the build machine, typical)

```
synchronous_commit = on   : latency average = 1.419 ms   tps = 704.7
synchronous_commit = off  : latency average = 0.035 ms   tps = 28743.9
4 clients, synchronous_commit = on : latency average = 6.057 ms   tps = 660.4
pg_test_fsync: fdatasync 183 ops/sec = 5465 microseconds per 8 kB flush
```

With `on`, the rate is bounded by the speed of the flush device: about 700 commits per second. With `off`, the commit returns without waiting, a factor of about 40 faster. With four clients the throughput did not rise, because the single disk performed the same number of flushes; group commit, which combines the flushes of concurrent commits, needs more concurrency or a faster device to show an effect.

Sreeram

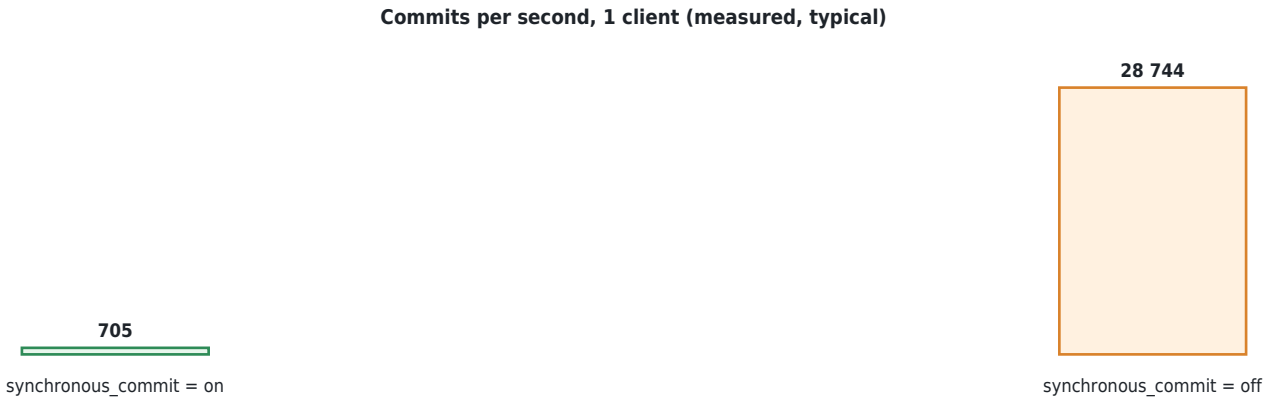


Figure 1.115. Commit throughput on the build machine with one client: the flush bounds synchronous commits at roughly 700 per second, and skipping it gives roughly 28 700 per second at the price of possible loss of the last commits.

Killing the server and watching recovery

Two sessions prepared a state, then a backend process was killed with signal 9 (`kill -9`), which forces the postmaster to restart all processes. Session A committed a row, session B had an open transaction that had inserted a row and updated row 1.

sessions before the crash (real output)

```
A: INSERT INTO crash_t VALUES (1, 'committed-before');          INSERT 0 1
B: BEGIN; INSERT INTO crash_t VALUES (2, 'uncommitted');
B: UPDATE crash_t SET v = 'dirty' WHERE id = 1;                 -- LSN 0/7B8F760
A: INSERT INTO crash_t VALUES (3, 'committed-after');           -- LSN 0/7B8F8F0
```

server log (real output)

```
LOG:  server process (PID 3880) was terminated by signal 9: Killed
DETAIL:  Failed process was running: select pg_backend_pid();
LOG:  terminating any other active server processes
LOG:  all server processes terminated; reinitializing
LOG:  database system was interrupted; last known up at 2026-10-18 06:54:22 ASIA
LOG:  database system was not properly shut down; automatic recovery in progress
LOG:  redo starts at 0/7B8F598
LOG:  invalid record length at 0/7B8F8F0: expected at least 24, got 0
LOG:  redo done at 0/7B8F8C8 system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.00 s
LOG:  checkpoint starting: end-of-recovery immediate wait
LOG:  checkpoint complete: wrote 6 buffers (0.1%); 0 WAL file(s) added, 0 removed, 0 recycled
LOG:  database system is ready to accept connections
```

Reading the log: `redo starts at 0/7B8F598` is the redo point of the last checkpoint. The message `invalid record length ... got 0` is the normal end of the log, where the writer stopped. `redo done at 0/7B8F8C8` is the last valid record, just before the final LSN of the committed row. After an end-of-recovery checkpoint the server accepts connections.

state after recovery (real output)

```
t=# SELECT * FROM crash_t ORDER BY id;
 id |        v
----+-----------------
  1 | committed-before
  3 | committed-after
```

Both committed rows survived, although their data pages had never been checkpointed. The uncommitted insert (id 2) and the uncommitted update to `dirty` are not visible, which is atomicity at work; no undo pass was necessary.

Checkpoint frequency

With a deliberately small `max_wal_size = 32MB`, `pgbench` (scale 5, 4 clients, 20 seconds) forced checkpoints because of WAL volume.

server log and statistics (real output, excerpt)

```
LOG:  checkpoint starting: wal
LOG:  checkpoint complete: wrote 1637 buffers (20.0%); 0 WAL file(s) added, 1 removed ...
LOG:  checkpoints are occurring too frequently (1 second apart)
HINT:  Consider increasing the configuration parameter "max_wal_size".

t=# SELECT checkpoints_timed, checkpoints_req, buffers_checkpoint FROM pg_stat_bgwriter;
 checkpoints_timed | checkpoints_req | buffers_checkpoint
-------------------+-----------------+--------------------
                 0 |              22 |              32445      -- max_wal_size = 32MB
                 0 |               0 |                  0      -- max_wal_size = 1GB (default)
```

The HINT line appears in the real server log directly after the warning. With the default of 1 GB no checkpoint was needed during the same 20-second run. The throughput on this machine was essentially identical (1603 versus 1617 transactions per second), so on a slow disk or a larger working set the effect would be larger; the write volume (32 445 pages versus none) is the reliable indicator.

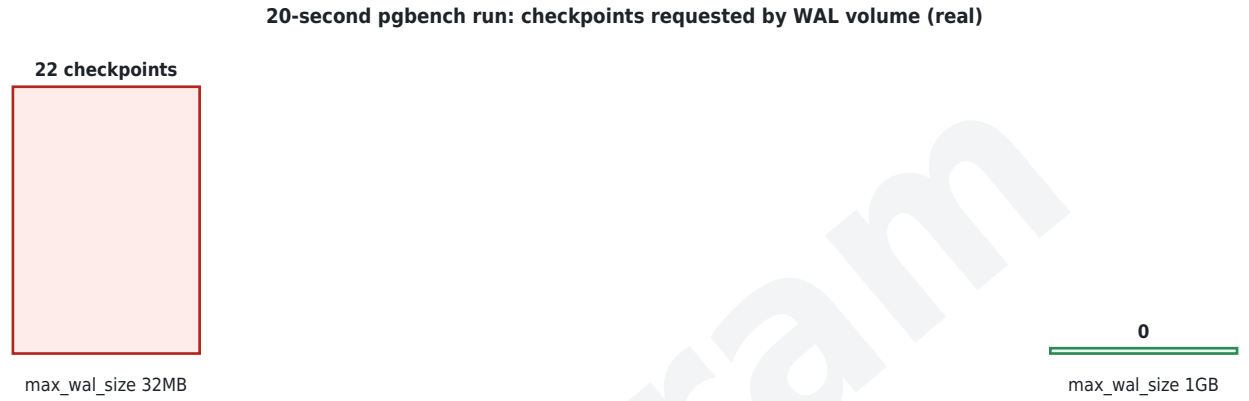


Figure 1.116. With a 32 MB max_wal_size the same 20-second workload triggered 22 requested checkpoints that wrote 32 445 buffers, against none with the default 1 GB.

1.24.4 Real-world problems and solutions

Incident: commit latency high because every commit waits for fsync

Scenario. An order service performs one small transaction per HTTP request. After a migration to a cloud volume, the 99th percentile latency of writes rises from 3 ms to 40 ms although CPU and queries are unchanged.

Symptoms. `pg_stat_activity` shows many sessions with `wait_event = WALSync` or `WALWrite`; throughput saturates at a few hundred commits per second.

Diagnosis.

1. Sample wait events: `SELECT wait_event_type, wait_event, count(*) FROM pg_stat_activity WHERE state = 'active' GROUP BY 1, 2 ORDER BY 3 DESC;`. A dominance of `IO / WALSync` points to the log flush.
2. Measure the device: `pg_test_fsync -f /path/on/the/wal/volume`. In this chapter's run, `fdatasync` took 5465 microseconds per flush; a local NVMe device would take well below a millisecond.
3. Check `pg_stat_wal` (`wal_sync`, `wal_sync_time` with `track_wal_io_timing = on`) for the flush count and time.

Root cause. Each commit requires one synchronous flush; the volume's flush latency bounds the number of commits per second per concurrent group.

Fix. Immediate: batch small writes into fewer transactions, and for non-critical writes set `SET LOCAL synchronous_commit = off` (accepting a loss window of a few hundred milliseconds). Permanent: place `pg_wal` on a low-latency device, and raise concurrency so that group commit amortises flushes (`commit_delay` and `commit_siblings` are available but rarely needed).

Prevention. Benchmark `pg_test_fsync` on every new volume type; verify that the storage honours flush requests (a volatile write cache without a battery silently breaks durability); keep a per-environment baseline of commit latency.

Incident: committed data lost after a crash

Scenario. To speed up an import, an operator set `fsync = off` and `synchronous_commit = off` in `postgresql.conf` and forgot to revert them. Weeks later the host lost power.

Symptoms. After the restart the latest minutes of orders are missing, or the server refuses to start with `invalid page in block ... of relation` or `could not locate a valid checkpoint record`.

Diagnosis. Check `SHOW fsync; SHOW synchronous_commit; SHOW full_page_writes;`, read the configuration history, and read the startup log for `redo` messages and page errors. `pg_controldata` shows the last checkpoint and cluster state.

Root cause. With `synchronous_commit = off` the acknowledgement precedes the log flush, so a crash loses the tail of the log (a bounded window, no corruption). With `fsync = off` the operating system may reorder or delay writes arbitrarily, and the cluster can be corrupted beyond recovery.

Fix. Immediate: restore from the latest backup and replay archived WAL (point-in-time recovery, covered in the chapter on backup); for a lost tail, re-ingest from the upstream source. Permanent: restore the defaults, and apply relaxed durability only per transaction (`SET LOCAL synchronous_commit = off`) for data that can be regenerated.

Prevention. Alert on non-default values of `fsync`, `full_page_writes` and `synchronous_commit` through configuration management; use `fsync = off` only for throwaway clusters, such as test fixtures, that are rebuilt from scratch.

Incident: periodic latency spikes from checkpoints

Scenario. Latency of a write-heavy service spikes every few minutes. The log contains `checkpoints are occurring too frequently`.

Symptoms. Spikes align with the lines `checkpoint starting: wal`; `pg_stat_bgwriter.checkpoints_req` grows faster than `checkpoints_timed`; WAL volume rises sharply after each checkpoint because of full-page images.

Diagnosis. Enable `log_checkpoints = on` (default on from PostgreSQL 15), compare `checkpoints_req` with `checkpoints_timed`, and measure the WAL rate with `pg_wal_lsn_diff` over a minute.

Root cause. `max_wal_size` is too small for the write rate, so WAL volume, not time, triggers checkpoints, each followed by a burst of full-page writes.

Fix. Raise `max_wal_size` (several GB is common for busy systems), leave `checkpoint_timeout` at 5 to 15 minutes and `checkpoint_completion_target` at 0.9, and consider `wal_compression = on` to shrink full-page images (off by default, as observed).

Prevention. Size `max_wal_size` to hold several minutes of peak WAL; keep an eye on recovery time, which grows with the checkpoint interval and the WAL volume.

1.24.5 Questions to expect

INTERVIEW ANGLE

Why is the log written before the data, and why is a commit fast although the data pages are not on disk? The log is sequential and small, so a flush is a single cheap write, and it contains enough information to redo the change after a crash. The data pages can be written later in any order, because recovery rebuilds them from the log. The write-ahead rule guarantees that no page reaches disk before its log records.

1. What is an LSN? A 64-bit position in the WAL stream; each page records the LSN of its last change, which enforces write-ahead and makes redo idempotent.
2. What does a checkpoint do and why does it matter for recovery time? It flushes dirty pages and records a redo point. Recovery replays only from that point, so the checkpoint interval and WAL volume bound recovery time.
3. What is a torn page and how is it handled? A partially written 8 kB page after a crash. PostgreSQL writes full-page images to WAL after each checkpoint; InnoDB uses the doublewrite buffer.
4. What does `synchronous_commit = off` risk? Loss of the most recent acknowledged transactions on a crash, without corruption. `fsync = off` risks corruption.
5. What are the phases of ARIES and which does PostgreSQL use? Analysis, redo, undo. PostgreSQL performs redo; uncommitted changes are invisible, so no undo pass is required.
6. How does `innodb_flush_log_at_trx_commit` compare? `1` flushes at each commit (durable), `2` writes at commit and flushes every second, `0` writes and flushes every second.
7. What does `kill -9` of a PostgreSQL backend cause? The postmaster terminates all backends and runs crash recovery; committed data is preserved, uncommitted data disappears.

8. Why can SSD or cloud storage break durability? A volatile cache that acknowledges flushes before the data is stable. The configuration must be tested with `pg_test_fsync` and verified with the provider.

KEY POINTS

Key points

- WAL: log first, flush at commit, write data pages later; this makes commits cheap and crash recovery possible.
- Checkpoints bound recovery time; full-page writes protect against torn pages.
- PostgreSQL recovery replays redo only; uncommitted changes are discarded by visibility rules.
- `synchronous_commit = off` trades a bounded loss window for speed; `fsync = off` risks corruption and must not be used on real data.
- Measure flush latency and checkpoint frequency with `pg_test_fsync`, `pg_stat_wal` and `pg_stat_bgwriter`.

1.25 Transactions in Java and Spring

The guarantees of the previous chapters apply only to statements that actually run inside the intended transaction, on the intended connection, for the intended duration. In a Java service this depends on JDBC settings, on the connection pool, on the Spring proxy that implements `@Transactional`, and on the exceptions that leave a method. Most production transaction defects are not database defects: they are boundary errors in application code, such as a transaction annotation that is never applied, a rollback that never happens, or a transaction that stays open for the duration of a remote call.

This chapter describes the JDBC transaction API, the Spring declarative model with its propagation and rollback rules, the interaction with the connection pool, and the patterns for operations that span several databases or services: idempotency, the outbox and sagas. No JDBC driver or Spring artefact can be downloaded on the build machine, so Spring and JDBC listings are marked as not run; the proxy mechanism and the rollback rule were reproduced with plain Java programs that were run (JDK 21), and the SQL-level behaviour was verified on PostgreSQL 16.15.

1.25.1 Concept and mechanism

JDBC transaction control

A JDBC `Connection` starts in autocommit mode. The transaction API consists of the following calls.

- `setAutoCommit(false)` begins an explicit transaction on the connection; the next statement opens it on the server.
- `commit()` and `rollback()` end it. `setSavepoint()` and `rollback(Savepoint)` provide partial rollback.
- `setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ)` selects the level for subsequent transactions. Constants exist for the four standard levels.
- `setReadOnly(true)` is a hint that PostgreSQL turns into `SET TRANSACTION READ ONLY`.

A connection belongs to one thread at a time, and a transaction belongs to a connection. Everything that must be atomic has to run on the same connection between `setAutoCommit(false)` and `commit()`. A pool returns the connection with `close()`; a pool that finds an open transaction rolls it back (HikariCP does so unless the application already finished it).

Spring declarative transactions

Spring implements `@Transactional` with a **proxy**. When the container creates a bean whose class or methods carry the annotation, it wraps the bean in a proxy (a JDK dynamic proxy for interfaces, a CGLIB subclass otherwise). A call from another bean reaches the proxy first. The proxy asks the `PlatformTransactionManager` (for JDBC and JPA, `DataSourceTransactionManager` or `JpaTransactionManager`) to begin a transaction according to the attribute, binds the connection to the current thread, invokes the target method, and then commits or rolls back according to the outcome.

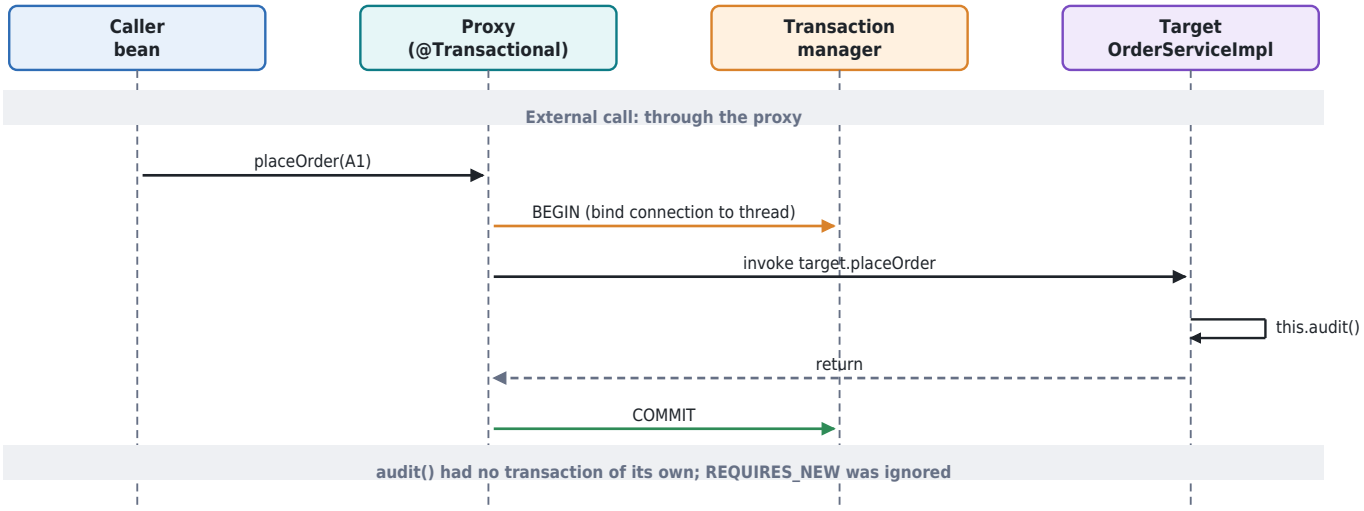


Figure 1.117. An external call passes through the transactional proxy and gets a transaction; a call from one method of the bean to another on this bypasses the proxy and gets nothing.

Three consequences follow from this design and explain most surprises.

1. **Self-invocation.** A call such as `this.audit()` inside the bean skips the proxy, so annotations on `audit` have no effect.
2. **Visibility.** Only public methods invoked through the proxy are intercepted in the default proxy mode; `private`, `final` or `static` methods and a call from the constructor are not.
3. **Thread binding.** The connection is bound to a thread (`TransactionSynchronizationManager`). Work handed to another thread, such as `@Async`, a parallel stream or a `CompletableFuture`, runs outside the transaction.

Attributes: propagation, isolation, read-only, timeout

Propagation says what happens when a transactional method is called while a transaction already exists.

Value	Behaviour
`REQUIRED`	Join the existing transaction, or start one if none exists.
`REQUIRES_NEW`	Suspend the existing transaction and run in a new one on a new connection; commits or fails independently.
`NESTED`	Run inside the existing transaction under a savepoint; failure rolls back to the savepoint (JDBC transaction managers only).
`SUPPORTS`	Join if one exists, otherwise run without a transaction.
`NOT_SUPPORTED`	Suspend any existing transaction and run without one.
`MANDATORY`	Require an existing transaction; throw if none exists.
`NEVER`	Throw if a transaction exists.

Propagation values of Spring `@Transactional` (default `REQUIRED`).

`REQUIRES_NEW` needs a second connection while the first is still held. With a small pool, nested `REQUIRES_NEW` calls from many threads at once can deadlock the pool itself, a point revisited in the incidents.

The `isolation` attribute issues the corresponding level at transaction start (the default, `DEFAULT`, uses the database default, READ COMMITTED on PostgreSQL and REPEATABLE READ on InnoDB). It takes effect only when the call starts a new transaction; a joined transaction keeps its level. `readOnly = true` allows optimisations (Hibernate skips dirty checking, PostgreSQL rejects writes) but does not make the code safe. `timeout` bounds the transaction duration in seconds.

Rollback rules

By default Spring rolls a transaction back when the method throws an unchecked exception (`RuntimeException` or `Error`), and commits when it throws a checked exception. This rule follows the EJB convention in which checked exceptions are treated as business outcomes. To change it, use `rollbackFor = Exception.class` or `noRollbackFor`. When a transactional method called through the proxy marks the shared transaction as rollback-only (for example because an inner `REQUIRED` method threw), the outer commit fails with `UnexpectedRollbackException`.

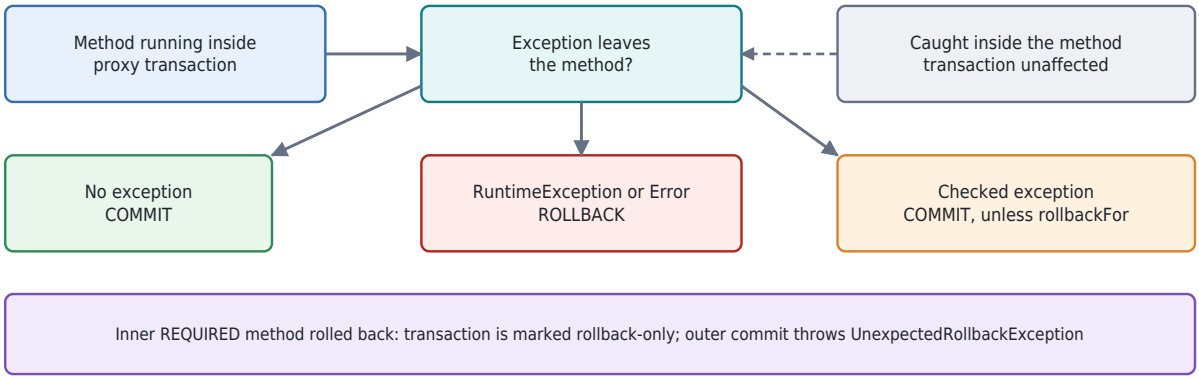


Figure 1.118. The exception type decides the outcome: unchecked exceptions roll back, checked exceptions commit unless rollbackFor says otherwise, and a caught exception inside the method leaves the transaction untouched.

Transaction boundaries and the connection pool

A pool such as HikariCP owns a fixed number of physical connections (`maximumPoolSize`, default 10) and lends them. With the default behaviour of the JDBC transaction manager, the connection is taken from the pool at the beginning of the transaction and returned at its end. The duration of the transaction is therefore the time during which the connection is unavailable to other threads. If the pool has 10 connections and each request holds a connection for 2 seconds while it waits for a remote service, the sustained capacity is 5 requests per second, regardless of how fast the database is. When all connections are in use, further callers wait for `connectionTimeout` (default 30 seconds) and then fail with `SQLTransientConnectionException`.

Two settings extend the lifetime unnoticed. With JPA, `spring.jpa.open-in-view` (default `true`) keeps the `EntityManager`, and under some configurations the connection, bound to the web request until the view is rendered, so lazy loading works in controllers but connections are held longer. Hibernate's `hibernate.connection.handling_mode` can release connections after each statement or after the transaction.

Idempotency, outbox and distributed work

A transaction covers one database. An operation that writes to a database and also publishes a message, calls another service or writes to a second store cannot be made atomic by `@Transactional` alone.

Idempotency. A retried request (after a timeout or a serialization failure) must not apply its effect twice. The usual technique is an idempotency key stored with a unique constraint: `INSERT INTO payment(idempotency_key, ...) ON CONFLICT (idempotency_key) DO NOTHING`, and the result of the first execution is returned to the retry.

Transactional outbox. The service writes the business row and an event row into an `outbox` table in the same local transaction. A separate relay process reads unpublished outbox rows (with `FOR UPDATE SKIP LOCKED`), publishes them to the broker and marks them as sent. Publication is at-least-once; consumers deduplicate by event id. This replaces a dual write, which can lose or duplicate messages, by one atomic local commit.

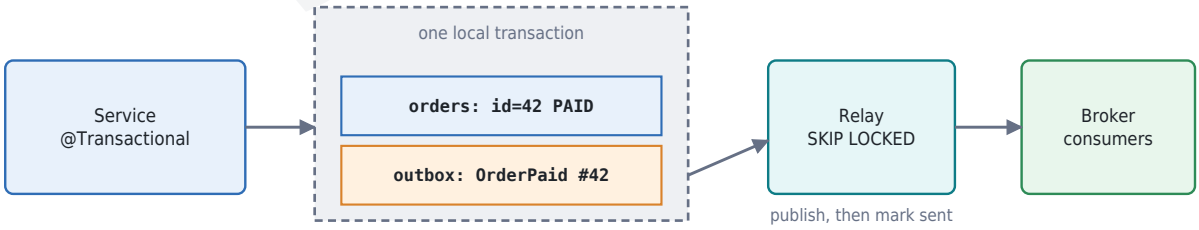


Figure 1.119. The order row and the outbox row are committed in one local transaction; the relay publishes afterwards, so a crash between commit and publish only delays the message.

Two-phase commit (2PC). The classic protocol for atomic commit across several resource managers has a coordinator and participants. In phase one the coordinator asks each participant to **prepare**: it makes its changes durable and promises that it can commit. In phase two the coordinator decides and tells all participants to commit or to roll back. PostgreSQL exposes the participant side as `PREPARE TRANSACTION 'gid'`, `COMMIT PREPARED 'gid'` and

`ROLLBACK PREPARED 'gid'`, with `max_prepared_transactions` set above its default of 0; JTA/XA implements the coordinator in Java.

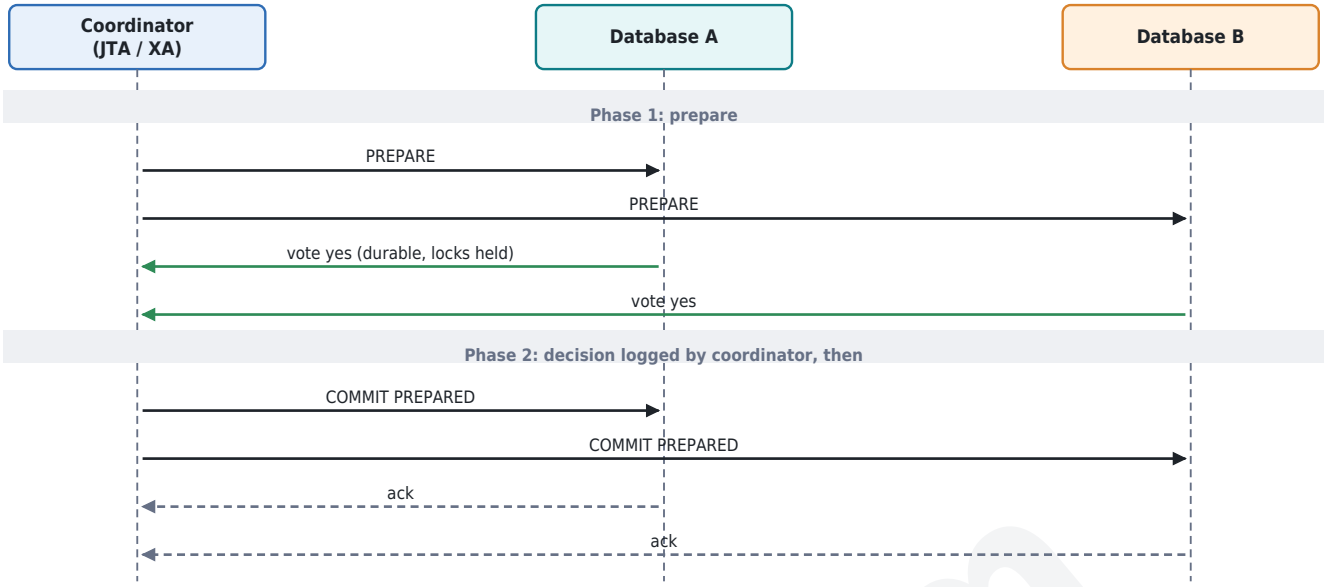


Figure 1.120. The coordinator collects prepare votes, writes the decision, then sends commit; a participant that has prepared must wait for the decision and holds its locks, which is the blocking weakness of 2PC.

A prepared transaction survives a crash and keeps its locks and its hold on the vacuum horizon until resolved, so an abandoned one causes the bloat described in the chapter on MVCC. For this and for availability reasons, 2PC is rare between services.

Sagas. A **saga** replaces one distributed transaction by a sequence of local transactions, each with a **compensating action** that semantically undoes it (for example `CancelPayment` for `CapturePayment`). If step *n* fails, the compensations for steps *n-1* to 1 run in reverse. Sagas provide atomicity of outcome but not isolation: other transactions can see intermediate states, so designs add semantic locks (an order in state `PENDING`) and make every step and compensation idempotent.

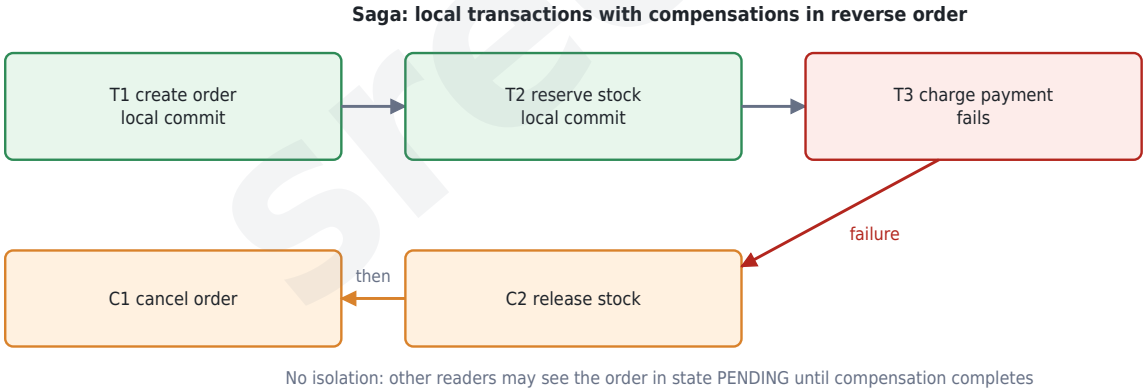


Figure 1.121. A three-step saga: the failure of the third local transaction triggers compensations for the first two in reverse order.

1.25.2 Worked example

The scenario is the self-invocation defect. `OrderService.placeOrder` is public and `@Transactional`; it calls `audit`, which is annotated `@Transactional(propagation = REQUIRES_NEW)` so that the audit row survives a rollback of the order. The expectation is that `audit` runs in its own transaction.

The following Java program reproduces the mechanism with a JDK dynamic proxy that logs when it would begin and commit a transaction. It is runnable on JDK 21 and stands in for the Spring proxy.

ProxyDemo.java JAVA

```java
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.List;

public class ProxyDemo {
    interface OrderService {
        void placeOrder(String id);
        void audit(String id);
    }

    static final List<String> log = new ArrayList<>();

    static class OrderServiceImpl implements OrderService {
        public void placeOrder(String id) {
            log.add("placeOrder body: " + id);
            audit(id);                  // self-invocation: plain call on this, bypasses the proxy
        }
        public void audit(String id) {
            log.add("audit body: " + id);
        }
    }

    public static void main(String[] args) {
        OrderService target = new OrderServiceImpl();
        InvocationHandler tx = (proxy, method, a) -> {
            log.add("BEGIN before " + method.getName());
            try {
                Object r = method.invoke(target, a);
                log.add("COMMIT after " + method.getName());
                return r;
            } catch (java.lang.reflect.InvocationTargetException e) {
                log.add("ROLLBACK after " + method.getName());
                throw e.getCause();
            }
        };
        OrderService svc = (OrderService) Proxy.newProxyInstance(
            OrderService.class.getClassLoader(), new Class<?>[] {OrderService.class}, tx);

        svc.placeOrder("A1");           // external call: goes through the proxy
        log.add("---");
        svc.audit("A1");                // direct external call to the second method
        log.forEach(System.out::println);
    }
}
```

Run the proxy demonstration TERMINAL

```
$ java ProxyDemo.java
BEGIN before placeOrder
placeOrder body: A1
audit body: A1
COMMIT after placeOrder
---
BEGIN before audit
audit body: A1
COMMIT after audit
```

What to notice: the first call shows one `BEGIN` and one `COMMIT`, both for `placeOrder`; the `audit` body ran inside the transaction of `placeOrder` and received no transaction of its own. Only the second call, made from outside, passed through the proxy and began a transaction for `audit`. In Spring the same applies, so a `REQUIRES_NEW` annotation on a method called through `this` is ignored, and a rollback of `placeOrder` also removes the audit row.

1.25.3 Hands-on

The rollback rule

The rule "roll back on unchecked, commit on checked" was verified with a runnable program that mirrors the default `TransactionAspectSupport` behaviour.

RollbackRule.java JAVA

```java
import java.io.IOException;
import java.util.concurrent.Callable;

public class RollbackRule {
    static String run(Callable<Void> work) {
        try {
            work.call();
            return "COMMIT";
        } catch (RuntimeException | Error e) {
            return "ROLLBACK (unchecked: " + e.getClass().getSimpleName() + ")";
        } catch (Exception e) {
            return "COMMIT (checked: " + e.getClass().getSimpleName()
                + " does not roll back by default)";
        }
    }

    public static void main(String[] args) {
        System.out.println(run(() -> { throw new IllegalStateException("x"); }));
        System.out.println(run(() -> { throw new IOException("x"); }));
        System.out.println(run(() -> { throw new StackOverflowError(); }));
        System.out.println(run(() -> null));
    }
}
```

Run the rollback rule demonstration TERMINAL

```
$ java RollbackRule.java
ROLLBACK (unchecked: IllegalStateException)
COMMIT (checked: IOException does not roll back by default)
ROLLBACK (unchecked: StackOverflowError)
COMMIT
```

A typical Spring service (not run)

The listing shows the correct form of a transfer service. It is a fragment that needs Spring Boot and a JDBC driver, neither of which is available on the build machine.

TransferService.java (fragment, not run) JAVA

```java
@Service
public class TransferService {
    private final AccountRepository accounts;
    private final AuditService audit;          // a separate bean: calls pass through its proxy

    public TransferService(AccountRepository accounts, AuditService audit) {
        this.accounts = accounts;
        this.audit = audit;
    }

    @Transactional(rollbackFor = Exception.class, timeout = 5)
    public void transfer(long from, long to, BigDecimal amount) throws InsufficientFunds {
        long first = Math.min(from, to), second = Math.max(from, to);
        accounts.lockInOrder(first, second);              // SELECT ... FOR UPDATE ORDER BY id
        if (accounts.debit(from, amount) == 0) {                  // guarded atomic UPDATE
            audit.record("REJECTED", from, to, amount);   // REQUIRES_NEW: survives rollback
            throw new InsufficientFunds(from);            // checked; rollbackFor covers it
        }
        accounts.credit(to, amount);
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(String kind, long from, long to, BigDecimal amount) {
        // inserts one audit row in its own transaction and commits on return
    }
}
```

What to notice: the audit call goes to another bean, so the proxy applies; the checked `InsufficientFunds` exception forces a rollback through `rollbackFor`; the lock order is fixed; and the transaction is bounded by `timeout`.

sreeram

Observing open transactions of a pool

When application threads hold transactions open, the database shows the same picture. Eight sessions executed `BEGIN` and a trivial query and then stayed idle, which is what a service does while waiting for a remote response.

psql session (real output)

```
t=# SELECT state, count(*), max(now() - xact_start) > interval '0' AS has_open_tx
    FROM pg_stat_activity WHERE datname = 't' AND pid <> pg_backend_pid() GROUP BY state;
        state        | count | has_open_tx
---------------------+-------+-------------
 idle in transaction |     8 | t

t=# SELECT count(*) AS sessions, current_setting('max_connections') AS max_conn
    FROM pg_stat_activity WHERE backend_type = 'client backend';
 sessions | max_conn
----------+----------
        9 | 30
```

Eight of the nine client sessions are `idle in transaction`. In a pool, each of them corresponds to a lent connection that does no database work.

1.25.4 Real-world problems and solutions

Incident: connection pool exhaustion from open transactions

Scenario. A checkout endpoint is annotated `@Transactional` and, inside the method, calls a payment provider through HTTP. The provider slows down from 200 ms to 8 seconds. The pool holds 10 connections.

Symptoms. Latency rises on every endpoint of the service, including those that use no payment. The log contains `HikariPool-1 - Connection is not available, request timed out after 30000ms` (typical text; recent versions append `(total=10, active=10, idle=0, waiting=N)`). Database CPU is low.

Diagnosis.

1. Check the pool metrics: `hikaricp.connections.active` equal to the maximum and `hikaricp.connections.pending` greater than 0.
2. On the database, group sessions by state; the real output above shows the shape. In the failing case most sessions are `idle in transaction`.
3. Identify what they wait for: the `query` column shows the last statement, and `now() - state_change` shows how long they have been idle.

idle_in_tx.sql SQL

```sql
SELECT pid, now() - state_change AS idle_for, left(query, 60) AS last_query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
ORDER BY state_change
LIMIT 10;
```

1. In the application, take a thread dump (`jcmd <pid> Thread.print`): the threads are blocked in the HTTP client below the transactional method.

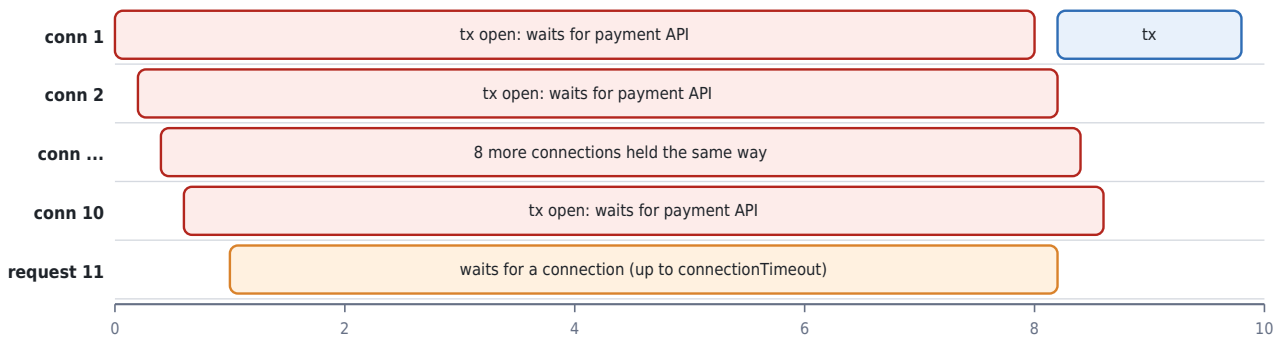


Figure 1.122. Each slow payment call holds a pooled connection inside an open transaction; once ten requests are in flight, every other request, even one that needs a millisecond of database time, waits for a connection.

sreeram

Root cause. The transaction boundary included a remote call, so connection hold time equalled the remote latency, and the pool capacity (10 divided by 8 seconds, about 1.2 requests per second) collapsed.

Fix. Immediate: add a client timeout and a circuit breaker for the provider; temporarily raise `maximumPoolSize` only if the database has headroom (a larger pool does not fix the cause and can overload the database). Permanent: restructure the method so that the transaction covers only the database work.

CheckoutService.java (fragment, not run) JAVA

```java
public void checkout(long orderId) {
    PaymentResult result = paymentClient.charge(orderId);      // no transaction open here
    orders.markPaid(orderId, result);                          // @Transactional, milliseconds
}

@Transactional
public void markPaid(long orderId, PaymentResult r) { /* short database work */ }
```

Because the two steps are no longer atomic, the charge call carries an idempotency key and a reconciliation job repairs orders that were charged but not marked.

Prevention. Set `idle_in_transaction_session_timeout` (for example 30 s), and statement and lock timeouts. Size the pool with the formula connections = (core count x 2) + effective spindle count as a starting point, and measure. Track connection hold time as a metric. A code rule: no network call inside a transaction.

Incident: @Transactional not applied (self-invocation)

Scenario. A developer annotates a private helper or a method called through `this` with `@Transactional(propagation = REQUIRES_NEW)` to write an audit row that must survive a failed order.

Symptoms. When the order fails, the audit row is missing as well. Or, a `@Transactional` method called from a scheduler inside the same class fails to roll back partial work; the log of `org.springframework.transaction` at `TRACE` shows no `Creating new transaction` line for the inner method.

Diagnosis.

1. Enable `logging.level.org.springframework.transaction.interceptor=TRACE` and `logging.level.org.springframework.orm.jpa=DEBUG`, and search for `Getting transaction for` and `Completing transaction for` around the call.
2. Verify that the call crosses a bean boundary: if the caller and callee are in the same class, the proxy is bypassed, as in the demonstration above.
3. Confirm at runtime with `TransactionSynchronizationManager.isActualTransactionActive()` and `getCurrentTransactionName()` inside the method.

Root cause. The annotation is implemented by a proxy; a call through `this` never reaches it.

Fix. Move the method to another bean (the simplest and most robust option), or inject the proxy of the own bean, or use `TransactionTemplate` programmatically for the block:

AuditWriter.java (fragment, not run) JAVA

```java
@Component
public class AuditWriter {
    private final TransactionTemplate requiresNew;
    private final JdbcTemplate jdbc;

    public AuditWriter(PlatformTransactionManager tm, JdbcTemplate jdbc) {
        this.jdbc = jdbc;
        this.requiresNew = new TransactionTemplate(tm);
        this.requiresNew.setPropagationBehavior(TransactionDefinition.PROPAGATION_REQUIRES_NEW);
    }

    public void record(String msg) {
        requiresNew.executeWithoutResult(
            status -> jdbc.update("insert into audit(msg) values (?)", msg));
    }
}
```

AspectJ mode (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)`) weaves the advice into the class and removes the limitation at the cost of build-time or load-time weaving.

Prevention. Cover every transactional behaviour that the business depends on with an integration test that checks the database state after a forced failure; make `@Transactional` visible only on public methods of service beans.

Incident: rollback does not happen on a checked exception

Scenario. A service method debits an account and then throws a checked `PaymentDeclinedException` to signal a business failure.

Symptoms. The exception reaches the controller, the customer sees an error, but the debit remains in the database.

Diagnosis. With `TRACE` logging on `org.springframework.transaction.interceptor`, the log shows `Completing transaction for [...]` followed by a commit rather than `Completing transaction for [...] after exception`. A unit test with an embedded database that asserts the balance after the exception fails.

Root cause. The default rule commits on checked exceptions, as shown by the real output of `RollbackRule`: `COMMIT (checked: IOException does not roll back by default)`.

Fix. Use `@Transactional(rollbackFor = PaymentDeclinedException.class)` or `rollbackFor = Exception.class` as a project default. Alternatively make the exception unchecked.

Prevention. Adopt a convention: business exceptions extend `RuntimeException`, or every service annotation carries `rollbackFor = Exception.class`. Review catch blocks: an exception that is caught inside the transactional method (for example to log it) also prevents the rollback, unless the code calls `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` or rethrows.

Incident: long transaction spanning a batch job

Scenario. A nightly job loads one million rows, processes each one and updates it, all in a single `@Transactional` method.

Symptoms. The transaction runs for two hours; table bloat grows, replication lag increases because the replica applies the changes in one burst at commit, locks held on updated rows block the daytime application, and a failure at row 900 000 rolls everything back.

Diagnosis. `pg_stat_activity` shows one session with `xact_start` hours old, and the horizon is held back (see the chapter on MVCC). `pg_locks` shows many row locks held by the same pid.

Root cause. The transaction boundary equals the job rather than a unit of work.

Fix. Process in chunks of a few hundred to a few thousand rows, each in its own transaction (a `TransactionTemplate` per chunk, or Spring Batch chunk-oriented steps), record the last processed key so that the job can be resumed, and make each chunk idempotent.

Prevention. Limit transaction duration through `transaction_timeout` (PostgreSQL 17 and later) or `statement_timeout`, and review jobs for unbounded transactions.

1.25.5 Questions to expect

INTERVIEW ANGLE

How does @Transactional work, and when does it silently fail? Spring wraps the bean in a proxy that begins a transaction, binds the connection to the thread, calls the method, and commits or rolls back. It fails silently when the call does not pass through the proxy (self-invocation, private or final methods), when the work moves to another thread, when the bean is not managed by Spring, and when a checked exception is thrown without `rollbackFor`.

1. What is the default rollback rule? Roll back on `RuntimeException` and `Error`, commit on checked exceptions. `rollbackFor` and `noRollbackFor` change it.
2. Difference between `REQUIRED`, `REQUIRES_NEW` and `NESTED`? `REQUIRED` joins or starts; `REQUIRES_NEW` suspends the current transaction and uses a second connection and a separate commit; `NESTED` uses a savepoint inside the same transaction.
3. How can a pool of 10 connections be exhausted by a database that is idle? Transactions that stay open while waiting for something else hold connections. The database sees `idle in transaction`.
4. Why should a remote call not occur inside a transaction? It extends connection and lock hold time to the remote latency and couples the failure of the remote system to the pool and the database.
5. What is the outbox pattern? The event is written to a table in the same transaction as the business data and published by a relay afterwards, giving at-least-once delivery without a distributed transaction.
6. Why is two-phase commit rarely used between services? Participants block holding locks while awaiting the coordinator, the coordinator is a single point of failure, and latency grows; sagas with compensations are preferred.
7. What is idempotency and how is it implemented for payments? Repeating a request has the same effect as running it once; an idempotency key with a unique constraint stores the first result and returns it to retries.

Freeram

8. What does `spring.jpa.open-in-view` change? It keeps the persistence context open for the whole request, enabling lazy loading in the view, at the cost of longer connection use and queries outside service boundaries. Many production setups disable it.
9. How does `isolation = SERIALIZABLE` change the code? Serialization failures (SQLSTATE 40001) must be retried around the whole transaction, outside the transactional method, for example through `@Retryable` placed on a caller bean.

KEY POINTS

Key points

- A Spring transaction is a thread-bound connection managed by a proxy; calls that bypass the proxy get no transaction.
- Default rollback is on unchecked exceptions only; use `rollbackFor` for business exceptions.
- Transaction duration equals connection hold time; never include remote calls or user waits inside a transaction.
- Use idempotency keys, the outbox and sagas instead of distributed transactions between services.
- Serialization failures and deadlocks require a retry of the whole transaction on a fresh connection.

1.26 Connections and Connection Pooling

A database connection is not a lightweight handle. It is an authenticated session with its own memory, its own transaction state and, in PostgreSQL, its own operating-system process. A Spring Boot service that opens a connection for every request spends more time connecting than querying, and a fleet of services that each open hundreds of connections can exhaust the server long before the CPU is busy. This chapter explains what a connection costs, how a pool bounds that cost, how JDBC talks to the server, and why prepared statements are both a performance feature and the only reliable defence against SQL injection.

1.26.1 Concept and mechanism

Process per connection versus thread per connection

PostgreSQL forks one backend process for every client connection. The postmaster process listens on the port, authenticates the client, and forks a backend that serves that session until it disconnects. Each backend has private memory (catalog caches, sort and hash memory governed by `work_mem`, prepared statement plans) and attaches to the shared memory segment that holds `shared_buffers`, the WAL buffers and the lock tables. MySQL with InnoDB uses one thread per connection inside a single `mysqld` process; threads are cheaper to create than processes, and a thread cache reuses them, but each thread still holds session buffers and a stack.

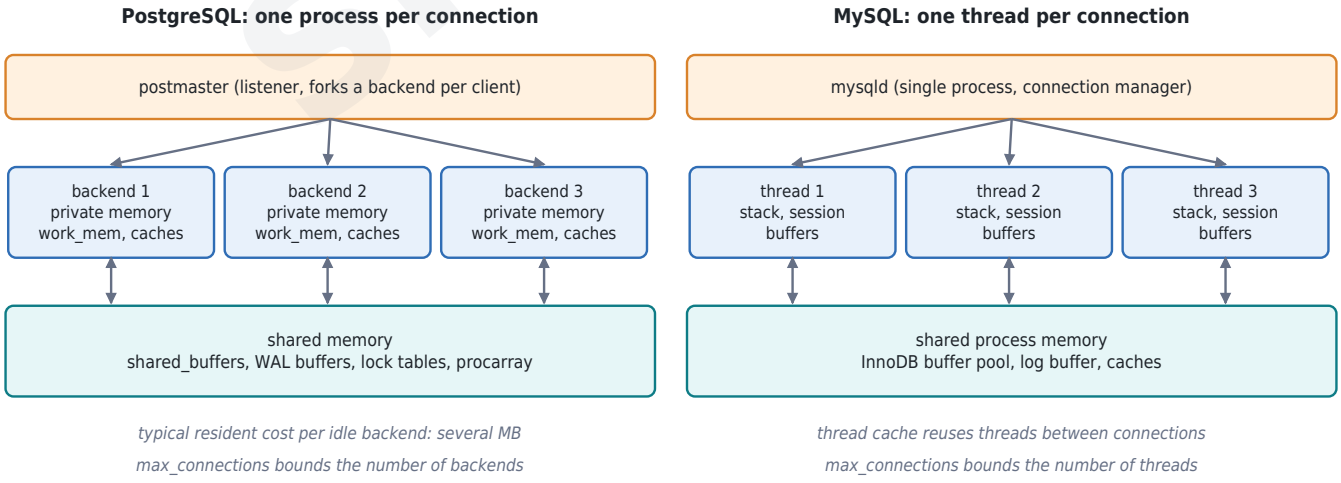


Figure 1.123. Both engines dedicate a server-side execution context to each client session; the number of contexts is capped by max_connections, and every context competes for the same CPUs and shared memory.

The consequences are the same in both engines. Connection establishment involves a TCP handshake, optional TLS negotiation, authentication and session initialisation. Measured on the build machine with `pgbench` (select-only, four clients, local socket), persistent connections sustained about 46 970 transactions per second at 0.085 ms average latency, while opening a new connection for every transaction (`pgbench -C`) sustained 233 transactions per second at

17.2 ms average latency, of which 8.3 ms was connection time. The query is identical in both runs; only the connection handling differs.

Persistent versus per-transaction connections (measured, typical) TERMINAL

```
$ pgbench -S -c 4 -j 2 -T 10 pgb
latency average = 0.085 ms
tps = 46968.644072 (without initial connection time)
$ pgbench -S -C -c 4 -j 2 -T 10 pgb
latency average = 17.153 ms
average connection time = 8.265 ms
tps = 233.192396 (including reconnection times)
```

max_connections and why more is not better

`max_connections` (default 100 in PostgreSQL; the experiments in this chapter use 30) is a hard ceiling. A fraction of it, `superuser_reserved_connections` (default 3), is kept for administrators so that a saturated server can still be diagnosed. When ordinary clients exceed the remaining slots, PostgreSQL refuses the connection immediately:

Connection refusal when the ordinary slots are used (real output) TERMINAL

```
$ psql -U appuser shop -c "select 1"
psql: error: connection to server on socket ".../.s.PGSQL.20500" failed:
FATAL:  remaining connection slots are reserved for roles with the SUPERUSER attribute
$ psql -U postgres shop -c "select 1 as superuser_still_ok"
 superuser_still_ok
--------------------
                  1
```

Raising `max_connections` is rarely the correct fix. Throughput is bounded by the hardware, not by the number of sessions. The measurement below drove a TPC-B-like workload (`pgbench` default script) against a server with two CPU cores. Throughput peaked at four clients and then fell, because additional sessions only add context switching and lock contention. The figures are noisy because the build machine is shared, but the shape is the general behaviour.

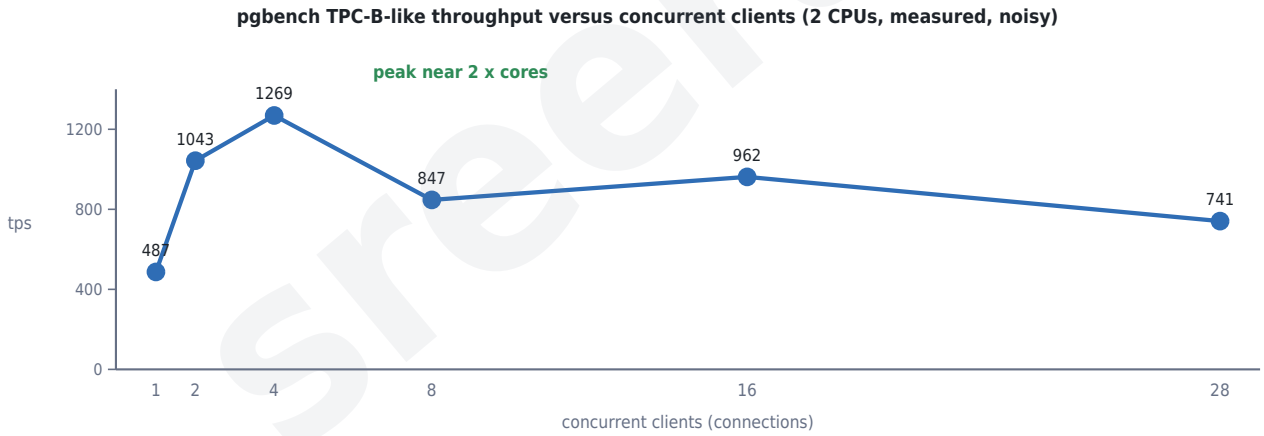


Figure 1.124. Throughput peaks when the number of active sessions is a small multiple of the CPU count and declines afterwards; a pool should therefore be small, and requests wait in the application rather than in the database.

Connection pools

A connection pool keeps a fixed set of established connections and lends them to request threads. A request borrows a connection, runs its statements, and returns the connection; the physical session stays open. HikariCP is the default pool in Spring Boot. Its central setting is `maximumPoolSize`: the maximum number of connections, idle plus in use. When all connections are borrowed, further requests wait up to `connectionTimeout` (30 000 ms by default) and then fail with an exception.

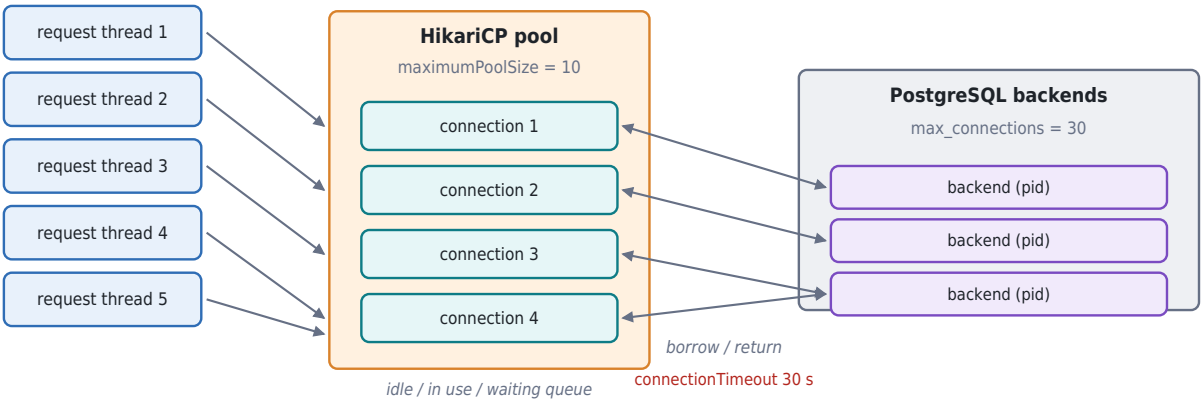


Figure 1.125. Five request threads share a pool of at most ten physical connections; the pool, not the request rate, determines how many backends the database sees.

The key HikariCP settings are listed below. Values are the documented defaults.

Property	Default	Meaning
`maximumPoolSize`	10	Upper bound of connections (idle plus active).
`minimumIdle`	same as maximumPoolSize	Idle connections kept ready; the documentation recommends leaving it unset for a fixed-size pool.
`connectionTimeout`	30000 ms	Maximum wait for a connection from the pool before an exception is thrown.
`idleTimeout`	600000 ms	Idle time before a connection above minimumIdle is retired.
`maxLifetime`	1800000 ms	Maximum age of a connection; should be several seconds shorter than any database or network limit.
`keepaliveTime`	120000 ms	Interval of liveness probes on idle connections (0 disables).
`leakDetectionThreshold`	0 (off)	Logs a warning if a connection is held longer than this many milliseconds.

HikariCP settings that matter in production.

In Spring Boot these are set with the prefix `spring.datasource.hikari`:

application.properties PROPERTIES

```
spring.datasource.url=jdbc:postgresql://db.internal:5432/shop
spring.datasource.username=shop_app
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-timeout=5000
spring.datasource.hikari.max-lifetime=1740000
spring.datasource.hikari.leak-detection-threshold=20000
```

A connection timeout of 5 000 ms instead of 30 000 ms makes an exhausted pool fail fast and visibly rather than piling up request threads.

Sizing the pool

The widely cited formula from the PostgreSQL and HikariCP guidance is a starting point, not a result:

pool sizing starting point

```
connections = (core_count * 2) + effective_spindle_count
```

For a database server with 4 cores and one SSD this gives about 9 connections for the whole database, shared by every application instance. The formula is derived from the observation in the measurement above: the number of useful concurrent sessions is bounded by cores plus the time sessions spend waiting for I/O. The practical procedure is:

1. Determine the database budget: `max_connections` minus the reserved slots minus the connections of administrative tools, replicas and migration jobs.
2. Divide the budget by the number of application instances at their maximum scale (including the extra instances present during a rolling deployment).

3. Set `maximumPoolSize` to that figure, and verify under load that the pool wait time (Hikari metric `hikaricp.connections.pending`) stays near zero.

With 20 application pods and a database budget of 80 connections, each pod receives 4 connections. A pool of 10 per pod would request 200 connections and exceed the limit, which is the mechanism behind the connection storm described later in this chapter.

An external pooler: PgBouncer

When many instances or many short-lived processes (serverless functions, batch jobs) need access, an external pooler such as PgBouncer multiplexes many client connections onto few server connections. In transaction pooling mode a server connection is assigned to a client only for the duration of one transaction, which allows thousands of clients to share a few dozen backends. The price is that session state does not survive between transactions: `SET` commands without `LOCAL`, session-level advisory locks, `LISTEN` and, in older versions, server-side prepared statements cannot be relied upon. Session pooling mode keeps the full semantics but multiplexes less.

1.26.2 Worked example: JDBC from first principles

JDBC is the standard Java API for relational databases. The sequence is always the same: obtain a `Connection`, create a `PreparedStatement`, bind parameters, execute, read a `ResultSet`, close everything (try-with-resources closes in reverse order). The following class is a complete service method in plain JDBC. It requires the PostgreSQL JDBC driver on the class path, which is not available on the build machine; the listing is therefore not run here, and the SQL it sends was executed through `psql` in the sections that follow.

CustomerDao.java JAVA

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Optional;

public class CustomerDao {
    private final String url;
    private final String user;
    private final String password;

    public CustomerDao(String url, String user, String password) {
        this.url = url;
        this.user = user;
        this.password = password;
    }

    public Optional<String> findNameByEmail(String email) throws SQLException {
        String sql = "SELECT name FROM customers WHERE email = ?";
        try (Connection con = DriverManager.getConnection(url, user, password);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, email);               // bound as data, never parsed as SQL
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(rs.getString("name")) : Optional.empty();
            }
        }
    }

    public static void main(String[] args) throws Exception {
        CustomerDao dao = new CustomerDao(
            "jdbc:postgresql://localhost:5432/shop", "shop_app", "secret");
        System.out.println(dao.findNameByEmail("c5@example.com").orElse("not found"));
    }
}
```

What to notice: `DriverManager.getConnection` opens a new physical connection on every call, which is exactly the per-request cost measured above. In a Spring application the `DataSource` (HikariCP) replaces `DriverManager`, and `con.close()` returns the connection to the pool instead of closing the socket. The parameter marker `?` is the essential line: the statement text and the values travel separately.

1.26.3 Hands-on: SQL injection and the fix

The following demonstration uses real PostgreSQL 16 on the build machine. A login check is written by concatenating strings. The Java program below builds the three statements that the vulnerable code would send for three different inputs; it was run with the source launcher.

Inject.java JAVA

```java
public class Inject {
    static String unsafe(String email, String password) {
        return "SELECT id, name, is_admin FROM customers WHERE email = '" + email
             + "' AND password = '" + password + "'";
    }
    public static void main(String[] args) {
        System.out.println(unsafe("c5@example.com", "pw5"));
        System.out.println(unsafe("c5@example.com", "x' OR '1'='1"));
        System.out.println(unsafe("c5@example.com'; DROP TABLE orders; --", "x"));
    }
}
```

Output of the program (the three statements) TERMINAL

```
$ java Inject.java
SELECT id, name, is_admin FROM customers WHERE email = 'c5@example.com' AND password = 'pw5'
SELECT id, name, is_admin FROM customers WHERE email = 'c5@example.com'
  AND password = 'x' OR '1'='1'
SELECT id, name, is_admin FROM customers WHERE email = 'c5@example.com'; DROP TABLE orders;
  --' AND password = 'x'
```

The second statement changes the logic: `AND` binds tighter than `OR`, so the condition becomes `(email = ... AND password = 'x') OR '1'='1'`, which is true for every row. Executing it in `psql` returns the whole table, administrator first:

Executing the second statement TERMINAL

```
shop=# SELECT id, name, is_admin FROM customers WHERE email = 'c5@example.com'
shop-#   AND password = 'x' OR '1'='1';
  id  |     name      | is_admin
------+---------------+----------
    1 | Customer 1    | t
    2 | Customer 2    | f
    3 | Customer 3    | f
    4 | Customer 4    | f
    5 | Customer 5    | f
    6 | Customer 6    | f
```

The third statement terminates the first query and appends a second one. The simple query protocol accepts several statements in one message, and the demonstration table was actually destroyed:

Executing the third statement (real output) TERMINAL

```
$ sed -n 3p out.txt | psql shop -f -
 id |    name    | is_admin
----+------------+----------
  5 | Customer 5 | f
(1 row)

DROP TABLE
shop=# select count(*) from orders;
ERROR:  relation "orders" does not exist
```

The fix is a parameterised statement. The server receives the statement text with placeholders first (Parse), then the values (Bind). The values are never part of the parsed SQL, so no quote character can change the structure. The same behaviour can be demonstrated in `psql` with `PREPARE`:

Parameterised statement with the malicious value (real output) TERMINAL

```
shop=# PREPARE login(text, text) AS
shop-#   SELECT id, name, is_admin FROM customers WHERE email = $1 AND password = $2;
PREPARE
shop=# EXECUTE login('c5@example.com', 'x'' OR ''1''=''1');
 id | name | is_admin
----+------+----------
(0 rows)
```

Parameterised statement with the malicious value (real output) (continued) TERMINAL

```
shop=# EXECUTE login('c5@example.com', 'pw5');
 id |    name    | is_admin
----+------------+----------
  5 | Customer 5 | f
(1 row)
```

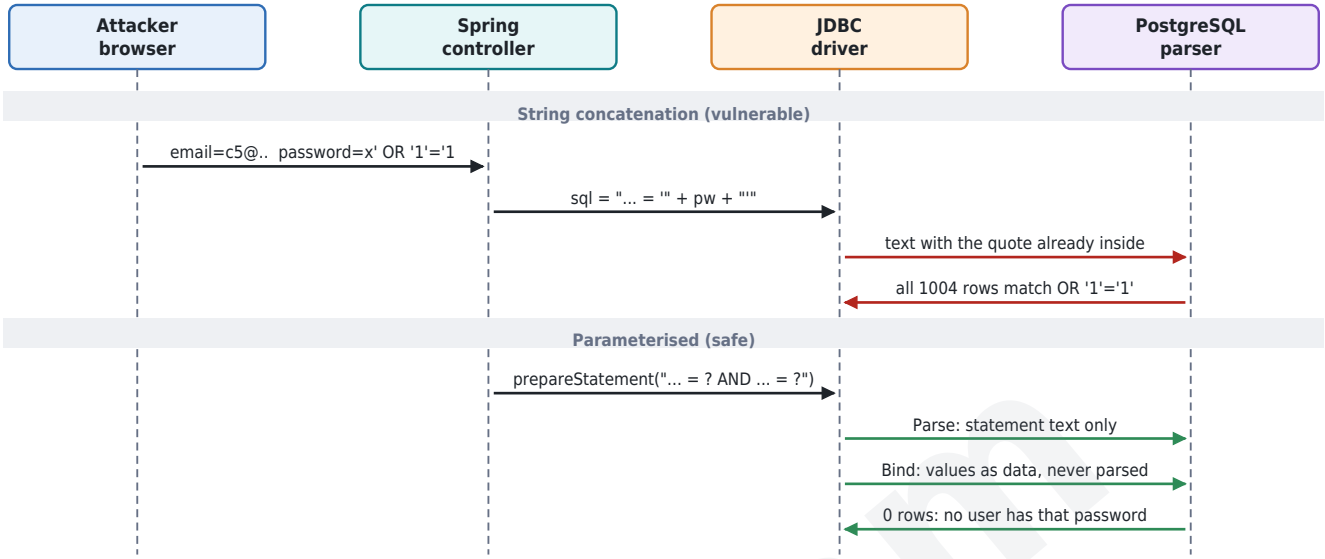


Figure 1.126. With concatenation the quote characters reach the parser as SQL syntax; with Parse and Bind the attacker-controlled text is only ever a value.

WATCH OUT

Parameters protect values, not identifiers. A table name, column name or `ORDER BY` direction cannot be a `?` parameter. For a dynamic sort column, map the user input to a fixed allow-list (`switch` on known names) and append the mapped literal; never append the raw input. Spring Data JPA derived queries and `Sort` objects validate property names against the entity, which is why they are safe for this case, whereas a native query concatenating `ORDER BY` + `sortParam` is not.

Prepared statements and plan caching

A prepared statement is parsed once and can be executed many times. The PostgreSQL JDBC driver switches to a server-side prepared statement after `prepareThreshold` executions (default 5). The server then keeps a named statement per connection; for the first five executions of a statement it plans with the actual parameter values (custom plans), after which PostgreSQL may switch to a cached generic plan if it is not estimated to be worse. Prepared statements are per connection, which is why PgBouncer transaction pooling historically needed care with them.

Batch inserts

Each statement sent in autocommit mode is its own transaction and costs a WAL flush and a network round trip. The measurement below loaded 20 000 rows into a table with a serial key using four methods, all against a local socket; on a real network the round-trip share is larger, so the relative gap grows.

Load of 20 000 rows (measured on the build machine, seconds) TERMINAL

```
single INSERT per statement, autocommit:        28.6 s
single INSERT per statement, one transaction:    0.91 s
20 multi-row INSERTs of 1000 rows, one txn:      0.126 s
COPY from a CSV file:                            0.082 s
```

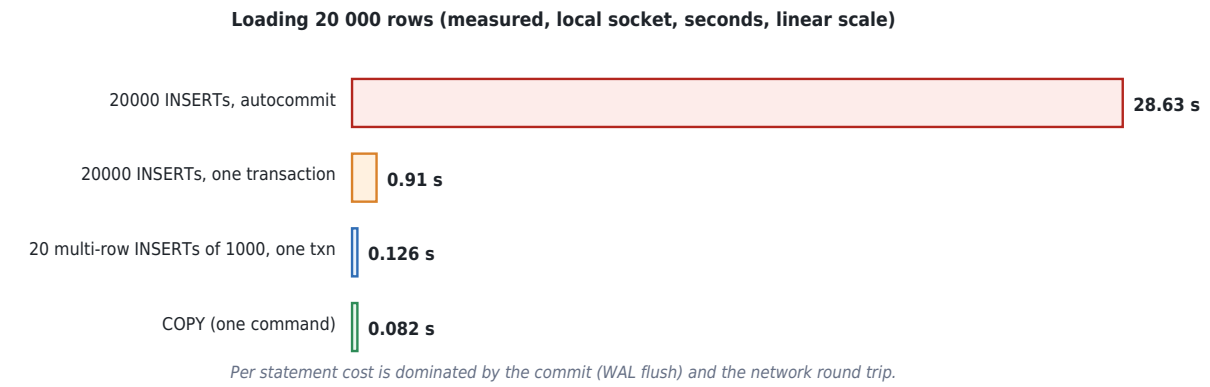


Figure 1.127. The commit per statement dominates the autocommit case; grouping statements into one transaction and then into multi-row statements removes almost all overhead.

In JDBC the corresponding technique is `addBatch()` and `executeBatch()` inside one transaction. The PostgreSQL driver can rewrite a batch of inserts into a multi-row statement when the property `reWriteBatchedInserts=true` is set on the URL. In Spring Data JPA, `saveAll` with `spring.jpa.properties.hibernate.jdbc.batch_size=50` and `spring.jpa.properties.hibernate.order_inserts=true` enables Hibernate batching; note that a `GenerationType.IDENTITY` key disables insert batching in Hibernate, whereas `SEQUENCE` with an allocation size permits it.

BatchInsert.java JAVA

```java
// Fragment: requires the PostgreSQL JDBC driver (not run on the build machine).
String sql = "INSERT INTO batch_t(a, b) VALUES (?, ?)";
con.setAutoCommit(false);
try (PreparedStatement ps = con.prepareStatement(sql)) {
    for (int i = 0; i < 20_000; i++) {
        ps.setInt(1, i);
        ps.setString(2, "row " + i);
        ps.addBatch();
        if ((i + 1) % 1000 == 0) {
            ps.executeBatch();
        }
    }
    ps.executeBatch();
    con.commit();
} catch (SQLException e) {
    con.rollback();
    throw e;
}
```

1.26.4 Real-world problems and solutions

Incident: FATAL too many connections after scaling out

Scenario. An order service runs 6 pods with the Spring Boot default pool of 10. Traffic grows and the platform autoscaler adds pods. The database allows 100 connections, of which administrative tooling and a reporting job use 15.

Symptoms. Intermittent errors in the application log: `FATAL: sorry, too many clients already` or `remaining connection slots are reserved for roles with the SUPERUSER attribute`. New pods fail their readiness probe. Existing requests succeed, so the outage looks partial.

Diagnosis. Count sessions by state and by application, from a superuser connection (the reserved slots make this possible):

diagnosis.sql SQL

```sql
SELECT state, count(*) FROM pg_stat_activity
WHERE backend_type = 'client backend' GROUP BY state ORDER BY count(*) DESC;

SELECT application_name, usename, count(*) FROM pg_stat_activity
WHERE backend_type = 'client backend' GROUP BY 1, 2 ORDER BY 3 DESC;

SHOW max_connections;
```

sreeram

Typical output during the incident (shape of the data) TERMINAL

```
 state              | count
--------------------+-------
 idle               |    71
 idle in transaction |    12
 active             |     4
```

Most sessions are `idle`: pools hold connections that no request is using. The few `idle in transaction` sessions point to code that begins a transaction and then waits (a remote call inside `@Transactional`).

Root cause. Total demand is `pods * maximumPoolSize` = 10 pods * 10 = 100, plus 15 external users, which exceeds `max_connections`. The sessions are mostly idle, so the pool size is far larger than the concurrency the database can use (see the throughput measurement above).

Fix. Immediate: lower `maximumPoolSize` to 4 on all pods (a configuration change and rolling restart), terminate idle sessions of the reporting job. Permanent: compute the per-pod pool from the budget (the procedure above), add PgBouncer in transaction mode if the number of pods must grow, and set `idle_in_transaction_session_timeout` so that sessions stuck inside a transaction are released.

Prevention. Alert at 80 percent of `max_connections`; load-test with the maximum pod count; include the rolling-update surge in the budget.

Incident: connection storm after a deployment

Scenario. A deployment restarts all pods at once. Each new pod eagerly opens its full pool and warms caches by running queries; old pods are still draining.

Symptoms. For about 30 seconds after the rollout starts the database refuses connections, latency spikes on queries that were healthy, and the application reports pool timeouts.

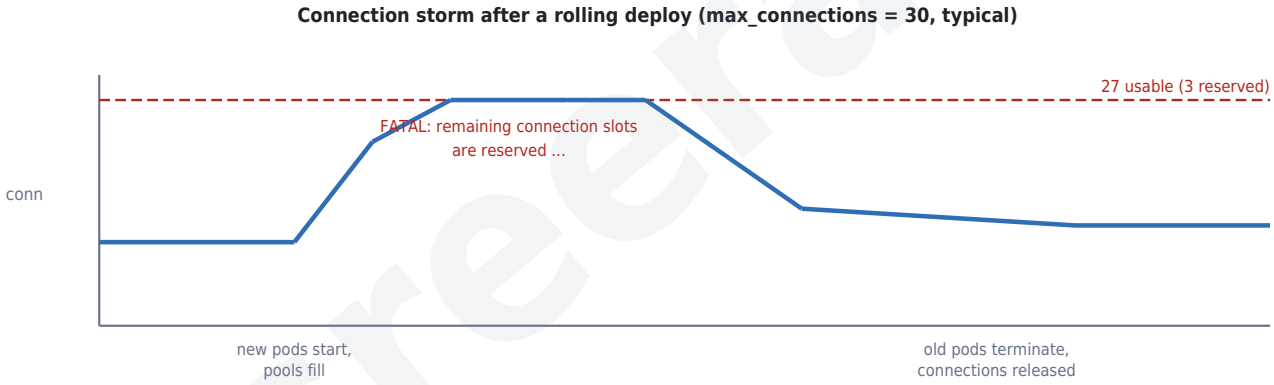


Figure 1.128. New pods fill their pools before old pods release theirs, so the total number of sessions crosses the usable limit during the overlap.

Diagnosis. `pg_stat_activity` grouped by `backend_start` shows a burst of sessions created in the same seconds; `log_connections` confirms the rate. Compare the number of pods during the rollout with the connection budget.

Root cause. The rolling update strategy allowed `maxSurge` extra pods and each pod opens `minimumIdle` (equal to `maximumPoolSize` by default) connections at start. During the overlap there are more pods than the budget assumed. Authentication of many simultaneous sessions also consumes CPU.

Fix. Immediate: pause the rollout, scale old pods down only after new pods are ready. Permanent: lower the pool size, set `maxSurge` to a small number, start the pool lazily (`initializationFailTimeout` and a small `minimumIdle`), and place PgBouncer between the services and the database so the server sees a stable number of connections.

Prevention. Rehearse deployments in a staging environment that has the real connection limit.

Incident: SQL injection through a search endpoint

Scenario. A product search builds `"SELECT * FROM products WHERE name LIKE '%" + q + "%'"` in a repository method using `JdbcTemplate.query(String)`.

Symptoms. The access log shows requests with `q=' UNION SELECT email, password, null FROM customers --`; the response contains customer emails. Alternatively errors such as `syntax error at or near "'"` appear for ordinary input containing an apostrophe (for example the name O'Brien), which is the benign symptom of the same defect.

Diagnosis. Search the code for string concatenation into SQL: `grep -rn "\" *+" src | grep -i "select\|update\|delete"`. Search the database log (`log_statement = 'all'` temporarily, or `log_min_error_statement`) for statements containing `UNION` or `--` that no code path should generate.

Root cause. User input reaches the SQL text. The application account was also over-privileged: it could read the `customers` table although the search feature needs only `products`.

Fix. Immediate: take the endpoint offline or block the pattern at the gateway, rotate credentials that may have been exposed. Permanent: use `?` placeholders (`jdbcTemplate.query(sql, mapper, "%" + q + "%")`), or a Spring Data derived query `findByNameContainingIgnoreCase`; restrict the role with `GRANT SELECT ON products` only (see the chapter on security).

Prevention. Static analysis (SpotBugs with Find Security Bugs, or CodeQL) in CI, code review rule against concatenated SQL, least-privilege roles so that an injection has little to read.

1.26.5 Questions to expect

What does a database connection cost and why do we pool?

Each connection is a server process (PostgreSQL) or thread (MySQL) with private memory and, on creation, a TCP handshake, optional TLS and authentication. Measured here, reconnecting for every transaction reduced throughput from about 47 000 to 233 transactions per second. A pool amortises the cost by reusing established sessions and bounds the number of sessions the database must serve.

How large should a HikariCP pool be?

Small: start near `cores * 2 + spindles` for the whole database and divide it among application instances. Throughput stops improving at a small multiple of the core count; beyond that, requests should queue in the application where waiting is cheap. Verify with pool wait metrics.

Why is a PreparedStatement safe against SQL injection?

The statement text is parsed and planned before the values arrive; values are bound as typed data. A quote inside a value cannot end a string literal because there is no literal to end. This protects values only; identifiers must be validated against an allow-list.

What is the difference between `connectionTimeout`, `maxLifetime` and `idleTimeout`?

`connectionTimeout` is how long a caller waits for a connection from the pool. `maxLifetime` retires a connection after a maximum age (to avoid stale connections closed by firewalls or the database). `idleTimeout` retires idle connections above `minimumIdle`.

Why can `GenerationType.IDENTITY` hurt batch inserts?

Hibernate must read the generated key immediately after each insert, so it cannot defer and group the statements. A sequence with an allocation size lets Hibernate assign keys in memory and batch the inserts.

What does transaction pooling in PgBouncer break?

Anything that relies on session state spanning transactions: session-level `SET`, session advisory locks, temporary tables and `LISTEN`.

KEY POINTS

- A connection is an expensive server-side session; reconnecting per transaction cost a factor of about 200 in the measurement.
- Pools should be small; the database connection budget is divided among all instances, including deployment surge.
- Parameterised statements separate code from data and are the only reliable defence against SQL injection; identifiers need allow-lists.
- Group statements: one transaction instead of autocommit, then batches or `COPY`, cut a 20 000-row load from 28.6 s to under 1 s.
- `idle in transaction` sessions and leaked connections are the usual cause of pool exhaustion.

sreram

1.27 Replication and High Availability

A single database server is a single point of failure and a single point of capacity. Replication keeps additional copies of the data on other servers. The copies serve three purposes: survival of a server loss (high availability), additional read capacity (read replicas) and a source for backups or analytics. Replication also introduces the most common source of subtle production defects in database-backed applications: a copy that is slightly behind the original.

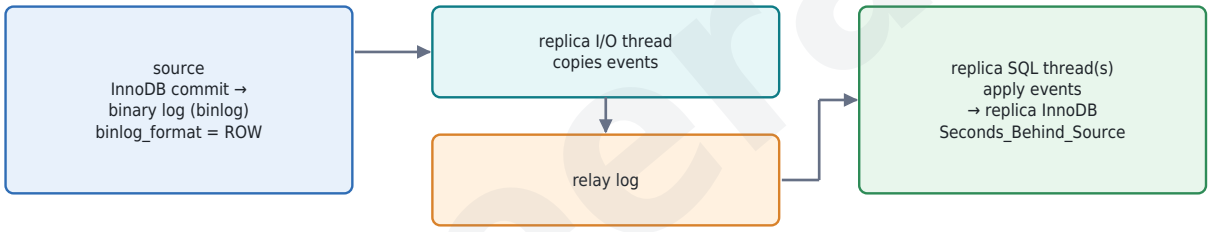
1.27.1 Concept and mechanism

Physical and logical replication

PostgreSQL streaming replication is physical: the standby receives the write-ahead log (WAL) of the primary, the byte-level record of every change to every page, and replays it. The standby is therefore an exact copy of the whole cluster, same major version, and it cannot accept writes while it follows. Logical replication decodes the WAL into row changes (INSERT, UPDATE, DELETE) per table and publishes them to subscribers; the subscriber is an independent, writable database that may have a different schema, version or additional tables. Logical replication is the tool for selective replication, zero-downtime major upgrades and change data capture; physical replication is the tool for high availability.

MySQL replication is logical in the sense that it ships the binary log (binlog) of committed transactions, in row, statement or mixed format, and the replica re-executes them through an applier thread. MySQL deployments use GTIDs (global transaction identifiers) to identify each transaction, which makes it possible to repoint a replica to a new source without computing file positions.

MySQL replication: binary log shipped to a relay log (typical, not run)



GTID: each transaction has an identifier, so failover can find the resume point

Figure 1.129. The MySQL replica copies binlog events into a relay log and applies them with separate threads; the delay between commit on the source and apply on the replica is reported as Seconds_Behind_Source (typical behaviour, not run on the build machine).

The streaming protocol in PostgreSQL

A commit writes WAL records to the WAL buffers and then flushes them to the local `pg_wal` directory. A walsender process on the primary reads WAL and sends it over a TCP connection to the walreceiver on the standby. The standby writes the received WAL to its own disk (`write_lsn`, `flush_lsn`) and the startup process replays it into the data files (`replay_lsn`). Queries on a hot standby see the state as of the last replayed position.

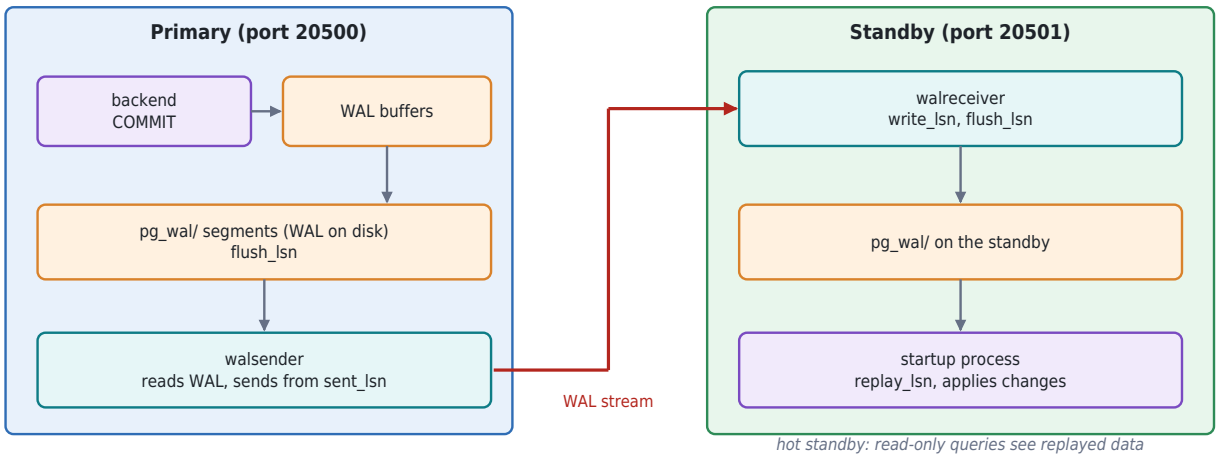


Figure 1.130. Three positions advance on the way to the standby: sent, flushed and replayed; the differences between them are the replication lag.

Synchronous and asynchronous replication

By default replication is asynchronous: the primary acknowledges a commit after its own WAL flush and streams to the standby afterwards. If the primary is lost, the most recent transactions may not have reached the standby and are lost on failover. Setting `synchronous_standby_names` makes the commit wait until the named standby has confirmed the WAL (according to `synchronous_commit`; the value `on` waits for the standby flush, `remote_apply` waits for replay). Synchronous replication gives a recovery point objective of zero for committed transactions at the price of commit latency, and it turns a standby outage into a primary outage unless several candidates are listed.

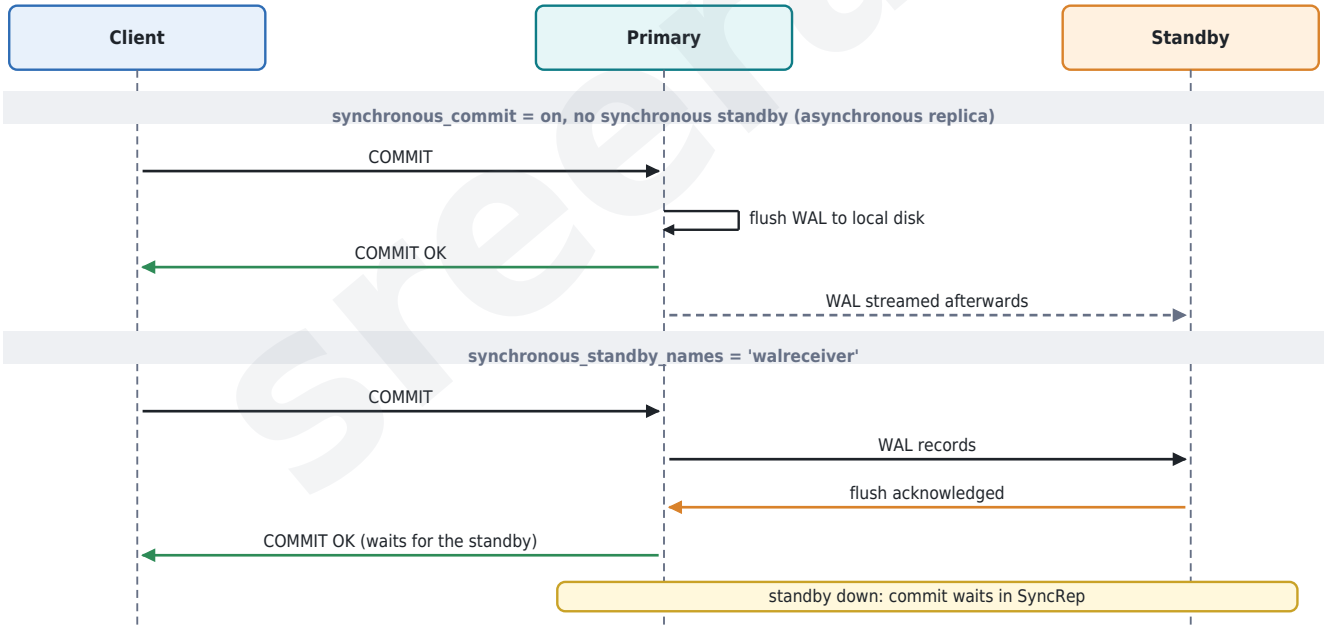


Figure 1.131. In the asynchronous case the acknowledgement does not depend on the standby; in the synchronous case the acknowledgement is delayed by one network round trip and one remote flush.

Replication lag and read-your-writes

Replication lag is the time or the amount of WAL between the primary's latest position and the position a replica has replayed. When an application routes reads to a replica, it may read a state older than its own last write. This violates read-your-writes consistency: a user updates a profile, is redirected, and sees the old profile. Approaches are: read from the primary for a short window after a write (a session flag), wait on the replica for a given LSN before reading (`pg_last_wal_replay_lsn()` compared with the LSN returned by `pg_current_wal_lsn()` at commit), use synchronous `remote_apply` for the few operations that need it, or accept staleness for views where it is harmless (catalogue pages, dashboards).

1.27.2 Worked example: streaming replication on one machine

Two PostgreSQL 16 clusters were created: the primary on port 20500 with `wal_level = replica`, and a standby cloned from it with `pg_basebackup`. The `-R` option writes `standby.signal` and the connection settings into `postgresql.auto.conf`; `-C -S` creates a physical replication slot named `replica1_slot`; `-X stream` includes the WAL needed to make the copy consistent; `-c fast` requests an immediate checkpoint (without it the base backup can wait for a spread checkpoint).

Creating the standby (real output, abridged) TERMINAL

```
$ pg_basebackup -h /tmp/osw_d5 -p 20500 -D /tmp/osw_d5/replica \
    -R -X stream -C -S replica1_slot -c fast -P -v
49568/49568 kB (100%), 1/1 tablespace
pg_basebackup: write-ahead log end point: 0/80001B0
pg_basebackup: syncing data to disk ...
pg_basebackup: base backup completed
$ cat replica/postgresql.auto.conf
primary_conninfo = 'user=postgres ... host=''/tmp/osw_d5'' port=20500 ...'
primary_slot_name = 'replica1_slot'
$ pg_ctl -D replica -l replica.log -w start
server started
```

On the standby the server reports recovery mode and refuses writes:

Standby behaviour (real output) TERMINAL

```
$ psql -p 20501 shop -c "select pg_is_in_recovery()"
 pg_is_in_recovery
-------------------
 t
$ psql -p 20501 shop -c "insert into customers(id,name) values (9999,'x')"
ERROR:  cannot execute INSERT in a read-only transaction
```

The primary describes the connected standby in `pg_stat_replication`:

pg_stat_replication on the primary (real output) TERMINAL

```
shop=# select pid, state, sent_lsn, write_lsn, flush_lsn, replay_lsn, sync_state,
shop-#   write_lag, flush_lag, replay_lag from pg_stat_replication \gx
-[ RECORD 1 ]---------------
pid        | 4432
state      | streaming
sent_lsn   | 0/9000060
write_lsn  | 0/9000060
flush_lsn  | 0/9000060
replay_lsn | 0/9000060
sync_state | async
write_lag  | 00:00:00.034336
flush_lag  | 00:00:00.034888
replay_lag | 00:00:00.034963
```

1.27.3 Hands-on: lag, synchronous commit and promotion

Creating measurable lag

Replay on the standby was paused with `pg_wal_replay_pause()`, a row was inserted on the primary, and the standby was queried. The WAL had arrived (received LSN advanced) but had not been replayed.

Stale read on the replica while replay is paused (real output) TERMINAL

```
standby=# select pg_wal_replay_pause();
primary=# insert into customers(id,name,email)
primary-#   values (5001,'Late Writer','late@example.com') returning id;
5001
standby=# select id,name from customers where id=5001;
 id | name
----+------
(0 rows)
primary=# select state, sent_lsn, replay_lsn,
primary-#   pg_wal_lsn_diff(sent_lsn, replay_lsn) as replay_gap_bytes from pg_stat_replication;
state           | streaming
sent_lsn        | 0/9002C10
replay_lsn      | 0/9000060
replay_gap_bytes | 11184
```

Stale read on the replica while replay is paused (real output) (continued) TERMINAL

```
standby=# select pg_last_wal_receive_lsn() as received,
standby-#   pg_last_wal_replay_lsn() as replayed;
 received  | replayed
-----------+-----------
 0/9002C10 | 0/9000060
standby=# select pg_wal_replay_resume();
standby=# select id,name from customers where id=5001;
  id  |    name
------+-------------
 5001 | Late Writer
```

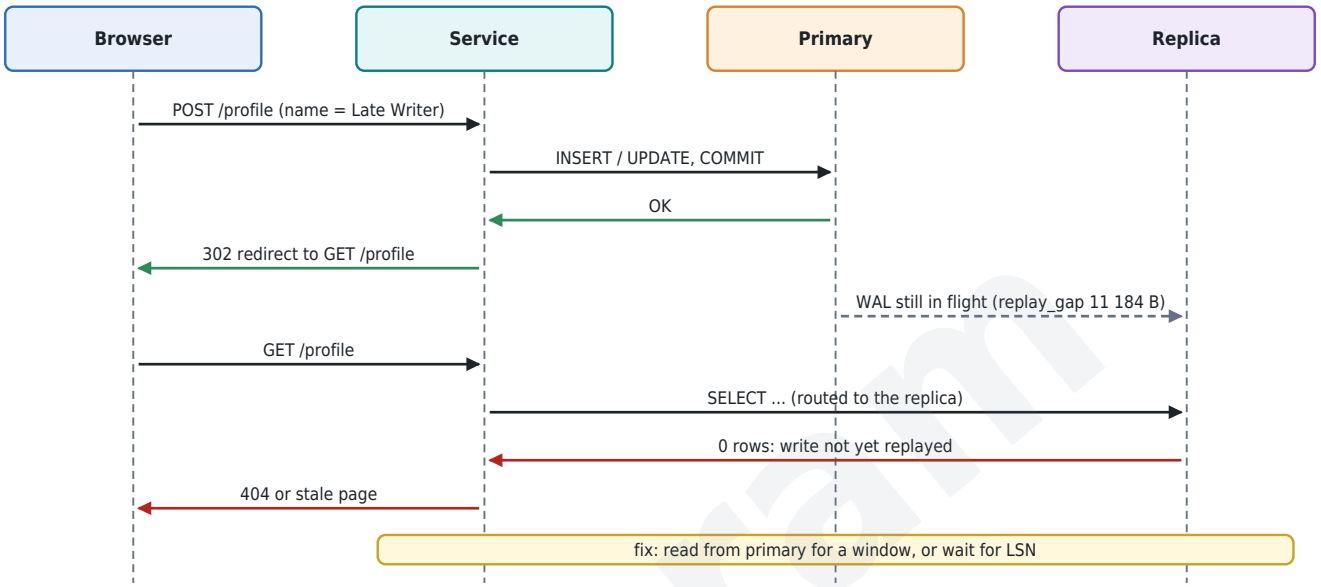


Figure 1.132. The redirect after a write reaches the replica before the write has been replayed there, so the second request sees no row.

Spring routing to replicas is usually implemented with a `AbstractRoutingDataSource` that selects a pool by the `readOnly` flag of the current transaction (`@Transactional(readOnly = true)` goes to the replica). The mechanism is sound, but it makes every read-only transaction subject to the lag described above; an operation that must see its own write should not be marked `readOnly`.

Synchronous commit

The standby registers with the default `application_name` `walreceiver`. After `ALTER SYSTEM SET synchronous_standby_names = 'walreceiver'` and a reload, the standby is reported as synchronous and commits wait for it. With the standby up the commit took 0.17 s through `psql`; with the standby stopped the commit never returned.

Commit waits when the synchronous standby is down (real output) TERMINAL

```
$ psql shop -c "select application_name, state, sync_state from pg_stat_replication"
 application_name |   state   | sync_state
------------------+-----------+------------
 walreceiver      | streaming | sync
$ pg_ctl -D replica -m fast -w stop
server stopped
$ timeout 5 psql shop -c "insert into customers(id,name) values (5003,'sync blocked')"
$ echo $?
124
primary=# select pid, state, wait_event_type, wait_event from pg_stat_activity
primary-#   where wait_event = 'SyncRep';
 pid  | state  | wait_event_type | wait_event
------+--------+-----------------+------------
 4510 | active | IPC             | SyncRep
```

The transaction is already durable on the primary (the row became visible locally) but the client has no acknowledgement. Cancelling the wait yields a warning that the transaction was committed locally but may not be replicated. The setting was reset with `ALTER SYSTEM RESET synchronous_standby_names`.

GOTCHA
Synchronous replication with a single standby couples availability. If the only synchronous standby fails, every commit blocks. Production configurations list at least two candidates, for example `ANY 1 (standby_a, standby_b)`, so that one failure does not stop writes.

Promotion and split brain

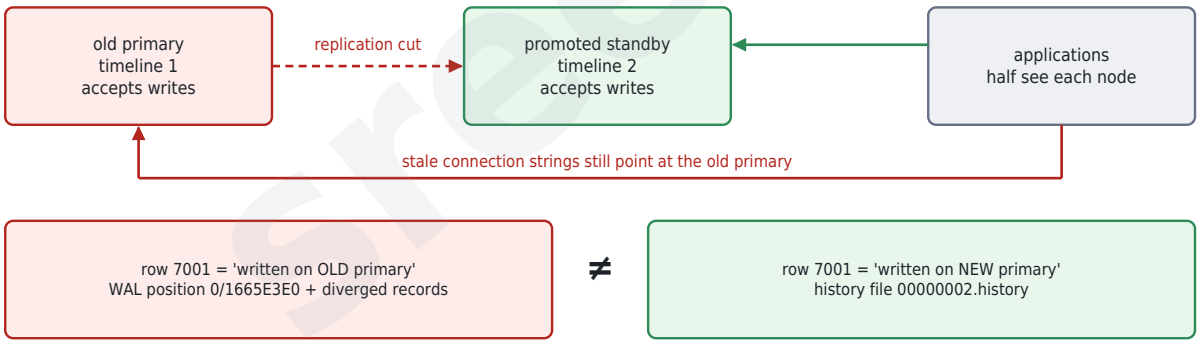
Failover replaces a failed primary with a standby. `pg_ctl promote` (or `SELECT pg_promote()`) ends recovery and starts a new timeline. On the build machine promotion took 0.23 s. Because the old primary was still running and reachable, both servers then accepted writes for the same key:

Two primaries (real output) TERMINAL

```
$ pg_ctl -D replica -w promote
server promoted
$ psql -p 20501 shop -c "select pg_is_in_recovery()"
 pg_is_in_recovery
-------------------
 f
$ psql -p 20501 shop -c "insert into customers(id,name) values (7001,'written on NEW primary')"
INSERT 0 1
$ psql -p 20500 shop -c "insert into customers(id,name) values (7001,'written on OLD primary')"
INSERT 0 1
$ psql -p 20501 shop -tA -c "select pg_walfile_name(pg_current_wal_lsn())"
000000020000000000000016
$ psql -p 20500 shop -tA -c "select pg_walfile_name(pg_current_wal_lsn())"
000000010000000000000016
$ cat replica/pg_wal/00000002.history
1       0/1665E3E0      no recovery target specified
```

The first eight hex digits of the WAL file name are the timeline: the promoted node is on timeline 2, the old primary is still on timeline 1. The history file records the point where timeline 2 branched. Both servers now contain a row with key 7001 and different content: this is split brain. Nothing in the databases themselves prevents it.

Split brain after promoting a standby while the old primary is still reachable



Defences: fencing (STONITH, revoke network or storage), a consensus-based manager (Patroni with etcd, repmgr witness), and rebuilding the old primary with pg_rewind or a new base backup.

Figure 1.133. The promoted standby and the unfenced old primary both accept writes; clients with stale connection settings diverge the data.

Prevention is external to the database: a failover manager (Patroni with a consensus store such as etcd, repmgr with a witness) that promotes only with a quorum, and fencing of the old primary (power or network isolation, or demoting it before routing traffic) before the new primary is announced. The old primary must be rebuilt as a standby, with `pg_rewind` (requires `wal_log_hints = on` or data checksums) or with a new base backup.

1.27.4 Real-world problems and solutions

Incident: stale reads after a write

Scenario. A Spring Boot service routes `@Transactional(readOnly = true)` methods to a replica. After checkout, the order confirmation page calls `getOrder(id)` and returns 404 for about one request in two hundred.

Symptoms. The order exists when queried a few seconds later. Support tickets say "my order disappeared". The failures cluster around batch jobs that write heavily.

Diagnosis. On the primary, `SELECT application_name, replay_lag, pg_wal_lsn_diff(sent_lsn, replay_lsn) AS gap_bytes FROM pg_stat_replication;` shows replay lag growing during the batch. On the replica, `SELECT now() - pg_last_xact_replay_timestamp();` gives the age of the replayed data (it is also non-zero on an idle primary, so interpret it together with the first query). Compare with the failing request time.

Root cause. Asynchronous replication plus read routing: the read after the write ran on a replica that had not replayed the commit. Replay also lags when a long query on the replica conflicts with WAL replay (`max_standby_streaming_delay`).

Fix. Immediate: route the confirmation read to the primary. Permanent: read-your-writes policy (primary for N seconds after a write by the same session, or LSN wait), alerts on `replay_lag` above a threshold, and keep long analytical queries on a dedicated replica with a larger delay allowance or `hot_standby_feedback` tuned deliberately.

Prevention. Mark only genuinely lag-tolerant reads as read-only and replica-routed; test with an artificially delayed replica (`recovery_min_apply_delay` on a test standby).

Incident: failed failover and split brain

Scenario. A network partition isolates the primary from the failover manager. The manager promotes a standby; the old primary keeps running and the application tier still holds connections to it.

Symptoms. Customer reports show different data depending on the request; unique-key violations appear when replication to a rebuilt standby is attempted; the old primary's replication connections have disappeared from `pg_stat_replication`.

Diagnosis. On each node run `SELECT pg_is_in_recovery();` and `SELECT pg_walfile_name(pg_current_wal_lsn());`. Two nodes answering `f` with different timeline numbers confirm split brain. Compare the divergent rows (as in the demonstration above).

Root cause. The old primary was not fenced before promotion, and clients were not forced to the new primary.

Fix. Immediate: stop the old primary (or block its client traffic), choose the authoritative node, export the writes made only on the other node (the difference of the two datasets) for manual reconciliation. Permanent: rebuild the loser as a standby from the winner, and run a failover manager with quorum-based leadership and fencing.

Prevention. Failover drills; a single connection endpoint (virtual IP, DNS with a short TTL or a proxy) that is switched as part of the fenced promotion.

Incident: disk full because of WAL growth from an inactive replication slot

Scenario. A standby was shut down for maintenance and never returned. Its replication slot remained on the primary. The `pg_wal` directory grows until the volume fills.

Symptoms. Monitoring shows the data volume filling at the rate of write activity; eventually the primary logs `PANIC: could not write to file "pg_wal/xlogtemp..."` and stops. Nothing is wrong with the tables themselves.

Sreeram

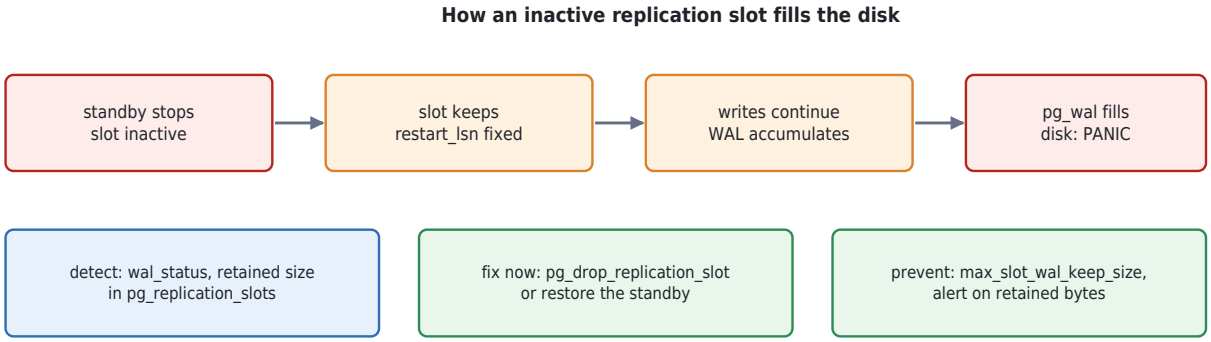


Figure 1.134. A slot is a promise to retain WAL for its consumer; when the consumer is gone the promise has no limit.

Diagnosis. The slot view shows an inactive slot with a large retained amount (real output from the build machine, after a bulk load and three full-table updates):

Slot state and WAL retained (real output) TERMINAL

```
primary=# select slot_name, active, wal_status,
primary-#   pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) as retained
primary-#   from pg_replication_slots;
   slot_name   | active | wal_status | retained
---------------+--------+------------+-----------
 replica1_slot | f      | reserved   | 211 MB
$ du -sh pgdata/pg_wal
241M    pgdata/pg_wal
```

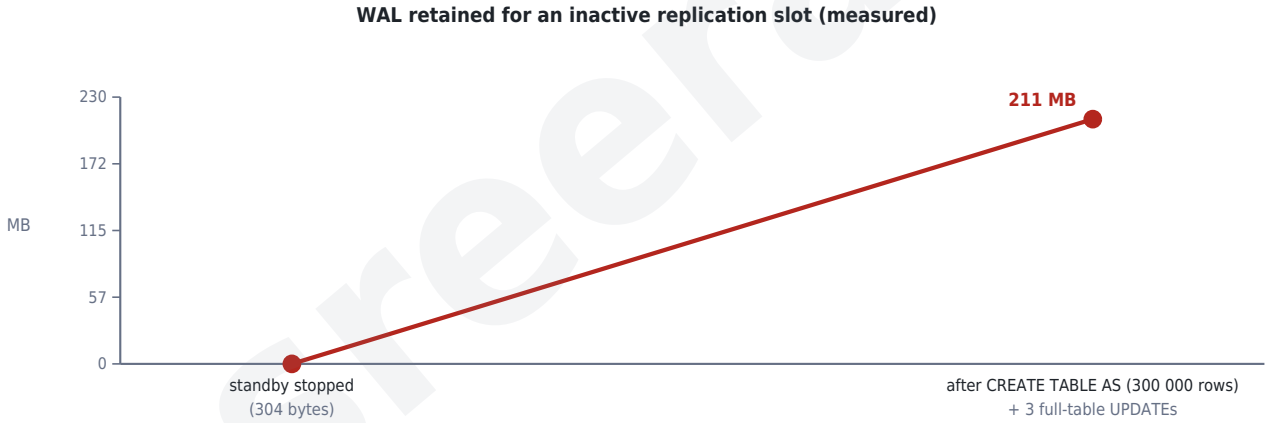


Figure 1.135. Retained WAL grew from 304 bytes to 211 MB while the standby was away, because the slot pins its restart position.

Root cause. A replication slot guarantees that the primary keeps all WAL the slot's consumer has not yet received. With the consumer gone, `restart_lsn` does not advance, and every change since then is retained.

Fix. Immediate: if the standby is not coming back, drop the slot with `SELECT pg_drop_replication_slot('replica1_slot');` and the next checkpoint recycles the WAL. If the standby must be kept, bring it back. Permanent: set `max_slot_wal_keep_size`, which bounds the retained WAL. With `max_slot_wal_keep_size = 64MB` the server invalidated the slot at the next checkpoint:

Bounded retention (real output) TERMINAL

```
LOG:  invalidating obsolete replication slot "replica1_slot"
DETAIL:  The slot's restart_lsn 0/9002D38 exceeds the limit by 150983368 bytes.
HINT:  You might need to increase max_slot_wal_keep_size.
   slot_name   | active | wal_status | restart_lsn | safe_wal_size
---------------+--------+------------+-------------+---------------
 replica1_slot | f      | lost       |             |
```

The standby, restarted afterwards, failed with `requested WAL segment 000000010000000000000009 has already been removed`. It caught up after a `restore_command` pointing to the WAL archive was added, which is the reason to keep an archive in addition to a slot (see the chapter on backups).

Prevention. Alert on `pg_replication_slots` where `active = false` and on retained bytes; alert at 70 percent disk usage of the WAL volume; set `max_slot_wal_keep_size` to a value smaller than the free space.

Incident: replica lag grows without bound

Scenario. A reporting query of two hours runs on the replica. The standby falls minutes behind and stays behind.

Symptoms. `replay_lag` rises; the replica logs `canceling statement due to conflict with recovery`, or, with `hot_standby_feedback = on`, the primary shows bloat.

Diagnosis. `SELECT * FROM pg_stat_database_conflicts;` on the replica, `pg_stat_replication` on the primary, and the settings `max_standby_streaming_delay` and `hot_standby_feedback`.

Root cause. WAL replay must remove row versions that the long query still needs. The standby either cancels the query after `max_standby_streaming_delay` (default 30 s) or delays replay. With feedback the primary retains the old versions instead.

Fix. Separate analytical replicas from failover replicas, tune the delay for the analytical one, or move the report to a materialised view refreshed on the primary.

Prevention. Monitor replay lag per replica and give reporting a replica of its own.

1.27.5 Questions to expect

Physical versus logical replication?

Physical replication ships WAL pages for the whole cluster; the standby is a byte-identical read-only copy of the same major version. Logical replication ships decoded row changes per table through publications and subscriptions; the subscriber is writable and may differ in version and schema. Use physical for high availability, logical for selective replication and upgrades.

What does asynchronous replication risk?

Loss of acknowledged transactions on failover: the primary acknowledges the commit before the standby has the WAL. Synchronous replication removes that loss at the cost of commit latency and of coupling availability to the standby.

What is split brain and how is it prevented?

Two nodes both believe they are the primary and accept writes. Prevented by quorum-based leader election, fencing the old primary before promotion, and a single controlled client endpoint.

How do you handle read-your-writes with replicas?

Route the reading request to the primary after a write for a time window, wait for the replica to replay the writer's LSN, or use synchronous `remote_apply` for critical flows. Avoid replica reads for data that the same user has just changed.

What does a replication slot do and why is it dangerous?

It makes the primary retain WAL until the consumer has received it. An abandoned slot retains WAL indefinitely and fills the disk; bound it with `max_slot_wal_keep_size` and alert on inactive slots.

What is a timeline?

A numbered branch of WAL history. Promotion starts a new timeline, recorded in a history file, so that WAL from the old and new branches cannot be confused.

KEY POINTS

- Physical replication copies WAL for high availability; logical replication copies row changes for selective use and upgrades.
- Asynchronous replication is fast but loses recent commits on failover; synchronous replication does not, but blocks commits if the standby disappears.
- Read replicas serve stale data by design; the application must define which reads tolerate lag.
- Promotion is quick (0.23 s here); safety comes from fencing and quorum, not from the promote command.
- An inactive replication slot retains WAL without limit; set `max_slot_wal_keep_size` and alert.

sreeram

1.28 Backup, Restore and Point-in-Time Recovery

Replication protects against the loss of a server. It does not protect against the loss of data: a `DELETE` without a `WHERE` clause, a faulty migration or a corrupting application bug is replicated to every standby within milliseconds. Protection against logical damage requires backups, and a backup is only worth the restore that has been tested. This chapter compares logical and physical backups, performs both on a real PostgreSQL 16 server, and recovers from an accidental mass delete to a chosen point in time.

1.28.1 Concept and mechanism

Logical and physical backups

A logical backup (`pg_dump`, `mysqldump`, `mysqlpump`) reads the data through SQL and writes the statements, or an archive of them, that recreate the objects and rows. It is portable across versions and operating systems, can restore a single table, and produces a consistent snapshot of one database (it runs in one `REPEATABLE READ` transaction). Restoring re-executes the inserts and rebuilds every index, so its duration grows with data volume. A physical backup copies the data files (`pg_basebackup`, Percona XtraBackup, filesystem or storage snapshots). It restores at the speed of file copy, includes all databases, and is tied to the same major version and architecture. Combined with archived WAL (PostgreSQL) or binary logs (MySQL) it supports point-in-time recovery.



Figure 1.136. The logical backup is smaller and portable but slow to restore; the physical backup is a copy of the cluster that can be rolled forward through archived WAL.

RPO and RTO

The recovery point objective (RPO) is the maximum acceptable amount of data loss, measured in time; the recovery time objective (RTO) is the maximum acceptable duration of the outage. A nightly dump gives an RPO of up to 24 hours. A base backup plus continuous WAL archiving gives an RPO of the archive delay (seconds, bounded by `archive_timeout` and segment switching). RTO is determined by restore speed and, for point-in-time recovery, the amount of WAL to replay. Both are properties of a rehearsed procedure, not of the configuration.

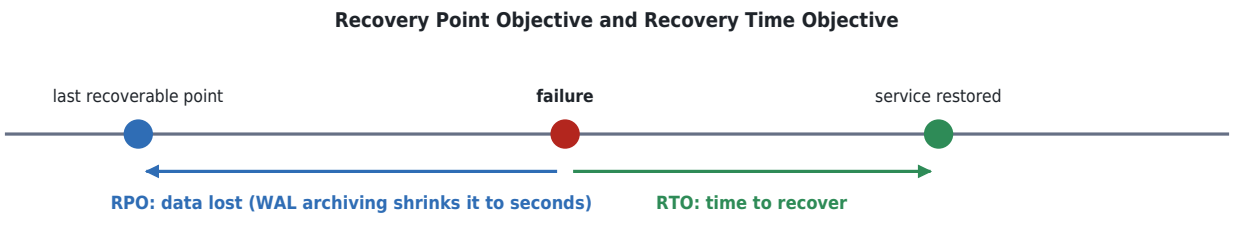


Figure 1.137. The interval between the last recoverable point and the failure is the data lost; the interval between the failure and the restored service is the downtime.

Point-in-time recovery in PostgreSQL

PostgreSQL archives WAL segments with `archive_command` (any command that copies `%p`, the segment path, to safe storage under the name `%f`). A base backup defines the starting state; recovery starts from the base backup, fetches WAL with `restore_command`, replays it, and stops at a recovery target (`recovery_target_time`, `recovery_target_lsn`, `recovery_target_xid` or a named restore point created with `pg_create_restore_point`). The server is told to enter recovery by an empty file `recovery.signal` in the data directory. `recovery_target_action = 'promote'` makes it open for writes when the target is reached.

MySQL has the equivalent procedure: restore a full backup, then replay the binary logs up to a position or time with `mysqlbinlog --stop-datetime=... | mysql`. The mechanism requires `log_bin` enabled and the binary logs retained (`binlog_expire_logs_seconds`); the commands below for MySQL are typical and were not run on the build machine.

1.28.2 Worked example: logical dump and restore

The sample database `shop` (customers and 200 000 orders, 24 MB on disk) was dumped in two formats. The custom format (`-Fc`) is compressed and supports selective and parallel restore; the plain format is a SQL script.

pg_dump in custom and plain format (real output; sizes in bytes) TERMINAL

```
$ time pg_dump -Fc -f dumps/shop.dump shop        # 0.77 s
$ ls -l dumps/shop.dump | awk '{print $5}'
2196136
$ time pg_dump -Fp -f dumps/shop.sql shop         # 0.32 s
$ ls -l dumps/shop.sql | awk '{print $5}'
10601932
$ grep -n -E "^(CREATE TABLE|COPY|ALTER TABLE ONLY)" dumps/shop.sql | head -6
29:CREATE TABLE public.customers (
44:CREATE TABLE public.orders (
80:ALTER TABLE ONLY public.orders ALTER COLUMN id SET DEFAULT nextval(...);
87:COPY public.customers (id, name, email, is_admin, password) FROM stdin;
1099:COPY public.orders (id, customer_id, amount, status, created_at) FROM stdin;
201114:ALTER TABLE ONLY public.customers
```

The ordering in the script is deliberate: tables, then data through `COPY`, then constraints and indexes, because loading data into an unindexed table and building the index afterwards is much faster than maintaining the index per row. `pg_restore -l` lists the table of contents of the custom archive, which can be edited to restore a subset (`-L`).

Restoring into a new database with two parallel jobs (real output) TERMINAL

```
$ createdb shop_restored
$ time pg_restore -j 2 -d shop_restored dumps/shop.dump      # 0.95 s
shop_restored=# select (select count(*) from customers) c,
shop_restored-#   (select count(*) from orders) o;
  c   |   o
------+--------
 1004 | 200000
$ pg_restore -l dumps/shop.dump | grep -v "^;" | head -4
215; 1259 16385 TABLE public customers postgres
217; 1259 16421 TABLE public orders postgres
216; 1259 16420 SEQUENCE public orders_id_seq postgres
3414; 0 0 SEQUENCE OWNED BY public orders_id_seq postgres
```

The MySQL counterparts (typical, not run): `mysqldump --single-transaction --routines --triggers shop > shop.sql` (the option takes a consistent InnoDB snapshot without locking tables) and `mysql shop < shop.sql`.

> **NOTE**
> A dump does not include roles and tablespaces. `pg_dump` saves one database. Cluster-wide objects (roles, their passwords, tablespaces) are saved by `pg_dumpall --globals-only`. A restore that lacks the roles fails with `role "shop_app" does not exist` on every `GRANT` and `OWNER TO`.

1.28.3 Hands-on: base backup, WAL archive and recovery from an accidental DELETE

Setup

The primary was configured with `archive_mode = on` and `archive_command = 'cp %p /tmp/osw_d5/archive/%f'`. A production archive command must be atomic and idempotent (copy to a temporary name and rename, never overwrite), and should write to storage outside the database server. A base backup was taken with `pg_basebackup` and verified with `pg_verifybackup`, which checks the backup manifest:

Base backup (real output) TERMINAL

```
$ pg_basebackup -D /tmp/osw_d5/base1 -X stream -c fast
$ pg_verifybackup /tmp/osw_d5/base1
backup successfully verified
$ du -sh base1
88M     base1
```

The accident

After the backup, 500 more orders were inserted. The time `T0` (06:59:46.461 UTC) was noted as the last good moment. Two seconds later a `DELETE FROM orders` was executed without a `WHERE` clause, the classic mistake of a console session pointed at production. A WAL segment switch (`pg_switch_wal()`) made sure the segment holding the delete reached the archive.

The mistake (real output) TERMINAL

```
shop=# select count(*) from orders;
 200500
shop=# delete from orders;
DELETE 200500
shop=# select count(*) from orders;
 0
$ ls archive | tail -3
000000010000000000000018.00000028.backup
000000010000000000000019
00000001000000000000001A
```

The recovery

The base backup was copied to a new directory (the pg_wal contents were left out, because recovery takes WAL from the archive), and recovery was configured in the new copy. The damaged production server was not touched, so the recovered copy could be inspected before anything was changed.

postgresql.conf (additions in the restored copy) INI

```
port = 20501
archive_mode = off
restore_command = 'cp /tmp/osw_d5/archive/%f %p'
recovery_target_time = '2026-10-18 06:59:46.461+00'
recovery_target_action = 'promote'
```

Starting the recovery (real output, abridged) TERMINAL

```
$ touch restore1/recovery.signal
$ pg_ctl -D restore1 -l restore1.log -w start
LOG:  starting point-in-time recovery to 2026-10-18 06:59:46.461+00
LOG:  starting backup recovery with redo LSN 0/18000028, checkpoint LSN 0/18000060, on timeline ID 1
LOG:  restored log file "000000010000000000000018" from archive
LOG:  redo starts at 0/18000028
LOG:  restored log file "000000010000000000000019" from archive
LOG:  consistent recovery state reached at 0/18000100
LOG:  restored log file "00000001000000000000001A" from archive
LOG:  recovery stopping before commit of transaction 60880, time 2026-10-18 06:59:48.597473+00
LOG:  redo done at 0/1A6EC638 system usage: CPU: user: 0.07 s, system: 0.00 s, elapsed: 0.15 s
LOG:  last completed transaction was at log time 2026-10-18 06:59:44.365904+00
LOG:  archive recovery complete
$ psql -p 20501 shop -c "select pg_is_in_recovery()" -c "select count(*) from orders"
 f
 200500
```

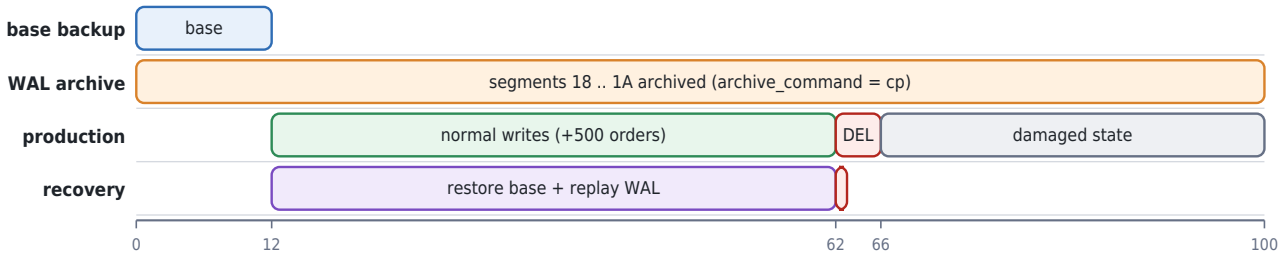


Figure 1.138. Recovery restores the base backup and replays archived WAL up to the target just before the transaction that deleted the rows; the deleting commit is never applied.

Recovery stopped "before commit of transaction 60880": the `DELETE` had made its changes in WAL, but its commit record lies after the target, so it is treated as an aborted transaction and the 200 500 rows are present. A production recovery would usually then export the damaged table from the recovered copy and load it into the live database (`pg_dump -t orders` from the copy, `psql` into production), rather than switching all traffic to the recovered cluster, because writes made after the accident would otherwise be lost.

The same flow for MySQL (typical, not run on the build machine): restore the last full backup, then `mysqlbinlog --start-position=<backup position> --stop-datetime="2026-10-18 06:59:47" binlog.000012 | mysql`. Locating the exact position before the `DELETE` is done with `mysqlbinlog --verbose` and a search for the statement.

WATCH OUT

A backup that has not been restored is a hypothesis. Typical failures found only at restore time: the archive command silently failed (`pg_stat_archiver.failed_count`), the WAL segment between two backups is missing, the backup is encrypted with a key that no longer exists, the restore takes 14 hours against an RTO of 2. Schedule a restore into a scratch environment, run an integrity check (`pg_amcheck`) and a row-count comparison, and record the measured duration.

1.28.4 Real-world problems and solutions

Incident: accidental DELETE without WHERE recovered with PITR

Scenario. At 07:00 a developer runs a clean-up statement on the production `orders` table from a SQL console in autocommit mode. The `WHERE` clause was not selected when the statement was executed.

Symptoms. `DELETE 200500` is displayed. Within seconds the application reports empty order lists; replicas show the same emptiness because the delete replicated.

Diagnosis. First stop the damage: revoke write access or stop the application. Determine the exact time of the damage: the database log (with `log_statement = 'mod'` or from the audit log), or inspect WAL with `pg_waldump -r Heap` around the time. Determine the last good time, and confirm that base backup and archive cover it (`pg_stat_archiver`, listing of the archive).

Root cause. A destructive statement was executed in autocommit mode by an account that holds `DELETE` on a business table, with no safety net such as an explicit transaction.

Fix. Immediate: restore to a separate server with `recovery_target_time` just before the delete (as demonstrated above; 0.15 s of replay and under a minute in total on the sample data), verify the data, then reload the table into production in a transaction. Permanent: keep continuous archiving and a tested procedure.

Prevention. Console sessions with `\set AUTOCOMMIT off` (psql) so that each statement requires `COMMIT`; read-only roles for people and write roles only for deployed services; four-eyes review for production data fixes; delayed replicas (`recovery_min_apply_delay = 1h`) as a cheap second line; `pg_create_restore_point('before_cleanup')` before planned destructive operations.

Incident: restore too slow for the RTO

Scenario. A 1.5 TB database must be restored after storage failure. The plan, written years earlier, uses a nightly `pg_dump -Fp`. The RTO is 2 hours.

Symptoms. The restore job is still creating indexes after 9 hours; the business has been offline all day.

Diagnosis. In a rehearsal (or the real event) measure phases: `COPY` data loading, index builds, constraint validation, `ANALYZE`. A restore of a logical dump is bound by CPU for index builds and by `maintenance_work_mem`. On the sample

data the 24 MB database restored in 0.95 s with two jobs, which hides the problem at small scale; the cost grows at least linearly with data and indexes.

Root cause. A logical dump was used as the only backup for a database whose volume exceeds the RTO. Plain format restores serially.

Fix. Immediate: restore with the custom or directory format and `pg_restore -j N`, with `maintenance_work_mem` raised and `fsync`, `synchronous_commit` and `full_page_writes` temporarily relaxed on the restore target only (never on a production server in service). Permanent: physical backups with WAL archiving (restore is file copy plus replay), a warm standby for failover, and a documented, rehearsed runbook.

Prevention. A quarterly restore test that records elapsed time against the RTO.

Incident: backups succeed but WAL archive has failed for days

Scenario. The archive destination fills up; `archive_command` returns an error. PostgreSQL retains unarchived WAL in `pg_wal` and the disk begins to fill, as in the slot incident of the previous chapter.

Symptoms. `pg_wal` grows; the log repeats `archive command failed with exit code 1`.

Diagnosis.

diagnosis.sql — SQL

```sql
SELECT archived_count, last_archived_wal, last_archived_time,
       failed_count, last_failed_wal, last_failed_time
FROM pg_stat_archiver;
```

A non-zero `failed_count` with `last_failed_time` newer than `last_archived_time` means the archive is broken now. The healthy server in this chapter showed `failed_count = 0` and a recent `last_archived_time`.

Root cause. Full archive volume, expired credentials to object storage or a permissions change.

Fix. Repair the destination so that the oldest unarchived segment is archived, which lets the server recycle WAL.
Prevention. Alert on `failed_count` growth and on the age of `last_archived_time`.

1.28.5 Questions to expect

What is the difference between a logical and a physical backup?

Logical: SQL or archive of statements and rows, portable, slow to restore, one database. Physical: copy of data files, fast restore, same major version, whole cluster, and combined with WAL archiving it enables point-in-time recovery.

How does point-in-time recovery work in PostgreSQL?

Restore a base backup, set `restore_command` and a recovery target, create `recovery.signal`, start the server. It replays archived WAL from the backup's start point to the target and then promotes. The commit that follows the target is not applied.

Define RPO and RTO and how each backup type affects them.

RPO is the tolerable data loss; RTO the tolerable downtime. Nightly dumps give an RPO of up to a day and a slow RTO; base backups with WAL archiving give an RPO of seconds and an RTO bounded by copy time plus replay time.

Why is a replica not a backup?

It replicates mistakes: a dropped table or mass delete is applied to it too. A delayed replica or PITR is required for logical damage.

Why does `mysqldump --single-transaction` matter?

It runs the dump in one consistent-snapshot transaction on InnoDB, so the dump is consistent without locking the tables. It does not give a consistent snapshot for non-transactional engines.

sreram

KEY POINTS

- Replication protects against hardware loss, backups protect against logical damage; both are needed.
- Logical dumps are portable and slow to restore; physical backups plus WAL archiving restore quickly and support point-in-time recovery.
- Recovery to a target time is deterministic: base backup, archived WAL, `recovery_target_time`, `recovery.signal`.
- Monitor `pg_stat_archiver`; a failing archive silently destroys the RPO.
- Measure restore time regularly; the RTO is what the rehearsal measured.

1.29 Partitioning and Sharding

A table that grows without bound eventually causes operational problems that are independent of query speed: indexes that no longer fit in memory, vacuum that runs for hours, and deletes of old data that take longer than the day they are meant to free. Partitioning splits one logical table into smaller physical tables inside one database. Sharding splits data across several database servers. The first is a physical design technique with transparent benefits; the second is an architectural commitment with permanent costs.

1.29.1 Concept and mechanism

Declarative partitioning in PostgreSQL

A partitioned table has no storage of its own. It is defined with `PARTITION BY RANGE`, `LIST` or `HASH` on one or more columns, and its partitions are ordinary tables that each hold the rows whose key falls in their bound. An `INSERT` is routed to the matching partition (an error occurs if none matches, unless a `DEFAULT` partition exists). A query that filters on the partition key allows the planner to exclude the other partitions (partition pruning), and `Append` nodes combine the remaining ones. MySQL 8 supports `PARTITION BY RANGE`, `LIST`, `HASH` and `KEY`, with the restriction that every unique key must include the partition columns, and without foreign keys on partitioned InnoDB tables.

Strategy	Typical use	Caution
Range	Time series, log data: one partition per month or day	The newest partition receives all writes; create partitions in advance.
List	Region, tenant class, status	Skew if a value dominates.
Hash	Even spread of a key such as user_id	No benefit for range queries; changing the modulus rewrites data.

Partitioning strategies.

The benefits are narrower than often assumed. Partitioning does not speed up queries that do not use the partition key; it can slow them, because the planner and executor handle several tables and every partition needs its own indexes. The decisive operational benefit is bulk removal: dropping or detaching a partition is a metadata operation.

1.29.2 Worked example: range partitioning with pruning

A table `events` was partitioned by month on `created_at`, with three partitions and 300 000 rows spread over August to October 2026.

range_partition.sql SQL

```sql
CREATE TABLE events (
  id bigint GENERATED ALWAYS AS IDENTITY,
  created_at timestamptz NOT NULL,
  user_id int,
  payload text
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2026_08 PARTITION OF events
  FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
CREATE TABLE events_2026_09 PARTITION OF events
  FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE events_2026_10 PARTITION OF events
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

CREATE INDEX ON events (created_at);   -- creates one index per partition
```

The bounds are inclusive at the lower end and exclusive at the upper end, which makes consecutive partitions contiguous without overlap. The row distribution confirms routing:

Rows per partition (real output) TERMINAL

```
shop=# select tableoid::regclass part, count(*) from events group by 1 order by 1;
      part      | count
----------------+--------
 events_2026_08 | 102009
 events_2026_09 |  98911
 events_2026_10 |  99080
```

A one-day query on the partition key touches one partition only; the other two do not appear in the plan:

Partition pruning (real output; plan abridged) TERMINAL

```
shop=# explain (analyze, costs off, timing off, summary off)
shop-#   select count(*) from events
shop-#   where created_at >= '2026-09-10' and created_at < '2026-09-11';
 Aggregate (actual rows=1 loops=1)
   ->  Bitmap Heap Scan on events_2026_09 events (actual rows=3178 loops=1)
         Heap Blocks: exact=629
         ->  Bitmap Index Scan on events_2026_09_created_at_idx (actual rows=3178 loops=1)
```

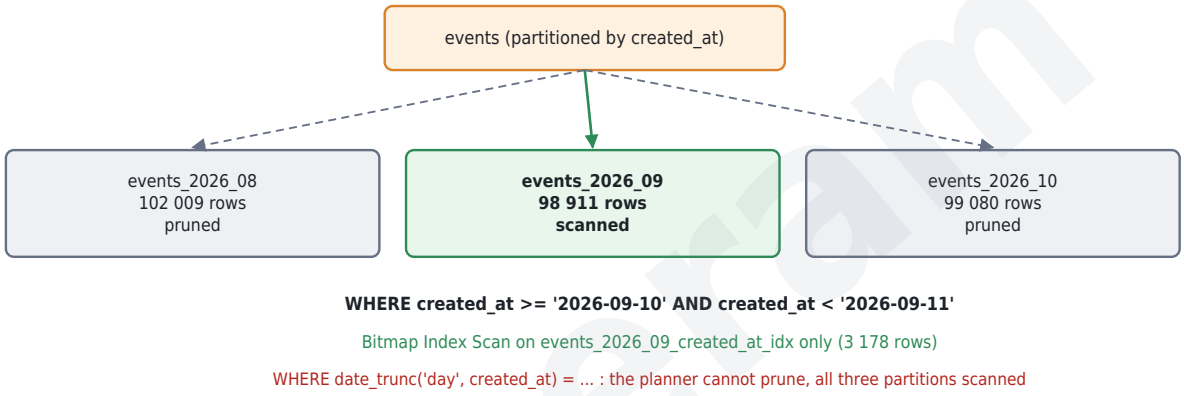


Figure 1.139. Only the September partition is scanned; the pruning decision is made from the constant bounds in the predicate.

Two predicates that do not allow pruning were tested. A filter on a non-key column scans all partitions (here with a parallel append), and wrapping the key in a function hides it from the planner:

Predicates without pruning (real output; plans abridged) TERMINAL

```
shop=# explain (costs off) select count(*) from events where user_id = 5;
   ->  Parallel Append
         ->  Parallel Seq Scan on events_2026_08 events_1   Filter: (user_id = 5)
         ->  Parallel Seq Scan on events_2026_10 events_3   Filter: (user_id = 5)
         ->  Parallel Seq Scan on events_2026_09 events_2   Filter: (user_id = 5)
shop=# explain (costs off) select count(*) from events
shop-#   where date_trunc('day', created_at) = '2026-09-10';
   ->  Parallel Append    (all three partitions scanned, Filter: date_trunc('day', ...))
```

Hash partitioning

Hash partitioning distributes rows by `hash(key) mod modulus`. It is useful when no natural range exists and the load must be spread. A `sessions` table with four hash partitions received 100 000 rows with 5 000 distinct user ids:

Hash partitions (real output) TERMINAL

```
shop=# select tableoid::regclass part, count(*) from sessions group by 1 order by 1;
    part     | count
-------------+-------
 sessions_p0 | 24380
 sessions_p1 | 25480
 sessions_p2 | 25600
 sessions_p3 | 24540
shop=# explain (costs off) select * from sessions where user_id = 42;
 Bitmap Heap Scan on sessions_p2 sessions
   Recheck Cond: (user_id = 42)
   ->  Bitmap Index Scan on sessions_p2_pkey
```

Hash partitions (real output) (continued) TERMINAL

```
         Index Cond: (user_id = 42)
shop=# explain (costs off) select * from sessions where token = 'abc';
 Append
   ->  Seq Scan on sessions_p0 sessions_1   Filter: (token = 'abc'::text)
   ->  Seq Scan on sessions_p1 sessions_2 ...  (all four partitions)
```

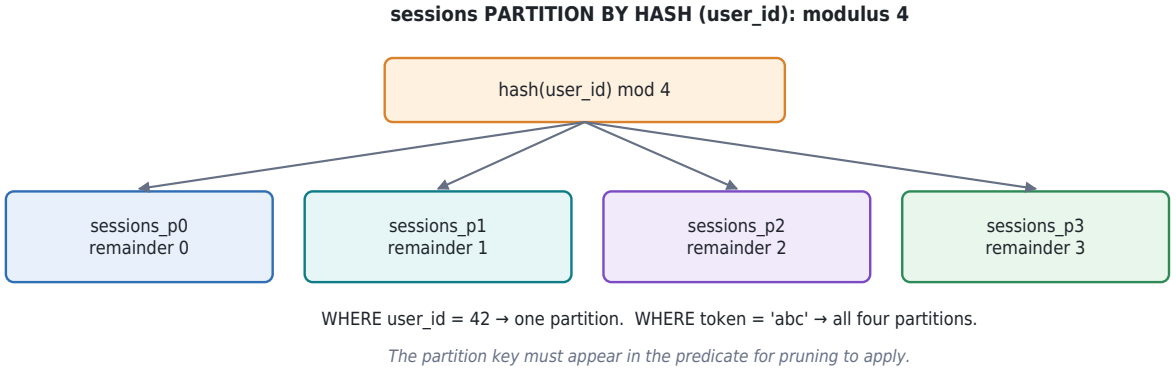


Figure 1.140. A predicate on the partition key reaches one partition; any other predicate reaches all of them.

Dropping old data

The operational benefit is retention. Deleting the 102 009 August rows from the unpartitioned perspective with `DELETE` took 1.996 s here, wrote WAL for every row and left dead tuples for vacuum. Detaching and dropping the partition holding the same kind of data took 2.1 ms and 7.6 ms:

Retention by DELETE versus DROP PARTITION (real output) TERMINAL

```
shop=# \timing on
shop=# delete from events where created_at < '2026-09-01';
DELETE 102009
Time: 1995.906 ms (00:01.996)
shop=# alter table events detach partition events_2026_09;
ALTER TABLE
Time: 2.145 ms
shop=# drop table events_2026_09;
DROP TABLE
Time: 7.571 ms
```

The detach and drop were applied to a different partition from the one the delete worked on, so the comparison is between a row-wise delete of about 100 000 rows and a metadata operation on a partition of about 100 000 rows. `DETACH PARTITION CONCURRENTLY` (PostgreSQL 14 and later) avoids the strongest lock on the parent.

GOTCHA

Create partitions before they are needed. A row whose key matches no partition causes `ERROR: no partition of relation "events" found for row`. Automate creation of next month's partition (a scheduled job or the `pg_partman` extension) and consider a `DEFAULT` partition as a safety net, remembering that a large default partition complicates later `CREATE TABLE ... PARTITION OF`.

1.29.3 Sharding

When partitioning is not enough

Partitioning keeps all data on one server. When the write rate, the working set or the storage exceeds what one server can provide, data is distributed across servers: sharding. A shard key determines which server owns a row. The application, a proxy (Vitess for MySQL, Citus for PostgreSQL) or the database itself performs the routing.

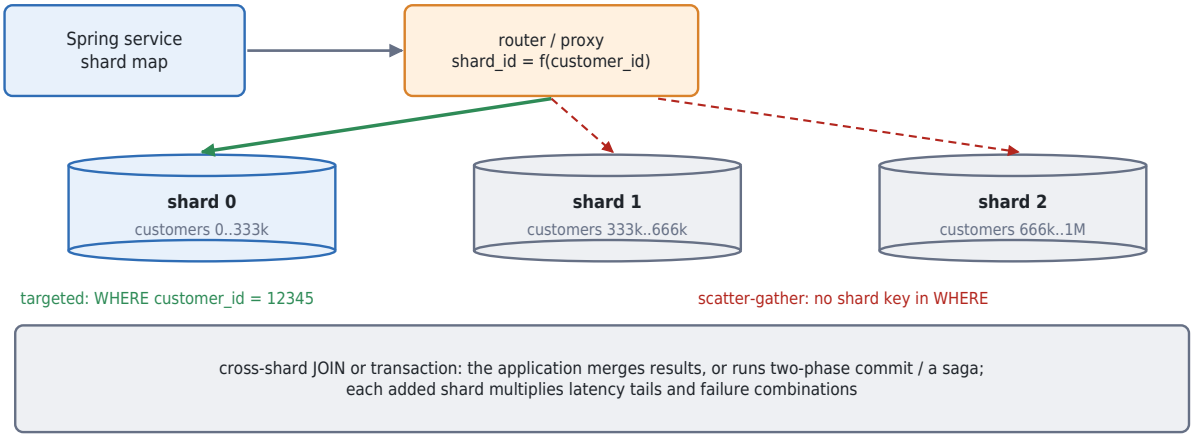


Figure 1.141. A query that contains the shard key is routed to one shard; a query that does not is sent to every shard and the results are merged.

Strategies and consistent hashing

Range sharding assigns contiguous key ranges to shards; it supports range scans and is simple to reason about, but monotonically increasing keys (timestamps, auto-increment ids) concentrate all new writes on the last shard. Hash sharding spreads keys evenly; range queries must visit all shards. Directory-based sharding keeps an explicit lookup table from key (or tenant) to shard; it is the most flexible and makes the directory a critical component.

Placing keys with `hash(key) mod N` has a severe weakness: when N changes, almost every key moves. Consistent hashing places both nodes and keys on a ring (the hash space as a circle); a key belongs to the first node clockwise from its position. Adding a node takes over keys only from its neighbours. Virtual nodes (each physical node appears at many ring positions) even out the load. The following Java program was run to measure both effects with 100 000 keys:

Ring.java JAVA

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.*;

public class Ring {
    static long hash(String s) throws Exception {
        byte[] d = MessageDigest.getInstance("MD5").digest(s.getBytes(StandardCharsets.UTF_8));
        return ((d[0] & 0xFFL) << 24) | ((d[1] & 0xFFL) << 16)
             | ((d[2] & 0xFFL) << 8) | (d[3] & 0xFFL);
    }
    static TreeMap<Long, String> build(List<String> nodes, int vnodes) throws Exception {
        TreeMap<Long, String> ring = new TreeMap<>();
        for (String n : nodes) for (int v = 0; v < vnodes; v++) ring.put(hash(n + "#" + v), n);
        return ring;
    }
    static String owner(TreeMap<Long, String> ring, String key) throws Exception {
        Map.Entry<Long, String> e = ring.ceilingEntry(hash(key));
        return (e != null ? e : ring.firstEntry()).getValue();
    }
    public static void main(String[] args) throws Exception {
        int keys = 100_000;
        for (int vn : new int[] {1, 16, 256}) {
            TreeMap<Long, String> ring = build(List.of("A", "B", "C"), vn);
            Map<String, Integer> cnt = new TreeMap<>();
            for (int i = 0; i < keys; i++) cnt.merge(owner(ring, "user:" + i), 1, Integer::sum);
            System.out.println("vnodes=" + vn + " load=" + cnt);
        }
        TreeMap<Long, String> before = build(List.of("A", "B", "C"), 256);
        TreeMap<Long, String> after = build(List.of("A", "B", "C", "D"), 256);
        int moved = 0, movedMod = 0;
        List<String> n3 = List.of("A", "B", "C"), n4 = List.of("A", "B", "C", "D");
        for (int i = 0; i < keys; i++) {
            String k = "user:" + i;
            if (!owner(before, k).equals(owner(after, k))) moved++;
            if (!n3.get((int) (hash(k) % 3)).equals(n4.get((int) (hash(k) % 4)))) movedMod++;
        }
        System.out.printf("ring, add node D: %d of %d keys moved (%.1f%%)%n",
```

Ring.java (continued) JAVA

```
            moved, keys, 100.0 * moved / keys);
        System.out.printf("hash mod N, 3 to 4 nodes: %d of %d keys moved (%.1f%%)%n",
            movedMod, keys, 100.0 * movedMod / keys);
        Map<String, Integer> hot = new TreeMap<>();
        Random r = new Random(1);
        TreeMap<Long, String> ring = build(List.of("A", "B", "C", "D"), 256);
        for (int i = 0; i < keys; i++) {
            String k = r.nextInt(100) < 30 ? "user:celebrity" : "user:" + r.nextInt(50_000);
            hot.merge(owner(ring, k), 1, Integer::sum);
        }
        System.out.println("hot key 30% of requests: " + hot);
    }
}
```

Output (real) TERMINAL

```
$ java Ring.java
vnodes=1 load={A=3057, B=56737, C=40206}
vnodes=16 load={A=35256, B=38280, C=26464}
vnodes=256 load={A=33982, B=31576, C=34442}
ring, add node D: 21984 of 100000 keys moved (22.0%)
hash mod N, 3 to 4 nodes: 74701 of 100000 keys moved (74.7%)
hot key 30% of requests: {A=19019, B=16488, C=49137, D=15356}
```

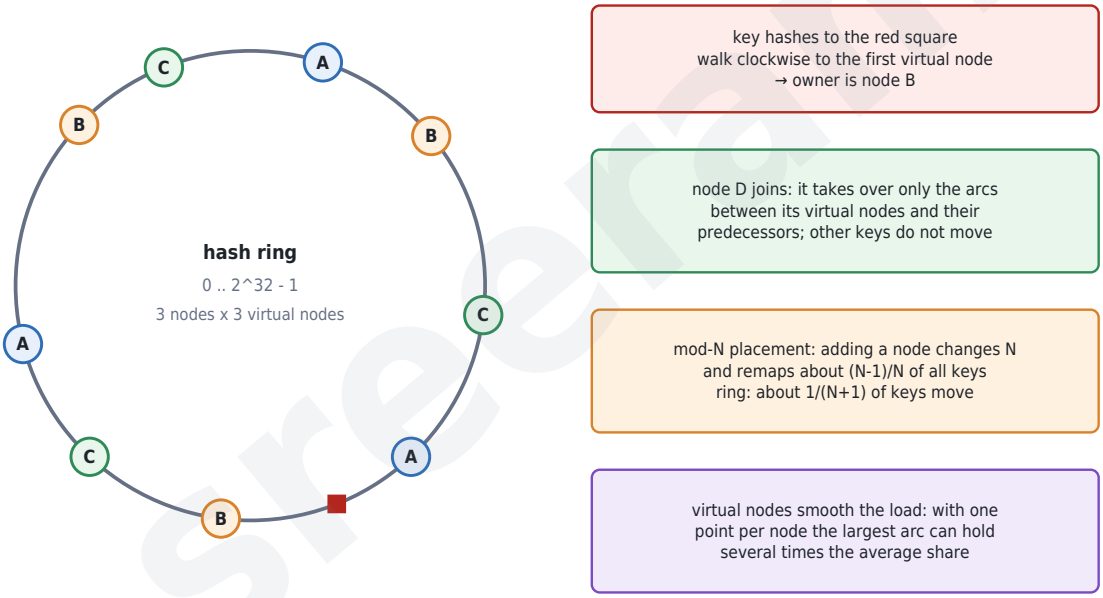


Figure 1.142. A key is owned by the first virtual node clockwise from its hash; a joining node takes keys only from its neighbours.

Three observations follow from the output. With one position per node, node A owned 3 percent of the keys and node B 57 percent; with 256 virtual nodes each owned about a third. Adding a fourth node moved 22 percent of the keys on the ring but 74.7 percent with modulo placement (the theoretical value is 3/4). Finally, a single hot key (30 percent of the requests) overloaded node C with 49 percent of the traffic, no matter how well the hash balances keys: consistent hashing balances keys, not load.

Cross-shard queries and transactions

A query without the shard key must visit all shards (scatter-gather); its latency is that of the slowest shard, and its failure probability is the combined failure probability of all shards. Joins across shards are performed in the application or avoided by co-locating related rows (all rows of a customer on the shard of the customer; in Citus, co-located distributed tables). A transaction touching two shards requires two-phase commit (PostgreSQL `PREPARE TRANSACTION`, XA transactions in MySQL), which blocks on coordinator failure, or an application-level saga with compensating actions. Global unique ids require a scheme such as Snowflake-style 64-bit ids, UUIDs or a block allocator. Resharding (moving ranges between servers while serving traffic) is the most expensive operation of a sharded system.

sreeram

INTERVIEW ANGLE
When would you shard, and what would you try first? Exhaust cheaper options first: indexes and query fixes, a larger server, read replicas, caching, archiving old data and partitioning. Shard when write throughput or data volume exceeds one server. Choose a shard key present in nearly all queries, with high cardinality and even load, and design so that most transactions stay inside one shard.

1.29.4 Real-world problems and solutions

Incident: hot partition from a poor key

Scenario. An `events` table is range-partitioned by month. A job floods the current month; a second system is sharded by `tenant_id` and one tenant produces 40 percent of the traffic.

Symptoms. One partition or shard shows high CPU and I/O while the others are idle; adding hardware to the whole cluster changes little. Lock waits and index contention appear on the newest partition's index tail.

Diagnosis. Compare per-partition activity:

diagnosis.sql SQL

```sql
SELECT relname, seq_scan, idx_scan, n_tup_ins, n_tup_upd, n_tup_del
FROM pg_stat_user_tables
WHERE relname LIKE 'events_%'
ORDER BY n_tup_ins DESC;
```

For shards, compare request rate and CPU per shard and the distribution of the top shard keys (`SELECT tenant_id, count(*) ... GROUP BY 1 ORDER BY 2 DESC LIMIT 10`). The Java run above shows the effect: with a key carrying 30 percent of the traffic, one of four nodes served 49 percent of the requests.

Root cause. Time-based partitioning concentrates writes by construction; a skewed key (one large tenant, one celebrity user) concentrates load on one shard even if the hash is uniform.

Fix. Immediate: split the large tenant onto a dedicated shard or partition; cache reads for the hot key. Permanent: a composite shard key (tenant plus a bucket number so one tenant spans several shards), smaller time partitions with the active one on fast storage, and capacity limits per tenant.

Prevention. Load tests with realistic key skew; dashboards of per-shard load; a shard key review that checks cardinality, frequency and monotonicity.

Incident: partitioning made the query slower

Scenario. A table of 300 million rows is partitioned by month. The most frequent query filters by `customer_id` only.

Symptoms. Planning time rises, and latency becomes the sum of 36 index probes instead of one.

Diagnosis. `EXPLAIN (ANALYZE)` shows an `Append` with one index scan per partition, as in the `user_id` plan above where all partitions were scanned.

Root cause. The partition key does not appear in the dominant predicate, so no pruning is possible.

Fix. Add the partition key to the query where the business logic permits (a date bound), or repartition by a key that matches the access path (hash on `customer_id`). **Prevention.** Choose partition keys from the query log, not from the schema.

Incident: cross-shard report that times out

Scenario. After sharding by customer, a dashboard computing revenue per product category queries all shards.

Symptoms. The dashboard takes 40 seconds; one slow shard determines the time; the failure of one shard makes the whole dashboard fail.

Diagnosis. The router log shows a fan-out to all shards, and the per-shard timings show a skewed tail.

Root cause. An analytical query was placed on an operational sharded store.

Fix. Aggregate through a pipeline into a reporting database (change data capture into a warehouse or a read replica per shard consolidated centrally). **Prevention.** Decide per query class whether it is operational (shard-local) or analytical (separate store).

1.29.5 Questions to expect

What is partition pruning?

The planner's exclusion of partitions that cannot contain matching rows, derived from the predicate on the partition key. It works with constant bounds or parameters that the planner can resolve, not with the key wrapped in a function.

Partitioning versus sharding?

Partitioning divides a table within one server; it helps maintenance and pruning. Sharding divides data across servers; it adds capacity for writes and storage but makes cross-shard queries and transactions hard.

Why is `hash(key) mod N` a poor way to place data on N servers?

Changing N remaps most keys (74.7 percent for 3 to 4 nodes in the measurement). Consistent hashing remaps only the keys of the neighbouring arcs (22 percent in the measurement).

What makes a good shard key?

High cardinality, even frequency, non-monotonic, and present in the main queries so that they are shard-local.

How do you delete old data from a large table without long locks and bloat?

Range-partition by time and drop or detach expired partitions; the measured drop took milliseconds against two seconds for the equivalent row delete.

KEY POINTS

- Partitioning helps maintenance and, when the predicate contains the key, pruning; it does not speed up other queries.
- Dropping a partition replaces a long delete and vacuum with a metadata operation.
- Sharding adds write and storage capacity at the cost of cross-shard queries, transactions and operational complexity; exhaust simpler options first.
- Consistent hashing with virtual nodes limits data movement when nodes change, but it balances keys, not hot-key load.
- Choose partition and shard keys from the access paths: cardinality, even frequency, no monotonic growth.

1.30 Caching and the Database

A cache stores the result of an expensive operation so that the next request can avoid it. For a database-backed service the expensive operation is a query, and the cache is typically an in-process map (Caffeine) or an external store such as Redis. Caching reduces read load and latency, but it creates a second copy of the data that can be wrong. This chapter treats the interaction between cache and database: the standard patterns, the consistency problems they leave open, and the failure that most often turns a cache into an outage, the stampede. The Redis data structures themselves are covered in the separate Redis guide and are not repeated here.

1.30.1 Concept and mechanism

Cache-aside

In cache-aside (lazy loading) the application owns the logic. A read first asks the cache; on a miss it reads the database, stores the result in the cache with a time to live (TTL) and returns it. A write updates the database and then invalidates (deletes) the cached entry, so that the next read repopulates it. In Spring this is `@Cacheable` for reads and `@CacheEvict` or `@CachePut` for writes.

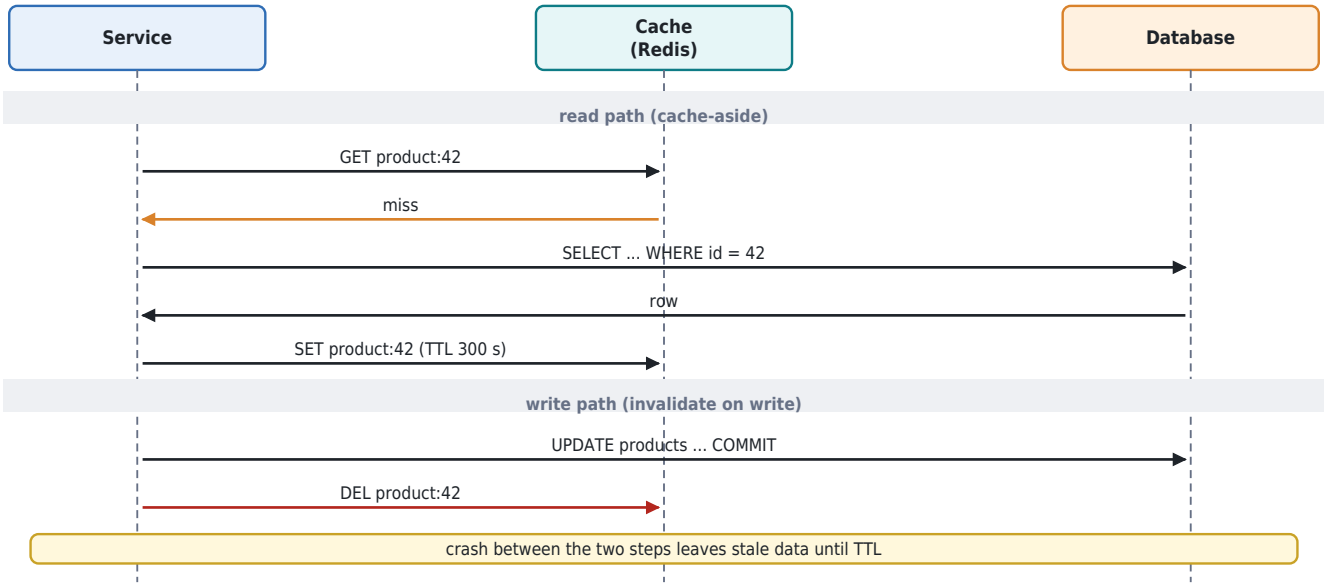


Figure 1.143. The database stays the source of truth; the cache is filled on demand and entries are deleted on write.

The write path contains two steps that cannot be made atomic, the database commit and the cache delete. If the process fails between them, the cache keeps the old value until the TTL expires. A second race exists between a reader and a writer: a reader misses, reads the old row, the writer commits and deletes the entry, and the reader then stores the old row in the cache. Deleting after the commit (not before) and keeping TTLs short narrow these windows; they do not close them. The TTL is therefore a correctness bound, not only a memory setting.

Pattern	Mechanism	Trade-off
Cache-aside	Application reads cache, falls back to the database, deletes on write	Simple and resilient to cache failure; stale windows; first request after a miss is slow.
Read-through	The cache library loads from the database on a miss	Same semantics, loading logic centralised in the cache layer.
Write-through	Every write goes to the cache, which writes to the database synchronously	Cache is fresh; write latency includes both; the cache must be highly available.
Write-behind	Writes go to the cache and are persisted asynchronously	Fast writes; data loss if the cache fails before persisting; ordering problems.

Cache patterns compared.

What to cache and what not

Cache data that is read much more often than written, expensive to compute, and tolerant of seconds of staleness: product catalogues, configuration, computed aggregates, authorisation lookups with short TTL. Do not cache data that must be exact at the moment of a decision: stock levels at checkout, account balances for a transfer, uniqueness checks. For those the database transaction is the only correct arbiter. A cache also hides query defects; a slow query that is "fixed" by caching still runs on every miss and on every restart.

The stampede (thundering herd)

When a popular key expires, all concurrent requests miss at the same moment and all query the database for the same value. The database receives N identical queries instead of one, which can overload it, which slows the loads, which keeps the entry missing, which brings more requests: a feedback loop. Remedies are single-flight (one loader per key; the others wait for its result), a lock in the cache (`SET key NX PX`), TTL jitter (random variation so that entries do not expire together), probabilistic early refresh, and serving stale data while one request refreshes.

1.30.2 Worked example: a measured stampede

The following program starts 100 threads that all request one key at the instant it is absent. The database call takes 200 ms. The first variant is naive cache-aside; the second shares one in-flight load per key through a `CompletableFuture`.

Stampede.java JAVA

```java
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

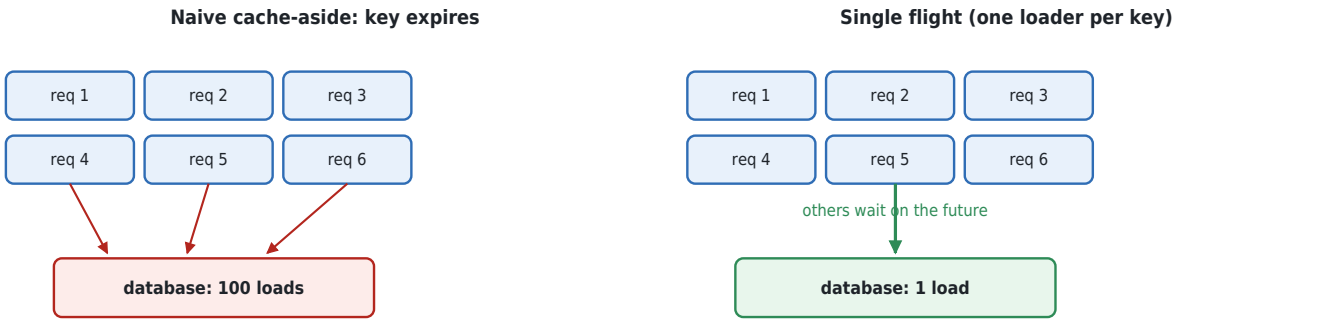
public class Stampede {
    static final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();
    static final ConcurrentHashMap<String, CompletableFuture<String>> inflight =
        new ConcurrentHashMap<>();
    static final AtomicInteger dbHits = new AtomicInteger();

    static String loadFromDb(String key) {
        dbHits.incrementAndGet();
        try { Thread.sleep(200); } catch (InterruptedException e) { throw new RuntimeException(e); }
        return "value-of-" + key;
    }
    static String naive(String key) {
        String v = cache.get(key);
        if (v == null) { v = loadFromDb(key); cache.put(key, v); }
        return v;
    }
    static String singleFlight(String key) {
        String v = cache.get(key);
        if (v != null) return v;
        CompletableFuture<String> f = inflight.computeIfAbsent(key, k ->
            CompletableFuture.supplyAsync(() -> {
                String x = loadFromDb(k); cache.put(k, x); return x;
            }).whenComplete((x, t) -> inflight.remove(k)));
        return f.join();
    }
    static int run(boolean safe) throws Exception {
        cache.clear(); dbHits.set(0);
        ExecutorService pool = Executors.newFixedThreadPool(100);
        CountDownLatch go = new CountDownLatch(1);
        for (int i = 0; i < 100; i++) {
            pool.submit(() -> { go.await(); return safe ? singleFlight("hot") : naive("hot"); });
        }
        go.countDown(); pool.shutdown(); pool.awaitTermination(10, TimeUnit.SECONDS);
        return dbHits.get();
    }
    public static void main(String[] a) throws Exception {
        System.out.println("100 concurrent requests, key just expired");
        System.out.println("naive cache-aside:   database loads = " + run(false));
        System.out.println("single-flight:       database loads = " + run(true));
    }
}
```

Output (real) TERMINAL

```
$ java Stampede.java
100 concurrent requests, key just expired
naive cache-aside:   database loads = 100
single-flight:       database loads = 1
```

sreeram



Measured with 100 threads, 200 ms loader (Java run in this chapter).

Figure 1.144. One hundred concurrent misses produce one hundred identical queries without coordination and one query with single-flight loading.

What to notice: the single-flight version registers the future before the load starts, using `computeIfAbsent`, which is atomic per key; there is a small window after `cache.put` and `inflight.remove` in which a request could start a second load, which is acceptable because the loads are idempotent. In a cluster of application instances the coordination is per instance; a distributed lock in Redis or a short negative-cache TTL provides cluster-wide protection. Spring offers `@Cacheable(sync = true)`, which makes the cache provider serialise the loading of one key within an instance.

1.30.3 Hands-on: Spring configuration (typical, not run)

ProductService.java JAVA

```java
// Fragment: Spring Cache with Caffeine; not run on the build machine.
@Service
public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    @Cacheable(cacheNames = "products", key = "#id", sync = true)
    public Product find(long id) {
        return repository.findById(id).orElseThrow();
    }

    @Transactional
    @CacheEvict(cacheNames = "products", key = "#product.id")
    public Product update(Product product) {
        return repository.save(product);
    }
}
```

application.properties PROPERTIES

```properties
spring.cache.type=caffeine
spring.cache.caffeine.spec=maximumSize=10000,expireAfterWrite=300s
```

GOTCHA

`@CacheEvict` runs before the transaction commits by default. The method returns inside the transaction, the cache entry is evicted, a concurrent reader repopulates the cache with the still uncommitted-old row, and the commit then leaves the stale value in the cache. Evict after commit, for example with `@TransactionalEventListener(phase = AFTER_COMMIT)` publishing an event that performs the eviction.

1.30.4 Real-world problems and solutions

Incident: cache stampede takes the database down

Scenario. The home page loads a ranked product list from a query that takes 800 ms. It is cached for 60 seconds. Traffic is 400 requests per second.

Symptoms. Every 60 seconds the database CPU reaches 100 percent for several seconds; `pg_stat_activity` shows dozens of identical active queries; the connection pool is exhausted and unrelated endpoints fail.

Diagnosis. Group active statements by text: `SELECT query, count(*) FROM pg_stat_activity WHERE state = 'active' GROUP BY 1 ORDER BY 2 DESC;`. The same ranking query appears 30 to 300 times at the same moment, aligned with the TTL boundary.

Root cause. About 320 requests arrive during the 800 ms of the load after expiry, and each of them starts the query. The entry expires for all instances at the same time.

Fix. Immediate: raise the TTL temporarily and warm the cache by hand. Permanent: single-flight loading (`sync = true` or a lock), TTL with random jitter (for example 60 s plus up to 20 percent), refresh in the background before expiry, and an index or materialised view that makes the query cheap in itself.

Prevention. Load test with a cold cache; metric of database queries per cache miss.

Incident: stale data after an update

Scenario. A price update is committed, yet the product page shows the old price for up to ten minutes; sometimes forever until the next deployment.

Symptoms. Direct queries show the new price; the cache shows the old one; users report inconsistent prices between the list and the detail page.

Diagnosis. Query the cache key directly and compare with the database row; inspect the TTL of the key (`TTL product:42`); look for write paths that bypass the service layer (a SQL script, a batch job, another service) and therefore never evict.

Root cause. Two causes occur: the write path does not invalidate (batch job updates the table directly), or the invalidation races with a concurrent read (the stale reload described above). A key without TTL turns the second cause into a permanent defect.

Fix. Immediate: delete the affected keys. Permanent: always set a TTL, evict after commit, and publish change events from the database (a trigger-driven outbox, or change data capture) so that every writer invalidates.

Prevention. Treat the cache as an optimisation for data whose staleness window is a written requirement.

Incident: cache hides a missing index until restart

Scenario. A slow query is masked by a cache hit rate of 99 percent. After a cache restart, all instances miss simultaneously.

Symptoms. Latency of several seconds and a connection storm just after the cache cluster restarts or after a deployment that changes the cache key prefix.

Diagnosis. `EXPLAIN (ANALYZE, BUFFERS)` on the query shows a sequential scan; the cache dashboard shows the hit rate falling to zero.

Root cause. The cache lowered the load but the query itself was never fixed, and the system lost its ability to start cold.

Fix. Add the index, rewrite the query, warm the cache gradually, limit concurrency of loaders with a semaphore.
Prevention. Run capacity tests with the cache disabled.

1.30.5 Questions to expect

Explain cache-aside and its consistency problems.

Read: cache, then database on a miss, then populate. Write: update the database, then delete the cache entry. The delete and the commit are not atomic and a reader can repopulate stale data between them, so entries need a TTL and eviction should follow the commit.

What is a cache stampede and how is it prevented?

Many concurrent requests miss the same expired key and all query the database. Prevent with single-flight loading, a lock, TTL jitter, early refresh, or serving stale data during refresh. Measured here: 100 loads versus 1.

Write-through versus write-behind?

Write-through writes cache and database synchronously, keeping them consistent at the cost of latency. Write-behind acknowledges after the cache write and persists later, which is faster but can lose data and reorder writes.

sreram

Which data must not be served from a cache?

Data on which an immediate decision depends, such as stock at checkout or an account balance for a transfer; the transactional database is the authority there.

KEY POINTS

- Cache-aside leaves the database authoritative; staleness is bounded by TTL and by the eviction discipline.
- Evict after commit, always set a TTL, and make every writer invalidate or publish a change event.
- A stampede multiplies one query by the concurrency; single-flight and TTL jitter turn 100 loads into 1.
- A cache must not be the reason a query is acceptable; the database must survive a cold cache.

1.31 Schema Migrations and Zero-Downtime Changes

The schema of a production database changes as the application changes. Each change is code, must be reviewed, versioned and repeatable, and must be applied to a database that is serving traffic. The central difficulty is that many DDL statements take locks that block the very queries the application is running. A migration that takes 30 ms in a test database can stop a production system for minutes when it queues behind a long transaction. This chapter covers the tools that version migrations, the pattern that avoids breaking running code (expand and contract), the locks of the main DDL statements, measured on PostgreSQL 16, and the ways to make index creation and column changes safe.

1.31.1 Concept and mechanism

Versioned migrations: Flyway and Liquibase

Flyway and Liquibase both record applied changes in a table of the database itself (`flyway_schema_history`, `DATABASECHANGELOG`) and apply pending ones at application start or from a pipeline. Flyway migrations are SQL files named `V<version>__<description>.sql` in `db/migration`; each is applied once, in version order, and its checksum is recorded so that editing an applied file is detected and rejected. Liquibase describes changes as change sets in XML, YAML or SQL, identified by id and author, with optional rollback blocks and preconditions. Spring Boot runs Flyway automatically when `flyway-core` is on the class path.

V7__add_order_channel.sql SQL

```sql
-- Flyway migration: applied exactly once; the file must never be edited afterwards.
ALTER TABLE orders ADD COLUMN channel text;
```

application.properties PROPERTIES

```properties
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.baseline-on-migrate=false
spring.jpa.hibernate.ddl-auto=validate
```

The setting `ddl-auto=validate` makes Hibernate verify that the entities match the schema without changing it; `update` and `create` must not be used against production, because the schema would then change outside review.

Running migrations at application startup has a hazard: with ten pods starting together, the migration runs once (Flyway takes a lock on its history table) but every pod waits, and a slow migration blocks the rollout and the readiness probes. Large migrations run as a separate job before the deployment.

Expand and contract

The application version and the schema version are never switched at the same instant: during a rolling deployment old and new code run together. A change must therefore be compatible with both. The expand-and-contract pattern splits one breaking change into compatible steps:

1. Expand: add the new column or table (nullable, no default that forces a rewrite). Old code ignores it.
2. Deploy code that writes both old and new structures and reads the old one.
3. Backfill existing rows in small batches.
4. Deploy code that reads the new structure.
5. Contract: after no running version uses the old structure, drop it.

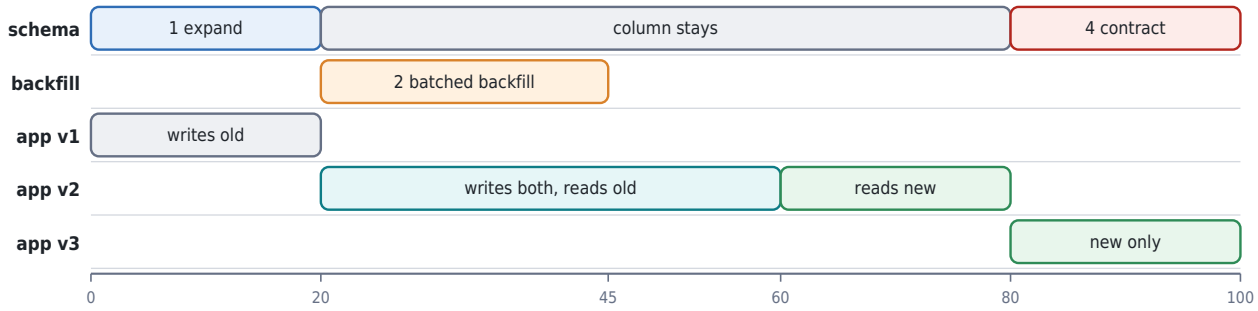


Figure 1.145. Each deployment is compatible with the schema before and after it; the destructive step comes last and only when no code needs the old column.

The same method renames a column (add the new, copy, switch, drop the old), splits a column, or changes a type. A direct `RENAME COLUMN` is instantaneous in the database but breaks every running instance that uses the old name.

1.31.2 Worked example: what each DDL statement locks

DDL statements acquire table locks, and the strongest one, `ACCESS EXCLUSIVE`, conflicts with every other lock including the `ACCESS SHARE` lock of a plain `SELECT`. The experiments below ran on `orders` with 1 000 000 rows.

Statement	PostgreSQL 16	MySQL 8.0 InnoDB
`ADD COLUMN nullable`	Metadata only (1.8 ms measured)	ALGORITHM=INSTANT (8.0.12+), metadata only
`ADD COLUMN NOT NULL DEFAULT constant`	Metadata only since PostgreSQL 11 (1.4 ms)	INSTANT
`ALTER COLUMN TYPE int to bigint`	Full table rewrite, ACCESS EXCLUSIVE (3.5 s)	Usually COPY algorithm, table rebuilt, DML blocked or online depending on the change
`CREATE INDEX`	Blocks writes for the duration (0.48 s)	Online (INPLACE) by default, concurrent DML allowed
`CREATE INDEX CONCURRENTLY`	Does not block writes; two table scans; cannot run in a transaction	Not applicable (online by default)

Behaviour of common ALTER TABLE operations (PostgreSQL 16 observed; MySQL 8.0 documented).

Adding columns

PostgreSQL stores a constant default of an added column in the catalog instead of rewriting the table. The catalog flag `atthasmissing` marks such columns:

Adding columns on one million rows (real output) TERMINAL

```
shop=# \timing on
shop=# alter table orders add column note text;
Time: 1.802 ms
shop=# alter table orders add column channel text not null default 'WEB';
Time: 1.355 ms
shop=# select attname, atthasmissing from pg_attribute
shop-#   where attrelid = 'orders'::regclass and attnum > 0 and not attisdropped;
   attname    | atthasmissing
--------------+---------------
 note         | f
 channel      | t
 amount_cents | f
```

A volatile default (`DEFAULT now()` is stable, `DEFAULT random()` is volatile) or a type change forces the rewrite. A rewrite creates a new file for the table; the file identifier changes:

Type change rewrites the table (real output) TERMINAL

```
shop=# select pg_relation_filenode('orders') before;
 16537
shop=# alter table orders alter column customer_id type bigint;
Time: 3511.160 ms (00:03.511)
shop=# select pg_relation_filenode('orders') after;
 24627
```

During those 3.5 seconds the table accepted neither reads nor writes. The safe pattern for a type change is expand and contract: add `customer_id_new bigint`, fill it in batches (`UPDATE ... WHERE id BETWEEN a AND b`), keep both in sync with a trigger or dual writes, switch the application and then drop the old column.

Indexes

A plain `CREATE INDEX` takes a `SHARE` lock: reads continue, writes wait. `CREATE INDEX CONCURRENTLY` builds the index with two table scans while writes continue; it takes longer in total (0.52 s against 0.48 s here, a larger gap on busy tables) and cannot run inside a transaction block, which means a Flyway migration needs `executeInTransaction=false` (a per-script configuration file) for it. If it fails, it leaves an invalid index that still costs write time:

A failed concurrent unique index remains INVALID (real output) TERMINAL

```
shop=# create unique index concurrently orders_cust_uq on orders(customer_id);
ERROR:  could not create unique index "orders_cust_uq"
DETAIL:  Key (customer_id)=(257) is duplicated.
shop=# select indexrelid::regclass, indisvalid from pg_index
shop-#   where indrelid = 'orders'::regclass;
    indexrelid     | indisvalid
-------------------+------------
 orders_pkey       | t
 orders_status_idx | t
 orders_cust_uq    | f
```

The invalid index must be dropped (`DROP INDEX orders_cust_uq`, or `DROP INDEX CONCURRENTLY` on a busy table) and the build repeated after the data is fixed.

Constraints without long locks

Adding a foreign key or a check constraint validates every existing row under a strong lock. The two-step form avoids that: `ALTER TABLE orders ADD CONSTRAINT chk_amount CHECK (amount >= 0) NOT VALID;` takes a brief lock and enforces the rule for new rows; `ALTER TABLE orders VALIDATE CONSTRAINT chk_amount;` then scans the table holding only a `SHARE UPDATE EXCLUSIVE` lock, which does not block reads and writes. A `NOT NULL` constraint can be added cheaply in PostgreSQL 12 and later when a validated `CHECK (col IS NOT NULL)` already exists.

1.31.3 Hands-on: the lock queue

The most dangerous property of `ACCESS EXCLUSIVE` is not its own duration but what happens while it waits. PostgreSQL grants locks in queue order: a lock request that conflicts with the waiting request also waits behind it. The experiment used three sessions on `orders`: pid 698 runs a `SELECT` in a transaction that stays open (it holds `ACCESS SHARE`), pid 708 issues `ALTER TABLE orders ADD COLUMN flag boolean` (it needs `ACCESS EXCLUSIVE` and must wait for 698), and pid 717 issues an ordinary `SELECT`.

Pile-up behind a waiting ALTER TABLE (real output) TERMINAL

```
shop=# select pid, state, wait_event_type, wait_event, pg_blocking_pids(pid) as blocked_by,
shop-#   left(query,48) as query from pg_stat_activity where datname = 'shop'
shop-#   and pid <> pg_backend_pid() order by backend_start;
 pid | state  | wait_event_type | wait_event | blocked_by |                  query
-----+--------+-----------------+------------+------------+----------------------------------------
 698 | active | Timeout         | PgSleep    | {}         | select pg_sleep(12)
 708 | active | Lock            | relation   | {698}      | alter table orders add col...
 717 | active | Lock            | relation   | {708}      | select count(*) from orders where id<10

shop=# select l.pid, l.mode, l.granted from pg_locks l
shop-#   where relation = 'orders'::regclass order by granted desc, pid;
 pid |        mode         | granted
-----+---------------------+---------
 698 | AccessShareLock     | t
 708 | AccessExclusiveLock | f
 717 | AccessShareLock     | f
```

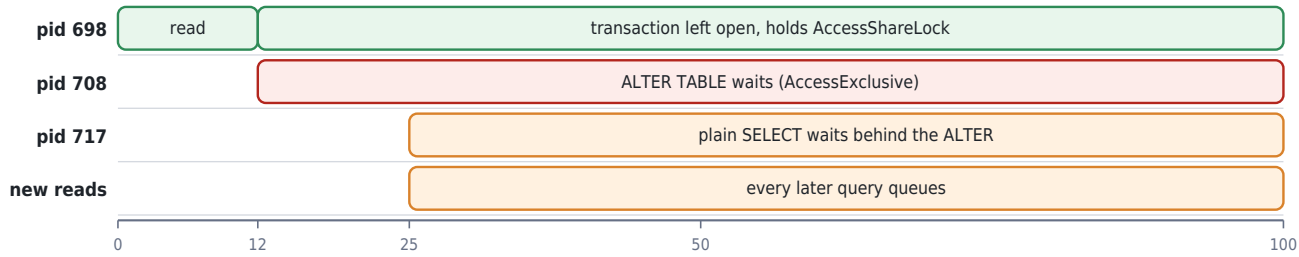


Figure 1.146. The ALTER does nothing, yet it blocks all later queries, because their ACCESS SHARE requests queue behind its pending ACCESS EXCLUSIVE request.

Session 717 asked only for a shared lock, which is compatible with the granted lock of 698, but it is blocked by 708. The statement that takes 1 ms when the table is quiet therefore stops all access to the table for as long as any older transaction runs. The defence is to bound the wait:

lock_timeout aborts a migration that cannot get its lock (real output) TERMINAL

```
shop=# set lock_timeout = '1s';
SET
shop=# alter table orders add column flag2 boolean;
ERROR:  canceling statement due to lock timeout
```

A migration script sets `lock_timeout` (for example 2 to 5 seconds) at its start and is retried when it fails; failing fast is cheaper than blocking the application. In MySQL the equivalent is `lock_wait_timeout` for metadata locks (default 31 536 000 seconds, about a year), and the same pile-up behind a pending metadata lock occurs; the statement `ALTER TABLE ... ALGORITHM=INPLACE, LOCK=NONE` makes the server refuse a change that cannot be done online instead of silently locking, which is a useful guard.

TIP
Use explicit algorithm and lock clauses in MySQL. `ALTER TABLE orders ADD COLUMN channel VARCHAR(10), ALGORITHM=INSTANT;` fails with an error if INSTANT is impossible, instead of falling back to a table copy. Tools such as gh-ost and pt-online-schema-change copy the table in the background for the changes that cannot be online.

1.31.4 Real-world problems and solutions

Incident: a migration locks a hot table

Scenario. A release adds a column to `orders` (800 requests per second). The migration runs as part of application start. A reporting job holds a transaction open for ten minutes.

Symptoms. Right after the release all order endpoints time out; the pool fills; the migration is "running" in the pipeline. Rolling back the application does not help.

Diagnosis. In order:

diagnosis.sql SQL

```sql
SELECT pid, state, wait_event_type, wait_event, pg_blocking_pids(pid) AS blocked_by,
       now() - xact_start AS xact_age, left(query, 60) AS query
FROM pg_stat_activity WHERE datname = current_database() ORDER BY xact_start;

SELECT l.pid, l.mode, l.granted FROM pg_locks l
WHERE l.relation = 'orders'::regclass ORDER BY l.granted DESC, l.pid;
```

The first query reveals the root of the blocking tree (a session with `blocked_by = {}` and a long `xact_age`, here pid 698); the second shows an ungranted `AccessExclusiveLock` of the migration and many ungranted `AccessShareLock` rows behind it.

Root cause. The `ALTER TABLE` waited for the long transaction, and every subsequent query queued behind the pending exclusive lock (the pile-up shown above).

Fix. Immediate: cancel the migration (`SELECT pg_cancel_backend(708);`), which releases the queue instantly; the long reporting transaction can then be ended or left. Permanent: `SET lock_timeout = '3s'` in migration scripts with automatic retry, run migrations in a window without long transactions, and alert on transactions older than one minute.

Prevention. `idle_in_transaction_session_timeout`, a statement timeout for reporting roles, and migrations reviewed for lock levels (a table in this chapter lists the common cases).

Incident: type change that rewrote a 400 GB table

Scenario. An identifier column is changed from `integer` to `bigint` because the sequence approaches 2 billion. The statement is `ALTER TABLE ... ALTER COLUMN id TYPE bigint`.

Symptoms. The statement runs for 40 minutes holding `ACCESS EXCLUSIVE`; the application is down; disk usage doubles temporarily because the rewrite builds a second copy.

Diagnosis. `pg_stat_progress_cluster` (for rewrites) and the changing `pg_relation_filenode`; free disk space. On the 1 000 000-row sample the rewrite took 3.5 s, and the time scales with size and with the number of indexes, which are rebuilt too.

Root cause. A type change that is not binary-compatible rewrites the table in a single exclusive operation.

Fix. Immediate: nothing shortens a started rewrite safely; cancel it if it is still early (the transaction rolls back) and plan properly. Permanent: expand and contract with a new `bigint` column, batched backfill, trigger or dual-write, switching reads, and finally dropping the old column; for a primary key the new unique index is built `CONCURRENTLY` and attached with `ADD CONSTRAINT ... USING INDEX`.

Prevention. Choose `bigint` or UUID for keys that can reach billions; monitor sequence usage (`pg_sequences`: `last_value` against the maximum).

Incident: invalid index left by a failed concurrent build

Scenario. `CREATE INDEX CONCURRENTLY` is cancelled by a statement timeout half way, or fails with a duplicate key as in the demonstration.

Symptoms. Writes to the table are slower than before although no index is used by queries; `\d table` shows an index marked `INVALID`.

Diagnosis. `SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;`.

Root cause. A concurrent build that fails does not clean up: the invalid index is still maintained on every write but is never used for reads.

Fix. Drop it and rebuild after correcting the cause. **Prevention.** Check `indisvalid` in the migration pipeline after every concurrent build.

1.31.5 Questions to expect

What is expand and contract?

A sequence of backwards-compatible schema and application changes: add the new structure, write to both, backfill, switch reads, then remove the old structure. It permits rolling deployments without downtime because old and new code both work against every intermediate schema.

Why can a fast `ALTER TABLE` take a production system down?

It needs an `ACCESS EXCLUSIVE` lock. While it waits for a running transaction, all new queries on the table queue behind it. Set `lock_timeout` and retry.

How do you add an index to a busy table in PostgreSQL?

`CREATE INDEX CONCURRENTLY`: it does not block writes, cannot run inside a transaction, and leaves an invalid index if it fails, which must be dropped.

What do Flyway checksums protect against?

Silent modification of an applied migration: the checksum of each applied file is stored, and a changed file causes validation to fail, so environments cannot drift unnoticed.

Which `ALTER TABLE` operations rewrite the table in PostgreSQL?

Changing a column type that is not binary-coercible, adding a column with a volatile default, and some operations on `SET LOGGED`. Adding a column with a constant default does not rewrite it since PostgreSQL 11.

KEY POINTS

- Version every schema change (Flyway or Liquibase) and never edit an applied migration.
- Expand and contract keeps old and new application versions working across a rolling deployment.
- `ALTER TABLE` is dangerous mostly because of lock queueing; use `lock_timeout` and retry.
- Metadata-only changes are milliseconds; type changes rewrite the table and hold an exclusive lock for its duration.
- Build indexes with `CREATE INDEX CONCURRENTLY` and verify `indisvalid` afterwards.

1.32 Database Security: Roles, Privileges and Row-Level Security

The database is the last line of defence for the data. An application that connects as a superuser, or as the owner of every table, turns any defect in the application (an injection, a leaked credential, an over-broad endpoint) into a total loss of the data. Security in the database layer rests on three ideas: every actor has a separate identity, each identity holds the minimum privileges for its task, and the connection and the data are protected against interception and theft.

1.32.1 Concept and mechanism

Roles and privileges

PostgreSQL has one concept, the role. A role with the `LOGIN` attribute is a user; a role without it is a group that can be granted to other roles. Privileges (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `REFERENCES`, `TRIGGER` on tables; `USAGE` on schemas and sequences; `EXECUTE` on functions; `CONNECT` on databases) are granted with `GRANT` and removed with `REVOKE`. The owner of an object holds all privileges implicitly, and superusers bypass all checks. MySQL separates users (`'app'@'10.0.%'`, identity includes the host pattern) from roles (introduced in 8.0) and uses the same statements with a different scope syntax (`GRANT SELECT ON shop.* TO ...`).

Least privilege means: the migration tool connects as the schema owner and is used only during deployments; the application connects as a role that can read and write the tables it needs but cannot alter the schema; people and reporting tools connect with read-only roles. PostgreSQL grants `CREATE` on the `public` schema to everyone in versions before 15 and `USAGE` to everyone in all versions, so the first hardening step is `REVOKE ALL ON SCHEMA public FROM PUBLIC`.

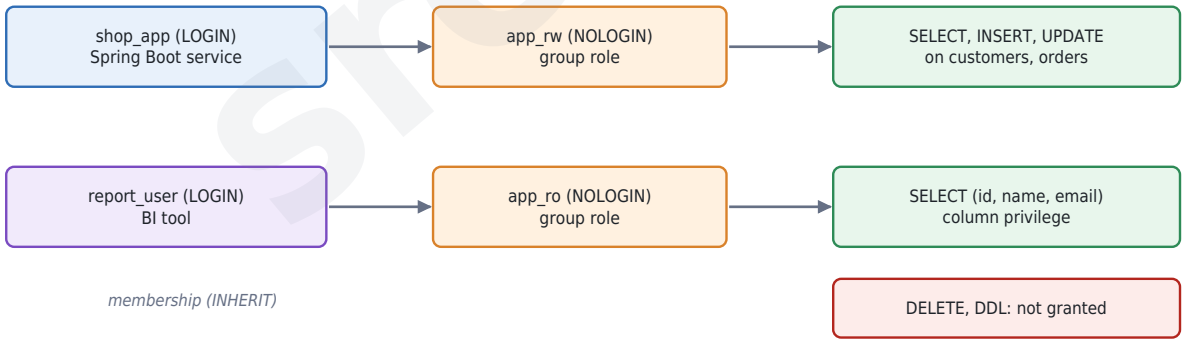


Figure 1.147. Login roles inherit from group roles, and the group roles hold the table and column privileges; deletes and DDL are granted to nobody in the application path.

1.32.2 Worked example: group roles, column privileges

The following script built the structure of the diagram on the sample database.

roles.sql SQL

```sql
CREATE ROLE app_rw NOLOGIN;
CREATE ROLE app_ro NOLOGIN;
CREATE ROLE report_user LOGIN PASSWORD 'x' IN ROLE app_ro;
CREATE ROLE shop_app    LOGIN PASSWORD 'secret' IN ROLE app_rw;

REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO app_rw, app_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_ro;
GRANT SELECT, INSERT, UPDATE ON customers, orders TO app_rw;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_rw;
```

The reporting login can read but neither delete nor write:

Privilege checks as report_user (real output) TERMINAL

```
$ psql -U report_user shop
shop=> select count(*) from customers;
 1004
shop=> delete from orders where id = 1;
ERROR:  permission denied for table orders
```

The same user could still read the `password` column of `customers`, which is a defect in the model. Column-level privileges close it: the table-wide `SELECT` is revoked and only three columns are granted.

Column privileges (real output) TERMINAL

```
shop=# revoke select on customers from app_ro;
shop=# grant select (id, name, email) on customers to app_ro;
$ psql -U report_user shop -c "select * from customers limit 1"
ERROR:  permission denied for table customers
$ psql -U report_user shop -c "select id, name from customers limit 1"
 id |    name
----+------------
  1 | Customer 1
```

`SELECT *` fails because it includes forbidden columns; this is the correct behaviour but it is a surprise for ORMs that map all columns, so a view with only the permitted columns (`CREATE VIEW customers_public AS SELECT id, name, email FROM customers`) is often a better interface for reporting. Passwords must never be stored in plain text at all; the demonstration data used plain values only to make the injection example visible. Application passwords are stored as slow salted hashes (bcrypt, Argon2) created in the application.

1.32.3 Hands-on: row-level security

Column privileges restrict which attributes a role sees; row-level security (RLS) restricts which rows. It is the database-side enforcement of tenant isolation in multi-tenant systems: even a query with a forgotten `WHERE tenant = ?` returns only the rows of the current tenant.

rls.sql SQL

```sql
CREATE TABLE tenant_orders (
  id serial PRIMARY KEY, tenant text NOT NULL, item text);
INSERT INTO tenant_orders(tenant, item) VALUES
  ('acme','bolt'), ('acme','nut'), ('globex','gear'), ('globex','cog'), ('globex','axle');
GRANT SELECT, INSERT ON tenant_orders TO app_rw;
GRANT USAGE ON SEQUENCE tenant_orders_id_seq TO app_rw;

ALTER TABLE tenant_orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON tenant_orders
  USING      (tenant = current_setting('app.tenant', true))
  WITH CHECK (tenant = current_setting('app.tenant', true));
```

`USING` filters the rows visible to `SELECT`, `UPDATE` and `DELETE`; `WITH CHECK` validates rows written by `INSERT` and `UPDATE`. `current_setting('app.tenant', true)` reads a session variable; the second argument makes it return NULL instead of an error when unset, so that an unset tenant matches no rows.

Behaviour for the application role (real output) TERMINAL

```
$ psql -U shop_app shop
shop=> select count(*) as without_setting from tenant_orders;
 0
shop=> set app.tenant = 'acme';
shop=> select * from tenant_orders;
 id | tenant | item
----+--------+------
  1 | acme   | bolt
  2 | acme   | nut
shop=> insert into tenant_orders(tenant,item) values ('globex','stolen');
ERROR:  new row violates row-level security policy for table "tenant_orders"
shop=> insert into tenant_orders(tenant,item) values ('acme','washer') returning id;
  7
```

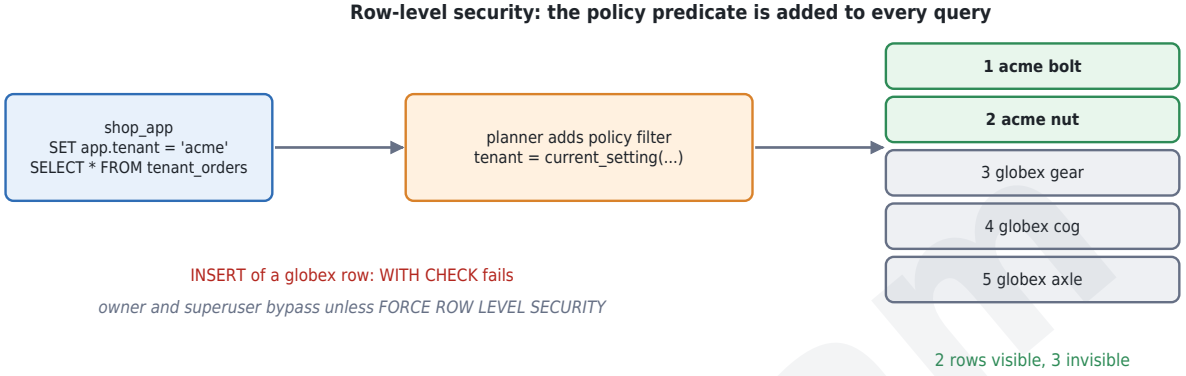


Figure 1.148. The policy is appended to every query as a filter, and writes of a foreign tenant fail the check.

The plan shows the mechanism: for the table owner, which is exempt, the plan is a plain scan; for the application role the policy appears as a filter.

Owner bypass and the policy filter in the plan (real output) TERMINAL

```
shop=# select count(*) as owner_sees from tenant_orders;
 6
shop=# select relname, relrowsecurity, relforcerowsecurity from pg_class
shop-#   where relname = 'tenant_orders';
    relname    | relrowsecurity | relforcerowsecurity
---------------+----------------+---------------------
 tenant_orders | t              | f
shop=> explain (costs off) select * from tenant_orders;     -- as shop_app, tenant set
 Seq Scan on tenant_orders
   Filter: (tenant = current_setting('app.tenant'::text, true))
```

In a Spring application the tenant variable must be set on every borrowed connection, and cleared when it is returned: `SET LOCAL app.tenant = ?` inside each transaction (it lasts until the end of the transaction) is safe with pooling, whereas a session-level `SET` leaks into the next request that receives the connection. The sequence value 7 in the output above, after the failed insert that consumed 6, shows that sequence values are not returned on failure.

WATCH OUT
Owners and superusers bypass row-level security. The table owner is exempt unless `ALTER TABLE ... FORCE ROW LEVEL SECURITY` is set, and superusers and roles with `BYPASSRLS` are always exempt. An application that connects as the owner (the common mistake of using the migration account at runtime) has no row-level protection at all.

Encryption and auditing

Encryption in transit: PostgreSQL enables TLS with `ssl = on` and certificate files; clients should use `sslmode=verify-full` (JDBC: `sslmode=verify-full` with `sslrootcert`), which verifies both the certificate chain and the host name; `require` encrypts but does not authenticate the server. In `pg_hba.conf`, `hostssl` lines accept only TLS connections and `scram-sha-256` is the recommended password method. MySQL uses `require_secure_transport=ON` on the server. Encryption at rest: PostgreSQL has no built-in transparent data encryption in community releases, so volume or filesystem encryption (LUKS, cloud disk encryption) is used, with column-level encryption through `pgcrypto` for specific secrets; MySQL InnoDB supports tablespace encryption with a keyring. Backups need the same protection as the live data. These settings were not exercised on the build machine, which uses local sockets; they are listed from the documentation.

Auditing records who did what. PostgreSQL logs statements with `log_statement = 'ddl'` or `'mod'` and, for a structured audit trail, the `pgaudit` extension; MySQL Enterprise Edition has an audit plugin and community deployments use the general log sparingly. Logs must contain the user and application name (`log_line_prefix = '%m [%p] %u@%d %a '`) and must not contain the values of sensitive parameters.

1.32.4 Real-world problems and solutions

Incident: application account with owner privileges exposes the schema

Scenario. After the SQL injection incident of the earlier chapter, the post-mortem finds that the service connects with the migration user, who owns every table.

Symptoms. The attacker's `DROP TABLE` statement succeeded (as in the injection demonstration), and `SELECT` of the password column returned hashes.

Diagnosis. List privileges and owners: `SELECT tablename, tableowner FROM pg_tables WHERE schemaname = 'public';` and `\dp` in `psql`; check the role of the service with `SELECT current_user, session_user;` and `SELECT rolsuper, rolbypassrls FROM pg_roles WHERE rolname = current_user;`.

Root cause. One credential for migration and runtime; the owner implicitly holds every privilege and is exempt from row-level security.

Fix. Create `shop_app` with only the privileges of the roles above, a separate `shop_migrator` owner, rotate the leaked credential. **Prevention.** Privilege checks in CI (a test that connects as the application role and verifies that `DROP TABLE` fails), secrets in a vault, short-lived credentials.

Incident: tenant data leaks through a pooled session variable

Scenario. RLS depends on `SET app.tenant = 'acme'`. The Spring filter sets it with a session-level `SET` and does not reset it.

Symptoms. Occasionally a user sees another tenant's rows; the effect is not reproducible in tests.

Diagnosis. On a borrowed pooled connection, `SHOW app.tenant;` returns the value of the previous request.

Root cause. Session state survives on a pooled connection. **Fix.** `SET LOCAL` within the transaction (or `set_config('app.tenant', ?, true)`), and a test that executes two tenants' requests on one connection. **Prevention.** Never rely on session state across pool borrows; HikariCP provides no automatic reset beyond transaction state.

1.32.5 Questions to expect

How do you apply least privilege to a Spring Boot service?

Separate roles for migration (owner) and runtime; the runtime role gets only `SELECT/INSERT/UPDATE` (and `DELETE` where needed) on the tables it uses, `USAGE` on sequences, no DDL; group roles for reporting with column privileges or views.

What is row-level security and when does it fail?

A per-table predicate the database adds to queries. It fails to protect when the connection role is the owner (without `FORCE`), a superuser or has `BYPASSRLS`, and when session variables leak across pooled connections.

`sslmode=require` versus `verify-full`?

`require` encrypts but accepts any certificate, so a man in the middle can succeed; `verify-full` validates the certificate chain and host name.

Why do parameterised queries not replace least privilege?

They prevent one class of injection; least privilege limits the damage of every other defect and of stolen credentials.

KEY POINTS

- Use separate identities for migrations, the application, people and reports; grant only what each needs.
- Column privileges and views protect attributes; row-level security protects rows and fails for owners and superusers unless forced.
- Session variables are unsafe on pooled connections unless set with `SET LOCAL` inside the transaction.
- Use TLS with `verify-full`, encrypt volumes and backups, and keep an audit trail without sensitive values.

1.33 Monitoring and Diagnosing a Slow Database

When the database is slow, the cost of a wrong guess is high: restarting a server that was merely blocked destroys the evidence, and adding an index to a system whose problem is lock contention wastes the incident window. A disciplined diagnosis names the bottleneck from cheap evidence first. This chapter lists the metrics worth watching, the queries that produce them, a decision tree, and the first five minutes of an incident.

1.33.1 Concept and mechanism

What to measure

Area	Metric	Source
Saturation	Connections used versus max_connections, pool wait	`pg_stat_activity`, HikariCP metrics
Concurrency	Sessions by state, `idle in transaction`, lock waits	`pg_stat_activity`, `pg_locks`, `pg_blocking_pids`
Throughput	Commits, rollbacks, deadlocks	`pg_stat_database`
Cache	Buffer hit ratio	`pg_stat_database` (blks_hit, blks_read)
Queries	Total time, calls, mean time per statement	`pg_stat_statements` (extension, needs `shared_preload_libraries`)
Replication	Replay lag, slot retained bytes	`pg_stat_replication`, `pg_replication_slots`
Storage	Disk and WAL directory size, dead tuples, archive failures	OS, `pg_ls_waldir()`, `pg_stat_user_tables`, `pg_stat_archiver`

Metrics and their sources in PostgreSQL.

MySQL equivalents: `SHOW PROCESSLIST` and `performance_schema.threads`, `SHOW ENGINE INNODB STATUS`, `sys.statement_analysis`, `performance_schema.events_statements_summary_by_digest`, `SHOW REPLICA STATUS` (`Seconds_Behind_Source`), and the slow query log (`slow_query_log`, `long_query_time`), typical, not run here.

1.33.2 Worked example: reading the first screen

A blocked update was staged (one session holding a row lock inside an open transaction, a second updating the same row), and the standard first queries were run. These queries are worth memorising because they answer the first three questions of any incident: how many sessions and in which state, who waits for whom, and how healthy is the cache.

Sessions by state and the oldest transactions (real output) TERMINAL

```
shop=# select state, count(*) from pg_stat_activity
shop-#   where backend_type = 'client backend' group by 1 order by 2 desc;
 state  | count
--------+-------
 active |     3

shop=# select pid, now()-xact_start as xact_age, state,
shop-#   wait_event_type||':'||wait_event as wait, left(query,40) q
shop-#   from pg_stat_activity where datname='shop' and pid<>pg_backend_pid() order by xact_start;
 pid  |    xact_age     | state  |        wait        |                    q
------+-----------------+--------+--------------------+---------------------------
 3874 | 00:00:02.032631 | active | Timeout:PgSleep    | select pg_sleep(9)
 3875 | 00:00:02.030803 | active | Lock:transactionid | update customers set name=n

shop=# select pid, pg_blocking_pids(pid) blockers from pg_stat_activity
shop-#   where cardinality(pg_blocking_pids(pid)) > 0;
 pid  | blockers
------+----------
 3875 | {3874}
```

Session 3875 waits for a transaction id lock held by 3874: a row lock conflict, visible as `wait_event_type = Lock`. The chain of `blockers` is followed until a pid with no blocker is found, and that session is the one to examine (it may be idle in a transaction).

Database-level counters including the cache hit ratio (real output) TERMINAL

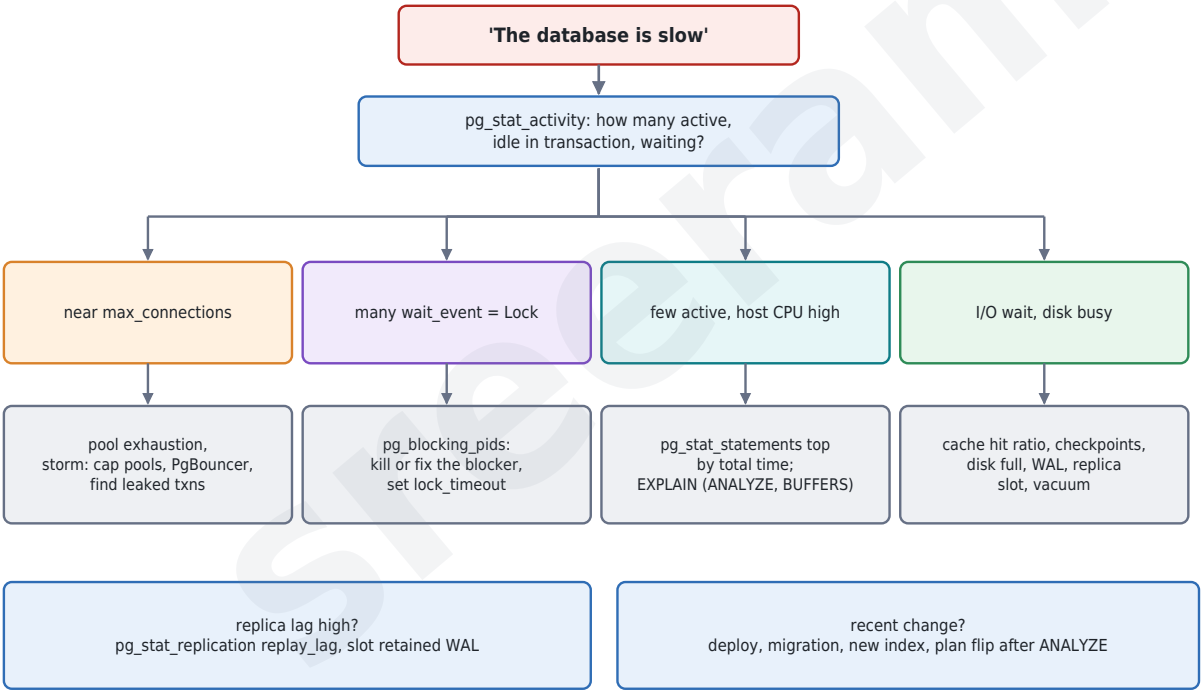
```
shop=# select datname, numbackends, xact_commit, xact_rollback, blks_hit, blks_read,
shop-#   round(100.0*blks_hit/nullif(blks_hit+blks_read,0),2) as hit_pct, deadlocks
shop-#   from pg_stat_database where datname='shop';
 datname | numbackends | xact_commit | xact_rollback | blks_hit | blks_read | hit_pct | deadlocks
---------+-------------+-------------+---------------+----------+-----------+---------+-----------
 shop    |           3 |         149 |             7 |  6909062 |     57694 |   99.17 |         0
```

A hit ratio above 99 percent on an OLTP workload is normal; a sudden fall indicates a working set larger than memory or a new scan-heavy query. The column `temp_files` (not shown; 14 in this database) counts spills of sorts and hashes to disk, a sign that `work_mem` is too small for some query. The counters are cumulative since the statistics reset: the rate of change matters, so monitoring systems store deltas.

Disk and archiving checks (real output) TERMINAL

```
shop=# select pg_size_pretty(sum(size)) wal_dir from pg_ls_waldir();
 336 MB
shop=# select archived_count, last_archived_wal, failed_count from pg_stat_archiver;
 archived_count | last_archived_wal        | failed_count
----------------+--------------------------+--------------
             20 | 000000010000000000000031 |            0
```

1.33.3 The decision tree and the first five minutes



Check the cheapest evidence first; change nothing until the bottleneck is named.

Figure 1.149. The first branch is the session picture: saturation, lock waits, CPU-bound queries or I/O; each branch has its own next query.

The tree reduces to four branches. If sessions are near `max_connections` or the pool is waiting, the problem is concurrency: look for idle-in-transaction sessions, leaked connections and storms. If many sessions wait on locks, find the root blocker with `pg_blocking_pids` and either end it or fix its transaction. If few sessions are active but CPU is high, query cost is the problem: sort `pg_stat_statements` by `total_exec_time`, then examine the top statement with `EXPLAIN (ANALYZE, BUFFERS)`. If CPU is low and I/O wait is high, check the cache hit ratio, checkpoints, a full disk, a vacuum storm, WAL growth and replication lag. Side branches are always asked: did something change (deployment, migration, new index, statistics refresh), and is a replica behind?

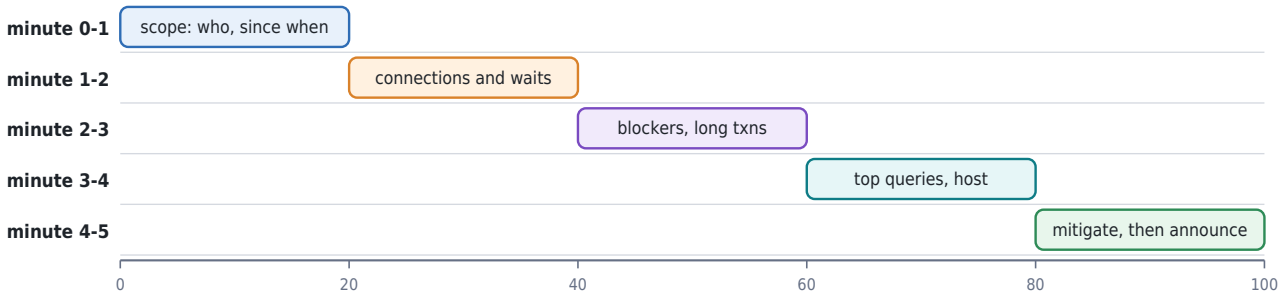


Figure 1.150. A fixed sequence for the first five minutes keeps the responder from changing the system before the cause is known.

First five minutes: checklist

1. Scope (minute 0 to 1): which endpoints, since when, which instances; correlate the start with deployments and scheduled jobs. Open the incident channel and name one coordinator.
2. Sessions (minute 1 to 2): run the session-by-state query and compare the total with `max_connections`.
3. Blockers (minute 2 to 3): run the blocking query and note the root blocker, its `xact_age` and query text.
4. Queries and host (minute 3 to 4): `pg_stat_statements` top five by total time, host CPU, memory, disk free and I/O wait, WAL directory size.
5. Mitigate and communicate (minute 4 to 5): the smallest reversible action: cancel the blocker with `pg_cancel_backend(pid)` (terminate with `pg_terminate_backend` only if cancel fails), roll back the last deployment, scale pools down, drop the abandoned slot. Record what was changed and when.

TIP

Set guard rails before the incident. `idle_in_transaction_session_timeout = '60s'`, `statement_timeout` for web roles, `lock_timeout` for migration roles, `log_min_duration_statement = 500`, `log_lock_waits = on`, and `deadlock_timeout` at its default of 1 s. None of these were set in the demonstration cluster (the settings query showed zeros), which is the default and the reason a blocked session can wait forever.

1.33.4 Real-world problems and solutions

Incident: the database is slow after a release

Scenario. After a release at 14:05, p95 latency rises from 40 ms to 2 s. CPU of the database host is 30 percent.

Symptoms. No errors, but pool wait time rises; `pg_stat_activity` shows 18 of 20 pool connections `active` and most waiting.

Diagnosis. Walk the tree. Sessions: pool nearly full, database below `max_connections`. Wait events: `Lock:relation` for a table, and one `idle in transaction` session of 4 minutes (the root blocker), originating from a new code path that calls an HTTP service inside `@Transactional`.

Root cause. A transaction held across a remote call keeps row locks; others queue; the pool fills.

Fix. Immediate: cancel the blocker, disable the feature flag. Permanent: move the remote call outside the transaction, set `idle_in_transaction_session_timeout`, `spring.datasource.hikari.leak-detection-threshold`. **Prevention.** A review rule on transaction scope and the alert on the oldest transaction age.

Incident: disk full on the database volume

Scenario. The volume holding the data directory reaches 100 percent at 03:10.

Symptoms. Writes fail with `could not extend file ... No space left on device`; the server may stop with a PANIC when `pg_wal` cannot grow.

Diagnosis. Measure which directory grew: `du -sh $PGDATA/pg_wal $PGDATA/base`, `pg_ls_waldir()`, slots (`pg_replication_slots` as in the replication chapter), archive failures (`pg_stat_archiver`), temporary files (`temp_files`, `log_temp_files`), and table and index size (`pg_total_relation_size`).

Root cause. The common causes are an inactive replication slot, a failing archive command, a runaway query spilling temporary files, and unbounded table growth.

Fix. Immediate: free space safely (drop the abandoned slot; never delete files from `pg_wal` by hand), extend the volume. Permanent: alerts at 70 and 85 percent, `max_slot_wal_keep_size`, `temp_file_limit`, partition-based retention.

Prevention. Capacity dashboards with a growth forecast.

Incident: slow restore blocks recovery of the service

This incident is described in the chapter on backups; its monitoring side is a recorded restore duration compared with the RTO.

1.33.5 Questions to expect

A web application reports that the database is slow. What do you check first?

Sessions by state and wait events in `pg_stat_activity`, compared with `max_connections`; then blocking chains with `pg_blocking_pids`; then top statements by total time; then host I/O, disk space and replication lag. Name the bottleneck before changing anything.

How do you find the session that blocks the others?

Follow `pg_blocking_pids(pid)` from a waiting session until a session with an empty list is found; check its state, `xact_start` and query. Cancel it with `pg_cancel_backend`.

What is the buffer cache hit ratio and what does a low value mean?

`blks_hit / (blks_hit + blks_read)`. Above 99 percent is typical for OLTP; a low value indicates a working set larger than memory or large scans.

Which timeouts should be configured?

`statement_timeout`, `lock_timeout`, `idle_in_transaction_session_timeout` per role, plus pool `connectionTimeout`.

KEY POINTS

- Diagnose by evidence: sessions and waits first, then blockers, then queries, then host resources and replication.
- `pg_stat_activity`, `pg_locks`, `pg_stat_database`, `pg_stat_statements`, `pg_stat_replication` and `pg_stat_archiver` answer most incident questions.
- Counters are cumulative; monitor rates and set alerts on saturation, the oldest transaction, replication lag, slot retention and disk.
- Make the smallest reversible change first, and record it.

1.34 Reference Architecture for Scaling and Operations

The previous chapters of this part each treated one mechanism. A production system uses them together, and the failure of one is often caused by the configuration of another: a pool that is too large causes a connection storm, a storm triggers a failover, the failover creates a replica lag, and the lag causes stale reads. This short chapter connects the pieces with one reference architecture and a review checklist, and ends with the interview-oriented summary of the whole part.

1.34.1 Reference architecture for a Spring Boot service

A typical medium-size deployment has the following elements, each tied to a decision made earlier.

Element	Configuration	Reason
Application pools	HikariCP, 4 to 10 connections per pod, `connectionTimeout` 5 s	Database connection budget divided among pods
Pooler	PgBouncer (transaction mode) when pods exceed the budget	Stable server connection count, deployment storms absorbed
Primary	Synchronous standby (`ANY 1` of two)	No data loss on failover, no blocking from one failure
Read replicas	Lag-tolerant reads only, `readOnly` routing	Read capacity with defined staleness

Element	Configuration	Reason
WAL archive and base backups	`archive_command` to object storage, weekly base backup, daily restore test	PITR with a measured RTO
Partitioning	Monthly range partitions for event tables	Retention by drop, pruned queries
Migrations	Flyway, `lock_timeout` 3 s, expand and contract	Zero-downtime schema change
Cache	Cache-aside with TTL jitter and single flight	Read offload without stampedes
Security	Separate migrator and runtime roles, TLS `verify-full`, RLS for tenants	Limited blast radius
Monitoring	Connections, oldest transaction, lag, slot retention, disk, archive failures	The five minutes of the decision tree

Elements of the data layer and the mechanism that justifies them.

1.34.2 Review checklist

1. Is the sum of all pools below `max_connections` including deployment surge and administrative sessions?
2. Does every statement that includes user input use bind parameters, and does the runtime role lack DDL and ownership?
3. Which reads tolerate replica lag, and how does the code route the others?
4. Has the last backup been restored, with the time recorded against the RTO, and is `pg_stat_archiver` clean?
5. Does every migration set `lock_timeout`, avoid table rewrites, and build indexes concurrently?
6. Is there an alert for inactive replication slots, the age of the oldest transaction, and WAL or disk usage?
7. Does the system start with an empty cache without overloading the database?

1.34.3 Questions to expect (part summary)

Walk through what happens to a write from a Spring service to a durable standby.

The request thread borrows a pooled connection, the `@Transactional` method sends parameterised statements, and `COMMIT` flushes WAL on the primary; the walsender streams it to the standby, which flushes it (synchronous setups wait for this acknowledgement) and replays it later. A read on the replica may therefore not yet see the write.

The database has high CPU and many identical queries. What are the candidates?

A cache stampede, an N+1 pattern from lazy loading, a missing index on a hot query, or retries after timeouts. `pg_stat_statements` ranks by total time and calls; compare with the cache miss rate and with the number of requests.

How would you make a risky migration safe?

Split it with expand and contract, set `lock_timeout`, avoid rewrites, build indexes concurrently, backfill in batches, test on a copy at production scale, and have a rollback that is a separate deployment.

How do you prove a backup works?

Restore it into a scratch cluster, run integrity checks and row-count comparisons, and record the elapsed time; for PITR, restore to a chosen target and verify the data at that time.

KEY POINTS

- Scaling mechanisms interact: pool sizes, replication lag, failover and caches must be designed as one system.
- Every guarantee in the architecture should have a measurement: the connection budget, the replica lag, the restore time, the stampede test.
- Operate with guard rails: timeouts, alerts on saturation and lag, least-privilege roles and rehearsed procedures.

Sieran

PART 2

Non-Relational Databases (NoSQL)

Data models, distribution and storage designs that trade some relational guarantees for scale, flexibility or specialised access paths.

In this part

- **2.1** Why NoSQL
- **2.2** Distributed data fundamentals
- **2.3** Storage engines: B-tree and LSM-tree
- **2.4** Key-value stores
- **2.5** Wide-column stores
- **2.6** Graph databases
- **2.7** Search and time-series stores
- **2.8** Choosing a database
- **2.9** MongoDB Architecture and the Document Model
- **2.10** CRUD in Depth
- **2.11** Query Operators, Arrays, and Embedded Documents
- **2.12** Schema Design for MongoDB
- **2.13** The Aggregation Framework
- **2.14** Indexes and explain in MongoDB
- **2.15** MongoDB Transactions and Consistency
- **2.16** Replication in MongoDB
- **2.17** Sharding in MongoDB
- **2.18** MongoDB Storage and Operations
- **2.19** Spring Data MongoDB
- **2.20** Relational versus Non-Relational: A Head-to-Head Comparison
- **2.21** Interview Question Bank
- **2.22** Cheat Sheets
- **2.23** Six-Week Study Plan
- **2.24** Putting It Together: One Order Through the Whole Data Layer

2.1 Why NoSQL

The first Part of this book treats the relational model as the default: a fixed schema, declarative queries, joins, and ACID transactions on a single server. For most applications this default remains correct. A relational database such as PostgreSQL 16 on a modern machine serves tens of thousands of simple queries per second and stores terabytes. The term NoSQL covers a family of systems that exist because a minority of workloads exceed what one server can do, or have a data shape that a relational schema represents awkwardly, or require a guarantee (for example availability during a network partition) that a single-primary relational deployment cannot provide.

NoSQL is not a single technology and the name is historically misleading. The label originated in 2009 as a hashtag for a meetup about distributed, non-relational stores. It is commonly read as "not only SQL": several systems in this Part, such as Cassandra (CQL), Elasticsearch (SQL plugin) and DynamoDB (PartiQL), offer a SQL-like language, and several relational systems (PostgreSQL with JSONB, MySQL with JSON columns) store documents. The useful distinction is not the query language but the data model and the set of guarantees the engine commits to.

This chapter states the problems that motivated NoSQL systems, classifies the families, compares them with relational databases on concrete axes, and ends with the architecture that most production systems adopt in practice: one primary relational store plus a small number of specialised stores.

sreeram

2.1.1 Concept and mechanism

The limits of a single node

A single database server is bounded by four resources: CPU cores, memory, storage capacity and I/O bandwidth, and by one further limit: the server is a single failure domain. Vertical scaling (a larger machine) is the first remedy and is frequently sufficient, because hardware capacity grows faster than most workloads. Two ceilings eventually appear.

1. **Capacity ceiling.** The largest practical machine has a finite number of cores, a finite amount of RAM and a finite write throughput to one log. The cost curve is also non-linear: the price of the largest machine is many times the price of two machines with half its capacity.
2. **Availability ceiling.** A single primary with an asynchronous standby can lose recent writes on failover, and a standby promotion takes seconds to minutes. A multi-region active-active write path cannot be built on a single-primary design without conflict handling.

Horizontal scaling (adding machines) removes the first ceiling only if the data and the load can be divided. Relational databases can be sharded manually or by tools, but the relational features that make the model attractive (joins across any tables, foreign keys, multi-row transactions with a single serialisable order) rely on all data being reachable from one place. Dividing the data forces each of these features to become either more expensive (a distributed transaction requires a commit protocol across machines) or unavailable (a join across shards must move data over the network).

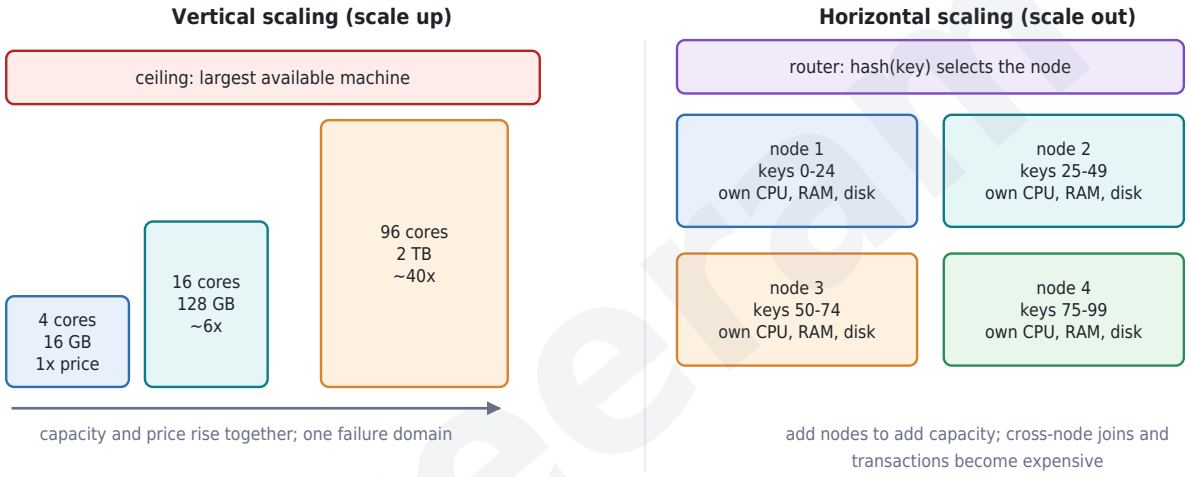


Figure 2.1. Vertical scaling raises one machine's limits at a rising price and keeps one failure domain; horizontal scaling divides the keys across machines, which is cheap per unit of capacity but restricts the operations that remain efficient.

What relational databases give up when distributed

The following relational capabilities depend on a single serialisation point. Distributed NoSQL systems typically weaken or remove them in exchange for scale and availability.

Capability	Why a single node provides it cheaply	Cost when data is partitioned
Join on any column	All tables share one buffer pool and one planner	Rows must be shipped between nodes; joins are limited to co-located keys
Foreign key enforcement	Constraint check is a local index lookup	The referenced row may live on another node; check needs a network round trip and a lock
Multi-row ACID transaction	One WAL and one lock manager decide commit order	Two-phase commit across nodes; blocking on coordinator failure; higher latency
Secondary index on any column	Index and table are updated in one transaction	Index is either local (query fans out to all partitions) or global (write touches a second node)
Strong consistency on read	Reads go to the single primary	Reads must contact a quorum or the leader, adding latency and reducing availability
Ad-hoc query	Planner chooses any access path	Efficient queries are those that name the partition key; others scan every partition

Relational capability and its cost in a partitioned system.

The four workload pressures

Four pressures motivate the departure from the relational default. A workload that has none of them should remain on a relational database.

1. **Volume and write rate.** A sustained write rate or data size that one server cannot absorb: event streams, sensor readings, click logs, messaging. The workload usually appends and rarely updates.
2. **Access pattern known in advance and simple.** Every read is a lookup by key or a short range under one key (session by id, product by sku, timeline by user). Such a pattern does not need a general query planner and can be served by hash partitioning.
3. **Schema variability.** Records of the same kind carry different attributes (product catalogues, content management, device telemetry from many firmware versions). A fixed schema either needs many nullable columns or an entity-attribute-value table.
4. **Specialised retrieval.** The primary operation is not row selection by predicate: traversal of relationships (graph), relevance-ranked text search (search engine), or aggregation over time buckets (time series).

2.1.2 Families of NoSQL databases

The families differ in the unit of storage and the way data is addressed. Figure order below follows increasing structure of the stored value.

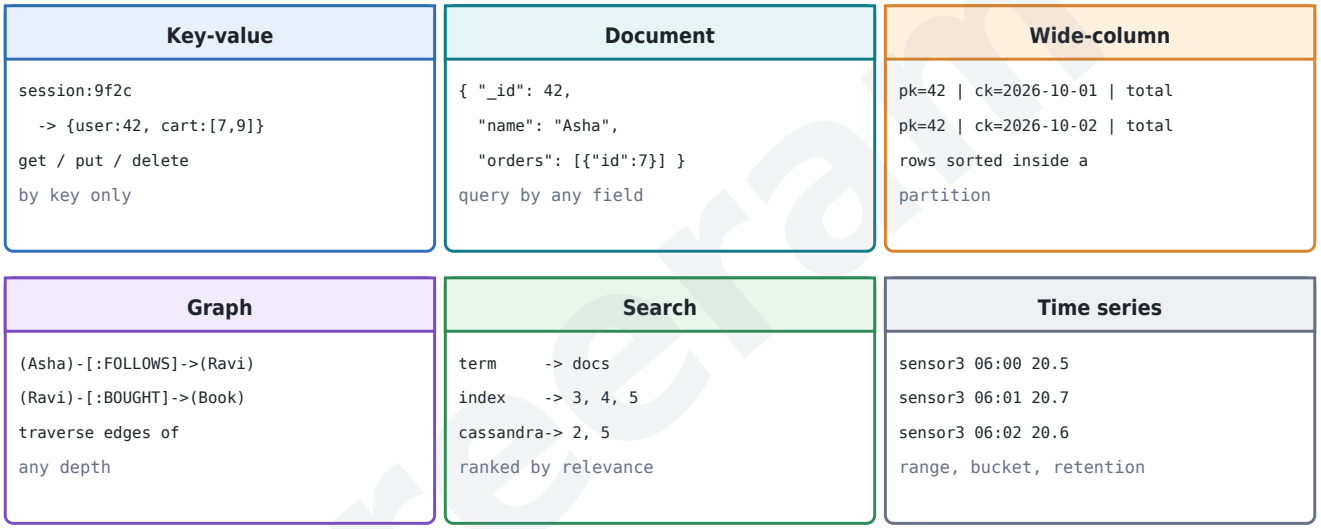


Figure 2.2. Five families classified by the shape of the stored value and the way it is addressed; each box shows one example record.

Family	Data model	Typical access	Representative systems
Key-value	Opaque value (or small typed map) addressed by key	get, put, delete by key; some stores add range scan	Redis, DynamoDB, Riak, etcd
Document	JSON-like nested document in a collection, with indexes on fields	Query by field values; update parts of a document	MongoDB, Couchbase, Firestore
Wide-column	Rows grouped under a partition key, sorted by a clustering key, sparse columns	Read a partition or a slice of it	Cassandra, HBase, ScyllaDB, Bigtable
Graph	Nodes and typed edges, both with properties	Traverse relationships of variable depth	Neo4j, Amazon Neptune, JanusGraph
Search	Inverted index over analysed text plus stored fields	Full-text and faceted queries ranked by relevance	Elasticsearch, OpenSearch, Solr
Time series	Timestamped measurements grouped by series	Range scans, down-sampling, retention	InfluxDB, TimescaleDB, Prometheus

The families and representative products.

Real products often straddle families. MongoDB offers a document model with secondary indexes and, since version 4.0, multi-document transactions. DynamoDB stores documents in values but is addressed like a wide-column store

(partition key and sort key). PostgreSQL stores documents in JSONB columns and graph data in ordinary tables. The classification describes the primary design centre, not an exclusive capability.

2.1.3 SQL and NoSQL compared

The comparison below describes tendencies, not rules; each row has exceptions in specific products.

Axis	Relational (PostgreSQL, MySQL)	Typical NoSQL (Cassandra, DynamoDB, MongoDB)
Schema	Declared before data; enforced by the engine	Implicit in the application; optionally validated (MongoDB $jsonSchema)
Data layout	Normalised across tables; joined at query time	Denormalised around the access pattern; duplicated data accepted
Query language	SQL, ad hoc, planner chooses the path	Key-oriented API or restricted query language; model the table per query
Scaling model	Vertical first; read replicas; sharding is added later	Horizontal from the design; partitioning by key is built in
Transactions	Multi-statement ACID, serialisable available	Single-item atomicity by default; limited or optional multi-item transactions
Consistency	Strong on the primary	Tunable or eventual; chosen per operation
Failure handling	Failover to a standby; brief unavailability	Replicas accept requests during node loss; conflicts resolved after the fact
Join and aggregate	First-class	Absent, limited, or done in the application or an analytics engine
Maturity of tooling	Decades; uniform ecosystem	Product-specific tools and operational practices

Relational and NoSQL systems compared on common axes.

The central difference is the direction of the design. In the relational approach the designer models the data and defers the queries; the engine finds an access path for any query. In the NoSQL approach the designer lists the queries first and shapes the data so that each query touches one partition. A change in the access pattern in a relational system usually needs a new index; in a partitioned NoSQL system it can need a new table holding a second copy of the data.

2.1.4 Worked example

The following example shows the same requirement, "show the 20 most recent orders of a customer", in both approaches, and the point at which the approach diverges.

Relational design

orders_relational.sql — SQL

```sql
CREATE TABLE customer (
  id    bigint PRIMARY KEY,
  email text   NOT NULL UNIQUE
);
CREATE TABLE orders (
  id          bigint PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customer(id),
  placed_at   timestamptz NOT NULL,
  total_cents bigint NOT NULL
);
CREATE INDEX orders_customer_time ON orders (customer_id, placed_at DESC);

SELECT id, placed_at, total_cents
FROM orders
WHERE customer_id = 42
ORDER BY placed_at DESC
LIMIT 20;
```

One B-tree index serves the query, the foreign key guarantees that every order has a customer, and a later requirement such as "orders above 100 in March across all customers" needs only a different index or a sequential scan. The design stays valid while the table fits on one server.

Wide-column design for the same query (Cassandra CQL, typical, not run)

orders_cassandra.cql — SQL

```sql
CREATE TABLE orders_by_customer (
  customer_id bigint,
  placed_at   timestamp,
  order_id    bigint,
  total_cents bigint,
  PRIMARY KEY ((customer_id), placed_at, order_id)
) WITH CLUSTERING ORDER BY (placed_at DESC, order_id ASC);

SELECT order_id, placed_at, total_cents
FROM orders_by_customer
WHERE customer_id = 42
LIMIT 20;
```

The partition key `customer_id` selects one replica set by hashing; the clustering columns keep rows of that partition sorted on disk, so the query reads 20 adjacent rows from one machine. The requirement "orders above 100 in March across all customers" cannot be answered efficiently from this table: it would contact every partition. The accepted remedy is a second table, `orders_by_month`, written by the application in the same operation as the first. Duplication of data is therefore a deliberate part of the design, and keeping the copies consistent becomes the application's responsibility.

2.1.5 Hands-on

A single relational node is much faster than the scaling folklore suggests, and a measurement is the correct starting point before adopting a distributed store. The following run uses `pgbench` against PostgreSQL 16 on the 2-core build machine (shared with other processes), scale factor 20 (2,000,000 account rows), 8 client connections, 15 seconds each.

pgbench on a single small PostgreSQL node (measured on the build machine; typical) — TERMINAL

```
$ pgbench -i -s 20 pb
$ pgbench -c 8 -j 2 -T 15 pb
scaling factor: 20
number of clients: 8
latency average = 6.987 ms
tps = 1145.016683 (without initial connection time)
$ pgbench -S -c 8 -j 2 -T 15 pb
latency average = 0.435 ms
tps = 18384.084044 (without initial connection time)
```

The first run is the TPC-B-like read-write mix (updates to three tables and an insert, each transaction committed with a durable WAL flush); the second is select-only. On two shared cores the node sustains roughly 1,100 durable multi-statement transactions per second and about 18,000 key lookups per second. These figures are specific to this machine; the conclusion is the order of magnitude. A workload of a few hundred writes per second and a few thousand reads per second does not justify a distributed database.

2.1.6 Polyglot persistence

Polyglot persistence is the practice of using several storage technologies in one system, each for the workload it serves best. A typical online shop built with Spring Boot illustrates it: PostgreSQL holds orders, payments and inventory, where transactions and constraints matter; Redis holds sessions and caches; Elasticsearch serves product search; a wide-column or time-series store holds clickstream and metrics. The relational database remains the system of record, and the other stores hold derived data that can be rebuilt.

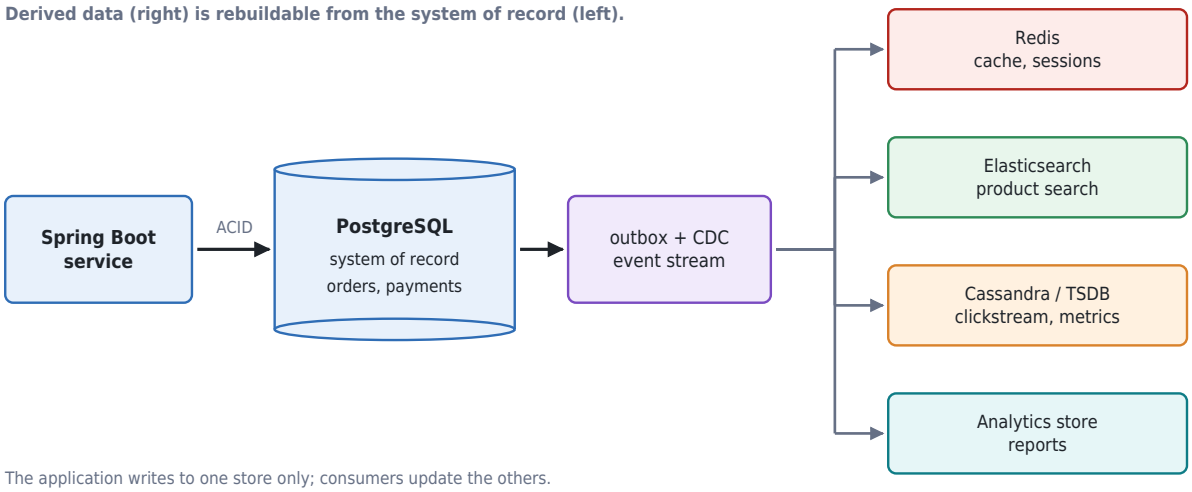


Figure 2.3. A system of record in PostgreSQL feeds derived stores through an event stream; each derived store can be rebuilt from the system of record, so its loss costs availability, not data.

The price of polyglot persistence is operational and consistency-related. Each additional store needs backup, monitoring, upgrades and expertise. Data copied between stores is eventually consistent; if a product is updated in PostgreSQL and the search index is updated by a consumer a few hundred milliseconds later, a search issued in that interval returns the old data. Two patterns keep the copies correct: the transactional outbox (the change event is written in the same database transaction as the data, and a relay publishes it) and change data capture (a connector reads the database log). Both are covered with the transaction chapters in Part 1; the rule is that the application does not write to two stores independently, because a crash between the two writes leaves them inconsistent with no record of which is correct.

2.1.7 Real-world problems and solutions

Incident 1: distributed store adopted for a workload one node handled

Scenario. A team building an internal order-tracking service (roughly 40 GB of data, 200 writes per second at peak, 1,500 reads per second) selects a Cassandra cluster of six nodes "to be ready for scale".

Symptoms. Feature delivery slows. Reports that were one SQL join become application loops over several tables. The ticket queue shows "stale status" complaints, because reads at consistency level ONE sometimes return the previous order state. Operations effort (repairs, compaction tuning, upgrades) exceeds the effort of building features.

Diagnosis. (1) Quantify the load: peak writes per second, peak reads per second, data size, growth per month. (2) Run the same load profile against a single PostgreSQL instance with `pgbench` custom scripts or an application load test. (3) List the queries the product owner expects within a year and count those that need joins, ad-hoc filters or multi-row transactions.

Root cause. The decision was based on anticipated scale rather than measured need. The measured workload was two orders of magnitude below the limit of a single node, and the required queries were relational.

Fix. Migrate the data to PostgreSQL with a primary and one streaming replica. Keep the order identifiers unchanged so that downstream systems are unaffected. Retire the Cassandra cluster after a dual-read validation period.

Prevention. Record scale requirements as numbers (rows, writes per second, p99 latency, availability target) in the design document, and require a measurement on the relational baseline before a distributed store is selected. A concrete threshold to trigger reconsideration: sustained write rate beyond the capacity of the largest affordable single node, a dataset beyond what one node can back up and restore within the recovery-time objective, or a multi-region write requirement.

2.1.8 Questions to expect

1. What does NoSQL mean, and is it accurate?

The term is a label for non-relational databases and is usually expanded as "not only SQL". It is imprecise: several NoSQL systems offer SQL-like languages, and some relational systems store documents. The meaningful classification is by data model (key-value, document, wide-column, graph, search, time series) and by the guarantees offered.

2. Why do NoSQL databases scale horizontally more easily than relational ones?

They restrict the operations that need data from several places at once. By limiting queries to a partition key, avoiding cross-partition joins and giving up multi-row transactions, each request is served by one or a few nodes, so adding nodes adds capacity almost linearly.

3. What is polyglot persistence?

The use of different storage engines in one system, each chosen for its workload: for example PostgreSQL for transactions, Redis for caching, Elasticsearch for search. A system of record is designated, and the other stores are derived from it.

4. When should a relational database be preferred?

When the data is structured and related, requirements include multi-row transactions and constraints, queries are not known in advance, and the load fits on one node with replicas. This describes most business applications.

5. What is the design difference between relational and NoSQL data modelling?

Relational modelling starts from the entities and normalises; queries come later. Partitioned NoSQL modelling starts from the queries and denormalises: one table or collection per access pattern, with duplication accepted.

6. Name the costs of a polyglot architecture.

Operational burden (more systems to run), consistency lag between the system of record and derived stores, and the need for a reliable propagation mechanism such as an outbox or change data capture.

7. Why is vertical scaling attractive and where does it stop?

It requires no change to the application and preserves all relational features. It stops at the capacity of the largest machine, at its price curve, and at the single failure domain.

KEY POINTS

NoSQL is a set of data models and design trade-offs, not a replacement for SQL.

- The motivations are volume and write rate, simple known access patterns, schema variability and specialised retrieval (graph, search, time series).
- Partitioning removes or weakens joins, foreign keys, multi-row transactions and ad-hoc queries.
- Partitioned NoSQL design starts from the queries and duplicates data; relational design starts from entities and normalises.
- A single PostgreSQL node handles a large workload; measure before adopting a distributed store.
- Polyglot persistence designates one system of record and derives other stores from it through an outbox or change data capture.

2.2 Distributed data fundamentals

Every NoSQL system in this Part rests on the same small set of distributed-systems techniques: copying data to several machines (replication), dividing data among machines (partitioning), deciding how many copies must agree before an operation succeeds (quorums), and repairing copies that diverge. Understanding these techniques once makes the individual products (Cassandra, DynamoDB, MongoDB, Riak, Redis Cluster) readable as different configurations of the same ideas. This chapter states each technique, works through numbers produced by programs run on the build machine, and ends with the consistency models and theorems (CAP, PACELC) stated precisely.

2.2.1 Concept and mechanism

Replication

Replication keeps a copy of the same data on several nodes. It serves two purposes: availability (a copy survives the loss of a node) and read capacity or proximity (a copy near the client serves reads). The number of copies is the replication factor, written N in this chapter and RF in Cassandra documentation. Three topologies exist.

1. **Single-leader (primary-replica).** One node accepts writes and streams a log to followers. Reads can be served by the leader or, with possible staleness, by followers. PostgreSQL streaming replication, MySQL, MongoDB replica sets and Redis replication follow this model. Write conflicts cannot occur because one node orders all writes; the leader is the bottleneck and the failover point.
2. **Multi-leader.** Several nodes, usually one per data centre, accept writes and exchange them asynchronously. Local writes remain fast and survive an inter-region outage, but two leaders can accept conflicting writes to the same record, so a conflict-resolution rule is mandatory.
3. **Leaderless.** Any replica accepts any request. The client or a coordinator node sends each write to all N replicas and waits for W acknowledgements, and reads from R replicas. Amazon Dynamo (2007), Cassandra, Riak and the DynamoDB design lineage use this model. Conflicts arise whenever concurrent writes reach replicas in different orders.

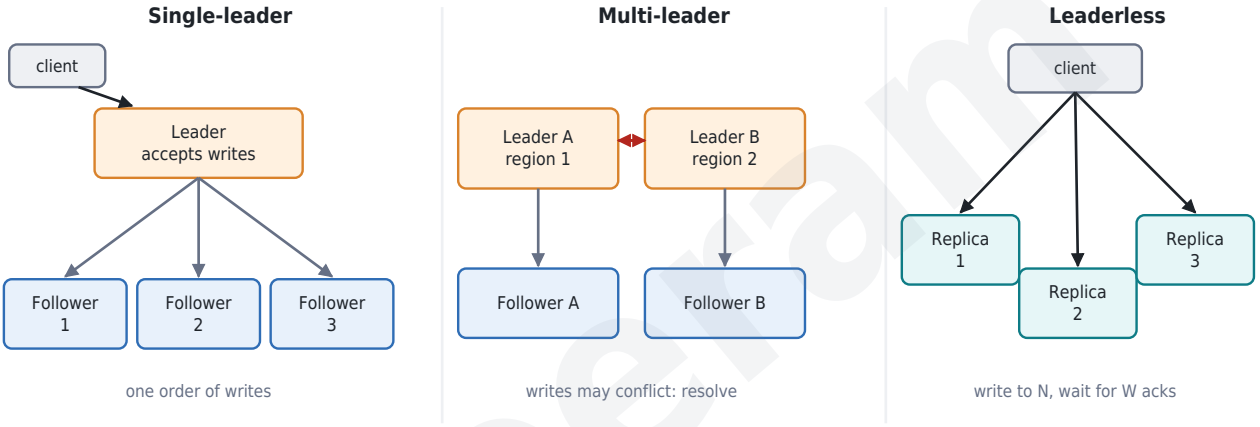


Figure 2.4. Three replication topologies: one leader orders every write, several leaders accept writes in parallel and must reconcile, and in the leaderless design the client writes to several replicas directly.

Replication can be synchronous (the writer waits for the replica) or asynchronous (the writer continues). Synchronous replication to every replica makes any single node failure block writes; asynchronous replication can lose acknowledged writes on leader failure and produces stale reads. Quorum replication, described below, is the intermediate position.

Partitioning (sharding)

When the dataset or write rate exceeds one node, the keys are divided into partitions, each stored on a different node (and each replicated). The two basic strategies differ in what they preserve.

Property	Range partitioning	Hash partitioning
Rule	Contiguous key ranges form a partition (a-f, g-m, ...)	The hash of the key selects the partition
Range queries on the key	Efficient: adjacent keys are co-located	Inefficient: neighbouring keys are scattered
Distribution of load	Skewed if keys are sequential (time stamps): the newest range receives all writes	Even for most key distributions, because the hash randomises placement
Rebalancing	Split or merge ranges	Move hash ranges or tokens
Used by	HBase, Bigtable, MongoDB ranged sharding, CockroachDB	Cassandra, DynamoDB, Riak, MongoDB hashed sharding

Range partitioning compared with hash partitioning.

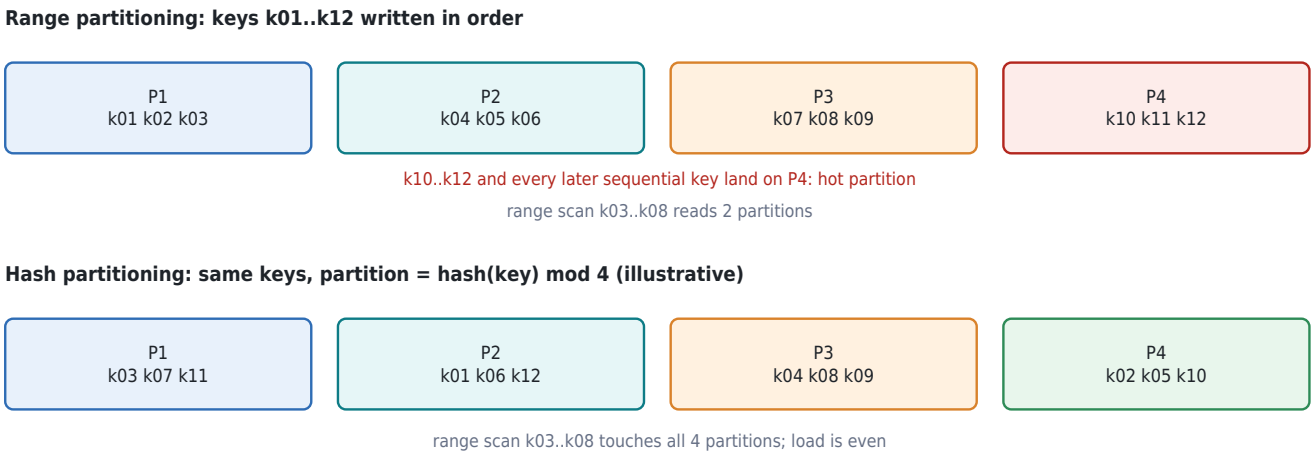


Figure 2.5. Range partitioning keeps adjacent keys together and sends all sequential-key writes to the last partition; hash partitioning spreads the same keys across all partitions and loses the order.

A frequent hybrid is a compound key: the first component is hashed to choose the partition, and the remaining components are kept in sorted order inside the partition. Cassandra's partition key plus clustering columns and DynamoDB's partition key plus sort key follow this design, and it is the reason a query such as "the latest 20 orders of customer 42" is efficient while "all orders in March" is not.

2.2.2 Consistent hashing

The naive way to map a hashed key to one of N nodes is `hash(key) mod N`. It distributes keys evenly, but changing N changes the result for most keys, so adding one node to a four-node cluster relocates most of the data. Consistent hashing (Karger et al., 1997; used in Amazon Dynamo, 2007) limits relocation to the data that must move.

The hash space is treated as a ring. Each node is placed at one or more positions on the ring by hashing its name. A key is hashed to a position on the same ring and is owned by the first node found moving clockwise from that position. With replication factor N, the key is also stored on the next N-1 distinct nodes clockwise (the preference list).

Worked placement

The program below uses the MD5 hash reduced modulo 360 (positions are degrees) so that the numbers are readable. Four nodes are placed by hashing their names.

ring.py (excerpt) PYTHON

```python
import hashlib, bisect

def h(s, m=360):
    return int(hashlib.md5(s.encode()).hexdigest(), 16) % m

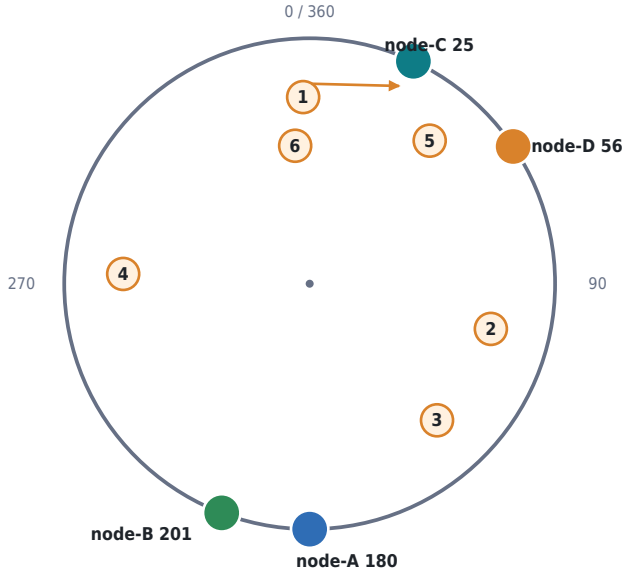
nodes = ["node-A", "node-B", "node-C", "node-D"]
ring = sorted((h(n), n) for n in nodes)

def owner(key):
    pos = h(key)
    points = [p for p, _ in ring]
    return pos, ring[bisect.bisect_left(points, pos) % len(ring)]
```

Positions and ownership (real output) TERMINAL

```
ring: [(25, 'node-C'), (56, 'node-D'), (180, 'node-A'), (201, 'node-B')]
user:1001 (358, (25, 'node-C'))
user:1002 (104, (180, 'node-A'))
user:1003 (137, (180, 'node-A'))
order:77 (273, (25, 'node-C'))
order:78 (40, (56, 'node-D'))
cart:9 (354, (25, 'node-C'))
```

The key `user:1001` hashes to position 358. Moving clockwise from 358 passes 360, wraps to 0 and reaches node-C at 25, so node-C owns it. The key `order:78` hashes to 40, which lies between node-C (25) and node-D (56), so node-D owns it. With N = 3 the preference list of `user:1001` is node-C, node-D, node-A (the next two distinct nodes clockwise from node-C).



key	position	owner	preference list (N=3)
1 user:1001	358	node-C	C D A
2 user:1002	104	node-A	A B C
3 user:1003	137	node-A	A B C
4 order:77	273	node-C	C D A
5 order:78	40	node-D	D A B
6 cart:9	354	node-C	C D A

Rule: the owner is the first node found moving clockwise from the key position, wrapping from 360 to 0. Key 1 (358) passes 360 and reaches node-C at 25. Replicas: next distinct nodes clockwise.

Figure 2.6. The ring with the four node positions and six keys from the program output; each key is owned by the first node clockwise, and the preference list for N = 3 continues clockwise.

Cost of changing the cluster

A fifth node, `node-E`, hashes to position 216 and takes over the arc between node-B (201) and itself, which formerly belonged to node-C (the arc from 201 round to 25). Only keys with positions in (201, 216] change owner. Counting 10,000 keys on the build machine:

Keys that change owner when a fifth node joins (real output) TERMINAL

```
consistent hashing: moved 430 of 10000   (4.3 percent)
hash(key) mod N   : moved 8027 of 10000  (80.3 percent)
```

Modulo placement relocates about 80 percent of the keys when N changes from 4 to 5 (the expected fraction is 1 - 1/5 under uniform hashing), while consistent hashing moves 4.3 percent in this instance, the arc owned by the new node. Relocated data is the cost of every scale-out operation, because it consumes network and disk bandwidth, so this difference determines whether a cluster can grow while serving traffic.

Keys that change owner when a fifth node joins (10,000 keys)

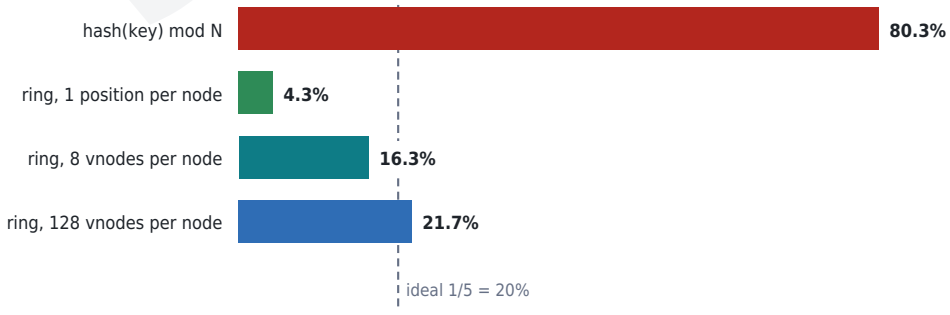


Figure 2.7. Bars compare the share of 10,000 keys that change owner when a fifth node joins: modulo placement 80.3 percent, consistent hashing with one position per node 4.3 percent (the 15-degree arc of the new node), and with 8 or 128 virtual nodes per node 16.3 and 21.7 percent against an ideal 20 percent.

Virtual nodes

One position per node has a defect: the arcs between positions are unequal. With the four positions above, node-C owns the arc from 201 round to 25 (184 degrees, 51 percent of the ring) while node-B owns only 21 degrees. Virtual nodes (vnodes) fix this by giving each physical node many positions, so that the arc lengths average out. A second program placed each node at 1, 8 or 128 positions (hashing the names `node-A#0`, `node-A#1`, and so on onto a 32-bit ring) and counted the owner of 10,000 keys.

Share of 10,000 keys per physical node (real output) TERMINAL

```
vnodes per node=1    A=612   B=245   C=5495  D=3648
vnodes per node=8    A=3277  B=3322  C=999   D=2402
vnodes per node=128  A=2768  B=2457  C=2645  D=2130
```

With one position per node, node-C holds 55 percent of the keys. With 8 positions the imbalance is still large (node-C 10 percent); with 128 the shares range from 21 to 28 percent against an ideal 25 percent. Virtual nodes also let nodes of different capacity hold different numbers of positions, and they spread the recovery work after a failure across all surviving nodes rather than only the neighbour. The disadvantage is a larger metadata table and more partial ranges to repair. Cassandra assigns vnodes by default; the documentation notes that modern versions use an allocation algorithm that needs far fewer tokens per node than the older random assignment with 256 tokens.

2.2.3 Quorum reads and writes

In a leaderless system the coordinator sends every write to all N replicas, but it reports success after W acknowledgements. A read contacts R replicas and returns the value with the newest version. The two sets of replicas, the W that acknowledged the write and the R that answer the read, are guaranteed to share at least one replica when

quorum rule

```
R + W > N
```

because the sizes then exceed N in total, so by the pigeonhole principle they overlap. The overlapping replica holds the latest acknowledged write, and the version comparison selects it.

N	W	R	R+W > N	Behaviour
3	1	1	no	Fastest; a read can miss the latest acknowledged write
3	2	2	yes	Quorum on both sides; tolerates one failed replica for reads and for writes
3	3	1	yes	Reads cheap; one failed replica blocks every write
3	1	3	yes	Writes cheap; one failed replica blocks every read

Settings for N = 3.

The program below draws random sets of W acknowledging replicas and R read replicas for 200,000 trials and counts reads that miss the write. A read is stale exactly when its set is disjoint from the write set, so the theoretical probability is C(N-W, R) / C(N, R).

Simulated stale-read rate for N = 3 (real output) TERMINAL

```
 N  W  R  W+R>N  stale reads (simulated, 200000 reads)  formula C(N-W,R)/C(N,R)
 3  1  1  False   66.6685%                             66.6667%
 3  1  2  False   33.4435%                             33.3333%
 3  2  1  False   33.4075%                             33.3333%
 3  2  2  True     0.0000%                              0.0000%
 3  3  1  True     0.0000%                              0.0000%
 3  1  3  True     0.0000%                              0.0000%
```

The figures describe the worst case in which the other replicas have not yet received the write when the read starts. In practice the remaining replicas receive the write within milliseconds, so the stale window is short; the table states what the system guarantees, not what usually happens.

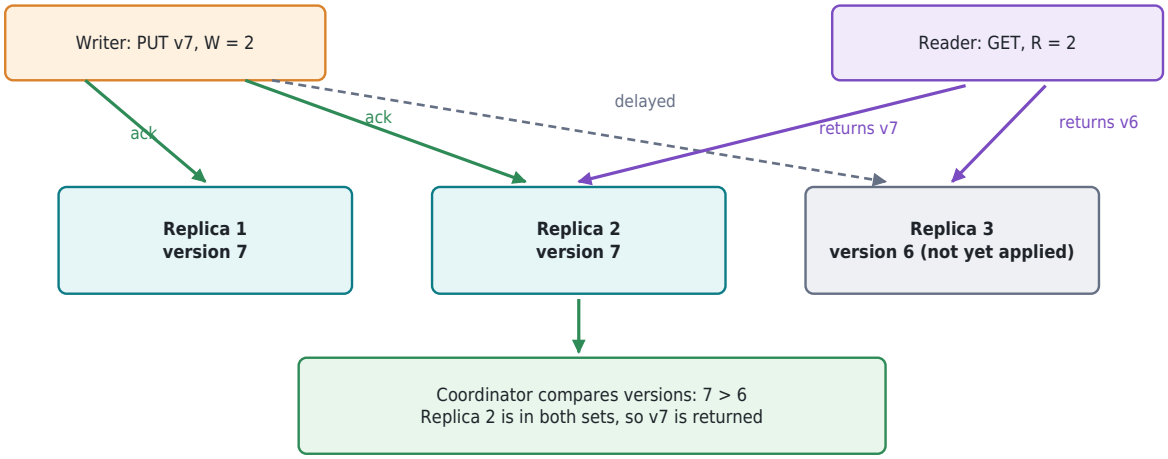


Figure 2.8. With N = 3, a write acknowledged by replicas 1 and 2 and a read answered by replicas 2 and 3 share replica 2, which returns version 7 and wins the version comparison; with W = 1 and R = 1 the sets can be disjoint.

WATCH OUT
R + W > N does not give linearizability by itself. Sloppy quorums (writes accepted by substitute nodes during a failure), concurrent writes with unsynchronised timestamps, a write that fails after reaching fewer than W replicas but remains visible on some of them, and replicas that are repaired from stale peers can all violate the guarantee. The condition ensures that an acknowledged write is visible to a later read only when none of these effects is present.

2.2.4 Worked example

A cart service stores one document per user under a Cassandra-style leaderless cluster with N = 3, W = 2 and R = 2. Replica nodes are node-C, node-D and node-A for the key `user:1001` (the preference list computed above).

1. The coordinator receives `PUT cart = [milk]` at version 1. It sends the write to node-C, node-D and node-A. node-C and node-D acknowledge; node-A is temporarily unreachable. W = 2 acknowledgements have arrived, so the client receives success.
2. A second request `GET cart` arrives. The coordinator asks node-D and node-A (R = 2). node-D returns version 1 and node-A does not answer, so only one response has arrived; the coordinator then asks node-C, receives version 1 and returns `[milk]`. Had node-A answered with an empty (older) state, the coordinator would still have returned version 1, the newest of the two versions received, because any two replicas out of three include at least one of the two that acknowledged the write.
3. node-A returns. It holds no cart. The coordinator, or a later read, detects the difference and repairs node-A. The repair mechanisms (hinted handoff, read repair, anti-entropy) are the subject of the next part of this chapter.

2.2.5 Real-world problems and solutions

Incident 2: stale read immediately after a write

Scenario. A profile service writes a changed email address with consistency level ONE and reads it with consistency level ONE (N = 3). After saving a profile the user sees the old address on the next page for a small fraction of requests.

Symptoms. Intermittent mismatch between the write response and the next read; the mismatch disappears after one to several seconds; the logs show no errors.

Diagnosis. (1) Identify the consistency levels in the driver configuration, for example `spring.cassandra.request.consistency` or the per-statement setting. (2) Compute R + W against N: 1 + 1 is not greater than 3. (3) Reproduce with a script that writes and immediately reads in a loop and counts mismatches. (4) Check replication lag indicators: hinted-handoff backlog (`nodetool tpstats`, HintsDispatcher) and dropped mutations (`nodetool tpstats`, MutationStage).

Root cause. W + R is not greater than N, so a read can be served by a replica that has not yet applied the write. The system behaves as specified.

Fix. Immediate: read at LOCAL_QUORUM or QUORUM for the affected statements. Permanent: write and read at QUORUM (W = 2, R = 2 for N = 3) for data that a user expects to read back immediately, or, where the read must stay at ONE, return the written value from the write path to the client and let the UI display it without re-reading.

Prevention. Declare the required consistency per use case (read-your-writes for profile data, eventual for counters and feeds) and set the level in code per statement rather than globally. Add an integration test that writes and reads at the configured levels against a multi-node test cluster.

Incident 3: modulo-based sharding in application code

Scenario. An application routes a user to one of four database instances using `userId % 4`. A fifth instance is added.

Symptoms. After the change, lookups for most users return "not found"; a migration job running for hours copies 80 percent of all rows.

Diagnosis. (1) Compare the routing function before and after: the result for a key changes unless `userId % 4 == userId % 5`. (2) Count the keys whose destination differs (the experiment above gives 80.3 percent for 10,000 keys). (3) Confirm that rows exist on the old instances.

Root cause. The placement function depends on N, so changing N invalidates the placement of most keys.

Fix. Immediate: route by a lookup table that holds the old and new instance for each moving key range while the migration runs. Permanent: use consistent hashing, a fixed large number of logical shards mapped to physical instances (for example 1,024 logical shards assigned to four, then five instances), or the cluster's own partitioning.

Prevention. Separate the stable logical partition (derived from the key and never changed) from the movable assignment of partitions to nodes. Redis Cluster uses exactly this design with 16,384 hash slots.

2.2.6 Questions to expect

1. What is consistent hashing and why is it used?

Keys and nodes are hashed onto a ring; a key belongs to the first node clockwise. Adding or removing a node relocates only the keys on the adjacent arc (about 1/N of the data) instead of nearly all keys as with `hash mod N`. It enables elastic scaling of partitioned stores.

2. What are virtual nodes?

Many ring positions per physical node. They equalise the share of keys (with one position per node the arcs differ widely), allow weighting by capacity, and spread rebalancing and recovery load over many nodes.

3. State the quorum condition and what it guarantees.

R + W > N: every read set intersects every acknowledged write set, so a read contacts at least one replica holding the latest acknowledged write. It does not by itself provide linearizability, because of sloppy quorums, partial failed writes and clock-based conflict resolution.

4. What are the trade-offs of choosing W = N?

Reads can use R = 1 and are fast, but a single failed replica makes writes unavailable and write latency equals the slowest replica.

5. Compare range and hash partitioning.

Range keeps order and supports range scans but concentrates sequential-key writes on one partition. Hash spreads load evenly but loses ordering, so range queries must contact all partitions. A compound key (hashed partition key, ordered clustering key) combines both.

6. Why can leaderless replication produce conflicts but single-leader replication cannot?

In single-leader replication one node assigns a total order to all writes. In leaderless replication two clients can write to different replicas concurrently, and replicas then hold different values with no ordering between them.

sreeram

KEY POINTS

Replication, partitioning and quorums are the common foundation of distributed NoSQL stores.

- Replication provides availability and read capacity; single-leader, multi-leader and leaderless designs differ in where conflicts can arise.
- Hash partitioning spreads load evenly; range partitioning keeps order; compound keys combine them.
- Consistent hashing relocates about 1/N of the keys when a node joins (4.3 percent in the run above) against 80 percent for modulo placement; virtual nodes even out the arcs.
- R + W > N makes read and write sets overlap; it is necessary but not sufficient for strong consistency.
- Choose consistency levels per use case; the default of one replica gives stale reads by design.

2.2.7 Conflict resolution

Leaderless and multi-leader systems allow two clients to update the same key on different replicas before the replicas have exchanged the updates. The system must then decide which value survives, or keep both. Four strategies are in common use.

Strategy	Mechanism	Consequence
Last write wins (LWW)	Each write carries a timestamp; the highest timestamp wins	Simple and convergent; silently discards concurrent writes; depends on clock quality
Version vectors / vector clocks	Each write carries a counter per node; comparison detects ordered versus concurrent writes	No silent loss; concurrent versions (siblings) are returned to the application to merge
Application merge	Siblings are merged by domain logic (union of cart items)	Correct for the domain; the code must be commutative and idempotent
CRDTs	Data types whose merge function is commutative, associative and idempotent (counters, sets, registers)	Replicas converge without coordination; limited to supported types

Conflict-resolution strategies.

Last write wins and its failure mode

Cassandra resolves conflicts per column with the write timestamp (supplied by the client or the coordinator, in microseconds); DynamoDB global tables use a last-writer-wins rule based on the time of the update. The rule is deterministic: every replica keeps the highest timestamp, so all replicas converge. It is not necessarily correct, because wall clocks on different machines disagree. The experiment below models two clients and two nodes whose clocks differ by 50 ms. Client X writes `qty=5` at real time 1.000 through node A; client Y writes `qty=7` 20 ms later through node B, whose clock runs 50 ms behind. The nodes then exchange their versions.

Last write wins with clock skew (real output) TERMINAL

```
node A: qty=5 (timestamp 1.000)
node B: qty=5 (timestamp 1.000)
write timestamps: X=1.000 Y=0.970
```

Both nodes converge, which satisfies eventual consistency, but they converge on the older write: node B stamped client Y's later write with 0.970, lower than client X's 1.000. The later update was lost without any error. Because the loss is silent and depends on clock skew, LWW is acceptable for data where any recent value is adequate (last-seen time, cache entries, idempotent overwrites of a whole record by one writer) and unacceptable for read-modify-write data such as balances, stock counts and carts.

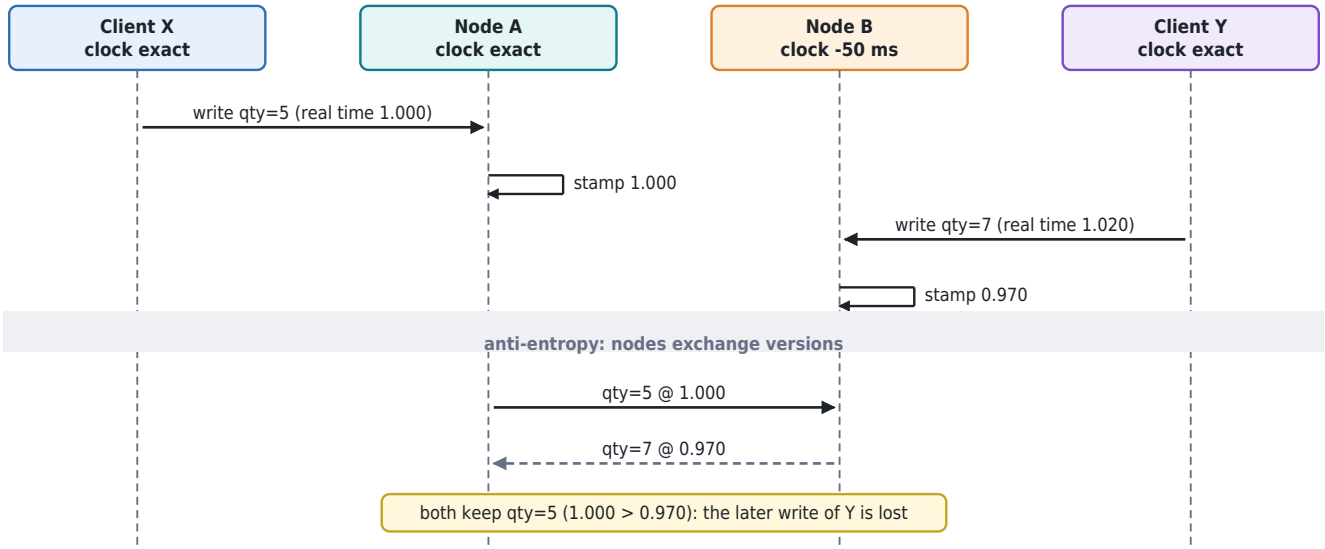


Figure 2.9. Timeline of the clock-skew experiment: the later write by client Y receives the lower timestamp 0.970 from the slow clock of node B, so the exchange of versions keeps client X's value on both nodes.

Vector clocks

A vector clock records, for each node that has coordinated a write, a counter. A version with clock *a* descends from a version with clock *b* if every counter in *a* is at least the counter in *b*; when neither descends from the other the versions are concurrent and both are kept. Amazon Dynamo used this scheme for its shopping cart; Riak implemented it (as dotted version vectors in later releases). The experiment follows a cart through three writes.

Comparing vector clocks (real output) TERMINAL

```
v1 vs v2: b descends from a
v2 vs v3: CONCURRENT
merged clock after client reconciles on node C: {'A': 2, 'B': 1, 'C': 1}
```

The sequence is: client 1 writes cart `[milk]` through node A with clock `{A:1}` (v1). Client 2 reads v1 and adds eggs through node B, producing `{A:1, B:1}` (v2); it descends from v1, so v1 can be discarded. Client 1, which has not seen v2, adds bread through node A, producing `{A:2}` (v3). The clocks of v2 and v3 are incomparable (v2 has B:1 that v3 lacks, v3 has A:2 that v2 lacks), so they are concurrent siblings: `[milk, eggs]` and `[milk, bread]`. The next reader receives both, merges them to `[milk, eggs, bread]` and writes the result with a clock that dominates both, here `{A:2, B:1, C:1}` after coordination by node C.

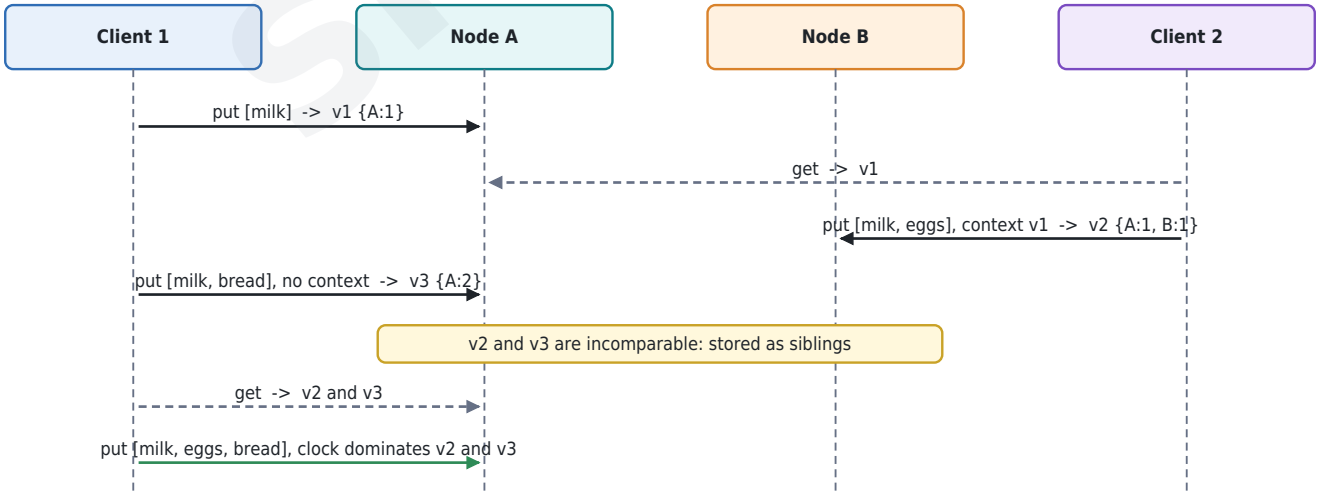


Figure 2.10. Three writes to a cart: v2 descends from v1 and replaces it, while v3 and v2 are incomparable siblings that the application merges into a new version whose clock dominates both.

A vector clock grows with the number of coordinating nodes, so implementations prune old entries, which can reintroduce false conflicts. Cassandra avoids the whole mechanism by using timestamps per cell; MongoDB avoids it by having one primary per shard; DynamoDB exposes conditional writes (`ConditionExpression`) so that a client can implement optimistic concurrency with a version attribute.

2.2.8 Keeping replicas converged

Replicas diverge when a node is down, when a message is lost or when writes arrive in different orders. Three mechanisms bring them back together, and a production cluster normally runs all three.

1. **Hinted handoff.** When a replica is unavailable during a write, the coordinator stores a hint (the mutation and the target node) and replays it when the node returns. Hints are kept for a bounded window (Cassandra: `max_hint_window`, three hours by default in recent versions, a setting to confirm in the version deployed); after that the node must be repaired by other means. A write accepted only through hints, when the preference-list replicas are down, is a sloppy quorum: the write is acknowledged but is not yet on the replicas that a quorum read will contact.
2. **Read repair.** When a read contacts several replicas and the returned versions differ, the coordinator returns the newest version and writes it back to the stale replicas. It repairs only data that is read.
3. **Anti-entropy repair.** A background process compares replicas using Merkle trees (hash trees over key ranges), finds ranges whose hashes differ and streams only the differing ranges. In Cassandra this is `nodetool repair`; it must run at least once within `gc_grace_seconds` (discussed with tombstones) to avoid resurrecting deleted data.

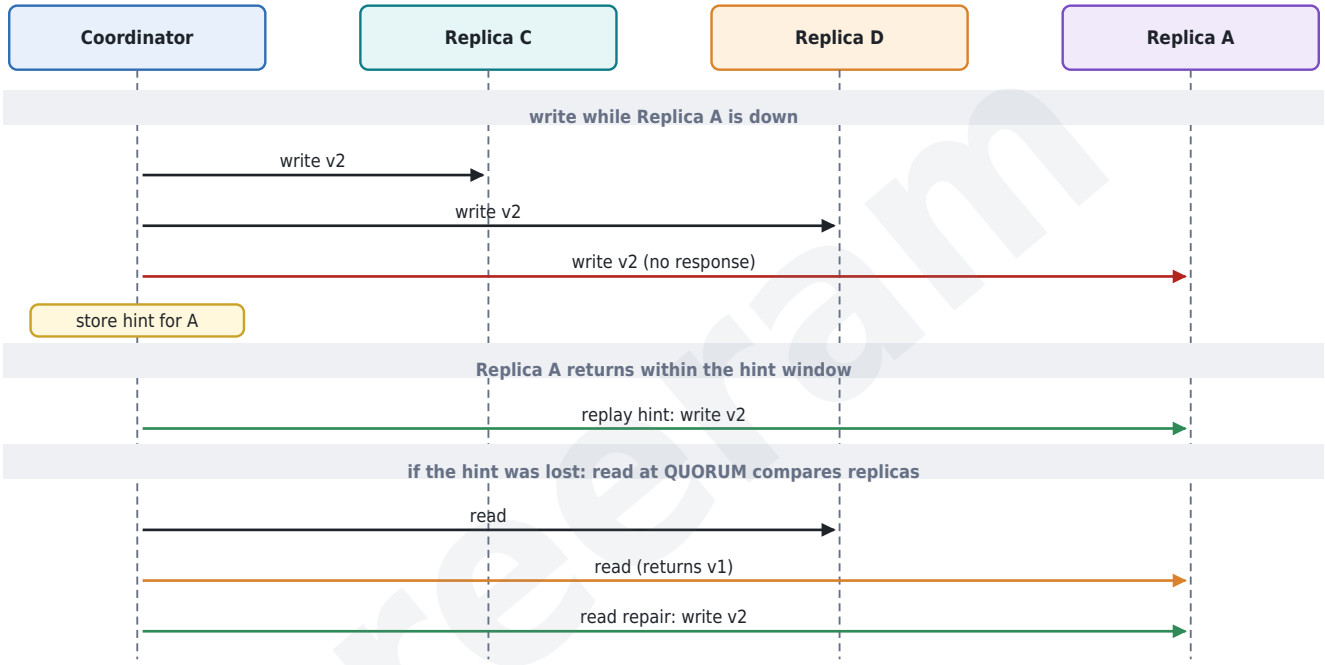


Figure 2.11. Hinted handoff stores the missed write at the coordinator and replays it after the node returns; read repair then fixes any remaining difference noticed during a read.

Gossip

Cluster membership (which nodes exist, which are up, which tokens they own) is spread by gossip: every second each node picks a random peer and exchanges its state; the receiver merges it with a version number per entry, so newer information replaces older. No node is a central registry, so there is no single point of failure for membership. A simulation of push-pull gossip with one exchange per node per round, started from one node that knows an update, gives the following times to reach every node.

Rounds until all nodes know an update (real output of a random simulation) TERMINAL

```
nodes=10     rounds until all nodes know the update:  3  (log2 n = 3.3)
nodes=100    rounds until all nodes know the update:  6  (log2 n = 6.6)
nodes=1000   rounds until all nodes know the update:  9  (log2 n = 10.0)
nodes=10000  rounds until all nodes know the update: 11  (log2 n = 13.3)
```

The number of rounds grows with the logarithm of the cluster size: the informed population roughly doubles in each round until most nodes know, then the last few nodes are reached. For a cluster of 1,000 nodes and a one-second gossip interval, an update is known everywhere in about ten seconds. Cassandra gossips every second, and its documentation notes that nodes decide locally whether a peer is up or down with a phi accrual failure detector, which adapts the suspicion threshold to the observed heartbeat intervals instead of using a fixed timeout.

sreeram

2.2.9 CAP and PACELC

The CAP theorem

The CAP theorem (conjectured by Eric Brewer in 2000; proved by Seth Gilbert and Nancy Lynch in 2002) concerns three properties of a replicated service:

- **Consistency (C):** linearizability. Every operation appears to take effect atomically at one point between its invocation and its response, and all clients observe the same order. This is not the C of ACID, which means that a transaction preserves the integrity constraints.
- **Availability (A):** every request received by a non-failed node eventually receives a response that is not an error.
- **Partition tolerance (P):** the system continues to operate when the network drops or delays messages between nodes.

The theorem states that in an asynchronous network, no implementation of a read/write register can guarantee both linearizability and availability when a network partition occurs. The argument is short. Two groups of nodes cannot communicate. A write is applied in one group. A read arrives at the other group. The group must either wait or return an error until the partition heals (giving up A) or answer with possibly stale data (giving up C).

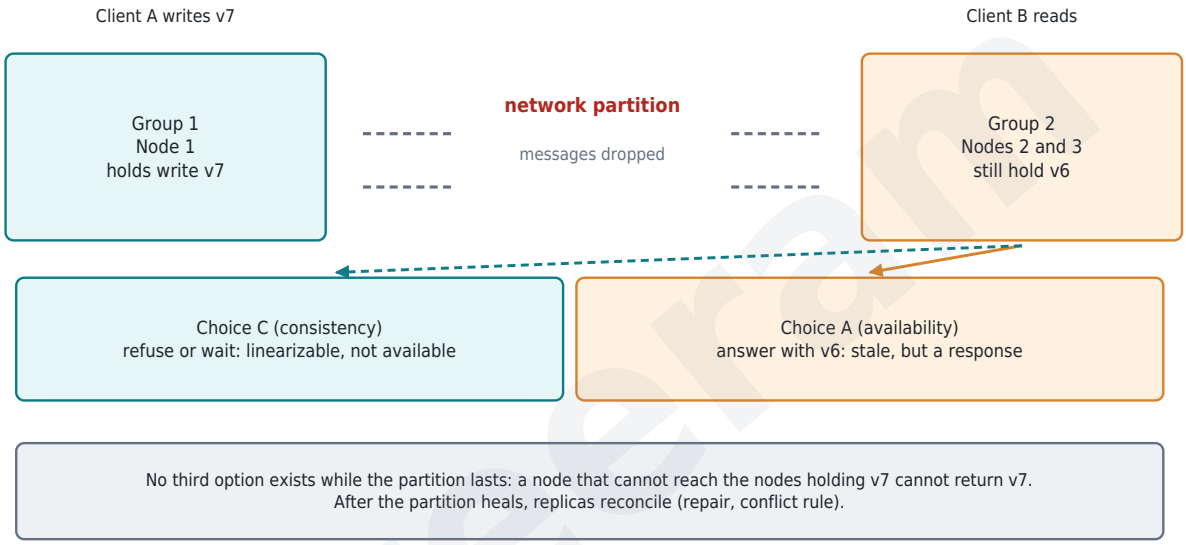


Figure 2.12. During a partition a replica in the minority group can refuse the request, preserving linearizability but losing availability, or it can answer from local state, preserving availability but returning stale data.

Common misconceptions

1. **"Pick any two of three."** Partitions are a fact of networks, not a choice. The real choice is between C and A while a partition lasts. A system that is not partition-tolerant is a single node.
2. **"CAP classifies a database permanently."** Behaviour depends on configuration and operation. Cassandra at consistency level ALL behaves as CP for that request; at ONE it behaves as AP. MongoDB with `w: "majority"` and `readConcern: "linearizable"` leans to C, while reads from secondaries lean to A.
3. **"CAP describes normal operation."** CAP says nothing when there is no partition. Most of the time the practical trade-off is latency against consistency, which PACELC captures.
4. **"Availability means high availability."** In the theorem every non-failed node must answer. A system that loses availability only in a minority partition may still report 99.99 percent uptime.
5. **"Eventual consistency means no guarantees."** Convergence is guaranteed when updates stop; stronger session guarantees can be layered on.

PACELC

Daniel Abadi (2012) extended CAP: if there is a partition (P), the system trades availability (A) against consistency (C); else (E), when running normally, it trades latency (L) against consistency (C). The second half reflects that replication to more nodes before answering costs round trips.

System	PACELC	Reasoning
Cassandra, DynamoDB (eventually consistent reads), Riak	PA/EL	Accepts writes during a partition; answers from the nearest replica in normal operation
ZooKeeper, etcd, HBase, a single-primary database with synchronous replication	PC/EC	Refuses or blocks without a majority; pays latency for agreement
MongoDB	PA/EC by default, PC/EC with majority concerns	Abadi's paper lists default MongoDB as PA/EC; with `w: "majority"` and majority reads a minority side cannot commit
PNUTS (Yahoo)	PC/EL	Chooses consistency during partition, low latency normally via timeline consistency

PACELC placement of typical configurations (classifications depend on configuration).

2.2.10 Consistency models and BASE

A consistency model is a contract between a data store and its clients about which histories of reads and writes are allowed. The models below are listed from strongest to weakest; each implies the ones below it on the main path.

Model	Guarantee	Typical implementation
Linearizable	Operations appear atomic at a single point in real time; a read returns the latest completed write	Consensus (Raft, Paxos), single leader with synchronous reads
Sequential	All clients see the same total order of operations, consistent with each client's program order, but not necessarily real time	Single log, some replicated state machines
Causal	Operations that are causally related are seen in the same order by everyone; concurrent operations may appear in different orders	Dependency tracking, version vectors (MongoDB causal sessions)
Read-your-writes	A client always sees its own earlier writes	Sticky sessions, reading from the leader, client-held version tokens
Monotonic reads	A client never sees data older than data it has already read	Pin a client to one replica
Eventual	If no new writes occur, all replicas converge	Anti-entropy, gossip

Consistency models.

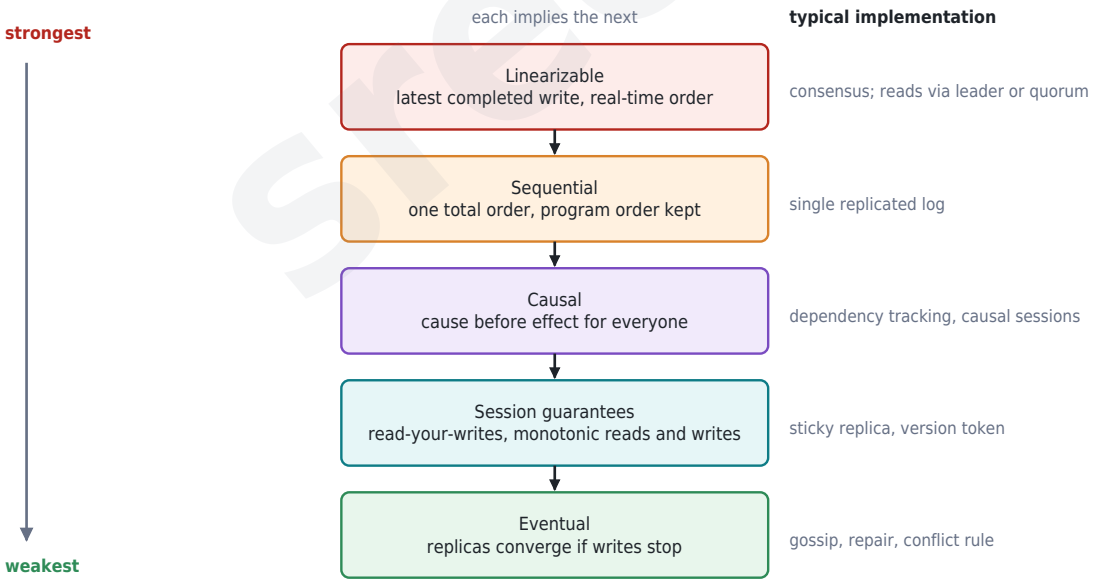


Figure 2.13. Consistency models ordered by strength: each stronger model permits a subset of the histories permitted by the weaker one below it, at the cost of more coordination and latency.

BASE (Basically Available, Soft state, Eventual consistency) was proposed by Dan Pritchett (2008) as a contrast to ACID. It describes a design stance rather than a precise property: the system stays available, state may change without input as replicas converge, and consistency arrives eventually. Applied to application design it means accepting that other users and services see intermediate states, and designing operations to be idempotent and commutative so that

reordering and retries are harmless. It is a trade-off, not a quality: money transfer and inventory decrement usually need stronger guarantees on the specific records involved, even in an otherwise eventually consistent system.

2.2.11 Worked example

A shop stores stock counts in a leaderless store (N = 3, QUORUM reads and writes) and uses a read-modify-write: read `stock`, subtract one, write `stock`. Two checkout services handle the last unit simultaneously.

1. Both read `stock = 1` from a quorum (version 10).
2. Both compute `stock = 0` and write version 11 with different timestamps.
3. Each write reaches a quorum; both return success. Two orders have been accepted for one unit.

The quorum condition R + W > N held at every step, yet the invariant (stock never below zero) was violated: quorums order reads after writes but do not make a read-modify-write atomic. Three remedies exist, in increasing strength. (a) Optimistic concurrency: write with a condition `IF stock = 1` (Cassandra lightweight transaction using Paxos, DynamoDB `ConditionExpression`, MongoDB `findOneAndUpdate` with a filter). One write succeeds and the other fails and retries. (b) Model the data as events: record `reserved(order_id)` rows and compute availability by counting, so that concurrent reservations are both stored and an overselling check runs in a single serialised consumer. (c) Keep the inventory in a store that supports transactions and treat the leaderless store as a read model.

2.2.12 Real-world problems and solutions

Incident 4: last write wins silently discards an update

Scenario. A user-preferences service on a Cassandra-style store is written by a mobile app and a web app. A user disables email notifications on the phone, then within the same minute changes the language in the browser. The web app writes the whole preferences document.

Symptoms. The user reports that the notification setting reverted. No error appears in logs. Both writes were acknowledged.

Diagnosis. (1) Retrieve write timestamps for the row: `SELECT notifications, writetime(notifications) FROM prefs WHERE user_id = ?;` (2) Compare with the request log times of both writes. (3) Check the clock offset between application servers (`chronyc tracking` on Linux) and database nodes. (4) Check whether the web write contained the full document including the stale notification value.

Root cause. Two defects combine: the web client wrote fields it had not changed (a full overwrite, based on a stale read), and the store resolved the conflict by timestamp per cell, so the stale value with the later timestamp replaced the newer setting.

Fix. Immediate: restore the value from the application audit log. Permanent: write only changed columns (partial update), so that unrelated fields are not overwritten; for fields that must not lose updates, use a conditional write on a version column, or a CRDT-style structure such as a set or map with per-field timestamps.

Prevention. Run NTP or chrony on all clients that supply timestamps, prefer server-assigned timestamps, avoid read-modify-write of whole records in eventually consistent stores, and document per field which conflict rule applies.

Incident 5: split brain after a network partition

Scenario. A three-node cluster stores a distributed lock and a job-scheduling flag in a single-leader store configured with automatic failover and no majority requirement. A network failure separates node 1 (the leader) from nodes 2 and 3.

Symptoms. Two nodes believe they are the leader. Both accept writes for the same job; the job runs twice and the two histories cannot be merged when the network heals.

Diagnosis. (1) Query each node for its role and term (for example `rs.status()` in MongoDB, `SELECT pg_is_in_recovery()` in PostgreSQL, `INFO replication` in Redis). (2) Compare last-applied log positions. (3) Review the failover configuration for a quorum requirement and a fencing mechanism.

Root cause. The failover decision did not require a majority, so the minority side (node 1) kept serving writes while the majority side (nodes 2 and 3) elected a new leader.

Fix. Immediate: stop writes to the old leader, choose the side with the authoritative history, and discard or manually reconcile divergent writes from the other. Permanent: require a majority for leadership (Raft-style election with terms), have a leader that cannot reach a majority step down, and use fencing tokens so that a stale leader's writes are rejected by storage.

Prevention. Use an odd number of voting members across at least three failure domains, test partitions with a fault-injection tool, and make clients send the leader's term or fencing token with each write.

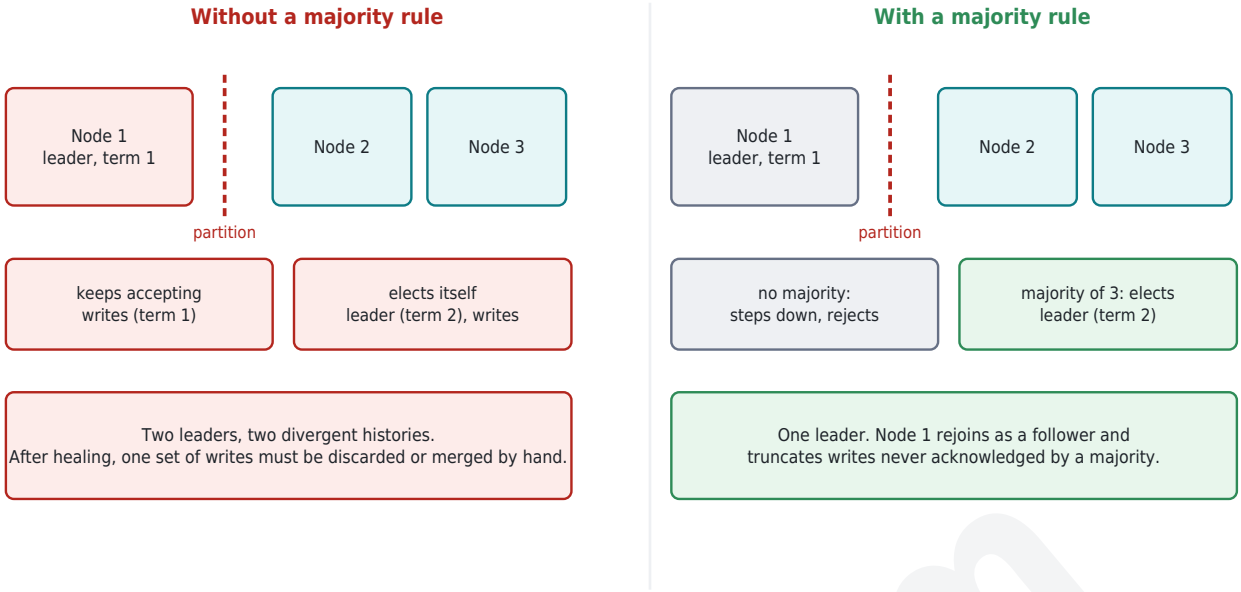


Figure 2.14. Without a majority rule, both sides of the partition elect a leader and accept writes; with a majority rule the minority side steps down, so only one history exists after the partition heals.

Incident 6: replicas diverge after a node is offline longer than the hint window

Scenario. A node of a six-node cluster is down for five days for a hardware replacement.

Symptoms. After the node returns, reads at consistency level ONE sometimes return old values or resurrect deleted rows. Dropped-hint counters are non-zero.

Diagnosis. (1) Check how long the node was down against the hint window and `gc_grace_seconds` (`nodetool describecluster`, table options in `system_schema.tables`). (2) Run a read with `CONSISTENCY ALL; TRACING ON;` for an affected key and compare the replica responses. (3) Inspect `nodetool netstats` for pending streams.

Root cause. The node was absent longer than the period for which hints are stored, so it never received the missed writes, and deletes made while it was away may already have had their tombstones purged elsewhere.

Fix. Run a full repair of the node (`nodetool repair -full` on the affected ranges) before it serves reads; when the downtime exceeded `gc_grace_seconds`, replace the node as a new member (wipe it and bootstrap) rather than rejoin it.

Prevention. Alert on nodes down longer than the hint window, schedule regular incremental repairs, and keep `gc_grace_seconds` longer than the repair interval.

2.2.13 Questions to expect

1. State the CAP theorem precisely.

When a network partition occurs, a replicated read/write service cannot be both linearizable and available: every non-failed node answering would require answering from state that may not reflect writes on the other side. The choice exists only during a partition; partitions cannot be excluded in a distributed system.

2. What does PACELC add to CAP?

CAP describes behaviour during a partition. PACELC adds that in normal operation (Else) there is still a trade-off between latency and consistency, because waiting for more replicas improves consistency at the cost of latency.

3. How do vector clocks differ from timestamps?

Timestamps impose a total order that may disagree with causality and discards concurrent updates. Vector clocks capture the partial order, so concurrent updates are detected and both preserved for merging.

4. What is a CRDT?

A conflict-free replicated data type: a structure whose merge operation is commutative, associative and idempotent, so replicas that have received the same updates in any order and any number of times converge to the same state. Examples are grow-only counters, observed-remove sets and last-writer-wins registers.

5. Distinguish hinted handoff, read repair and anti-entropy repair.

Hinted handoff replays writes a replica missed while down, for a limited window. Read repair fixes stale replicas for data that is read. Anti-entropy repair compares Merkle trees across replicas and streams the differences for all data, and is the only one that guarantees full convergence.

6. What is a sloppy quorum?

A quorum in which writes may be accepted by nodes outside the key's preference list when the preferred nodes are unreachable. It increases write availability but weakens the R + W > N guarantee until the hints are delivered.

7. Which consistency model describes "I read my own profile update immediately"?

Read-your-writes (a session guarantee). It can be provided by reading from the leader, by sticky routing to a replica that has the write, or by passing a version token with the read.

8. Why does eventual consistency not mean data loss?

It states that replicas converge when updates stop. Loss occurs only through the conflict rule (for example last write wins) or through a failure that loses all replicas holding a write.

KEY POINTS

Conflicts, repair and the theorems determine what a distributed store can promise.

- Last write wins converges but can silently discard the later of two writes when clocks differ; vector clocks and CRDTs preserve concurrent updates.
- Hinted handoff, read repair and anti-entropy repair (Merkle trees) bring replicas together; only anti-entropy covers data that is never read.
- Gossip spreads membership in about log2(n) rounds without a central registry.
- CAP applies during a partition and concerns linearizability and availability; PACELC adds the latency versus consistency trade-off of normal operation.
- Quorum overlap does not make a read-modify-write atomic; use conditional writes or event modelling for invariants.
- A majority rule for leadership prevents split brain.

2.3 Storage engines: B-tree and LSM-tree

A storage engine is the component of a database that decides how records are laid out on disk, how they are found, and how updates reach durable storage. The distributed techniques of the previous chapter decide which node holds a record; the storage engine decides what that node does with it. Two designs dominate. Update-in-place B-trees, used by PostgreSQL, MySQL InnoDB, SQLite, WiredTiger (MongoDB) and most relational systems, keep a sorted tree of fixed-size pages and modify pages where they lie. Log-structured merge-trees (LSM-trees), used by Cassandra, HBase, RocksDB, LevelDB and ScyllaDB, never modify a file: every write is appended to memory and to a log, flushed to immutable sorted files, and merged in the background. The choice determines write throughput, read latency, disk usage and the shape of operational problems such as compaction backlogs and tombstone accumulation.

2.3.1 Concept and mechanism

B-tree engines

A B+tree divides the key space into pages of typically 4 to 16 KB. Interior pages hold separator keys and child pointers; leaf pages hold the records (or pointers to them). A lookup reads one page per level, three to four levels for hundreds of millions of keys. An update locates the leaf page, modifies it in memory (the buffer pool) and, before acknowledging, appends a description of the change to the write-ahead log (WAL) so that a crash can be repaired. Dirty pages are written back later by a checkpoint. The structure of B+trees and their insertion and split procedure are covered with indexes in Part 1; the relevant properties here are these.

- Reads are cheap and predictable: a bounded number of page reads, and records of one range are adjacent on disk.

- Writes are random: an insert modifies one leaf page wherever its key sorts, so a random-key workload touches random pages. The same logical update is written twice, once to the WAL and once to the data page, and a page of 8 KB is rewritten to change a 100-byte record.
- Space is reclaimed in place, but pages split and become partly empty, so a B-tree typically uses 50 to 90 percent of its allocated pages.

LSM-tree engines

An LSM-tree turns random writes into sequential ones. The components, in write order, are:

1. **Write-ahead log (commit log).** Each write is appended to a sequential log first, giving durability for the in-memory structure.
2. **Memtable.** An in-memory sorted structure (skip list or balanced tree) holds recent writes. A read checks the memtable first.
3. **SSTable (sorted string table).** When the memtable reaches a size limit it is written, already sorted, as one immutable file, and the corresponding log segment is discarded. An SSTable contains sorted key-value pairs, a sparse index and a bloom filter.
4. **Compaction.** A background process merges several SSTables into one, keeping only the newest version of each key and dropping deleted data once it is safe. Without compaction the number of files, and therefore the cost of a read, grows without bound.

A delete does not remove data, because older SSTables are immutable and may hold the key. It writes a **tombstone**, a marker that hides older versions. The tombstone is removed only when a compaction has merged every file that could contain an older version, and, in Cassandra, only after a grace period.

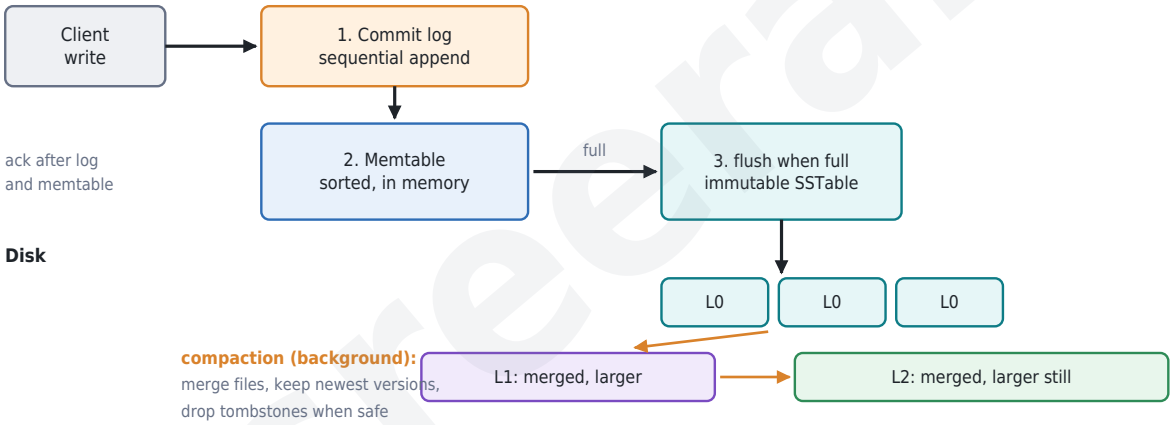


Figure 2.15. The LSM write path: each write is appended to the commit log and inserted into the memtable; a full memtable becomes an immutable SSTable at level 0, and compaction merges SSTables into larger files at deeper levels.

The read path and bloom filters

A read must find the newest version of a key. It checks the memtable, then every SSTable that may hold the key, from newest to oldest, until a version (or tombstone) is found. A key that does not exist must be checked against every SSTable, which is the expensive case. A **bloom filter** keeps a small bit array per SSTable: it answers "definitely not present" or "possibly present". It has no false negatives; its false-positive probability for m bits, n keys and k hash functions is approximately (1 - e^(-kn/m))^k. With 10 bits per key and 4 hash functions this is (1 - e^(-0.4))^4 = 1.2 percent, so about 99 of 100 useless file reads are avoided at a cost of 1.25 bytes per key.

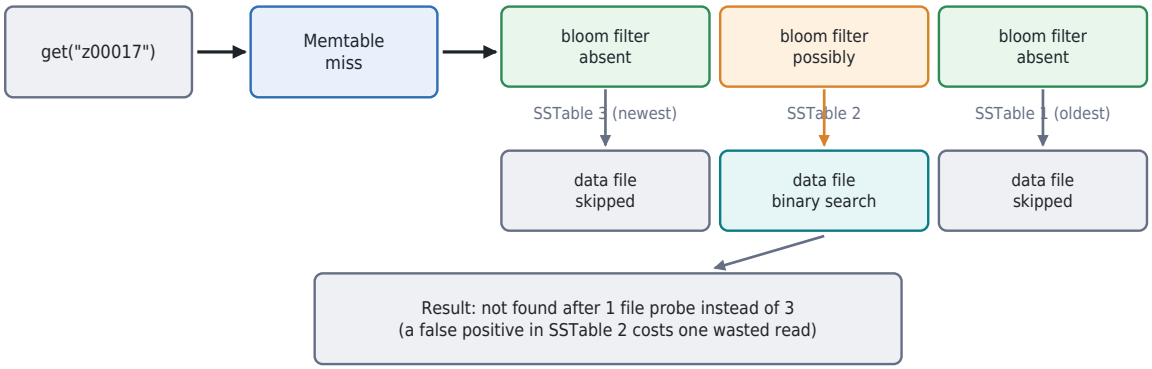


Figure 2.16. The LSM read path: the memtable is checked first, then each SSTable's bloom filter; only files whose filter answers possibly present are searched, so a read for an absent key usually touches no data file.

Compaction strategies

Two classic strategies trade write against read and space amplification. Size-tiered compaction merges several SSTables of similar size into one larger table. It writes each record fewer times (low write amplification) but several overlapping files can hold the same key, so reads may consult many files, and a merge can need free space equal to its inputs. Leveled compaction divides data into levels of geometrically growing size (each level about ten times the previous), where the files inside one level do not overlap in key range. A read consults at most one file per level, so reads and space use are better, but a record is rewritten each time it moves down a level, which raises write amplification. Cassandra documents SizeTieredCompactionStrategy as its default, LeveledCompactionStrategy for read-heavy or update-heavy workloads, TimeWindowCompactionStrategy for TTL-based time series, and the Unified Compaction Strategy (UCS) as a recommended option for new tables in recent versions.

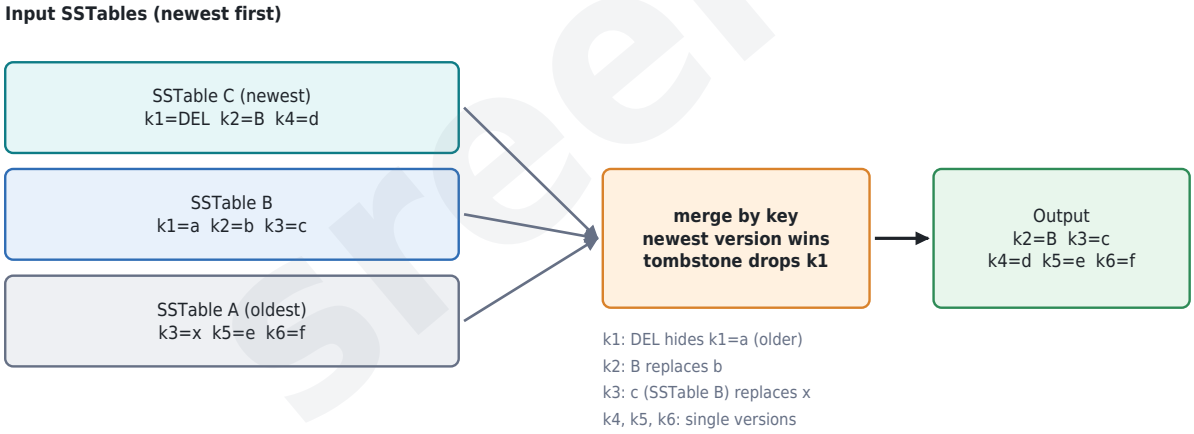


Figure 2.17. Compaction merges three overlapping SSTables into one: for every key only the newest version survives, and the tombstone for k1 removes the key because no older file remains below.

Amplification

Three ratios describe the cost of an engine and cannot all be minimised together.

Factor	Definition	B-tree	LSM-tree
Write amplification	Bytes written to storage per byte written by the application	Page rewrite plus WAL; high for random small updates	Rewrites during compaction; low (size-tiered) to high (leveled)
Read amplification	Storage reads per application read	Low: one page per level	Higher: several files per read; reduced by bloom filters
Space amplification	Bytes stored per byte of live data	Page fill and fragmentation	Stale versions awaiting compaction; temporary space during merges

The three amplification factors.

2.3.2 Worked example

The listing below is a complete simulation of the LSM design in about 70 lines of Python: a memtable, immutable sorted tables with bloom filters, tombstones, size-tiered compaction and a read path. The commit log is omitted because the simulation does not crash.

lsm.py PYTHON

```python
import hashlib, random

TOMB = "DEL"

class Bloom:
    def __init__(self, items, bits_per_key=10, k=4):
        self.n, self.k = max(8, len(items) * bits_per_key), k
        self.bits = bytearray(self.n)
        for key in items:
            for i in self._idx(key):
                self.bits[i] = 1
    def _idx(self, key):
        d = hashlib.md5(key.encode()).digest()
        return [int.from_bytes(d[4 * i:4 * i + 4], "big") % self.n for i in range(self.k)]
    def might_contain(self, key):
        return all(self.bits[i] for i in self._idx(key))

class SSTable:                      # immutable, sorted by key
    def __init__(self, data):
        self.rows = dict(sorted(data.items()))
        self.bloom = Bloom(self.rows)
        self.lo, self.hi = min(self.rows), max(self.rows)

class LSM:
    def __init__(self, memtable_limit=100, fanout=4, use_bloom=True):
        self.mem, self.levels = {}, [[]]
        self.limit, self.fanout, self.use_bloom = memtable_limit, fanout, use_bloom
        self.user_writes = self.rows_written = self.probes = 0

    def put(self, key, value):          # real systems append to a WAL first
        self.mem[key] = value
        self.user_writes += 1
        if len(self.mem) >= self.limit:
            self.flush()

    def delete(self, key):
        self.put(key, TOMB)

    def flush(self):
        if not self.mem:
            return
        self.levels[0].insert(0, SSTable(self.mem))     # newest first
        self.rows_written += len(self.mem)
        self.mem = {}
        for lvl in range(len(self.levels)):             # size-tiered compaction
            if len(self.levels[lvl]) < self.fanout:
                break
            merged = {}
            for t in reversed(self.levels[lvl]):        # oldest first, newest wins
                merged.update(t.rows)
            self.levels[lvl] = []
            if lvl + 1 == len(self.levels):
                self.levels.append([])
                merged = {k: v for k, v in merged.items() if v != TOMB}   # bottom: drop tombstones
            if merged:
                self.levels[lvl + 1].insert(0, SSTable(merged))
                self.rows_written += len(merged)

    def get(self, key):
        if key in self.mem:
            v = self.mem[key]
            return None if v == TOMB else v
        for lvl in self.levels:
            for t in lvl:
                if self.use_bloom and not t.bloom.might_contain(key):
                    continue
                self.probes += 1
```

lsm.py (continued) PYTHON

```
                if key in t.rows:
                    v = t.rows[key]
                    return None if v == TOMB else v
        return None
```

The driver below runs two experiments. The first inserts 5,000 random keys into a memtable of 100 entries, then reads 2,000 keys that do not exist, with and without bloom filters. The second replays a small trace with a memtable of 3 entries and a compaction fan-out of 3 to show the structure of the files.

lsm_driver.py PYTHON

```
import random
from lsm import LSM

random.seed(7)
for use_bloom in (False, True):
    db = LSM(use_bloom=use_bloom)
    for i in range(5000):
        db.put("k%05d" % random.randrange(100000), i)
    db.probes = 0
    for i in range(2000):
        db.get("z%05d" % i)                      # keys that do not exist
    print("bloom=%-5s sstables per level=%s probes=%d write amplification=%.2f"
          % (use_bloom, [len(l) for l in db.levels], db.probes, db.rows_written / db.user_writes))

db = LSM(memtable_limit=3, fanout=3)
ops = [("put", "k1", "a"), ("put", "k2", "b"), ("put", "k3", "c"), ("put", "k2", "B"),
       ("put", "k4", "d"), ("del", "k1", None), ("put", "k5", "e"), ("put", "k6", "f"),
       ("put", "k7", "g")]
for op, k, v in ops:
    db.put(k, v) if op == "put" else db.delete(k)
    fmt = lambda rows: ",".join("%s=%s" % kv for kv in rows.items())
    lv = " | ".join("L%d:" % i + " ".join("[" + fmt(t.rows) + "]" for t in lvl)
                    for i, lvl in enumerate(db.levels))
    print("%s %s: mem[%s] %s" % (op, k, fmt(db.mem), lv))
print("get k1 ->", db.get("k1"), "| get k2 ->", db.get("k2"))
```

Real output of lsm_driver.py TERMINAL

```
$ python3 lsm_driver.py
bloom=False sstables per level=[1, 0, 3] probes=8000 write amplification=2.89
bloom=True  sstables per level=[1, 0, 3] probes=100 write amplification=2.89
put k1: mem[k1=a] L0:
put k2: mem[k1=a,k2=b] L0:
put k3: mem[] L0:[k1=a,k2=b,k3=c]
put k2: mem[k2=B] L0:[k1=a,k2=b,k3=c]
put k4: mem[k2=B,k4=d] L0:[k1=a,k2=b,k3=c]
del k1: mem[] L0:[k1=DEL,k2=B,k4=d] [k1=a,k2=b,k3=c]
put k5: mem[k5=e] L0:[k1=DEL,k2=B,k4=d] [k1=a,k2=b,k3=c]
put k6: mem[k5=e,k6=f] L0:[k1=DEL,k2=B,k4=d] [k1=a,k2=b,k3=c]
put k7: mem[] L0: | L1:[k2=B,k3=c,k4=d,k5=e,k6=f,k7=g]
get k1 -> None | get k2 -> B
```

Reading the output. After the first experiment there are four SSTables (one at level 0, three at level 2). A miss without bloom filters probes all four tables: 2,000 reads times 4 tables = 8,000 probes. With bloom filters only 100 probes occurred, a false-positive rate of 100 / 8,000 = 1.25 percent, matching the formula above. The write amplification of 2.89 means that each user write caused 2.89 rows to be written to SSTables in total (the flush plus the rewrites during compaction), measured as rows written divided by user writes.

In the trace, the third write fills the memtable (limit 3) and flushes `{k1:a, k2:b, k3:c}`. The write `put k2=B` goes to the memtable and shadows the old `k2=b`. After `del k1` the second flush holds a tombstone `DEL` for k1; the two level-0 tables now disagree about k1, and a read examines the newer table first and returns not found. When a third table would have been created the fan-out of 3 triggers compaction: the merge keeps `k2=B` (newest), drops `k1` (tombstone with nothing below it) and writes one table at level 1. A read of `k2` returns `B` and `k1` returns nothing.

Cost of tombstones

The next program deletes 99,990 of 100,000 keys and then asks for the first five live keys, counting the entries that the read had to examine. Entries are merged across files in memory in this simulation, which isolates the effect: the scan must step over every tombstone.

Reading past tombstones (real output of tomb_demo.py) TERMINAL

```
before deletes: ['k000000', 'k000001', 'k000002', 'k000003', 'k000004'] entries examined: 5
after 99990 deletes: ['k099990', 'k099991', 'k099992', 'k099993', 'k099994'] entries examined: 99995
```

Five rows are returned in both cases, but the second query examined 99,995 entries. The cost of a range read in an LSM store depends on the dead data in the range, not on the size of the answer. Wide-column systems keep tombstones for a grace period, so a partition that is the target of frequent deletes (a queue implemented as a table) becomes slower with every delete. This is revisited in the wide-column chapter.

2.3.3 Hands-on

The B-tree side can be measured on PostgreSQL 16. The first test inserts 300,000 rows with random 32-character text keys (a primary-key B-tree receives inserts at random positions), the second inserts 300,000 rows with ascending bigint keys. In both cases the row holds a 100-byte value. The server WAL position is read before and after to obtain the WAL bytes written.

B-tree insert cost in PostgreSQL 16 (measured on the build machine; typical) TERMINAL

```
postgres=# select pg_current_wal_lsn() as lsn0 \gset
postgres=# insert into kv select md5(i::text), repeat('x',100) from generate_series(1,300000) i;
INSERT 0 300000
postgres=# select pg_size_pretty(pg_wal_lsn_diff(:'lsn1',:'lsn0')) wal_written;
 wal_written
-------------
 93 MB
postgres=# select pg_size_pretty(pg_relation_size('kv')) heap,
postgres-#        pg_size_pretty(pg_relation_size('kv_pkey')) idx;
 heap  |  idx
-------+-------
 48 MB | 22 MB
(sequential bigint keys, same row count: 67 MB of WAL)
```

The 300,000 rows carry about 40 MB of user data (32-byte key plus 100-byte value), so the WAL alone is about 2.3 times the user data for random keys and about 1.7 times for sequential keys, before the later checkpoint writes the heap and index pages again. Random keys cost more WAL because inserts land on pages that have not been modified since the last checkpoint, so PostgreSQL logs full page images (`full_page_writes`). This is the B-tree write amplification that LSM engines avoid by writing sequentially. It does not make a B-tree slow: the measured node sustained thousands of transactions per second in the first chapter. The cost appears when the sustained write rate approaches the device write bandwidth.

Workload	Favours B-tree	Favours LSM-tree
Read-heavy point and range queries, moderate writes	Yes: predictable read latency	Only with leveled compaction and caching
Sustained high ingest of small records (events, metrics)	Write amplification and random I/O limit throughput	Yes: sequential writes, batch merges
Frequent updates of the same keys	In-place update, no stale versions	Stale versions until compaction; space amplification
Many deletes within ranges	Space reused after vacuum	Tombstones slow range reads until purged
Predictable latency under load	Yes, apart from checkpoints	Compaction competes with foreground I/O
Storage on SSD with endurance limits	Higher write amplification wears flash	Size-tiered: lower write volume

Selection guide between engine families.

2.3.4 Real-world problems and solutions

Incident 7: compaction storm degrades read latency

Scenario. A time-series ingestion cluster running Cassandra with size-tiered compaction receives a bulk back-fill of several days of data in addition to live traffic. Read p99 latency rises from 15 ms to over 800 ms for several hours.

Symptoms. `nodetool compactionstats` shows a long list of pending compactions; disk I/O utilisation is at the device limit on every node; reads time out; `nodetool tablestats` reports a high SSTable count for the table.

Diagnosis. In order: (1) `nodetool compactionstats` to read the pending task count and the compaction throughput. (2) `nodetool tablestats keyspace.table` and read the lines "SSTable count" and "Local read latency". (3) `nodetool tpstats` and look at the CompactionExecutor and pending MutationStage values. (4) `iostat -x 5` to confirm the disk is saturated. (5) `nodetool getcompactionthroughput` to see the throttle.

Root cause. The back-fill produced SSTables faster than compaction could merge them. As the file count rose, each read had to consult more files (read amplification), while the compactions themselves consumed the I/O the reads needed.

Fix. Immediate: throttle the back-fill at the client; raise compaction throughput temporarily (`nodetool setcompactionthroughput 128`, in MB per second) if the disks have headroom; do not start a manual major compaction, which would need free space equal to the table and block the node. Permanent: pace bulk loads, use a compaction strategy that matches the data (time-window compaction for TTL time series), and provision I/O capacity for compaction (a common planning rule is to keep a size-tiered table below about half of the disk so that a large merge can complete).

Prevention. Alert on pending compactions and SSTable count, load-test with a realistic ingest rate including back-fills, and keep compaction throughput and disk headroom documented per cluster.

Incident 8: disk fills up although the dataset did not grow

Scenario. A table with frequent overwrites of the same keys is stored in an LSM engine. The live data is 200 GB, yet disk usage reaches 90 percent of 1 TB.

Symptoms. Disk usage grows with write volume rather than with the number of distinct keys; compaction falls behind; the node refuses writes at the disk-full threshold.

Diagnosis. (1) Compare `nodetool tablestats` "Space used (live)" and "Space used (total)". (2) List data files by age and size (`ls -lS` under the table directory) to find old large SSTables not yet merged. (3) Check the compaction strategy and its settings (`DESCRIBE TABLE`).

Root cause. Overwrites create new versions that coexist with the old ones until a compaction merges the files that contain them (space amplification). With size-tiered compaction, tables of the largest size tier are merged rarely, so stale versions persist.

Fix. Immediate: free space by removing snapshots and unneeded backups (`nodetool clearsnapshot`), then run compaction on the affected table with spare capacity (`nodetool compact` only when free space exceeds the table size). Permanent: switch the table to leveled compaction, which bounds space amplification to roughly 10 percent for a stable workload, and size disks so that usage stays under about 50 to 60 percent for size-tiered tables.

Prevention. Alert on disk usage at 60 and 75 percent, and track the ratio of total to live space.

2.3.5 Questions to expect

1. How does an LSM-tree store and read data?

Writes go to a commit log and a sorted in-memory memtable; a full memtable is flushed as an immutable SSTable; background compaction merges SSTables. A read checks the memtable and then SSTables from newest to oldest, using bloom filters to skip files that cannot contain the key.

2. Why are LSM writes fast?

They are sequential appends (log and flush) and avoid random in-place page updates; the cost of organising data is deferred to batched merges.

3. What is a bloom filter and why is a false negative impossible?

A bit array filled by k hash functions per key. A key that was inserted sets all of its k bits, so a lookup finds them all set. A key that was never inserted may find all bits set by other keys (false positive) but a stored key can never find a bit unset.

4. Explain write, read and space amplification.

Write amplification is bytes physically written per byte logically written; read amplification is the number of storage reads per logical read; space amplification is stored bytes per live byte. Engines trade one against the others.

sreram

5. Compare size-tiered and leveled compaction.

Size-tiered merges similarly sized tables: low write amplification, higher read and space amplification. Leveled keeps non-overlapping files per level: low read and space amplification, higher write amplification.

6. Why are deletes expensive in LSM stores?

A delete writes a tombstone that must be retained until all older versions are compacted away; range reads must skip tombstones, and in Cassandra tombstones are kept for `gc_grace_seconds` (default 864,000 s, ten days) to prevent deleted data from reappearing after a repair.

7. When is a B-tree preferable?

For read-heavy workloads, range scans, frequent in-place updates and predictable latency, which is why relational databases use it.

KEY POINTS

The storage engine determines the cost profile of a database node.

- B-trees update pages in place: cheap, predictable reads; random writes and WAL double writes.
- LSM-trees append to a log and memtable, flush immutable SSTables and compact in the background: high write throughput, deferred cost.
- Bloom filters (about 1.2 percent false positives at 10 bits per key) make reads of absent keys cheap.
- Tombstones make range reads proportional to dead data until compaction removes them; the simulation examined 99,995 entries to return 5 rows.
- Compaction backlog and disk headroom are the principal operational risks of LSM stores.

2.4 Key-value stores

A key-value store maps a key to a value and offers little else: `get(key)`, `put(key, value)` and `delete(key)`. The value is opaque to the engine, or a small typed structure. The restriction is the source of the design's strength. Because every operation names its key, the key can be hashed to a partition and the request routed to exactly one replica set, without a query planner and without cross-partition coordination. Latency stays flat as the cluster grows. The price is that all other access paths (find by attribute, aggregate, join) are absent or must be built by the application.

Three systems illustrate the range. Redis is an in-memory store with rich value types (strings, hashes, lists, sets, sorted sets, streams) used for caching, sessions, counters and queues; a complete guide exists elsewhere, and only the points that matter for database selection are recapitulated here. Riak and the original Amazon Dynamo are leaderless stores built on consistent hashing and vector clocks, described in the chapter on distributed fundamentals. Amazon DynamoDB (2012) is a managed service whose design descends from the Dynamo paper but whose behaviour differs from it: it offers a table abstraction with a composite primary key, secondary indexes, conditional writes and transactions, and a leader per partition rather than leaderless replication. The remainder of this chapter concentrates on key design, because the choice of key determines what can be queried efficiently.

2.4.1 Concept and mechanism

Redis recapitulated

Redis keeps the dataset in memory and executes commands on a single thread, so each command is atomic without locks. Durability is optional: RDB snapshots at intervals and an append-only file (AOF) with a configurable fsync policy (`always`, `everysec`, `no`). Replication is asynchronous from a primary to replicas; Redis Cluster divides the key space into 16,384 hash slots assigned to primaries. Data larger than memory is not supported without eviction (`maxmemory-policy`). Typical roles are cache in front of a relational store, session store, rate limiter, leaderboard (sorted set) and lightweight queue. Because replication is asynchronous, an acknowledged write can be lost on failover; Redis is therefore normally not the system of record.

DynamoDB data model

A DynamoDB table holds items, each a set of attributes (strings, numbers, binary, booleans, lists, maps, sets). Only the primary key attributes are mandatory and typed. The primary key is either a **partition key** alone, or a partition key plus a **sort key**. DynamoDB hashes the partition key to choose the physical partition; items sharing a partition key are stored together ordered by the sort key, which makes a query for a key range inside one partition efficient.

Documented limits that shape designs (Amazon DynamoDB Developer Guide): an item is at most 400 KB including attribute names; a partition key value is at most 2,048 bytes and a sort key value 1,024 bytes; a `Query` returns at most 1 MB per call and continues with `LastEvaluatedKey`; `BatchWriteItem` accepts up to 25 put or delete requests; a transaction covers up to 100 unique items and 4 MB. A table with local secondary indexes limits each item collection (all items with one partition key value) to 10 GB.

Operation	What it reads	Cost behaviour
GetItem	One item by full primary key	One item read; the cheapest operation
Query	Items of one partition key, optionally a sort-key range, optionally filtered	Reads only the addressed items; a filter expression is applied after reading and does not reduce consumed capacity
Scan	Every item of the table or index	Reads the whole table in 1 MB pages; consumes capacity proportional to table size

DynamoDB read operations.

Capacity and partitions

Throughput is measured in read and write capacity units. One read unit is one strongly consistent read per second of an item up to 4 KB (two eventually consistent reads); one write unit is one write per second of an item up to 1 KB. The documentation states that each partition is designed to deliver up to 3,000 read units and 1,000 write units per second, and that adaptive capacity (available in both on-demand and provisioned modes) shifts capacity towards busier partitions. A single partition key value therefore has a hard ceiling of about 1,000 item writes per second regardless of table capacity. The design rule that follows from the documentation is to spread activity evenly across partition key values.

Secondary indexes

A **global secondary index** (GSI) maintains a second copy of selected attributes under a different partition and sort key; it is updated asynchronously, so a read from a GSI is eventually consistent, and it has its own capacity. A **local secondary index** (LSI) shares the partition key of the table with a different sort key, must be created together with the table, supports strongly consistent reads and carries the 10 GB item-collection limit. Both exist because the primary key serves exactly one access pattern.

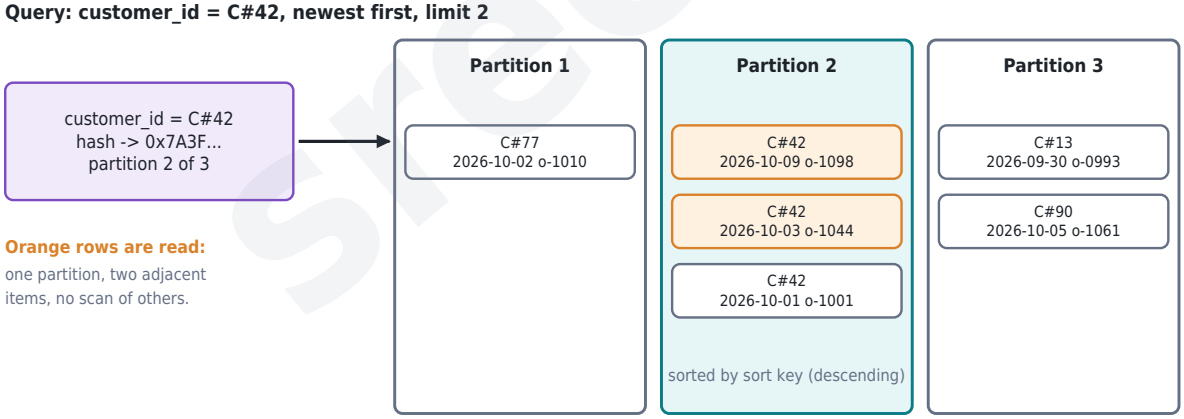


Figure 2.18. An orders table with partition key customer_id and sort key order_date: the partition key is hashed to one of the partitions, and inside a partition items are ordered by the sort key, so the latest orders of one customer are adjacent.

2.4.2 Worked example

Requirements: show a customer's most recent orders; fetch one order by id; list all orders in status SHIPPED for a day. The primary key serves the first requirement directly.

customer_id	order_ts_id	status	total
C#42	2026-10-01T09:14#o-1001	SHIPPED	1999
C#42	2026-10-03T18:02#o-1044	PLACED	450

customer_id	order_ts_id	status	total
C#42	2026-10-09T11:40#o-1098	SHIPPED	7200
C#77	2026-10-02T08:30#o-1010	PLACED	990

Items in the table Orders (the partition key is customer_id and the sort key is order_ts_id).

1. **Latest 10 orders of customer 42.** `Query` with `customer_id = C#42`, `ScanIndexForward = false`, `Limit = 10`. DynamoDB hashes `C#42`, reaches one partition and reads ten adjacent items in reverse sort order. Cost: about ten items.
2. **One order by id.** The id alone is not a key. Either the client supplies the customer and timestamp, or a GSI with partition key `order_id` is added (`GSI1`), which copies the item under the new key.
3. **Shipped orders for a day.** No key serves this. A GSI with partition key `ship_day` (for example `2026-10-09`) and sort key `order_id` serves it, but all orders of one day share one partition key value: at 5,000 orders per second the index partition receives more than its 1,000 writes per second limit. The fix is write sharding: `ship_day#<n>` with `n` in 0..9 chosen by hash of the order id; the reader queries the ten shards and merges. The cost of the access pattern is explicit in the model.

The same table in Java with the AWS SDK v2 Enhanced Client is shown below. It is typical code and was not run on the build machine (no AWS service or SDK is available here).

OrderRepository.java (fragment, not run) JAVA

```java
public List<Order> latestOrders(String customerId, int n) {
    QueryConditional key = QueryConditional.keyEqualTo(Key.builder()
            .partitionValue(customerId).build());
    QueryEnhancedRequest request = QueryEnhancedRequest.builder()
            .queryConditional(key)
            .scanIndexForward(false)          // newest first: descending sort key
            .limit(n)
            .build();
    return table.query(request).items().stream().limit(n).toList();
}

public void markShipped(String customerId, String orderTsId) {
    Expression expected = Expression.builder()
            .expression("#s = :placed")
            .putExpressionName("#s", "status")
            .putExpressionValue(":placed", AttributeValue.fromS("PLACED"))
            .build();
    UpdateItemEnhancedRequest<Order> req = UpdateItemEnhancedRequest.builder(Order.class)
            .item(new Order(customerId, orderTsId, "SHIPPED"))
            .conditionExpression(expected)     // optimistic concurrency, fails if not PLACED
            .build();
    table.updateItem(req);
}
```

What to notice: the query names the partition key and nothing else, so it is a single-partition read; the conditional update is the DynamoDB mechanism for the read-modify-write safety discussed in the consistency chapter, and a `ConditionalCheckFailedException` signals the conflict.

2.4.3 Hands-on

Hot keys can be examined before any database is bought, by simulating the key distribution of a workload against a hash partitioner. The program hashes 100,000 writes into 8 partitions using five candidate key designs.

Writes per partition for five key designs (real output of hot.py; hash is MD5 modulo 8) TERMINAL

```
key = device_id (10000 devices)
  [12276, 12672, 12710, 12578, 12785, 12071, 12583, 12325]  max/avg=1.02
key = date (one day of traffic)
  [100000, 0, 0, 0, 0, 0, 0, 0]  max/avg=8.00
key = country (skewed 60/20/10..)
  [0, 0, 19891, 70138, 0, 0, 4013, 5958]  max/avg=5.61
key = date#bucket(0..15)
  [6213, 18742, 18549, 18851, 6338, 6287, 12611, 12409]  max/avg=1.51
key = celebrity user (1 of 3 hits)
  [8305, 8232, 8522, 8283, 8208, 8453, 41513, 8484]  max/avg=3.32
```

A high-cardinality key (device id) gives the ideal ratio 1.02 of the busiest partition to the average. A low-cardinality key such as the date sends all traffic to one partition (ratio 8.00): the table is as fast as one partition. A skewed key

(country) fails in proportion to the skew. Appending a random bucket number from 0 to 15 spreads the date over 16 key values; with only 8 partitions and a hash, the result is better but not uniform (1.51), because 16 random placements over 8 partitions collide. A single very popular item (the celebrity) concentrates one third of the load on one partition even though cardinality is high; no choice of hash fixes this, and the remedies are caching in front of the store, replicating the popular item under several keys (`user:celeb#0..9`) and reading from a random copy, or batching its updates.

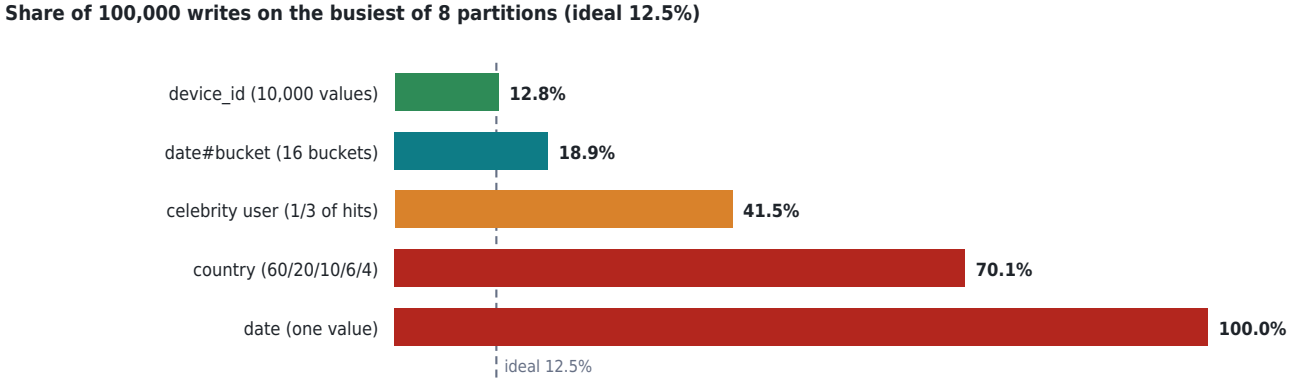


Figure 2.19. Share of 100,000 writes received by the busiest of eight partitions under five key designs: low-cardinality and skewed keys overload one partition, while a high-cardinality key or a bucketed key stays near the 12.5 percent ideal.

2.4.4 Real-world problems and solutions

Incident 9: hot partition from a poor key

Scenario. An IoT platform stores readings in a DynamoDB table with partition key `date` (YYYY-MM-DD) and sort key `device_id#timestamp`, provisioned at 20,000 write units. Every device writes continuously.

Symptoms. `ProvisionedThroughputExceededException` (HTTP 400) in the application although the CloudWatch metric for consumed write capacity at table level is only 10 percent of the provisioned value. Latency of puts rises; throttled-request metrics are non-zero.

Diagnosis. (1) Compare `ConsumedWriteCapacityUnits` with `WriteThrottleEvents` in CloudWatch: throttling with low average consumption indicates a hot key. (2) Enable CloudWatch Contributor Insights for DynamoDB, which lists the most accessed and most throttled partition key values. (3) Inspect the key design: all writes of a day carry the same partition key value. (4) Compute the write rate per key value: 15,000 writes per second against the limit of about 1,000 per partition.

Root cause. The partition key has one value per day, so the entire write load of that day maps to one partition, whose capacity is bounded by the documented per-partition limit, independent of table capacity.

Fix. Immediate: reduce the write rate (client back-off with jitter, queue in front); adaptive capacity may absorb a part, but it does not remove the limit. Permanent: change the partition key to `device_id` (or `device_id#date` to bound the partition size) and keep the timestamp as sort key; for a time-range query across devices, query per device or maintain a GSI with a sharded day key (`date#shard`). The table must be rewritten, which is done by creating a new table and back-filling it.

Prevention. Evaluate the key against the access pattern and the write distribution before launch, using the simulation above or a load test with production-like key skew. Review Contributor Insights after each release.

Incident 10: a scan in a request path

Scenario. A "find user by email" endpoint is implemented as a `Scan` with a filter expression on `email`.

Symptoms. The endpoint is fast in testing with 1,000 users and takes seconds and consumes thousands of read units per call with 20 million users; the bill for read capacity rises with traffic.

Diagnosis. (1) Check the SDK call: `scan(...)` with `FilterExpression`. (2) Compare `ScannedCount` and `Count` in the response: millions scanned, one returned. (3) Look at the consumed capacity per call (`ReturnConsumedCapacity=TOTAL`).

Root cause. A filter is applied after items are read, so the whole table is read to return one item.

Fix. Create a GSI with partition key `email` and query it (eventually consistent), or store a second item `EMAIL#<address>` pointing to the user id and read it with `GetItem`, which also allows a conditional put to enforce uniqueness.

Prevention. Forbid `Scan` in request paths in code review; use scans only for batch jobs and exports.

2.4.5 Questions to expect

1. How does DynamoDB decide where an item is stored?

The partition key value is hashed to select a partition; the sort key orders items within the partition. All items with the same partition key are stored together.

2. What is a hot partition and how is it avoided?

A partition receiving a disproportionate share of traffic, limited by the per-partition throughput (documented as 1,000 write and 3,000 read units per second). It is avoided with high-cardinality partition keys, write sharding with a bucket suffix, caching for popular items and, where needed, replicas of hot items.

3. Query versus Scan?

Query reads one partition key (and optionally a sort-key range) and costs in proportion to the items read. Scan reads the whole table or index; a filter expression only trims the result and does not reduce the capacity consumed.

4. GSI versus LSI?

A GSI has its own partition and sort key and capacity, is eventually consistent and can be added later. An LSI keeps the table's partition key with another sort key, offers strong consistency, must be defined at table creation and bounds each item collection to 10 GB.

5. Why is a key-value store not a good fit for ad-hoc reporting?

Only key-addressed access is efficient; arbitrary predicates need a scan. Reporting data is usually exported to an analytics store.

6. When is Redis acceptable as the primary store?

When data loss of the last moments is tolerable or the data is reproducible: caches, sessions, rate-limit counters. Because replication is asynchronous and persistence is optional or batched, it is not normally the system of record for orders or payments.

KEY POINTS

A key-value store trades query flexibility for predictable, partition-local operations.

- Every efficient operation names the key; design the key from the access patterns.
- DynamoDB partition key plus sort key supports single-partition range queries; other patterns need GSIs or duplicated items.
- A single partition key value is bounded by per-partition throughput (1,000 write units per second in the documentation); a low-cardinality or skewed key throttles the table although table capacity is unused.
- Filters do not reduce the cost of a Scan; avoid scans on request paths.
- Use conditional writes for read-modify-write safety.

2.5 Wide-column stores

Wide-column stores descend from Google Bigtable (2006) and Amazon Dynamo (2007). Apache Cassandra combines the Bigtable data model with the Dynamo distribution model: rows are grouped into partitions placed on a consistent-hash ring, replicated to N nodes without a leader, and stored in an LSM engine. Apache HBase and Google Bigtable follow the Bigtable design more closely: range-partitioned tablets or regions served by one node each, on a distributed file system. ScyllaDB is a Cassandra-compatible implementation in C++. This chapter uses Cassandra and its query language CQL. The Cassandra commands in this chapter are written from the Apache Cassandra documentation and are typical output; Cassandra is not installed on the build machine and the outputs are marked accordingly.

The name is misleading: a Cassandra table looks like a relational table in CQL, and the "wide" refers to the physical layout, where one partition can hold many rows with their own columns, sorted by clustering key.

2.5.1 Concept and mechanism

Data model

A keyspace holds tables and declares the replication strategy and factor. A table has a **primary key** composed of a **partition key** (one or more columns, hashed to place the partition on the ring) and optional **clustering columns** (which determine the sort order of rows inside the partition). The partition key decides where; the clustering columns decide the order. Every query must supply the full partition key; clustering columns can be restricted by equality or range in their declared order.

sensor_readings.cql (typical, not run) SQL

```sql
CREATE KEYSPACE iot WITH replication =
  {'class': 'NetworkTopologyStrategy', 'dc1': 3};

CREATE TABLE iot.readings_by_sensor_day (
  sensor_id  text,
  day        date,
  ts         timestamp,
  temperature double,
  PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC)
  AND default_time_to_live = 7776000;      -- 90 days

INSERT INTO iot.readings_by_sensor_day (sensor_id, day, ts, temperature)
VALUES ('s-17', '2026-10-18', '2026-10-18 06:00:01', 21.4);

SELECT ts, temperature FROM iot.readings_by_sensor_day
WHERE sensor_id = 's-17' AND day = '2026-10-18' AND ts >= '2026-10-18 06:00:00'
LIMIT 100;
```

The partition key is the pair `(sensor_id, day)`, so one sensor-day is one partition: it is bounded (at most one day of readings) and placed by hash. The clustering column `ts` sorts readings newest first, so the latest hundred readings are the first hundred rows of the partition. Adding the day to the key is the standard technique to bound partition size, called bucketing.

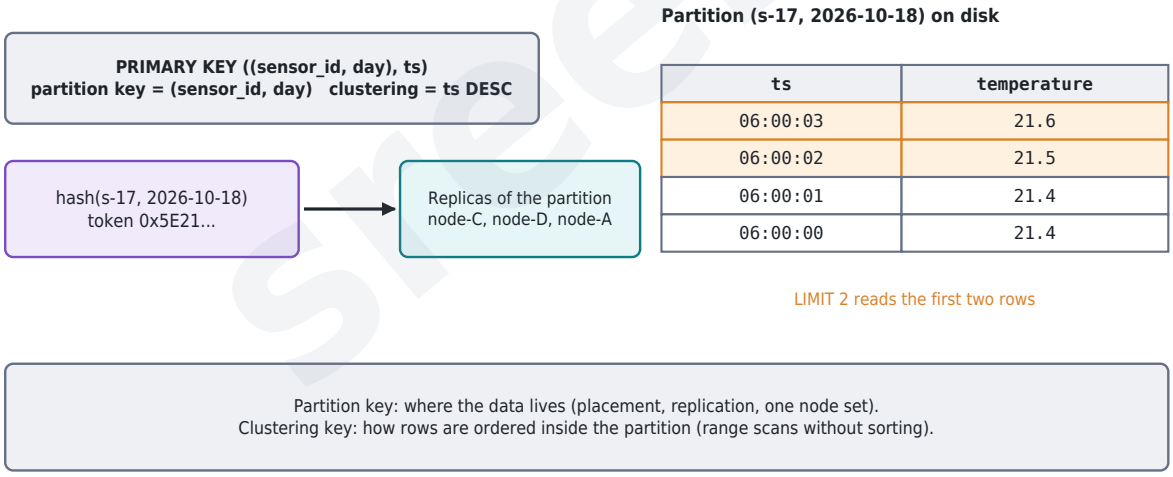


Figure 2.20. Anatomy of a Cassandra primary key: the partition key (sensor_id, day) is hashed to a token that selects replicas; inside the partition, rows are stored sorted by the clustering column ts.

CQL rejects queries that cannot be served from a single partition unless `ALLOW FILTERING` is added; the documentation states that such queries have unpredictable latency because the data scanned is not bounded by the answer. The rejection is intentional: it exposes at write time the access paths that the data model does not support.

Consistency levels

Cassandra lets each request choose how many replicas must respond. The coordinator always sends a write to all replicas; the level only controls how many acknowledgements it waits for. The documentation defines the levels as follows.

Level	Replicas that must respond
ANY	Write only: any replica, or a stored hint
ONE, TWO, THREE	One, two or three replicas
QUORUM	A majority of all replicas (RF/2 + 1, rounded down)
LOCAL_QUORUM	A majority of the replicas in the coordinator's data centre
EACH_QUORUM	A majority in every data centre (writes)
LOCAL_ONE	One replica in the local data centre
ALL	Every replica
SERIAL / LOCAL_SERIAL	Paxos phase for lightweight transactions (`IF NOT EXISTS`, `IF` conditions)

Cassandra consistency levels.

With RF = 3, QUORUM is 2; writing and reading at QUORUM gives R + W = 4 > 3. A multi-data-centre deployment normally uses LOCAL_QUORUM for both so that requests do not wait for a remote data centre, which keeps the guarantee within one data centre. Lightweight transactions run a Paxos round and cost several round trips; the documentation advises to use them sparingly.

Write path and tombstones

A write is appended to the commit log, applied to the memtable and acknowledged; memtables flush to SSTables and compaction merges them (the storage chapter explains the mechanism). A `DELETE`, an `UPDATE` that sets a column to null, and an expired TTL all create tombstones. The documentation states that tombstones are retained until `gc_grace_seconds` (default 864,000 seconds, ten days) has passed and a compaction then removes them; the grace period gives a node that was down time to receive the delete through repair, because otherwise it would still hold the old value and repair it back into the cluster (a zombie). Consequently, repair must complete on every node within `gc_grace_seconds`.

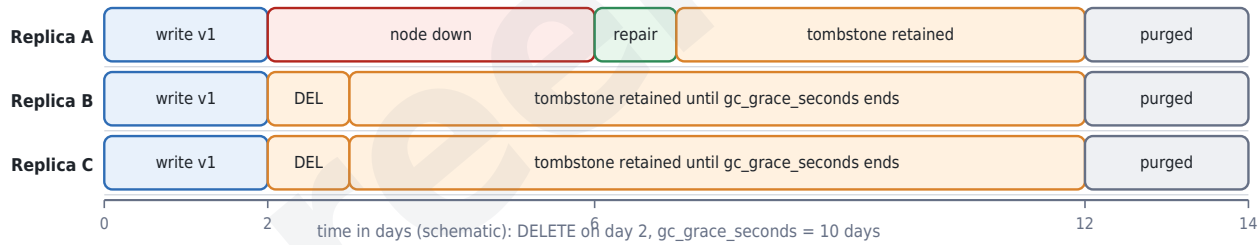


Figure 2.21. Life of a delete: the tombstone is written at once, survives for gc_grace_seconds so that every replica can receive it by repair, and is purged by the first compaction after the grace period; a node that missed the delete and is repaired too late brings the deleted row back.

A read of a partition must merge live cells and tombstones from the memtable and every SSTable that overlaps. Cassandra emits a warning when a read scans more tombstones than `tombstone_warn_threshold` (1,000 in the default configuration of recent versions) and aborts the query at `tombstone_failure_threshold` (100,000); these defaults should be confirmed in the `cassandra.yaml` of the version deployed.

Modelling rules

1. Model one table per query. The same data is stored in several tables with different keys (`orders_by_customer`, `orders_by_day`), written by the application, optionally in a logged batch.
2. Keep partitions bounded. A commonly cited guideline is to stay below roughly 100 MB or 100,000 cells per partition, and certainly below the point where a single partition no longer fits comfortably in a repair or compaction. The figure is guidance from practice, not a hard limit.
3. Avoid read-before-write patterns and avoid secondary indexes on high-cardinality columns; both turn into multi-node operations.
4. Do not model queues or frequently deleted rows in one partition (the tombstone anti-pattern).
5. Avoid multi-partition batches except for keeping denormalised copies in step; the documentation notes that a logged batch across partitions pays a performance penalty and is not a substitute for a transaction.

2.5.2 Worked example

A messaging service stores chat messages and reads "the latest 50 messages of a conversation".

First design: `PRIMARY KEY (conversation_id, message_time)`. A very active group chat accumulates 40 million messages over two years in one partition. Reads of the latest messages are still fast (they read the head of a clustering order), but compaction, repair and the in-memory index of the partition become expensive, and the node holding the replicas of that partition becomes a hot spot.

Second design: `PRIMARY KEY ((conversation_id, bucket), message_time)` with `bucket` = month (`2026-10`). Each partition holds one month. A "latest 50" query reads the current bucket, and if it returns fewer than 50 rows, the previous bucket. The application computes the buckets. The partition size is bounded by one month of traffic; a chat with 100,000 messages per month at 300 bytes holds about 30 MB per partition.

Typical output (not run on the build machine): inspecting a table with nodetool TERMINAL

```
$ nodetool tablestats chat.messages_by_conversation
Table: messages_by_conversation
    SSTable count: 14
    Space used (live): 5.21 GiB
    Compacted partition maximum bytes: 31457280
    Average live cells per slice (last five minutes): 52.0
    Average tombstones per slice (last five minutes): 0.0
```

The line "Compacted partition maximum bytes" is the first check for oversized partitions; in the second design it stays near 30 MB.

2.5.3 Real-world problems and solutions

Incident 11: tombstone accumulation slows reads (queue in a table)

Scenario. A task queue is stored in a Cassandra table with partition key `queue_name` and clustering key `enqueued_at`. Workers read the oldest task, process it and delete it.

Symptoms. Read latency of "next task" grows from milliseconds to seconds over days. The log contains lines such as `Read 1 live rows and 18432 tombstone cells for query SELECT ... (see tombstone_warn_threshold)` (typical message form), and eventually `TombstoneOverwhelmingException` aborts reads.

Diagnosis. (1) `nodetool tablestats keyspace.table` and read "Average tombstones per slice". (2) `nodetool tablehistograms keyspace.table` for the distribution of cells per read. (3) `TRACING ON` in `cqlsh` for the query, which prints the number of live rows and tombstone cells scanned. (4) Check `gc_grace_seconds` and the compaction strategy.

Root cause. The query reads the oldest rows first, which are exactly the deleted ones. Deleted rows remain as tombstones for ten days by default, and the read must scan past all of them to reach a live row; the cost of the read is proportional to the number of deletes, not to the number of live tasks. The simulation in the storage chapter shows the same effect: 5 rows returned after 99,995 entries examined.

Fix. Immediate: lower `gc_grace_seconds` for this table only if repair runs more frequently than the new value, and force a compaction of the table (`nodetool compact keyspace table`) to purge expired tombstones. Permanent: time-bucket the queue into partitions that are dropped as a whole (`PRIMARY KEY ((queue_name, hour), enqueued_at)` with TTL or whole-partition deletion, and time-window compaction), or move the queue to a system designed for it (a message broker or Redis streams).

Prevention. Treat delete-heavy and queue-like access as a modelling smell in Cassandra reviews; alert on the tombstone-per-slice statistic.

Incident 12: unbounded growth of a partition

Scenario. Per-user activity events are stored under `PRIMARY KEY (user_id, event_time)`. After a year the largest users own several gigabytes in one partition.

Symptoms. Timeouts on a handful of users; one node with high heap usage and long garbage collection pauses; repair streams of those partitions fail; `nodetool tablestats` shows "Compacted partition maximum bytes" in the gigabytes.

Diagnosis. (1) `nodetool tablestats` and `nodetool tablehistograms` (partition size percentiles). (2) Identify the offending keys with a sampling tool (`nodetool toppartitions`). (3) Confirm that the key has no time component.

Root cause. The partition key is unbounded: the partition grows for the life of the user, so its size is not controlled by the data model.

Fix. Add a time bucket to the partition key (`(user_id, month)`), migrate by copying old data into the new table (application job or Spark), and switch reads to query the latest buckets first. Permanent: set a partition size budget in the design and test it with the heaviest realistic user.

Prevention. Review every Cassandra table for a statement of the maximum partition size and for its growth rate; include the heaviest user in load tests.

2.5.4 Questions to expect

1. Explain partition key and clustering key.

The partition key (hashed) selects the replicas that store the partition; the clustering columns sort rows within it. Queries must name the partition key; range queries are possible on clustering columns.

2. Why does Cassandra not support joins?

Rows of different partitions are on different nodes, and a join would require cross-node data movement at read time. The model instead stores a denormalised table per query.

3. What is the effect of consistency level QUORUM with RF = 3 on reads and writes?

Both wait for two replicas; since 2 + 2 > 3 a read overlaps every acknowledged write. A single replica failure is tolerated for both.

4. What is a tombstone and why is gc_grace_seconds important?

A delete marker that hides older data. It is kept for the grace period so that replicas that missed the delete can receive it through repair. If repair is slower than the grace period, deleted data can return; if the period is long and deletes are heavy, reads scan many tombstones.

5. How is a partition kept bounded?

By adding a time or hash bucket to the partition key so that each partition has a maximum size, and by querying the relevant buckets.

6. Compare Cassandra and HBase.

Cassandra is leaderless with consistent-hash partitioning and tunable consistency; HBase uses range-partitioned regions served by one region server each, built on HDFS, with strong consistency per row and sorted scans across keys.

7. When are lightweight transactions appropriate?

For rare operations that need compare-and-set semantics, such as claiming a unique username, because they use Paxos and cost several round trips.

KEY POINTS

Wide-column stores serve known queries at scale by pre-arranging data per partition.

- Primary key = partition key (placement) + clustering columns (order within the partition).
- Model one table per query, duplicate data deliberately, and bound partition size with buckets.
- Consistency is chosen per request; QUORUM on reads and writes (RF = 3) gives overlap.
- Deletes create tombstones kept for gc_grace_seconds (864,000 s default); repair must complete within it.
- Queues, unbounded partitions and scans with ALLOW FILTERING are the classic failure patterns.

sheeran

2.6 Graph databases

Relational databases store relationships as foreign keys and recover them with joins. For queries of fixed depth this is efficient. For queries whose depth is not known in advance (friends of friends of friends, all components of an assembly, shortest route between two accounts) every additional level adds a self-join, and the query text and the planner's work grow with the depth. A graph database stores relationships as first-class data and offers a query language whose basic operation is traversal. This chapter describes the property graph model, the Cypher query language used by Neo4j, and a measurement that shows where a relational database with a recursive query matches a graph database and where it stops being a good tool.

2.6.1 Concept and mechanism

The property graph model

A property graph consists of **nodes** (vertices) and **relationships** (edges). A node has zero or more **labels** (Person, Product) and a set of key-value **properties**. A relationship connects two nodes, has exactly one **type** (FOLLOWS, BOUGHT) and a direction, and can also carry properties (`since`, `rating`). Two other models exist: RDF triples (subject, predicate, object), queried with SPARQL and used for linked data, and the multi-model approach of storing graphs in another engine.

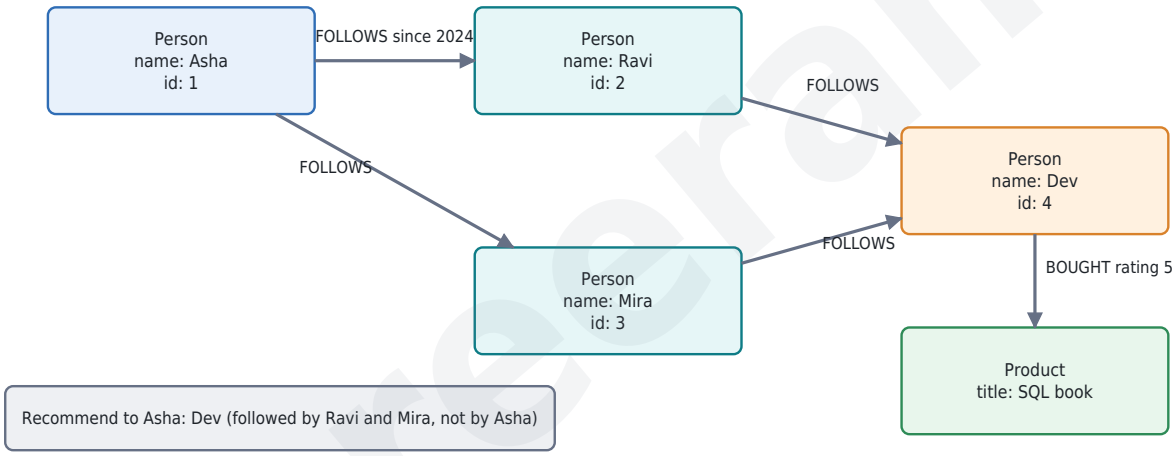


Figure 2.22. A property graph: labelled nodes with properties, typed directed relationships with their own properties; a recommendation query follows two FOLLOWS edges from Asha and excludes people she already follows.

Native graph storage

Neo4j documents a storage design called index-free adjacency: each node record points directly to its first relationship record, and relationship records link to the next relationship of the same node, so moving from a node to its neighbours follows record pointers instead of searching an index. The cost of a traversal is proportional to the number of nodes and relationships visited, not to the total size of the graph. A relational database reaches the neighbours through an index lookup per step (logarithmic in the table size). For graphs that fit the cache both costs are small; the practical difference emerges for deep or unbounded traversals and for the readability of the query.

Cypher in brief

Cypher expresses a pattern drawn with ASCII art: parentheses for nodes, square brackets for relationships, arrows for direction. The following statements are typical Cypher and were not run on the build machine (Neo4j is not installed there); the relational equivalent of the recommendation query was run on PostgreSQL and is shown below.

social.cypher (typical, not run)

```
// create two people and a relationship
CREATE (a:Person {id: 1, name: 'Asha'})-[:FOLLOWS {since: 2024}]->(b:Person {id: 2, name: 'Ravi'});

// everyone Asha reaches in one to three hops, without duplicates
MATCH (a:Person {id: 1})-[:FOLLOWS*1..3]->(f:Person)
RETURN DISTINCT f.name;
```

social.cypher (typical, not run) (continued)

```
// recommendation: friends of friends that Asha does not yet follow, ranked by common contacts
MATCH (me:Person {id: 1})-[:FOLLOWS]->(:Person)-[:FOLLOWS]->(rec:Person)
WHERE rec <> me AND NOT (me)-[:FOLLOWS]->(rec)
RETURN rec.name, count(*) AS common
ORDER BY common DESC LIMIT 5;

// shortest path between two people, any depth up to 6
MATCH p = shortestPath((a:Person {id: 1})-[:FOLLOWS*..6]-(b:Person {id: 77777}))
RETURN p;
```

The variable-length pattern `[:FOLLOWS*1..3]` replaces a hand-written chain of joins, and `shortestPath` is a built-in operation. Indexes in a graph database are used to find the start nodes (`:Person(id)`); after that the query follows relationships.

2.6.2 Worked example

A social-network schema is loaded into PostgreSQL 16 to compare the relational form with the Cypher form. The graph has 100,000 persons and 999,945 FOLLOWS edges (each person follows about ten random others), stored as follows.

graph_schema.sql SQL

```sql
CREATE TABLE person  (id int PRIMARY KEY, name text);
CREATE TABLE follows (
  src int NOT NULL,
  dst int NOT NULL,
  PRIMARY KEY (src, dst)
);
CREATE INDEX ON follows (dst);      -- reverse direction lookups
```

The recommendation query for person 1 in SQL has the same logical structure as the Cypher pattern: two edge steps and an exclusion of existing edges.

recommend.sql SQL

```sql
SELECT f2.dst AS recommended, count(*) AS common
FROM follows f1
JOIN follows f2 ON f2.src = f1.dst
WHERE f1.src = 1 AND f2.dst <> 1
  AND NOT EXISTS (SELECT 1 FROM follows x WHERE x.src = 1 AND x.dst = f2.dst)
GROUP BY f2.dst
ORDER BY common DESC, recommended
LIMIT 5;
```

Plan of the recommendation query (real output; warm cache) TERMINAL

```
Hash Anti Join (actual rows=100 loops=1)
  ->  Nested Loop (actual rows=100 loops=1)
        ->  Index Only Scan using follows_pkey on follows f1 (actual rows=10 loops=1)
              Index Cond: (src = 1)
        ->  Index Only Scan using follows_pkey on follows f2 (actual rows=10 loops=10)
              Index Cond: (src = f1.dst)
Execution Time: 0.254 ms
```

The query visits 10 neighbours and their 100 edges, using index lookups only; 0.25 ms of execution time (1.8 ms including planning and client time; the first execution on a cold cache took 1.5 seconds, the cost of reading uncached pages). A native graph database would perform the same traversal. At fixed depth two, a relational database with a suitable index is competitive.

2.6.3 Hands-on

For variable depth the relational form is a recursive common table expression. The statement below computes the nodes reachable from person 1 within `d` steps (UNION removes duplicate pairs of node and depth, which keeps cycles from looping forever) and is run for increasing depth limits.

reach.sql SQL

```sql
WITH RECURSIVE reach(node, depth) AS (
  SELECT 1, 0
  UNION
  SELECT f.dst, r.depth + 1
  FROM reach r JOIN follows f ON f.src = r.node
  WHERE r.depth < 5            -- depth limit
)
SELECT max(depth), count(DISTINCT node) FROM reach;
```

Method	Depth	Result	Time
Chain of joins	exactly 2	100 distinct nodes	5 ms
Chain of joins	exactly 3	996 distinct nodes	15 ms
Chain of joins	exactly 4	9,475 distinct nodes	70 ms
Recursive CTE	limit 4	pairs per level 10, 100, 996, 9,475	48 ms
Recursive CTE	limit 5	64,922 distinct nodes in total	646 ms
Recursive CTE	limit 6	99,844 distinct nodes in total	1,794 ms

Reachability from person 1 in a graph of 100,000 nodes and about 1 million edges (PostgreSQL 16; measured on the build machine, typical).

The figures show the behaviour of any traversal in a graph with an average out-degree of 10: the frontier grows tenfold per level (10, 100, 996, 9,475) until it covers most of the graph; at depth 6 nearly all 100,000 nodes (99,844) have been reached. The time grows with the number of edges visited: 5 ms for a few hundred edge visits, 1.8 seconds for the order of a million. A native graph database would also visit these million edges at depth 6; its advantage is constant-factor (pointer-following against index probes, perhaps an order of magnitude) and in query expressiveness, not an escape from the growth of the frontier.

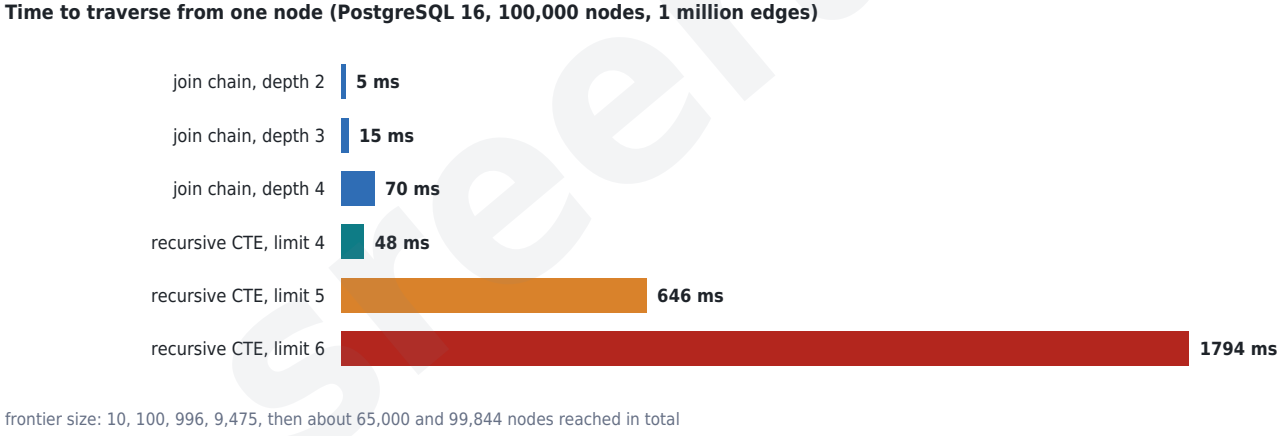


Figure 2.23. Query time against traversal depth for the SQL join chain and recursive CTE on a graph of 100,000 nodes and 1 million edges: the time follows the growth of the frontier (tenfold per level), not the size of the table.

When a graph database is the better choice

- The central queries are traversals of unknown or large depth: shortest path, reachability, influence, fraud rings, network and dependency analysis.
- The pattern is the question: "accounts that share a device with an account that shares a card with a flagged account" is a short Cypher path and a many-join SQL statement.
- Relationships change as often as entities, and relationship properties (weights, timestamps) matter.
- A relational database remains the better choice when queries are mostly entity lookups with at most two or three joins, when the dataset requires strict constraints and reporting, or when the team already operates PostgreSQL: recursive CTEs, `ltree` and extensions cover moderate graph needs.

Graph databases have costs: sharding a graph is hard because edges cross any partition boundary, so most graph databases scale reads by replicas and keep the graph on one writer; the operational ecosystem is smaller; and analytical whole-graph algorithms (PageRank, community detection) are usually run in a separate engine.

2.6.4 Real-world problems and solutions

Incident 13: unbounded traversal exhausts the database

Scenario. A "network of influence" feature runs `MATCH (a)-[:FOLLOWS*]->(b)` (no upper bound) in a request path, or its SQL equivalent without a depth predicate.

Symptoms. Most requests finish in milliseconds; requests for well-connected users run for minutes, consume memory and cause timeouts for other requests. In the relational version the recursive CTE returns hundreds of thousands of rows to the application.

Diagnosis. (1) Identify the long-running statements: `SELECT pid, now() - query_start, query FROM pg_stat_activity WHERE state = 'active' ORDER BY 2 DESC;` in PostgreSQL, or `SHOW TRANSACTIONS` in Neo4j 5 or `CALL dbms.listQueries()` in 4.x. (2) Test the statement for one hub node and count rows per level (the table above shows the tenfold growth). (3) Check the degree of the starting node (`SELECT count(*) FROM follows WHERE src = ?`).

Root cause. The frontier of a traversal multiplies by the average out-degree at every level; a hub node with high degree or a graph with a small diameter makes an unbounded traversal touch most of the graph.

Fix. Immediate: cancel the statement (`SELECT pg_cancel_backend(pid)`), set a statement timeout (`SET statement_timeout = '2s'` or the transaction timeout setting of Neo4j, `dbms.transaction.timeout` in 4.x and `db.transaction.timeout` in 5.x; verify the name for the version deployed). Permanent: bound the depth (`*1..3`), limit the result (`LIMIT`), cap high-degree nodes (sample their neighbours), precompute popular results and cache them, and run whole-graph analytics offline.

Prevention. Reject unbounded variable-length patterns in code review, set server-side query timeouts and test every traversal against the highest-degree node in production data.

2.6.5 Questions to expect

1. What is a property graph?

A data model of labelled nodes and typed, directed relationships, both with key-value properties.

2. What problem does a graph database solve better than a relational database?

Queries whose depth is variable or large, and queries expressed as patterns over relationships (shortest path, recommendations, fraud rings). The relational model needs self-joins or recursive CTEs whose cost and complexity grow with depth.

3. What is index-free adjacency?

A storage design in which a node record holds a direct pointer to its relationships, so that a traversal step follows pointers rather than performing an index lookup; traversal cost depends on the visited subgraph, not on the graph size.

4. How is a graph query written in a relational database?

With a recursive CTE: an anchor query selects the start node, the recursive term joins the previous level to the edge table, and a depth or visited-set predicate terminates it.

5. Why are graph databases harder to shard?

Any partitioning of the nodes cuts edges, and traversals then cross machines at every hop, which is expensive; a good partitioning of a general graph is itself a hard problem.

6. When would a recursive CTE be sufficient?

For hierarchies and traversals of moderate depth and size on a database that already exists, such as an organisation chart, a category tree or a bill of materials.

sreeram

KEY POINTS

Graph databases make relationships first-class and traversal the basic operation.

- Property graph: nodes with labels and properties, typed directed relationships with properties; Cypher expresses patterns.
- On a 1-million-edge graph, fixed two-step queries run in about a millisecond in PostgreSQL; the cost of deeper traversals follows the frontier (tenfold per level), in any engine.
- Choose a graph database when traversal depth is unbounded or the pattern is the question; use a recursive CTE for moderate needs.
- Always bound depth and set query timeouts.

2.7 Search and time-series stores

Two further families are defined by the retrieval they optimise instead of by their record shape. Search engines (Elasticsearch, OpenSearch, Apache Solr) answer queries over text: which documents contain these words, ranked by relevance. Time-series databases (InfluxDB, TimescaleDB, Prometheus, and Cassandra or PostgreSQL used with a time-oriented key) store measurements indexed by time and answer range, aggregation and retention queries. Both are derived stores in most architectures: the system of record remains elsewhere and the specialised store is filled from it.

2.7.1 Concept and mechanism

The inverted index

A B-tree maps a key to rows. For a text query that is the wrong direction: the question is which rows contain the word, not what the word of a row is. An inverted index maps each **term** to the list of documents containing it (the **posting list**), with the term frequency and, optionally, positions. A query for several terms retrieves the posting lists and combines them: intersection for AND, union for OR. Lucene, the library beneath Elasticsearch, OpenSearch and Solr, stores these lists in immutable segments, merged in the background in the same way as LSM SSTables.

Before indexing, text passes through an **analyzer**: a tokenizer splits the text into tokens, and filters lower-case them, remove stop words and reduce words to a stem, so that "indexes" and "indexed" match "index". The same analyzer must be applied to the query.

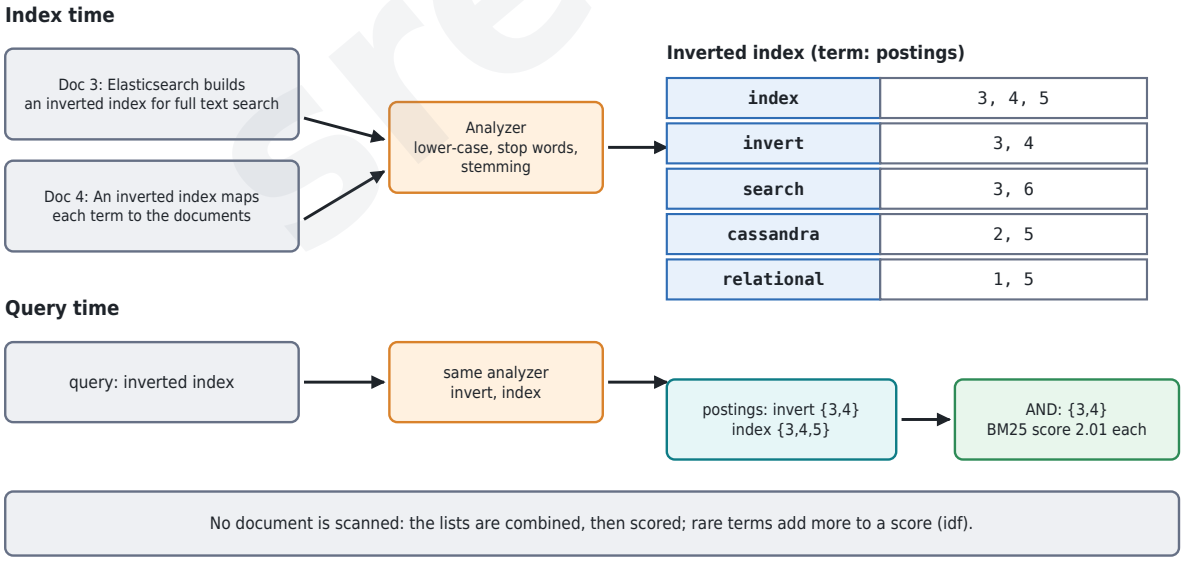


Figure 2.24. Documents pass through an analyzer into an inverted index; the query inverted index is analysed to the terms invert and index, the posting lists {3,4} and {3,4,5} are intersected or merged, and the result is ranked by score.

Worked example: an inverted index built for real

The program below indexes seven short documents with a crude analyzer (lower-case, stop words removed, a few suffix rules), stores term frequencies per document, and ranks a query with BM25, the default scoring function in Lucene since version 6. For a query term the score of a document is idf x tf x (k1 + 1) / (tf + k1 x (1 - b + b x length / average length)), where idf = ln(1 + (N - n + 0.5) / (n + 0.5)), N is the number of documents and n the number that contain the term; k1 = 1.2 and b = 0.75.

inv.py PYTHON

```
import math, re
from collections import defaultdict

DOCS = {
    1: "PostgreSQL is a relational database with strong transactional guarantees",
    2: "Cassandra is a wide column database designed for write heavy workloads",
    3: "Elasticsearch builds an inverted index for full text search",
    4: "An inverted index maps each term to the documents containing the term",
    5: "Relational databases use B-tree indexes while Cassandra uses LSM trees",
    6: "Full text search ranks documents by relevance using BM25",
    7: "Redis is an in-memory key value store",
}
STOP = {"is", "a", "an", "for", "the", "to", "each", "by", "while", "with", "using", "in"}

def analyze(text):
    toks = re.findall(r"[a-z0-9]+", text.lower())
    out = []
    for t in toks:
        if t in STOP: continue
        if t.endswith("xes"): t = t[:-2]                      # indexes -> index
        elif t.endswith("ed") and len(t) > 5: t = t[:-2]      # inverted -> invert
        elif t.endswith("ing") and len(t) > 6: t = t[:-3]     # containing -> contain
        elif t.endswith("s") and not t.endswith("ss"): t = t[:-1]
        out.append(t)
    return out

index = defaultdict(dict)       # term -> {doc_id: term frequency}
length = {}
for d, text in DOCS.items():
    toks = analyze(text); length[d] = len(toks)
    for t in toks: index[t][d] = index[t].get(d, 0) + 1
avg = sum(length.values()) / len(length); N = len(DOCS)

def bm25(query, k1=1.2, b=0.75):
    score = defaultdict(float)
    for t in analyze(query):
        post = index.get(t, {})
        idf = math.log(1 + (N - len(post) + 0.5) / (len(post) + 0.5))
        for d, tf in post.items():
            score[d] += idf * tf * (k1 + 1) / (tf + k1 * (1 - b + b * length[d] / avg))
    return sorted(score.items(), key=lambda x: -x[1])

for term in ("index", "invert", "cassandra", "search", "relational"):
    print("%-10s -> %s" % (term, dict(sorted(index[term].items()))))
print("terms:", len(index), "docs:", N, "avg length: %.2f" % avg)
for q in ("inverted index", "cassandra write", "full text search relevance"):
    print(repr(q), [(d, round(s, 2)) for d, s in bm25(q)])
# AND query: intersection of posting lists
a, b = set(index["index"]), set(index["invert"]); print("index AND invert ->", sorted(a & b))
```

Real output of inv.py TERMINAL

```
$ python3 inv.py
index      -> {3: 1, 4: 1, 5: 1}
invert     -> {3: 1, 4: 1}
cassandra  -> {2: 1, 5: 1}
search     -> {3: 1, 6: 1}
relational -> {1: 1, 5: 1}
terms: 36 docs: 7 avg length: 7.14
'inverted index' [(3, 2.01), (4, 2.01), (5, 0.71)]
'cassandra write' [(2, 2.7), (5, 1.0)]
'full text search relevance' [(6, 5.21), (3, 3.52)]
index AND invert -> [3, 4]
```

Reading the output. The term `index` occurs in documents 3, 4 and 5; `invert` (from "inverted") in 3 and 4. The AND query `index AND invert` is the intersection of two posting lists, `[3, 4]`. For the query "inverted index" documents 3 and 4 score 2.01 and document 5, which has `index` but not `invert`, scores 0.71. For "full text search relevance" document 6 scores highest (5.21) because it matches all four query terms (`full`, `text`, `search`, `relevance`), while document 3 matches three of them (3.52). The rare term gets a larger idf: `cassandra` (2 of 7 documents) contributes more to a score than a term found in most documents would.

Elasticsearch architecture

An Elasticsearch **index** is divided into **shards**, each a Lucene index. Every shard has a primary and a configurable number of replicas; shards are spread over nodes. A search is sent to a coordinating node, which fans it out to one copy of every shard, each shard returns its top results, and the coordinator merges them. Newly indexed documents become searchable after a **refresh** (by default every second), which makes the engine near-real-time and not real-time. Documents are JSON; a **mapping** declares each field as `text` (analysed, for full-text search) or `keyword` (not analysed, for exact match, sorting and aggregation).

products_index.json (typical, not run)

```
PUT /products
{
  "settings": { "number_of_shards": 3, "number_of_replicas": 1 },
  "mappings": { "properties": {
    "name":     { "type": "text" },
    "category": { "type": "keyword" },
    "price":    { "type": "double" } } }
}

GET /products/_search
{
  "query": { "bool": {
    "must":   [ { "match": { "name": "wireless headphones" } } ],
    "filter": [ { "term":  { "category": "audio" } },
                { "range": { "price": { "lte": 100 } } } ] } },
  "sort": [ "_score" ],
  "size": 10
}
```

The `must` clause is scored by relevance, whereas `filter` clauses only include or exclude documents and can be cached. This is the idiom for faceted product search.

Time-series data

Time-series workloads write measurements (a metric, tags such as sensor and region, a timestamp, a value) in time order, rarely update them, query recent ranges and aggregates, and delete old data in bulk. Specialised stores exploit this: append-only writes, time-based partitioning so that old data is dropped by removing a partition, compression of timestamps and values (consecutive values are similar), and down-sampling (replacing raw points by hourly averages after a period). The principal modelling hazard is **series cardinality**: the number of distinct tag combinations. A tag with unbounded values (user id, request id) makes the index of series grow without limit, which exhausts memory in InfluxDB- and Prometheus-style stores.

PostgreSQL handles moderate time-series loads well. Two features matter: BRIN indexes, which keep one summary entry per range of pages and suit columns correlated with physical order (time stamps inserted in order), and `date_bin` for bucketed aggregation. The experiment below loads 2,000,000 readings (one per second, 20 sensors in rotation, 23 days) and compares a B-tree and a BRIN index on the timestamp.

Index sizes and a one-hour range query (real output; PostgreSQL 16) TERMINAL

```
postgres=# select pg_size_pretty(pg_relation_size('reading')) heap,
postgres-#        pg_size_pretty(pg_relation_size('reading_btree')) btree,
postgres-#        pg_size_pretty(pg_relation_size('reading_brin')) brin;
  heap  | btree | brin
--------+-------+-------
 100 MB | 43 MB | 32 kB
postgres=# explain (analyze, buffers, costs off, timing off) select avg(val) from reading
postgres-#  where ts >= '2026-10-10 06:00+00' and ts < '2026-10-10 07:00+00';
-- with the BRIN index only:
 Bitmap Heap Scan on reading (actual rows=3600 loops=1)
   Heap Blocks: lossy=32
   ->  Bitmap Index Scan on reading_brin (actual rows=320 loops=1)
```

Index sizes and a one-hour range query (real output; PostgreSQL 16) (continued) TERMINAL

```
 Execution Time: 0.623 ms
-- with the B-tree index only:
 Index Scan using reading_btree on reading (actual rows=3600 loops=1)
 Execution Time: 0.664 ms
```

The BRIN index occupies 32 kB against 43 MB for the B-tree, and answers the one-hour query in the same time (0.62 ms against 0.66 ms), reading 32 heap pages marked lossy (the index narrows the search to a range of pages, which are then scanned and re-checked). The saving is possible only because the rows were inserted in time order; on a table whose physical order is unrelated to time, BRIN is useless. Down-sampling uses `date_bin`:

Down-sampling into six-hour buckets (real output) TERMINAL

```
postgres=# select date_bin('6 hours', ts, timestamptz '2026-10-01 00:00+00') as bucket, sensor,
postgres-#        round(avg(val)::numeric,2) avg_val, count(*)
postgres-# from reading where sensor=3 and ts < '2026-10-02 00:00+00' group by 1,2 order by 1;
         bucket         | sensor | avg_val | count
------------------------+--------+---------+-------
 2026-10-01 00:00:00+00 |      3 |   20.53 |  1080
 2026-10-01 06:00:00+00 |      3 |   20.58 |  1080
 2026-10-01 12:00:00+00 |      3 |   20.65 |  1080
 2026-10-01 18:00:00+00 |      3 |   20.69 |  1080
```

Retention is implemented by partitioning the table by time (declarative partitioning, covered with partitioning in Part 1) and dropping old partitions, which is a metadata operation, in contrast to `DELETE` of millions of rows; in Elasticsearch the equivalent is index lifecycle management with one index per period; in Cassandra it is a TTL on the table with time-window compaction.

2.7.2 Real-world problems and solutions

Incident 14: the search index no longer matches the database

Scenario. Product data is written to PostgreSQL by the application and indexed in Elasticsearch by a second call in the same request handler.

Symptoms. Some products are missing from search although they exist in the database; deleted products still appear; the counts differ between the two systems.

Diagnosis. (1) Compare identifiers: export ids from both stores and compute the set differences. (2) Check the application log for failed indexing calls and for requests where the database commit succeeded and the indexing call timed out. (3) Check the Elasticsearch bulk rejection and refresh settings.

Root cause. The application writes to two systems without a shared transaction. A crash or timeout between the writes, or a rejected indexing request, leaves them different, and nothing records the discrepancy.

Fix. Immediate: re-index from the database (full rebuild into a new index and switch an alias). Permanent: write the change event to an outbox table in the same database transaction, publish it to a queue, and let a consumer update the index idempotently using the product id and a version number (Elasticsearch accepts `version_type=external`); or use change data capture from the database log.

Prevention. Treat the search index as rebuildable derived data, keep a re-index job tested and scheduled, and alert on the lag between the outbox and the index.

Incident 15: series cardinality explosion

Scenario. A metrics pipeline adds the label `request_id` to a latency metric in a Prometheus-style store.

Symptoms. Memory use of the server rises steadily until it is killed; queries slow down; the number of active series grows by millions per hour.

Diagnosis. (1) Read the series count metric of the server (for Prometheus, `prometheus_tsdb_head_series`). (2) Find the metric and label with the most distinct values (the TSDB status page or a series count query by metric name). (3) Inspect the instrumentation for labels with unbounded values.

Root cause. Each distinct label combination is a separate series with its own index entry and in-memory buffer; an identifier with unbounded values creates a new series for each request.

Fix. Remove the label from the metric, drop the existing series (relabelling at scrape time), and record per-request detail in logs or traces instead.

Prevention. Allow only bounded label values (route template, status class, region), enforce a limit on series per target at the scraper, and review metric names in code review.

2.7.3 Questions to expect

1. What is an inverted index?

A map from each term to the list of documents containing it (with frequencies and positions). It lets a text query combine posting lists instead of scanning documents.

2. What does an analyzer do and why must queries use the same one?

It converts text into index terms (tokenising, lower-casing, removing stop words, stemming). If the query used a different analysis, its terms would not match the terms stored in the index.

3. How does BM25 rank documents?

By term frequency in the document (with saturation controlled by k1), inverse document frequency of the term (rare terms count more) and document length normalisation (controlled by b).

4. Why is Elasticsearch not a system of record?

Indexing is near-real-time and asynchronous with respect to other stores, it offers no multi-document transactions, and mapping changes require re-indexing. It is normally filled from a primary database and can be rebuilt.

5. What is time-series cardinality and why does it matter?

The number of distinct series (metric plus tag combinations). Each series has index and memory overhead, so unbounded tag values exhaust memory.

6. Why does a BRIN index suit time-ordered data?

It stores a summary (minimum and maximum) per range of heap pages. When values correlate with physical order, a range predicate excludes most ranges, at a tiny fraction of the size of a B-tree (32 kB against 43 MB in the experiment).

7. How is retention implemented efficiently?

By partitioning by time and dropping whole partitions (PostgreSQL), index-per-period with lifecycle policies (Elasticsearch), or TTL with time-window compaction (Cassandra), rather than deleting rows individually.

KEY POINTS

Search and time-series stores optimise a retrieval pattern, not a record shape.

- An inverted index maps terms to posting lists; analyzers normalise text; BM25 ranks by tf, idf and length.
- Elasticsearch shards Lucene indexes, is near-real-time (refresh about every second) and is a derived store fed through an outbox or CDC.
- Time-series data is append-mostly and queried by range: partition by time, down-sample, drop partitions for retention, and control series cardinality.
- A BRIN index of 32 kB served the same one-hour query as a 43 MB B-tree because the data was inserted in time order.

2.8 Choosing a database

The preceding chapters describe the mechanisms; this chapter turns them into a selection procedure. A database choice is hard to reverse because data volume, query code and operational practice accumulate around it. The procedure below starts from requirements stated as numbers, eliminates families that cannot meet them, and defaults to a relational database unless a measured requirement excludes it.

Steeran

2.8.1 Concept and mechanism

Requirements to state first

1. **Data shape and relationships.** Entities with many relationships and constraints, self-contained aggregates, key-addressed blobs, time-ordered measurements, text, or a graph.
2. **Access patterns.** The list of queries with their frequency and latency target: lookups by key, range reads, ad-hoc filters, joins, aggregations, full-text search, traversals.
3. **Consistency and transactions.** Which operations must be atomic and across which records; which reads must observe the latest write; what the system does during a partition.
4. **Scale.** Data size now and in three years, peak reads and writes per second, number of regions.
5. **Availability and recovery.** Recovery time objective, recovery point objective, tolerance for downtime during failover.
6. **Operational capacity.** Whether the team can run the system or must buy it as a managed service.

Workload	Relational	Document	Key-value	Wide-column	Specialised
Transactions across several records	Strong: ACID	Medium: single-document atomic, multi-document optional	Weak: item or limited transaction	Weak: partition-local	Not offered
Ad-hoc queries and reporting	Strong: SQL, planner	Medium: indexes and aggregation	Weak: key only	Weak: partition key only	Search: strong for text
Known key lookups at very high rate	Medium	Medium	Strong	Strong	Not intended
Sustained write ingest beyond one node	Weak without sharding	Medium: sharding	Strong	Strong: LSM, leaderless	Time series: strong
Flexible or evolving record structure	Medium: JSONB	Strong	Medium	Weak: schema in CQL	Search: strong
Relationship traversal of unknown depth	Weak: recursive CTE	Weak	Not intended	Not intended	Graph: strong
Multi-region active-active writes	Weak	Medium	Strong: global tables	Strong	Varies
Operational simplicity	Strong: mature tooling	Medium	Managed: strong	Weak: repair, compaction	Medium

Fit of the families to workload requirements (rows: workloads; entries: fit and the reason).

sreeram

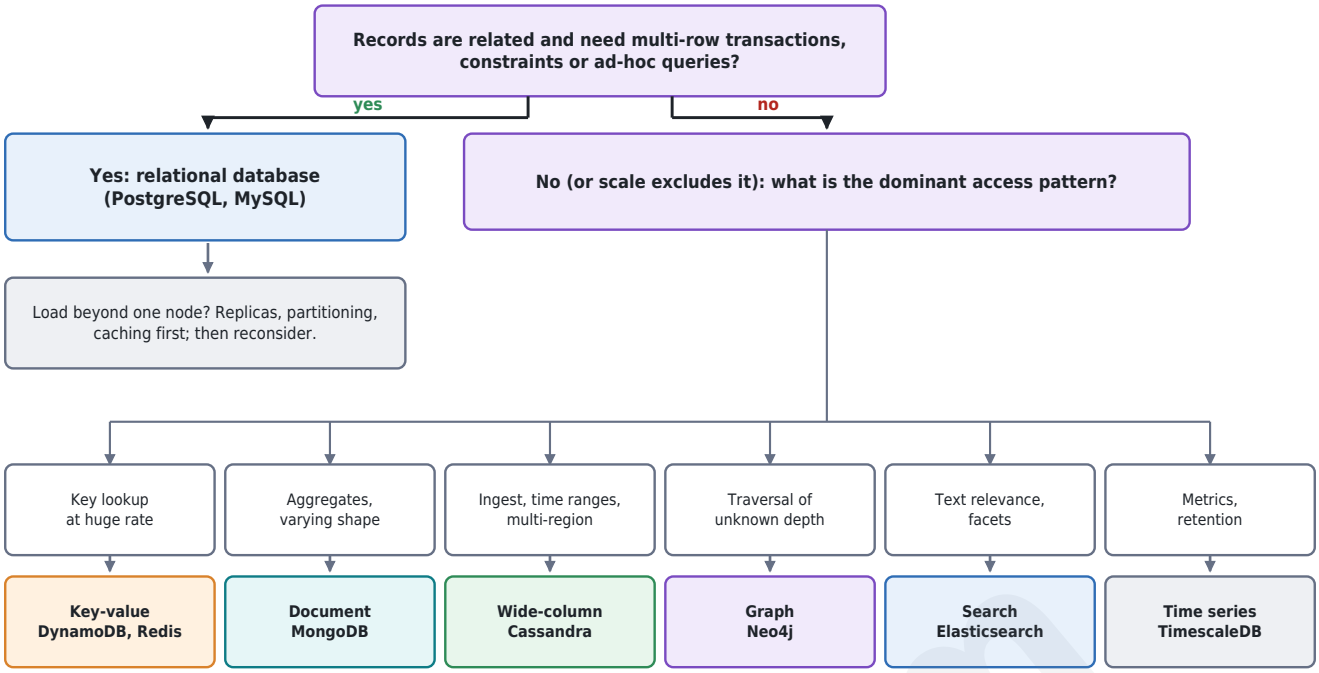


Figure 2.25. A selection procedure: default to a relational database and move to another family only when a stated requirement (scale, shape, retrieval pattern or availability) excludes it.

Rules of thumb with their reasons

- **Start relational.** PostgreSQL 16 on one node handled about 1,100 durable read-write transactions per second and about 18,000 key lookups per second on two shared cores (first chapter), and JSONB covers semi-structured fields. The relational model supports queries that were not anticipated, which most products need.
- **Choose a document database** when the unit of access is an aggregate read and written as a whole (a product page, a customer profile with embedded preferences), the structure varies between records, and multi-document transactions are rare. The next chapters treat MongoDB in detail.
- **Choose a key-value store** for cache, session and simple lookups where the key is known, and a managed key-value service (DynamoDB) when horizontal scale and operations-free capacity matter more than flexible queries.
- **Choose a wide-column store** for very high write volume, known queries, multi-data-centre replication and time-ordered data, when the team accepts query-first modelling and operates repair and compaction.
- **Choose a graph database** when deep traversals or path patterns are the product (fraud rings, recommendations, network analysis).
- **Add a search engine** when text relevance, faceting or typo tolerance is needed; keep it derived.
- **Add a time-series store** when metrics volume and retention policies dominate; PostgreSQL with partitioning and BRIN indexes covers moderate volumes.

Cost of the choice

Each family moves work to a different place. A relational database performs joins, constraint checks and planning at query time. A document database moves integrity rules into the application and the schema into code. A wide-column or key-value store moves joins and secondary access paths into application-maintained copies of the data. The effort does not vanish; the decision is where it is cheapest to carry. The cost of operations (upgrades, repairs, capacity planning, backup and restore tested end to end) is usually larger than the cost of writing the first version.

2.8.2 Worked example

Four systems, each reduced to the decision table.

System	Workload facts	Recommended store	Reason
Online shop (orders, payments, inventory)	300 writes per second, strict consistency, ad-hoc reporting	PostgreSQL, with Redis cache and Elasticsearch for product search	Transactions and constraints dominate; scale fits one node plus replicas; search and cache are derived

System	Workload facts	Recommended store	Reason
Fleet telemetry (500,000 devices, one reading per 10 s)	50,000 writes per second, queries by device and time range, 90-day retention	Cassandra (partition key device and day, TTL) or TimescaleDB	Sustained ingest, time-ordered, bounded queries, bulk expiry
Social timeline	Fan-out of posts to followers, read by user, tolerate seconds of staleness	Wide-column or key-value store for timelines; graph or relational store for the follow graph	Per-user partitioned reads, eventual consistency acceptable
Product catalogue with variable attributes	Read-heavy, records of 50 different shapes, search by attributes	MongoDB or PostgreSQL JSONB; Elasticsearch for faceted search	Aggregate access with varying structure

Case studies and recommended stores.

The telemetry example can be checked for partition size by arithmetic before any software is chosen: one device writes one reading per 10 seconds, which is 8,640 readings per day; with 50 bytes per cell (a typical order of magnitude, to be measured) a `(device_id, day)` partition holds about 430 KB, well below the 100 MB guidance for Cassandra partitions. The write rate of 500,000 / 10 = 50,000 per second spreads over 500,000 partition keys per day, which is the uniform distribution the earlier experiment labelled ideal.

2.8.3 Real-world problems and solutions

Incident 16: relational data in a document store, joined in application code

Scenario. A team stores customers and orders in two MongoDB collections with references (`customerId` in each order document) because the document model was chosen "for flexibility". The order list page shows each order with the customer name, the product names and the shipping address.

Symptoms. The page makes 1 + N + M database calls (orders, then one call per customer, then one per product); latency grows linearly with page size; consistency bugs appear (an order references a deleted customer); reports that group orders by customer region are implemented as application loops that load whole collections.

Diagnosis. (1) Enable the database profiler or driver command logging and count commands per request (`db.setProfilingLevel(1, { slowms: 0 })` for a test run, then group by collection). (2) Count round trips per page in the application trace. (3) Review the query list against the document design: queries that always combine two collections show that the data is relational. (4) Check for referential-integrity defects with a query for orders whose `customerId` is absent from the customers collection.

Root cause. The data has many-to-one and many-to-many relationships queried together, and the integrity rules of a relational database are absent. Application code implements joins as loops of point reads. The cost of the loop is the number of calls multiplied by the network round trip: on the build machine a local socket round trip of one PostgreSQL statement cost on the order of 30 microseconds (1,000 separate lookups took roughly 30 ms longer than one join returning the same data), and across a network with a 0.5 ms round trip the same loop costs about 500 ms before any processing.

Fix. Immediate: replace the per-row calls by one batched query (`find({_id: {$in: ids}})` for all customers of the page) and cache reference data. Permanent: decide per relationship whether to embed (copy the customer name and the shipping address into the order, as the order records the facts at purchase time), use `$lookup` for occasional joins, or move the data to a relational database. In the Spring Data MongoDB sketch below, the first version is the loop and the second the batch (typical code, not run on the build machine).

OrderViewService.java (fragment, not run) JAVA

```
// Before: one query per order for the customer (N + 1 round trips)
List<OrderView> slow(List<Order> orders) {
    List<OrderView> out = new ArrayList<>();
    for (Order o : orders) {
        Customer c = customerRepository.findById(o.customerId()).orElseThrow();
        out.add(new OrderView(o.id(), c.name(), o.total()));
    }
    return out;
}

// After: one batched query, joined in memory by id
List<OrderView> batched(List<Order> orders) {
    Set<String> ids = orders.stream().map(Order::customerId).collect(Collectors.toSet());
    Map<String, Customer> byId = customerRepository.findAllById(ids).stream()
            .collect(Collectors.toMap(Customer::id, Function.identity()));
```

OrderViewService.java (fragment, not run) (continued) JAVA

```
    return orders.stream()
            .map(o -> new OrderView(o.id(), byId.get(o.customerId()).name(), o.total()))
            .toList();
}
```

Prevention. Before choosing a document database list the ten most frequent queries; if more than a few join two or more entity types, use a relational database. Model facts that must not change after creation (the price and address at the time of an order) as embedded copies. Run a constraint-violation report regularly if references are not enforced.

Incident 17: store chosen for a benchmark, not for the workload

Scenario. A new analytics feature selects a database because a published benchmark showed the highest write throughput.

Symptoms. The production workload (many small point updates and joins across three entities) performs worse than the previous database; the throughput advantage never appears because the benchmark used an append-only, single-key workload.

Diagnosis. (1) Compare the benchmark's workload definition (operation mix, key distribution, record size, consistency settings, durability setting) with production traces. (2) Replay a sample of production queries against both systems. (3) Check whether the benchmark disabled durability (for example acknowledged writes before fsync).

Root cause. Benchmark results depend on the workload, configuration and durability settings, and do not transfer to a different mix.

Fix. Select with a test that replays production-shaped traffic, including key skew, with the durability and consistency level that the product requires.

Prevention. Write a decision record listing the requirements, the candidates, the measurements and the rejected options with reasons; re-evaluate when the numbers change.

2.8.4 Questions to expect

1. How is a database selected for a new service?

By listing data shape, access patterns, consistency needs, scale and operational constraints; eliminating families that cannot satisfy a stated requirement; defaulting to a relational database unless a measured requirement excludes it; and validating the shortlist with a production-shaped test.

2. When is MongoDB or another document store appropriate?

When data is accessed as aggregates, structure varies between records, and transactions rarely span documents. It is poorly suited to data with many cross-entity queries and strict integrity rules.

3. What is polyglot persistence and when does it become a liability?

Using several stores, each for its strength. It becomes a liability when stores hold copies without a reliable propagation mechanism, or when the team cannot operate them all.

4. How can a relational database be used for semi-structured data?

With JSONB columns and GIN indexes in PostgreSQL, or JSON columns with generated columns in MySQL, keeping core attributes as typed columns and variable attributes in JSON. The comparison chapter later in this Part measures it.

5. Give a case where wide-column is the right choice and one where it is wrong.

Right: a telemetry ingest of tens of thousands of writes per second, queried by device and time range. Wrong: an order system with ad-hoc reporting and multi-row transactions, where the query-first model and the absence of joins add cost without benefit.

6. Why is "we may need to scale later" a weak reason to choose a distributed store?

A single relational node handles a large load, scaling paths exist (replicas, partitioning, later migration), and the distributed store imposes immediate costs in modelling, consistency and operations.

sreeram

7. Which questions reveal that a document model was a mistake?

Do most queries combine several collections? Are there references that must never dangle? Are reports implemented as loops over collections? If yes, the data is relational.

KEY POINTS

Choose from requirements, not from fashion.

- State data shape, access patterns, consistency, scale, availability and operational capacity as numbers.
- Default to a relational database; move to a family only when a stated requirement excludes it.
- Each family relocates work: joins, integrity and secondary access paths move to application code in key-value and wide-column stores.
- Search and time-series stores are normally derived from a system of record.
- Joining in application code over a document store costs one round trip per row; batch the lookups, embed facts that do not change, or use a relational database.
- Validate the shortlist with production-shaped traffic and the real durability setting.

2.9 MongoDB Architecture and the Document Model

MongoDB is a document database: the unit of storage is a self-describing record, called a document, that may contain nested objects and arrays. A relational system splits one business entity such as an order across several tables and reassembles it with joins. A document database stores the entity as one unit, so that the data an application reads together is also stored together. This chapter defines the vocabulary, the storage format (BSON), the structure of the `_id` field, and the limits that constrain every schema decision in later chapters.

Version scope: statements refer to MongoDB 7.x and 8.x as documented. The build machine used for this book has PostgreSQL 16 but no `mongod`, so every mongosh output in this Part is labelled **typical output (not run on the build machine)**. Where a mechanism can be demonstrated for real, the demonstration uses a reference implementation written for this book in Python (BSON encoder, ObjectId generator, query matcher, update operators, aggregation stages) or PostgreSQL JSONB; such results are labelled accordingly, and the aggregation results were cross-checked against relational queries executed in PostgreSQL 16.

2.9.1 Concept and mechanism

Deployment, database, collection, document

A running `mongod` process hosts databases. A database holds collections. A collection holds documents. A document holds fields, and a field value is a scalar, an embedded document, or an array. A collection corresponds loosely to a table, but the correspondence ends at the level of role: a collection does not impose a column list on its documents unless a schema validator is attached (see the chapter on schema design). Databases and collections are created implicitly on the first write to them.

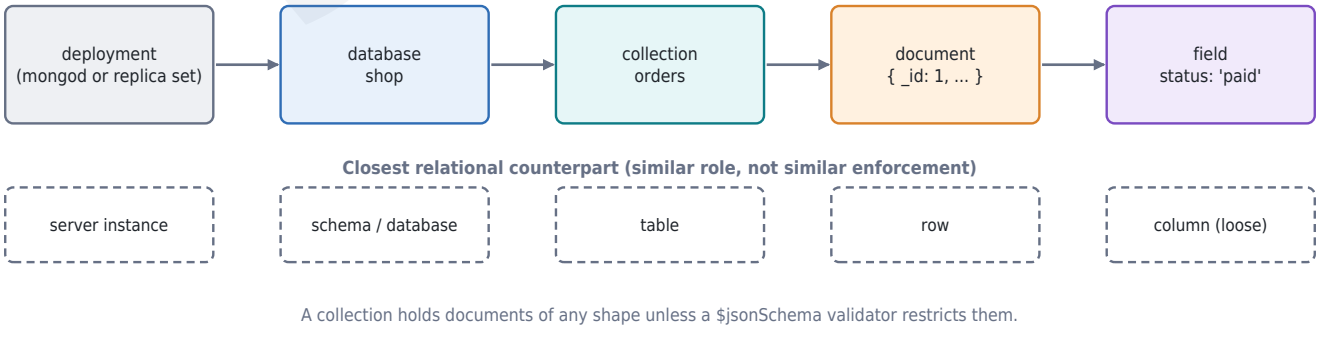


Figure 2.26. Containment hierarchy of MongoDB with the nearest relational counterparts. The last level differs most: fields are defined per document, not per collection.

Relational term	MongoDB term	Difference that matters
database / schema	database	Same role: a namespace and unit of access control.
table	collection	No fixed columns; created implicitly on first insert.

Relational term	MongoDB term	Difference that matters
row	document	Variable shape, nested values, up to 16 MiB.
column	field	Exists only in the documents that contain it.
primary key	`_id`	Mandatory, immutable, unique within the collection.
foreign key	reference (manual)	No enforcement; the application or the schema design keeps it valid.
join	`$lookup` or embedding	Embedding avoids the join; `$lookup` is a left outer join stage.
index	index	B-tree based; the `_id` index always exists.
transaction	single-document write is atomic; multi-document transaction available	Covered in the chapter on transactions in the operations Part.

Vocabulary mapping between relational systems and MongoDB.

Server architecture

A client (application driver, mongosh, a tool) opens a TCP connection to `mongod`, which listens on port 27017 by default. The server decodes each request, authenticates it, plans queries, and delegates storage to a pluggable storage engine. Since MongoDB 3.2 the default engine is WiredTiger, which keeps recently used B-tree pages in an in-memory cache, protects changes with a write-ahead journal, and writes checkpoints to the data files `collection-*.wt` and `index-*.wt` below the `dbPath`. Storage internals (cache sizing, journaling, compression) are treated in the operations chapters; this Part concentrates on the data model, queries, aggregation, and indexes.

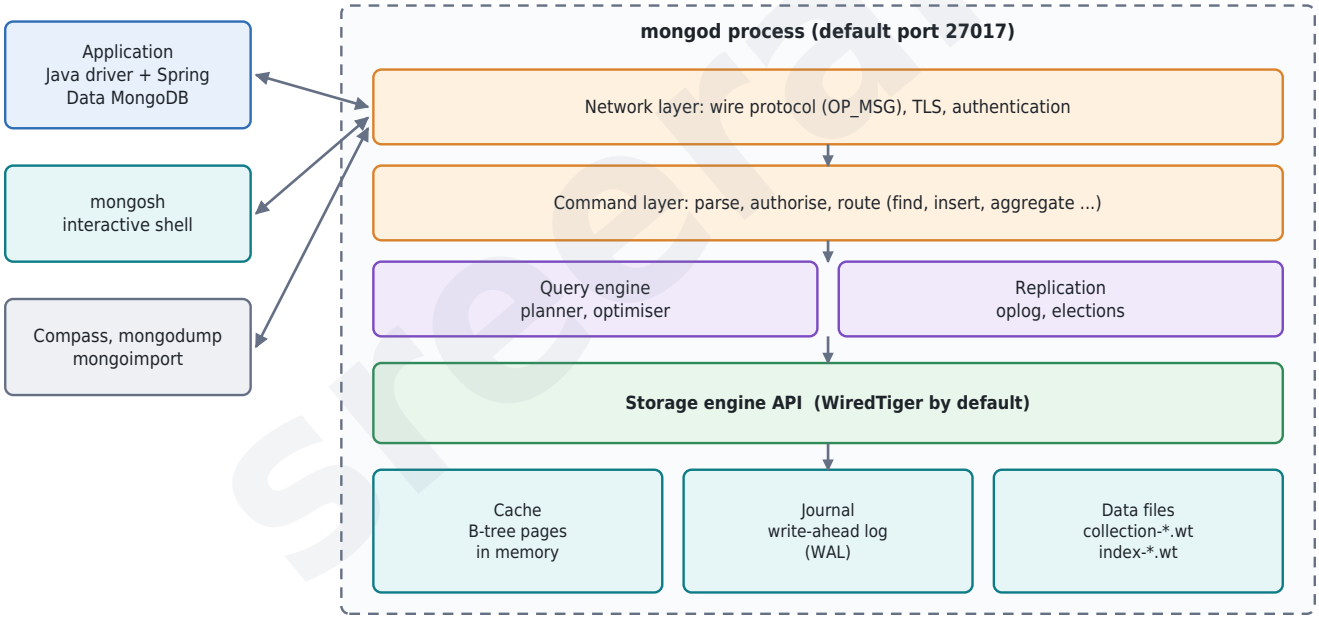


Figure 2.27. Request path inside a single mongod: network layer, command layer, query engine, then the storage engine API with the WiredTiger cache, journal, and data files.

Three databases exist on every deployment and are not application data: `admin` (users, roles, administrative commands), `local` (instance-local data such as the replication oplog), and `config` (cluster metadata and session records). Application databases are separate, for example `shop`.

The two ways to talk to the server

The interactive shell `mongosh` is a JavaScript (Node.js based) REPL. It replaces the legacy `mongo` shell and prints documents in a relaxed extended-JSON style with helpers such as `ObjectId('...')` and `ISODate('...')`. Applications use a driver. The official Java driver (`mongodb-driver-sync`, or the reactive-streams driver) exposes `MongoClient`, `MongoDatabase`, `MongoCollection<Document>`, and the `Filters`, `Updates`, `Projections`, `Sorts`, `Aggregates`, and `Indexes` builders; Spring Data MongoDB builds repositories and `MongoTemplate` on top of it (covered in the Spring chapter of this Part). All of them speak the same wire protocol and send BSON.

BSON and JSON

Documents are written by humans as JSON-like text but stored and transmitted as BSON (Binary JSON). BSON adds types that JSON lacks, stores lengths in front of variable-size values so that a reader can skip them, and preserves field order. The principal types are listed below; a query can test a type with `$type` using either the number or the alias.

Alias	Code	Java / mongosh	Notes
`double`	1	`double`, number	IEEE 754 binary64; inexact for decimal fractions.
`string`	2	`String`	UTF-8, length-prefixed, NUL-terminated.
`object`	3	`Document`	Embedded document; nesting limited to 100 levels.
`array`	4	`List`	Stored as a document with keys "0", "1", ...
`binData`	5	`byte[]`, `UUID`	Subtype 4 is the standard UUID.
`objectId`	7	`ObjectId`	12 bytes; default `_id` type.
`bool`	8	`boolean`	One byte.
`date`	9	`Date`, `Instant`	Signed 64-bit milliseconds since the Unix epoch, always UTC.
`null`	10	`null`	Distinct from a missing field.
`int`	16	`int`	32-bit; mongosh stores integral numbers that fit as int.
`long`	18	`long`	64-bit integer.
`decimal`	19	`Decimal128`, `BigDecimal`	34 decimal digits; the correct type for money.

Principal BSON types.

The distinction between `double` and `decimal` is the same as the one between `float` and `numeric` in SQL. A price of `0.1 + 0.2` stored as a double is `0.30000000000000004`; monetary amounts belong in `Decimal128`, or in an integer count of the smallest currency unit (the approach used in this Part: fields named `priceCents`, `totalCents`).

The `_id` field and ObjectId

Every document has an `_id` field that is unique within its collection and immutable; the server creates a unique index on it automatically. If an insert supplies no `_id`, the driver generates an **ObjectId**: 12 bytes composed of a 4-byte timestamp (seconds since the epoch, big-endian), a 5-byte random value generated once per client process, and a 3-byte counter that starts at a random value and increments for each id created. Because the timestamp occupies the most significant bytes, ObjectIds generated by different clients sort approximately by creation time and the creation time can be read back from the id.

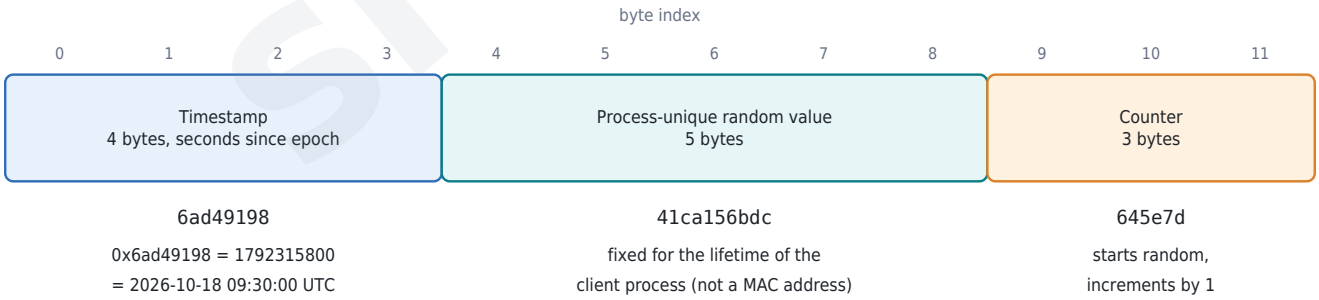


Figure 2.28. Layout of the ObjectId 6ad49198 41ca156bdc 645e7d produced by the reference implementation: the leading four bytes decode to 2026-10-18 09:30:00 UTC.

The id is generated on the client, not on the server, which removes a round trip and a central sequence from the insert path and lets a client know a document identifier before the write is acknowledged. The cost is that the 12-byte value is larger than a 4-byte integer key, and that inserts with random 5-byte process values and a monotonic prefix are mostly appended at the right edge of the `_id` index. Alternatives are legitimate: any BSON value except an array or a regular expression may serve as `_id`, for example a natural key (`'P0041'`), a UUID (binary subtype 4), or a compound document. Choose a natural `_id` only if it is immutable in the business domain, because `_id` cannot be updated.

Document structure and size

Field names within one document must be unique and may not contain the NUL character. Names beginning with `$` or containing `.` are permitted since MongoDB 5.0 but are awkward in queries and should be avoided. The maximum BSON document size is **16 MiB** (16,777,216 bytes) and nesting is limited to 100 levels, where each embedded document or array adds a level. These limits are hard; larger binary content belongs in GridFS or object storage. The server returns the field names in the order in which they were written, so `{a: 1, b: 2}` and `{b: 2, a: 1}` are different documents for equality purposes when compared as whole embedded documents.

2.9.2 Worked example: BSON bytes of one document

The document `{ _id: ObjectId('68f3a1d8b4c2e19a7d05f3c1'), sku: 'P0041', qty: 3, price: 4.99, paid: true, at: ISODate('2026-10-18T09:30:00Z') }` was serialised with a BSON encoder written from the BSON specification (little-endian integers, type byte, NUL-terminated field name, value). The result is 80 bytes.

Hex dump produced by the reference encoder

```
0000  50 00 00 00 07 5f 69 64 00 68 f3 a1 d8 b4 c2 e1  P...._id.h......
0010  9a 7d 05 f3 c1 02 73 6b 75 00 06 00 00 00 50 30  .}....sku.....P0
0020  30 34 31 00 10 71 74 79 00 03 00 00 00 01 70 72  041..qty......pr
0030  69 63 65 00 f6 28 5c 8f c2 f5 13 40 08 70 61 69  ice..(\....@.pai
0040  64 00 01 09 61 74 00 c0 b9 58 4e a1 01 00 00 00  d...at...XN.....
```

One 80-byte BSON document, element by element (little-endian integers)



Figure 2.29. The same 80 bytes decoded element by element. Every element repeats its field name, which is why short field names reduce document size in collections with millions of small documents.

Observations that follow from the byte layout:

- The first four bytes (`50 00 00 00`) hold the total length, 0x50 = 80. A reader can skip a whole document, or an embedded document, without parsing it.
- The integer 3 occupies 4 bytes as an `int32`; in the same encoder an integer above 2^31 - 1 occupies 8 bytes as an `int64`, and a double always occupies 8 bytes. The type is part of the value: the double `1.0` and the int `1` are stored differently, although they compare equal in queries.
- The equivalent compact JSON text is 113 bytes: the ObjectId becomes 24 characters instead of 12 bytes and the date 24 characters instead of 8 bytes, but the numbers and short strings are about the same size. BSON is therefore not systematically smaller than JSON; its benefit is traversal speed and richer types.
- Field names are stored in every document. The name `priceCents` costs 11 bytes in each embedded line item, and a collection of 20 million orders with 4 items each carries about 880 MB of repeated field names for that one field alone before compression. WiredTiger compresses data blocks (snappy by default for collections), which reduces the disk cost but not the cache or network cost.

sreeram

2.9.3 Hands-on

Starting a server and connecting (typical)

Start a local mongod and the shell (commands; output omitted) — BASH

```bash
mkdir -p /data/db
mongod --dbpath /data/db --port 27017 --bind_ip 127.0.0.1 &
mongosh "mongodb://127.0.0.1:27017/shop"
```

mongosh session (typical output, not run on the build machine)

```
shop> db.orders.insertOne({ customerId: 17, status: 'pending', totalCents: 1998 })
{
  acknowledged: true,
  insertedId: ObjectId('68f3a1d8b4c2e19a7d05f3c1')
}
shop> show dbs
admin   40.00 KiB
config  72.00 KiB
local   72.00 KiB
shop     8.00 KiB
shop> db.getCollectionNames()
[ 'orders' ]
shop> db.orders.findOne()
{
  _id: ObjectId('68f3a1d8b4c2e19a7d05f3c1'),
  customerId: 17,
  status: 'pending',
  totalCents: 1998
}
shop> db.orders.findOne()._id.getTimestamp()
ISODate('2026-10-18T09:30:00.000Z')
shop> Object.bsonsize(db.orders.findOne())
80
shop> db.orders.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

The database `shop` and the collection `orders` did not exist before the `insertOne` call. The only index is the automatic `_id_` index. `Object.bsonsize()` returns the BSON size in bytes of a document, the quantity limited to 16 MiB.

The same idea in Java (not run: the driver jar is not available on the build machine)

InsertOrder.java — JAVA

```java
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.result.InsertOneResult;
import org.bson.Document;
import org.bson.types.ObjectId;

public class InsertOrder {
    public static void main(String[] args) {
        // One MongoClient per application: it owns the connection pool and is thread-safe.
        try (MongoClient client = MongoClients.create("mongodb://127.0.0.1:27017")) {
            MongoCollection<Document> orders =
                client.getDatabase("shop").getCollection("orders");

            Document order = new Document("customerId", 17)
                .append("status", "pending")
                .append("totalCents", 1998);

            InsertOneResult result = orders.insertOne(order);
            ObjectId id = result.getInsertedId().asObjectId().getValue();
            System.out.println("inserted " + id.toHexString() + " created at " + id.getDate());
        }
    }
}
```

The driver creates the `ObjectId` on the client when the document has no `_id`; `insertOne` also writes it back into the `Document` instance. The `MongoClient` is a long-lived, thread-safe object with its own connection pool and server monitoring threads; creating one per request is an error that exhausts connections.

Comparison with PostgreSQL JSONB (run for real on PostgreSQL 16)

PostgreSQL offers the nearest relational analogue, the `jsonb` type. Its semantics differ from BSON in ways that influence a migration decision, and the differences can be observed directly.

PostgreSQL 16: jsonb does not keep key order, drops duplicate keys, and has one number type TERMINAL

```
$ psql -c "select '{\"qty\":3,\"b\":1,\"a\":2,\"aa\":3,\"a\":9}'::jsonb as j"
                    j
-------------------------------------
 {"a": 9, "b": 1, "aa": 3, "qty": 3}
(1 row)

$ psql -c "select '{\"x\":1.0}'::jsonb = '{\"x\":1}'::jsonb as equal"
 equal
-------
 t
(1 row)

$ psql -c "select jsonb_typeof('{\"at\":\"2026-10-18T09:30:00Z\"}'::jsonb->'at') as date_type"
 date_type
-----------
 string
(1 row)
```

`jsonb` orders keys by length and then bytewise, keeps the last duplicate key, has no date or binary type (a timestamp is a string), and has a single numeric type. BSON keeps the written order, distinguishes `int`, `long`, `double`, and `decimal`, and has native date and binary types. A later chapter in this book compares the two models in depth.

2.9.4 Real-world problems and solutions

Incident: a lookup by id returns nothing

Scenario. A support tool receives an order identifier as a string in a URL (`/orders/68f3a1d8b4c2e19a7d05f3c1`) and queries `db.orders.findOne({ _id: req.params.id })`. The document exists, yet the result is `null`. In Java the same defect appears as `collection.find(eq("_id", idString))`.

Symptoms. `findOne` returns `null` for identifiers copied from the database; `countDocuments({ _id: ... })` returns 0; a query with `ObjectId('...')` in mongosh returns the document.

Diagnosis. Compare the BSON type of the stored value and of the query value.

Diagnosis in mongosh JAVASCRIPT

```
db.orders.aggregate([
  { $match: { customerId: 17 } },
  { $limit: 1 },
  { $project: { idType: { $type: '$_id' } } }
])
// typical output: [ { _id: ObjectId('68f3a1d8b4c2e19a7d05f3c1'), idType: 'objectId' } ]
db.orders.findOne({ _id: '68f3a1d8b4c2e19a7d05f3c1' })                // null  (string)
db.orders.findOne({ _id: ObjectId('68f3a1d8b4c2e19a7d05f3c1') })      // the document
```

The reference matcher gives the same result for the two predicates: `match(doc, {'_id': ObjectId(...)})` is true and `match(doc, {'_id': '68f3...'})` is false. MongoDB comparisons are type-bracketed: values of different BSON types are never equal, even if their textual form is identical.

Root cause. The query supplies a `string`; the stored `_id` is an `objectId`. No implicit conversion exists.

Fix. Convert at the boundary: `new ObjectId(idString)` in Java (after validating with `ObjectId.isValid(idString)` to return HTTP 400 instead of an exception for malformed input), or declare the controller parameter as `ObjectId`/`String` mapped by Spring Data, which converts a 24-character hexadecimal `String` annotated as `@Id` automatically.

Prevention. Decide the `_id` type per collection once and expose it through one conversion function. Reject malformed identifiers at the API edge. Integration tests should load a document through the repository and then read it back through the HTTP route.

Incident: monetary totals drift by fractions of a unit

Scenario. An invoice collection stores `total: 0.1 + 0.2` computed as a Java `double` and summed per day by an aggregation.

Symptoms. Daily totals differ from the ledger by amounts such as `0.00000000000005`; an equality filter `{ total: 0.3 }` does not match a document that was computed from `0.1 + 0.2`.

Diagnosis. `db.invoices.find({ total: { $type: 'double' } }).limit(3)` shows `double` values; evaluating `0.1 + 0.2` in any IEEE 754 environment gives `0.30000000000000004` (this value was confirmed in the Python reference implementation).

Root cause. Binary floating point cannot represent most decimal fractions exactly.

Fix. Store `Decimal128` (Java `BigDecimal`, mapped by the driver), or integer minor units. Migrate with an aggregation-based update: `db.invoices.updateMany({}, [{ $set: { totalCents: { $toLong: { $round: [{ $multiply: ['$total', 100] }, 0] } } } }])`, verify with a checksum query, and then remove the old field.

Prevention. A schema validator that requires `totalCents` to be `long` (see the schema chapter) turns the mistake into a rejected write.

2.9.5 Questions to expect

1. **What is BSON and why does MongoDB not store plain JSON?** BSON is a binary, typed, length-prefixed serialisation of documents. It adds types missing from JSON (int32, int64, double, decimal128, date, binary, ObjectId), allows a reader to skip values by length, and preserves field order. It is not necessarily smaller than JSON.
2. **What are the components of an ObjectId?** Twelve bytes: a 4-byte creation timestamp in seconds, a 5-byte random value per client process, and a 3-byte counter initialised randomly. It is generated by the driver, so it can be created without a server round trip, and it sorts roughly by creation time.
3. **What is the maximum document size and why does it exist?** 16 MiB. It bounds memory use per document, network message size, and the cost of rewriting a document. Larger content uses GridFS or an external object store; the limit is a design signal against unbounded embedded arrays.
4. **Can `_id` be changed? Can it be something other than an ObjectId?** `_id` is immutable; changing it means deleting and re-inserting. It can be any BSON type except array or regex, including a natural key or a compound subdocument.
5. **How does a collection differ from a table?** A collection has no mandatory column definition, enforces no foreign keys, and holds variable-shape documents. Optional schema validation and unique indexes provide the constraints that the application requires.
6. **Why is `Decimal128` preferred over `double` for money?** `double` is binary floating point and cannot represent most decimal fractions exactly; `Decimal128` stores 34 decimal digits exactly. Integer minor units are an acceptable alternative.
7. **What is the difference between a missing field and `null`?** They are different states; the query semantics (`{ x: null }` matches both) are covered in the chapter on query operators.
8. **Which storage engine does MongoDB use and what does it contribute?** WiredTiger (default since 3.2): B-tree storage, an in-memory cache, a write-ahead journal for durability, checkpoints, document-level concurrency control, and block compression.

KEY POINTS

MongoDB stores BSON documents in collections inside databases; databases and collections are created on first write.

- BSON is typed, length-prefixed, ordered, and not always smaller than JSON; field names are repeated in every document.
- `_id` is mandatory, unique, immutable, and by default a 12-byte client-generated ObjectId (timestamp, process random, counter).
- A document is limited to 16 MiB and 100 nesting levels; these limits shape schema design.
- Query comparisons are type-bracketed: a string never equals an ObjectId, and an int equals a double of the same value.
- Use `Decimal128` or integer minor units for money, never `double`.

2.10 CRUD in Depth

CRUD operations in MongoDB are methods on a collection object that take documents as arguments: a filter document selects, an update document modifies, an options document tunes. All examples in this chapter use a small `orders` collection of five documents so that every result can be verified by inspection; the same collection shape (order with embedded `items`) is used in the later chapters at a scale of 20,000 documents. The mongosh results below are **typical output (not run on the build machine)**; the document-level effects (matching, sorting, update operators, array updates) were additionally computed with the Python reference implementation described in the first chapter and agree with the results shown.

Sample data used throughout this chapter JAVASCRIPT

```
db.orders.insertMany([
  { _id: 1, customerId: 17, status: 'paid',      placedAt: ISODate('2026-10-01T09:05:00Z'),
    items: [ { sku: 'P0041', qty: 1, priceCents: 499 }, { sku: 'P0421', qty: 2, priceCents: 199 } ],
    totalCents: 897, tags: ['gift'] },
  { _id: 2, customerId: 17, status: 'pending',   placedAt: ISODate('2026-10-03T14:20:00Z'),
    items: [ { sku: 'P0323', qty: 1, priceCents: 9999 } ], totalCents: 9999, tags: [] },
  { _id: 3, customerId: 42, status: 'shipped',   placedAt: ISODate('2026-10-05T11:00:00Z'),
    items: [ { sku: 'P0041', qty: 3, priceCents: 499 },
             { sku: 'P0007', qty: 1, priceCents: 1299 } ],
    totalCents: 2796, tags: ['gift', 'priority'] },
  { _id: 4, customerId: 42, status: 'cancelled', placedAt: ISODate('2026-10-07T08:45:00Z'),
    items: [ { sku: 'P0007', qty: 2, priceCents: 1299 } ], totalCents: 2598 },
  { _id: 5, customerId: 58, status: 'paid',      placedAt: ISODate('2026-10-09T16:30:00Z'),
    items: [ { sku: 'P0421', qty: 5, priceCents: 199 } ], totalCents: 995, tags: ['priority'] }
])
```

2.10.1 Concept and mechanism

Create: insertOne, insertMany

`insertOne(doc)` and `insertMany([docs])` return an acknowledgement containing the generated `_id` values. A document without `_id` receives an ObjectId from the driver. Inserting a second document with an existing `_id` fails with error code 11000 (`E11000 duplicate key error`). `insertMany` is **ordered** by default: the server stops at the first failing document and the documents after it are not inserted, while the documents before it remain. With `{ ordered: false }` the server attempts every document, reports all failures together, and may reorder the work for throughput. Ordered behaviour is correct when the documents depend on one another; unordered behaviour is correct for independent bulk loads.

Read: find, filter, projection, sort, skip, limit

`find(filter, projection)` returns a **cursor**, not an array. The server returns the first batch (101 documents or 16 MiB, whichever is smaller) and the rest on demand through `getMore`; mongosh prints 20 documents and waits for `it`. `findOne` returns the first document or `null`. An empty filter `{}` matches every document. Multiple top-level fields in the filter are combined with logical AND.

Projection limits the fields returned, which reduces network traffic and, when the projection is covered by an index, avoids reading documents (see the indexes chapter). An inclusion projection lists the fields to keep (`{ status: 1, totalCents: 1 }`); an exclusion projection lists the fields to drop (`{ items: 0 }`). The two styles cannot be mixed in one projection, with the single exception of `_id`, which is returned by default and can be suppressed with `_id: 0` in either style. Array fields accept the projection operators `$slice` (return a sub-range of the array) and `$elemMatch` (return the first element that satisfies a condition).

Sort, skip, limit are cursor modifiers. Regardless of the order in which they are called, the server applies them in the order sort, skip, limit. A sort without a supporting index is a blocking in-memory sort that is limited to 100 MB unless `allowDiskUse` is permitted. When two documents tie on the sort key their relative order is not defined, so a sort that feeds pagination must end with a unique field such as `_id`: `sort({ placedAt: -1, _id: -1 })`.

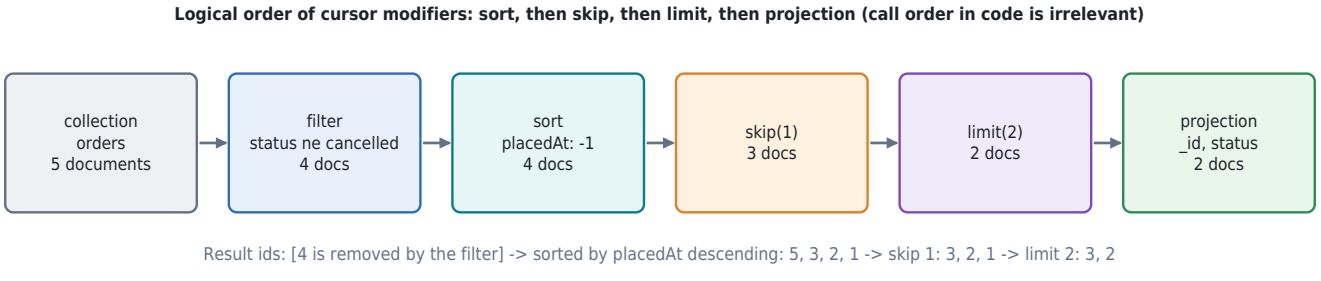


Figure 2.30. Cursor pipeline for find({ status: { $ne: 'cancelled' } }).sort({ placedAt: -1 }).skip(1).limit(2) over the five sample orders; the document counts after each step are those of the sample data.

`countDocuments(filter)` executes an aggregation that counts the matching documents and is exact. `estimatedDocumentCount()` reads collection metadata, is constant time, accepts no filter, and may be inaccurate after an unclean shutdown or in a sharded cluster with orphaned documents. `distinct(field, filter)` returns the unique values of a field and is limited by the 16 MiB result size.

Update: operators, not assignment

An update document consists of **update operators**; a plain document without operators is rejected by `updateOne` and `updateMany`, and is accepted only by `replaceOne`, which substitutes the entire document except `_id`. Operators change fields in place on the server, so the application neither reads nor rewrites the document, and the whole update to one document is atomic.

Operator	Effect	Typical use
`$set`	Sets a field, creating it and intermediate embedded documents if needed.	`{ $set: { status: 'shipped', 'shipTo.city': 'Pune' } }`
`$unset`	Removes a field.	`{ $unset: { tempFlag: '' } }`
`$inc`	Adds a number (negative to subtract); missing field starts at 0.	`{ $inc: { stock: -1 } }`
`$mul, $min, $max`	Multiply; keep the smaller or larger of current and given value.	`{ $max: { highScore: 87 } }`
`$rename`	Renames a field.	`{ $rename: { zip: 'pin' } }`
`$push`	Appends to an array; modifiers `$each`, `$position`, `$sort`, `$slice`.	`{ $push: { tags: { $each: ['a'], $slice: -10 } } }`
`$addToSet`	Appends only if the value is not yet present.	`{ $addToSet: { tags: 'gift' } }`
`$pull`	Removes all array elements that equal a value or satisfy a condition.	`{ $pull: { items: { sku: 'P0007' } } }`
`$pop`	Removes the first (-1) or last (1) array element.	`{ $pop: { queue: -1 } }`
`$setOnInsert`	Sets a field only when an upsert inserts.	`{ $setOnInsert: { createdAt: new Date() } }`
`$currentDate`	Sets the field to the server time.	`{ $currentDate: { updatedAt: true } }`

Update operators used most often.

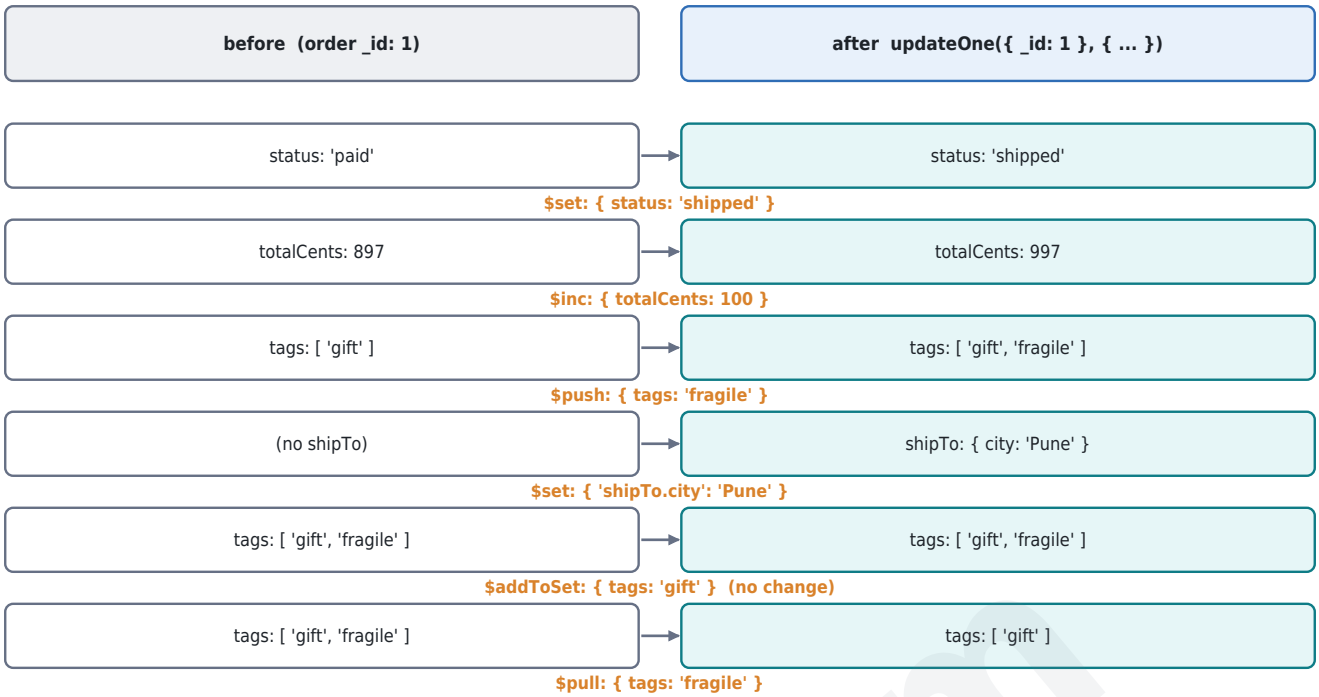
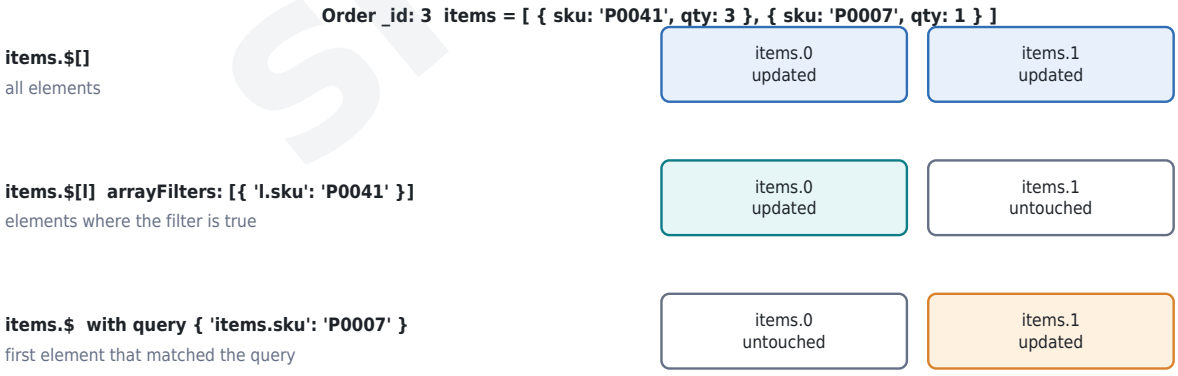


Figure 2.31. Effect of six update operators on order 1: each operator touches only the named field and the document keeps its `_id` and all other fields.

Since MongoDB 4.2 the update argument may also be an **aggregation pipeline** (an array of `$set`, `$unset`, `$replaceWith` stages), which allows the new value to be computed from other fields of the same document, for example `[{ $set: { totalCents: { $sum: '$items.priceCents' } } }]`. Each update returns `matchedCount` and `modifiedCount`; an update that matches a document but changes nothing, such as `$addToSet` of an existing value, has `modifiedCount: 0`.

Updating inside arrays: positional operators and arrayFilters

Three syntaxes address array elements. The **positional operator** `$` updates the first element that matched the array condition in the query part. The **all-positional operator** `$[]` updates every element. The **filtered positional operator** `$[identifier]` updates the elements for which a condition in the `arrayFilters` option is true; it is the only form that can update several, selected elements.



Update document: { $set: { 'items.$[l].qty': 4 } } changes only items.0.qty (3 to 4).

Figure 2.32. The three array update forms applied to order 3, which has two items. Only items.0 changes when `items.$[l]` is used with the filter `l.sku = 'P0041'`.

Upsert, findOneAndUpdate, delete

With `{ upsert: true }` an update inserts a document when no document matches. The inserted document is built from the equality conditions of the filter, then the update operators are applied; `$setOnInsert` fields apply only in that case. An upsert is atomic per document, but two concurrent upserts on a filter that no unique index protects may both insert, so the filter fields must be covered by a unique index (see the incident on duplicate documents in the indexes chapter).

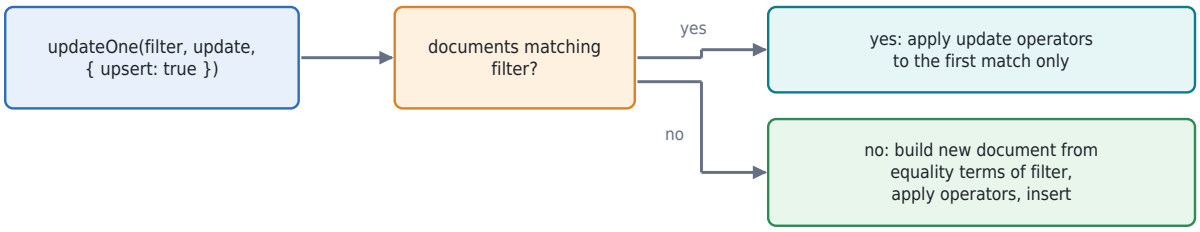


Figure 2.33. Decision made by an upsert: apply the operators to the first match, or insert a new document built from the filter equalities and the operators.

`findOneAndUpdate(filter, update, { returnDocument: 'after', sort, projection })` modifies one document and returns it in a single atomic step; with `sort` it can claim the next job from a queue. `findOneAndReplace` and `findOneAndDelete` are the equivalents for replacement and removal. `deleteOne` removes the first match and `deleteMany` removes all matches; `deleteMany({})` removes every document one by one and keeps the collection and its indexes, whereas `drop()` removes the collection with its indexes and is far cheaper. MongoDB has no `TRUNCATE` statement.

Bulk writes

`bulkWrite([ { insertOne: ... }, { updateOne: ... }, { deleteOne: ... } ], { ordered: false })` sends many different write operations in few round trips. In ordered mode the first error stops the batch; in unordered mode the server continues and reports every error in a `BulkWriteError` (`MongoBulkWriteException` in Java). The driver splits very large batches automatically (the documented limit is 100,000 operations per batch). MongoDB 8.0 added a new `bulkWrite` command that can address several namespaces in one request.

Write concern, read concern, and atomicity

A **write concern** states how many members must acknowledge a write before the client sees success: `w: 1` (primary only), `w: 'majority'` (a majority of data-bearing voting members), `j: true` (journaled), and `wtimeout`. Since MongoDB 5.0 the implicit default is `w: 'majority'` for replica sets and sharded clusters, except when arbiters make a majority unattainable; earlier versions defaulted to `w: 1`. A **read concern** states the consistency of the data returned: `local` (default; the node's latest data, which may later be rolled back), `majority` (data acknowledged by a majority), `snapshot` (inside transactions), `linearizable` (strongest, primary only), and `available` (for sharded clusters, may return orphans). The replication Part of this book treats both at length; the point here is that a write acknowledged with `w: 1` can disappear if the primary fails before replication.

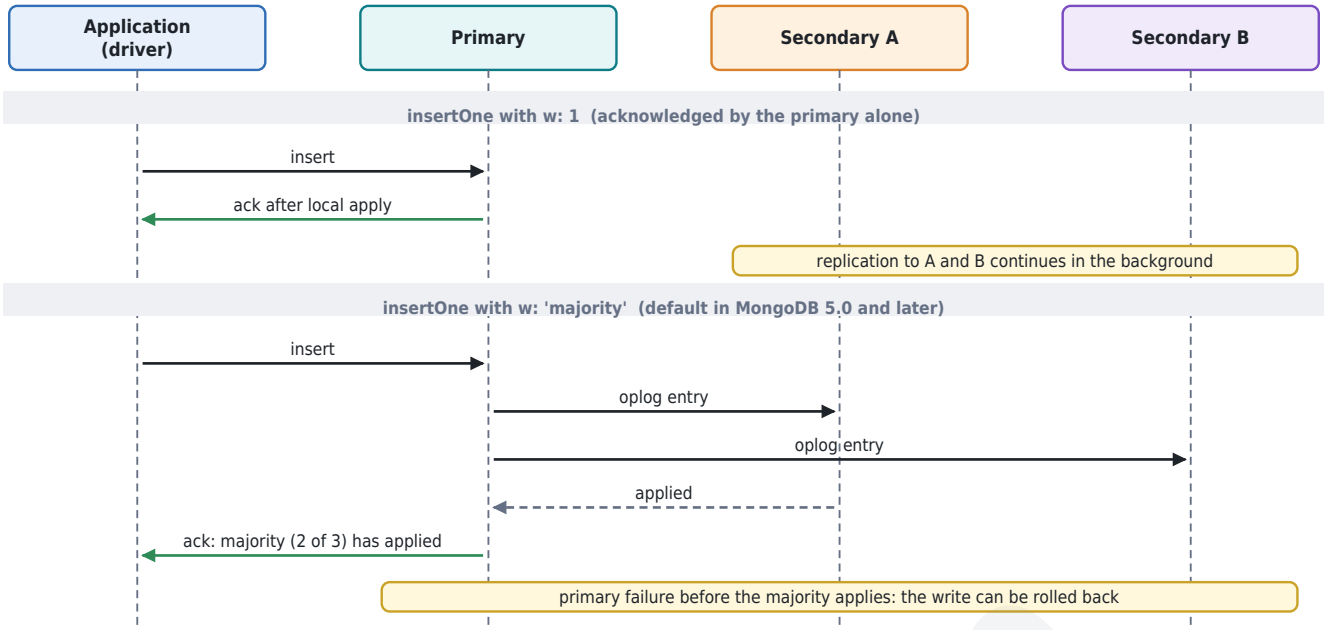


Figure 2.34. Acknowledgement timing: with w: 1 the primary answers immediately; with w: 'majority' it answers after one secondary has applied the oplog entry, so the write survives a failover.

Atomicity. A write to a single document is atomic, including its embedded documents and arrays: concurrent readers see either the old or the new document, never a mixture. A multi-document operation such as `updateMany` is **not** atomic as a whole; each document is updated atomically and a failure part-way leaves some documents updated. Multi-document ACID transactions exist (covered in the operations Part) but the schema design chapter shows how to avoid needing them.

2.10.2 Worked example: updating one line item inside an order

Task: change the quantity of the line `P0041` in order 3 from 3 to 4 and recompute the order total, without reading the order into the application.

Step 1: the document before the update.

Order 3 before JAVASCRIPT

```
{ _id: 3, customerId: 42, status: 'shipped',
  items: [ { sku: 'P0041', qty: 3, priceCents: 499 }, { sku: 'P0007', qty: 1, priceCents: 1299 } ],
  totalCents: 2796, tags: ['gift', 'priority'] }
```

Step 2: the update. The filtered positional operator `$[l]` introduces the identifier `l`; the `arrayFilters` entry `{ 'l.sku': 'P0041' }` states which elements `l` denotes. The increment of the total uses the price of the line (499 cents multiplied by the quantity difference 1).

Update with arrayFilters JAVASCRIPT

```
db.orders.updateOne(
  { _id: 3 },
  { $set: { 'items.$[l].qty': 4 }, $inc: { totalCents: 499 } },
  { arrayFilters: [ { 'l.sku': 'P0041' } ] }
)
```

Step 3: evaluation, as the server performs it:

1. The filter `{ _id: 3 }` selects one document through the `_id` index.
2. The path `items.$[l].qty` is expanded: for each element of `items` the filter `l.sku == 'P0041'` is evaluated. Element 0 satisfies it (`'P0041'`), element 1 does not (`'P0007'`). The path therefore resolves to the single concrete path `items.0.qty`.
3. `$set` writes 4 to `items.0.qty`; `$inc` adds 499 to `totalCents`, giving 3295 (2796 + 499; this equals 4 x 499 + 1 x 1299).
4. The modified document is written atomically. The result is `{ acknowledged: true, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }`.

Order 3 after (computed with the reference implementation) JAVASCRIPT

```
{ _id: 3, customerId: 42, status: 'shipped',
  items: [ { sku: 'P0041', qty: 4, priceCents: 499 }, { sku: 'P0007', qty: 1, priceCents: 1299 } ],
  totalCents: 3295, tags: ['gift', 'priority'] }
```

If no element satisfied the filter the update would match the document, change nothing, and report `modifiedCount: 0`; if the identifier `l` appeared in the update without a corresponding `arrayFilters` entry the server would reject the command with an error.

2.10.3 Hands-on

mongosh session on the sample data (typical output, not run on the build machine)

```
shop> db.orders.find({ status: 'paid' }, { customerId: 1, totalCents: 1 }).sort({ totalCents: -1 })
[
  { _id: 5, customerId: 58, totalCents: 995 },
  { _id: 1, customerId: 17, totalCents: 897 }
]
shop> db.orders.find(
...   { placedAt: { $gte: ISODate('2026-10-03'), $lt: ISODate('2026-10-08') },
...     status: { $ne: 'cancelled' } },
...   { status: 1 })
[ { _id: 2, status: 'pending' }, { _id: 3, status: 'shipped' } ]
shop> db.orders.find({}, { _id: 1 }).sort({ placedAt: -1 }).skip(1).limit(2)
[ { _id: 4 }, { _id: 3 } ]
shop> db.orders.updateOne({ _id: 1 },
...   { $set: { status: 'shipped', 'shipTo.city': 'Pune' }, $inc: { totalCents: 100 },
...     $push: { tags: 'fragile' } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
shop> db.orders.updateOne({ _id: 3 }, { $addToSet: { tags: 'gift' } })
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 0, upsertedCount: 0 }
shop> db.counters.updateOne({ _id: 'invoice' }, { $inc: { seq: 1 } }, { upsert: true })
{
  acknowledged: true,
  insertedId: 'invoice',
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
shop> db.orders.findOneAndUpdate({ status: 'pending' }, { $set: { status: 'paid' } },
...   { sort: { placedAt: 1 }, returnDocument: 'after', projection: { status: 1 } })
{ _id: 2, status: 'paid' }
shop> db.orders.insertOne({ _id: 1, status: 'x' })
MongoServerError: E11000 duplicate key error collection: shop.orders index: _id_ dup key: { _id: 1 }
shop> db.orders.deleteMany({ status: 'cancelled' })
{ acknowledged: true, deletedCount: 1 }
```

Three details deserve attention. The second `find` shows that the date range and `$ne` filters compose with AND. In the third `find` the sort precedes the skip, so `_id: 5` (newest) is skipped and the next two documents are returned. The upsert on `counters` created the document `{ _id: 'invoice', seq: 1 }` in one atomic step, which is the standard way to implement a sequence when one is genuinely needed.

preram

The same operations from Java (not run: the driver jar is not available on the build machine)

OrderCrud.java JAVA

```java
import static com.mongodb.client.model.Filters.*;
import static com.mongodb.client.model.Updates.*;

import com.mongodb.WriteConcern;
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.model.*;
import com.mongodb.client.result.UpdateResult;
import java.util.List;
import org.bson.Document;
import org.bson.conversions.Bson;

public class OrderCrud {
    public static void main(String[] args) {
        try (MongoClient client = MongoClients.create("mongodb://127.0.0.1:27017")) {
            MongoCollection<Document> orders = client.getDatabase("shop")
                .getCollection("orders")
                .withWriteConcern(WriteConcern.MAJORITY);   // explicit: durable across failover

            // Filtered positional update (arrayFilters)
            UpdateResult r = orders.updateOne(
                eq("_id", 3),
                combine(set("items.$[l].qty", 4), inc("totalCents", 499)),
                new UpdateOptions().arrayFilters(List.of(eq("l.sku", "P0041"))));
            System.out.println("matched=" + r.getMatchedCount()
                + " modified=" + r.getModifiedCount());

            // Atomic claim of the oldest pending order
            Document claimed = orders.findOneAndUpdate(
                eq("status", "pending"),
                set("status", "paid"),
                new FindOneAndUpdateOptions().sort(Sorts.ascending("placedAt"))
                    .returnDocument(ReturnDocument.AFTER));
            System.out.println(claimed);

            // Unordered bulk: continue past individual failures
            List<WriteModel<Document>> ops = List.of(
                new UpdateOneModel<>(eq("_id", 1), set("status", "shipped")),
                new UpdateOneModel<>(eq("_id", 5), push("tags", "fragile")),
                new DeleteOneModel<>(eq("_id", 4)));
            System.out.println(orders.bulkWrite(ops, new BulkWriteOptions().ordered(false)));
        }
    }
}
```

The builders (`Filters`, `Updates`, `Sorts`) produce the same BSON that the mongosh commands send. The `WriteConcern.MAJORITY` setting on the collection handle makes the acknowledgement level visible in code even though it is the default for current deployments.

2.10.4 Real-world problems and solutions

Incident: concurrent requests lose updates to a counter

Scenario. A stock service decrements `stock` in the `products` collection with a read-modify-write sequence: it loads the product, computes `stock - qty`, and saves the whole document (`replaceOne` or Spring Data `save`). Two requests for the same product arrive within a few milliseconds.

Symptoms. After 100 sold units the stock counter shows 14 fewer decrements than the number of orders. No error appears in any log. The loss grows with traffic.

Diagnosis. (1) Compare the order count with the stock difference: `db.orders.countDocuments({ 'items.sku': 'P0041' })` against the starting stock minus `db.products.findOne({ _id: 'P0041' }).stock`. (2) Read the code path for a pattern of `find` followed by `save` or `replaceOne` with no condition on the version read. (3) Reproduce with two threads that both read stock 10 and both write 9.

Root cause. Both requests read the same value (10), both computed 9, and the second write overwrote the first: a lost update. The write itself is atomic, but the sequence read, compute, write is not.

Fix. Immediate: replace the application-side arithmetic with an atomic operator and a guard that also prevents overselling.

Atomic decrement with a guard JAVASCRIPT

```
db.products.updateOne(
  { _id: 'P0041', stock: { $gte: 2 } },     // guard: enough stock
  { $inc: { stock: -2 } }
)
// matchedCount: 0  means the stock was insufficient; no document was modified
```

When the new value depends on several fields, use a version field: `updateOne({ _id, version: v }, { $set: {...}, $inc: { version: 1 } })` and retry when `matchedCount` is 0 (optimistic concurrency; Spring Data offers it through `@Version`).

Prevention. Treat every update that depends on the current value as an operator expression. Code review rule: no `save` of a whole document that other writers may touch.

Incident: an update wiped most of a document

Scenario. A developer migrates code from `updateOne` to `replaceOne` (or to a Spring Data `save` on a partially loaded entity) in order to "update the status".

Symptoms. After a status change the order has only `_id` and `status`; `items`, `totalCents`, and `customerId` are gone. Downstream reports show empty orders.

Diagnosis. `db.orders.find({ items: { $exists: false } }).count()` shows the damaged documents; the application log shows `replaceOne` calls with a filter and a replacement document of one field. If the database has an oplog, `db.getSiblingDB('local').oplog.rs.find({ ns: 'shop.orders', op: 'u' })` lists the update entries (a replacement appears as a full document under `o`, an operator update as `$v` and `diff`).

Root cause. `replaceOne` substitutes the whole document by design. A projection-based entity (a DTO with three fields) saved through a mapper sends only those three fields.

Fix. Restore from the most recent backup or point-in-time recovery for the damaged `_id` values, then re-apply later changes. Change the code to `updateOne` with `$set`.

Prevention. Use `$set` for partial changes, validate with a `$jsonSchema` validator that requires `items` and `totalCents` (the write fails with a document validation error), and keep a delayed secondary or periodic backups for recovery.

Incident: a bulk import stops half way

Scenario. A nightly job loads 50,000 products with `insertMany`. The batch contains a duplicate `_id` at position 18,214.

Symptoms. The job reports `E11000 duplicate key` and exits; the collection contains exactly 18,214 of the new products; the remaining 31,786 are missing, and a naive re-run inserts the first 18,214 again and fails at the same document.

Diagnosis. `insertMany` defaults to `ordered: true`, which stops at the first error. The exception (`MongoBulkWriteException` in Java) carries `getWriteResult().getInsertedCount()` and the list of `WriteError` objects with the failing index.

Root cause. Ordered semantics plus a non-idempotent insert.

Fix. Use `ordered: false` so that every valid document is inserted and all failures are reported together; make the load idempotent with `bulkWrite` of `replaceOne` or `updateOne` operations that use `upsert: true` on the natural key, so that a re-run converges to the same state.

Prevention. Load into a staging collection, check counts and duplicates with an aggregation, and then merge into the target with `$merge`.

2.10.5 Questions to expect

1. **What is the difference between `updateOne` and `replaceOne`?** `updateOne` applies update operators to selected fields; `replaceOne` substitutes the entire document except `_id`. Passing a document without operators to `updateOne` is an error.
2. **How is a read-modify-write race avoided in MongoDB?** Use an atomic operator (`$inc`, `$push`, `$addToSet`) with a guarding filter, or `findOneAndUpdate`, or a version field checked in the filter. Single-document operations are atomic.

Sreeram

3. **What does `upsert: true` do and what is its pitfall?** It inserts a document built from the filter equalities and the update if nothing matches. Without a unique index on the filter fields, concurrent upserts can create duplicates.
4. **Is `updateMany` atomic?** No. Each document is updated atomically, but the set is not; a failure can leave some documents modified. A multi-document transaction is required for all-or-nothing behaviour across documents.
5. **What are `$`, `$[]`, and `$[id]` in updates?** `$` updates the first array element matched by the query; `$[]` updates all elements; `$[id]` updates the elements that satisfy an `arrayFilters` condition.
6. **Ordered versus unordered bulk operations?** Ordered stops at the first error and preserves order; unordered continues after errors and may run in parallel. Choose unordered for independent writes.
7. **What do `w: 1` and `w: 'majority'` mean?** `w: 1` acknowledges after the primary applied the write; `w: 'majority'` after a majority of voting data-bearing members applied it, which protects against rollback during failover. The default is `majority` since 5.0.
8. **What does the order of `sort`, `skip`, and `limit` calls mean?** None: the server always applies sort, then skip, then limit.
9. **When should `countDocuments` be used instead of `estimatedDocumentCount`?** Use `countDocuments` for any filtered count or when exactness is required; `estimatedDocumentCount` is constant time metadata and takes no filter.

KEY POINTS

CRUD in MongoDB is expressed as filter, update-operator, and options documents applied to a collection.

- Use update operators (`$set`, `$inc`, `$push`, `$pull`, `$addToSet`) instead of read-modify-write; single-document writes are atomic, multi-document operations are not.
- Address array elements with `$`, `$[]`, or `$[id]` plus `arrayFilters`; use `upsert` only with a unique index behind the filter.
- Sort, skip, and limit always run in that order; add a unique tiebreaker to every sort that feeds pagination.
- `insertMany` and `bulkWrite` are ordered by default; use unordered, idempotent operations for loads.
- The write concern controls durability across failover; `w: 'majority'` is the default since MongoDB 5.0.

2.11 Query Operators, Arrays, and Embedded Documents

A MongoDB filter is a document whose keys are field paths or logical operators and whose values are literals or operator expressions. Most defects in MongoDB queries do not come from syntax but from three semantic rules that differ from SQL: comparisons are type-bracketed, a condition on an array field applies to the array and to its elements, and a missing field is not the same state as `null`. This chapter states the rules, demonstrates them on a five-order sample derived from the one in the previous chapter (listed below; order 5 has no status and a null shipping address), and checks the array semantics against PostgreSQL on the 20,000-order dataset used in the later chapters.

2.11.1 Concept and mechanism

Operator families

Family	Operators	Remark
Comparison	`$eq $ne $gt $gte $lt $lte $in $nin`	`$gt` and similar compare only values of the same BSON type class.
Logical	`$and $or $nor $not`	Top-level fields are an implicit AND.
Element	`$exists $type`	`$exists` tests presence; `$type` tests the BSON type.
Evaluation	`$regex $expr $mod $jsonSchema $text`	`$expr` compares fields of one document and uses aggregation expressions.
Array	`$all $elemMatch $size`	Equality and comparison on arrays match any element.
Geospatial	`$geoWithin $near $geoIntersects`	Need a `2dsphere` index for `$near`; see the indexes chapter.

Query operators by family.

Type bracketing and cross-type order

A comparison operator only matches values of the same type class: `{ qty: { $gt: '1' } }` compares with a string and therefore matches no document whose `qty` is a number, with no error. All numeric types (int, long, double, decimal) form one class and compare by value. Across classes, sorting uses a fixed order: MinKey, null, numbers, strings, objects, arrays, binary data, ObjectId, booleans, dates, timestamps, regular expressions, MaxKey. This order governs `sort()` on a field with mixed types and explains why a sort on a field that is sometimes missing places the missing documents first in ascending order (a missing field sorts as null).

Logical operators and the duplicate-key trap

Conditions on different fields are ANDed implicitly. Two conditions on the same field must not be written as two JavaScript keys, because the second key silently replaces the first when the object literal is built: `{ status: 'paid', status: 'shipped' }` is the filter `{ status: 'shipped' }`. Use `$and`, `$or`, or an operator document: `{ status: { $in: ['paid', 'shipped'] } }`. The Java `Filters.and`, `Filters.or` builders avoid the problem because they take separate objects.

Embedded documents: dot notation and whole-document equality

A condition on a nested field uses a dotted path: `{ 'shipTo.city': 'Pune' }`. This matches every document whose `shipTo` contains that field irrespective of the other fields. The alternative `{ shipTo: { city: 'Pune' } }` is **whole-value equality**: it matches only a `shipTo` that has exactly the field `city` with value `'Pune'`, no other fields, and in the same order. Equality on an embedded document is therefore both exact and order-sensitive, which makes it fragile; dotted paths are the general rule.

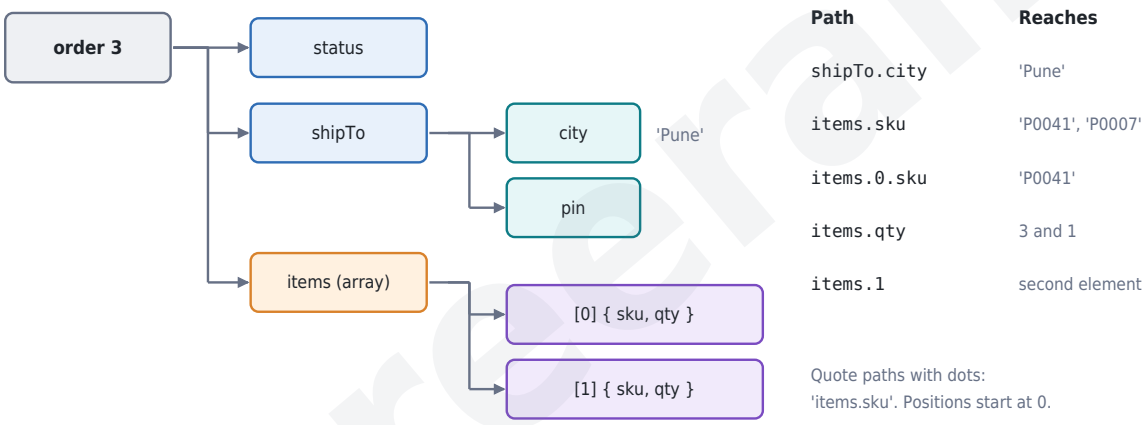


Figure 2.35. Document tree of order 3 and the values reached by dotted paths; a path that crosses an array reaches the values of every element.

Arrays

For a condition on an array field, MongoDB tests the array as a whole and each element. Hence `{ tags: 'gift' }` matches `['gift', 'priority']` (element match), and `{ tags: ['gift'] }` matches only an array that equals `['gift']` exactly (whole-array equality, order and length included). `{ tags: { $all: ['gift', 'priority'] } }` requires every listed value to be present; `{ tags: { $size: 2 } }` tests the length (only an exact length; there is no range form, a counter field or an index on `'tags.1'` is the usual substitute). `{ 'items.0.sku': 'P0041' }` addresses the first element.

When two conditions concern fields inside the elements of an array of embedded documents, each condition is evaluated independently and may be satisfied by different elements. `{ 'items.sku': 'P0041', 'items.qty': { $gt: 2 } }` therefore means "some element has sku P0041 and some element has qty greater than 2", which is rarely the intention. `$elemMatch` binds the conditions to a single element: `{ items: { $elemMatch: { sku: 'P0041', qty: { $gt: 2 } } } }`.

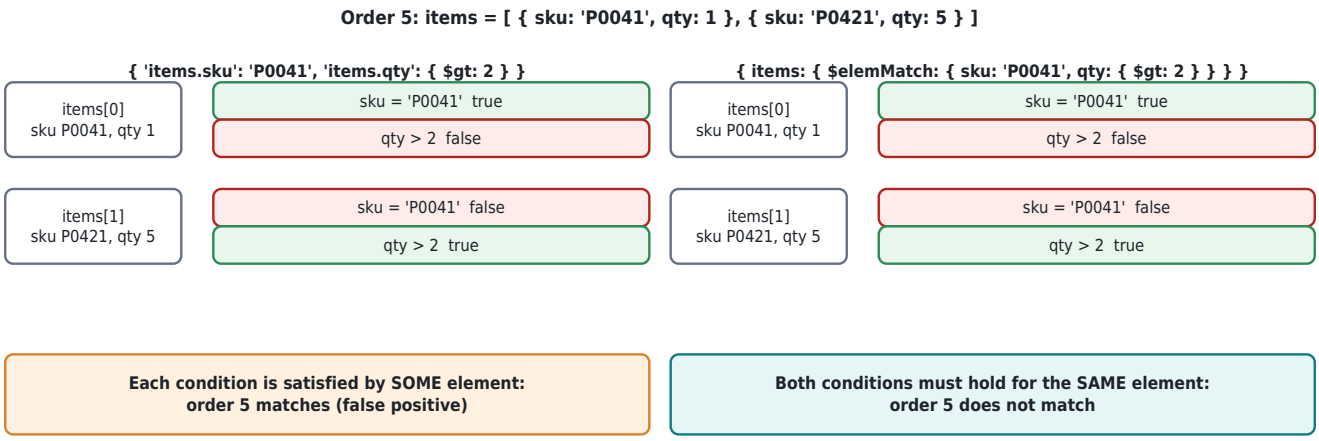


Figure 2.36. The same two conditions evaluated per element. Order 5 has the SKU in element 0 and the large quantity in element 1; the dotted query accepts it, `$elemMatch` rejects it.

Null, missing, and $exists

An absent field and a field with the value `null` are different states. `{ x: null }` matches both, and also an array that contains `null`. `{ x: { $exists: false } }` matches only absent fields. `{ x: { $type: 'null' } }` matches explicit null (and arrays containing null). `{ x: { $ne: null } }` matches documents where the field exists with a non-null value. Negative operators `$ne`, `$nin`, and `$not` also match documents in which the field is absent, because "absent" is "not equal" to every value.

document	{ x: null }	{ x: { $exists: false } }	{ x: { $type: 'null' } }	{ x: { $ne: null } }	{ x: { $in: [null] } }
a { x: null }	**match**	-	**match**	-	**match**
b { } (x missing)	**match**	**match**	-	-	**match**
c { x: 0 }	-	-	-	**match**	-
d { x: [null] }	**match**	-	**match**	-	**match**
e { x: [ ] }	-	-	-	**match**	-
f { x: 'null' }	-	-	-	**match**	-

Query { x: null } matches null, a missing field, and an array that contains null; $type: 'null' separates explicit null from missing.

Figure 2.37. Six documents against five predicates, computed with the reference matcher. Only $exists: false isolates the missing field, and only $type: 'null' isolates explicit null.

2.11.2 Worked example: why the dotted query accepts order 5

The extended sample contains the following orders (only the relevant fields are shown).

_id	items (sku, qty)	tags
1	(P0041, 1), (P0421, 2)	gift
2	(P0323, 1)	(empty)
3	(P0041, 3), (P0007, 1)	gift, priority
4	(P0007, 2)	(absent)
5	(P0041, 1), (P0421, 5)	priority

Sample for the array experiments.

Query A: `{ 'items.sku': 'P0041', 'items.qty': { $gt: 2 } }`. The first condition is true for orders 1, 3, 5 (each has a P0041 line). The second is true for orders 3 (qty 3) and 5 (qty 5). The AND of the two sets is {3, 5}. Order 5 qualifies although its P0041 line has quantity 1; the quantity 5 belongs to a different line.

Query B: `{ items: { $elemMatch: { sku: 'P0041', qty: { $gt: 2 } } } }`. Element by element: order 1 has no element with qty above 2; order 3 has `(P0041, 3)`, which satisfies both; order 5 has `(P0041, 1)`, failing the quantity, and `(P0421, 5)`, failing the sku. The result is {3}.

PostgreSQL shows the same distinction with JSONB path expressions, run for real on PostgreSQL 16: two independent `jsonb_path_exists` conditions behave like Query A, and one filter expression over the same element behaves like Query B.

PostgreSQL 16: independent conditions versus one element (table o holds the four documents' items) TERMINAL

```
$ psql -c "select id from o \
    where jsonb_path_exists(doc,'$.items[*].sku ? (@ == \"P0041\")') \
    and jsonb_path_exists(doc,'$.items[*].qty ? (@ > 2)') order by id"
 id
----
  3
  5
(2 rows)

$ psql -c "select id from o \
    where jsonb_path_exists(doc,'$.items[*] ? (@.sku == \"P0041\" && @.qty > 2)') order by id"
 id
----
  3
(1 row)
```

sreeram

2.11.3 Hands-on

Filter	_id values
`{ 'shipTo.city': 'Pune' }`	1, 2, 3
`{ shipTo: { city: 'Pune' } }`	3
`{ shipTo: { city: 'Pune', pin: '411001' } }`	1
`{ shipTo: { pin: '411001', city: 'Pune' } }`	2
`{ tags: 'gift' }`	1, 3
`{ tags: ['gift'] }`	1
`{ tags: { $all: ['gift', 'priority'] } }`	3
`{ tags: { $size: 2 } }`	3
`{ tags: { $in: ['priority', 'sale'] } }`	3, 5
`{ tags: [] }`	2
`{ 'items.sku': 'P0041' }`	1, 3, 5
`{ 'items.sku': 'P0041', 'items.qty': { $gt: 2 } }`	3, 5
`{ items: { $elemMatch: { sku: 'P0041', qty: { $gt: 2 } } } }`	3
`{ 'items.0.sku': 'P0041' }`	1, 3, 5
`{ items: { $size: 2 } }`	1, 3, 5
`{ status: { $ne: 'paid' } }`	2, 3, 4, 5
`{ status: { $nin: ['paid', 'shipped'] } }`	2, 4, 5
`{ status: { $exists: false } }`	5
`{ status: null }`	5
`{ status: { $regex: '^p' } }`	1, 2
`{ 'items.qty': { $gt: '1' } }`	(none)
`{ shipTo: null }`	5
`{ 'shipTo.pin': null }`	3, 5
`{ 'shipTo.pin': { $exists: true } }`	1, 2, 4
`{ 'items.qty': { $not: { $gt: 2 } } }`	1, 2, 4
`{ $or: [ { status: 'paid' }, { tags: 'priority' } ] }`	1, 3, 5
`{ items: { $not: { $elemMatch: { qty: { $gt: 2 } } } } }`	1, 2, 4

Results of 27 filters on orders 1 to 5 (reference matcher; mongosh returns the same _id sets).

Order 5 in this table has no `status` and `shipTo: null`; this is why it appears in `$ne`, `$nin`, `{ status: null }`, and `{ shipTo: null }` results. For the regular-expression filter the anchor `^` is deliberate: an anchored, case-sensitive prefix pattern can use an index range, an unanchored one cannot (see the indexes chapter).

On the 20,000-order dataset the semantic difference has a measurable size. The counts below come from the reference matcher on documents and were cross-checked with relational queries on PostgreSQL 16 (tables `orders` and `order_items`); both sides gave identical numbers.

Filter	Orders	PostgreSQL check
SKU P0041 on some line	102	`count(distinct order_id) ... where sku = 'P0041'`
SKU P0041 on some line AND qty 3 or more on some line (dotted form)	16	two `exists` subqueries
SKU P0041 with qty 3 or more on the same line (`$elemMatch`)	6	single `where sku = 'P0041' and qty >= 3`
Orders with exactly 3 lines (`$size: 3`)	4040	`having count(*) = 3`
Status `paid` or `pending`	3646	`status in (...)`

Filter	Orders	PostgreSQL check
Status not `delivered`	8062	`status <> 'delivered'`

Array filters on 20,000 orders (matcher and PostgreSQL agree).

The dotted form overstates the true answer by a factor of 2.7 (16 against 6) on this data, so a report built on it would be wrong by more than half.

From Java and Spring Data

ArrayQueries.java JAVA

```java
import static com.mongodb.client.model.Filters.*;

import com.mongodb.client.model.Filters;
import org.bson.conversions.Bson;

public class ArrayQueries {
    public static void main(String[] args) {
        // Dotted form: conditions may be met by different elements (rarely intended)
        Bson naive = and(eq("items.sku", "P0041"), gte("items.qty", 3));
        // Same-element form
        Bson correct = elemMatch("items", and(eq("sku", "P0041"), gte("qty", 3)));
        System.out.println(naive.toBsonDocument().toJson());
        System.out.println(correct.toBsonDocument().toJson());
    }
}
```

Spring Data MongoDB derives `findByItemsSku(String sku)` as the dotted form `{ 'items.sku': ?0 }`; for same-element conditions use `@Query("{ items: { $elemMatch: { sku: ?0, qty: { $gte: ?1 } } } }")` (the annotation syntax is documented; this snippet was not run because the driver and Spring jars are not available on the build machine).

2.11.4 Real-world problems and solutions

Incident: a report counts orders that do not satisfy the rule

Scenario. A loyalty rule awards points for "an order that contains at least 3 units of one specific product". The report job runs `find({ 'items.sku': 'P0041', 'items.qty': { $gte: 3 } })`.

Symptoms. The award total is 2.7 times higher than finance expects; sampling shows orders with one unit of the product and 5 units of another.

Diagnosis. (1) Print the filter that the code sends (`toBsonDocument().toJson()`), looking for two dotted paths with the same array prefix. (2) Run the filter and the `$elemMatch` form with `countDocuments` and compare: 16 against 6 on the sample dataset. (3) Fetch one extra order with `find(...)` and inspect the element positions.

Root cause. Conditions on dotted paths below an array are evaluated independently, so different elements may satisfy them.

Fix. Replace the filter with `$elemMatch`, repeat the count, and correct the awarded points with a compensating run restricted to the difference of the two result sets.

Prevention. Adopt the rule "two or more conditions on the same array of subdocuments require `$elemMatch`" in code review; add a unit test with a document whose two lines separately satisfy each condition.

Incident: the "not premium" list includes unmigrated records

Scenario. A migration adds the field `tier` to customers. The campaign query `find({ tier: { $ne: 'premium' } })` is intended to select customers that are not premium.

Symptoms. The campaign reaches customers that have no tier at all, including deleted-account stubs.

Diagnosis. `db.customers.countDocuments({ tier: { $exists: false } })` returns a non-zero count; `$ne` results include those documents.

Root cause. `$ne`, `$nin`, and `$not` match documents in which the field is missing; the matcher result above for `{ status: { $ne: 'paid' } }` includes order 5, which has no status.

Fix. Add the existence condition: `{ tier: { $exists: true, $ne: 'premium' } }`, or the positive form `{ tier: { $in: ['basic', 'gold'] } }`.

Prevention. Prefer positive conditions; backfill new fields with a default; enforce required fields with a `$jsonSchema` validator.

Incident: an exact embedded-document filter stops matching after a code change

Scenario. Orders were saved with `shipTo` fields in the order `city`, `pin`. A new service version maps the address through a different class whose fields are declared in the order `pin`, `city`. The lookup `find({ shipTo: { city: 'Pune', pin: '411001' } })` returns nothing for new orders.

Symptoms. Old orders match, new ones do not; the same values are visible in the documents.

Diagnosis. Print both documents; compare field order. The matcher result above shows `{ city, pin }` matches order 1 and `{ pin, city }` matches order 2, never both.

Root cause. Equality on a whole embedded document is order-sensitive and requires identical field sets.

Fix. Query dotted paths: `{ 'shipTo.city': 'Pune', 'shipTo.pin': '411001' }`.

Prevention. Never use embedded-document equality in application queries; if one value identifies a composite, use an explicit field.

2.11.5 Questions to expect

1. **How does a condition on an array field behave?** It matches if the array equals the value or any element satisfies the condition; `{ tags: 'gift' }` matches `['gift', 'priority']`.
2. **When is `$elemMatch` required?** When two or more conditions must hold for the same element of an array of embedded documents. Without it each condition may be satisfied by a different element.
3. **What is the difference between `{ x: null }`, `{ x: { $exists: false } }`, and `{ x: { $type: 'null' } }`?** The first matches null and missing; the second only missing; the third only explicit null.
4. **Why does `$ne` return documents without the field?** Absent fields are not equal to the value, so negative operators include them; add `$exists: true`.
5. **What is the difference between `{ shipTo: { city: 'Pune' } }` and `{ 'shipTo.city': 'Pune' }`?** The first is exact, order-sensitive whole-document equality; the second matches the nested field regardless of other fields.
6. **What does type bracketing mean?** Range comparisons only match values of the same type class, so a numeric field is never greater than a string; no conversion or error occurs.
7. **How are two conditions on one field written?** As one operator document (`{ qty: { $gt: 1, $lt: 5 } }`) or with `$and`; two identical keys in a literal collapse to one.
8. **How is an array length range queried?** `$size` accepts only an exact number; use `$expr` with `$size` for ranges (not indexable) or store a length field.

KEY POINTS

Array, embedded-document, and null semantics are where MongoDB filters differ most from SQL predicates.

- Comparisons are type-bracketed; numeric types compare together, everything else needs the same type.
- A condition on an array matches any element; use `$elemMatch` to bind several conditions to one element (16 against 6 orders on the sample data).
- Use dotted paths for embedded documents; whole-document equality is order-sensitive.
- `{ x: null }` matches missing and null; `$ne`, `$nin`, `$not` also match missing fields.
- Prefer positive conditions and validated, backfilled fields over negations.

2.12 Schema Design for MongoDB

A relational design starts from the data and normalises it; a MongoDB design starts from the access patterns and decides which data is stored together. The principle can be stated as: **data that is read together should be stored together, provided that the resulting document stays bounded and that the duplication it introduces is tolerable.** The flexible schema does not remove the need for design; it moves the constraints from the database (table definitions, foreign keys) to the application and to optional validation rules, and it makes some mistakes, in particular unbounded arrays, fatal at a large scale rather than at deployment time.

2.12.1 Concept and mechanism

Embedding versus referencing

Embedding stores related data as a nested document or array inside the parent. **Referencing** stores the related data in its own collection and keeps the identifier of one side in the other, as a manual foreign key. The consequences are symmetrical.

Aspect	Embedding	Referencing
Reads	One read returns the parent with its children.	One read per collection, or a `$lookup`; the application joins.
Writes	One atomic single-document write covers parent and children.	Several documents; atomicity across them needs a multi-document transaction.
Growth	The document grows with the children; bounded by 16 MiB.	Children grow independently without limit.
Duplication	A child shared by many parents is copied into each.	One copy; changes are made once.
Consistency	Copies can become stale (acceptable for snapshots such as prices at purchase time).	Always current; no copy to maintain.
Typical fit	Address, order lines, tags, a bounded list of latest items.	Shared catalogue entries, high-volume children, data read on its own.

Embedding compared with referencing.

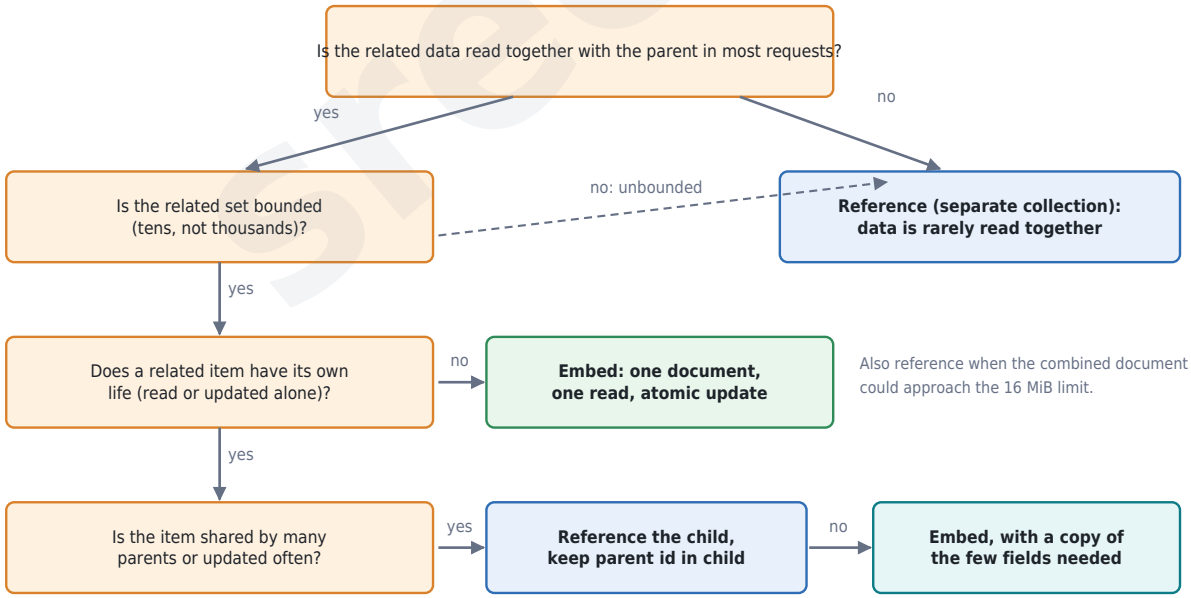


Figure 2.38. Decision path for embedding or referencing: read together, bounded, independent lifetime, shared or frequently updated.

Rules that follow from the table: embed when the relationship is "contains" and the child is meaningful only with its parent, when the set is bounded, and when the data is read together on the hot path. Reference when the child has its own lifecycle, when it is shared by many parents, when it is large or unbounded, or when it is updated far more often

than the parent is read. Duplicating a few fields of a referenced entity into the parent (an **extended reference**, such as the product name and unit price in an order line) is the usual compromise.

Relationship shapes

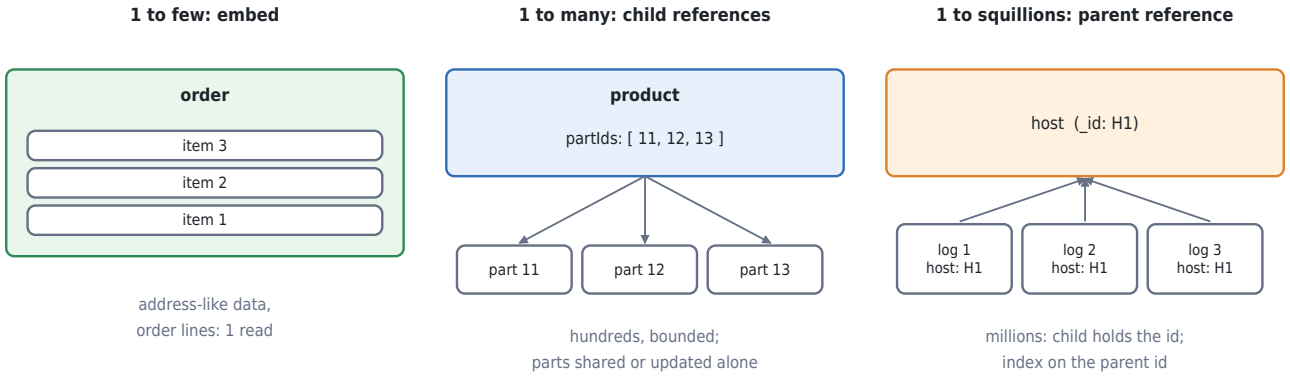


Figure 2.39. Three one-to-many shapes: embed for few, an array of child references for many, a parent reference stored in each child for very many.

One-to-one: embed unless the two parts have different access frequency or size (a rarely read, large profile document may live separately). **One-to-few**: embed as an array. **One-to-many** (hundreds): either embed or keep an array of child `_id` values in the parent. **One-to-squillions** (millions of log lines per host): store the parent `_id` in each child and index it; the parent never holds the list. **Many-to-many**: store an array of references on the side that is smaller or queried more often (a book has a few `authorIds`; an author may have many books, found with an index on `authorIds`); store references on both sides only if both directions are hot and the application can keep them consistent.

Tree and graph shapes

Hierarchies (categories, organisation charts, comments) have four standard encodings. **Parent reference** stores `parent` in each node: cheap to update, one query per level to descend. **Child references** store an array of children in the parent: easy to list children, awkward to move. **Materialised path** stores the path of ancestors as a string (`',electronics,audio,headphones,'`): a subtree is one anchored regular-expression query on an indexed field, at the price of rewriting descendants when a node moves. **Ancestor array** stores an array of all ancestor ids, indexed as a multikey index: both ancestors and descendants are single index lookups. For recursive traversals over many levels, `$graphLookup` (an aggregation stage) walks a reference chain on the server.

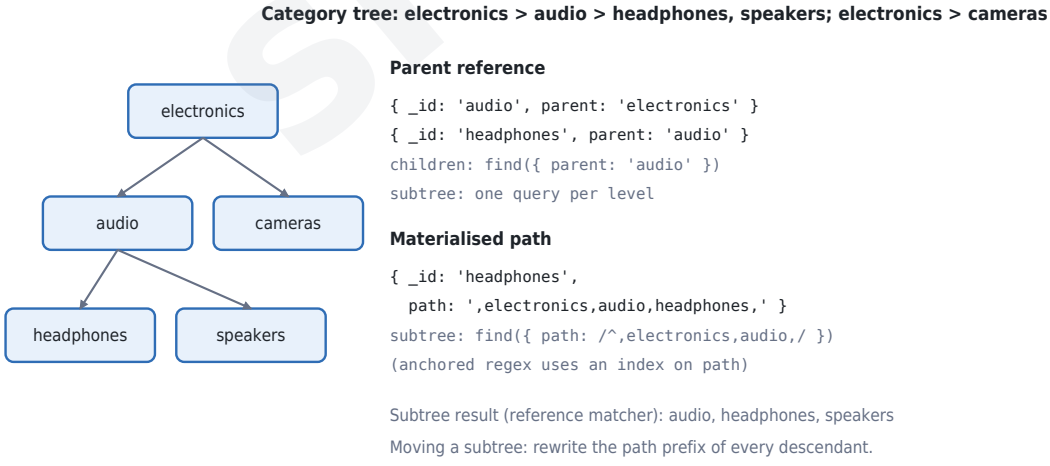


Figure 2.40. The same category tree encoded with parent references and with materialised paths. The subtree query on the path field returned audio, headphones, speakers on the sample data.

Design patterns

Pattern	Idea	Used for
Bucket	Group many small time-ordered items into one document per time window or per count.	Sensor readings, events, chat messages.
Subset	Embed only the most relevant few items (latest reviews) and keep the rest in another collection.	Product page showing 3 reviews of 50,000.
Computed	Store a derived value (sum, count, average) updated on write instead of recomputed on read.	Order totals, counters, hourly means.
Extended reference	Copy the few fields of a referenced document that the parent needs.	Customer name and city in an order.
Outlier	Flag the rare document with huge arrays and handle it separately (overflow documents).	A product with a million reviews among normal ones.
Schema versioning	A `schemaVersion` field on every document so that old and new shapes coexist during migration.	Zero-downtime evolution.
Attribute	Turn many sparse fields into an array of key-value pairs that one index can cover.	Product specifications that differ per category.

Frequently used MongoDB schema patterns.

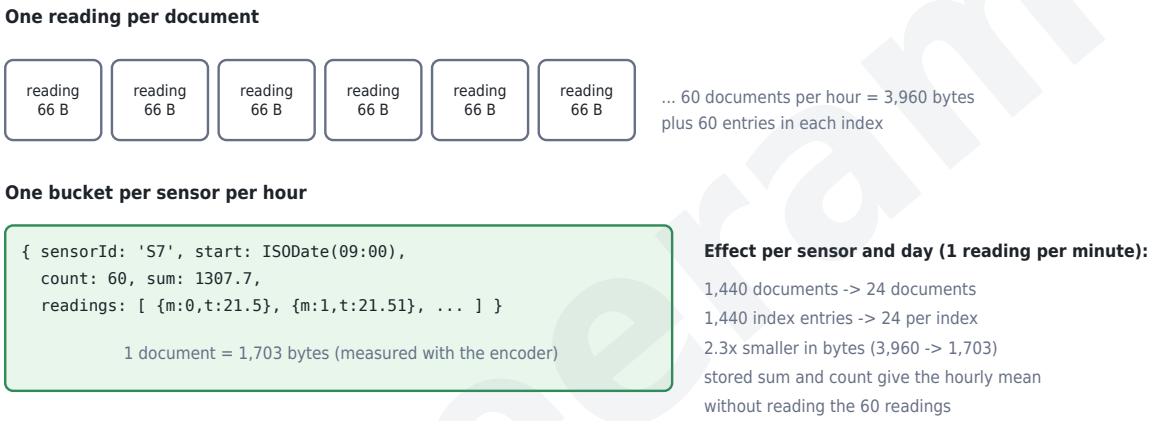


Figure 2.41. The bucket pattern measured with the BSON encoder: 60 one-minute readings of 66 bytes each occupy 3,960 bytes as 60 documents or 1,703 bytes as one bucket document, and the index entries shrink by 60 times.

Anti-patterns

- **Unbounded arrays**: an array that grows with usage (reviews, comments, events). The document approaches the 16 MiB limit, every update touches a larger document, and the array becomes slower to scan. Use child documents with a parent reference, or a bounded bucket or subset.
- **Massive documents**: large documents loaded for a small field. Project only the needed fields, or split hot and cold fields into two documents.
- **Many collections**: one collection per tenant, per day, or per user multiplies indexes and file handles (every collection and every index is a file in WiredTiger). One collection with a tenant field and an index on it is the normal solution.
- **Application-side joins everywhere**: the symptom of modelling a relational schema one-to-one as collections; the fix is to embed what is read together.
- **Case and type inconsistency**: `'Paid'` and `'paid'`, or a number stored as string in some documents, which break equality filters and indexes silently.
- **Field names as data**: documents such as `{ '2026-10-01': 5, '2026-10-02': 7 }` that cannot be indexed or queried; use an array of key-value subdocuments.

sreeram

Validation with $jsonSchema

A collection can carry a validator that checks every insert and update. `validationLevel` (`strict` for all writes, `moderate` for documents that already conform) and `validationAction` (`error` rejects, `warn` logs) tune the rollout. A validator restores part of the constraint system that relational tables provide: required fields, types, ranges, enumerations. It does not provide referential integrity or cross-document uniqueness; uniqueness requires a unique index.

Validator for the orders collection JAVASCRIPT

```
db.createCollection('orders', {
  validator: { $jsonSchema: {
    bsonType: 'object',
    required: ['customerId', 'status', 'placedAt', 'items', 'totalCents', 'schemaVersion'],
    properties: {
      customerId:    { bsonType: ['int', 'long'] },
      status:        { enum: ['pending', 'paid', 'shipped', 'delivered', 'cancelled'] },
      placedAt:      { bsonType: 'date' },
      totalCents:    { bsonType: ['int', 'long'], minimum: 0 },
      schemaVersion: { bsonType: 'int' },
      items: {
        bsonType: 'array', minItems: 1, maxItems: 200,
        items: {
          bsonType: 'object',
          required: ['sku', 'qty', 'priceCents'],
          properties: { sku: { bsonType: 'string' }, qty: { bsonType: 'int', minimum: 1 },
                        priceCents: { bsonType: ['int', 'long'] } }
        }
      }
    }
  } },
  validationLevel: 'strict',
  validationAction: 'error'
})
```

The `maxItems: 200` on `items` turns the unbounded-array rule into a database-enforced limit.

2.12.2 Worked example: the e-commerce model redesigned for MongoDB

The relational design used in this book has four tables: `customers`, `products`, `orders`, and `order_items` (an order has one or more lines). The access patterns that drive the redesign are: (1) show an order with its lines (the dominant read, by order id and by customer), (2) place an order, (3) list a customer's latest orders, (4) show a product with a few recent reviews, (5) report revenue by day and category.

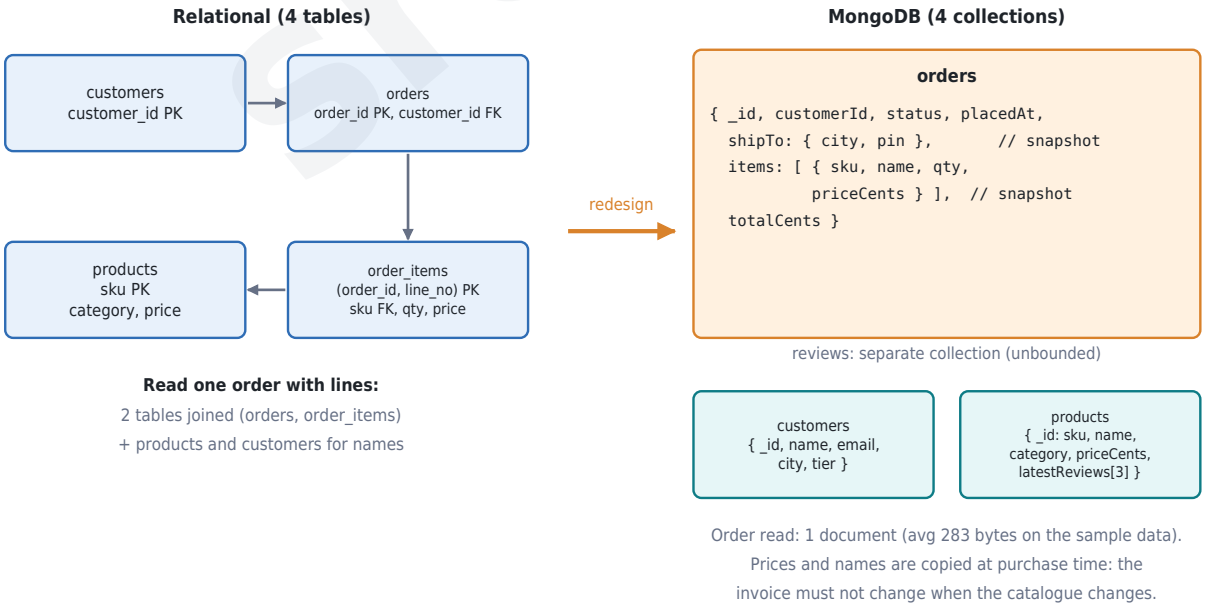


Figure 2.42. The relational model (left) and its MongoDB redesign (right): order lines and the shipping address are embedded in the order as snapshots; reviews are kept separately because they are unbounded.

Decisions:

1. **Order lines are embedded in the order.** They are created and read with the order, never alone, and bounded (at most 4 lines in the sample data, `maxItems: 200` in the validator). Placing an order becomes one atomic insert instead of one insert into `orders` plus one per line inside a transaction.
2. **Name and unit price are copied into each line** (extended reference and snapshot). An invoice must keep the price paid; a later catalogue price change must not alter history. The `sku` remains as the reference.
3. **The shipping city and pin are copied into the order** for the same reason, and `totalCents` is stored (computed pattern) and kept equal to the sum of `qty * priceCents`.
4. **Customers and products are separate collections**: they have their own lifecycle and are referenced by many orders. `products` embeds the three most recent reviews (subset pattern) for the product page.
5. **Reviews are a separate collection** with `productId`, because a popular product may collect hundreds of thousands of them.

Measured on the 20,000-order sample dataset with the BSON encoder, and checked against the relational data in PostgreSQL 16:

Quantity	MongoDB (embedded)	Relational / referenced
Average order document with lines	283 bytes	131.6 bytes order row plus 1.99 line rows of 97 bytes each (as separate documents)
Average lines per order	1.99 (inside the order)	1.99 (rows in `order_items`)
Documents or rows read to show one order	1	1 + 1.99 on average (or one join)
Total storage of orders and lines	5.66 MB (20,000 documents)	6.50 MB (20,000 + 39,874 documents) if lines were separate documents
Statements to place an order (avg)	1 insert	1 + 1.99 inserts in one transaction

Cost of the two designs on the sample data.

The referenced variant is larger because every line document repeats an `_id` and the `orderId`. The relational side rebuilds the embedded form with a join and an aggregate; the query below ran on PostgreSQL 16 for order 1 and returns the shape that MongoDB stores directly.

PostgreSQL 16: rebuilding the document shape for order 1 (output wrapped) TERMINAL

```
$ psql -c "select o.order_id, o.status, o.total_cents, \
    json_agg(json_build_object('sku',i.sku,'qty',i.qty,'priceCents',i.price_cents) \
             order by i.line_no) as items \
  from orders o join order_items i using(order_id) where o.order_id=1 group by o.order_id"
 order_id |  status   | total_cents | items
----------+-----------+-------------+------------------------------------------------------------
        1 | delivered |       10697 | [{"sku" : "P0041", "qty" : 1, "priceCents" : 499},
                                      {"sku" : "P0421", "qty" : 1, "priceCents" : 199},
                                      {"sku" : "P0323", "qty" : 1, "priceCents" : 9999}]
(1 row)
```

The trade-off appears on the analytic side. Pattern (5), revenue by category, needs the product category for each line; because the line stores only `sku`, `name`, `qty`, and `priceCents`, the aggregation chapter joins to `products` with `$lookup` (or the line also embeds `category`, a one-field extended reference chosen when the report is hot). The rule is to decide by measured frequency: the dominant read is the order page, so the order is optimised for it and the report pays a join.

Resulting order document (order 1 of the sample dataset plus a schemaVersion field) JSON

```
{
  "_id": 1, "customerId": 383, "status": "delivered", "schemaVersion": 3,
  "placedAt": { "$date": "2026-08-03T09:23:47Z" },
  "items": [
    { "sku": "P0041", "name": "Product 041", "qty": 1, "priceCents": 499 },
    { "sku": "P0421", "name": "Product 421", "qty": 1, "priceCents": 199 },
    { "sku": "P0323", "name": "Product 323", "qty": 1, "priceCents": 9999 }
  ],
  "totalCents": 10697,
  "shipTo": { "city": "Vizag", "pin": "500069" }
}
```

2.12.3 Hands-on

mongosh: validator rejects an invalid order (typical output, not run on the build machine)

```
shop> db.orders.insertOne({ customerId: 17, status: 'shipping', placedAt: new Date(),
...   items: [], totalCents: -5, schemaVersion: 3 })
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('68f3a1d8b4c2e19a7d05f3c2'),
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      { operatorName: 'properties', propertiesNotSatisfied: [
        { propertyName: 'status', description: 'attribute "status" value was not in the enum' },
        { propertyName: 'totalCents', description: 'attribute "totalCents" failed minimum' },
        { propertyName: 'items', description: 'array did not meet minItems' } ] } ]
  }
}
```

The exact layout of the error detail varies between server versions; the stable parts are the code `121` (`DocumentValidationFailure`) and the list of failed properties.

Bucket writes: one upsert per reading

Appending a reading to the current hourly bucket JAVASCRIPT

```
db.readings.updateOne(
  { sensorId: 'S7', start: ISODate('2026-10-18T09:00:00Z'), count: { $lt: 60 } },
  { $push: { readings: { m: 7, t: 21.57 } }, $inc: { count: 1, sum: 21.57 } },
  { upsert: true }
)
```

The filter includes `count: { $lt: 60 }` so that a full bucket no longer matches and the upsert opens a new one; an index on `{ sensorId: 1, start: 1 }` keeps the lookup cheap. A unique index on `{ sensorId, start }` would make the opening of a second bucket for the same hour an error, and is the safer choice when overflow buckets are not intended.

Migrating the shape with schema versioning

Upgrade v2 documents to v3 in batches (aggregation-pipeline update) JAVASCRIPT

```
db.customers.updateMany(
  { schemaVersion: 2 },
  [ { $set: {
        firstName: { $arrayElemAt: [ { $split: ['$name', ' '] }, 0 ] },
        lastName:  { $arrayElemAt: [ { $split: ['$name', ' '] }, -1 ] },
        schemaVersion: 3 } },
    { $unset: 'name' } ]
)
```

In production the same statement is applied in key ranges (for example by `_id` ranges of 10,000) so that the load and the replication lag stay bounded; the application reads both shapes until the migration has finished.

The order entity in Spring Data MongoDB (not run: Spring jars are not available on the build machine)

Order.java JAVA

```java
import java.time.Instant;
import java.util.List;
import org.springframework.data.annotation.Id;
import org.springframework.data.annotation.Version;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "orders")
public class Order {
    @Id
    private Long id;
    @Indexed
    private long customerId;
    private String status;
    private Instant placedAt;
    private List<OrderItem> items;      // embedded, bounded
    private long totalCents;
    private ShipTo shipTo;              // snapshot copied at purchase time
    private int schemaVersion = 3;
    @Version
    private Long version;               // optimistic locking: updates check this field

    public record OrderItem(String sku, String name, int qty, long priceCents) { }
    public record ShipTo(String city, String pin) { }
    // getters and setters omitted
}
```

`@Version` makes Spring Data add the version to the update filter and increment it, which turns the read-modify-write race of the CRUD chapter into an `OptimisticLockingFailureException` that the caller can retry.

2.12.4 Real-world problems and solutions

Incident: product reviews grow until writes fail

Scenario. The first design embedded reviews in the product: `{ _id, name, reviews: [ ... ] }`. A promotional campaign drives one product to a very large number of reviews.

Symptoms. The product page slows down over weeks, loading a multi-megabyte document to show three reviews. Then `$push` to `reviews` fails for that product only, with an error of the following form (typical output; the byte counts vary).

Typical error

```
MongoServerError: BSONObj size: 16793618 (0x1200AD10) is invalid.
  Size must be between 0 and 16793600(16MB)
  code: 10334, codeName: 'BSONObjectTooLarge'
```

Diagnosis. (1) Find the largest documents: `db.products.aggregate([{ $project: { n: { $size: '$reviews' }, bytes: { $bsonSize: '$$ROOT' } } }, { $sort: { bytes: -1 } }, { $limit: 5 }])` (`$bsonSize` is available since MongoDB 4.4). (2) Check the array growth rate by date of the newest review. (3) Compute the headroom: with 92-byte reviews and about 7 bytes of element overhead the encoder measured 168,882 reviews as the maximum (16,777,118 bytes) and 168,883 as over the limit.

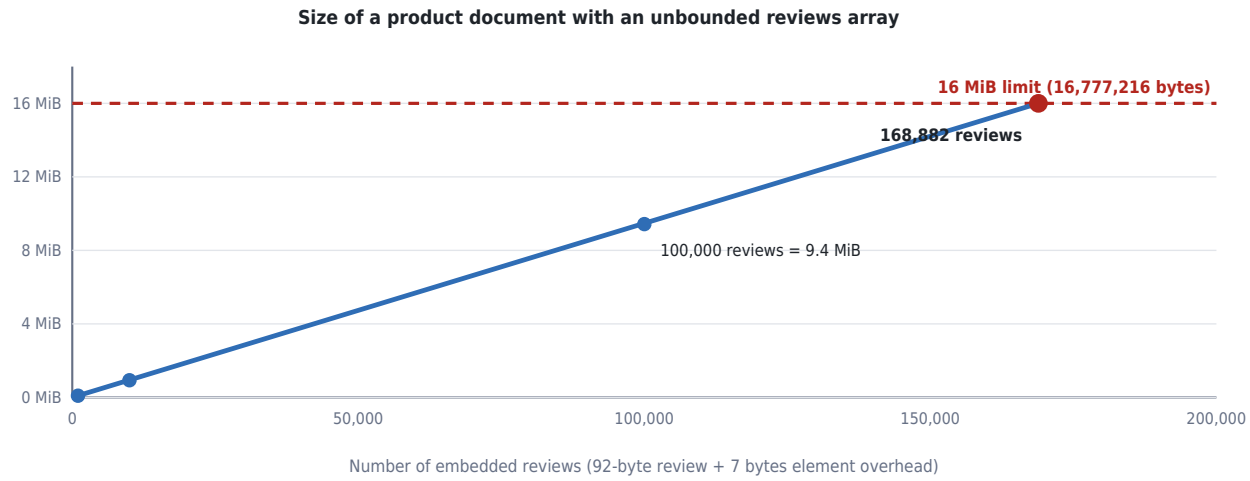


Figure 2.43. Size of a product document against the number of embedded 92-byte reviews, computed with the BSON encoder: 100,000 reviews occupy 9.4 MiB and the 16 MiB limit is reached at 168,882.

Root cause. An array whose length is determined by user behaviour, not by the domain, was embedded. Every update also rewrites a larger document.

Fix. Immediate: move reviews of the affected product to a `reviews` collection with a `productId` field using an aggregation with `$unwind` and `$merge`, then `$unset` the array and restore the writes. Permanent: keep the `reviews` collection indexed on `{ productId: 1, createdAt: -1 }`, embed only the latest three in the product (subset pattern, updated with `$push` with `$each`, `$sort`, and `$slice: 3`), and keep `reviewCount` and `ratingSum` as computed values updated with `$inc`.

Prevention. Write `maxItems` into the `$jsonSchema` validator for every embedded array, and add a monitoring query for the 99th percentile of `$bsonSize` per collection.

Incident: a new release cannot read old customer documents

Scenario. Release 3 splits `name` into `firstName` and `lastName` and changes `phone` (string) into `phones` (list). Documents written by releases 1 and 2 stay in the collection.

Symptoms. After the deployment, the customer service API returns 500 for old customers with a mapping exception (typical text: a `phone` string cannot be converted to `List`), or returns null names for others. New customers work.



Figure 2.44. Three generations of customer documents in one collection read by an entity class that knows only the newest shape.

Diagnosis. (1) Group by shape: `db.customers.aggregate([{ $group: { _id: { v: '$schemaVersion', hasName: { $gt: ['$name', null] } }, n: { $sum: 1 } } }])`. A missing `schemaVersion` marks v1 or v2 documents. (2) Match the stack trace field name to the shapes found. (3) Confirm with `db.customers.countDocuments({ phone: { $type: 'string' } })`.

Root cause. The deployment assumed that the schema changes atomically; MongoDB does not rewrite existing documents when a class changes, so every historical shape is still read by the new code.

Fix. Immediate: roll back or deploy a tolerant reader (a custom Spring Data `@ReadingConverter` or `Document`-based mapping that upcasts v1 and v2 to the v3 object). Permanent: run the batch migration shown above, verify `countDocuments({ schemaVersion: { $ne: 3 } })` is zero, then remove the upcasting code.

Prevention. Follow expand and contract: first deploy readers that understand both shapes, then writers of the new shape, then migrate, and only then remove the old reader. Require `schemaVersion` in the validator for new writes.

Incident: one collection per customer

Scenario. A multi-tenant application creates `orders_<tenantId>` for each of 4,000 tenants, each with three indexes.

Symptoms. Start-up and failover take minutes, memory use is high with little data, and cache efficiency is poor; `db.getCollectionNames()` returns thousands of names.

Diagnosis. `db.stats()` shows `collections: 4000` and `indexes: 12000`; each collection and index is a separate file in the WiredTiger data directory, so the open-file and metadata overhead dominates.

Root cause. A tenant boundary was modelled as a namespace boundary.

Fix. Merge into one `orders` collection with `tenantId` as the first field of the relevant compound indexes, migrating with `$merge` per tenant; enforce tenant scoping in a repository layer. Permanent: use a database-per-tenant only for a small number of large tenants with isolation requirements.

Prevention. Keep collection counts in the hundreds, not thousands; review any design in which collection names contain data values.

2.12.5 Questions to expect

1. **When should data be embedded and when referenced?** Embed data that is read together, owned by the parent, and bounded. Reference data that is shared, unbounded, large, or updated independently.
2. **What is the bucket pattern?** Grouping many small time-ordered records in one document per time window, with stored counts and sums; it cuts the number of documents and index entries and speeds range reads.
3. **What is the subset pattern?** Embedding only the few most relevant items of a large set (latest reviews) while the full set lives in a separate collection.
4. **What is wrong with an unbounded array?** It can reach the 16 MiB limit, makes each update and read heavier, and has no good indexing story; it signals a one-to-many relationship that should use references.
5. **How is a hierarchy modelled?** Parent references (cheap updates), child references, materialised paths (indexable subtree queries), or ancestor arrays; `$graphLookup` handles recursion on the server.
6. **How is referential integrity ensured without foreign keys?** By design (embedding), by application logic, by transactions for multi-document changes, and by validators and unique indexes for local rules; the database does not enforce references.
7. **Why copy fields into a document?** For read speed and for historical correctness (price at purchase); the cost is stale copies, acceptable only for snapshot data or when a background job refreshes them.
8. **How does schema versioning work?** A `schemaVersion` field is written on every document; readers upcast old versions; a batch job migrates; the field is later required by validation.
9. **What does `$jsonSchema` validation provide and not provide?** Required fields, types, ranges, enums, and array sizes on writes; not references between documents, and not uniqueness (a unique index does that).
10. **How does the design of the same domain differ between PostgreSQL and MongoDB?** PostgreSQL normalises and joins at query time; MongoDB embeds the aggregate that is read as a unit, snapshots volatile data, and accepts duplication; the single-document write replaces a small transaction.

KEY POINTS

Schema design in MongoDB is driven by access patterns, with the 16 MiB limit and update behaviour as the boundary conditions.

- Embed bounded, co-read, owned data; reference shared, unbounded, or independently updated data.
- Use the patterns by name: bucket, subset, computed, extended reference, outlier, schema versioning.
- Never let user behaviour define the length of an embedded array: the 16 MiB limit is reached at 168,882 reviews of 92 bytes.
- Validators enforce shape and bounds, not references or uniqueness; a unique index enforces the latter.
- Evolve schemas with a `schemaVersion` field and expand-and-contract deployments; old shapes stay in the data.

2.13 The Aggregation Framework

Aggregation is MongoDB's general data-processing facility: a **pipeline** of stages, each of which receives a stream of documents, transforms it, and passes the result to the next stage. Filtering, projection, grouping, sorting, joining, reshaping, and writing results back are all stages. The pipeline replaces the `GROUP BY`, `JOIN`, `HAVING`, and `INSERT ... SELECT` constructs of SQL and, because the stages run inside the server, avoids moving raw documents to the application.

All examples use the 20,000-order sample dataset (collections `orders`, `customers`, `products`; the order document has an embedded `items` array). Each of the 15 analytic queries in the mapping section was executed with a Python reference implementation of the stages **and** as an equivalent SQL query on PostgreSQL 16 on the same data; the result sets were identical in all 15 cases. The mongosh output is labelled **typical output (not run on the build machine)**; the numbers in it are the verified ones.

2.13.1 Concept and mechanism

Pipeline model

`db.orders.aggregate([ stage1, stage2, ... ])` returns a cursor over the documents emitted by the last stage. The first stage reads from the collection; if it is `$match` (optionally followed by `$sort`) it can use an index. A stage never sees the collection directly except `$lookup` and `$unionWith`, which read a second collection.

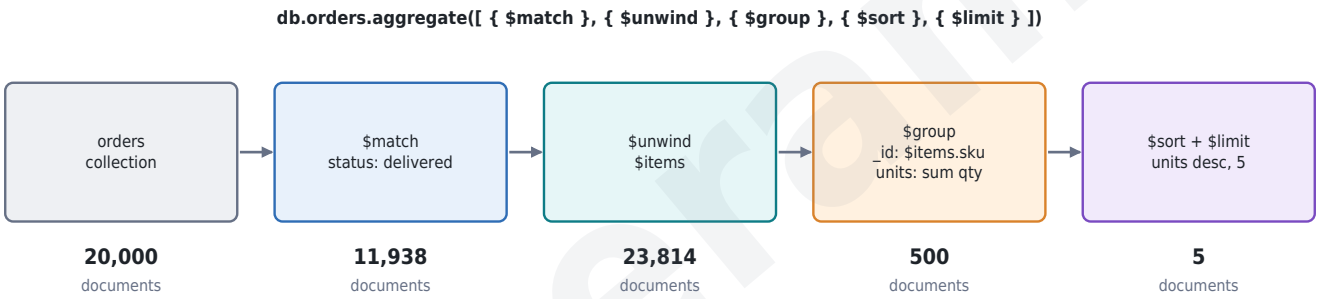


Figure 2.45. A five-stage pipeline with the document count after each stage on the sample data. $unwind raises the count from 11,938 orders to 23,814 lines; $group reduces it to the 500 distinct SKUs.

Stage	Effect	SQL counterpart
`$match`	Keeps documents that satisfy a query filter.	`WHERE`, `HAVING`
`$project`	Includes, excludes, renames, or computes fields.	`SELECT` list
`$addFields / $set`	Adds or overwrites fields, keeping the others.	computed column
`$unset`	Removes fields.	column list
`$group`	Groups by `_id` expression and computes accumulators.	`GROUP BY`
`$sort`	Orders documents.	`ORDER BY`
`$limit / $skip`	Keeps or discards a number of documents.	`LIMIT / OFFSET`
`$unwind`	Emits one document per array element.	lateral join over an array
`$lookup`	Left outer join to another collection; matches arrive as an array.	`LEFT JOIN`
`$count`	Emits one document with the count.	`COUNT(*)`
`$facet`	Runs several sub-pipelines over the same input.	several queries
`$bucket / $bucketAuto`	Groups by value ranges (fixed or automatic).	`CASE` + `GROUP BY`
`$sortByCount`	Group by value, count, sort descending.	`GROUP BY ... ORDER BY count DESC`

Stage	Effect	SQL counterpart
`$replaceRoot / $replaceWith`	Promotes a subdocument to the document.	select of a nested record
`$unionWith`	Appends the documents of another collection.	`UNION ALL`
`$setWindowFields`	Window functions (running totals, ranks, moving averages).	`OVER (...)` (MongoDB 5.0)
`$graphLookup`	Recursive traversal of references.	recursive CTE
`$merge / $out`	Writes the result into a collection.	`INSERT ... SELECT`

Core stages.

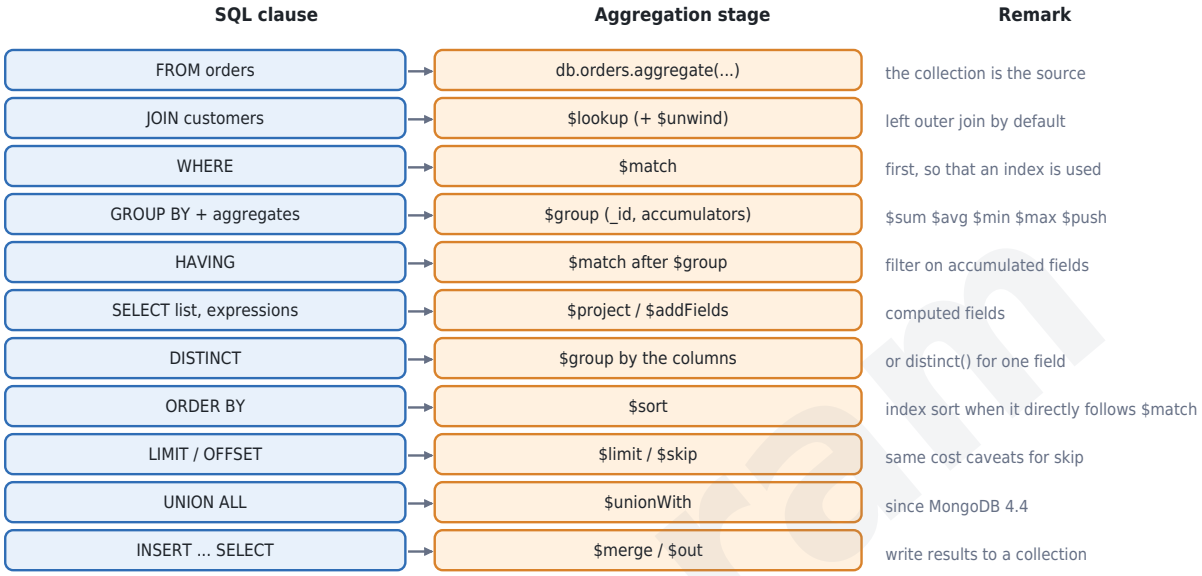


Figure 2.46. Mapping of SQL clauses to pipeline stages. Both express the logical order filter, group, filter again, project, sort, limit; in a pipeline the order is written explicitly.

Expressions and accumulators

Stage arguments contain **expressions**: field paths (`'$items.qty'`), literals, and operators such as `$add`, `$multiply`, `$cond`, `$ifNull`, `$dateToString`, `$size`, `$filter`, `$map`, `$arrayElemAt`, `$concat`. `$group` computes **accumulators** per group: `$sum`, `$avg`, `$min`, `$max`, `$first`, `$last` (meaningful only after a `$sort`), `$push` (collect all values), `$addToSet` (collect distinct values), and `$count`. The group key `_id` can be any expression, including a document for a compound key (`{ city: '$shipTo.city', status: '$status' }`) or `null` for one overall group.

$unwind, $lookup, $facet

`$unwind: '$items'` replaces each order by one document per line item, and the field `items` then holds one subdocument instead of an array. An order with an empty or missing array disappears unless `preserveNullAndEmptyArrays: true` is set. After an unwind, every order-level field is repeated on each of its lines, which is the cause of the double-counting incident in this chapter.

`$lookup` has two forms. The equality form `{ from, localField, foreignField, as }` matches one local field to one foreign field and is served by an index on the foreign field. The pipeline form `{ from, let, pipeline, as }` runs a sub-pipeline per input document with variables and supports conditions that are not simple equality, at a higher cost. In both forms the result is an array in `as`, empty when nothing matches.

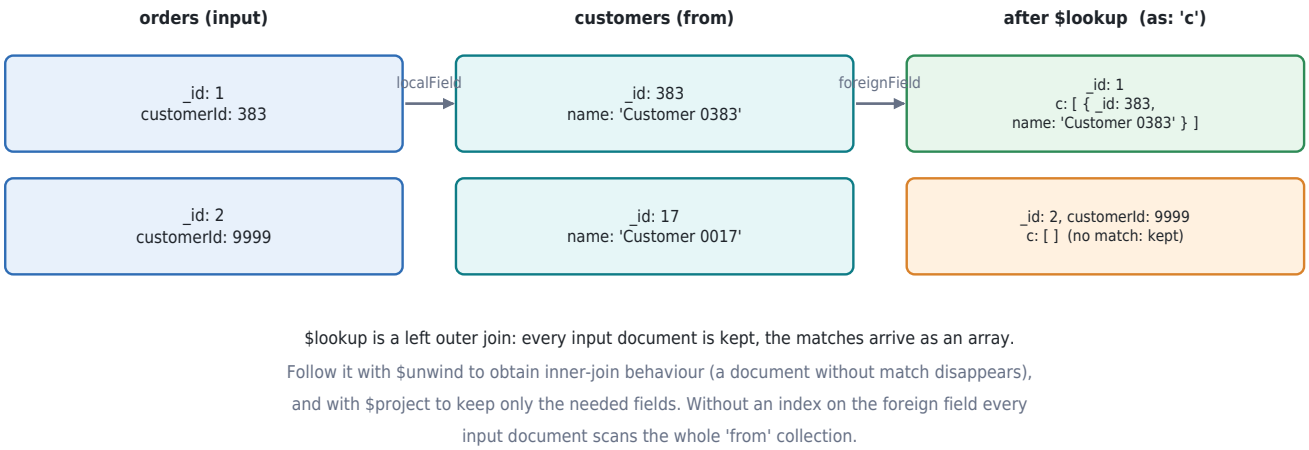


Figure 2.47. $lookup keeps every input document: order 2 refers to a customer that does not exist and receives an empty array. Matches are always returned as an array.

`$facet` evaluates several sub-pipelines over the same input and returns one document whose fields are the arrays of results; it is the usual way to return a page of results together with counts by category. The whole output is a single document and is therefore limited to 16 MiB, and the stages inside a facet cannot use indexes, so a `$match` must precede the `$facet`.

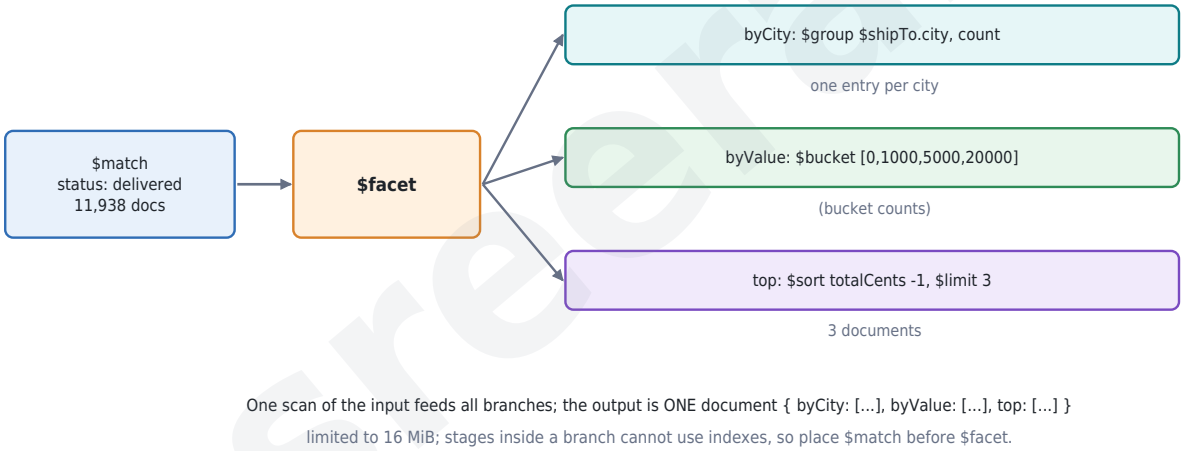
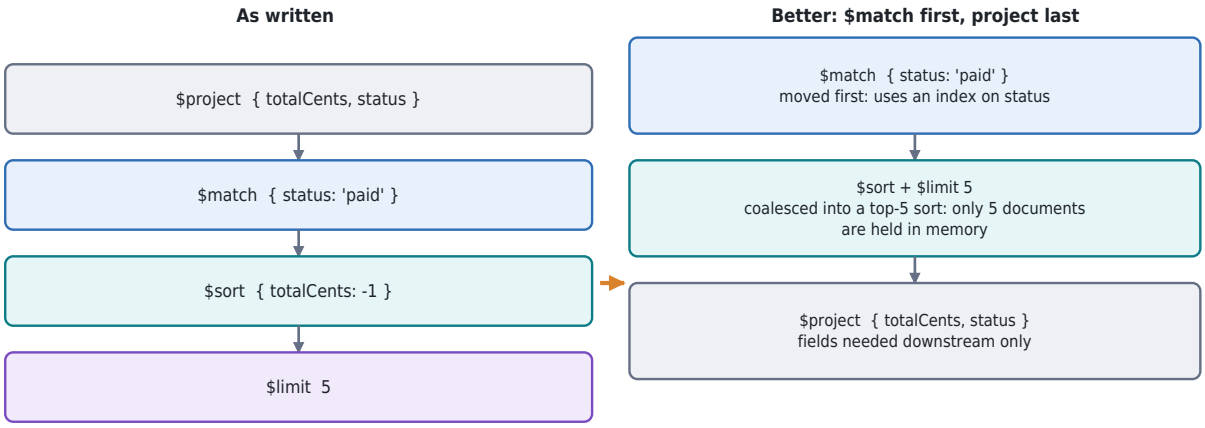


Figure 2.48. $facet runs three sub-pipelines on the filtered input and returns one document containing three arrays.

How the optimiser rewrites a pipeline

The server rewrites a pipeline before running it. The documented rewrites include moving a `$match` ahead of a `$project`, `$addFields`, or `$sort` when it does not depend on the fields they create, merging adjacent `$match` stages, merging `$sort` with a following `$limit` into a top-k sort that holds only k documents, merging `$limit` and `$skip`, merging `$lookup` with the `$unwind` that follows it, and computing only the fields that later stages need. Writing the stages in the efficient order remains good practice because not every case can be rewritten (a `$match` on a computed field cannot move before the computation).

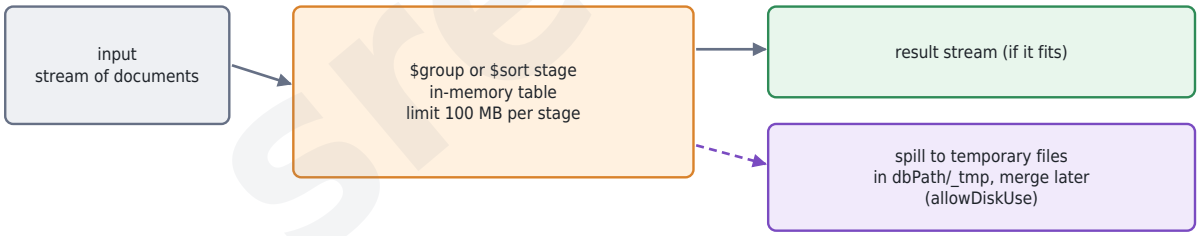


Same result, fewer documents examined. The optimiser can move a $match ahead of a $project and merge $sort with $limit, but writing the stages in the efficient order does not depend on it; check with explain().

Figure 2.49. A pipeline that projects before filtering (left) and the efficient order (right): `$match` first so that an index applies, then `$sort` with `$limit` as a top-k sort, then the projection.

Memory, disk, and output stages

Each blocking stage (`$sort` without index support, `$group`, `$bucket`, `$setWindowFields`) is limited to **100 MB** of RAM. If the limit is exceeded and disk use is not allowed, the command fails with error 292 (`QueryExceededMemoryLimitNoDiskUseAllowed`). Since MongoDB 6.0 the server parameter `allowDiskUseByDefault` is true, so the stage spills to temporary files; older deployments and any with that parameter set to false need `{ allowDiskUse: true }` in the options. `$merge` writes results into a collection, inserting, replacing, merging, or failing per document through `whenMatched` and `whenNotMatched`, and can maintain an incrementally refreshed summary collection (an on-demand materialised view); `$out` replaces a whole collection atomically. The pipeline length is limited to 1,000 stages.



Memory limit exceeded and disk use not allowed: the command fails (QueryExceededMemoryLimitNoDiskUseAllowed, code 292).

MongoDB 6.0 and later: allowDiskUseByDefault is true, so the stage spills instead of failing, at the cost of disk I/O.

Reduce first: $match early, $project only needed fields, group by fewer keys, avoid $push of whole documents.

Figure 2.50. Behaviour of a blocking stage at the 100 MB limit: it fails, or it spills to temporary files in the data directory, depending on allowDiskUse.

2.13.2 Worked example: tracing a pipeline

Task: the five most sold products by units on delivered orders. A three-document extract makes every stage visible: order A (delivered; P0041 x1, P0421 x2), order B (delivered; P0041 x3), order C (paid; P0421 x1, P0007 x1).

Pipeline JAVASCRIPT

```
db.orders.aggregate([
  { $match:  { status: 'delivered' } },
  { $unwind: '$items' },
  { $group:  { _id: '$items.sku', units: { $sum: '$items.qty' } } },
  { $sort:   { units: -1, _id: 1 } },
  { $limit:  5 }
])
```

1. `$match` keeps A and B; C is discarded (3 documents become 2).
2. `$unwind` emits (A, P0041 x1), (A, P0421 x2), (B, P0041 x3): 3 documents.
3. `$group` creates a running state per key. P0041 receives 1 and then 3, giving `units: 4`; P0421 receives 2, giving `units: 2`: 2 documents.
4. `$sort` places P0041 (4) before P0421 (2); `$limit: 5` keeps both.

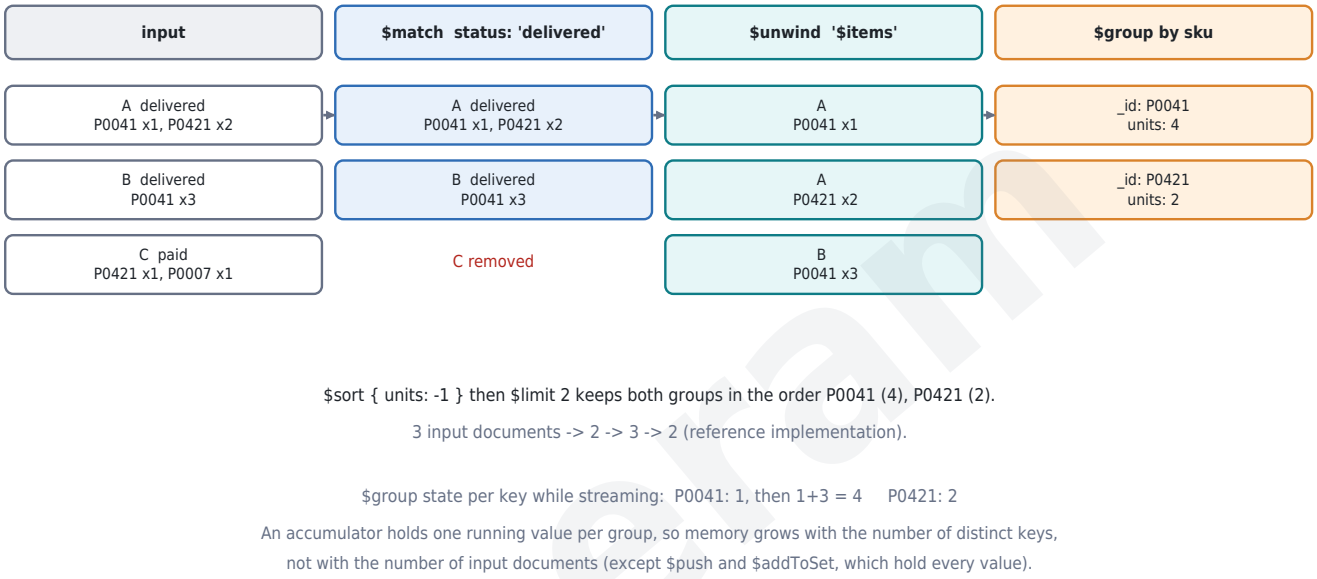


Figure 2.51. The same trace as documents: the three input orders, the filtered orders, the unwound lines, and the two groups.

On the full dataset the same pipeline processes 20,000 orders, passes 11,938 of them, produces 23,814 lines, forms 500 groups, and returns the five best sellers.

mongosh on the full dataset (typical output; values verified against PostgreSQL 16)

```
shop> db.orders.aggregate([ ...pipeline above... ])
[
  { _id: 'P0029', units: 99 },
  { _id: 'P0165', units: 98 },
  { _id: 'P0108', units: 94 },
  { _id: 'P0411', units: 91 },
  { _id: 'P0305', units: 89 }
]
```

The PostgreSQL check was `select i.sku, sum(i.qty) from order_items i join orders o using(order_id) where o.status = 'delivered' group by 1 order by 2 desc, 1 limit 5`, which returned the same five SKUs and unit counts. A pipeline with `$limit` after `$sort` needs a tiebreaker (`_id: 1`) to be deterministic, exactly as `ORDER BY` needs a unique final column.

2.13.3 Hands-on

SQL queries rewritten as pipelines

The table lists 15 analytic queries over the sample schema. Column 2 is the PostgreSQL query (tables `orders`, `order_items`, `customers`, `products`); column 3 is the pipeline over the embedded documents. Both were executed and returned identical result sets (compared row by row, including order).

#	SQL (PostgreSQL 16)	Pipeline
1	`select status, count(*) from orders group by 1 order by 2 desc, 1`	`[{$group:{_id:'$status',n:{$sum:1}}}, {$sort:{n:-1,_id:1}}]`
2	`select sum(total_cents) from orders where status='delivered'`	`[{$match:{status:'delivered'}}, {$group:{_id:null,rev:{$sum:'$totalCents'}}}]`
3	`select ship_city, count(*), sum(total_cents) from orders where status<>'cancelled' group by 1 order by 3 desc limit 3`	`[{$match:{status:{$ne:'cancelled'}}}, {$group:{_id:'$shipTo.city', orders:{$sum:1},rev:{$sum:'$totalCents'}}}, {$sort:{rev:-1}}, {$limit:3}]`
4	`select to_char(placed_at at time zone 'UTC','YYYY-MM'), count(*), sum(total_cents) from orders where placed_at >= '2026-01-01' and status='delivered' group by 1 order by 1`	`[{$match:{placedAt:{$gte:ISODate('2026-01-01')},status:'delivered'}}, {$group:{_id:{$dateToString:{format:'%Y-%m',date:'$placedAt'}}, n:{$sum:1},rev:{$sum:'$totalCents'}}}, {$sort:{_id:1}}]`
5	`select sku, sum(qty) from order_items group by 1 order by 2 desc, 1 limit 5`	`[{$unwind:'$items'}, {$group:{_id:'$items.sku',units:{$sum:'$items.qty'}}}, {$sort:{units:-1,_id:1}}, {$limit:5}]`
6	`select c.name, count(*), sum(o.total_cents) from orders o join customers c using(customer_id) where o.status in ('paid','shipped','delivered') group by c.customer_id, c.name order by 3 desc limit 3`	`[{$match:{status:{$in:['paid','shipped','delivered']}}}, {$group:{_id:'$customerId',orders:{$sum:1},spent:{$sum:'$totalCents'}}}, {$sort:{spent:-1}}, {$limit:3}, {$lookup:{from:'customers',localField:'_id',foreignField:'_id',as:'c'}}, {$unwind:'$c'}]`
7	`select c.tier, count(*), avg(o.total_cents) from orders o join customers c using(customer_id) group by 1 order by 1`	`[{$lookup:{from:'customers',localField:'customerId',foreignField:'_id',as:'c'}}, {$unwind:'$c'}, {$group:{_id:'$c.tier',n:{$sum:1},avg:{$avg:'$totalCents'}}}, {$sort:{_id:1}}]`
8	`select p.category, sum(i.qty*i.price_cents) from order_items i join products p using(sku) join orders o using(order_id) where o.status<>'cancelled' group by 1 order by 2 desc`	`[{$match:{status:{$ne:'cancelled'}}}, {$unwind:'$items'}, {$lookup:{from:'products',localField:'items.sku',foreignField:'_id',as:'p'}}, {$unwind:'$p'}, {$group:{_id:'$p.category',rev:{$sum:{$multiply:['$items.qty','$items.priceCents']}}}}, {$sort:{rev:-1}}]`
9	`select customer_id, count(*) from orders group by 1 having count(*) >= 18 order by 2 desc, 1`	`[{$group:{_id:'$customerId',n:{$sum:1}}}, {$match:{n:{$gte:18}}}, {$sort:{n:-1,_id:1}}]`
10	`select case when total_cents<1000 then 0 when total_cents<5000 then 1000 when total_cents<20000 then 5000 else 20000 end b, count(*) from orders group by 1 order by 1`	`[{$bucket:{groupBy:'$totalCents',boundaries:[0,1000,5000,20000],default:20000,output:{n:{$sum:1}}}}]`
11	`select distinct on (customer_id) customer_id, order_id from orders order by customer_id, placed_at desc, order_id desc`	`[{$sort:{customerId:1,placedAt:-1,_id:-1}}, {$group:{_id:'$customerId',last:{$first:'$_id'}}}]`
12	`select count(distinct customer_id) from orders where status='cancelled'`	`[{$match:{status:'cancelled'}}, {$group:{_id:'$customerId'}}, {$count:'n'}]`
13	`select count(*) from (select order_id from order_items group by 1 having count(*)>=3) t`	`[{$match:{$expr:{$gte:[{$size:'$items'},3]}}}, {$count:'n'}]`
14	`select order_id, total_cents, (select sum(qty) from order_items i where i.order_id=o.order_id) from orders o where order_id<=3`	`[{$match:{_id:{$lte:3}}}, {$addFields:{units:{$sum:'$items.qty'}}}, {$project:{totalCents:1,units:1}}]`
15	`select order_id from orders where status='paid' order by placed_at desc, order_id desc offset 20 limit 3`	`[{$match:{status:'paid'}}, {$sort:{placedAt:-1,_id:-1}}, {$skip:20}, {$limit:3}, {$project:{_id:1}}]`

SQL query and equivalent pipeline (all 15 pairs verified).

#	Result
1	delivered 11,938; shipped 2,983; paid 2,114; pending 1,532; cancelled 1,433
2	147,270,475 cents
3	Bengaluru 2,789 orders / 34,885,192; Delhi 2,630 / 33,759,947; Hyderabad 2,551 / 31,242,170
4	9 months; 2026-01: 578 orders, 7,740,281; 2026-02: 493, 5,899,841; 2026-03: 592, 7,756,262
5	P0165 159; P0108 151; P0055 145; P0429 144; P0411 143
6	Customer 0001: 554 orders, 6,969,808; Customer 0002: 366, 4,361,408; Customer 0003: 244, 2,896,139
7	basic 14,331 orders avg 12,365.3; gold 3,604 avg 11,837.9; platinum 2,065 avg 12,957.7
8	10 categories; fashion 30,520,955; sports 25,501,079; kitchen 24,893,577
9	193 customers with at least 18 orders (customer 1: 653; customer 2: 410)
10	below 1,000: 3,770; 1,000 to 4,999: 5,663; 5,000 to 19,999: 6,030; 20,000 and above: 4,537
11	latest order of customers 1, 2, 3: orders 11963, 6423, 274

Srgerm

#	Result
12	792 distinct customers with a cancelled order
13	5,976 orders with three or more lines
14	order 1: 10,697 cents, 3 units; order 2: 9,998, 2; order 3: 6,294, 6
15	orders 19938, 14987, 17059

Verified results (identical on both sides).

Three observations. First, rows 6 and 7 show that a join in MongoDB is an explicit `$lookup` plus `$unwind`; row 6 runs the `$lookup` only for the three surviving groups because the `$limit` precedes it, the cheap order. Second, row 8 needs two `$unwind`/`$lookup` steps because the lines carry only `sku`; if the report is frequent, embedding `category` in each line removes both. Third, customer 1 has 653 orders against an average of 10: per-customer result sizes therefore vary widely, which matters for pagination and for the per-customer index queries of the indexes chapter.

The same pipeline in Java and in Spring Data MongoDB (not run: jars not available on the build machine)

TopProducts.java JAVA

```java
import static com.mongodb.client.model.Aggregates.*;
import static com.mongodb.client.model.Accumulators.sum;
import static com.mongodb.client.model.Filters.eq;
import static com.mongodb.client.model.Sorts.*;

import com.mongodb.client.AggregateIterable;
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import java.util.List;
import org.bson.Document;

public class TopProducts {
    public static void main(String[] args) {
        try (MongoClient client = MongoClients.create("mongodb://127.0.0.1:27017")) {
            AggregateIterable<Document> result = client.getDatabase("shop")
                .getCollection("orders")
                .aggregate(List.of(
                    match(eq("status", "delivered")),
                    unwind("$items"),
                    group("$items.sku", sum("units", "$items.qty")),
                    sort(orderBy(descending("units"), ascending("_id"))),
                    limit(5)))
                .allowDiskUse(true);
            result.forEach(d -> System.out.println(d.toJson()));
        }
    }
}
```

Spring Data MongoDB equivalent (fragment) JAVA

```java
import static org.springframework.data.mongodb.core.aggregation.Aggregation.*;

import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.aggregation.Aggregation;
import org.springframework.data.mongodb.core.query.Criteria;
import org.bson.Document;

public record UnitsBySku(String id, long units) { }

class ReportService {
    private final MongoTemplate template;
    ReportService(MongoTemplate template) { this.template = template; }

    java.util.List<Document> topProducts() {
        Aggregation agg = newAggregation(
            match(Criteria.where("status").is("delivered")),
            unwind("items"),
            group("items.sku").sum("items.qty").as("units"),
            sort(Sort.by(Sort.Direction.DESC, "units").and(Sort.by("_id"))),
            limit(5));
        return template.aggregate(agg, "orders", Document.class).getMappedResults();
    }
}
```

2.13.4 Real-world problems and solutions

Incident: revenue report is 2.5 times too high

Scenario. A report sums `totalCents` of delivered orders after unwinding the lines because it also needs a per-category breakdown: `[{ $match }, { $unwind: '$items' }, { $group: { _id: null, rev: { $sum: '$totalCents' } } }]`.

Symptoms. The total is 366,393,617 cents against 147,270,475 in the finance ledger; per-category totals add up to far more than the grand total.

Diagnosis. (1) Compare with `db.orders.aggregate([{ $match: { status: 'delivered' } }, { $group: { _id: null, rev: { $sum: '$totalCents' } } }])`, which returns 147,270,475. (2) Compute the ratio 366,393,617 / 147,270,475 = 2.49, close to the 1.99 lines per order weighted by order value. (3) Search the pipeline for an order-level field summed after `$unwind`.

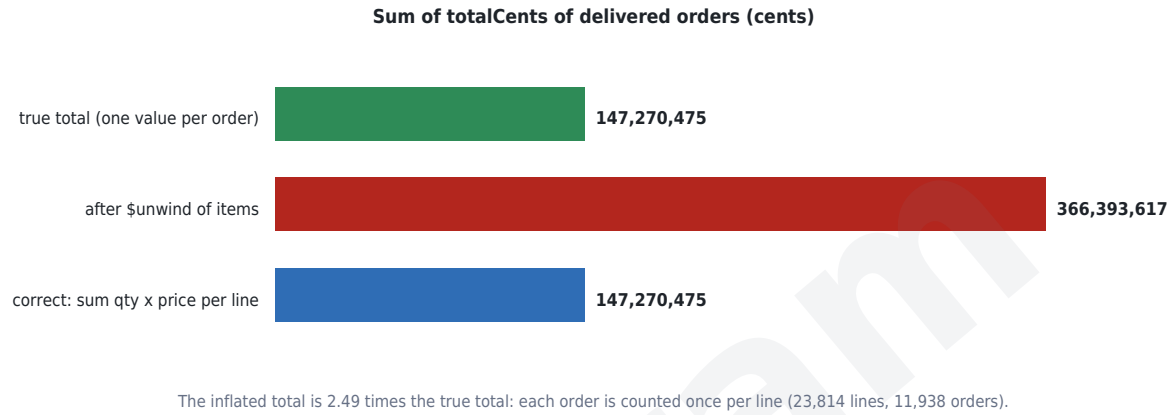


Figure 2.52. The unwound sum counts every order once per line; the correct line-level sum equals the true order-level total.

Root cause. `$unwind` repeats order-level fields on each line, the same fan-out as a SQL join between `orders` and `order_items`. PostgreSQL gave the identical wrong value (366,393,617) for the corresponding join, which confirms that the error is logical and not specific to MongoDB.

Fix. Sum the line-level quantity and price (`$multiply: ['$items.qty', '$items.priceCents']`), or aggregate order-level fields before the unwind, or use `$sum` over the array without unwinding (`{ $sum: { $sum: { $map: ... } } }`).

Prevention. State the grain of every aggregation (one document per order, or per line) and never sum a field of a coarser grain. Add a reconciliation test that compares the report total with the ledger.

Incident: the order list endpoint slows with every release because of $lookup

Scenario. The "my orders" endpoint became `orders.aggregate([{ $match: { customerId } }, { $lookup: { from: 'customers', ... } }, { $lookup: { from: 'products', ... } }])` to show names. It was fast in tests with 50 orders.

Symptoms. p99 latency of 2 seconds under production load; CPU high on the primary; `currentOp` shows the aggregation running with growing `docsExamined` on `customers` and `products`.

Diagnosis. (1) `db.orders.explain('executionStats').aggregate([...])` shows `$lookup` stages with `totalDocsExamined` far above the number of orders returned. (2) `db.customers.getIndexes()` shows that the foreign field used (`email`, not `_id`) has no index. (3) Arithmetic: 20,000 input documents times 2,000 customers is 40,000,000 comparisons without an index and 20,000 probes with one.

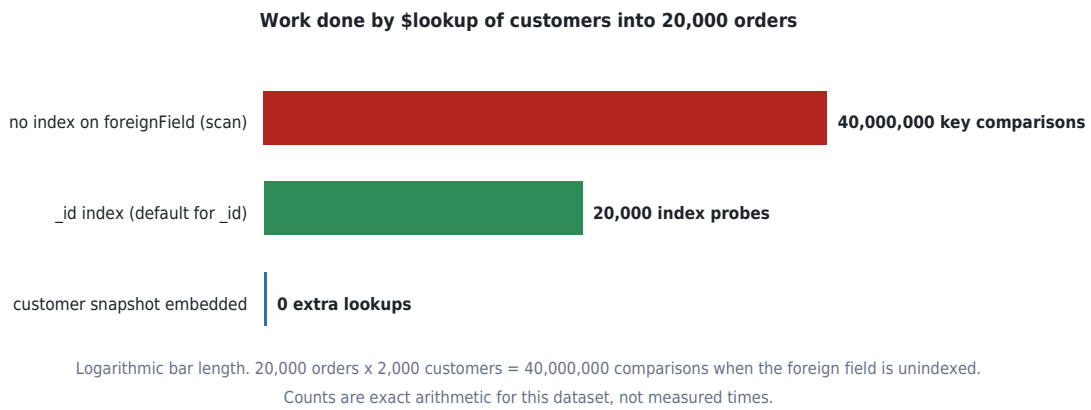


Figure 2.53. Work of the $lookup by access path for the sample sizes; the lengths are logarithmic.

Root cause. The equality `$lookup` performs one lookup in the foreign collection for every input document; without an index on the foreign field each lookup is a collection scan. Even with an index, a join per request multiplies the round-trips into storage on the hot path.

Fix. Immediate: create the index on the foreign field (`db.customers.createIndex({ email: 1 })`) and move the `$lookup` after a `$limit`. Permanent: apply the extended reference pattern, copying `name` and the line `name` into the order at write time, so that the hot read needs no join; keep `$lookup` for reports and rare screens.

Prevention. Treat a `$lookup` in a request path as a design smell to be justified by measurement; run `explain` with realistic data volumes; alert on `docsExamined / nReturned` ratios.

Incident: a nightly aggregation fails with a memory limit error

Scenario. A job groups 50 million events by user and collects the event list: `{ $group: { _id: '$userId', events: { $push: '$$ROOT' } } }`.

Symptoms. The job fails after several minutes with an error of this form (typical output; the wording differs by version):

Typical error

```
MongoServerError: PlanExecutor error during aggregation :: caused by :: Exceeded memory limit for
$group, but didn't allow external spilling; pass allowDiskUse:true to opt in
  code: 292, codeName: 'QueryExceededMemoryLimitNoDiskUseAllowed'
```

Diagnosis. (1) The error names the stage. (2) `explain('executionStats')` on a sample reports the stage; in versions that expose it, `usedDisk: true` indicates spilling. (3) `$push: '$$ROOT'` retains every document of a group, so memory grows with the data size and not only with the number of groups.

Root cause. A blocking `$group` accumulates state per group in RAM up to 100 MB per stage, and `$push` of whole documents makes that state as large as the input.

Fix. Immediate: add `{ allowDiskUse: true }` (default behaviour from MongoDB 6.0 when `allowDiskUseByDefault` is true) and run in a quiet window. Permanent: reduce the data entering the stage (`$match` on a date range, `$project` of the needed fields before the group), group in increments by day and combine with `$merge` into a summary collection, and avoid `$push` of whole documents in favour of counts and sums.

Prevention. Load-test pipelines with the full data volume, set `maxTimeMS` on batch aggregations, and monitor temporary-file usage under the `_tmp` directory of `dbPath`.

2.13.5 Questions to expect

1. **What is the aggregation pipeline?** An ordered list of stages in which each stage transforms the stream of documents produced by the previous one; the server executes it and can optimise the order.
2. **How is a SQL `GROUP BY ... HAVING` expressed?** `$group` with accumulators followed by a `$match` on the accumulated fields.
3. **What does `$unwind` do and what is its main hazard?** It emits one document per array element, repeating the other fields; summing a parent-level field afterwards double counts.

4. **How does `$lookup` behave and when is it expensive?** It is a left outer join returning matches as an array; it costs one foreign lookup per input document and is a scan without an index on the foreign field.
5. **Why put `$match` first?** Only a leading `$match` (and `$sort`) can use an index; it also reduces the documents processed by later stages.
6. **What limits apply to a pipeline stage?** 100 MB RAM for blocking stages (spilling to disk when permitted, default since 6.0), 16 MiB for any single output document including `$facet`, and 1,000 stages per pipeline.
7. **What is `$facet` for and what are its limits?** Several sub-pipelines over one input, returning one document; the document is limited to 16 MiB and the branches cannot use indexes.
8. **How is a result stored for reuse?** `$merge` (into a collection, with per-document merge behaviour) or `$out` (replaces the collection); this implements incrementally refreshed summaries.
9. **When should aggregation be used instead of the application?** When the data reduction is large (millions of documents to a few hundred), because only the result crosses the network.
10. **How is "top N per group" done?** `$sort` then `$group` with `$first`, or `$setWindowFields` with `$rank` and a `$match` on the rank (MongoDB 5.0 and later); `$topN` and `$bottomN` accumulators exist since 5.2.

KEY POINTS

The aggregation framework runs a pipeline of stages inside the server; most defects come from grain, join cost, and memory.

- Put `$match` first and `$project` only what later stages need; `$sort` with `$limit` becomes a top-k sort.
- `$unwind` repeats parent fields: summing them afterwards inflated revenue from 147,270,475 to 366,393,617 cents on the sample data.
- `$lookup` is a left outer join with one foreign lookup per input document; index the foreign field or embed a snapshot.
- Blocking stages are limited to 100 MB; spilling to disk is the default since MongoDB 6.0, but reduce the input first.
- All 15 SQL analytic queries of this chapter were reproduced as pipelines with identical results.

2.14 Indexes and explain in MongoDB

Without an index MongoDB reads every document of a collection to answer a query (a collection scan, `COLLSCAN`). An index is an ordered structure of keys, stored as a B-tree in WiredTiger, that maps field values to the location of documents so that a query reads only the relevant range. Indexes are the single most effective tuning tool and also the most common source of avoidable cost: each index is updated on every insert, delete, and relevant update, and it occupies memory. This chapter explains the index types, the ESR rule for compound indexes, and how to read the output of `explain('executionStats')`.

The quantitative statements use the 20,000-order sample dataset. The effect of index column order was reproduced for real with PostgreSQL 16 B-tree indexes with the same column orders (`EXPLAIN (ANALYZE, BUFFERS)`), and the key counts of the MongoDB model were computed with a reference index model in Python; mongod outputs are labelled typical.

2.14.1 Concept and mechanism

Sperran

The index types

Type	What it indexes	Typical use
`Single field`	One field, ascending or descending (direction matters only in compound sorts).	Equality and range on one field.
`Compound`	An ordered list of fields; usable for any left prefix.	Multi-field filters with sort (ESR rule).
`Multikey`	Created automatically when an indexed field holds an array: one entry per element.	Tags, `items.sku`.
`Text`	Tokenised, stemmed words of string fields; one text index per collection.	Basic `$text` search with relevance score.
`` `2dsphere` ``	GeoJSON points and shapes on a sphere.	`$near`, `$geoWithin` for stores within 5 km.
`Hashed`	Hash of one field.	Equality lookups; hashed shard keys; no range queries.
`TTL`	A single date field with `expireAfterSeconds`; a background task removes expired documents (it runs every 60 seconds).	Sessions, caches, temporary data.
`Partial`	Only documents that match a `partialFilterExpression`.	Index only `status: 'pending'` orders; smaller index.
`Sparse`	Only documents in which the field exists.	Optional fields; partial indexes are the more general tool.
`Unique`	Rejects a second document with the same key.	Natural keys, one cart per user.
`Wildcard`	All fields under a path (`attrs.$**`).	Attribute pattern with unpredictable field names.

Index types and their use.

A **compound index** `{ a: 1, b: -1 }` orders entries by `a` and, within equal `a`, by `b` descending. It supports queries on `a` and on `a` and `b` (the left prefix), and sorts on `a`, on `{ a: 1, b: -1 }`, and on the exact reverse `{ a: -1, b: 1 }`. It does not serve a query on `b` alone. A **multikey** index holds one entry per array element (the 20,000 sample orders produce 39,874 entries for `items.sku`); a compound index can contain at most one array field per document.

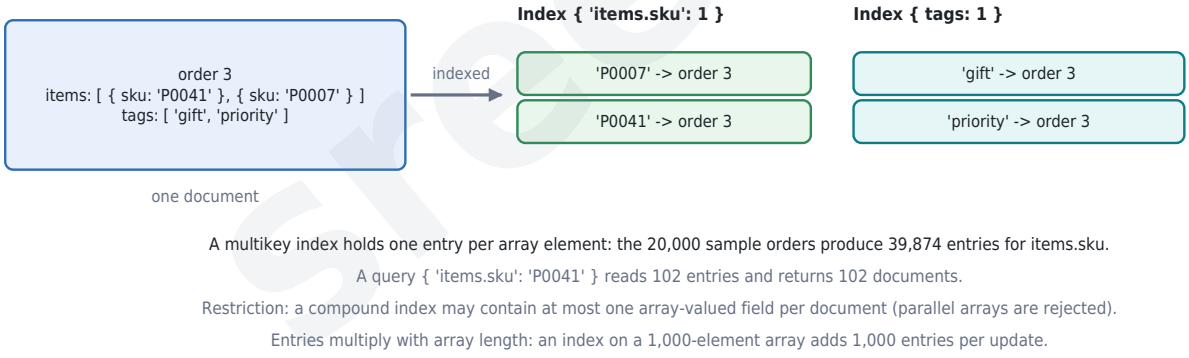


Figure 2.54. A multikey index stores one entry per array element; the index over `items.sku` is larger than the number of documents.

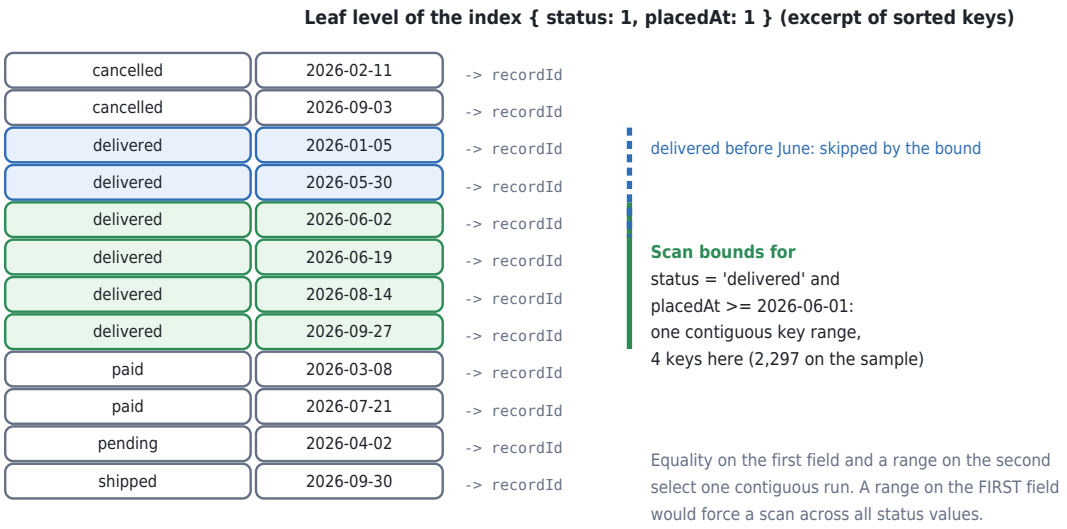


Figure 2.55. Sorted leaf keys of the index { status, placedAt }: an equality on the first field and a range on the second field define one contiguous range of keys.

The ESR guideline

Field order in a compound index determines how much of it a query can use. The documented guideline is **Equality, Sort, Range**: fields tested by equality first, then the fields used for sorting, then the fields tested by range or inequality (`$gt`, `$lt`, `$ne`, `$nin`, `$regex`). Equality fields narrow the scan to a contiguous run; sort fields placed next let the index deliver the documents already ordered, so that no blocking in-memory sort is needed; range fields last filter within that order. If the range predicate is very selective, putting it before the sort field (ERS) can examine fewer keys at the price of an in-memory sort of a small result. `$in` with up to 200 values behaves like equality with a merged sort; larger arrays behave like a range.

Covered queries

A query is **covered** when every field in the filter and in the projection is in the index and `_id` is excluded (or is in the index): the server answers from the index alone and `totalDocsExamined` is 0. A multikey index cannot cover a query that projects the array field.

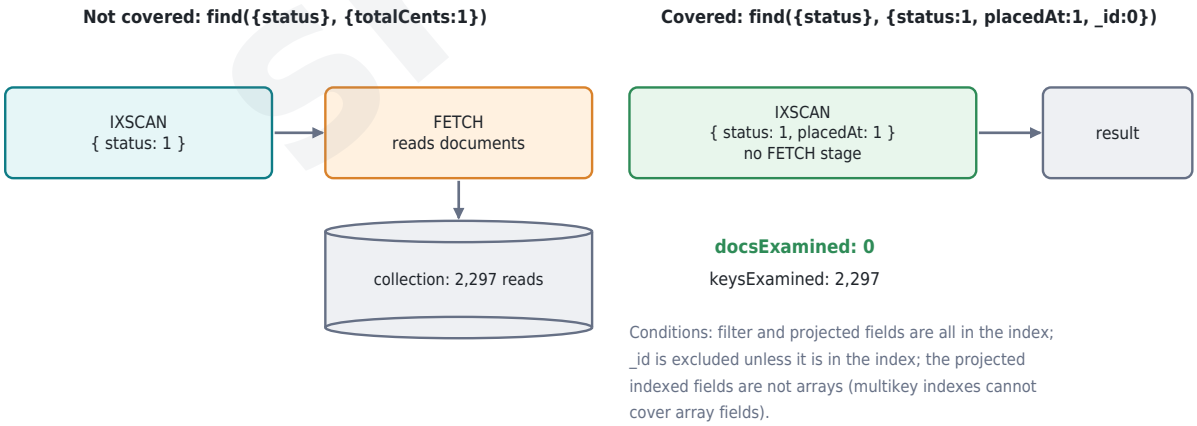


Figure 2.56. A normal query reads documents through a FETCH stage; a covered query ends at the IXSCAN stage and reads no documents.

Reading explain

`collection.find(...).explain('executionStats')` (or `aggregate(...)` with `explain`) runs the query and returns the plan and counters. The three verbosity levels are `queryPlanner` (plan only, no execution), `executionStats` (executes the winning plan and reports counters), and `allPlansExecution` (also reports partial counters for rejected plans). The important elements:

Field	Meaning
queryPlanner.winningPlan	Tree of stages: leaves read an index (IXSCAN) or the collection (COLLSCAN); parents FETCH, SORT, PROJECTION_*, LIMIT, SKIP.
indexBounds	The key ranges scanned in each indexed field, for example ["delivered","delivered"] or [MaxKey, MinKey].
executionStats.nReturned	Documents returned.
executionStats.totalKeysExamined	Index entries visited.
executionStats.totalDocsExamined	Documents read, including those later discarded by a filter or by skip.
executionStats.executionTimeMillis	Time for plan selection and execution, excluding network transfer.
rejectedPlans	Candidate plans that lost the trial run.

Fields of explain('executionStats').

A SORT stage in the plan means a blocking in-memory sort (limited to 100 MB unless allowDiskUse); its absence means the index supplied the order. The efficiency indicators are the ratios nReturned / totalDocsExamined and nReturned / totalKeysExamined: values near 1 mean that the query reads little more than it returns. Version note: the slot-based execution engine reports a different plan tree (winningPlan.queryPlan) from the classic engine, and field names such as queryHash were renamed in 8.0; the stage names IXSCAN, FETCH, SORT, COLLSCAN remain.

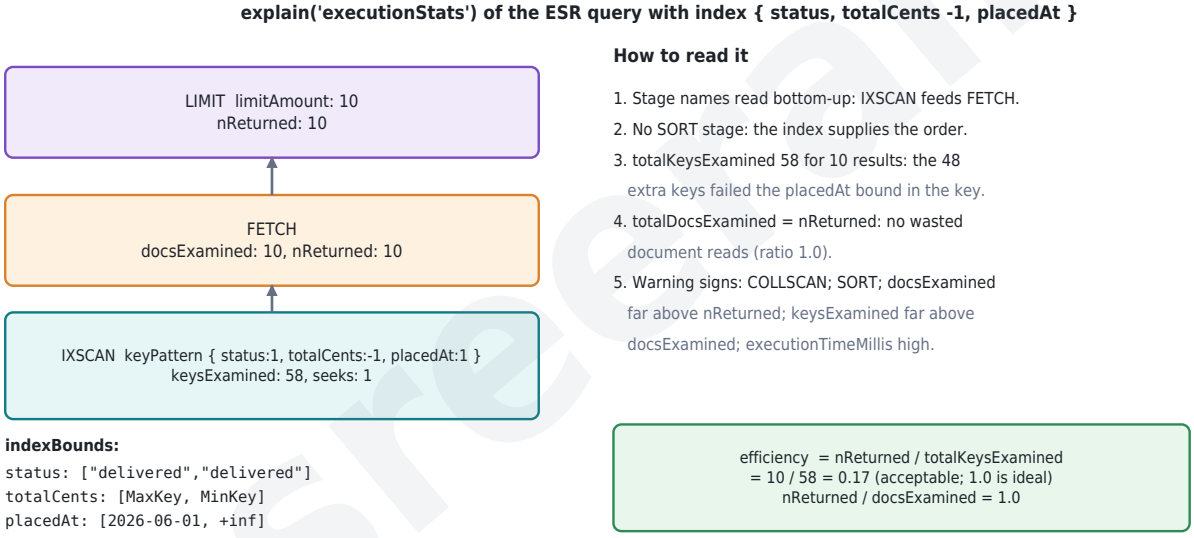


Figure 2.57. A plan for the ESR query: IXSCAN with its bounds feeds FETCH and LIMIT; there is no SORT stage and docsExamined equals nReturned.

2.14.2 Worked example: four orders of the same three fields

Query: delivered orders from 1 June 2026 onwards, the ten with the highest totalCents.

The query JAVASCRIPT

```
db.orders.find({ status: 'delivered', placedAt: { $gte: ISODate('2026-06-01') } })
         .sort({ totalCents: -1 })
         .limit(10)
```

The data: 20,000 orders; 11,938 delivered; 2,297 delivered since 1 June; 3,863 orders of any status since 1 June. Four candidate indexes were evaluated, with the counts computed by the reference index model and the buffer counts measured on PostgreSQL 16 with B-tree indexes of the same column order.

Index	keys examined	docs examined	sort	PostgreSQL buffers
none (collection scan)	20,000	20,000	in memory	211

Index	keys examined	docs examined	sort	PostgreSQL buffers
`{ status }`	11,938	11,938	in memory	211 (seq scan chosen)
`{ status, placedAt, totalCents }` (E, R, S)	2,297	2,297	in memory	217
`{ placedAt, status, totalCents }` (R, E, S)	3,863	2,297	in memory	222
`{ status, totalCents: -1, placedAt }` (E, S, R)	58	10	none	12

Effect of index column order on the same query (20,000 orders).

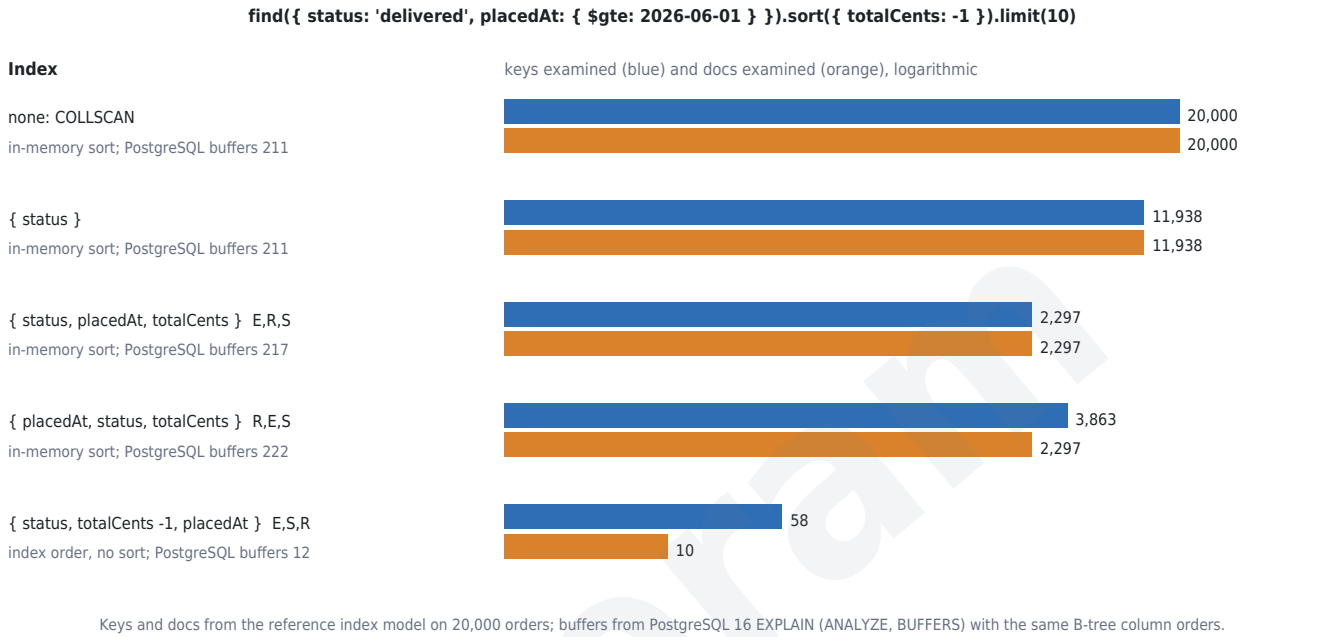


Figure 2.58. The same query under five index choices. Only the E, S, R order yields the top ten without sorting: it walks the status = 'delivered' keys in descending totalCents order and stops after 10 matches, examining 58 keys.

Explanation of the last row. Entries for `status = 'delivered'` are ordered by `totalCents` descending, and within equal totals by `placedAt`. The scan starts at the highest total and checks the `placedAt` bound inside each key without touching the document; 48 of the first 58 keys fail the date bound (the highest totals include many old orders), 10 pass, and the `LIMIT` stage stops the scan. In the E, R, S order the index contains all 2,297 qualifying keys in `placedAt` order, so all 2,297 documents must be fetched and sorted to find the ten largest; the R, E, S order is worse because the range is the first field and the keys of other statuses inside the date range (3,863 in total) have to be traversed.

PostgreSQL showed the same behaviour, with the characteristic difference that its planner may choose a bitmap scan. The two plans that matter, trimmed:

PostgreSQL 16 plan (run for real), index (status, placed_at, total_cents)

```
Limit (actual rows=10 loops=1)
  Buffers: shared hit=201 read=16
  ->  Sort (actual rows=10 loops=1)
        Sort Key: total_cents DESC
        Sort Method: top-N heapsort  Memory: 25kB
        ->  Bitmap Heap Scan on orders (actual rows=2297 loops=1)
              Heap Blocks: exact=198
              ->  Bitmap Index Scan on ix_t (actual rows=2297 loops=1)
                    Index Cond: ((status = 'delivered') AND (placed_at >= '2026-06-01'))
```

PostgreSQL 16 plan (run for real), index (status, total_cents DESC, placed_at)

```
Limit (actual rows=10 loops=1)
  Buffers: shared hit=10 read=2
  ->  Index Scan using ix_t on orders (actual rows=10 loops=1)
        Index Cond: ((status = 'delivered') AND (placed_at >= '2026-06-01'))
```

The first plan reads 217 buffers and sorts 2,297 rows to return 10; the second reads 12 buffers, has no sort node, and stops after 10 rows. The mechanism is identical to the one that MongoDB's ESR guideline describes.

2.14.3 Hands-on

mongosh: creating indexes and running explain (typical output, not run on the build machine)

```
shop> db.orders.createIndex({ status: 1, totalCents: -1, placedAt: 1 })
status_1_totalCents_-1_placedAt_1
shop> db.orders.find({ status: 'delivered', placedAt: { $gte: ISODate('2026-06-01') } })
...   .sort({ totalCents: -1 }).limit(10).explain('executionStats')
{
  queryPlanner: {
    namespace: 'shop.orders',
    winningPlan: {
      stage: 'LIMIT', limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { status: 1, totalCents: -1, placedAt: 1 },
          indexName: 'status_1_totalCents_-1_placedAt_1',
          isMultiKey: false, direction: 'forward',
          indexBounds: {
            status: [ '["delivered", "delivered"]' ],
            totalCents: [ '[MaxKey, MinKey]' ],
            placedAt: [ '[new Date(1780272000000), new Date(9223372036854775807)]' ]
          } } } },
    rejectedPlans: [ ... ]
  },
  executionStats: {
    executionSuccess: true, nReturned: 10, executionTimeMillis: 1,
    totalKeysExamined: 58, totalDocsExamined: 10
  }
}
```

The counters in this listing are the values produced by the reference index model (58 keys, 10 documents); the timing is illustrative. `rejectedPlans` is abbreviated; with several candidate indexes it lists the plans that lost the plan-selection trial.

Other index definitions used in this Part JAVASCRIPT

```
db.orders.createIndex({ customerId: 1, placedAt: -1 })            // customer's latest orders
db.orders.createIndex({ 'items.sku': 1 })                           // multikey
db.orders.createIndex({ status: 1, placedAt: 1 },
                      { partialFilterExpression: { status: { $in: ['pending', 'paid'] } } })
db.sessions.createIndex({ lastSeen: 1 }, { expireAfterSeconds: 3600 })  // TTL
db.carts.createIndex({ userId: 1 }, { unique: true })                // one cart per user
db.stores.createIndex({ location: '2dsphere' })
db.products.createIndex({ name: 'text', description: 'text' })
db.orders.createIndex({ status: 1, totalCents: -1 }, { hidden: true }) // test removal safely
```

A **hidden** index (MongoDB 4.4 and later) is maintained but ignored by the planner, so that the effect of dropping it can be tested and reversed instantly. `db.orders.aggregate([{ $indexStats: {} }])` reports for every index how many times it was used since the last restart, the basis for finding unused indexes.

Indexes from Java and Spring Data (not run: jars not available on the build machine)

IndexSetup.java JAVA

```java
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.model.IndexOptions;
import com.mongodb.client.model.Indexes;
import org.bson.Document;

public class IndexSetup {
    public static void main(String[] args) {
        try (MongoClient client = MongoClients.create("mongodb://127.0.0.1:27017")) {
            MongoCollection<Document> orders = client.getDatabase("shop").getCollection("orders");
            orders.createIndex(Indexes.compoundIndex(
                Indexes.ascending("status"),
                Indexes.descending("totalCents"),
                Indexes.ascending("placedAt")));
            client.getDatabase("shop").getCollection("carts")
                .createIndex(Indexes.ascending("userId"), new IndexOptions().unique(true));
        }
    }
}
```

In Spring Data MongoDB the annotations `@Indexed`, `@CompoundIndex(def = "{'status': 1, 'totalCents': -1, 'placedAt': 1}")`, and `@TextIndexed` on the `@Document` class declare the same indexes. Automatic index creation from annotations is disabled by default in current Spring Data MongoDB versions (property `spring.data.mongodb.auto-index-creation`); production indexes are best created by a migration script or deployment step, not at application start.

2.14.4 Real-world problems and solutions

Each incident below starts from the decision path used to diagnose slow reads: capture the slow operation, run `explain('executionStats')`, and compare the plan stages and counters with the result size.

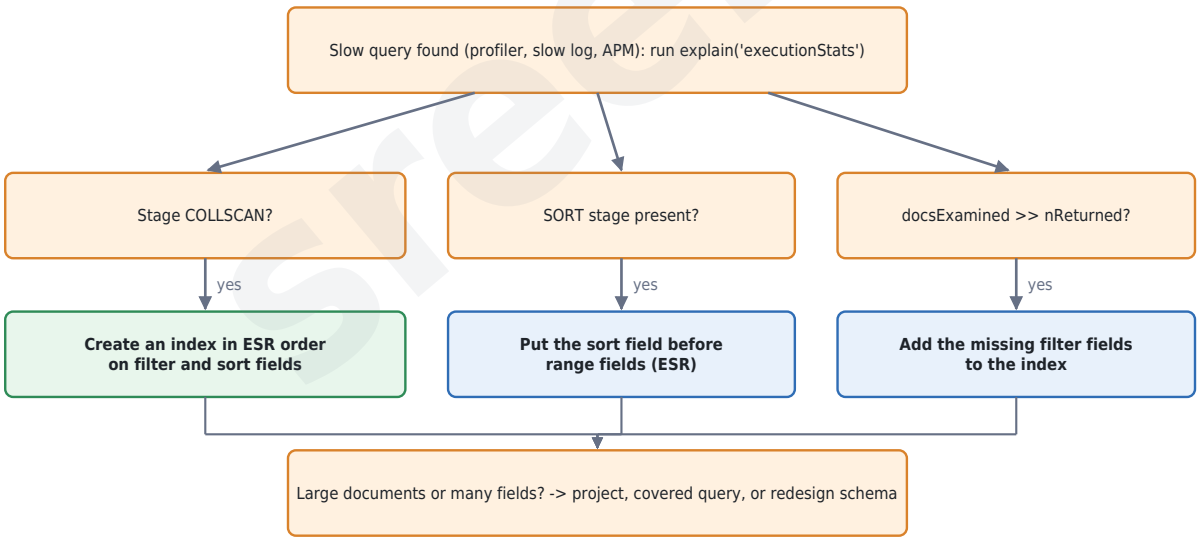


Figure 2.59. Diagnosis path for a slow query: look for COLLSCAN, for a SORT stage, and for docsExamined far above nReturned; each answer points to a specific index change.

Incident: a hot query performs a collection scan

Scenario. The "my orders" page runs `db.orders.find({ customerId: 383 }).sort({ placedAt: -1 }).limit(20)`. The collection grew from 20,000 to several million documents.

Symptoms. p99 latency rises with the data volume; CPU on the primary is high; the profiler lists the same query shape with durations in the hundreds of milliseconds.

Diagnosis. In order:

Diagnosis steps — JAVASCRIPT

```
db.setProfilingLevel(1, { slowms: 100 })            // record operations slower than 100 ms
db.system.profile.find({ ns: 'shop.orders' }).sort({ ts: -1 }).limit(3)
db.orders.find({ customerId: 383 }).sort({ placedAt: -1 }).limit(20).explain('executionStats')
// plan: SORT <- COLLSCAN; totalDocsExamined: 20000 (the whole collection), nReturned: 20
db.orders.getIndexes()                                // only _id_
```

On the sample dataset the plan examines 20,000 documents to return 20.

Root cause. No index supports the filter `customerId` and the sort `placedAt`, so every execution scans the collection and sorts the matches in memory; the cost grows linearly with the collection.

Fix. Immediate: `db.orders.createIndex({ customerId: 1, placedAt: -1 })`. Index builds are online since MongoDB 4.2 (they block only briefly at start and end) but consume I/O and cache, so run them off-peak. After the build the plan is `LIMIT <- FETCH <- IXSCAN` with `totalKeysExamined: 20`, `totalDocsExamined: 20`, and no `SORT` (equality on `customerId`, then the sort field: the ESR rule). Permanent: keep the index definitions in version-controlled migration scripts.

Prevention. Run `explain` for every new query shape against production-sized data in a staging environment, assert in tests that the winning plan contains `IXSCAN`, and review `$indexStats` for unused and missing indexes.

Incident: a compound index exists but the query is still slow

Scenario. A dashboard query `find({ status: 'delivered', placedAt: { $gte: D } }).sort({ totalCents: -1 }).limit(10)` has the index `{ placedAt: 1, status: 1, totalCents: 1 }`, created by a developer who ordered the fields by how they are written in the code.

Symptoms. An index is used (`IXSCAN` in the plan) yet the query is slow, and the plan contains a `SORT` stage; `totalKeysExamined` is 3,863 and `totalDocsExamined` is 2,297 on the sample for 10 returned documents.

Diagnosis. `explain('executionStats')` shows a blocking `SORT` and counters far above `nReturned`. Classifying the fields: `status` is equality, `totalCents` is the sort, `placedAt` is a range; the index has the range first.

Root cause. The index violates ESR: the range on the first field forces a traversal of all keys in the date range for every status, and the sort on `totalCents` cannot use the index order, so all qualifying documents are fetched and sorted.

Fix. Create `{ status: 1, totalCents: -1, placedAt: 1 }`. The plan then has no `SORT`, examines 58 keys and 10 documents on the sample (the PostgreSQL B-tree with the same order reads 12 buffers instead of 222). Validate with `.hint('status_1_totalCents_-1_placedAt_1')`, hide the old index (`db.orders.hideIndex(...)`), watch the metrics for a week, and then drop it.

Prevention. Name the fields by role (equality, sort, range) when designing an index, and check the explain counters. Remember that the best order depends on selectivity: if the date range typically selects fewer than a few hundred documents, the E, R, S order with an in-memory sort of those few documents can win (the ERS exception).

Incident: deep pages of a list become progressively slower

Scenario. An export page uses `find({ status: 'delivered' }).sort({ placedAt: -1 }).skip(page * 20).limit(20)`. An automated client walks through all pages.

Symptoms. Page 1 returns in a few milliseconds; page 500 takes seconds; total time for the walk grows quadratically; the profiler shows `docsExamined` equal to the skip plus 20.

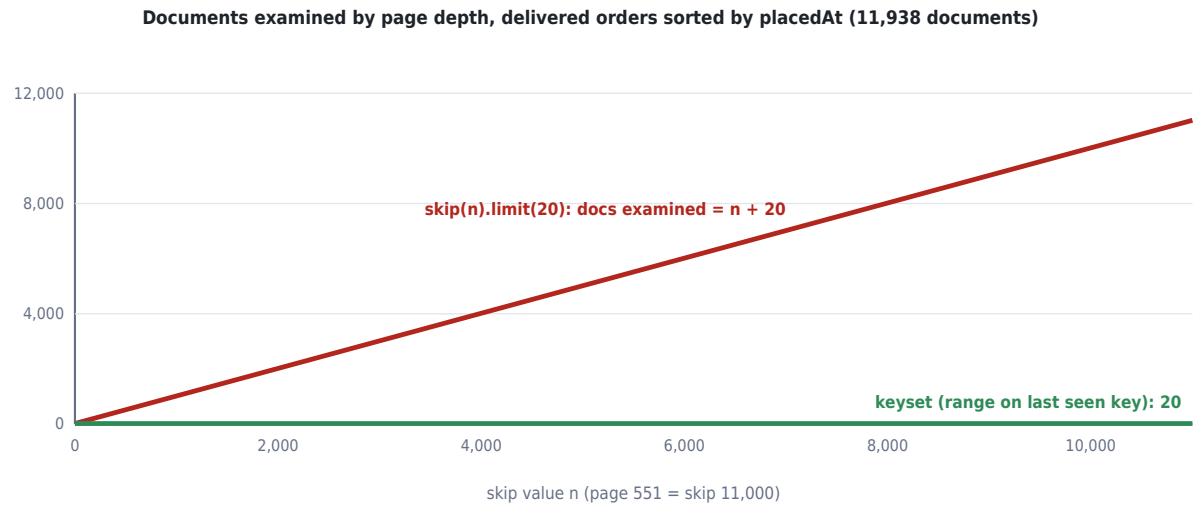


Figure 2.60. Documents examined by `skip(n).limit(20)` grow linearly with n (11,020 at page 551 of 11,938 delivered orders) while keyset pagination examines 20 at every depth.

Diagnosis. `explain` of the page-551 query shows `IXSCAN` with `totalKeysExamined: 11020`, `totalDocsExamined: 11020`, `nReturned: 20`, and a `SKIP` stage above `FETCH`. The numbers equal `skip + limit`.

Root cause. `skip` cannot jump: the server walks and fetches the skipped documents and then discards them. Unstable ordering adds a second defect: documents with equal `placedAt` may appear on two pages or on none unless the sort has a unique tiebreaker.

Fix. Keyset (seek) pagination with a unique sort key: sort by `{ placedAt: -1, _id: -1 }`, remember the last pair, and query the next page with a range on it.

Keyset pagination JAVASCRIPT

```
db.orders.createIndex({ status: 1, placedAt: -1, _id: -1 })
db.orders.find({
  status: 'delivered',
  $or: [ { placedAt: { $lt: lastPlacedAt } },
         { placedAt: lastPlacedAt, _id: { $lt: lastId } } ]
}).sort({ placedAt: -1, _id: -1 }).limit(20)
// every page: totalKeysExamined about 21, totalDocsExamined 20
```

Spring Data MongoDB 4.1 (the 2023.0 release train) and later offers keyset scrolling through `ScrollPosition.keyset()` and `Window<T>` return types on repository methods (not run on the build machine). When random page access (jump to page 500) is a requirement, bound the maximum page, or offer search filters instead.

Prevention. Reject `skip` values above a threshold in the API, include `_id` in every paginated sort, and use `countDocuments` sparingly on large collections.

Incident: customer search by email fragment times out

Scenario. Support searches `find({ email: /19@example/ })` and `find({ email: /^USER19/i })` on 2,000,000 customers; an index on `email` exists.

Symptoms. Both searches are slow although `IXSCAN` appears in the plan; `totalKeysExamined` equals the number of index entries.

Diagnosis. `explain` shows `indexBounds: { email: [ '[ "", {} ]', '[ /19@example/, /19@example/ ]' ] }`: the bounds span the whole index. On the sample data (2,000 customers) the unanchored search `/19/` examines all 2,000 keys for 139 results, and `/^USER19/i` also scans all 2,000 keys; the anchored case-sensitive `/^user19/` examines 111 keys for 111 results.

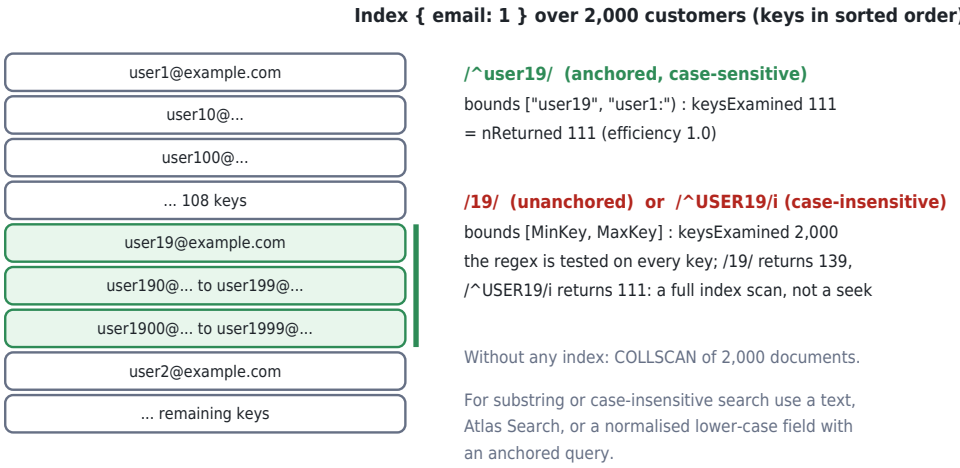


Figure 2.61. Key ranges for an anchored prefix regular expression (tight bounds, 111 keys) and for an unanchored or case-insensitive expression (whole index, 2,000 keys).

Root cause. Only a case-sensitive expression anchored at the start (`^prefix`) converts into a key range. All other regular expressions are evaluated against every key (or every document).

Fix. Store a normalised lower-case field (`emailLower`), index it, and query it with an anchored expression; or create the index with a case-insensitive collation (`{ locale: 'en', strength: 2 }`) and use equality queries with the same collation; or use a text index (whole words) or Atlas Search for substring and fuzzy needs.

Prevention. Ban unanchored `$regex` from hot paths in review, and test search queries with `explain` at production cardinality.

Incident: duplicate documents despite "unique" business rules

Scenario. The cart service runs `updateOne({ userId }, { $setOnInsert: { createdAt: now }, $push: { items: item } }, { upsert: true })`. A double-click or two browser tabs send two requests at once.

Symptoms. Some users have two carts; reads with `findOne({ userId })` return either one, so items "disappear" intermittently.

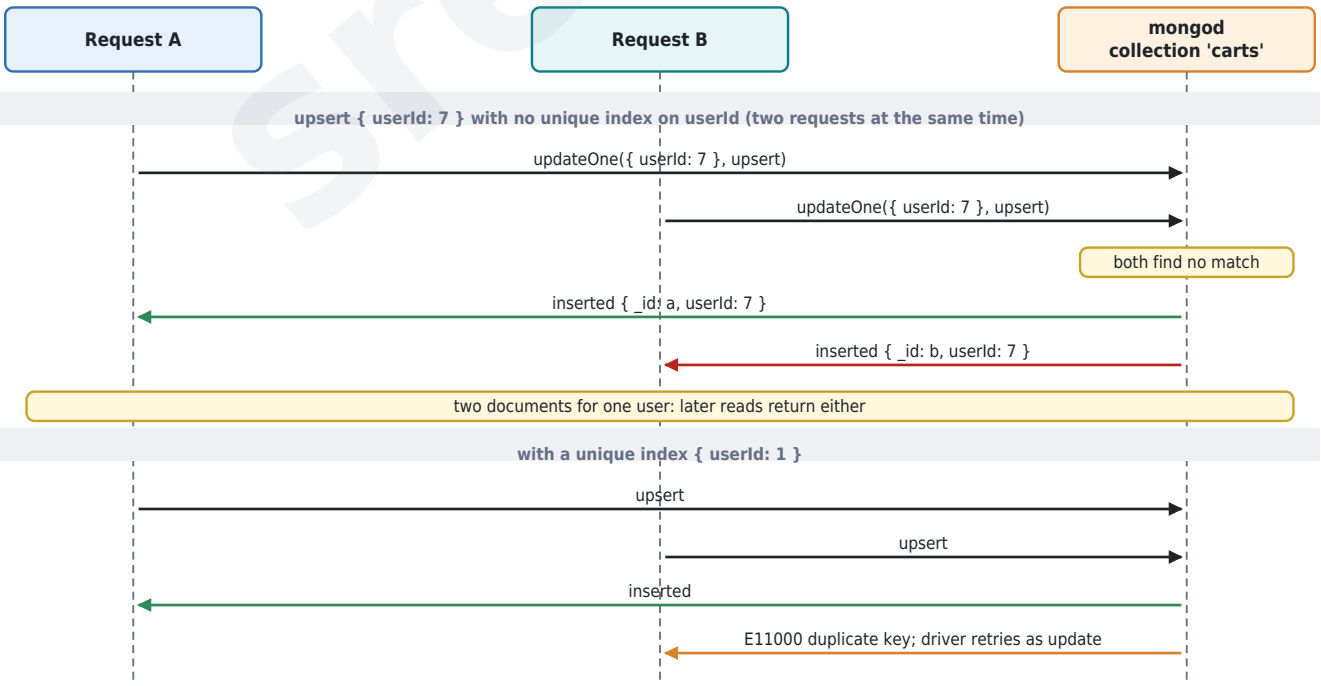


Figure 2.62. Two concurrent upserts without a unique index both find no document and both insert; with a unique index the second insert fails with E11000 and the driver can retry as an update.

Diagnosis. Find duplicates and measure them:

Finding duplicate keys JAVASCRIPT

```
db.carts.aggregate([
  { $group: { _id: '$userId', n: { $sum: 1 }, ids: { $push: '$_id' } } },
  { $match: { n: { $gt: 1 } } },
  { $sort: { n: -1 } }, { $limit: 20 }
])
db.carts.getIndexes()      // no unique index on userId
```

Root cause. An upsert is atomic per document but not across concurrent operations on a filter that matches nothing: both operations see no match and insert. Only a unique index serialises the decision. The same defect occurs with a check-then-insert sequence in application code.

Fix. Immediate: merge the duplicate carts (keep the newest, `$push` the items of the others with `$each`, delete the rest) in a maintenance script. Then create the constraint: `db.carts.createIndex({ userId: 1 }, { unique: true })`, which fails with `E11000 duplicate key error` while duplicates still exist. In the application, treat error code 11000 from an upsert as "retry once as an update"; since MongoDB 4.2 the server itself retries such an upsert once under documented conditions (the filter matches the unique index key exactly), which narrows but does not remove the need for the application-side handling.

Prevention. Every business key that must be unique has a unique index (use a partial unique index when uniqueness applies only to a subset, for example `{ partialFilterExpression: { deletedAt: { $exists: false } } }`). A collection without such an index plus a read-before-write is a defect by design.

2.14.5 Questions to expect

1. **What is the ESR rule?** Order compound index fields as Equality, Sort, Range: equality narrows to a contiguous run, the sort field provides order without a blocking sort, and the range filters within that order. The exception ERS applies when the range is very selective.
2. **How is explain read?** Read the stage tree bottom-up: `COLLSCAN` or `IXSCAN`, then `FETCH`, `SORT`, `LIMIT`. Compare `nReturned` with `totalKeysExamined` and `totalDocsExamined`; a `SORT` stage means an in-memory sort.
3. **What is a covered query?** One answered entirely from an index: filter and projection fields are in the index, `_id` is excluded or indexed, and `totalDocsExamined` is 0.
4. **What is a multikey index and what are its limits?** An index over an array field that stores an entry per element; a compound index may include at most one array field per document, and large arrays multiply the entries and the write cost.
5. **When does a regular expression use an index efficiently?** Only when it is case-sensitive and anchored at the start; the index range is the prefix. Otherwise the whole index (or collection) is scanned.
6. **How do partial, sparse, and TTL indexes differ?** Partial indexes only documents matching a filter; sparse only those with the field; TTL deletes documents after a period via a date field (checked about every 60 seconds).
7. **Why is `skip` slow for deep pages and what replaces it?** The server must read and discard the skipped documents; keyset pagination with a range on the sort key (plus `_id` as tiebreaker) examines a constant number.
8. **What does a unique index guarantee, and why is an upsert unsafe without it?** The unique index rejects the second insert of a key atomically; without it concurrent upserts both insert.
9. **What does an index cost?** Memory and disk space, and extra work on every write that touches an indexed field; at most 64 indexes per collection are allowed, and unused indexes should be found with `$indexStats` and removed.
10. **How can an index be tested before dropping?** Hide it with `hideIndex`; the planner ignores it but it is still maintained, and `unhideIndex` restores it instantly.

KEY POINTS

Indexes turn a scan of the collection into a seek and a short range read; their design is governed by ESR and verified with explain.

- Order compound index fields Equality, Sort, Range; on the sample data the right order examined 58 keys where others examined thousands and avoided the in-memory sort.
- Read explain bottom-up and compare `nReturned` with `totalKeysExamined` and `totalDocsExamined`; `COLLSCAN` and `SORT` are the warning signs.
- Anchored case-sensitive regular expressions use index ranges; unanchored and case-insensitive ones scan every key.
- Replace deep `skip` with keyset pagination and a unique tiebreaker; enforce business uniqueness with unique indexes, never with read-then-write.
- Every index costs writes and memory: measure with `$indexStats`, test removal with hidden indexes.

2.15 MongoDB Transactions and Consistency

A MongoDB deployment is almost always a replica set or a sharded cluster, so every write has two questions attached to it: how much of the write is atomic, and how far the write has travelled when the client is told that it succeeded. The first question is answered by the transaction model, the second by write concern, read concern and read preference. A Java developer who treats these settings as defaults will eventually lose an acknowledged write during a failover, read stale data after an update, or watch a transaction abort under load. This chapter states the guarantees precisely and ties each setting to the failure that it prevents. Behaviour described here is documented for MongoDB 7.x and 8.x; no MongoDB server is available on the build machine, so every mongosh output in this chapter is labelled typical output. Experiments that could be run (snapshot conflict behaviour on PostgreSQL, a failover simulation) are marked as such.

2.15.1 Concept and mechanism

Single-document atomicity

A write to one document is atomic, including every embedded document and array element inside it. A concurrent reader observes either the state before the write or the state after it, never a mixture. This guarantee is free: it needs no session, no transaction and no extra round trip. It is the central reason why MongoDB data models embed data that changes together. A statement such as `updateMany` is not atomic as a whole: each matching document is updated atomically, but another client can observe some documents updated and others not, and a failure part-way leaves a partial result.

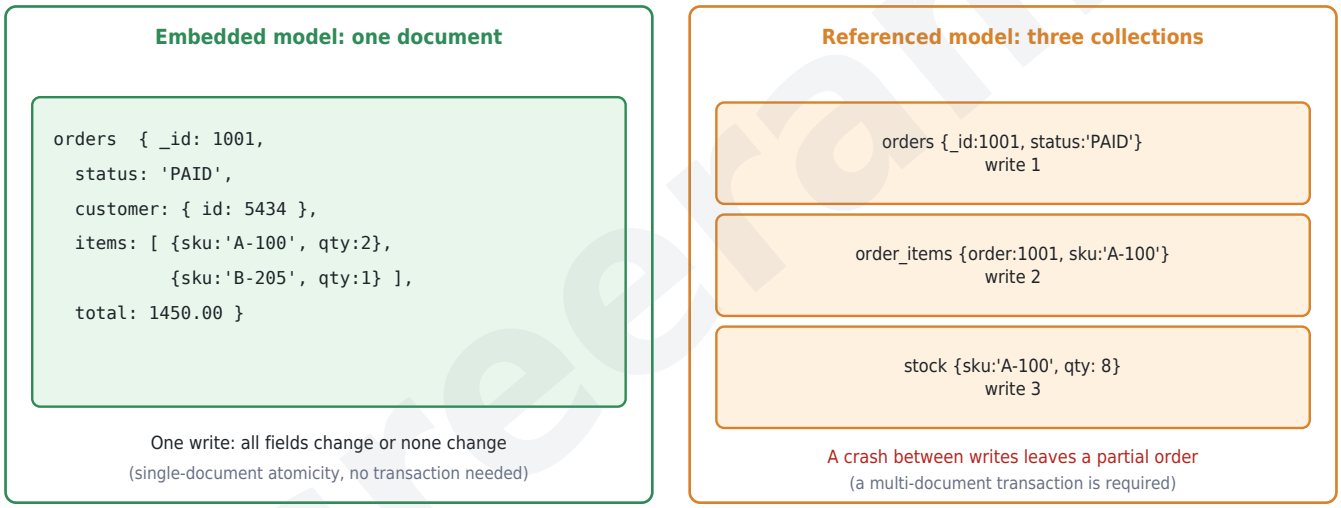


Figure 2.63. An embedded order changes in one atomic write; the referenced model needs three writes and therefore a multi-document transaction to prevent a partial order.

Multi-document transactions

Multi-document transactions are available on replica sets since MongoDB 4.0 and on sharded clusters since 4.2. A transaction runs inside a client session and provides ACID semantics across documents, collections, databases and, on a sharded cluster, shards. The contract has four parts.

- **Atomicity.** All writes of the transaction become visible together at commit, or none becomes visible after an abort.
- **Isolation.** Reads inside the transaction use the snapshot read concern: a consistent snapshot taken at the start of the transaction. Writes of other clients committed after that point are not visible to it. This is snapshot isolation, comparable to PostgreSQL `REPEATABLE READ`.
- **Conflict handling.** The first writer to modify a document wins. If a second transaction (or a non-transactional write) attempts to modify a document that the first has modified, the second fails immediately with a `WriteConflict` error carrying the label `TransientTransactionError`. MongoDB does not queue the second writer behind the first the way PostgreSQL row locks do for plain `UPDATE` statements.
- **Durability.** The transaction commits with the write concern given to the transaction, and `w: "majority"` is the correct choice for data that must survive a failover.

Snapshot isolation is not serializability. Two concurrent transactions that read overlapping data and write different documents can both commit, which is the write skew anomaly described in the transactions part of this book. The

remedy is the same as in a relational database: make both transactions write to a common guard document so that the conflict detector sees the overlap.

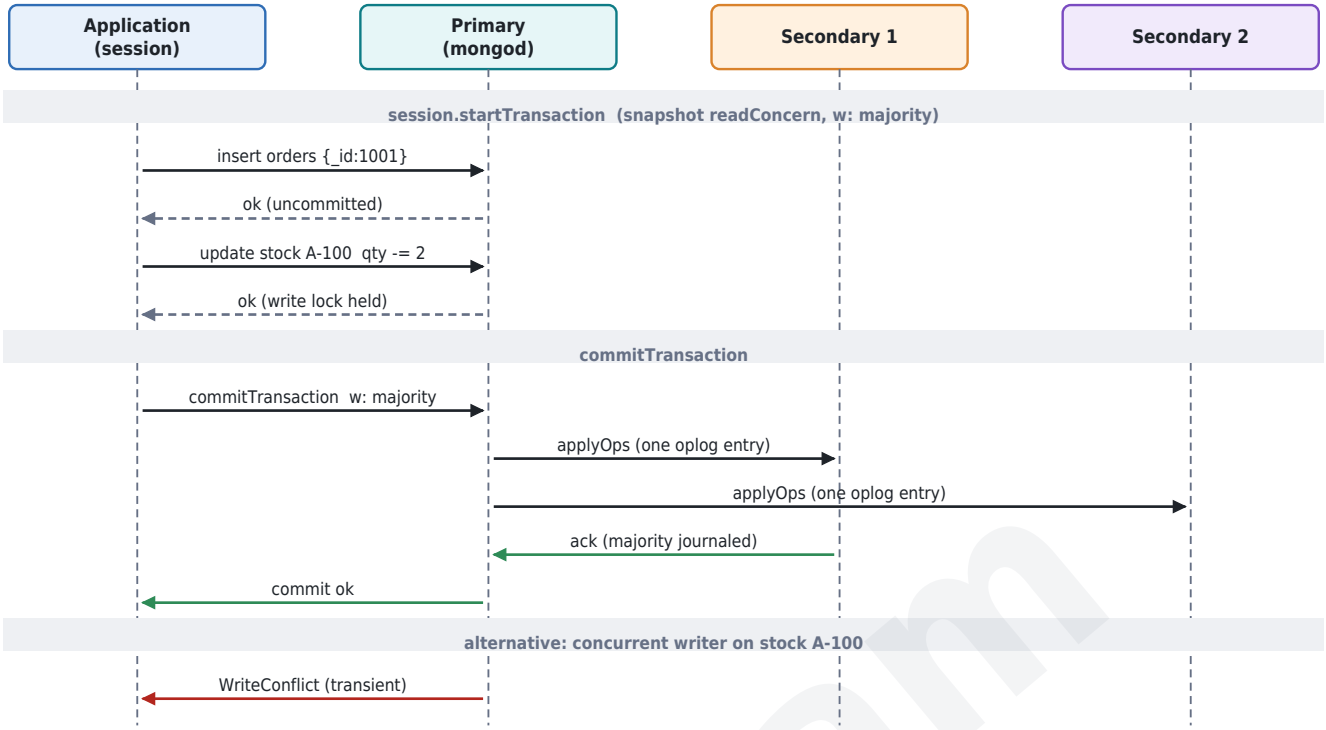


Figure 2.64. A transaction sends every operation with the same session and transaction number, holds document write locks until commit, and commits as one majority-acknowledged oplog entry; a concurrent writer on the same document receives a transient conflict.

Limits and costs

Transactions are deliberately bounded. The documented defaults for MongoDB 7.x and 8.x are listed below.

Parameter	Default	Effect
`transactionLifetimeLimitSeconds`	`60`	A transaction older than this is aborted by a periodic cleanup.
`maxTransactionLockRequestTimeoutMillis`	`5`	A transaction waits at most 5 ms for a lock before aborting.
`Oplog entry size`	`16 MB per entry`	A large transaction is split over several oplog entries; each entry must fit the BSON limit.
`Read preference`	`primary`	A transaction that reads must use the primary and one member throughout.
`Cache pressure`	`n/a`	A transaction that dirties too much cache aborts (write conflict or TransactionTooLargeForCache).

Operational limits of multi-document transactions.

Two practical consequences follow. A transaction should touch tens of documents, not millions, and it should never wait for a user or an external service. A bulk migration is written as many small independent batches, each idempotent, rather than one large transaction. An in-flight transaction also blocks DDL on the collections that it touches, and a pending DDL operation in turn makes new transactions on those collections wait, so index builds and `drop` commands belong outside peak hours.

Write concern

Write concern defines when the server acknowledges a write. It has three fields: `w` (how many members must have applied the write), `j` (whether the members counted by `w` must have flushed the journal) and `wtimeout` (how long the primary waits for `w` before reporting a write concern error, which does not undo the write).

Setting	Acknowledged when	Survives
w: 0	never acknowledged	nothing; errors are not reported

Setting	Acknowledged when	Survives
w: 1	primary has applied the write in memory	process stays up; lost on primary crash or failover
w: 1, j: true	primary has flushed its journal	primary crash and restart; still lost if the primary is replaced
w: majority	a majority of voting members have durably written the oplog entry	loss of the primary and an election
w: 3 (a number)	that many members have applied the write	depends on which members; blocks if fewer are up

Write concern values and what an acknowledgement means.

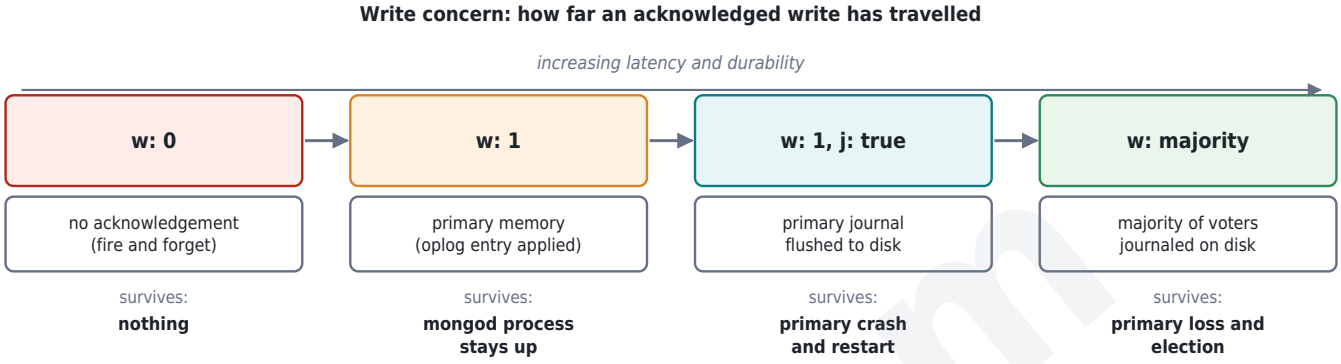


Figure 2.65. Each step to the right adds latency and a stronger durability guarantee; only majority acknowledgement survives the loss of the primary.

Since MongoDB 5.0 the implicit default write concern is `w: "majority"` for most deployments. The exception concerns arbiters: when the replica set has an arbiter and the number of data-bearing voting members does not exceed the voting majority (for example primary, one secondary and one arbiter), the implicit default is `w: 1`. With `w: "majority"` the journal flag defaults to true through `writeConcernMajorityJournalDefault`, so an acknowledged majority write is on disk on a majority of members. In MongoDB 8.0 and later a majority acknowledgement is sent once a majority has durably written the oplog entry; members apply the change asynchronously, so a secondary read immediately afterwards can still miss the write unless a causally consistent session is used.

GOTCHA

A topology of one primary, one secondary and one arbiter has two data-bearing members. A majority write needs both data-bearing members; when either is down, every `w: "majority"` write blocks (or times out with `wtimeout`). Production replica sets use three data-bearing members instead of an arbiter.

Read concern and read preference

Read concern controls which version of the data a read may return; read preference controls which member serves the read.

Level	Meaning
local	Returns the member's latest data; may later be rolled back after a failover. Default for reads outside transactions.
available	Like local; on a sharded cluster it may return orphaned documents. Lowest latency.
majority	Returns only data acknowledged by a majority; it cannot be rolled back.
linearizable	Single-document reads on the primary that reflect all majority-acknowledged writes completed before the read started; slow, requires a majority write concern for the writes.
snapshot	A consistent point-in-time view; used inside multi-document transactions.

Read concern levels.

Read preference modes are `primary` (default), `primaryPreferred`, `secondary`, `secondaryPreferred` and `nearest`. Every mode except `primary` can return data that is behind the primary, because replication is asynchronous. The option `maxStalenessSeconds` (minimum 90 seconds) excludes secondaries whose estimated lag exceeds the given value, and

tag sets restrict reads to members with given tags, for example one data centre. A replica set member that is briefly isolated can serve stale reads even with `primary` mode, until it steps down.

Causal consistency

A causally consistent session guarantees read your writes, monotonic reads, monotonic writes and writes follow reads, even when reads and writes go to different members. The driver records the `operationTime` of each write and sends it as `afterClusterTime` with later reads; the serving member waits until its data has reached that time. The guarantees hold only when the session uses read concern `majority` for reads and write concern `majority` for writes. Operations outside the session are not ordered with respect to it.

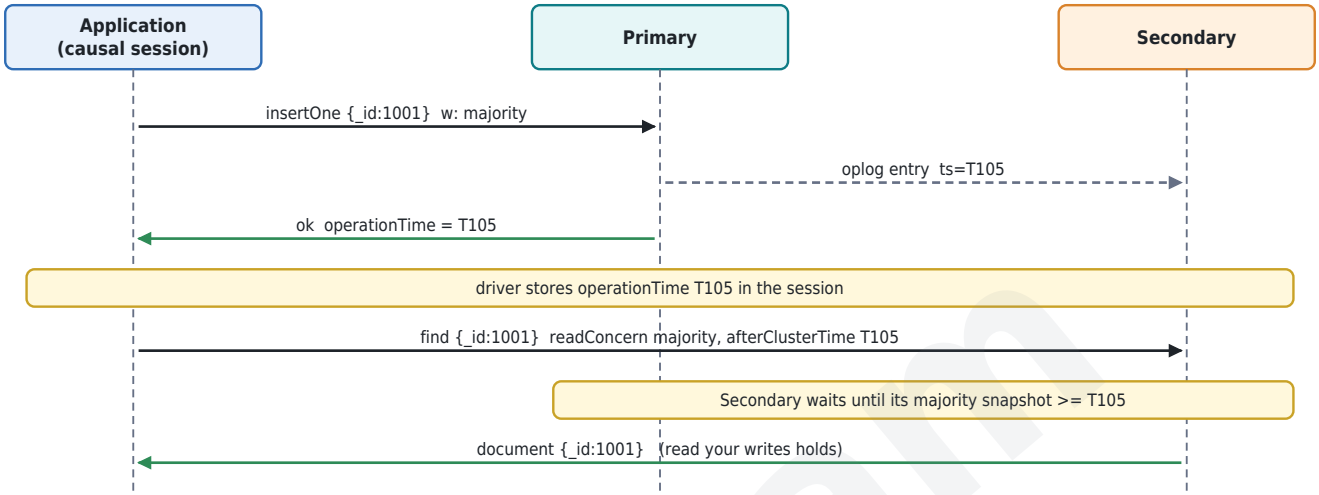


Figure 2.66. The driver carries the operation time of the write into the read, so a secondary delays the read until it has replicated past that time instead of returning stale data.

Retryable writes and reads

A retryable write is retried exactly once by the driver after a transient network error or a replica set election, without applying the write twice. The driver attaches a session id and a transaction number to the command, and the server keeps a record of the outcome of each, so a repeated command returns the stored result. Drivers for MongoDB 4.2 and later enable retryable writes by default (`retryWrites=true`). Retryable operations are the single-document ones: `insertOne`, `insertMany`, `updateOne`, `replaceOne`, `deleteOne`, `findOneAndUpdate`, `findOneAndReplace`, `findOneAndDelete`, and `bulkWrite` consisting only of those. `updateMany` and `deleteMany` are not retryable, and neither is a write with `w: 0`. Commit and abort of a transaction are retryable. Retryable reads (`retryReads=true`) retry a read once in the same way.

2.15.2 Worked example

The order service must create an order and reduce stock for each item. Stock lives in its own collection because it is updated by the warehouse system independently of orders, so the invariant spans two collections. The mongosh session below places order 1001 for two units of `A-100`. The statement `startTransaction` is local; no server round trip happens until the first operation. Typical output (not run on the build machine):

Multi-document transaction in mongosh (typical output, not run on the build machine)

```
shop> const session = db.getMongo().startSession()
shop> session.startTransaction({ readConcern: { level: 'snapshot' },
...     writeConcern: { w: 'majority' } })
shop> const s = session.getDatabase('shop')
shop> s.orders.insertOne({ _id: 1001, customerId: 5434, status: 'NEW',
...   items: [{ sku: 'A-100', qty: 2 }] })
{ acknowledged: true, insertedId: 1001 }
shop> s.stock.updateOne({ sku: 'A-100', qty: { $gte: 2 } }, { $inc: { qty: -2 } })
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
shop> session.commitTransaction()
shop> session.endSession()
```

Reading the sequence in order: the insert and the update execute inside the snapshot, and neither is visible to other clients until `commitTransaction`. The filter `qty: { $gte: 2 }` is the application-level invariant: when `matchedCount` is 0 the stock is insufficient and the code aborts instead of committing. If a second client had decremented the same `A-100`

document after the snapshot was taken, the `updateOne` would have failed with a `WriteConflict` error labelled `TransientTransactionError`, and the whole transaction would have to be retried from `startTransaction`.

The same first-writer-wins behaviour can be reproduced for real on PostgreSQL `REPEATABLE READ`, which also uses snapshot isolation. Session A reads the stock row and sleeps; session B reads, updates and commits; session A then attempts its update. The output below was produced on the build machine (PostgreSQL 16).

First writer wins under snapshot isolation (PostgreSQL 16, real output) TERMINAL

```
$ psql shop <<'EOF'   # session A, started first
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT qty FROM stock WHERE sku='A-100';
SELECT pg_sleep(2);
UPDATE stock SET qty=qty-1 WHERE sku='A-100';
EOF
 qty
-----
  10
(1 row)

 pg_sleep
----------

(1 row)

ERROR:  could not serialize access due to concurrent update
```

Session B (not shown) ran `UPDATE stock SET qty=qty-2` and committed during the sleep, so the final quantity is 8 and the update of session A was rejected. MongoDB reports the same situation as `WriteConflict` and expects the application to retry; PostgreSQL reports SQLSTATE 40001. The difference is that PostgreSQL makes a later writer wait for the first one to finish before deciding, while MongoDB aborts the later writer at once.

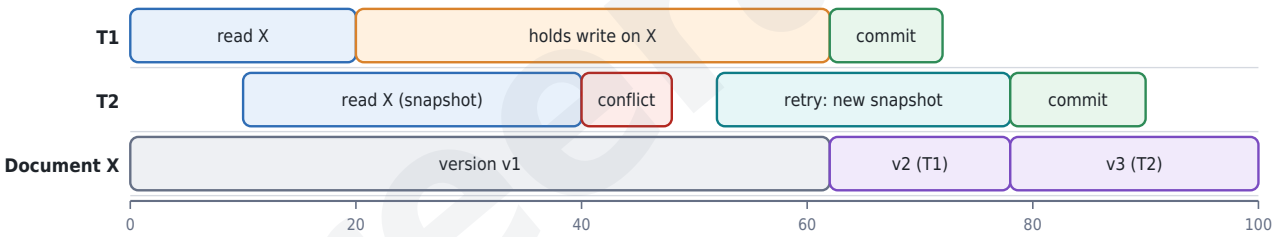


Figure 2.67. Transaction T2 reads document X from its snapshot, finds that T1 holds a write on X and aborts with a conflict; the retry takes a new snapshot that includes the version committed by T1.

2.15.3 Hands-on

Transaction code in Java with the callback API

The driver offers `ClientSession.withTransaction`, a callback API that starts the transaction, runs the body, commits, and automatically retries on `TransientTransactionError` and on `UnknownTransactionCommitResult` until a time limit of 120 seconds is reached. The listing uses the synchronous Java driver and is not run here because no driver jar is available on the build machine.

PlaceOrder.java (not run on the build machine) JAVA

```
import com.mongodb.ReadConcern;
import com.mongodb.TransactionOptions;
import com.mongodb.WriteConcern;
import com.mongodb.client.ClientSession;
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Updates;
import com.mongodb.client.result.UpdateResult;
import org.bson.Document;

public class PlaceOrder {
    public static void main(String[] args) {
        try (MongoClient client = MongoClients.create(
```

PlaceOrder.java (not run on the build machine) (continued) JAVA

```
            "mongodb://db1:27017,db2:27017,db3:27017/shop?replicaSet=rs0&w=majority")) {
        MongoCollection<Document> orders = client.getDatabase("shop").getCollection("orders");
        MongoCollection<Document> stock = client.getDatabase("shop").getCollection("stock");

        TransactionOptions options = TransactionOptions.builder()
                .readConcern(ReadConcern.SNAPSHOT)
                .writeConcern(WriteConcern.MAJORITY)
                .build();

        try (ClientSession session = client.startSession()) {
            String result = session.withTransaction(() -> {
                UpdateResult r = stock.updateOne(session,
                        Filters.and(Filters.eq("sku", "A-100"), Filters.gte("qty", 2)),
                        Updates.inc("qty", -2));
                if (r.getModifiedCount() == 0) {
                    throw new IllegalStateException("insufficient stock");
                }
                orders.insertOne(session, new Document("_id", 1001).append("status", "NEW"));
                return "ordered";
            }, options);
            System.out.println(result);
        }
    }
}
```

The body of the lambda can run more than once, therefore it must not have side effects outside the database (no emails, no messages). An exception thrown from the body aborts the transaction; `withTransaction` retries only the two labelled error types and rethrows everything else.

Observing write concern and read concern in mongosh

Choosing durability per operation (typical output, not run on the build machine)

```
shop> db.orders.insertOne({ _id: 1002, status: 'NEW' },
...   { writeConcern: { w: 'majority', j: true, wtimeout: 5000 } })
{ acknowledged: true, insertedId: 1002 }
shop> db.orders.find({ _id: 1002 }).readConcern('majority').readPref('secondary')
[ { _id: 1002, status: 'NEW' } ]
shop> db.adminCommand({ getDefaultRWConcern: 1 })
{
  defaultWriteConcern: { w: 'majority', wtimeout: 0 },
  defaultReadConcern: { level: 'local' },
  localUpdateWallClockTime: ISODate('2026-10-12T08:21:07.113Z'),
  ok: 1
}
```

Write concern simulation of a failover

The effect of `w: 1` during a failover can be measured with a small model: one write per millisecond, an exponentially distributed replication delay (mean 12 ms) to each of two secondaries, a primary crash at 1.0 s, and the election of the more up-to-date secondary. Acknowledged writes that are absent from the new primary are counted as lost. The model is a simplification (no network partition, ordered oplog application, instant election), run on the build machine with a fixed random seed.

Failover loss simulation (real output of the model, 2000 failovers) TERMINAL

```
$ python3 wc_sim.py
w1        acknowledged= 2002000 lost= 58224 failovers with loss=2000/2000 mean lost=29.11
majority  acknowledged= 1943521 lost=     0 failovers with loss=   0/2000 mean lost=0.00
```

The result is structural and not an artefact of the parameters: with `w: 1` the client is told "done" at the moment the primary applies the write, so every write that has not yet reached a secondary when the primary fails is acknowledged and then lost. With `w: "majority"` the acknowledgement is delayed until a secondary has the write, so the number of acknowledged writes is slightly lower (about 29 writes per failover were still in flight and are reported as errors, to be retried by the client), and none of the acknowledged ones disappears.

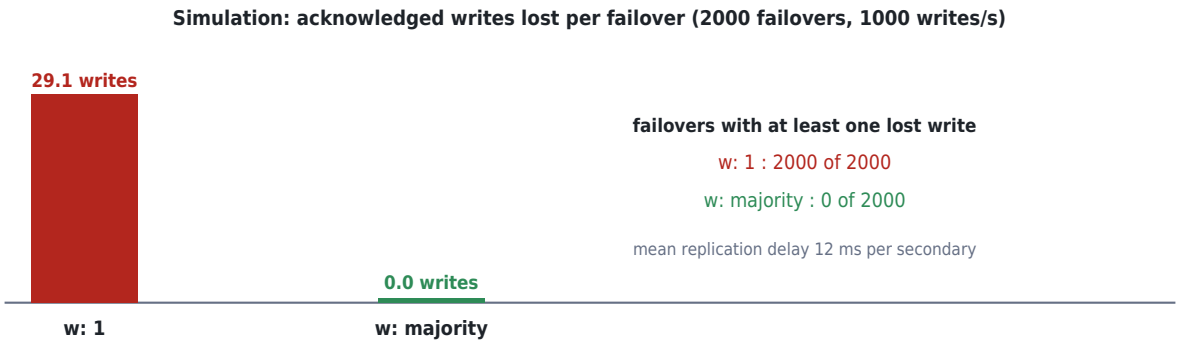


Figure 2.68. Under the simulated workload each failover loses about 29 acknowledged writes with w: 1 and none with w: majority.

2.15.4 Real-world problems and solutions

Incident: stale reads from a secondary

Scenario. A Spring Boot service reads orders with `readPreference=secondaryPreferred` to offload the primary. After a status update, a customer reloads the order page and still sees the previous status.

Symptoms. Support tickets state that "the status did not change". The write is present when checked in mongosh against the primary. The effect is intermittent and worse during imports and backups.

Diagnosis. In order:

1. Confirm that the read was served by a secondary: the connection string or `@ReadPreference` setting, or `db.currentOp()` on the secondaries.
2. Measure replication lag. Typical output (not run on the build machine):

Replication lag per secondary (typical output, not run on the build machine)

```
shop> rs.printSecondaryReplicationInfo()
source: db2.example.net:27017
        { syncedTo: 'Mon Oct 12 2026 08:21:05 GMT+0000', replLag: '1.8 secs (0 hrs) behind' }
source: db3.example.net:27017
        { syncedTo: 'Mon Oct 12 2026 08:21:06 GMT+0000', replLag: '0.9 secs (0 hrs) behind' }
```

1. Check whether the lag correlates with load: `db.serverStatus().wiredTiger.concurrentTransactions` (or `queues.execution` on 7.0 and later), oplog window in `rs.printReplicationInfo()`, and disk latency on the secondary.

Root cause. Replication is asynchronous, so a secondary always shows a past state. The service issues a write to the primary and an immediate read to a secondary with no ordering between them.

Fix. Immediate: route the read-after-write path to the primary (`primaryPreferred` or `primary`). Permanent: use a causally consistent session with `majority` read and write concern for flows that must read their writes, and keep `secondaryPreferred` only for reads that tolerate lag, with `maxStalenessSeconds` set.

Prevention. Alert on replication lag above a threshold (for example 10 seconds); document which endpoints tolerate stale data; test read-after-write behaviour with an artificial delay on a secondary.

sreram

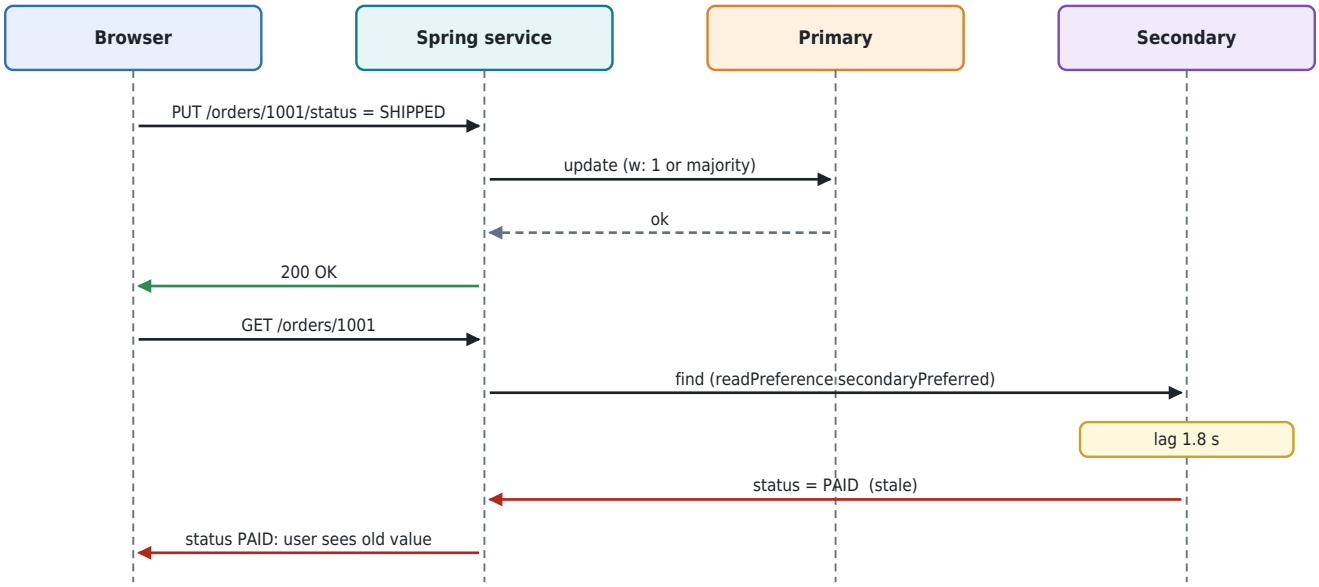


Figure 2.69. The update is acknowledged by the primary, the following read is served by a secondary that is 1.8 seconds behind, and the user sees the old status.

Incident: wrong write concern loses acknowledged writes on failover

Scenario. A payment-event collection is written with `w: 1` (set explicitly in the connection string years ago to reduce latency). A maintenance window triggers a primary step-down during peak traffic.

Symptoms. After the failover the application reports no errors, but reconciliation finds events that the application logged as stored and that are missing in MongoDB. The old primary logs a rollback.

Diagnosis.

1. Search the log of the former primary for the rollback and for the saved files.

Finding rolled-back data (typical output, not run on the build machine) TERMINAL

```
$ grep -i "rollback" /var/log/mongodb/mongod.log | grep -i "files\|complete" | tail -3
{"t":{"$date":"2026-10-12T02:14:51.201+00:00"},"s":"I","c":"ROLLBACK","msg":"Rollback complete"}
$ ls /var/lib/mongodb/rollback/*/
removed.2026-10-12T02-14-50.0.bson
$ bsondump /var/lib/mongodb/rollback/*/removed.2026-10-12T02-14-50.0.bson | wc -l
37
```

1. Compare the effective write concern: the connection string, `db.adminCommand({getDefaultRWConcern: 1})`, and the per-operation options in the code.

Root cause. A `w: 1` write is acknowledged before replication. The old primary held 37 acknowledged writes that no secondary had received; the new primary never saw them, so on rejoining, the old primary rolled them back.

Fix. Immediate: recover the documents from the BSON files in the `rollback` directory (inspect with `bsondump`, reinsert after verification). Permanent: set `w=majority` in the connection string or `WriteConcern.MAJORITY` on the collection, and set the cluster default with `setDefaultRWConcern`.

Prevention. Keep write concern explicit in configuration; add a failover test (`rs.stepDown()` under load in staging) to the release checklist, and treat `w: 1` for business data as a defect.

Incident: multi-document transaction timeouts

Scenario. A nightly job closes expired orders in one transaction: it iterates 200,000 orders, updates each, and decrements counters. The job worked in testing with 2,000 orders.

Symptoms. The job fails with `NoSuchTransaction` or `TransactionExpired` after about 60 seconds; during the run, online order updates fail with `WriteConflict`.

Diagnosis.

1. `db.currentOp({ 'transaction': { $exists: true } })` shows the transaction age and its lock count; `db.serverStatus().transactions` shows `totalAborted` growing.
2. The log contains the abort reason (`transactionLifetimeLimitSeconds`).

Root cause. The transaction exceeds the 60-second lifetime and holds write locks on 200,000 documents, so every concurrent writer of those documents conflicts with it.

Fix. Immediate: run the job in batches of 500 documents, each in its own short transaction (or, when no cross-document invariant exists, with plain `updateMany` over a bounded batch). Permanent: keep an idempotent marker (`closedAt`) so that a failed batch is re-run safely. Raising `transactionLifetimeLimitSeconds` is not a fix: it extends lock holding and cache pressure.

Prevention. Load-test with production volumes; alert on `transactions.totalAborted`; keep transactions below tens of documents and a few seconds.

2.15.5 Questions to expect

Questions and model answers

1. **Is a single MongoDB write atomic?** A write to one document is atomic including embedded documents and arrays. A write that spans several documents (`updateMany`) is atomic per document only; a multi-document transaction is needed for atomicity across documents.
2. **What isolation level do MongoDB transactions provide?** Snapshot isolation: reads see a consistent snapshot, conflicting writers abort with `WriteConflict` (first writer wins). It is not serializable, so write skew is possible.
3. **What does `w: "majority"` guarantee and what does it not?** The write has been durably written by a majority of voting members and will survive an election. It does not make the data visible on every secondary yet.
4. **When is `w: 1` acceptable?** For data that can be regenerated or is of little value (metrics, caches). Not for business records, because a failover can roll back acknowledged writes.
5. **What is the difference between read preference and read concern?** Read preference chooses the member; read concern chooses the consistency and durability level of the data returned. `secondary` with `majority` still returns possibly stale but never rolled-back data.
6. **How does a retryable write avoid a double insert?** The driver sends a session id and transaction number; the server records the result and returns it if the same pair is received again.
7. **Why can a transaction abort with `TransientTransactionError` and what does the application do?** A write conflict, a failover or a lock timeout aborted it. The application restarts the whole transaction; the callback API does this automatically.
8. **Why prefer embedding to multi-document transactions?** Embedding makes the invariant a single-document write: atomic, one round trip, no lock holding, no 60 second limit.

KEY POINTS

Single-document writes are atomic; model data so that invariants fit in one document.

- Multi-document transactions give snapshot isolation with first-writer-wins conflicts, a 60 second default limit and a 5 ms lock wait; keep them small and retry on `TransientTransactionError`.
- `w: "majority"` is required for acknowledged data to survive a failover; `w: 1` loses roughly the in-flight writes of every failover.
- Read preference selects the member, read concern the version of data; non-primary reads can be stale.
- Causal sessions with majority concerns give read your writes across members.
- Retryable writes cover single-document operations only.

2.16 Replication in MongoDB

A single mongod process is a single point of failure for availability and, until the disk is backed up, for data. MongoDB replicates data with replica sets: a group of mongod processes that hold copies of the same data and elect one member as primary to accept writes. Replication provides redundancy, automatic failover and the ability to serve reads, backups and analytics from additional members. It does not provide horizontal write scaling; that is the task of sharding. The chapter explains the mechanisms in enough depth to predict what happens during a failure and to size the settings that govern it.

2.16.1 Concept and mechanism

Members and roles

A replica set has up to 50 members, of which at most 7 vote. One member is the primary; it is the only member that accepts writes. The other data-bearing members are secondaries that copy the primary's operations. Members can take special roles through configuration.

Member	Configuration	Purpose
Primary	elected	Accepts writes and, by default, reads
Secondary	votes: 1, priority: 1	Replicates; electable; may serve reads
Priority 0	priority: 0	Never becomes primary; used for a remote data centre or dedicated reads
Hidden	hidden: true, priority: 0	Invisible to drivers; used for reporting and backups
Delayed	secondaryDelaySecs: N, hidden, priority 0	Holds a state N seconds old; protection against logical errors
Arbiter	arbiterOnly: true	Votes only, stores no data

Replica set member types.

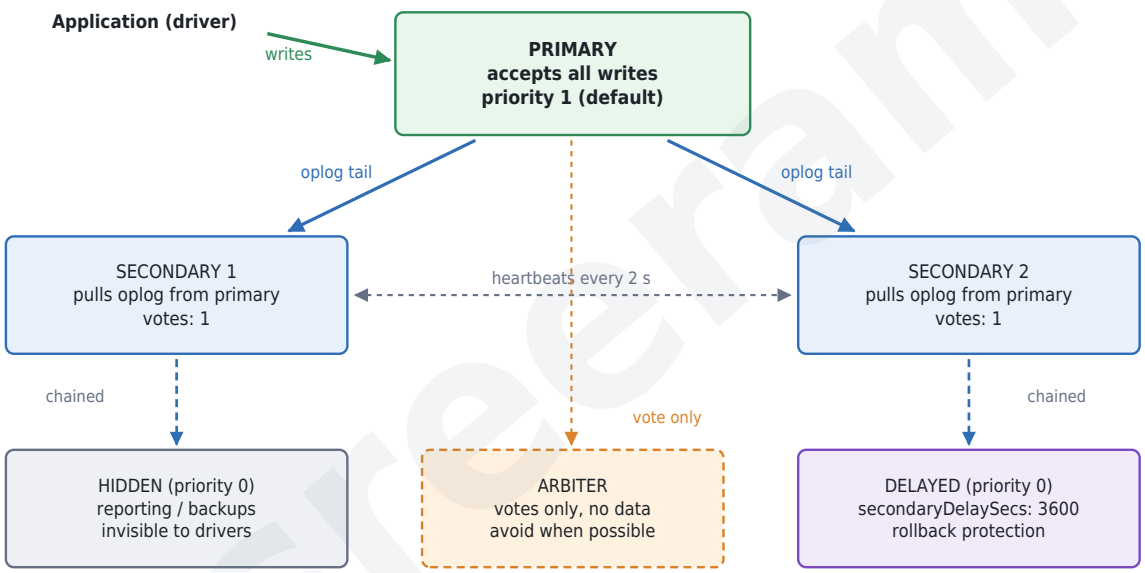


Figure 2.70. A replica set of one primary and two voting secondaries, with a hidden reporting member, a delayed member and an optional arbiter; secondaries pull the oplog and all members exchange heartbeats.

An arbiter provides a vote without providing a copy of the data. It cannot acknowledge writes, so it reduces the availability of `w: "majority"` (see the previous chapter), and it prevents some operations such as changing a shard key in a transaction. The recommended deployment is three data-bearing members in three failure domains.

The oplog

The primary applies every write to its data and records it in the oplog, a capped collection named `local.oplog.rs`. Each entry has a timestamp `ts`, a term `t`, an operation type `op` (`i` insert, `u` update, `d` delete, `c` command, `n` no-op), the namespace `ns` and the operation `o`. Entries are idempotent: applying one twice yields the same state, which makes replication and recovery safe to repeat. A secondary fetches entries from its sync source (normally the primary, optionally another secondary, which is chained replication), writes them to its own oplog and applies them in batches.

The oplog is a fixed-size ring: when it is full, the oldest entries are overwritten. The interval covered by the oplog is the oplog window. A secondary that stays down longer than the window cannot catch up from the oplog and must perform an initial sync (full copy). The default size on WiredTiger is a percentage of free disk space with documented minimum and maximum bounds, and it can be changed online with `replSetResizeOplog`.

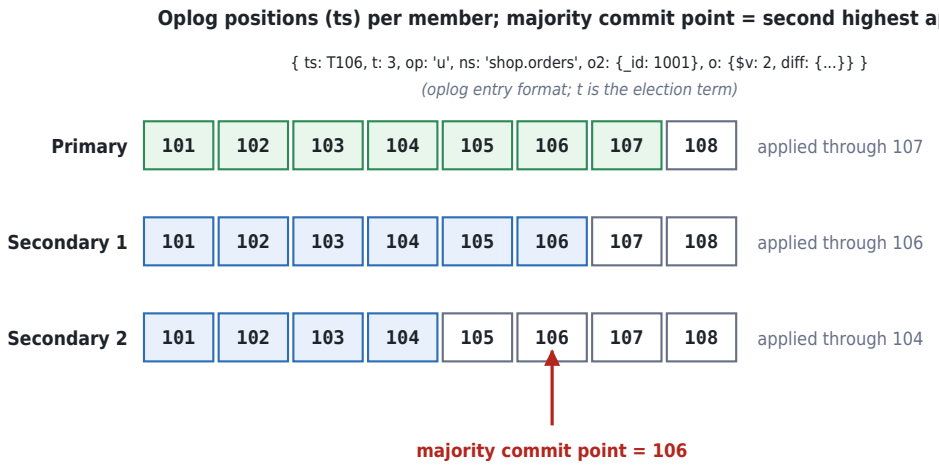


Figure 2.71. Each member has applied the oplog up to a different position; the majority commit point is the highest position present on a majority, here 106, which bounds what read concern majority returns.

An oplog entry (typical output, not run on the build machine)

```
shop> db.getSiblingDB('local').oplog.rs.find({ ns: 'shop.orders', op: 'u' })
...   .sort({ $natural: -1 }).limit(1)
[
  {
    op: 'u', ns: 'shop.orders', ui: UUID('3c1f4b8e-7a5d-4f0e-9d6b-2f6b0a7c11aa'),
    o: { '$v': 2, diff: { u: { status: 'SHIPPED' } } }, o2: { _id: 1001 },
    ts: Timestamp({ t: 1760257265, i: 4 }), t: Long('3'), v: Long('2'),
    wall: ISODate('2026-10-12T08:21:05.114Z')
  }
]
```

Elections

MongoDB uses a Raft-like protocol (replication protocol version 1). Time is divided into terms, each beginning with an election. Members exchange heartbeats every 2 seconds. A secondary that has not heard from the primary for `electionTimeoutMillis` (default 10 seconds) first runs a dry-run election to check that it would win, and then starts a real election with an incremented term. Each member votes at most once per term and refuses a candidate whose oplog is behind its own, so the winner holds the most recent majority-committed data. A candidate that gathers votes from a majority of voting members becomes primary. A primary that cannot reach a majority steps down on its own, which prevents two primaries from accepting writes that both survive.

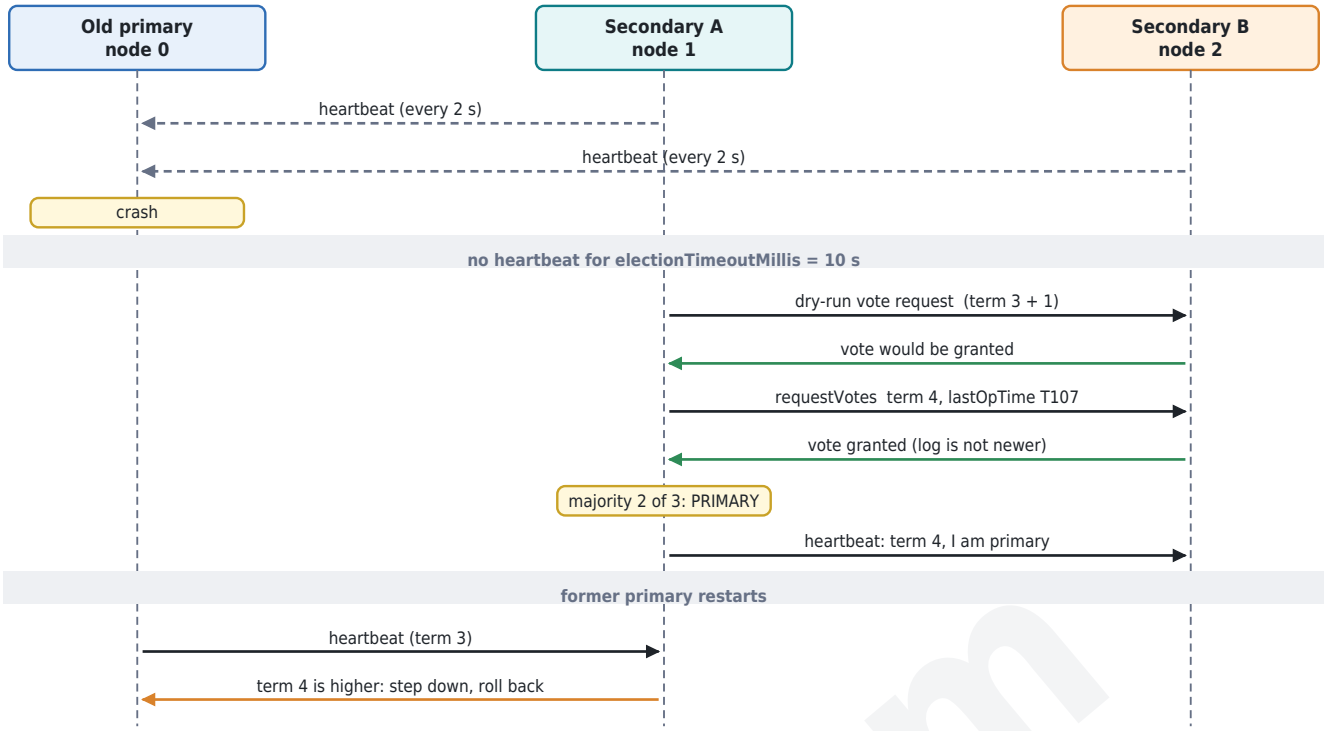


Figure 2.72. After the primary crashes, a secondary waits for the election timeout, wins a majority vote in a higher term, and the returning old primary sees the higher term, steps down and rolls back.

Priority influences who is elected: a higher-priority member calls elections sooner and is favoured, but priority is not a guarantee. Use `priority`, not `votes`, to prefer a member. The MongoDB documentation gives a median time to elect a new primary of no more than 12 seconds with default settings. Lowering `electionTimeoutMillis` shortens detection but increases spurious elections and rollbacks of `w: 1` writes. Writes are unavailable between the loss of the primary and the election of its successor; reads from secondaries remain possible under a suitable read preference.

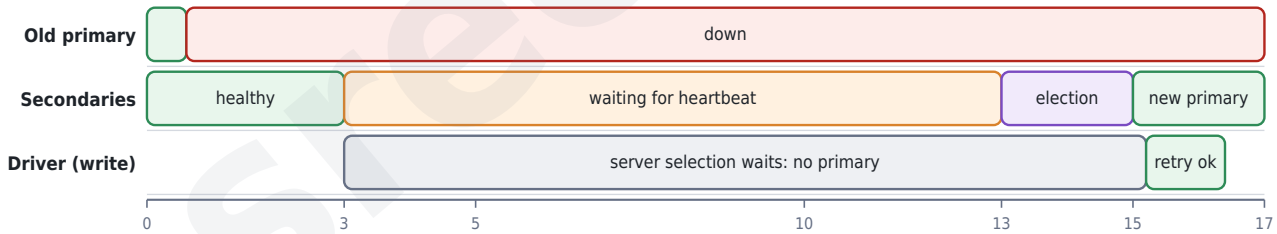


Figure 2.73. A write issued three seconds into an outage waits in server selection, and the retryable write succeeds once the new primary is elected, in this timeline roughly 15 seconds after the failure with default settings (illustrative values).

Rollback

If the old primary had accepted writes that no member received before it lost its primacy, those writes are on the old primary only. When it rejoins as a secondary it finds that its oplog diverges from the new primary's. It finds the common point, reverts its data to that point and applies the new primary's entries. The reverted inserts and updates are written to BSON files under `dbpath/rollback/<collection UUID>/removed.<timestamp>.bson`; reverted deletes and drops are not saved. Rollback is rare and, for writes with `w: "majority"`, impossible by construction. A member whose divergence exceeds `rollbackTimeLimitSecs` (24 hours by default) cannot roll back and must be resynchronised.

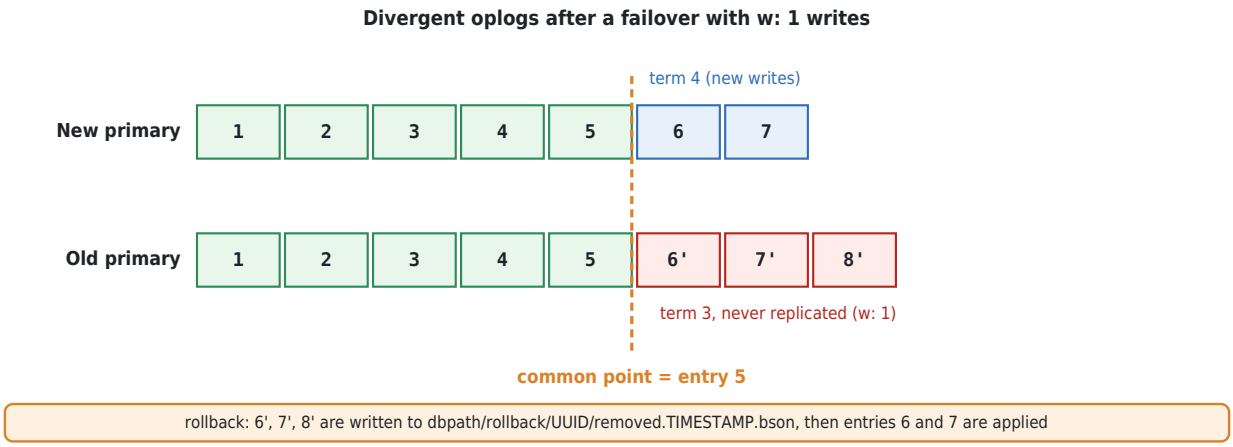


Figure 2.74. The old primary holds entries 6', 7' and 8' from term 3 that no other member received; after the common point at entry 5 the new primary's history wins and the divergent entries are saved to a file and removed.

Flow control and initial sync

A new or stale member runs an initial sync: it clones all databases, applies the oplog that accumulated during the clone, and then switches to normal replication. Initial sync of a large dataset takes hours and competes for I/O with production traffic. Since MongoDB 4.2 a flow control mechanism throttles the primary's writes when the majority commit point lags more than a target (10 seconds by default), trading write throughput for bounded lag.

2.16.2 Worked example

A replica set `rs0` has three members. The primary holds oplog position 107, secondary 1 position 106 and secondary 2 position 104. The question is which write is acknowledged to a `w: "majority"` client, which writes a `majority` reader sees, and what a failover would lose.

1. The majority of three voting members is two. The positions sorted are 107, 106, 104. The second highest position, 106, is present on at least two members: this is the majority commit point.
2. A write at position 106 or lower is acknowledged to a `w: "majority"` client. The write at 107 is acknowledged to a `w: 1` client already, but a `w: "majority"` client waits.
3. A `majority` reader sees the state up to position 106 only. Position 107 might still be rolled back.
4. If the primary now fails, secondary 1 (position 106) and secondary 2 (position 104) hold an election. Secondary 2 cannot win: the voter secondary 1 refuses a candidate with a shorter oplog. Secondary 1 wins and becomes primary at term 4. Entry 107 existed only on the old primary and is lost: it was acknowledged only to `w: 1` clients.
5. Secondary 2 catches up from the new primary (entries 105 and 106). When the old primary returns, it rolls back entry 107 to a file and resumes as a secondary.

The simulation in the previous chapter reproduces exactly this step 4 at scale: every acknowledged `w: 1` write beyond the surviving prefix is lost.

2.16.3 Hands-on

Starting a three-member set and reading its status

The commands below create a development replica set on one machine (three mongod processes on different ports). They are shown for reference; MongoDB is not installed on the build machine, so the output is typical.

Starting a local replica set (typical output, not run on the build machine) TERMINAL

```
$ mongod --replSet rs0 --port 27017 --dbpath /data/rs0-0 --bind_ip localhost \
    --fork --logpath /data/rs0-0.log
$ mongod --replSet rs0 --port 27018 --dbpath /data/rs0-1 --bind_ip localhost \
    --fork --logpath /data/rs0-1.log
$ mongod --replSet rs0 --port 27019 --dbpath /data/rs0-2 --bind_ip localhost \
    --fork --logpath /data/rs0-2.log
$ mongosh --port 27017 --eval 'rs.initiate({_id: "rs0", members: [ \
    {_id: 0, host: "localhost:27017"}, {_id: 1, host: "localhost:27018"}, \
    {_id: 2, host: "localhost:27019"}]})'
{ ok: 1 }
```

rs.status() reduced to the essential fields (typical output, not run on the build machine)

```
rs0 [direct: primary] test> rs.status().members.map(m =>
...   ({ name: m.name, state: m.stateStr, optimeDate: m.optimeDate }))
[
  { name: 'localhost:27017', state: 'PRIMARY', optimeDate: ISODate('2026-10-12T08:21:05.000Z') },
  { name: 'localhost:27018', state: 'SECONDARY', optimeDate: ISODate('2026-10-12T08:21:05.000Z') },
  { name: 'localhost:27019', state: 'SECONDARY', optimeDate: ISODate('2026-10-12T08:21:04.000Z') }
]
```

Planned failover and configuration changes

Step down and prefer a member (typical output, not run on the build machine)

```
rs0 [direct: primary] test> rs.stepDown(60)
{ ok: 1, '$clusterTime': { clusterTime: Timestamp({ t: 1760257300, i: 1 }) } }
rs0 [direct: secondary] test> cfg = rs.conf(); cfg.members[1].priority = 2; rs.reconfig(cfg)
{ ok: 1 }
```

`rs.stepDown(60)` makes the primary ineligible for election for 60 seconds, so the failover is controlled and quick (the primary waits for an electable secondary to catch up first). Planned maintenance always uses `stepDown`, never `kill` of the primary.

Connection string for a replica set

application.properties — PROPERTIES

```
spring.data.mongodb.uri=mongodb://db1:27017,db2:27017,db3:27017/shop?replicaSet=rs0\
  &w=majority&readPreference=primary&retryWrites=true&retryReads=true
```

The driver discovers the members through the seed list, monitors them, and reroutes after an election. Listing all members in the seed list lets the driver connect even when one is down; the `replicaSet` name must match.

2.16.4 Real-world problems and solutions

Incident: failover causing write errors despite retryable writes

Scenario. A planned primary step-down is executed during business hours. The order service uses retryable writes and expects no visible errors.

Symptoms. For about 12 seconds the service logs `MongoNotPrimaryException` or `MongoSocketReadException`, and some HTTP requests fail with timeouts. A few `updateMany` batches fail outright.

Diagnosis.

1. Check which operations failed: single-document writes are retried once; `updateMany` and `deleteMany` are not retryable; a `w: 0` write is never retried.
2. Compare the client timeouts with the election time. `serverSelectionTimeoutMS` (default 30 seconds) bounds how long the driver waits for a primary; an HTTP timeout of 5 seconds is shorter than the election.
3. Review the log of the members for the election time: `"Starting an election"` and `"Election succeeded"` messages.

Root cause. Retryable writes retry once after the new primary is found, but the retry cannot start before the election completes (typically up to about 12 seconds). Non-retryable operations return the error to the caller, and short upstream timeouts convert the wait into failures.

Fix. Immediate: retry failed bulk operations in the application with backoff (they are safe when idempotent). Permanent: make bulk updates idempotent and retried by the caller; set HTTP and driver timeouts consistently (`timeoutMS` or `serverSelectionTimeoutMS` longer than the expected election); schedule `stepDown` for low-traffic periods.

Prevention. Run failover drills in staging with production-like load; track the error rate during each maintenance; expose the number of retries as a metric.

Incident: replica set unavailable for writes after losing one data centre

Scenario. A five-member replica set has two members in data centre A and three in data centre B. Data centre B loses power.

Symptoms. The two members in A remain up but are both secondaries; applications report `NotWritablePrimary`.

Diagnosis. `rs.status()` shows the members in B as `(not reachable/healthy)`, and the members in A as `SECONDARY`. Two of five votes are not a majority.

Root cause. A majority of voting members (3 of 5) is required both to elect a primary and for `w: "majority"`. The majority lived in the failed site, so the surviving site cannot elect.

Fix. Immediate: when B cannot be restored, reconfigure A with `rs.reconfig(cfg, { force: true })` after confirming that B is permanently down (this can lose writes that only B held). Permanent: place members so that no single site holds a majority: for five voters use 2 + 2 + 1 across three sites.

Prevention. Design the topology from the failure it must survive; test by isolating a site in staging.

2.16.5 Questions to expect

Questions and model answers

1. **How does MongoDB elect a primary?** A secondary that misses heartbeats for the election timeout (10 s default) runs a dry-run and then a real election in a higher term. Voters grant one vote per term and only to a candidate whose oplog is at least as recent as their own. A majority wins.
2. **What is the oplog and why is it idempotent?** A capped collection of operations in `local.oplog.rs`. Each entry records the resulting change in a form that can be applied repeatedly with the same result, which makes replay after a crash or during initial sync safe.
3. **What is a rollback and how is it avoided?** A former primary reverts writes that no other member received. Using `w: "majority"` guarantees that acknowledged writes are not rolled back.
4. **Why are two data members plus an arbiter a poor design?** Majority writes require both data members; one failure blocks them and leaves only one copy. Three data-bearing members avoid both problems.
5. **What happens if a secondary is down longer than the oplog window?** It cannot resume from the oplog and needs a full initial sync.
6. **How do reads from secondaries relate to consistency?** They can be stale by the replication lag. Use causal sessions with majority concerns when a read must reflect a prior write.
7. **What do hidden and delayed members do?** Hidden members serve reporting and backups without receiving driver traffic. A delayed member keeps a historical state as protection against logical errors such as an accidental drop.
8. **How does the number of members affect fault tolerance?** The set tolerates the loss of a minority: 3 members tolerate 1 failure, 5 tolerate 2. An even number adds no tolerance over the odd number below it.

KEY POINTS

A replica set elects one primary by majority vote; the oplog carries idempotent operations to secondaries.

- Election takes about 10 to 12 seconds with default settings; writes are unavailable during that time.
- Only `w: "majority"` writes are protected from rollback; rolled-back inserts and updates are saved as BSON files.
- Put voting members in at least three failure domains and avoid arbiters in new designs.
- Reads from secondaries are stale by the replication lag; bound it with `maxStalenessSeconds` or avoid it for read-after-write.

2.17 Sharding in MongoDB

A replica set stores the complete dataset on every member and sends every write through one primary. When the dataset exceeds the storage or memory of one machine, or the write rate exceeds what one primary can apply, the data must be divided. Sharding partitions a collection across several replica sets (shards), each holding a subset of the documents. The application sees one logical collection; routing, balancing and metadata are handled by the cluster. The price is a design decision that is hard to reverse: the shard key. A poor shard key produces a hot shard, jumbo chunks or queries that touch every shard. This chapter covers the components, the criteria for a shard key, the mechanics of chunks and the balancer, query routing, zones and resharding. Experiments that the build machine could run (simulations of key distributions) are marked as such; cluster commands are typical output.

2.17.1 Concept and mechanism

Components

A sharded cluster has three kinds of process.

- **Shards.** Each shard is a replica set that stores a subset of the sharded data (and the complete data of unsharded collections assigned to it as the primary shard of a database).
- **Config servers.** A replica set (CSRS) that stores the cluster metadata: the list of shards, the chunk map of each collection, zone definitions and the state of the balancer.
- **mongos routers.** Stateless processes that applications connect to. A mongos caches the chunk map, routes each operation to the shards that own the relevant data, and merges results. Several mongos instances are deployed for availability, commonly one per application host.

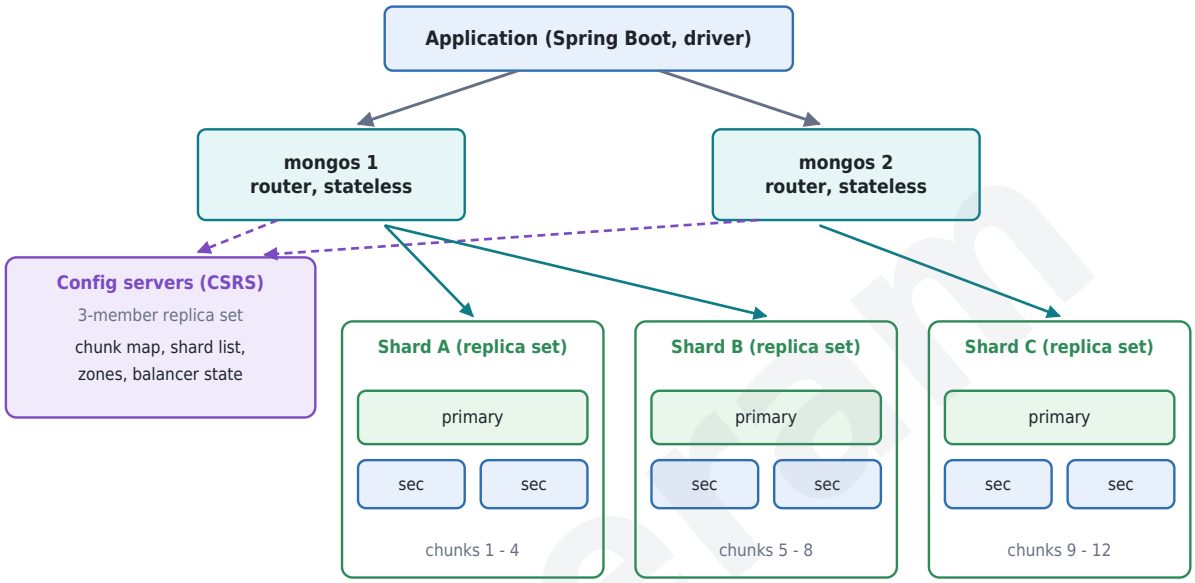


Figure 2.75. The application talks to mongos routers, which consult the config servers for the chunk map and send operations to the shard replica sets that own the data.

The shard key

The shard key is one field or several fields of every document that determines the shard on which the document lives. It must be backed by an index (created automatically on an empty collection) and every document must have its value (missing fields are treated as null). Since MongoDB 4.2 the value of a shard key field can be modified, except `_id`, and since 4.4 the key can be refined by appending fields (`refineCollectionShardKey`); since 5.0 the whole key can be changed by resharding. A shard key is therefore no longer permanent, but changing it moves data and costs hours on a large collection.

Four properties determine the quality of a shard key.

1. **Cardinality.** The number of distinct values bounds the number of chunks. A key such as `country` with 200 values cannot spread beyond 200 chunks, and a key such as `status` with 5 values cannot spread at all.
2. **Frequency.** The distribution of the values matters as much as their number. When one value covers 55 percent of the documents, the chunk for that value cannot be split, because a split needs two different key values. Such a chunk is flagged jumbo.
3. **Monotonic change.** A key that grows with time (timestamp, `ObjectId`, auto-increment number) in a range-sharded collection directs all new inserts to the chunk that holds the maximum value, and therefore to a single shard. The balancer redistributes stored data but cannot redistribute the insert load.
4. **Query isolation.** The key should appear in the filters of the frequent queries. A query that contains the shard key (or a prefix of a compound key) is targeted at one shard or a few; a query without it is broadcast to all shards (scatter-gather).

A hashed shard key (`{ customerId: 'hashed' }`) applies a hash function to the value, which spreads monotonic and clustered values uniformly. The price is that range queries on the key become scatter-gather, because neighbouring values are on different shards. A ranged key keeps neighbours together, so range queries are targeted, at the risk of

hot spots. Compound keys combine both: `{ region: 1, customerId: 'hashed' }` is supported since MongoDB 7.0, and a hashed field in the second position is the common pattern to spread within one region.

Candidate key	Cardinality / frequency	Insert distribution	Verdict
{ _id: 1 } (ObjectId)	high / unique	all inserts to the last chunk	hot shard
{ _id: 'hashed' }	high / unique	uniform	good for writes; range queries scatter
{ customerId: 'hashed' }	high / even	uniform	good; customer queries target one shard
{ country: 1 }	8 values / 55% in one	skewed	jumbo chunk, poor
{ country: 1, customerId: 1 }	high	spread inside each country	good; supports zones

Shard key candidates for the orders collection (100,000 documents in the experiment below).

Chunks and the balancer

The range of shard key values is divided into chunks: contiguous ranges, each assigned to one shard. A chunk is a metadata concept; documents are not stored per chunk. The default maximum chunk size ("range size") is 128 MB. When a chunk grows beyond it, it is split at a median key value. The balancer, a process running on the config server primary, compares the data volume of each shard per collection and migrates ranges from the shard with the most data to the shard with the least when the difference exceeds three times the range size (384 MB with the default). Each shard takes part in one migration at a time, and a cluster with n shards runs at most n/2 migrations concurrently.

A migration copies the range to the recipient, forwards writes that occurred during the copy, briefly blocks writes to the range while the ownership is committed in the config metadata (the critical section), and then deletes the donor copy asynchronously. The balancer can be restricted to a window with `sh.setBalancerState` and the `balancerWindow` setting so that migrations do not compete with peak traffic.

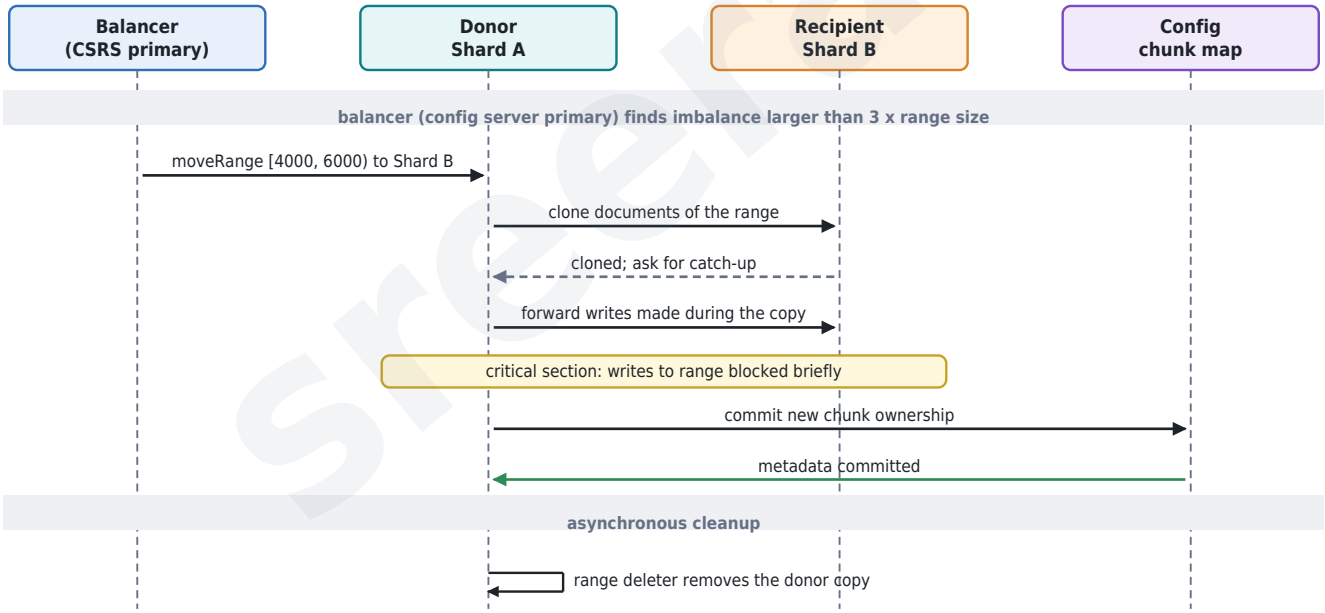


Figure 2.76. A chunk migration clones documents, replays concurrent writes, commits the new owner in the config metadata inside a short critical section, and cleans up the donor copy in the background.

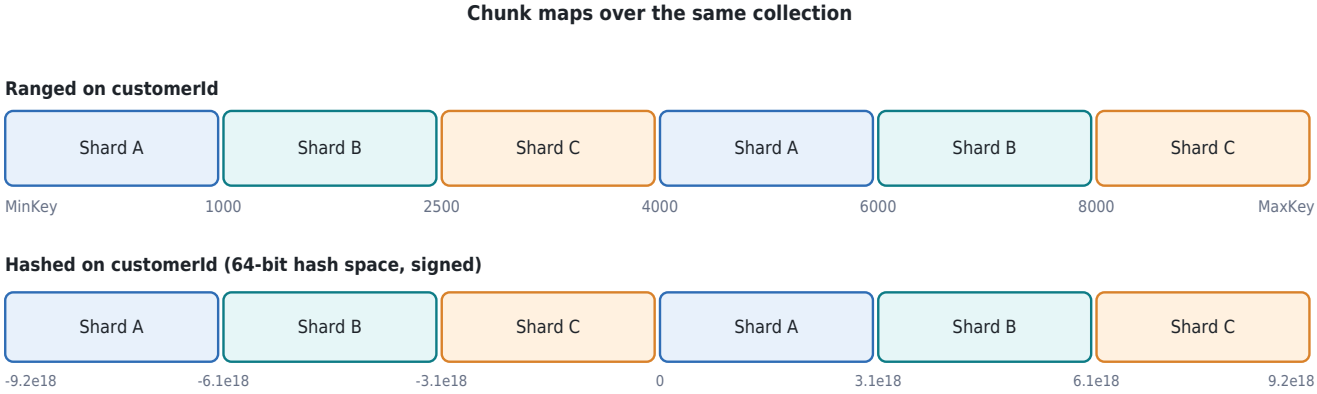


Figure 2.77. The same collection divided by ranged and hashed keys: ranged chunks keep neighbouring customer ids together, hashed chunks cover consecutive intervals of the hash space so neighbouring ids are scattered.

A chunk with too many documents to move (above roughly twice the range size divided by the average document size) or a single key value that cannot be split is a jumbo chunk. It remains on its shard, grows without bound and skews the cluster. Since MongoDB 6.0 the balancer balances by data size and splitting is performed lazily (when a chunk must be moved), which removed most automatic split work, but the jumbo condition itself still follows from the key.

Query routing

For a read or write, mongos extracts the shard key conditions from the filter and consults its chunk map. If the filter contains an equality on the full shard key, exactly one chunk is addressed. A range or `$in` on the shard key addresses the chunks that overlap. A filter without the leftmost shard key field is sent to every shard (scatter-gather), and mongos merges sorted partial results. Aggregation pipelines are split: stages that can run per shard (`$match`, `$project`, partial `$group`) run on the shards, and a merging stage runs on one shard or on mongos. Inserts need the full shard key in the document; updates and deletes that address a single document should include the shard key in the filter or use the `_id` together with it.

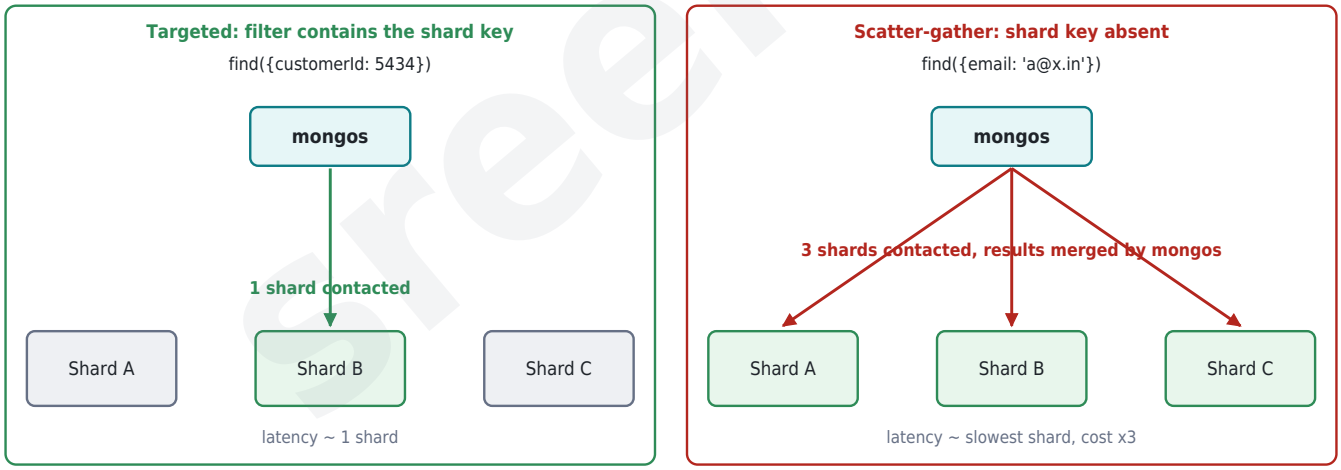


Figure 2.78. A filter that contains the shard key is routed to one shard; a filter on another field is sent to all shards and merged, so latency follows the slowest shard.

Unique indexes on a sharded collection must include the shard key as a prefix, because each shard can enforce uniqueness only within its own data. A unique constraint on `email` therefore requires either shard key `{ email: 1 }` or a separate lookup collection keyed by email.

Zone sharding

Zones associate ranges of the shard key with a subset of shards. The balancer then places chunks of a range only on shards of the matching zone. The typical uses are data residency (European customers on European shards), tiered hardware (recent data on fast shards, old data on cheap shards) and isolating a tenant. The shard key must begin with the field that defines the zone.

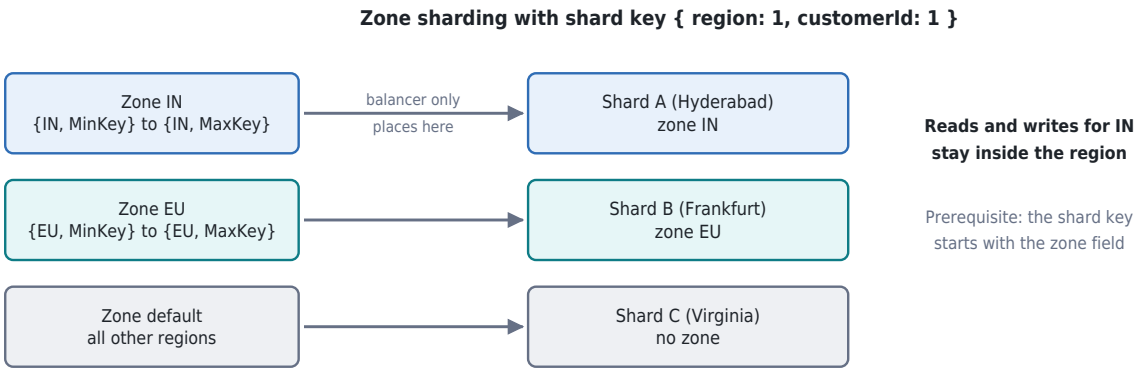


Figure 2.79. With a shard key that begins with the region, zone ranges pin each region's chunks to the shards of that zone, so reads and writes for a region stay in its data centre.

Resharding

`db.adminCommand({ reshardCollection: 'shop.orders', key: { customerId: 'hashed' } })` (MongoDB 5.0 and later) rebuilds the collection under a new key online: it copies the data to a new collection layout in the background, applies the changes that occur during the copy, and then switches over in a short critical section. It needs spare storage (a full copy), I/O and a window of low write traffic near the end, and it is the documented remedy for a shard key that proved wrong. MongoDB 8.0 added `moveCollection` and `unshardCollection` and a faster rebalance with the same key; 7.0 added `analyzeShardKey`, which samples queries and reports the cardinality, frequency and monotonicity of candidate keys before committing to them.

2.17.2 Worked example

The orders collection receives 100,000 inserts distributed over four shards. The question is which shard key keeps the insert load even. The experiment is a simulation with a Python model of the chunk mechanism: chunks split at the median key when they exceed 1,000 documents (a scaled-down range size), a balancer moves chunks from the shard with most chunks to the one with fewest every 5,000 inserts, and each insert is routed to the shard that owns its chunk at that moment. Three keys are compared: a monotonic key (the insertion order, as `ObjectId` or a timestamp would give), the hash of `_id`, and a uniformly random key. The hash is MD5-based and stands in for the MongoDB hash function. Real output of the model:

Shard key simulation (real output of the model, columns reformatted to fit the page) TERMINAL

```
$ python3 shard_sim.py
key                    chunks  inserts per shard                docs per shard
ranged, monotonic key     199  [100000, 0, 0, 0]                [25500, 25000, 25000, 24500]
ranged, hashed(_id)       128  [29056, 23674, 24085, 23185]     [25382, 25036, 25327, 24255]
ranged, random key        131  [29367, 23494, 23137, 24002]     [25853, 23272, 25583, 25292]
country key: 8 distinct values, most frequent value IN = 55.0% of documents (55,040)
```

Reading the result: with the monotonic key the stored data ends up balanced (about 25,000 documents on each shard, because the balancer migrated the old chunks away), yet shard 1 received all 100,000 inserts. The balancer repairs the storage skew but not the load skew, because every new document belongs to the newest chunk. The hashed and random keys spread the inserts within 18 percent of the ideal 25,000 per shard (the first shard is somewhat higher because chunks on it are split before they migrate). The last two lines show the frequency problem: an eight-valued key has 55 percent of its documents under one value, so the chunk of that value cannot be divided.

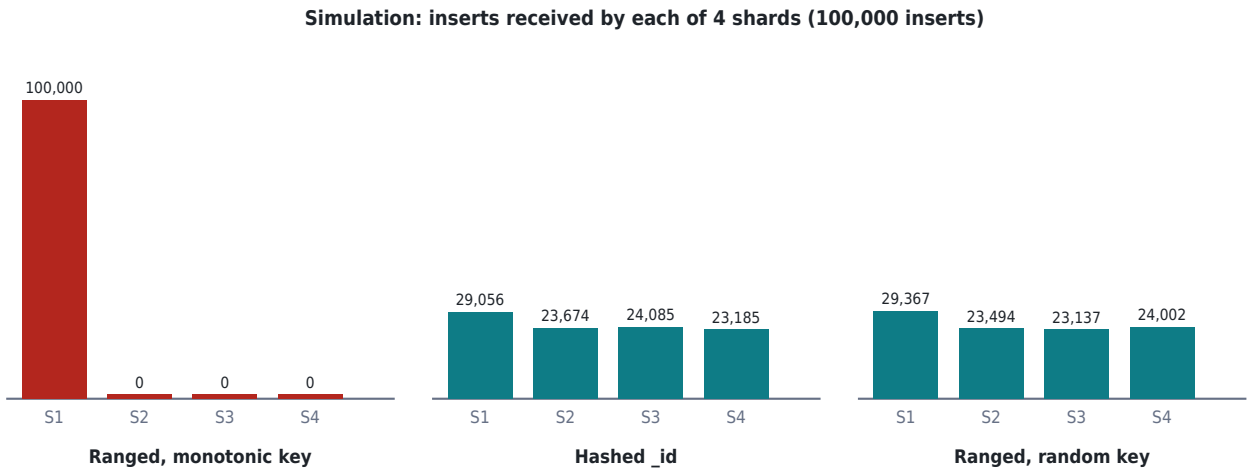


Figure 2.80. Under a monotonic ranged key one shard receives every insert; hashed and random keys give each shard roughly a quarter of the inserts.

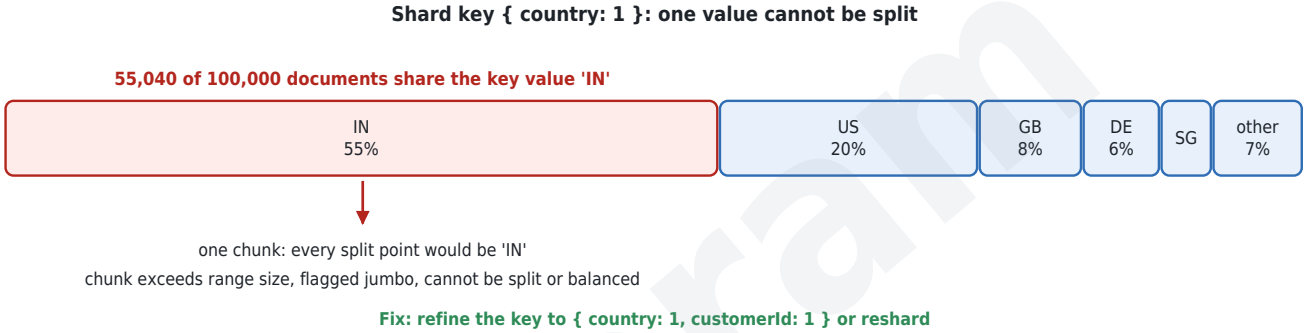


Figure 2.81. A shard key with one dominant value produces a chunk that cannot be split because every split point would equal the same key value; refining the key with a second field removes the restriction.

2.17.3 Hands-on

Creating a sharded collection

The commands below show the common sequence for the orders collection with a hashed customer key. They are typical output; no MongoDB cluster is available on the build machine.

mongosh session against a mongos (typical output, not run on the build machine)

```
[direct: mongos] test> sh.shardCollection('shop.orders', { customerId: 'hashed' })
{
  collectionsharded: 'shop.orders',
  ok: 1,
  '$clusterTime': { clusterTime: Timestamp({ t: 1760257500, i: 8 }) }
}
[direct: mongos] test> db.getSiblingDB('shop').orders.getShardDistribution()
Shard shardA at shardA/a1:27018,a2:27018,a3:27018
  data : 3.91GiB docs : 4012331 chunks : 2
  estimated data per chunk : 1.95GiB estimated docs per chunk : 2006165
Shard shardB at shardB/b1:27018,b2:27018,b3:27018
  data : 3.88GiB docs : 3991020 chunks : 2
  estimated data per chunk : 1.94GiB estimated docs per chunk : 1995510
Totals
  data : 7.79GiB docs : 8003351 chunks : 4
  Shard shardA contains 50.13% data, 50.13% docs in cluster, avg obj size on shard : 1KiB
  Shard shardB contains 49.87% data, 49.87% docs in cluster, avg obj size on shard : 1KiB
```

A hashed key on an empty collection creates initial chunks automatically (the option `numInitialChunks` controls how many), so the load is spread from the first insert; a ranged key starts with a single chunk on one shard and needs time to split and migrate.

Explaining a targeted and a scatter-gather query

explain output reduced to the routing information (typical output, not run on the build machine)

```
[direct: mongos] shop> db.orders.find({ customerId: 5434 }).explain().queryPlanner.winningPlan
{ stage: 'SINGLE_SHARD',
  shards: [ { shardName: 'shardB', winningPlan: { stage: 'FETCH',
      inputStage: { stage: 'IXSCAN', indexName: 'customerId_hashed' } } } ] }
[direct: mongos] shop> db.orders.find({ status: 'NEW' }).explain().queryPlanner.winningPlan
{ stage: 'SHARD_MERGE',
  shards: [ { shardName: 'shardA', winningPlan: { stage: 'COLLSCAN' } },
            { shardName: 'shardB', winningPlan: { stage: 'COLLSCAN' } } ] }
```

`SINGLE_SHARD` means one shard was contacted. `SHARD_MERGE` means several shards ran the query and mongos merged the results. The second query also shows a `COLLSCAN` on every shard: scatter-gather multiplies a missing index by the number of shards.

Zone commands

Pinning regions to shards (typical output, not run on the build machine) JAVASCRIPT

```
sh.shardCollection('shop.customers', { region: 1, customerId: 1 })
sh.addShardToZone('shardA', 'IN')
sh.addShardToZone('shardB', 'EU')
sh.updateZoneKeyRange('shop.customers', { region: 'IN', customerId: MinKey },
                      { region: 'IN', customerId: MaxKey }, 'IN')
sh.updateZoneKeyRange('shop.customers', { region: 'EU', customerId: MinKey },
                      { region: 'EU', customerId: MaxKey }, 'EU')
```

2.17.4 Real-world problems and solutions

Incident: shard key causing a hot shard

Scenario. An event collection is sharded on `{ createdAt: 1 }` ("because queries are by date"). The cluster has four shards and the dashboards show a balanced disk usage.

Symptoms. The primary of one shard runs at 95 percent CPU and has a growing write queue; the other three are almost idle. Insert latency rises with load although the cluster has spare capacity. `sh.status()` reports balanced chunk counts.

Diagnosis. In order:

1. Compare per-shard operation counters: `db.serverStatus().opcounters` on each shard primary (or the monitoring graphs per shard). One shard shows nearly all inserts.
2. Identify the chunk that receives the inserts: `db.getSiblingDB('config').chunks.find({ ns: 'shop.events' }).sort({ min: -1 }).limit(1)` shows the chunk with the highest key range and its shard.
3. Check the shard key analysis for monotonicity: `db.events.analyzeShardKey({ createdAt: 1 })` (MongoDB 7.0 and later) reports a monotonic change in the key; for earlier versions the inference follows from the key being a timestamp.

Root cause. A monotonically increasing ranged key sends every insert to the chunk with the maximum key, which lives on one shard. The balancer moves older chunks to other shards, so storage is balanced but the write load is not. The experiment in the worked example reproduces the effect: 100,000 inserts on one shard and none on the others.

Fix. Immediate: add capacity to the hot shard's primary (vertical scaling) and reduce write amplification (fewer indexes, batching). Permanent: reshard to a key with a non-monotonic component, for example `{ deviceId: 1, createdAt: 1 }` (writes spread by device, time queries per device targeted) or `{ createdAt: 'hashed' }` when range queries on time are rare. Run `reshardCollection` outside the peak and verify spare disk space (a full copy of the collection).

Prevention. Evaluate candidate keys with `analyzeShardKey` and a load test before sharding; reject timestamps, `ObjectId` and sequences as the first field of a ranged key; monitor operation counters per shard.

Incident: jumbo chunks

Scenario. Orders are sharded by `{ country: 1 }`. The business operates mostly in one country.

Symptoms. One shard holds 55 percent of the data and its disk fills first. `sh.status()` marks chunks as `jumbo: true`. Migrations of those chunks fail or are skipped, and the balancer logs that the chunk "exceeds the maximum size and cannot be split".

Diagnosis.

1. `db.getSiblingDB('config').chunks.find({ ns: 'shop.orders', jumbo: true })` lists the flagged chunks and their key ranges. In the typical case min and max of the chunk have the same `country` value.
2. `db.orders.aggregate([{ $group: { _id: '$country', n: { $sum: 1 } } }, { $sort: { n: -1 } }])` shows the frequency distribution: here 55 percent of the documents carry `IN`.
3. `db.orders.stats().avgObjSize` multiplied by the document count of the range gives the chunk size and confirms that it exceeds the range size.

Root cause. A chunk is bounded by key values; documents with the same key value cannot be separated. The key has too few distinct values and a dominant one.

Fix. Immediate: for a temporary relief, `sh.splitAt` cannot split a single value, so free space by archiving old data or increasing the shard's disk; `moveRange` with `forceJumbo` can migrate an oversized range explicitly when the destination has room. Permanent: refine the key by appending a high-cardinality field, `db.adminCommand({ refineCollectionShardKey: 'shop.orders', key: { country: 1, customerId: 1 } })` (the index on the new key must exist first), after which the dominant value can be split by `customerId`; or reshard.

Prevention. Check the frequency distribution of candidate keys against production data; use compound keys that end with a high-cardinality field; alert when any chunk is flagged jumbo.

Incident: scatter-gather queries after sharding

Scenario. A collection is sharded on `{ customerId: 'hashed' }`. A support page searches orders by `orderNumber`.

Symptoms. The query took 4 ms on a replica set and takes 60 ms on the cluster and consumes CPU on every shard. Explain shows `SHARD_MERGE`.

Diagnosis. `explain()` on mongos lists all shards; the filter lacks the shard key. `db.currentOp()` on each shard shows the same query running concurrently.

Root cause. The filter does not contain the shard key, so mongos cannot target a shard. The cost grows with the number of shards.

Fix. Immediate: add an index on `orderNumber` on every shard (each shard still runs the query, but as an index lookup). Permanent: include the customer id in the lookup (the page knows the customer), or create a lookup collection `orderIndex` keyed by `orderNumber` that stores the customer id, or choose a shard key that serves the dominant access path.

Prevention. Review the top queries for the presence of the shard key before sharding; track scatter-gather ratio in monitoring (`serverStatus().metrics.query.scatterGather` where available).

2.17.5 Questions to expect

Questions and model answers

1. **What is a shard key and why is it important?** The field(s) that decide which shard owns a document. It determines write distribution, query targeting and the ability to split chunks; a poor key causes hot shards, jumbo chunks or scatter-gather queries.
2. **Ranged versus hashed sharding?** Ranged keeps neighbouring values together (good range queries, risk of hot spots with monotonic keys). Hashed spreads values uniformly (good write distribution, range queries become scatter-gather).
3. **What are cardinality, frequency and monotonicity?** The number of distinct values, how evenly they occur, and whether new values only increase. Low cardinality, skewed frequency and monotonic growth each cause imbalance.
4. **What does mongos do?** It routes operations using the cached chunk map, merges results from several shards, and hides the cluster layout from the application.
5. **How does the balancer work?** It runs on the config server primary, compares data size per shard, and migrates ranges when the difference exceeds three range sizes, one migration per shard at a time.
6. **What is a jumbo chunk?** A chunk that exceeds the range size and cannot be split or moved, usually because all documents share one key value. Remedy: refine the shard key or reshard.
7. **Can the shard key be changed?** Yes: refine it by adding fields (4.4), or reshard to a different key online (5.0). Field values can be updated (4.2) except `_id`.

sreram

8. **Why must unique indexes include the shard key?** Each shard enforces uniqueness only locally; including the shard key guarantees that equal values land on the same shard.
9. **What does zone sharding provide?** Placement of key ranges on chosen shards, for data residency or hardware tiers.

KEY POINTS

The shard key is the most consequential design decision of a sharded cluster.

- Good keys have high cardinality, even frequency, non-monotonic writes and appear in the common filters.
- Chunks default to 128 MB; the balancer evens out stored data, not insert load.
- Queries with the shard key are targeted; others are broadcast to all shards.
- Zones pin key ranges to shards; resharding and key refinement correct a wrong choice at a cost in time and disk space.

2.18 MongoDB Storage and Operations

Operating MongoDB means understanding where data lives, what must fit in memory, how a write reaches the disk, how to take a backup that restores, how to find the operation that is slow right now, and how to close the default security gaps. This chapter explains the WiredTiger storage engine, sizing by working set, backup methods, monitoring tools and the security controls that matter, and it ends with a decision path for a slow cluster. Atlas, the managed service, is mentioned only where it changes a recommendation; the facts below apply to self-managed deployments.

2.18.1 Concept and mechanism

WiredTiger

WiredTiger has been the default storage engine since MongoDB 3.2. It organises each collection and each index as a B-tree file (`collection-*.wt`, `index-*.wt`) and keeps two representations of the data: in-memory pages in its own cache, and compressed blocks on disk.

- **Cache.** The default size is the larger of 50 percent of (RAM minus 1 GB) and 256 MB. The cache holds uncompressed pages for collections and indexes, plus dirty pages waiting to be written. The operating system file cache holds the compressed blocks as a second level. Pages are evicted when the cache fills; when dirty data exceeds thresholds, application threads help with eviction and latency rises.
- **Journal.** Every change is first recorded in a write-ahead journal. The journal is flushed to disk about every 100 ms, and immediately for writes with `j: true` or `w: "majority"` (which implies the journal on a majority). After a crash, mongod replays the journal from the last checkpoint.
- **Checkpoints.** About every 60 seconds WiredTiger writes a consistent snapshot of all data to the data files. The previous checkpoint stays valid until the new one is complete, so the data files are always consistent as of some checkpoint.
- **Compression.** Collections use snappy block compression by default (zstd and zlib are options; time series collections default to zstd). Indexes use prefix compression, which stores shared key prefixes once.
- **Concurrency.** Locking is at document level. Operations that touch different documents of the same collection proceed in parallel; conflicting writes to one document are retried internally. Since MongoDB 7.0 the number of concurrent storage transactions (read and write tickets) is adjusted dynamically up to a maximum of 128 each.

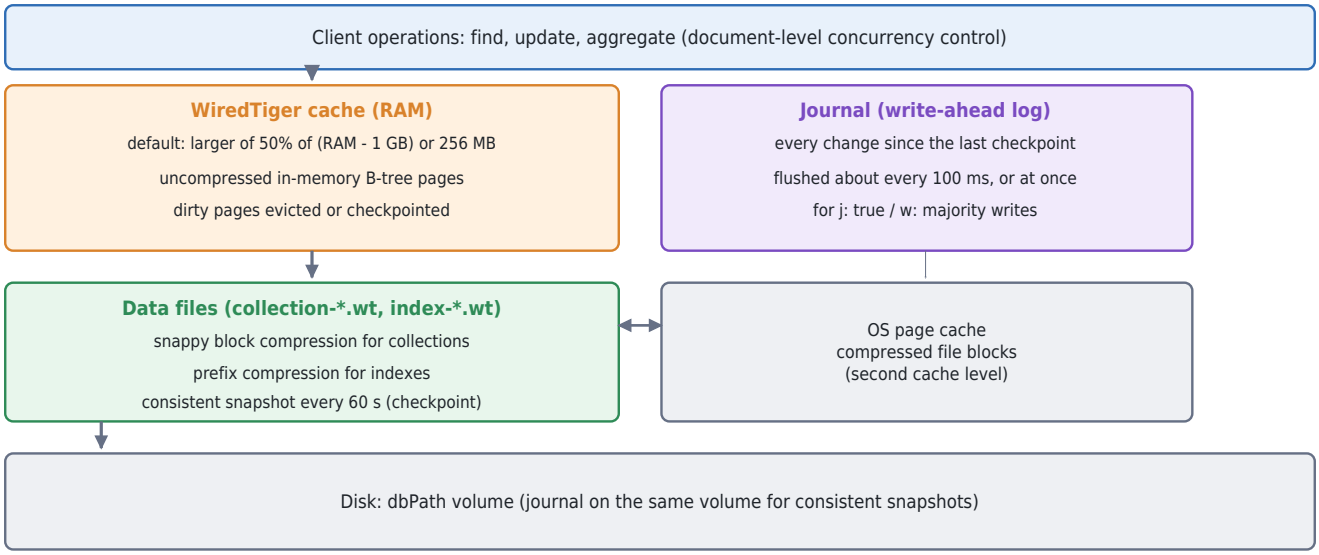


Figure 2.82. Writes modify pages in the WiredTiger cache and are recorded in the journal; checkpoints write the pages to compressed data files, and the operating system cache holds the compressed blocks.

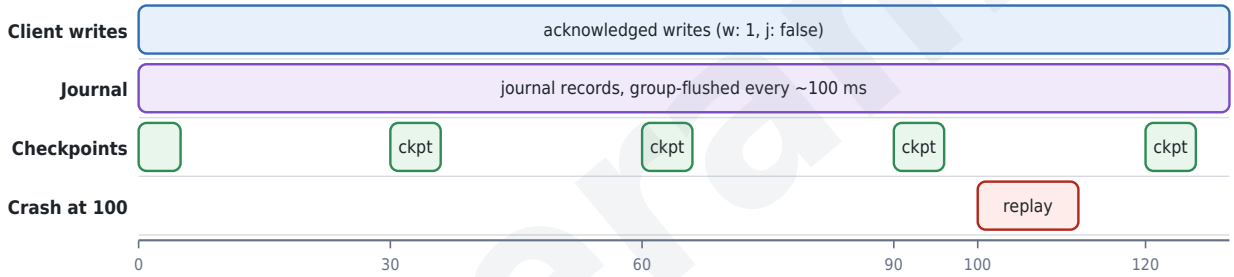


Figure 2.83. Writes are protected by the journal between checkpoints; after a crash at time 100 the engine starts from the checkpoint at time 90 (the last completed) and replays the journal for the changes after it. Illustrative times in seconds.

A crash with the default settings can lose at most the last journal interval (about 100 ms) of writes acknowledged with `w: 1` and no `j`. For `w: "majority"` the loss window does not exist on a majority of members. This matters only for a standalone or for `w: 1`; in a replica set the surviving members hold the data.

Working set and memory

The working set is the data and indexes that the workload touches regularly. When it fits in the WiredTiger cache plus the operating system cache, reads are served from memory; when it does not, every cache miss costs a disk read of a page, one or two orders of magnitude slower. Indexes matter most: an index that does not fit forces page reads at every level of every lookup. A practical sizing rule is that all indexes and the hot part of the data fit in memory. The size of the indexes is reported by `db.collection.stats().indexSizes`, and the cache occupancy by `serverStatus().wiredTiger.cache`.

The access pattern determines how much memory is enough. A model run on the build machine shows the effect: an LRU cache in front of 200,000 pages (a page per document), with accesses drawn from a skewed (Zipf) distribution that resembles real data, versus a uniform distribution that touches everything equally. The average latency assumes 0.1 ms for a memory hit and 5 ms for a disk read, which are illustrative values.

Cache hit ratio versus cache size (real output of the model) TERMINAL

```
$ python3 cache_sim.py
cache%  zipf(s=1.0)  uniform   avg latency zipf (0.1ms mem, 5ms disk)  uniform
   2%     59.7%       2.0%   2.07 ms   4.90 ms
   5%     68.4%       5.0%   1.65 ms   4.75 ms
  10%     75.2%      10.0%   1.32 ms   4.51 ms
  25%     84.8%      25.0%   0.85 ms   3.77 ms
  50%     93.5%      50.0%   0.42 ms   2.55 ms
  75%     97.6%      75.0%   0.22 ms   1.32 ms
 100%    100.0%     100.0%   0.10 ms   0.10 ms
```

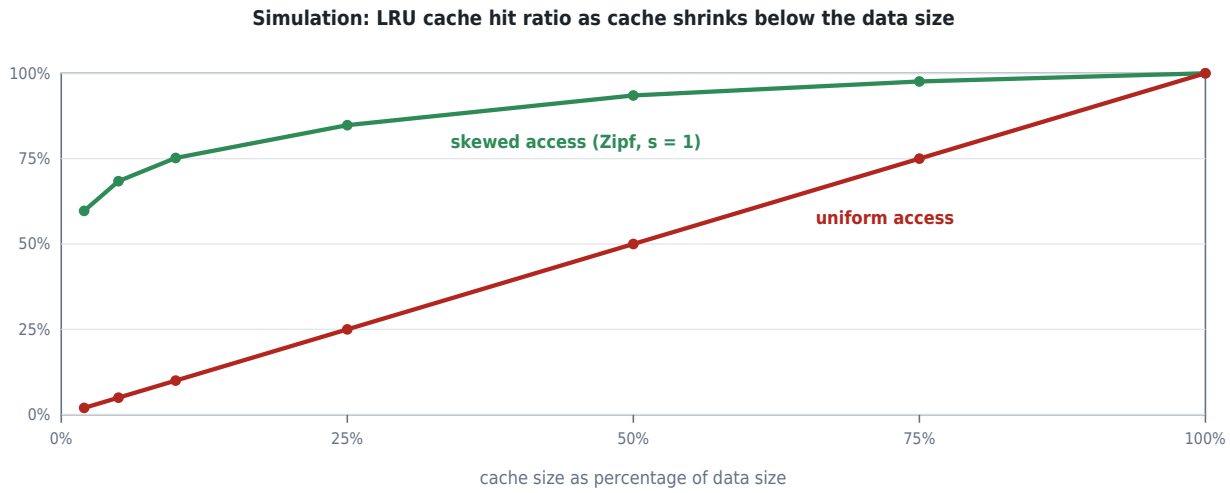


Figure 2.84. With skewed access a cache holding 25 percent of the data already serves 85 percent of reads; with uniform access the hit ratio equals the cache fraction, so any shortfall of memory costs proportionally.

The conclusion for sizing is that the shape of the access pattern matters more than the dataset size: a skewed workload tolerates a cache much smaller than the data, whereas a workload that scans or samples uniformly (analytics over the whole collection, random lookups over all customers) needs the data in memory or tolerates disk latency.

Backup and restore

Method	Consistency	Point in time	Notes
mongodump and mongorestore	logical; with --oplog a consistent dump of a replica set	the instant the dump ends	slow to dump and to restore (rebuilds indexes); suited to small data sets and migrations
Filesystem or volume snapshot	consistent when journal and data are on the same volume	the snapshot time	fast; needs a snapshot-capable volume; sharded clusters need coordinated snapshots
Snapshot plus oplog archive	consistent	any time inside the archived oplog	the basis of point-in-time recovery; offered by managed services and backup tools
Delayed member	live copy N seconds old	N seconds before the error	a safety net, not a backup

Backup methods for MongoDB.

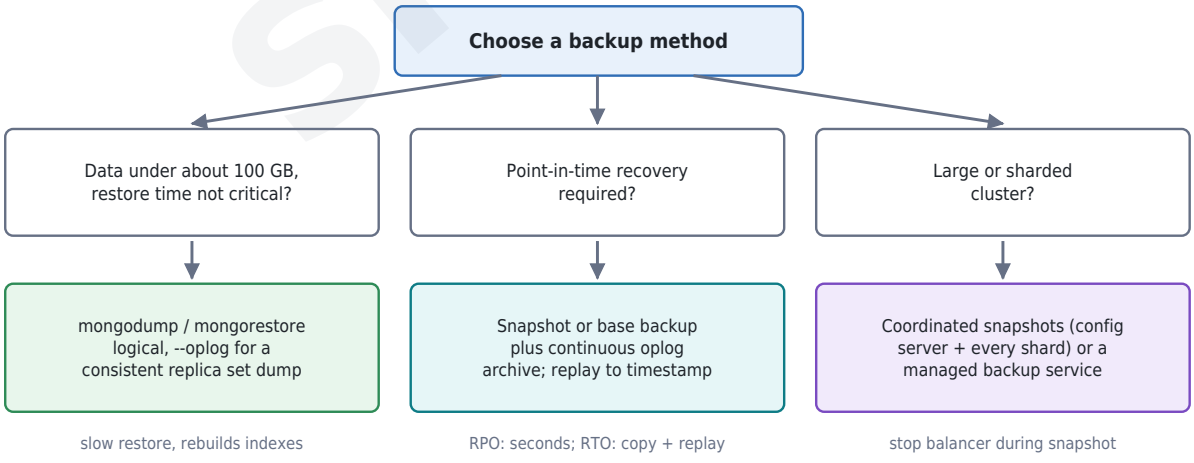


Figure 2.85. The backup method follows from the data size, the need for point-in-time recovery and whether the deployment is sharded.

For a sharded cluster a consistent backup needs all shards and the config servers captured at the same logical moment, with the balancer stopped (`sh.stopBalancer()`), or a tool that coordinates this. The recovery point objective

(RPO) is the age of the newest recoverable data; the recovery time objective (RTO) is the time to restore. A restore must be rehearsed regularly: a backup that was never restored is an assumption. A typical logical backup and restore:

Logical backup with the oplog and restore (typical output, not run on the build machine) TERMINAL

```
$ mongodump --uri "mongodb://db1:27017/?replicaSet=rs0" --oplog --gzip --out /backup/2026-10-12
2026-10-12T02:00:01.114+0000    writing shop.orders to /backup/2026-10-12/shop/orders.bson.gz
2026-10-12T02:00:41.870+0000    done dumping shop.orders (8003351 documents)
2026-10-12T02:00:42.002+0000    writing captured oplog to
$ mongorestore --uri "mongodb://newdb1:27017" --oplogReplay --gzip /backup/2026-10-12
```

Monitoring and diagnosis tools

Four sources answer most operational questions.

- `db.serverStatus()` returns counters and gauges: `opcounters` (operations per second when sampled twice), `connections` (current and available), `wiredTiger.cache` (bytes in cache, dirty bytes, maximum), `globalLock`, `queues.execution` (queued read and write tickets), `repl` and `metrics`. Monitoring systems sample it and plot the deltas.
- `db.currentOp()` lists running operations with their duration (`secs_running`), the namespace, the query and whether they are waiting for a lock (`waitingForLock`). `db.killOp(opid)` stops one.
- The database profiler records slow operations in the capped collection `system.profile` of each database. `db.setProfilingLevel(1, { slowms: 100 })` records operations slower than 100 ms (the default threshold); level 2 records everything and is for short diagnostic windows only.
- `explain('executionStats')` shows how one query ran; the ratio `totalDocsExamined` to `nReturned` is the first indicator of a missing or poor index (see the chapter on indexes and explain).

Finding slow and blocked operations (typical output, not run on the build machine)

```
shop> db.currentOp({ active: true, secs_running: { $gt: 5 } }).inprog.map(o =>
...   ({ opid: o.opid, ns: o.ns, secs: o.secs_running, waiting: o.waitingForLock, op: o.op }))
[ { opid: 982341, ns: 'shop.orders', secs: 41, waiting: false, op: 'query' } ]
shop> db.getSiblingDB('shop').system.profile.find({ millis: { $gt: 100 } })
...   .sort({ ts: -1 }).limit(1)
[ { op: 'query', ns: 'shop.orders', command: { find: 'orders', filter: { status: 'NEW' } },
    keysExamined: 0, docsExamined: 4012331, nreturned: 118, planSummary: 'COLLSCAN',
    millis: 2874, ts: ISODate('2026-10-12T08:41:00.221Z') } ]
shop> db.serverStatus().wiredTiger.cache['tracked dirty bytes in the cache']
412763904
```

The profile entry shows the typical signature of a missing index: `docsExamined` of 4,012,331 for 118 documents returned, with plan `COLLSCAN`.

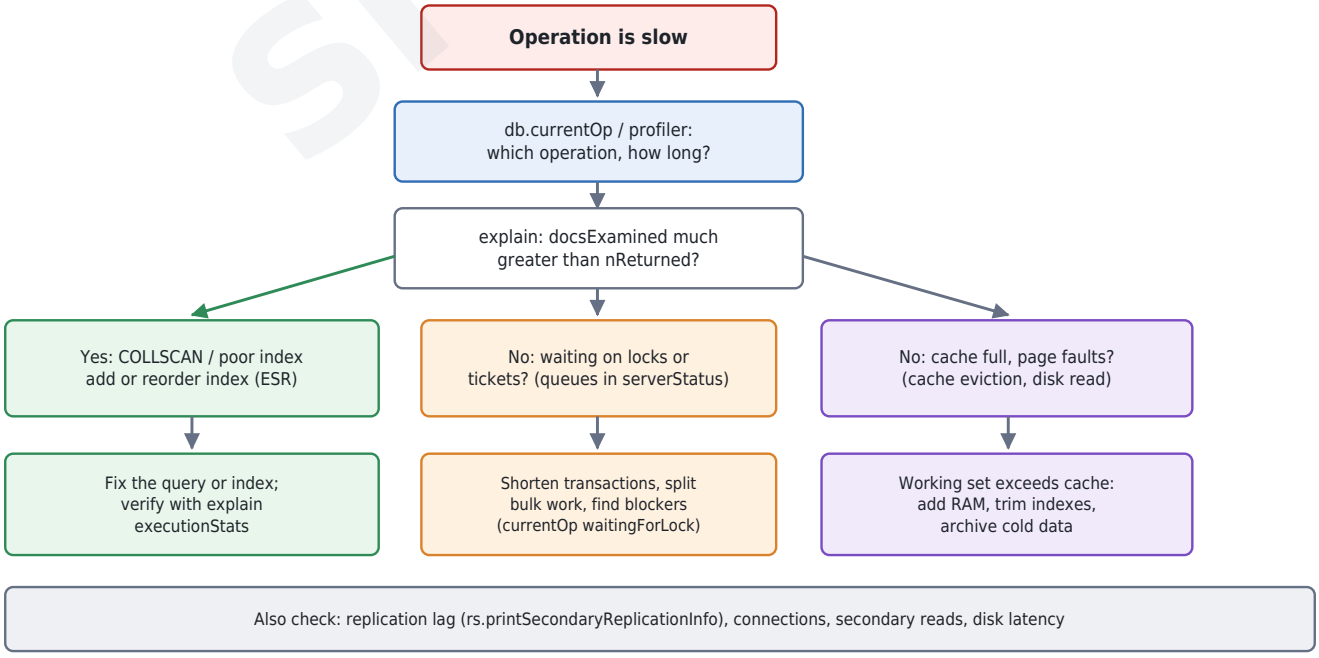


Figure 2.86. Diagnosis starts from the slow operation, tests the examined-to-returned ratio first, then lock or ticket waits, then cache pressure, and finally replication and connection limits.

Security

MongoDB started with authentication disabled by default in older versions, which produced a long list of exposed databases. The controls below are the baseline for any deployment.

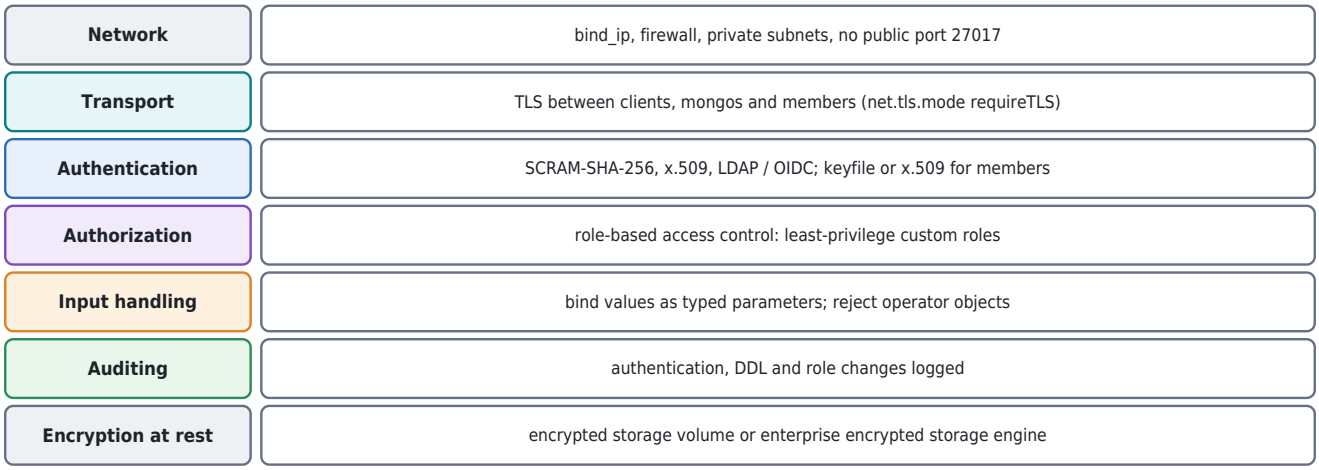


Figure 2.87. Security is layered: the network, transport encryption, authentication, role-based authorization, input handling in the application, auditing and encryption at rest each address a different threat.

- **Authentication and authorization.** `security.authorization: enabled` turns on access control; clients authenticate with SCRAM-SHA-256 (default), x.509 certificates or an external provider. Members of a replica set authenticate to one another with a keyfile or x.509 certificates. Roles grant privileges on resources: `read`, `readWrite`, `dbAdmin`, `clusterMonitor`, and custom roles. The application user receives `readWrite` on its own database only.
- **Transport encryption.** `net.tls.mode: requireTLS` encrypts client and internal traffic.
- **Network.** `net.bindIp` restricts listening addresses; the port is never reachable from the internet; a firewall or private subnet restricts clients to the application hosts.
- **Encryption at rest.** An encrypted volume protects against lost disks; MongoDB Enterprise offers an encrypted storage engine and client-side field-level encryption for selected fields.
- **Auditing.** Authentication events, DDL and role changes are logged (Enterprise audit log, or Atlas).

Creating a least-privilege application user (typical output, not run on the build machine)

```
admin> use shop
shop> db.createUser({ user: 'orders_app', pwd: passwordPrompt(),
...   roles: [{ role: 'readWrite', db: 'shop' }] })
{ ok: 1 }
```

Operator injection

MongoDB queries are documents, so a value that the application takes from a request can itself be an operator document. A login endpoint that passes the parsed JSON body straight into a filter accepts `{ "username": "admin", "password": { "$ne": null } }`: the password condition becomes "not equal to null" and matches every user. This is the NoSQL counterpart of SQL injection; it needs no string concatenation at all.

VulnerableLogin.java (fragment, not run: Spring Data MongoDB not available) JAVA

```java
// Vulnerable: the body is a Map, so the password value can be {"$ne": null}
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody Map<String, Object> body) {
    Query q = new BasicQuery(new Document(body));          // operators pass through
    User u = mongoTemplate.findOne(q, User.class);
    return u != null ? ResponseEntity.ok(u.getName()) : ResponseEntity.status(401).build();
}

// Safe: typed DTO, fixed fields, hashed password comparison
public record LoginRequest(@NotBlank String username, @NotBlank String password) {}

@PostMapping("/login")
public ResponseEntity<?> login(@Valid @RequestBody LoginRequest req) {
```

VulnerableLogin.java (fragment, not run: Spring Data MongoDB not available) (continued) JAVA

```
    User u = userRepository.findByUsername(req.username()).orElse(null);
    boolean ok = u != null && passwordEncoder.matches(req.password(), u.getPasswordHash());
    return ok ? ResponseEntity.ok(u.getName()) : ResponseEntity.status(401).build();
}
```

The fix has three parts: bind request bodies to typed objects (a `String` field cannot hold an operator document), never build queries from raw maps or from JSON strings, and compare hashed passwords in application code. Server-side JavaScript (`$where`, `mapReduce` functions) can be disabled with `security.javascriptEnabled: false` when no application uses it.

2.18.2 Worked example

A cluster has a WiredTiger cache of 15 GB (a 32 GB host: 0.5 times (32 minus 1) is 15.5 GB, rounded down). The orders collection has 90 GB of data and 14 GB of indexes. The hot part of the data is the last 30 days, about 8 GB. The question is whether this host is adequately sized.

1. Indexes alone (14 GB) nearly fill the cache of 15 GB. Together with the hot data (8 GB) the working set is 22 GB, larger than the cache.
2. The operating system cache holds compressed blocks: with snappy compression at roughly 2:1 for typical JSON-like documents, 8 GB of hot data occupies about 4 GB compressed. The OS cache has about 32 minus 15 minus 3 (process and connections) = 14 GB available, so the compressed hot data fits there, but every access through it needs a decompression and a copy into the WiredTiger cache.
3. Expect a cache hit ratio well below the skewed curve in the experiment: the cache is full of index pages and constant eviction takes place. `serverStatus().wiredTiger.cache['pages read into cache']` grows steadily and the dirty fraction is high.
4. Options in order of cost: drop unused indexes (`db.orders.aggregate([{ $indexStats: {} }])` shows `accesses.ops` per index; unused indexes consume cache for nothing), reduce index size with partial indexes, add RAM (64 GB host gives a 31.5 GB cache), or shard so each shard holds a smaller working set.

The first step costs nothing and is applied first; the decision to add RAM follows the measurement after cleaning the indexes.

2.18.3 Hands-on

A first look at a deployment

Health snapshot of a mongod (typical output, not run on the build machine)

```
shop> db.serverStatus().connections
{ current: 118, available: 51082, totalCreated: 40211, active: 9 }
shop> const c = db.serverStatus().wiredTiger.cache
shop> ({ max: c['maximum bytes configured'], used: c['bytes currently in the cache'],
...        dirty: c['tracked dirty bytes in the cache'] })
{ max: 16106127360, used: 15988113408, dirty: 412763904 }
shop> db.serverStatus().queues.execution
{ read: { out: 3, available: 125, totalTickets: 128 },
  write: { out: 1, available: 127, totalTickets: 128 } }
shop> rs.printSecondaryReplicationInfo()
source: db2:27017
        { syncedTo: 'Mon Oct 12 2026 08:50:01 GMT+0000', replLag: '0 secs (0 hrs) behind' }
```

The numbers read as follows: the cache is at 99 percent of its maximum, which is normal for a busy server (a full cache is not a problem by itself; high eviction and read rates are); the dirty fraction is 2.6 percent, below the eviction thresholds; tickets are plentiful, so there is no concurrency bottleneck; replication lag is zero. A low `available` ticket count alone does not indicate overload; a growing queue does.

Index usage report

Finding indexes that waste cache (typical output, not run on the build machine)

```
shop> db.orders.aggregate([{ $indexStats: {} },
...   { $project: { name: 1, ops: '$accesses.ops', since: '$accesses.since' } }])
[ { name: '_id_', ops: Long('881204551'), since: ISODate('2026-09-01T00:00:00.000Z') },
  { name: 'customerId_1_createdAt_-1', ops: Long('12044871'),
    since: ISODate('2026-09-01T00:00:00.000Z') },
  { name: 'status_1', ops: Long('0'), since: ISODate('2026-09-01T00:00:00.000Z') } ]
```

The index `status_1` was never used since the statistics began; it costs write time and cache space and is a candidate for removal (hide it first with `db.orders.hideIndex('status_1')` to confirm that nothing slows down, then drop it).

2.18.4 Real-world problems and solutions

Incident: working set exceeding RAM

Scenario. A product search service runs for months with latency of 5 ms. Over three weeks the catalogue doubles and a new index is added for a filter.

Symptoms. p95 latency rises to 200 ms. CPU is low, disk read IOPS are at the volume limit. No query plan changed.

Diagnosis.

1. `db.serverStatus().wiredTiger.cache` compared over 60 seconds: `pages read into cache` is rising rapidly and `bytes currently in the cache` sits at the maximum.
2. `db.products.stats().indexSizes` summed against the cache size: the indexes now exceed the cache.
3. OS metrics: `iostat -x` shows high read utilisation on the data volume; the page cache hit rate dropped.
4. `$indexStats` shows an index with no use.

Root cause. The working set (indexes plus hot data) no longer fits in memory, so lookups that used to hit the cache now read from disk. The access pattern is close to uniform because the new filter makes users page through the whole catalogue, which corresponds to the "uniform" column of the model: hit ratio equals the cache fraction.

Fix. Immediate: hide and drop the unused index; scale the host to more memory. Permanent: reduce index size (partial indexes, remove redundant prefixes: `{a:1}` is redundant to `{a:1,b:1}`), archive cold documents, and, when a single host cannot hold the working set, shard by a key that keeps related data together.

Prevention. Alert on cache read rates and index size relative to cache; review index usage monthly; capacity-plan memory from index size growth.

Incident: operator injection in a login endpoint

Scenario. A Spring Boot endpoint authenticates users through MongoDB and passes a request map to the query.

Symptoms. The security scan reports that sending `{"username":"admin","password":{"$ne":null}}` returns a valid session; access logs show unusual JSON bodies containing `$`-prefixed keys.

Diagnosis. Reproduce with `curl -X POST -H 'Content-Type: application/json' -d '{"username":"admin","password":{"$ne":null}}' https://host/login`; inspect the controller for `Map<String, Object>` or `BasicQuery` constructed from the body; confirm with the profiler that the logged filter contains the operator.

Root cause. Untyped input is interpreted as a query document, so the attacker chooses the operators.

Fix. Immediate: block requests with keys starting with `$` at the gateway and rotate sessions. Permanent: typed DTOs, validation, server-side password hashing and comparison (see the listing above), and the least-privilege database user so that the application account cannot read other databases.

Prevention. Static analysis rule against `BasicQuery(new Document(map))` and `Document.parse(userInput)`; security tests that send operator payloads.

Incident: restore that did not work

Scenario. A collection is dropped by mistake. The team restores last night's `mongodump`.

Symptoms. The restore takes 9 hours and the application is down; afterwards indexes are missing or the restore reports duplicate key errors because the target was not empty.

Diagnosis. `mongorestore` output and the restore log; `db.orders.getIndexes()` after the restore; timing of the dump (it was taken without `--oplog` on a busy primary, so it is not a consistent point in time across collections).

Root cause. A logical backup restores documents and then rebuilds every index, which is slow for large data; the dump was not consistent; and the procedure was never rehearsed.

Fix. Immediate: restore into a separate database to recover the dropped collection only (`--nsInclude shop.orders --nsFrom ... --nsTo ...`), then copy back. Permanent: take volume snapshots with oplog archiving for point-in-time recovery, use `--oplog` for dumps, and schedule restore drills that record the elapsed time as the real RTO.

Prevention. Keep a delayed secondary for logical errors, restrict `drop` privileges, and test the restore monthly.

2.18.5 Questions to expect

Questions and model answers

1. **What is the WiredTiger cache and how large is it by default?** The engine's cache of uncompressed pages; default is the larger of 50 percent of (RAM minus 1 GB) or 256 MB. The OS cache holds the compressed file blocks.
2. **How does MongoDB survive a crash?** Changes are written to the journal before they are acknowledged with `j: true` or `w: "majority"`; after a crash, the data is restored from the last checkpoint (taken every 60 seconds) and the journal is replayed.
3. **What is the working set?** The data and indexes touched by the regular workload. Performance degrades sharply when it exceeds available memory.
4. **How do you find a slow query?** The profiler (`system.profile`), the slow query log, `db.currentOp()` for running operations, and `explain('executionStats')` to compare `docsExamined` with `nReturned`.
5. **mongodump or snapshots?** `mongodump` is logical and portable but slow to restore; snapshots are fast and, combined with oplog archiving, give point-in-time recovery. Both need rehearsed restores.
6. **What is NoSQL injection and how is it prevented?** Operator objects in user input (`$ne`, `$gt`, `$where`) change the query semantics. Prevent with typed DTOs, never passing raw maps into queries, validating input, and disabling server-side JavaScript.
7. **What are the minimum security settings?** Authentication enabled with least-privilege roles, TLS, restricted `bindIp` and network access, encrypted storage, and auditing.
8. **Why can a full WiredTiger cache be normal?** The cache is meant to be used; the signals of trouble are a high rate of pages read into cache, heavy eviction and a high dirty percentage, not the fill level.

KEY POINTS

WiredTiger keeps uncompressed pages in a cache, protects changes with a journal and writes checkpoints every 60 seconds.

- Size memory for indexes plus the hot data; skewed access tolerates less memory than uniform access.
- Backups are logical (`mongodump`) or snapshot based; point-in-time recovery needs oplog archiving; rehearse restores.
- `serverStatus`, `currentOp`, the profiler and `explain` cover most diagnosis; start with examined-to-returned ratio.
- Enable authentication, TLS and least privilege, and never pass untyped request data into a query.

2.19 Spring Data MongoDB

Spring Data MongoDB maps Java objects to BSON documents and offers two programming models over the MongoDB Java driver: repositories, which derive queries from method names, and `MongoTemplate`, which exposes the full query, update and aggregation API. A Spring Boot application obtains a configured `MongoClient`, a `MongoTemplate` and repository support from the `spring-boot-starter-data-mongodb` dependency and a connection URI. The abstraction is thin: every repository method becomes a driver call, and every design decision from the previous chapters (embedding, write concern, indexes, transactions) remains the responsibility of the developer. This chapter shows a complete service, explains where the abstraction hides cost (references, lazy loading, missing indexes, transactions), and lists the configuration that production needs. No Spring or MongoDB artifacts are available on the build machine, so all code in this chapter is unverified against a running system: it was written against the documented API of Spring Boot 3.x with Spring Data MongoDB 4.x (the API shown is also valid in the 5.x line that follows it) and checked only for Java syntax by the book's linter. Class names and annotations should be checked against the version in use.

2.19.1 Concept and mechanism

Layers of the stack

A call such as `orderRepository.findByStatus(NEW)` passes through the repository proxy (query derivation), `MongoTemplate` (execution), the `MappingMongoConverter` (translation between Java objects and `Document`), and the driver (connection pool, server selection, retries), before reaching mongod or mongos. The converter writes the field `_class` with the fully qualified class name into each document so that polymorphic reads can instantiate the right type; this is a frequent cause of trouble after package renames, because stored documents still carry the old name.

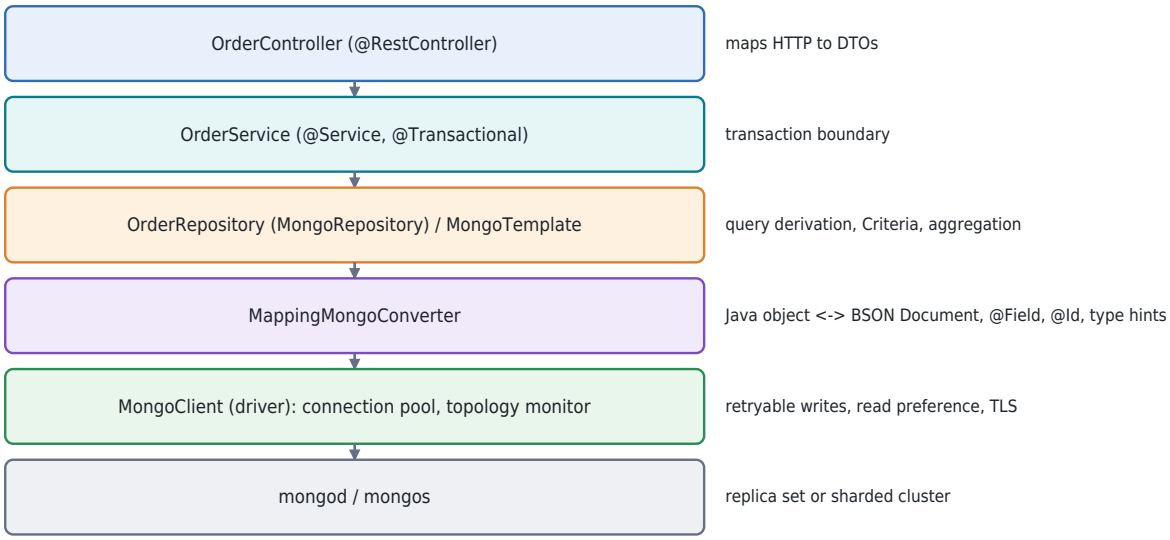


Figure 2.88. A repository call is translated by query derivation, executed by MongoTemplate, converted between objects and BSON, and sent through the driver's pool to the cluster.

Mapping annotations

Annotation	Effect
`@Document(collection = "orders")`	Marks an aggregate root and names the collection (default: decapitalised class name)
`@Id`	The `_id` field; a `String` or `ObjectId` property is generated by the driver on save
`@Field("cust_id")`	Stored field name; shorter names save space in every document
`@Indexed`	Single-field index (created only when auto index creation is enabled)
`@CompoundIndex(def = "{'customerId': 1, 'createdAt': -1}")`	Compound index on the class
`@Version`	Optimistic locking: the version is checked and incremented on every save
`@CreatedDate, @LastModifiedDate`	Filled by auditing when `@EnableMongoAuditing` is present
`@Transient`	Property is not persisted
`@DBRef`	Stores a reference to a document in another collection and loads it (eagerly by default, lazily with `lazy = true`)
`@DocumentReference`	Reference with a configurable lookup (3.3 and later); stores a plain id, loads through a query

Core mapping annotations.

Repositories, templates and aggregation

`MongoRepository<T, ID>` provides `save`, `findById`, `findAll(Pageable)`, `deleteById` and `count`, plus derived queries: the method name `findByCustomerIdAndStatusOrderByCreatedAtDesc` is parsed into a filter and a sort. `@Query` takes a JSON filter with positional parameters (`?0`), and `@Aggregation` accepts pipeline stages as JSON. `MongoTemplate` offers the programmatic API: `Query` with `Criteria`, `Update` for partial updates (`$set`, `$inc`, `$push`), `findAndModify` for atomic read-modify-write, and `aggregate` with a typed `Aggregation` pipeline. Partial updates through `Update` are preferable to `save(entity)` for hot fields, because `save` replaces the whole document and can overwrite concurrent changes to other fields.

sreeram

Transactions in Spring

`@Transactional` on a service method uses the registered `PlatformTransactionManager`. For MongoDB the manager is `MongoTransactionManager`, and it must be declared as a bean: Spring Boot does not register one automatically, so without it `@Transactional` has no effect on MongoDB operations (the methods simply run without a transaction). The manager starts a driver session and a transaction at method entry, binds the session to the thread, and commits at normal return or aborts on a runtime exception (checked exceptions do not trigger rollback unless `rollbackFor` says so, as for any Spring transaction). Transactions require a replica set or sharded cluster; on a standalone mongod, the first operation fails with "Transaction numbers are only allowed on a replica set member or mongos". The same rules as for relational transactions apply: the proxy is bypassed by self-invocation, so a `@Transactional` method called from another method of the same class does not start a transaction.

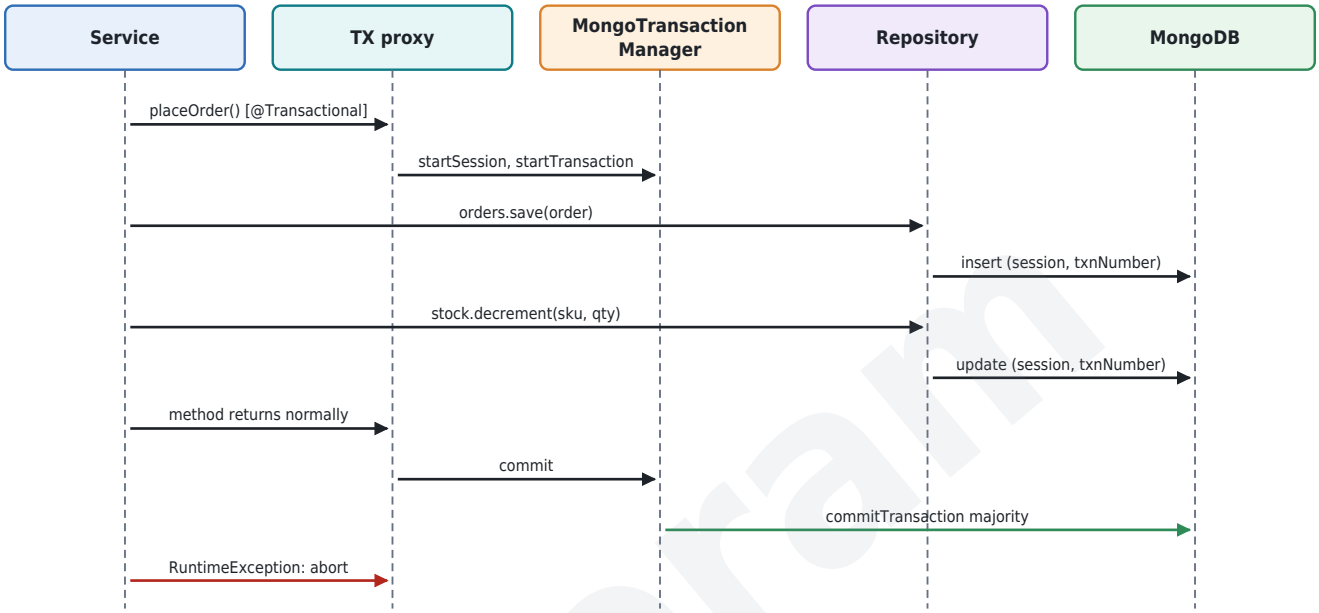


Figure 2.89. The transaction proxy asks the MongoTransactionManager to start a session; repository calls inside the method use that session; a normal return commits with the configured write concern and a runtime exception aborts.

2.19.2 Worked example

The example is the order service of the e-commerce domain used in this book. It consists of the build file entries, the entity, the repository, a service with a transaction, a controller and the configuration. The orders embed their items and a snapshot of the customer's name, following the design rules of the schema design chapter; stock is a separate collection because it is updated independently.

Dependencies and configuration

pom.xml (fragment, unverified) XML

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
</dependencies>
```

application.properties (unverified) PROPERTIES

```properties
spring.data.mongodb.uri=mongodb://db1:27017,db2:27017,db3:27017/shop?replicaSet=rs0\
  &w=majority&retryWrites=true&maxPoolSize=50&minPoolSize=5\
  &serverSelectionTimeoutMS=15000&connectTimeoutMS=5000
spring.data.mongodb.auto-index-creation=false
```

The URI carries the write concern, the pool bounds and the timeouts; `auto-index-creation` is false by default and stays false in production, where indexes are created by a controlled migration so that an application start never triggers a large index build. The driver pool defaults to a maximum of 100 connections per server.

Entity

Order.java (unverified) JAVA

```java
import java.math.BigDecimal;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.Id;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.annotation.Version;
import org.springframework.data.mongodb.core.index.CompoundIndex;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "orders")
@CompoundIndex(name = "customer_created", def = "{'customerId': 1, 'createdAt': -1}")
public class Order {

    public enum Status { NEW, PAID, SHIPPED, CANCELLED }

    public record Item(String sku, String name, int qty, BigDecimal unitPrice) {}

    @Id
    private String id;

    private long customerId;
    private String customerName;            // snapshot copy: extended reference pattern

    @Indexed
    private Status status = Status.NEW;

    private List<Item> items = new ArrayList<>();
    private BigDecimal total = BigDecimal.ZERO;

    @Version
    private Long version;

    @CreatedDate
    private Instant createdAt;

    @LastModifiedDate
    private Instant updatedAt;

    public String getId() { return id; }
    public long getCustomerId() { return customerId; }
    public void setCustomerId(long customerId) { this.customerId = customerId; }
    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) { this.customerName = customerName; }
    public Status getStatus() { return status; }
    public void setStatus(Status status) { this.status = status; }
    public List<Item> getItems() { return items; }
    public BigDecimal getTotal() { return total; }
    public void setTotal(BigDecimal total) { this.total = total; }
    public Long getVersion() { return version; }
    public Instant getCreatedAt() { return createdAt; }
}
```

What to notice: `BigDecimal` is stored as a string by default in Spring Data MongoDB unless a Decimal128 representation is configured through the converter customisation (`MongoCustomConversions`; recent versions offer a BigDecimal representation option, to be confirmed for the version in use), so arithmetic and range queries on `total` need that configuration; the field `version` makes `save` fail with `OptimisticLockingFailureException` when two writers update the same order concurrently; the enum is stored by name.

Repository

OrderRepository.java (unverified) JAVA

```java
import java.util.List;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.mongodb.repository.Aggregation;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.data.mongodb.repository.Query;

public interface OrderRepository extends MongoRepository<Order, String> {

    List<Order> findByCustomerIdOrderByCreatedAtDesc(long customerId, Pageable page);

    Page<Order> findByStatus(Order.Status status, Pageable page);

    @Query("{ 'status': ?0, 'items.sku': ?1 }")
    List<Order> findByStatusAndSku(Order.Status status, String sku);

    @Aggregation(pipeline = {
        "{ $match: { status: { $in: ['PAID', 'SHIPPED'] } } }",
        "{ $group: { _id: '$customerId', revenue: { $sum: '$total' }, orders: { $sum: 1 } } }",
        "{ $sort: { revenue: -1 } }",
        "{ $limit: 10 }"
    })
    List<CustomerRevenue> topCustomers();

    record CustomerRevenue(long id, double revenue, int orders) {}
}
```

The first method derives the filter `{ customerId: ?, }` and the sort `{ createdAt: -1 }` and uses the compound index `customer_created`. Paging with `Pageable` translates into `skip` and `limit`; for deep pages the offset cost described in the indexes chapter applies, so a range filter on `createdAt` or `_id` is preferable for long lists. The `@Aggregation` method is the repository form of the revenue pipeline from the aggregation chapter. (Note: in `topCustomers`, the `$sum` of a total stored as a string yields 0; see the `BigDecimal` remark above.)

sreeram

Service with a transaction

OrderService.java (unverified) JAVA

```java
import java.math.BigDecimal;
import java.util.List;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;
import org.springframework.data.mongodb.core.query.Update;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    private final OrderRepository orders;
    private final MongoTemplate template;

    public OrderService(OrderRepository orders, MongoTemplate template) {
        this.orders = orders;
        this.template = template;
    }

    public static class OutOfStockException extends RuntimeException {
        public OutOfStockException(String sku) { super("insufficient stock for " + sku); }
    }

    @Transactional
    public Order placeOrder(long customerId, String customerName, List<Order.Item> items) {
        BigDecimal total = BigDecimal.ZERO;
        for (Order.Item item : items) {
            Query q = new Query(Criteria.where("sku").is(item.sku()).and("qty").gte(item.qty()));
            var result = template.updateFirst(q, new Update().inc("qty", -item.qty()), "stock");
            if (result.getModifiedCount() == 0) {
                throw new OutOfStockException(item.sku());      // RuntimeException: abort
            }
            total = total.add(item.unitPrice().multiply(BigDecimal.valueOf(item.qty())));
        }
        Order order = new Order();
        order.setCustomerId(customerId);
        order.setCustomerName(customerName);
        order.getItems().addAll(items);
        order.setTotal(total);
        return orders.save(order);
    }

    @Transactional
    public void markPaid(String orderId) {
        Query q = new Query(Criteria.where("_id").is(orderId).and("status").is(Order.Status.NEW));
        Update paid = new Update().set("status", Order.Status.PAID);
        var result = template.updateFirst(q, paid, Order.class);
        if (result.getModifiedCount() == 0) {
            throw new IllegalStateException("order not found or not in state NEW: " + orderId);
        }
    }
}
```

What to notice: the stock decrement uses a conditional update (`qty >= requested`) so that two concurrent orders cannot both take the last unit, even without a transaction, because the update of one document is atomic. The transaction adds the property that a failure in the third item restores the stock taken for the first two. `markPaid` is a single-document write and does not need `@Transactional`; the annotation is kept for the example and costs a session start. The state transition `NEW` to `PAID` is expressed as a filter, which makes it an atomic compare-and-set.

Controller, configuration, auditing

OrderController.java (unverified) JAVA

```java
import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import java.math.BigDecimal;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    public record ItemDto(@NotBlank String sku, @NotBlank String name, @Min(1) int qty,
                          BigDecimal unitPrice) {}
    public record PlaceOrderRequest(long customerId, @NotBlank String customerName,
                                    @NotEmpty List<@Valid ItemDto> items) {}

    private final OrderService service;

    public OrderController(OrderService service) { this.service = service; }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public String place(@Valid @RequestBody PlaceOrderRequest req) {
        List<Order.Item> items = req.items().stream()
                .map(i -> new Order.Item(i.sku(), i.name(), i.qty(), i.unitPrice())).toList();
        return service.placeOrder(req.customerId(), req.customerName(), items).getId();
    }

    @ExceptionHandler(OrderService.OutOfStockException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public String outOfStock(OrderService.OutOfStockException e) { return e.getMessage(); }
}
```

MongoConfig.java (unverified) JAVA

```java
import com.mongodb.WriteConcern;
import java.util.concurrent.TimeUnit;
import org.springframework.boot.autoconfigure.mongo.MongoClientSettingsBuilderCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.mongodb.MongoDatabaseFactory;
import org.springframework.data.mongodb.MongoTransactionManager;
import org.springframework.data.mongodb.config.EnableMongoAuditing;

@Configuration
@EnableMongoAuditing
public class MongoConfig {

    @Bean
    MongoTransactionManager transactionManager(MongoDatabaseFactory factory) {
        return new MongoTransactionManager(factory);
    }

    @Bean
    MongoClientSettingsBuilderCustomizer mongoSettings() {
        return builder -> builder
                .writeConcern(WriteConcern.MAJORITY)
                .applyToConnectionPoolSettings(p -> p
                        .maxSize(50)
                        .minSize(5)
                        .maxWaitTime(2, TimeUnit.SECONDS));
    }
}
```

The customizer sets the properties that cannot be expressed as readable URI options: majority write concern and a short wait for a pooled connection (`maxWaitTime`), so that pool exhaustion surfaces as an error after two seconds and not as a hung request thread.

References: @DBRef versus embedding

Reference variants (fragment, unverified) JAVA

```java
// 1) @DBRef: stores {$ref: 'customers', $id: ...}; every load issues an extra query
@Document("orders_ref")
public class OrderWithRef {
    @Id private String id;
    @DBRef private Customer customer;                 // eager: one query per order
    @DBRef(lazy = true) private Customer lazyCustomer; // proxy: query on first access
}

// 2) @DocumentReference: stores the plain id; can be resolved by $lookup in aggregations
@Document("orders_docref")
public class OrderWithDocRef {
    @Id private String id;
    @DocumentReference private Customer customer;
}

// 3) Extended reference (recommended for display data): id plus the fields that are read
@Document("orders")
public class OrderWithSnapshot {
    @Id private String id;
    private long customerId;
    private String customerName;
}
```

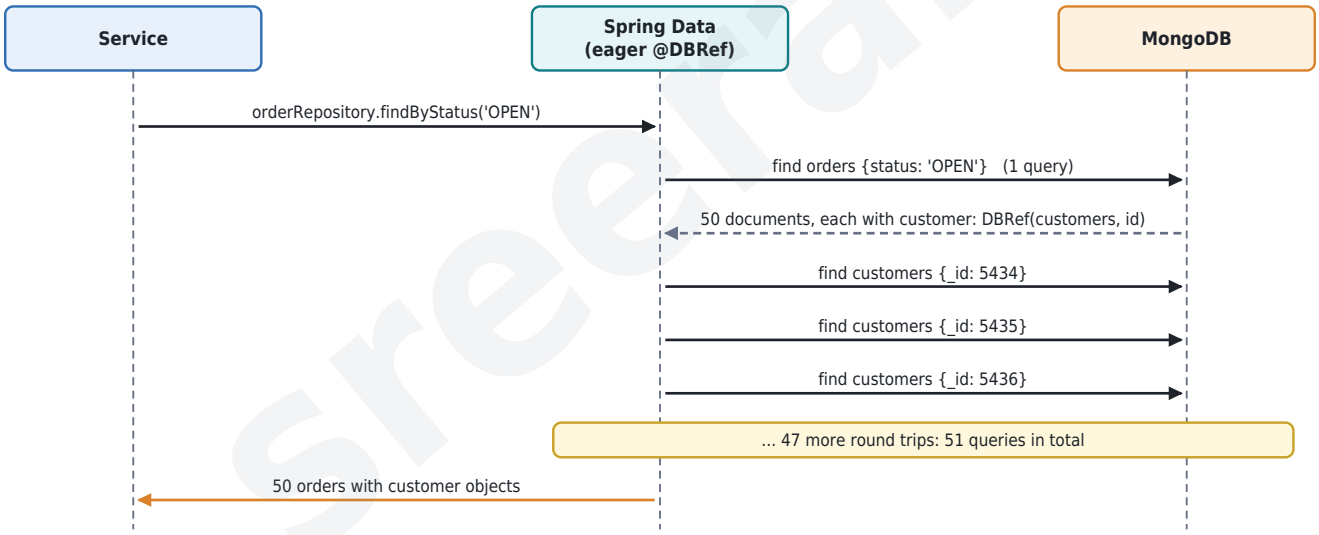


Figure 2.90. Loading 50 orders with an eager @DBRef customer issues one query for the orders and one query per customer, 51 round trips for one request.

Testing

Repository and service tests need a real MongoDB because query semantics, indexes and transactions are server behaviour. Two options exist, both described here and not run on the build machine. The first uses Testcontainers: the class `MongoDBContainer` starts a single-node replica set (which is what transactions require), and in Spring Boot 3.1 and later the annotation `@ServiceConnection` wires the application to it. The second embeds a MongoDB binary through Flapdoodle, which is lighter but trails the server version. Unit tests of controllers mock the service and need no database.

OrderRepositoryIT.java (unverified; requires Docker and Testcontainers) JAVA

```java
import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.mongo.DataMongoTest;
import org.springframework.data.domain.PageRequest;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
```

OrderRepositoryIT.java (unverified; requires Docker and Testcontainers) (continued) JAVA

```java
import org.testcontainers.containers.MongoDBContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

@DataMongoTest
@Testcontainers
class OrderRepositoryIT {

    @Container
    @ServiceConnection
    static MongoDBContainer mongo = new MongoDBContainer("mongo:7.0");

    @Autowired
    OrderRepository repository;

    @Test
    void savesAndFindsByStatus() {
        Order order = new Order();
        order.setCustomerId(5434L);
        repository.save(order);
        var page = repository.findByStatus(Order.Status.NEW, PageRequest.of(0, 10));
        assertThat(page.getTotalElements()).isEqualTo(1);
    }
}
```

2.19.3 Hands-on

The queries behind the repository

Enabling the logger `org.springframework.data.mongodb.core.MongoTemplate` at `DEBUG` prints each query that Spring Data sends. The log lines below are typical output (not run); they show the filters that the repository methods generate.

Log output with MongoTemplate at DEBUG (typical output, not run)

```
find using query: { "customerId" : 5434 } for class: class Order in collection: orders
find using query: { "status" : "NEW", "items.sku" : "A-100" } in collection: orders
Executing aggregation: [{ "$match" : { "status" : { "$in" : ["PAID", "SHIPPED"] } } }, ..
```

Creating the indexes outside the application

With `auto-index-creation=false` the indexes that the annotations describe must exist from another source. A migration script run by the deployment pipeline (a shell script, or a migration library such as Mongock) creates them. In a replica set, indexes build on every member; on a large collection the build runs in the background of the server but uses I/O and cache.

Index migration (typical output, not run on the build machine)

```
shop> db.orders.createIndex({ customerId: 1, createdAt: -1 }, { name: 'customer_created' })
customer_created
shop> db.orders.createIndex({ status: 1 }, { name: 'status_1' })
status_1
shop> db.orders.getIndexes().map(i => i.name)
[ '_id_', 'customer_created', 'status_1' ]
```

Detecting N+1 from the server side

The server's profiler or a monitoring counter shows the pattern independent of the framework: many `find` commands on the referenced collection with `_id` filters within one request.

Profiler view of an N+1 pattern (typical output, not run on the build machine)

```
shop> db.setProfilingLevel(2)
shop> db.system.profile.aggregate([
...   { $match: { op: 'query', ns: 'shop.customers' } },
...   { $group: { _id: { $dateTrunc: { date: '$ts', unit: 'second' } }, queries: { $sum: 1 } } },
...   { $sort: { _id: -1 } }, { $limit: 3 } ])
[ { _id: ISODate('2026-10-12T09:12:44.000Z'), queries: 5100 },
  { _id: ISODate('2026-10-12T09:12:43.000Z'), queries: 4980 },
  { _id: ISODate('2026-10-12T09:12:42.000Z'), queries: 5075 } ]
```

About 5,000 point queries per second on `customers` while the endpoint receives 100 requests per second is the signature: 50 customer lookups per request.

2.19.4 Real-world problems and solutions

Incident: N+1 queries from @DBRef

Scenario. The order list endpoint returns 50 orders with the customer's name. The `Order` class holds `@DBRef Customer customer`. The endpoint was fast in development with 5 orders.

Symptoms. In production the endpoint takes 400 ms at p50; MongoDB CPU is high with a very high query rate on `customers`; the application log with MongoTemplate at DEBUG prints dozens of `find using query: { "_id" : ... }` lines per request.

Diagnosis. In order:

1. Count the queries of one request in the application log (DEBUG logger above): 51 for 50 orders.
2. Confirm from the server: the profiler query shown above, or `db.serverStatus().opcounters.query` sampled during one request.
3. Inspect the mapping: `@DBRef` without `lazy` loads the reference eagerly, one query per referenced document, and lazy loading only moves the same queries to the first access.

Root cause. A DBRef is resolved by the client, one lookup per reference; MongoDB performs no join on read. The N+1 pattern of Hibernate lazy loading appears here in a different place, with a network round trip per reference.

Fix. Immediate: replace the data that the list needs by a snapshot in the order (`customerId`, `customerName`), written at order creation (extended reference pattern); the list endpoint then runs one query. If the live customer data is required, resolve it with one batched query: `customerRepository.findAllById(ids)` and map in memory, or one aggregation with `$lookup`. Permanent: use `@DocumentReference` or plain ids for references, embed the fields that are read with the parent, and keep the number of queries per request as a metric.

Prevention. A request-level assertion in integration tests on the number of commands (using a `CommandListener` registered with the driver); code review rule against `@DBRef` on read-heavy paths.

Incident: @Transactional has no effect on MongoDB

Scenario. A service method debits stock and saves an order inside `@Transactional`. A test forces an exception after the stock update and expects the stock to be restored.

Symptoms. The stock stays decremented after the exception. Alternatively, on a development laptop with a standalone mongod, every call fails with `Transaction numbers are only allowed on a replica set member or mongos`.

Diagnosis.

1. Check that a `MongoTransactionManager` bean exists: `applicationContext.getBeansOfType(PlatformTransactionManager.class)`. Spring Boot does not create one for MongoDB.
2. Check how the method is called: from another method of the same class (self-invocation bypasses the proxy), or the method is `private` or `final`, or it throws a checked exception (no rollback by default).
3. Check the server topology: `db.hello()` returns no `setName` field on a standalone mongod; transactions need a replica set.

Root cause. Without a `MongoTransactionManager`, the annotation is processed against no Mongo-aware manager (or another manager, for example a JPA one, that is irrelevant to MongoDB), so the writes run outside any transaction and each commits independently. The standalone error is the server refusing transaction numbers.

Fix. Declare the transaction manager bean (see the configuration class), run development databases as single-node replica sets (`mongod --replSet rs0` followed by `rs.initiate()`), call transactional methods through the Spring proxy (from another bean), and add `rollbackFor` for checked exceptions.

Prevention. An integration test per transactional use case that forces a failure and asserts the rollback, against the same MongoDB topology class as production (Testcontainers `MongoDBContainer`).

sfeir

Incident: missing indexes in production because annotations were trusted

Scenario. The entity carries `@Indexed` and `@CompoundIndex`; the team assumes the indexes exist. In production `spring.data.mongodb.auto-index-creation` is false (the default).

Symptoms. A list endpoint that was fast on developers' machines (where the property was true) takes seconds in production; the profiler shows `COLLSCAN` with `docsExamined` in the millions.

Diagnosis. `db.orders.getIndexes()` in production lists only `_id_`; the profiler entries show `planSummary: 'COLLSCAN'`; `explain('executionStats')` of the repository's query confirms it.

Root cause. The annotations declare intent; indexes are created only when automatic creation is enabled, which has been off by default since Spring Data MongoDB 3.0.

Fix. Create the indexes with the migration script shown in the hands-on section, in a quiet period for a large collection. Permanent: version index definitions with the code (migration tool), and verify in the deployment pipeline that every annotated index exists.

Prevention. A startup check or integration test that compares `getIndexes()` with the annotated indexes; do not enable auto creation in production, because an application start would trigger index builds.

2.19.5 Questions to expect

Questions and model answers

1. **How do `MongoRepository` and `MongoTemplate` differ?** The repository derives queries from method names and is the default for simple access; `MongoTemplate` provides the full API (partial updates, `findAndModify`, aggregation, bulk operations) when derived queries are insufficient.
2. **How are transactions enabled with Spring Data MongoDB?** Register a `MongoTransactionManager` bean and annotate service methods with `@Transactional`; the deployment must be a replica set or sharded cluster.
3. **What is the N+1 problem with `@DBRef`?** Each reference is resolved by a separate client query, so loading N parents costs N+1 round trips. Embed a snapshot, batch the lookups with `findAllById`, or use `$lookup`.
4. **Why is `save` risky for hot documents?** It writes the entire document, so a concurrent change to another field can be lost; partial updates with `Update` (`$set`, `$inc`) or `@Version` avoid this.
5. **What does `@Version` do?** It adds optimistic locking: the update filter includes the expected version and a mismatch raises `OptimisticLockingFailureException`.
6. **How do you configure the connection pool?** Through URI options (`maxPoolSize`, `minPoolSize`, `maxIdleTimeMS`) or a `MongoClientSettingsBuilderCustomizer`; the wait time for a connection should be bounded.
7. **How do you test code that uses MongoDB?** Against a real server: Testcontainers with `MongoDBContainer` (a single-node replica set), or an embedded binary; mocking the repository tests only the service logic.
8. **Why is `_class` stored in documents and when does it hurt?** It enables polymorphic mapping; after a class is renamed or moved, old documents fail to map unless a `@TypeAlias` or a migration handles them.
9. **How does the application set the write concern and read preference?** In the URI (`w=majority`, `readPreference=...`) or on the client settings; `MongoTemplate` can override per template instance.

KEY POINTS

Spring Data MongoDB is a thin mapping layer: modelling, indexes, write concern and transactions remain explicit decisions.

- Declare a `MongoTransactionManager` and run on a replica set for `@Transactional` to take effect.
- Prefer embedded snapshots or batched lookups to `@DBRef`, which causes N+1 queries.
- Use partial updates and `@Version` instead of whole-document `save` on contended documents.
- Create indexes through migrations; do not rely on annotations or on automatic creation in production.
- Test against a real MongoDB replica set.

2.20 Relational versus Non-Relational: A Head-to-Head Comparison

The choice between a relational database and a document database is a choice of data model, consistency boundary and operating model, and it is often made on reputation. This chapter replaces reputation with measurement and rules. It models one domain, the e-commerce orders used throughout the book, in both ways, compares the two on queries, transactions, scale and operations, runs a benchmark on PostgreSQL 16 that includes the middle path (a JSONB column in a relational table), and gives decision rules, a migration procedure and hybrid architectures. PostgreSQL results are real (measured on the build machine, 2 CPUs, 8 GB shared with other jobs, `shared_buffers` 64 MB, so absolute timings are typical and only the ratios are meaningful). MongoDB statements are taken from the documentation and are not measured.

2.20.1 Concept and mechanism

The same domain, two models

The domain consists of customers, products and orders with several items per order. The relational model normalises it into four tables and joins them at read time. The document model stores an order as one document that embeds its items and a copy of the data that is displayed with the order (customer name and city, product name and category), because an order must show what was true when it was placed.

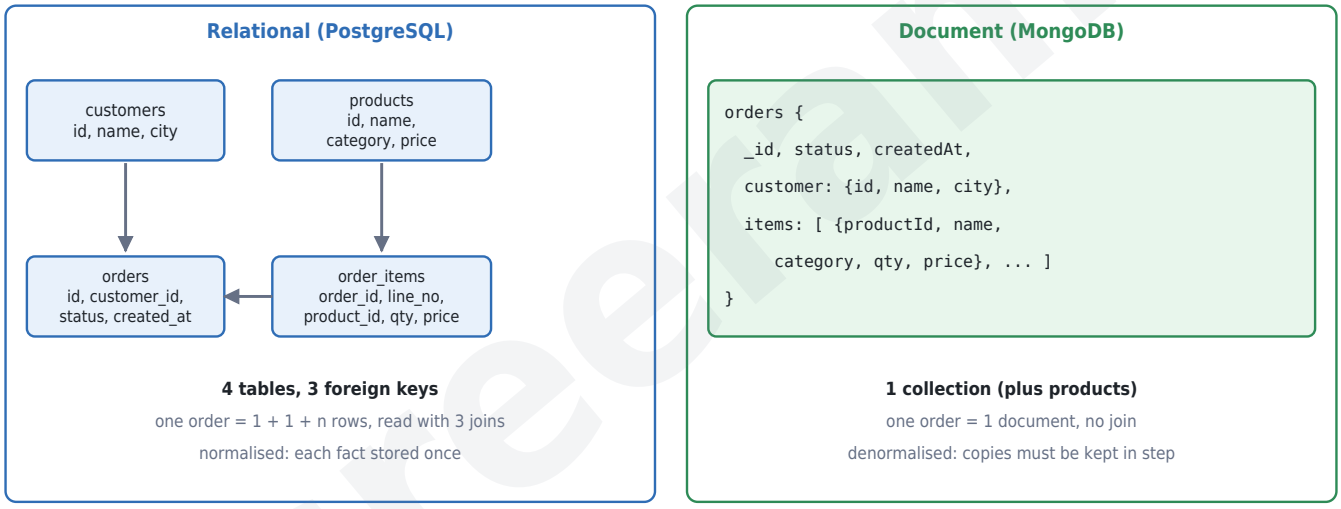


Figure 2.91. The relational model stores each fact once and reconstructs an order with joins; the document model stores the order as one self-contained aggregate with copied display data.

Relational schema (PostgreSQL) SQL

```
CREATE TABLE customers(id int PRIMARY KEY, name text, city text);
CREATE TABLE products(id int PRIMARY KEY, name text, category text, price numeric(10,2));
CREATE TABLE orders(id int PRIMARY KEY, customer_id int REFERENCES customers,
                    status text, created_at timestamptz);
CREATE TABLE order_items(order_id int REFERENCES orders, line_no int,
                         product_id int REFERENCES products, qty int,
                         unit_price numeric(10,2), PRIMARY KEY(order_id, line_no));
```

Document model (mongosh)

```
shop> db.orders.findOne({ _id: 7 })
{
  _id: 7,
  status: 'SHIPPED',
  createdAt: ISODate('2026-10-18T06:44:42.978Z'),
  customer: { id: 5434, name: 'Customer 5434', city: 'Mumbai' },
  items: [
    { productId: 235, name: 'Product 235', category: 'electronics', qty: 1, unitPrice: 785 },
    { productId: 252, name: 'Product 252', category: 'books', qty: 2, unitPrice: 424 }
  ]
}
```

The document above has the shape of the real document that the benchmark generated in PostgreSQL, so the two models contain identical information. The benchmark data set has 50,000 customers, 2,000 products, 500,000 orders and 1,250,000 order items (between one and four items per order).

Comparison by dimension

Dimension	Relational	Document (MongoDB)
Data model	Normalised tables, foreign keys, one fact in one place	Aggregates with embedded data and copies; references where needed
Schema	Declared and enforced by the server (types, NOT NULL, CHECK, FK)	Flexible; optional `$jsonSchema` validation; enforced mainly by the application
Reads of an aggregate	Joins across tables; cost grows with joins	One document read; cost grows with document size
Ad hoc queries and reporting	Strong: any join, any aggregate, mature optimiser	Aggregation pipeline and `$lookup`; weaker for many-way joins
Writes touching several entities	Multi-statement ACID transactions are the normal tool	Single-document atomicity; multi-document transactions are supported but bounded
Update of shared data (customer city)	Update one row	Update every copy, or accept stale copies
Consistency default	Serializable or read committed per transaction; immediate consistency on the primary	Majority write concern by default since 5.0; reads from secondaries are eventually consistent
Horizontal write scale	Not built in (replicas scale reads; sharding by extension or partitioning)	Built in: sharding with balancer and zones
Failover	Streaming replication with external orchestration (Patroni, managed services)	Replica set elections built in
Indexing	B-tree, hash, GIN, GiST, BRIN, partial, expression	B-tree (single, compound, multikey), text, geospatial, TTL, wildcard
Typical failure modes	Missing index, lock contention, long transactions, bloat	Unbounded arrays, hot shard key, working set above RAM, schema drift
Java access	JDBC, Spring Data JPA (entities, lazy loading, N+1)	MongoDB driver, Spring Data MongoDB (embedded documents, @DBRef N+1)

Relational (PostgreSQL, MySQL) versus document (MongoDB).

JSONB, the middle path

PostgreSQL stores JSON in the binary type `jsonb` with operators for containment (`@>`), key existence (`?`), path extraction (`->`, `->>`, `#>>`), the SQL/JSON path language (`@?`, `jsonb_path_query`), and index support: GIN indexes (`jsonb_ops` or the smaller `jsonb_path_ops`) for containment and existence queries, and B-tree expression indexes for scalar paths. A `jsonb` column inside a relational table provides flexible attributes while keeping transactions, constraints, joins and the existing operational tooling. MySQL 8 offers a `JSON` type with functional indexes on generated columns and multi-valued indexes over arrays, with a narrower feature set. The trade-offs are structural: a `jsonb` value is stored and updated as a whole, so changing one field rewrites the entire value (and its TOAST data when large); statistics are not collected inside the value, so the planner estimates containment predicates with fixed defaults; and the shape is not enforced unless a `CHECK` constraint says so.

2.20.2 Worked example

The same five tasks are run against both representations in PostgreSQL. The `orders_doc` table has two columns, `id int` and `doc jsonb`; each document is the order with embedded customer and items, built from the normalised tables with `jsonb_build_object` and `jsonb_agg` (500,000 rows, average document size 499 bytes). A GIN index `jsonb_path_ops` and a B-tree expression index on the customer id are created on the document table. Timings are `EXPLAIN (ANALYZE)` execution times after one warm-up run.

Task 1: fetch one order with its items

Relational: four tables joined SQL

```sql
SELECT o.id, o.status, c.name, i.line_no, p.name, i.qty, i.unit_price
FROM orders o JOIN customers c ON c.id = o.customer_id
              JOIN order_items i ON i.order_id = o.id
              JOIN products p ON p.id = i.product_id
WHERE o.id = 250007;

-- Document: one primary key lookup
SELECT doc FROM orders_doc WHERE id = 250007;
```

The relational plan is three nested loops over four index scans (`orders_pkey`, `customers_pkey`, `order_items_pkey`, and `products_pkey` four times) with 17 shared buffer hits and 6 reads; the execution time is 0.169 ms. The document plan is one `Index Scan using orders_doc_pkey` with 1 hit and 3 reads; the execution time is 0.049 ms. The document needs a third of the time, and the saving comes from fetching one tuple instead of seven (one order, one customer, four items, four products through a hash of repeated lookups). For a read path that serves one aggregate at a time, this is the legitimate advantage of the document model; in absolute terms both are well below a millisecond, so the benefit becomes visible only at high request rates or when the joined tables are not cached.

Task 2: search by attributes of the order and its items

The query counts orders from customers in Pune that contain a product of category toys.

Relational and document forms SQL

```sql
SELECT count(DISTINCT o.id)
FROM orders o JOIN customers c ON c.id = o.customer_id
              JOIN order_items i ON i.order_id = o.id
              JOIN products p ON p.id = i.product_id
WHERE c.city = 'Pune' AND p.category = 'toys';

SELECT count(*) FROM orders_doc
WHERE doc @> '{"customer":{"city":"Pune"},"items":[{"category":"toys"}]}';
```

Plan	Execution time	Buffers touched	Note
`Relational: parallel hash joins`	328 ms	20,642	two parallel workers, seq scans of orders and order_items
`Document, no index: seq scan`	367 ms	33,334	evaluates the containment on all 500,000 documents
`Document with GIN jsonb_path_ops`	168 ms	24,535	bitmap index scan (about 10 ms) then 24,465 heap blocks to recheck

Task 2: 33,250 matching orders.

The GIN index finds the candidate rows in about 10 ms, but the query still reads 24,465 heap blocks, because the predicate matches 6.6 percent of the table (33,250 of 500,000 rows) and the matches are spread over nearly all pages. The index brought a factor of 2.2 over the sequential scan, not a factor of 1,000, because the predicate is not selective. A selective predicate (a customer id, an order id) is where indexes dominate; for non-selective filters the cost is the number of pages that hold results.

Task 3: selective lookup by a nested field

Customer's orders: B-tree on the relational column and an expression index on the document SQL

```sql
CREATE INDEX ON orders(customer_id);
CREATE INDEX idx_doc_cust ON orders_doc (((doc->'customer'->>'id')::int));

SELECT id FROM orders WHERE customer_id = 5434;
SELECT id FROM orders_doc WHERE (doc->'customer'->>'id')::int = 5434;
```

Both queries return 10 rows in 0.07 ms (`Bitmap Heap Scan` on `orders` in 0.074 ms, `Index Scan` on `idx_doc_cust` in 0.072 ms). A single-path B-tree index on a JSONB expression performs like an ordinary index. The expression in the query must match the expression in the index exactly, including the cast.

sreeram

Task 4: analytics over all items

Revenue by category SQL

```sql
SELECT p.category, sum(i.qty * i.unit_price) AS rev
FROM order_items i JOIN products p ON p.id = i.product_id
GROUP BY 1 ORDER BY 1;

SELECT it->>'category' AS cat, sum((it->>'qty')::int * (it->>'unitPrice')::numeric) AS rev
FROM orders_doc d, jsonb_array_elements(d.doc->'items') it
GROUP BY 1 ORDER BY 1;
```

The relational query scans the narrow `order_items` table in parallel (about 125 MB of pages) and joins to the small `products` table by hash: 762 ms. The document query must read every document (all 261 MB of the table, including the document payload), expand each array with `jsonb_array_elements` (500,000 calls), extract and cast text from JSON values, and takes 2,076 ms, 2.7 times as long. This is the generic cost of analytics on a document layout: the data that the query needs is spread inside larger values, and the aggregation processes strings and casts. A column store or a relational table has no equivalent overhead. MongoDB's aggregation pipeline has the same structure (`$unwind` on items, `$group` by category) and the same character: it reads every document.

Task 5: update one field

Set the status of 20,000 orders SQL

```sql
UPDATE orders SET status = 'SHIPPED' WHERE id BETWEEN 100001 AND 120000;
UPDATE orders_doc SET doc = jsonb_set(doc, '{status}', '"SHIPPED"')
WHERE id BETWEEN 100001 AND 120000;
```

The relational update took 176 ms, the document update 833 ms (4.7 times). The cause is write amplification. The relational row is a few dozen bytes, the whole `jsonb` value about 500 bytes, and PostgreSQL writes a new version of the entire row for any change. Moreover, the document table has a GIN index over the whole value, so every update must add new index entries, and the statistics after the run show that none of the 20,000 updates were HOT updates (heap-only tuple updates that avoid index maintenance), whereas for the `orders` table the number of HOT updates was 68 of 20,000 (a likely reason for the small count is that the pages had no free space, since the table was loaded at full page density, and a HOT update needs room on the same page). In MongoDB an update of one field in a large document is likewise a rewrite of the document in the storage engine's B-tree page and an oplog entry that carries only the delta; the write-amplification argument applies to documents that are large and updated often, which is one of the reasons for the bucket and subset patterns.

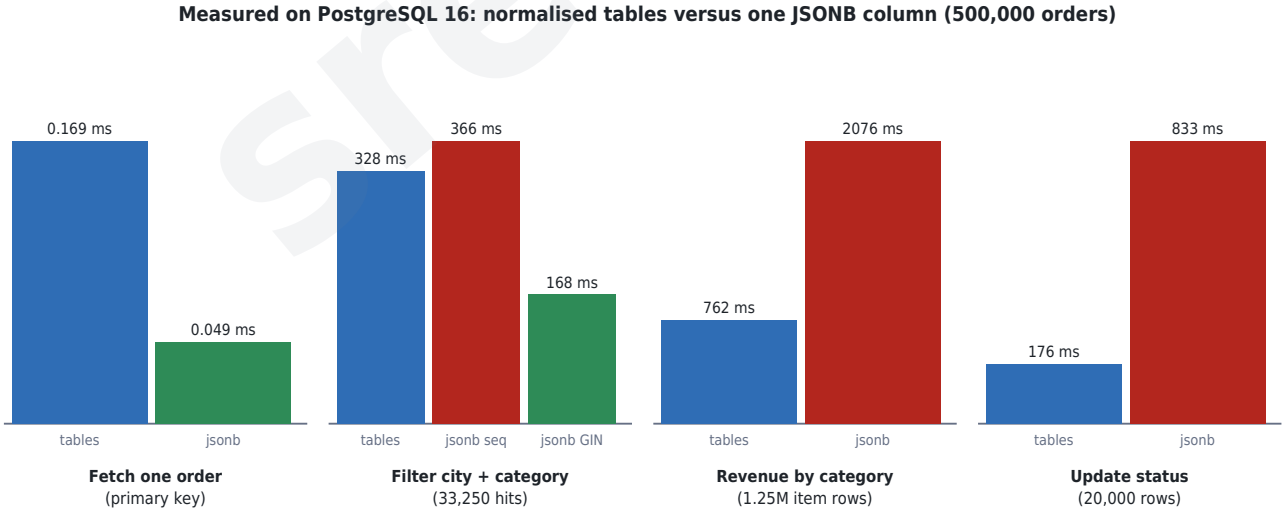


Figure 2.92. Measured on PostgreSQL 16: the single-aggregate read is fastest in the document layout, a non-selective search is helped only moderately by a GIN index, and analytics and updates favour the normalised layout.

Space and constraints

Layout	Size	Content
Relational tables (orders, order_items, customers, products)	154 MB	heap pages only, excluding indexes
Document table orders_doc	261 MB heap + 72 MB indexes	includes copies of customer and product fields in every document
GIN index (jsonb_path_ops)	57 MB	part of the 72 MB above

Storage cost of the two layouts after loading and updating (measured).

The document layout needs about 1.7 times the heap space because each order repeats the customer name, city, product name and category. This is the cost of denormalisation, paid once in space and each time in keeping the copies correct. A JSONB column can regain some of the relational guarantees with constraints and generated columns. Real output:

Constraints on a JSONB column (PostgreSQL 16, real output) TERMINAL

```
$ psql shop
shop=# ALTER TABLE orders_doc ADD CONSTRAINT doc_has_status
shop-#   CHECK (doc ? 'status' AND jsonb_typeof(doc->'items') = 'array');
ALTER TABLE
shop=# INSERT INTO orders_doc VALUES (900001, '{"customer":{"id":1}}');
ERROR:  new row for relation "orders_doc" violates check constraint "doc_has_status"
DETAIL:  Failing row contains (900001, {"customer": {"id": 1}}).
shop=# ALTER TABLE orders_doc ADD COLUMN status text GENERATED ALWAYS AS (doc->>'status') STORED;
ALTER TABLE
shop=# CREATE INDEX idx_doc_status ON orders_doc(status);
CREATE INDEX
```

The check constraint rejects a document without a status, which is the equivalent of MongoDB `$jsonSchema` validation, and the generated column exposes a document field as an ordinary typed, indexable column with statistics, so the planner estimates it correctly (in the final run, a filter on that column returned 96,001 rows through an ordinary bitmap scan on the B-tree index).

2.20.3 Hands-on

The MongoDB form of the five tasks

The following mongosh and Spring Data forms correspond to the tasks. They are typical, not run on the build machine.

Document-store equivalents (typical, not run on the build machine)

```
// Task 1: one read
db.orders.findOne({ _id: 250007 })

// Task 2: search with a multikey compound index on the embedded fields
db.orders.createIndex({ 'customer.city': 1, 'items.category': 1 })
db.orders.countDocuments({ 'customer.city': 'Pune', 'items.category': 'toys' })

// Task 3: selective lookup
db.orders.createIndex({ 'customer.id': 1 })
db.orders.find({ 'customer.id': 5434 }, { _id: 1 })

// Task 4: analytics
db.orders.aggregate([
  { $unwind: '$items' },
  { $group: { _id: '$items.category',
              rev: { $sum: { $multiply: ['$items.qty', '$items.unitPrice'] } } } },
  { $sort: { _id: 1 } } ])

// Task 5: update one field
db.orders.updateMany({ _id: { $gte: 100001, $lte: 120000 }, status: { $ne: 'SHIPPED' } },
                     { $set: { status: 'SHIPPED' } })
```

Two details differ from the SQL forms. The compound multikey index of task 2 contains entries for every combination of city and item category, so documents with several items produce several entries; and the `countDocuments` can be answered from the index alone only when the filter is fully covered, which a multikey path on arrays does not guarantee. In task 5, `$ne` makes the update idempotent and skips documents that are already shipped.

Exporting relational rows as documents

Migration to a document store often starts with generating documents from the relational data. The statement below builds the order documents (the same as in the benchmark) and writes them as newline-delimited JSON; `mongoimport` then loads them. The first two lines of the real output follow, cut for the page.

Generating NDJSON from PostgreSQL (real output, each line cut to 95 characters) TERMINAL

```
$ psql shop -t -A -c "SELECT jsonb_build_object('_id', id) || doc \
    FROM orders_doc ORDER BY id LIMIT 2" | cut -c1-95
{"_id": 1, "items": [{"qty": 3, "name": "Product 49", "category": "electronics", "productId": 4
{"_id": 2, "items": [{"qty": 4, "name": "Product 80", "category": "grocery", "productId": 80, "
```

Loading the documents into MongoDB (typical, not run on the build machine) BASH

```
psql shop -t -A -c "SELECT jsonb_build_object('_id', id) || doc FROM orders_doc" > orders.ndjson
mongoimport --uri "mongodb://db1:27017/shop" --collection orders --file orders.ndjson
```

Two conversions need care. JSON numbers lose the distinction between integers and decimals: `833.00` becomes a double, so monetary values should be converted to Decimal128 (`$convert`) or stored as integer minor units. Timestamps arrive as strings and must be converted to BSON dates (`$toDate` in an update pipeline, or the extended JSON form `{"$date": "..."}` in the export).

2.20.4 Real-world problems and solutions

Incident: document store used for relational data

Scenario. A team picks MongoDB for a reporting product because "the schema will change a lot". Customers, orders, invoices and payments become separate collections with ids referencing each other, and the application joins them in Java code.

Symptoms. Endpoints issue dozens of queries; the service holds large lists in memory to join them; reports are inconsistent when two collections are read at different moments; a bug leaves orphaned invoices because no constraint exists.

Diagnosis. Count queries per request (profiler or driver command listener). Look for `find` calls on one collection driven by the ids in the result of another. Review the schema: collections with many references and no embedded data are relational data in a document store.

Root cause. The data is highly connected and queried across entity boundaries, the access pattern that relational databases are designed for. The document model rewards aggregates read as a unit; here no such unit exists.

Fix. Immediate: cache reference data and batch lookups with `$in` or `$lookup` to cut round trips. Permanent: either redesign the hot read paths as aggregates (embed what is read together, copy display data) or move the connected data to PostgreSQL, keeping MongoDB for the document-shaped parts. A migration follows the staged procedure described below.

Prevention. Decide the model from the list of queries and invariants, not from the expected rate of schema change; JSONB or `$jsonSchema` handle schema change in either world.

Incident: JSONB column used as a catch-all

Scenario. To avoid schema migrations, all product attributes are stored in one `jsonb` column of the `products` table, including price and stock, which are updated frequently and used in joins.

Symptoms. Updates of stock become slow and the table bloats; queries on price filters are mis-estimated by the planner (the plan flips between nested loop and hash join); reports require casts everywhere; occasionally a string appears where a number was expected.

Diagnosis. `EXPLAIN (ANALYZE)` shows estimated rows far from actual rows for predicates on JSON paths (the planner has no statistics inside the value); `pg_stat_user_tables` shows many updates without HOT updates and a growing `n_dead_tup`; `SELECT DISTINCT jsonb_typeof(attrs->'price') FROM products` returns more than one type.

Root cause. Frequently updated, constrained, relational attributes were put in a structure that rewrites the whole value, has no column statistics and no type enforcement.

Fix. Immediate: add generated columns (`price numeric GENERATED ALWAYS AS ((attrs->>'price')::numeric) STORED`) with B-tree indexes, and a `CHECK` constraint on the type. Permanent: move stable, queried, updated attributes to real columns and keep only the variable long tail (colour, size, material, vendor-specific keys) in `jsonb`.

Prevention. Rule of thumb: a field that appears in a `WHERE`, `JOIN`, `ORDER BY` or `UPDATE` of a hot path belongs in a column.

Incident: premature sharding or premature NoSQL for scale

Scenario. A start-up chooses a sharded MongoDB cluster for a service that stores 20 GB and handles 300 requests per second, "to be ready for scale".

Symptoms. Operations cost three shards, three config servers and routers; development slows because of shard-key constraints (unique indexes, targeted queries); a scatter-gather query is slower than the same query on one node would be.

Diagnosis. Compare the actual load with single-node capacity: 20 GB fits in the memory of a modest server, and a single PostgreSQL or MongoDB replica set handles thousands of simple operations per second.

Root cause. Scale-out machinery was adopted for a hypothetical load, with its constraints paid immediately.

Fix. Collapse to a replica set (a single primary with replicas); defer sharding until measurements show a limit that vertical scaling, indexing, caching and read replicas cannot meet.

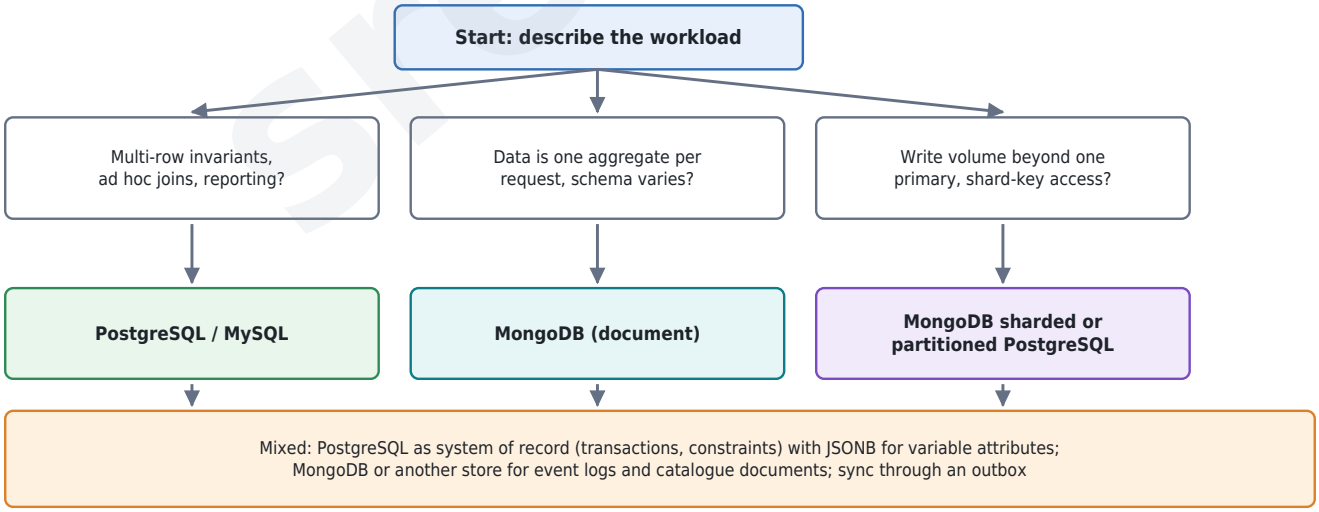
Prevention. Document the capacity thresholds that trigger a re-architecture and review them against measured load every quarter.

2.20.5 When to choose which

Decision rules

The rules below apply in order; the first that matches decides.

1. The data has strong cross-entity invariants (balances, inventory, uniqueness across entities) or needs ad hoc joins and reporting: choose a relational database.
2. The unit of work is one self-contained aggregate (a product page, a user profile, a shopping cart, an event) read and written as a whole, and its shape varies between records: a document store fits; embed what is read together.
3. Write throughput or data volume exceeds one primary and the access path has a natural partition key (customer, device, tenant): a sharded document store, or a partitioned relational database, fits.
4. The data is mostly relational with a variable tail of attributes: PostgreSQL with `jsonb` for the tail.
5. Otherwise choose the technology the team operates well; the operational cost of an unfamiliar database often exceeds the gain of a better data model.



Decide by access pattern, consistency need, scale and team skill, not by fashion

Figure 2.93. Workload questions lead from invariants and ad hoc queries towards relational storage, from per-request aggregates towards documents, and from partition-friendly high write volume towards sharding; mixed systems combine them through an outbox.

Hybrid architectures

A single system often uses both. The relational database is the system of record for orders, payments and inventory, where constraints and transactions matter. MongoDB stores documents that are read as a whole and tolerate eventual consistency: the product catalogue with variable attributes, a denormalised order history view for the customer-facing app, and an append-only event log. The two are kept in step through the outbox pattern: the service writes the order and an outbox row in one relational transaction; a relay publishes the outbox rows to a message broker or writes them to MongoDB idempotently (the `_id` of the document is the id of the outbox row, so a repeated delivery overwrites instead of duplicating). Reads of the history view go to MongoDB; writes that must be consistent go to PostgreSQL.

Migrating between them

A migration is a sequence of reversible stages, each deployed and observed separately.

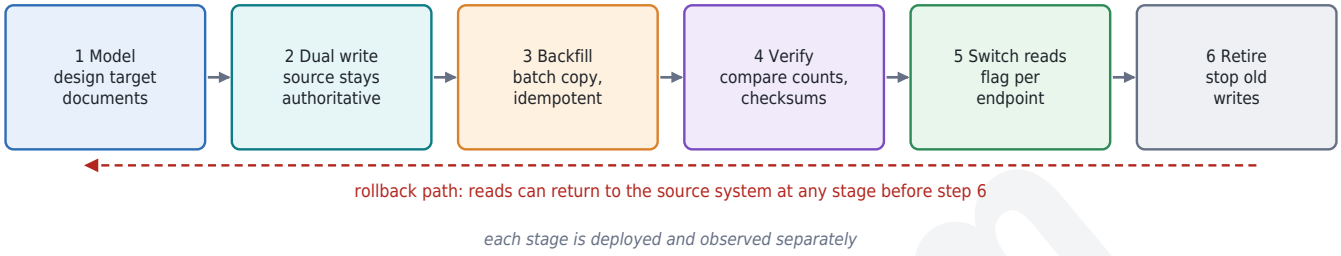


Figure 2.94. A staged migration designs the target model, dual-writes, backfills history idempotently, verifies, switches reads gradually and only then retires the source, with a rollback path until the last stage.

1. **Model.** Derive the target from the query list. Relational to document: pick the aggregate root, embed the child rows read with it, copy display fields, keep references for shared entities. Document to relational: map each embedded array to a child table with a foreign key and extract the stable fields to typed columns.
2. **Dual write.** The application writes to both stores; the old store remains authoritative, and a failure to write the new store is logged and queued, not surfaced to the user.
3. **Backfill.** A batch job copies history in key order with checkpoints; every copy is an idempotent upsert (`_id` or primary key preserved).
4. **Verify.** Compare counts per key range and checksums of selected fields; run shadow reads that query both stores and diff the responses.
5. **Switch reads.** Move endpoints one at a time behind a feature flag, starting with low-risk ones, watching latency and error rates.
6. **Retire.** Stop writing to the old store after a rollback window, then archive it.

The most common errors are copying `_id` or key values inconsistently, converting decimals and dates through floating point or strings, and skipping the verification stage.

2.20.6 Questions to expect

Questions and model answers

1. **When would you choose MongoDB over PostgreSQL?** When the unit of access is a self-contained aggregate with varying shape, when horizontal write scale through sharding is required, or when documents map naturally onto application objects. For relational integrity, joins and reporting, PostgreSQL.
2. **Can PostgreSQL replace a document database?** For many workloads yes: `jsonb` with GIN and expression indexes, constraints and transactions cover flexible attributes. It does not provide built-in sharding with automatic balancing, and whole-value rewrites make large frequently updated documents expensive.
3. **What does denormalisation cost?** Space, and the work of keeping copies consistent: updating shared data (a customer's city) means updating every copy or accepting staleness, which is acceptable only where the copy is a historical snapshot.
4. **Why was the analytic query slower on JSONB in the benchmark?** It had to read every document, expand arrays and cast text values; the normalised layout scans a narrow table of typed columns.
5. **Why did the GIN index help only moderately in task 2?** The predicate matched 6.6 percent of the rows spread over almost all pages; the index finds them quickly but the heap pages still have to be read.

6. **How does a transaction across entities differ in the two systems?** PostgreSQL treats it as the ordinary case; MongoDB prefers to make it a single-document write and offers bounded multi-document transactions (60 second limit, snapshot isolation).
7. **What is the outbox pattern?** Writing the business change and an event row in one local transaction and publishing the event later, so that two stores stay consistent without a distributed transaction.
8. **How do you migrate with no downtime?** Dual writes, idempotent backfill, verification, gradual switch of reads, then retirement of the old store, with a rollback path at every stage.

KEY POINTS

Choose by access pattern and invariants, then verify with measurements on your own data.

- Document layouts win single-aggregate reads (here 0.049 ms against 0.169 ms); normalised layouts win analytics (762 ms against 2,076 ms) and small updates (176 ms against 833 ms).
- JSONB is a good home for the variable tail of attributes; keep queried and frequently updated fields in typed columns.
- A GIN index helps selective containment queries; a non-selective predicate remains bounded by the pages that hold the result.
- Sharding and NoSQL adopted for hypothetical scale cost complexity immediately; migrate in reversible stages when it is justified.

2.21 Interview Question Bank

The questions below are grouped by topic and cover the full book. Each answer is a model answer of two to five lines: the points an interviewer expects, in the order in which they should be stated. A strong candidate adds one concrete example (a query, a plan node, a configuration key) and names the trade-off. The scenario questions at the end combine several topics and are answered as a diagnosis procedure. The answers state PostgreSQL behaviour as observed in PostgreSQL 16 and MySQL and MongoDB behaviour as documented for MySQL 8.x and MongoDB 7.x and 8.x.

2.21.1 DBMS basics

Questions 1 to 10

1. **What is a DBMS and what does it provide beyond files?** Software that stores data and provides a query language, concurrency control, transactions, recovery after failure, access control and integrity constraints. A file system provides none of these; each application would need to implement them.
2. **What is the three-schema architecture?** External schemas (views per user group), the conceptual schema (the logical structure of the whole database) and the internal schema (physical storage). It gives logical data independence (changing the conceptual schema without breaking views) and physical data independence (changing storage without changing the logical schema).
3. **What is the difference between OLTP and OLAP?** OLTP serves many short transactions that read and write a few rows by key (orders, payments) and favours row storage and indexes. OLAP runs few long queries that scan and aggregate many rows and favours column storage and parallel scans.
4. **What is the difference between a primary key, a candidate key and a super key?** A super key is any set of attributes that identifies a row; a candidate key is a minimal super key; the primary key is the candidate key chosen as the main identifier. Remaining candidate keys become unique constraints.
5. **Surrogate or natural key?** A surrogate key (identity, UUID) is stable and compact and does not change when business data changes. A natural key (email, ISBN) has meaning but can change or be reused. Common practice is a surrogate primary key plus a unique constraint on the natural key.
6. **What integrity constraints exist?** Domain (type, CHECK), NOT NULL, entity integrity (primary key), referential integrity (foreign key) and UNIQUE. Constraints put the rule in the database so that every application is subject to it.
7. **What is a foreign key and what do the referential actions do?** A column that references a key in another table. On delete or update of the parent, `RESTRICT` and `NO ACTION` reject, `CASCADE` propagates, `SET NULL` and `SET DEFAULT` replace the value.
8. **What is NULL and how does it behave in comparisons?** NULL means unknown. Comparisons with NULL yield unknown (three-valued logic), so `x = NULL` is never true; use `IS NULL`. `WHERE` keeps only rows where the predicate is true, and `NOT IN` with a NULL in the list returns no rows.

9. **What is the difference between DELETE, TRUNCATE and DROP?** `DELETE` removes selected rows, is logged per row and can be rolled back; `TRUNCATE` removes all rows quickly by deallocating storage (transactional in PostgreSQL, implicit commit in MySQL); `DROP` removes the table definition too.
10. **What is a view and what is a materialised view?** A view is a stored query evaluated at each use. A materialised view stores the result and must be refreshed (`REFRESH MATERIALIZED VIEW`), trading freshness for speed.

2.21.2 SQL

Questions 11 to 22

1. **In which order does SQL logically process a SELECT?** FROM and JOIN, WHERE, GROUP BY, HAVING, SELECT (including window functions), DISTINCT, ORDER BY, LIMIT. This explains why a select alias cannot be used in WHERE but can be used in ORDER BY.
2. **What is the difference between WHERE and HAVING?** `WHERE` filters rows before grouping and cannot use aggregates; `HAVING` filters groups after aggregation.
3. **Explain the join types.** Inner returns matching pairs; left (right) keeps all rows of one side with NULLs where no match exists; full keeps both; cross returns the Cartesian product; a self join joins a table to itself; semi and anti joins (`EXISTS`, `NOT EXISTS`) test for existence of matches without returning columns of the other side.
4. **What is the difference between a filter in ON and in WHERE for a left join?** A condition on the right table in `WHERE` removes the NULL-extended rows and turns the join into an inner join; in `ON` it restricts which rows match while keeping all left rows.
5. **What is the difference between the three COUNT forms?** `COUNT(*)` counts rows; `COUNT(col)` counts non-NULL values; `COUNT(DISTINCT col)` counts distinct non-NULL values.
6. **How do you find the second highest salary?** `SELECT max(salary) FROM emp WHERE salary < (SELECT max(salary) FROM emp)`, or `DENSE_RANK() OVER (ORDER BY salary DESC)` and filter on rank 2, which also handles ties and the Nth case.
7. **How do you find duplicate rows?** `SELECT col, count(*) FROM t GROUP BY col HAVING count(*) > 1`; to delete duplicates keep one row per group with `ROW_NUMBER() OVER (PARTITION BY col ORDER BY id)` and delete rows with number above 1.
8. **What is a window function and how does it differ from GROUP BY?** It computes a value over a set of related rows without collapsing them, so each input row stays in the output. Examples: `ROW_NUMBER`, `RANK`, `LAG`, `SUM() OVER (ORDER BY ...)` for running totals.
9. **ROW_NUMBER, RANK and DENSE_RANK?** `ROW_NUMBER` gives unique consecutive numbers; `RANK` gives ties the same rank and leaves gaps after them; `DENSE_RANK` gives ties the same rank with no gaps.
10. **What is a CTE and what is a recursive CTE?** A named subquery (`WITH`) used for readability and reuse. A recursive CTE has an anchor and a recursive member joined by `UNION ALL` and walks hierarchies such as organisation trees; a termination condition or a depth limit prevents endless recursion.
11. **EXISTS or IN?** They are logically equivalent for non-NULL values and the optimiser usually produces the same semi-join plan. `NOT IN` differs from `NOT EXISTS` when the subquery can return NULL: `NOT IN` then returns nothing, so `NOT EXISTS` is the safe form.
12. **What is an upsert?** Insert or update in one statement: PostgreSQL `INSERT ... ON CONFLICT (key) DO UPDATE`, MySQL `INSERT ... ON DUPLICATE KEY UPDATE`. It is atomic and avoids the race of select-then-insert.

2.21.3 Normalisation and modelling

Questions 23 to 30

1. **What is normalisation and why is it done?** Structuring tables so that each fact is stored once, to avoid insertion, update and deletion anomalies. It is based on functional dependencies.
2. **State 1NF, 2NF and 3NF.** 1NF: atomic values and no repeating groups. 2NF: 1NF and no non-key attribute depends on part of a composite key. 3NF: 2NF and no non-key attribute depends on another non-key attribute (no transitive dependency).
3. **What is BCNF and how does it differ from 3NF?** In BCNF every determinant of a non-trivial functional dependency is a super key. 3NF allows a dependency whose right side is a prime attribute; BCNF does not, and a BCNF decomposition can lose dependency preservation.

sheeran

4. **What are update, insertion and deletion anomalies?** Redundant copies get out of step on update; a fact cannot be stored without an unrelated fact on insertion; deleting a row destroys an unrelated fact.
5. **When would you denormalise?** When measured read cost of joins is a problem and the duplicated data changes rarely or is a historical snapshot (price at order time); the price is extra writes and the risk of inconsistency, managed by triggers, application logic or periodic reconciliation.
6. **How is a many-to-many relationship modelled?** With an association table holding the foreign keys of both sides as a composite primary key, plus attributes of the relationship (quantity, date).
7. **What is a weak entity?** An entity without its own key, identified through an owner entity (order line identified by order id and line number); it is implemented with a composite key containing the owner's key.
8. **What is a lossless decomposition?** A decomposition whose natural join of the parts reproduces the original table exactly, with no spurious tuples. It holds when the common attributes form a key of one of the parts.

2.21.4 Indexes and query performance

Questions 31 to 42

1. **What is an index and what does it cost?** An auxiliary structure (usually a B+tree) mapping key values to row locations so that lookups avoid a full scan. It costs space and slows writes, because every insert, update of an indexed column and delete maintains each index.
2. **How does a B+tree index work?** Balanced tree with internal nodes holding separator keys and leaf nodes holding keys with row pointers, leaves linked in key order. A lookup descends from the root in a few page reads (typically three or four levels for millions of rows) and range scans follow the leaf links.
3. **What is the leftmost prefix rule for composite indexes?** An index on (a, b, c) serves filters on a, on a and b, and on a, b and c; it does not serve a filter on b alone. Equality columns come first, then the range or sort column.
4. **What is a covering index?** An index that contains all columns a query needs (with `INCLUDE` in PostgreSQL), so the query is answered by an index-only scan without visiting the table, provided the visibility map shows the pages as all-visible.
5. **When is an index not used?** When the predicate is not sargable (a function on the column, `LIKE '%x'`, an implicit cast), when the predicate matches a large fraction of the table, when statistics are stale, or when the table is small. The fix depends on the cause: expression index, rewrite, `ANALYZE`, or none.
6. **What is selectivity and cardinality?** Cardinality is the number of distinct values; selectivity is the fraction of rows a predicate returns. Indexes pay off for low fractions (high selectivity in the sense of few rows).
7. **What do Seq Scan, Index Scan, Index Only Scan and Bitmap Heap Scan mean?** Read the whole table; walk the index and fetch each row; answer from the index alone; collect matching row locations from one or more indexes into a bitmap and read the heap pages in physical order, which suits mid-sized result sets.
8. **How do you read EXPLAIN ANALYZE?** Compare estimated rows with actual rows at each node (a large mismatch points to stale statistics), look at `actual time`, `loops` (multiply per-loop time by loops), buffers (`shared hit` and `read`), and rows removed by filter. The most expensive node by actual time is the starting point.
9. **Nested loop, hash join and merge join?** Nested loop probes the inner side per outer row, best with an index and few outer rows; hash join builds a hash table on one input, good for large unsorted inputs; merge join merges two sorted inputs and is good when both are already sorted.
10. **Why is OFFSET pagination slow for deep pages?** The server must produce and discard all skipped rows. Keyset pagination (`WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT n`) uses an index seek and has constant cost.
11. **What is the N+1 query problem?** Loading N parents and then issuing one query per parent for a lazy association: N+1 statements. Fix with `JOIN FETCH`, entity graphs, batch fetching (`@BatchSize`) or a projection query.
12. **How do you find slow queries in production?** `pg_stat_statements` (total and mean time per normalised query), `log_min_duration_statement`, `auto_explain` for plans, and the MySQL slow query log with `performance_schema`; then `EXPLAIN (ANALYZE, BUFFERS)` on the worst offenders.

Preview

2.21.5 Transactions and isolation

Questions 43 to 52

1. **What does ACID mean?** Atomicity (all or nothing), consistency (constraints hold before and after), isolation (concurrent transactions do not see each other's partial work) and durability (committed data survives a crash, via the write-ahead log).
2. **Name the concurrency anomalies.** Dirty read (reading uncommitted data), non-repeatable read (a row read twice changes), phantom read (a repeated query returns new rows), lost update (two read-modify-write cycles overwrite each other) and write skew (two transactions read overlapping data and write disjoint rows, breaking an invariant).
3. **What are the SQL isolation levels?** Read uncommitted, read committed, repeatable read and serializable, each forbidding more anomalies. PostgreSQL treats read uncommitted as read committed and its repeatable read as snapshot isolation.
4. **What are the defaults in PostgreSQL and MySQL?** PostgreSQL: read committed. MySQL InnoDB: repeatable read, with next-key locks that prevent most phantoms.
5. **How does PostgreSQL implement serializable?** Serializable snapshot isolation: snapshot reads plus detection of dangerous read-write dependency cycles, aborting one transaction with SQLSTATE 40001, which the application must retry.
6. **How do you prevent a lost update?** An atomic update (`SET qty = qty - 1`), `SELECT ... FOR UPDATE`, optimistic locking with a version column, or a higher isolation level with retry.
7. **What is optimistic versus pessimistic locking?** Pessimistic locks the row when reading (`FOR UPDATE`) and blocks others; optimistic reads without locking and checks at update time that the version has not changed, retrying on conflict. Optimistic suits low contention.
8. **What is a deadlock and how is it handled?** Two transactions each wait for a lock the other holds. The database detects the cycle and aborts one (PostgreSQL after `deadlock_timeout`, 1 s by default). Prevent by locking rows in a consistent order and keeping transactions short.
9. **What do `SKIP LOCKED` and `NOWAIT` do?** `NOWAIT` fails at once if the row is locked; `SKIP LOCKED` skips locked rows, which allows several workers to take jobs from a queue table without blocking each other.
10. **What is a savepoint?** A named point inside a transaction to which part of the work can be rolled back without aborting the whole transaction (`SAVEPOINT`, `ROLLBACK TO`).

2.21.6 MVCC, WAL and recovery

Questions 53 to 60

1. **What is MVCC?** Multi-version concurrency control: writers create new row versions while readers continue to see the version that was current for their snapshot, so readers do not block writers and writers do not block readers.
2. **How does PostgreSQL store versions?** Each tuple has `xmin` (creating transaction) and `xmax` (deleting or locking transaction); an update marks the old tuple with `xmax` and inserts a new tuple. A snapshot decides visibility from these ids and the commit status.
3. **How does InnoDB differ?** It updates the row in place in the clustered index and keeps old versions in undo logs; a read view reconstructs old versions from the undo chain. There is no heap bloat but long transactions grow the undo history.
4. **What is VACUUM and why is it needed?** It removes dead tuples that no snapshot can see, updates the visibility map and free space map, and freezes old transaction ids. Without it tables bloat and transaction id wraparound approaches.
5. **What is transaction id wraparound?** Transaction ids are 32 bits and compared modulo 2^32; if old rows are not frozen, they appear to be in the future and become invisible. PostgreSQL warns and finally refuses writes to protect data; anti-wraparound autovacuum prevents it.
6. **What is the write-ahead log?** Changes are first appended to a sequential log and flushed at commit; data pages are written later. After a crash the log is replayed (redo) to restore committed changes, so durability needs one sequential flush instead of random page writes.
7. **What is a checkpoint?** A point at which all dirty pages are flushed so that recovery needs to replay only the WAL after it; checkpoint frequency trades recovery time against write bursts.

freeram

8. **What does `synchronous_commit` control?** Whether a commit waits for the WAL flush (and, with synchronous replication, for the standby). Turning it off can lose the last few commits on a crash but never corrupts the database.

2.21.7 PostgreSQL and MySQL differences

Questions 61 to 68

1. **How are rows stored in PostgreSQL and in InnoDB?** PostgreSQL stores rows in an unordered heap with all indexes secondary; InnoDB stores rows in the clustered primary key B+tree, and secondary indexes hold the primary key value, so a secondary lookup needs a second tree descent.
2. **What follows from the clustered index for primary key choice?** In InnoDB a large or random primary key (a random UUID) enlarges every secondary index and causes page splits; a short monotonic key is preferred. In PostgreSQL the key size affects only the index itself.
3. **How do auto-increment columns differ?** PostgreSQL uses sequences (`GENERATED ... AS IDENTITY`) that are not rolled back, so gaps appear; MySQL uses `AUTO_INCREMENT` per table with similar gaps after rollback or failed inserts.
4. **What type differences matter?** PostgreSQL has `timestamptz`, arrays, `jsonb`, native `boolean` and `uuid`; MySQL has `DATETIME` and `TIMESTAMP` (with its range limit and time zone conversion) and `JSON`. Both treat empty string and NULL differently from Oracle.
5. **How do DDL and locking differ?** Most PostgreSQL DDL is transactional; MySQL DDL commits implicitly. Adding a column with a constant default is instant in PostgreSQL 11 and later and MySQL 8 uses instant algorithms for many changes; both can still take locks that queue behind long transactions.
6. **How do replication mechanisms compare?** PostgreSQL streams WAL (physical) or uses logical replication; MySQL replicates the binary log (statement, row or mixed) and supports group replication and semi-synchronous modes.
7. **Which has better JSON support?** PostgreSQL `jsonb` has GIN indexing and a rich path language; MySQL `JSON` has generated-column and multi-valued indexes. Both are adequate for a variable tail of attributes.
8. **What is a PostgreSQL feature with no direct MySQL equivalent?** Partial indexes, `RETURNING`, rich index types (GIN, GiST, BRIN), `LISTEN/NOTIFY`, extensions such as PostGIS; MySQL has no partial indexes and had no `RETURNING` for inserts (MariaDB does).

2.21.8 Scaling, replication and sharding

Questions 69 to 78

1. **Why use a connection pool?** Opening a database connection is expensive (a process in PostgreSQL) and the server supports a limited number. A pool such as HikariCP reuses connections and bounds concurrency; its size should be small (roughly cores times two plus disks), not equal to the number of request threads.
2. **What happens when the pool is exhausted?** Threads wait for a connection up to `connectionTimeout` and then fail. Common causes are transactions that stay open while calling remote services and queries that are too slow.
3. **What is replication lag and why does it matter?** Asynchronous replicas apply changes after the primary, so reads from them can be stale. Read-your-writes flows must read from the primary or wait for the replica to pass a known position.
4. **Synchronous versus asynchronous replication?** Synchronous waits for the standby before commit (no loss of acknowledged data, higher latency, availability depends on the standby); asynchronous does not wait (low latency, possible loss on failover).
5. **What is split brain?** Two nodes both act as primary after a network partition and accept divergent writes. Prevent with quorum, fencing and a consensus-backed failover manager.
6. **Partitioning versus sharding?** Partitioning splits a table into parts inside one database server (pruning, maintenance); sharding distributes data across independent servers, each holding a subset, with routing and cross-shard limits.
7. **What makes a good shard key?** High cardinality, even frequency, no monotonic growth, and presence in common queries; hashed keys spread load, ranged keys support range queries.
8. **What is consistent hashing?** Keys and nodes are mapped onto a ring and a key belongs to the next node clockwise; adding or removing a node moves only the keys of one neighbour, and virtual nodes smooth the distribution.

9. **How do you take a backup that you can trust?** Physical base backup plus archived WAL for point-in-time recovery, or logical dumps for small data, and, above all, a regular restore test that measures the real recovery time.
10. **What are RPO and RTO?** Recovery point objective: the maximum acceptable data loss in time. Recovery time objective: the maximum acceptable time to restore service.

2.21.9 NoSQL and the CAP theorem

Questions 79 to 88

1. **Why NoSQL?** Horizontal scale, flexible schema, and data models matched to access patterns (documents, key-value, wide-column, graph), accepting weaker joins or transactions. The name does not mean the absence of SQL.
2. **State the CAP theorem precisely.** In the presence of a network partition a distributed system must choose between consistency (linearizability) and availability (every non-failing node answers). Without a partition both are possible; CAP does not say "pick two of three".
3. **What does PACELC add?** If there is a partition, choose availability or consistency; else (normal operation) choose between latency and consistency. It captures the latency cost of strong consistency.
4. **What is eventual consistency?** If no new updates are made, all replicas converge to the same value eventually. Between updates readers can see stale data.
5. **What is quorum replication?** With N replicas, a write waits for W acknowledgements and a read queries R replicas; if R + W > N, every read set overlaps the latest write set, so the latest acknowledged write is visible to the read.
6. **What are vector clocks and last-write-wins?** A vector clock records the causal history of each version so concurrent versions can be detected and merged; last-write-wins keeps the version with the highest timestamp, which silently drops concurrent updates and depends on clock accuracy.
7. **What is the difference between a B-tree and an LSM-tree storage engine?** B-trees update pages in place and read with few I/Os; LSM-trees buffer writes in memory, flush sorted immutable files and merge them by compaction, giving high write throughput at the cost of read and space amplification.
8. **What is a tombstone and why can it hurt?** A delete marker in LSM-based stores; the data is removed only at compaction, and reads must skip accumulated tombstones, which slows queries on frequently deleted partitions.
9. **When is a graph database better than relational?** For queries that traverse many relationships of variable depth (friends of friends, fraud rings); each hop is an index-free adjacency step instead of a join, whereas a relational recursive query repeats joins.
10. **What is polyglot persistence?** Using several database types in one system, each for the data it fits, for example PostgreSQL for orders, a document store for catalogue, a search engine for text and a cache for hot reads.

2.21.10 MongoDB

Questions 89 to 100

1. **What is the document model?** Data stored as BSON documents (nested fields, arrays, rich types) in collections without a fixed schema; related data is embedded in one document when it is read and changed together.
2. **When do you embed and when do you reference?** Embed for one-to-few relationships read with the parent and bounded in size; reference for large or unbounded children, data shared by many parents, or data updated independently. Unbounded arrays must be avoided (16 MB document limit).
3. **What is the aggregation pipeline?** A sequence of stages (`$match`, `$group`, `$sort`, `$lookup`, `$unwind`, `$project`) in which each stage transforms the document stream; `$match` and `$sort` early use indexes and reduce work.
4. **What is the ESR rule?** For compound indexes order fields as Equality, Sort, Range, so equality filters narrow the scan, the sort uses the index order, and the range is applied last.
5. **How do you read `explain('executionStats')`?** Compare `nReturned`, `totalKeysExamined` and `totalDocsExamined`; a large ratio of examined to returned documents or a `COLLSCAN` stage indicates a missing or poorly ordered index.
6. **How does MongoDB provide atomicity?** A single-document write is atomic; multi-document transactions provide snapshot isolation on replica sets and sharded clusters, with a 60-second default limit.

7. **What is a replica set and how does failover work?** Members replicate the primary's oplog; if the primary is unreachable for the election timeout (10 s by default) the secondaries elect a new primary by majority vote in a higher term, typically within about 12 seconds.
8. **What are write concern, read concern and read preference?** Write concern: how many members must acknowledge a write (`w: majority` survives failover). Read concern: the consistency level of returned data (`majority`, `snapshot`). Read preference: which member serves reads.
9. **How does sharding work in MongoDB?** A shard key partitions data into chunks assigned to shards (replica sets); mongos routes by the chunk map from the config servers; the balancer migrates chunks to even out data.
10. **What causes a hot shard and a jumbo chunk?** A hot shard comes from a monotonic or skewed key sending most writes to one chunk; a jumbo chunk from a key with too few distinct values so that a chunk cannot be split.
11. **What is the oplog?** A capped collection of idempotent operations on the primary, tailed by secondaries; its size determines how long a secondary may be offline before it needs a full resync.
12. **What is NoSQL injection?** Operator objects in user input (`{"$ne": null}`) change the meaning of a query. Prevent with typed request objects and validation instead of passing raw maps into queries.

2.21.11 Spring and JPA data access

Questions 101 to 110

1. **What does `@Transactional` do?** A proxy starts a transaction before the method, commits on normal return and rolls back on a runtime exception (not on checked exceptions by default). It works only on calls that pass through the proxy.
2. **Why does `@Transactional` sometimes not apply?** Self-invocation within the same class, a non-public or final method, a missing transaction manager (for MongoDB it must be declared), or a swallowed exception.
3. **What are the propagation types?** `REQUIRED` (join or start), `REQUIRES_NEW` (suspend and start a new one), `SUPPORTS`, `MANDATORY`, `NEVER`, `NOT_SUPPORTED` and `NESTED` (savepoint). `REQUIRES_NEW` holds a second connection, which can exhaust a pool.
4. **Lazy versus eager loading in JPA?** Lazy fetches an association on first access, eager with the parent. Lazy access outside the session raises `LazyInitializationException`; both can cause N+1 queries when used in loops.
5. **How do you fix N+1 in JPA?** `JOIN FETCH` in JPQL, `@EntityGraph`, `@BatchSize` or `default_batch_fetch_size`, or a DTO projection that selects the needed columns in one query.
6. **What does `open-in-view` do?** It keeps the persistence context open during web view rendering, hiding lazy loading problems and holding a connection longer; production configurations usually disable it (`spring.jpa.open-in-view=false`).
7. **How does optimistic locking work in JPA?** A `@Version` field is included in the `UPDATE ... WHERE id = ? AND version = ?`; zero affected rows raises `OptimisticLockException`, which the application retries.
8. **What does a HikariCP pool need configured?** `maximumPoolSize` (small), `connectionTimeout`, `maxLifetime` (below any database or proxy timeout), `idleTimeout`, and leak detection (`leakDetectionThreshold`) during diagnosis.
9. **How do you prevent SQL injection in JDBC?** Use `PreparedStatement` parameters (or JPA parameters) so values are never concatenated into the SQL text; whitelist dynamic identifiers such as sort columns.
10. **How does Flyway work?** It applies versioned SQL migrations in order, recording them in a history table with checksums; changed applied migrations fail validation. Zero-downtime changes use expand and contract: add, migrate data, switch code, remove.

2.21.12 Scenario questions

Questions 111 to 120

1. **An endpoint became slow after a release. What do you do first?** Identify the query (slow query log, `pg_stat_statements`), run `EXPLAIN (ANALYZE, BUFFERS)`, compare estimates with actuals, check for a new query pattern (N+1) and for a missing index or changed plan; fix and verify with the same measurement.
2. **The database CPU is at 100 percent. Where do you look?** Top queries by total time, sequential scans of large tables, a sudden increase in executions (a loop or a cache failure), missing indexes, connection storms; then act on the heaviest query.

3. **Two users bought the last item. Why and how do you fix it?** A read-check-write race (lost update). Use an atomic conditional update (`UPDATE ... SET qty = qty - 1 WHERE qty > 0`) and check the affected row count, or `SELECT FOR UPDATE`.
4. **Disk usage grows quickly though data volume is stable. What are the suspects?** Table bloat from long transactions blocking vacuum, an inactive replication slot retaining WAL, a growing log or temporary files, or index bloat.
5. **Application threads hang waiting for a connection. How do you diagnose?** Check pool metrics and `pg_stat_activity` for `idle in transaction` sessions and long queries; look for transactions open across remote calls; fix the transaction boundary and add timeouts (`idle_in_transaction_session_timeout`).
6. **A migration that adds an index blocked production. What happened?** A plain `CREATE INDEX` blocks writes for its duration; use `CREATE INDEX CONCURRENTLY`. Also, a DDL statement waiting on a long transaction queues all later queries behind its lock; set `lock_timeout`.
7. **Reads from a replica return old data after a write. How do you fix it?** Route read-after-write requests to the primary, or use causal consistency (MongoDB sessions, PostgreSQL LSN waiting), or bound staleness and show the user the pending state.
8. **A report shows different totals each run. Why?** Reading in read committed while data changes, a join fan-out duplicating rows, or non-deterministic pagination; run in a repeatable read snapshot, check join cardinality and add a unique ordering.
9. **How would you design the data layer for an order system with search and an activity feed?** PostgreSQL for orders, items, payments and inventory with constraints and transactions; an outbox feeding a search index and a MongoDB event log; a cache for product pages; replicas for reporting.
10. **What do you do when a primary fails in a three-node set?** Confirm the election result and the new primary, check that applications reconnect (retryable writes, timeouts), examine the former primary for rollback data once it returns, review replication lag and write concern effects, and record the timeline.

KEY POINTS

The bank is a revision tool: state the answer, give an example, name the trade-off.

- Basics and SQL questions test precision of definitions; index and plan questions test the ability to read evidence.
- Isolation, MVCC and recovery answers should name the mechanism, not only the property.
- Scaling and NoSQL answers should state the guarantee and the failure it prevents.
- Scenario answers are procedures: observe, narrow, fix, verify, prevent.

2.22 Cheat Sheets

The tables and listings in this chapter condense the book into reference form: SQL syntax, a command map between PostgreSQL, MySQL and MongoDB, the isolation matrix, an index decision table, a guide to reading EXPLAIN output and a mongosh crib. They state facts for PostgreSQL 16, MySQL 8.x and MongoDB 7.x and 8.x, and each entry points back to the chapter that explains the mechanism.

Preprint

2.22.1 SQL syntax

SQL reference (PostgreSQL syntax; differences for MySQL in comments) SQL

```sql
-- DDL
CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,   -- MySQL: BIGINT AUTO_INCREMENT
  customer_id bigint NOT NULL REFERENCES customers(id) ON DELETE RESTRICT,
  status      text   NOT NULL DEFAULT 'NEW' CHECK (status IN ('NEW','PAID','SHIPPED')),
  created_at  timestamptz NOT NULL DEFAULT now()                  -- MySQL: DATETIME or TIMESTAMP
);
ALTER TABLE orders ADD COLUMN note text;
CREATE INDEX CONCURRENTLY idx_orders_cust ON orders (customer_id, created_at DESC);

-- DML
INSERT INTO orders (customer_id) VALUES (5434) RETURNING id;      -- MySQL: LAST_INSERT_ID()
INSERT INTO stock (sku, qty) VALUES ('A-100', 5)
  ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;  -- MySQL: ON DUPLICATE KEY UPDATE
UPDATE stock SET qty = qty - 2 WHERE sku = 'A-100' AND qty >= 2;
DELETE FROM orders WHERE status = 'CANCELLED' AND created_at < now() - interval '1 year';

-- Query
SELECT c.city, count(*) AS orders, sum(o.total) FILTER (WHERE o.status = 'PAID') AS paid
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.created_at >= date '2026-01-01'
GROUP BY c.city HAVING count(*) > 100
ORDER BY orders DESC LIMIT 10;

-- Window and CTE
WITH ranked AS (
  SELECT o.*, row_number() OVER (PARTITION BY customer_id ORDER BY created_at DESC) AS rn
  FROM orders o)
SELECT * FROM ranked WHERE rn <= 3;                               -- latest 3 orders per customer

-- Locking reads and queues
SELECT id FROM jobs WHERE state = 'READY' ORDER BY id
  FOR UPDATE SKIP LOCKED LIMIT 10;

-- Transaction
BEGIN ISOLATION LEVEL REPEATABLE READ;   -- MySQL: SET TRANSACTION ISOLATION LEVEL ..
COMMIT;
```

sreeram

2.22.2 PostgreSQL, MySQL and MongoDB command map

Task	PostgreSQL	MySQL 8	MongoDB (mongosh)
List databases	`\l`	`SHOW DATABASES;`	`show dbs`
Switch database	`\c shop`	`USE shop;`	`use shop`
List tables or collections	`\dt`	`SHOW TABLES;`	`show collections`
Describe structure	`\d orders`	`DESCRIBE orders;`	`db.orders.findOne()`
List indexes	`\di or \d orders`	`SHOW INDEX FROM orders;`	`db.orders.getIndexes()`
Explain a query	`EXPLAIN (ANALYZE, BUFFERS) q`	`EXPLAIN ANALYZE q`	`db.orders.find(q).explain('executionStats')`
Running queries	`pg_stat_activity`	`SHOW FULL PROCESSLIST;`	`db.currentOp()`
Cancel a query	`pg_cancel_backend(pid)`	`KILL QUERY id;`	`db.killOp(opid)`
Lock waits	`pg_locks, pg_blocking_pids()`	`performance_schema.data_locks`	`currentOp waitingForLock`
Slow query capture	`pg_stat_statements, log_min_duration_statement`	`slow_query_log, long_query_time`	`db.setProfilingLevel(1, {slowms: 100})`
Table size	`pg_total_relation_size('orders')`	`information_schema.TABLES`	`db.orders.stats()`
Statistics refresh	`ANALYZE orders;`	`ANALYZE TABLE orders;`	`(automatic; $indexStats)`
Reclaim dead space	`VACUUM (ANALYZE) orders;`	`OPTIMIZE TABLE orders;`	`compact (rarely needed)`
Logical backup	`pg_dump -Fc shop`	`mysqldump --single-transaction shop`	`mongodump --oplog`
Physical backup	`pg_basebackup`	`xtrabackup or mysqlbackup`	`volume snapshot`
Replication status	`pg_stat_replication`	`SHOW REPLICA STATUS;`	`rs.status()`
Create user	`CREATE ROLE app LOGIN PASSWORD ...; GRANT ...`	`CREATE USER ...; GRANT ...`	`db.createUser({...})`
Begin a transaction	`BEGIN;`	`START TRANSACTION;`	`session.startTransaction()`
Upsert	`ON CONFLICT DO UPDATE`	`ON DUPLICATE KEY UPDATE`	`updateOne(f, u, {upsert: true})`
Generated id	`IDENTITY or SERIAL`	`AUTO_INCREMENT`	`ObjectId (default _id)`

Equivalent administrative tasks.

2.22.3 Isolation matrix

Level (engine)	Dirty read	Non-repeatable read	Phantom read	Lost update	Write skew
Read uncommitted (PostgreSQL: behaves as read committed)	no	yes	yes	yes	yes
Read committed (PostgreSQL default)	no	yes	yes	yes	yes
Repeatable read (PostgreSQL: snapshot isolation)	no	no	no	no (error 40001)	yes
Repeatable read (MySQL default)	no	no	not for plain reads	yes, unless locking reads	yes
Serializable (PostgreSQL: SSI)	no	no	no	no	no (retry on 40001)
Serializable (MySQL: locking reads)	no	no	no	no	no (deadlocks possible)
Snapshot (MongoDB transactions)	no	no	no	no (WriteConflict)	yes

Anomalies possible at each level (PostgreSQL 16 and MySQL InnoDB, as documented).

streram

2.22.4 Index decision table

Query shape	Index	Remark
Equality or range on one column	B-tree	Default; supports ORDER BY and prefix LIKE
Equality on several columns, range on the last	Composite B-tree, equality columns first	Leftmost prefix rule; MongoDB: equality, sort, range
Only some rows are queried (state = 'OPEN')	Partial index	Smaller and cheaper to maintain; PostgreSQL and MongoDB
Query on an expression (lower(email))	Expression index	The query must use the identical expression
Avoid visiting the table	Covering index (INCLUDE)	Index-only scan needs a current visibility map
Containment, arrays, full-text, JSONB	GIN	MongoDB: multikey and text indexes
Substring search (LIKE '%x%')	GIN with pg_trgm	A B-tree cannot help
Ranges, geometry, nearest neighbour	GiST (PostgreSQL), 2dsphere (MongoDB)	Spatial queries
Huge append-only table ordered by time	BRIN	Tiny index, coarse block ranges
Expire rows by age	none in SQL (partition and drop)	MongoDB: TTL index
Unique business rule	Unique index or constraint	Sharded MongoDB: must include the shard key

Choosing an index.

2.22.5 Reading EXPLAIN output

Node	Reading
`Seq Scan`	Reads the whole table; acceptable for small tables or when most rows are needed
`Index Scan`	Walks the index and fetches rows; good for few rows
`Index Only Scan`	Answers from the index; check Heap Fetches (0 is ideal)
`Bitmap Heap Scan`	Mid-sized result: index builds a bitmap, heap read in page order; Recheck Cond is normal
`Nested Loop`	Inner side probed per outer row; fast with an index and few outer rows, dangerous when loops is large
`Hash Join`	Hash table built on the smaller input; Batches above 1 means spill to disk (raise work_mem)
`Merge Join`	Both inputs sorted; efficient for large sorted inputs
`Sort`	Method: quicksort (memory) or external merge (disk)
`Aggregate and HashAggregate`	Grouping; Batches above 1 spills
`Gather and Parallel`	Worker processes; Workers Launched below Planned indicates contention

Plan nodes and what they imply (PostgreSQL).

The checklist for a plan: first compare `rows=` (estimate) with `actual rows=` at every node, because a factor of ten or more points to stale statistics or correlated columns (`ANALYZE`, extended statistics). Second, multiply `actual time` by `loops`. Third, read `Buffers: shared hit` and `read` to see cache behaviour. Fourth, look at `Rows Removed by Filter`: a large number means the index does not match the predicate. In MongoDB the corresponding checks are `COLLSCAN` versus `IXSCAN` and the ratios `totalDocsExamined` to `nReturned` and `totalKeysExamined` to `nReturned`.

2.22.6 mongosh crib

mongosh quick reference

```
use shop                                          // switch database
db.orders.insertOne({ _id: 1, status: 'NEW', items: [] })
db.orders.find({ status: 'NEW', 'items.sku': 'A-100' }, { status: 1 }).sort({ _id: -1 }).limit(5)
db.orders.updateOne({ _id: 1 }, { $set: { status: 'PAID' }, $inc: { version: 1 } })
db.orders.updateOne({ _id: 1 }, { $push: { items: { sku: 'B-205', qty: 1 } } })
db.orders.updateOne({ _id: 1, 'items.sku': 'B-205' }, { $inc: { 'items.$.qty': 1 } })
db.orders.updateOne({ sku: 'X' }, { $setOnInsert: { qty: 0 } }, { upsert: true })
db.orders.findOneAndUpdate({ _id: 1, status: 'NEW' }, { $set: { status: 'PAID' } },
                           { returnDocument: 'after' })
db.orders.deleteMany({ status: 'CANCELLED' })
db.orders.aggregate([ { $match: { status: 'PAID' } },
  { $group: { _id: '$customerId', revenue: { $sum: '$total' } } },
  { $sort: { revenue: -1 } }, { $limit: 10 } ])
db.orders.createIndex({ customerId: 1, createdAt: -1 })     // equality first, then sort/range
db.orders.createIndex({ createdAt: 1 }, { expireAfterSeconds: 86400 })   // TTL
db.orders.find({ customerId: 5434 }).explain('executionStats')
db.orders.getIndexes();  db.orders.stats();  db.orders.aggregate([{ $indexStats: {} }])
const s = db.getMongo().startSession(); s.startTransaction({ writeConcern: { w: 'majority' } })
rs.status();  rs.printSecondaryReplicationInfo();  rs.stepDown(60)
sh.status();  sh.shardCollection('shop.orders', { customerId: 'hashed' })
db.currentOp({ active: true, secs_running: { $gt: 5 } });  db.killOp(opid)
db.setProfilingLevel(1, { slowms: 100 });  db.system.profile.find().sort({ ts: -1 }).limit(5)
```

KEY POINTS

Reference tables help recall, not understanding; each row refers to a chapter that explains why.

- The isolation matrix differs between engines: PostgreSQL repeatable read is snapshot isolation, MySQL repeatable read is not.
- Choose an index from the query shape: equality columns first, then sort, then range.
- Read every plan by comparing estimated and actual rows before anything else.

2.23 Six-Week Study Plan

The plan assumes about two hours on weekdays and four hours on one weekend day, which is 14 to 16 hours per week. Each week has a theme, chapters to read by name, labs that must be executed (reading output is not a substitute for running it), and a checkpoint of interview questions from the question bank that must be answered aloud without notes. A week is complete when the checkpoint answers are correct and the labs produce the documented results on the learner's own machine.

sreram

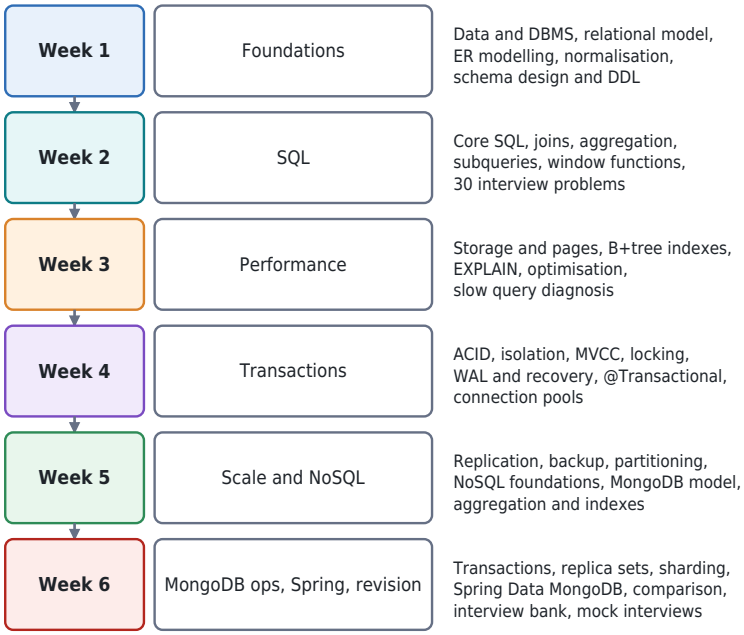


Figure 2.95. Six weekly stages progress from relational foundations through SQL, performance, transactions and scaling to MongoDB operations, Spring integration and revision.

2.23.1 The six weeks

Week	Chapters to read	Labs to run	Checkpoint
1	Data and DBMS; The relational model; Entity-relationship modelling; Normalisation; Schema design and DDL	Install PostgreSQL 16 with initdb; model the e-commerce schema; reproduce an update anomaly and fix it by normalising	Questions 1 to 10 and 23 to 30
2	SQL fundamentals; Joins; Aggregation; Subqueries and CTEs; Window functions; the 30 interview SQL problems	Load the sample schema; solve ten of the interview problems without hints; reproduce the NOT IN with NULL bug	Questions 11 to 22
3	Storage and pages; Indexes; Reading query plans; The query planner; Query optimisation; Slow query diagnosis	Generate one million rows; compare Seq Scan, Index Scan and Bitmap Scan; fix a function-on-column query; show an N+1 from Hibernate in the SQL log	Questions 31 to 42
4	Transactions and ACID; Isolation levels; MVCC; Locking; WAL and recovery; Transactions in Java and Spring	Reproduce a lost update and write skew with two psql sessions; cause and read a deadlock log; kill a postgres and watch recovery	Questions 43 to 60 and 101 to 103
5	Connections and pooling; Replication; Backup and PITR; Partitioning and sharding; Why NoSQL; Distributed data fundamentals; MongoDB model, CRUD, aggregation, indexes	Set up streaming replication; run a PITR restore; compute a quorum example; write five aggregation pipelines	Questions 61 to 88 and 89 to 93
6	MongoDB transactions, replication, sharding, storage and operations; Spring Data MongoDB; Relational versus non-relational; Putting it together	Run the JSONB benchmark; write the Spring order service; run mock interviews on the whole bank	Questions 94 to 120

Weekly plan mapped to chapters by name.

2.23.2 Method

1. Read a chapter once at normal speed, then close it and write the mechanism from memory in five lines; compare with the text.
2. Run every lab; change one parameter and predict the result before running again.
3. At the end of each day answer three questions from the bank aloud, then check the model answers; repeat the failures two days later.
4. In the last two days of week 6 give each scenario question as a procedure: observe (which tool), narrow (which query or plan), fix (immediate and permanent) and prevent.
5. Keep a one-page error log of every wrong answer with the correct mechanism; review it before each mock interview.

KEY POINTS

A plan works only when each week ends with a checkpoint that is answered without notes.

- Labs convert knowledge into evidence: run them, vary them, predict the result.
- Revisit weak questions after two days and again after a week.
- Treat scenario questions as diagnosis procedures with commands, not as definitions.

2.24 Putting It Together: One Order Through the Whole Data Layer

The preceding chapters examined components separately. This chapter follows one request, `POST /orders`, from the HTTP layer to every system that stores a trace of it: the connection pool, the PostgreSQL transaction, the index structures, the write-ahead log, the replica, the cache and the MongoDB event log. The PostgreSQL part is a real trace obtained on the build machine; the Spring, Redis and MongoDB parts follow the documented behaviour and are labelled as such. The goal is that the reader can say, for any statement in the request, which structure it touches and what fails if that structure is slow or unavailable.

2.24.1 Concept and mechanism

The request path

The design is the common one for a Java service: the relational database is the system of record and the other stores are derived from it.

1. **Spring Boot receives the request.** The controller validates the body and calls a service method annotated `@Transactional`. The proxy asks the transaction manager to begin a transaction.
2. **The pool lends a connection.** HikariCP returns an idle connection in microseconds, or waits up to `connectionTimeout` when all are in use. The connection is bound to the thread until the method ends, so everything done in the method is one database transaction.
3. **PostgreSQL executes the statements.** `INSERT INTO orders`, two `INSERT INTO order_items` and an `UPDATE stock`. Each statement searches indexes (B-tree descents), checks foreign keys (row locks on the referenced customer and products) and writes new tuple versions to heap pages in shared buffers.
4. **The WAL records the changes.** Every page modification generates a WAL record before the page may be written. At `COMMIT` the commit record is appended and the log is flushed to disk (`fsync`) before the client receives the acknowledgement.
5. **The replica replays the WAL.** A walsender streams the records to the standby, which replays them; reads from the replica are behind by the replication lag.
6. **The cache is invalidated.** The service deletes the cache key of the affected product's stock so that the next read reloads it (cache-aside).
7. **An event is written to MongoDB.** The order event is stored in the document store for history and analytics, written with `w: "majority"` and keyed by the outbox id so that a repeated delivery is harmless.

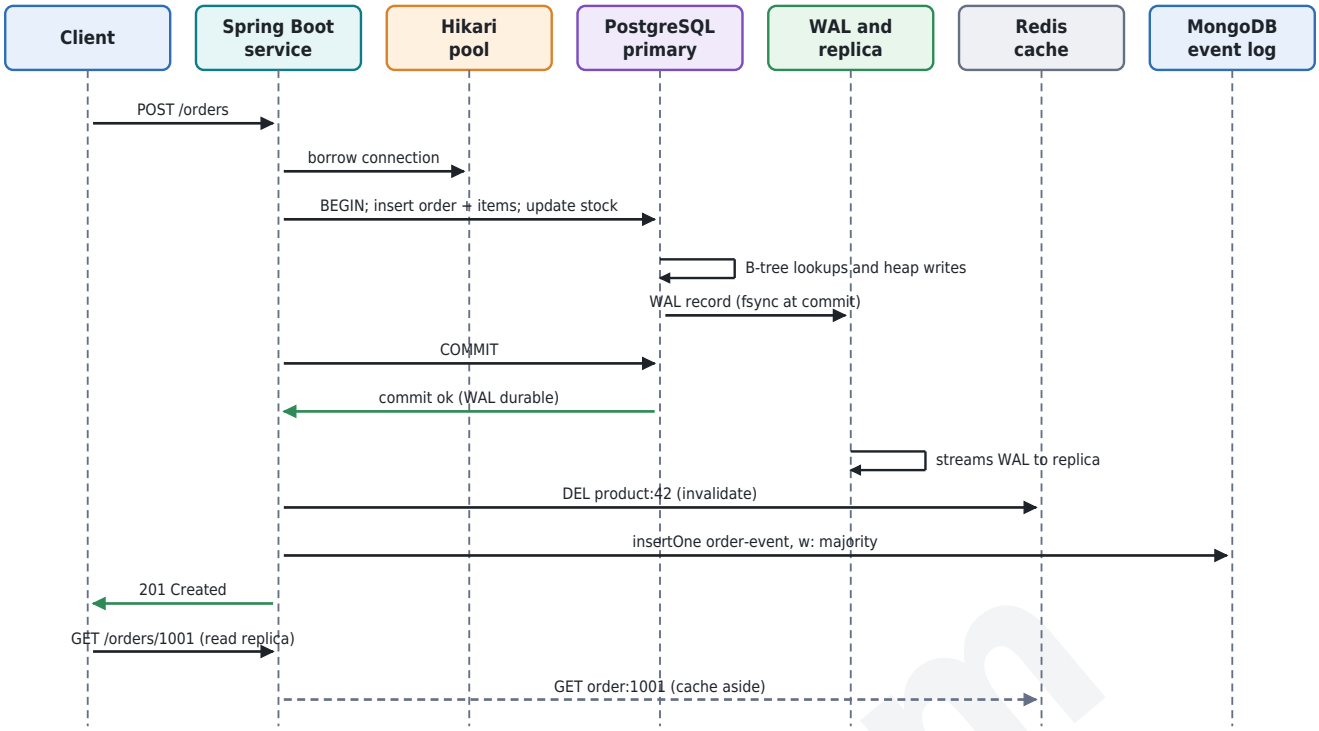


Figure 2.96. One order crosses the pool, the PostgreSQL transaction with its write-ahead log, the replica, the cache and the MongoDB event log; the commit is acknowledged to the client only after the WAL flush.

Consistency between the stores

The transaction in step 3 and the event in step 7 are in different systems, so they cannot commit atomically without two-phase commit. Writing the event directly after the commit leaves a failure window: if the process crashes between the commit and the MongoDB write, the order exists and the event does not. The outbox pattern removes the window: the service inserts an `outbox` row in the same PostgreSQL transaction as the order, and a relay process reads unpublished rows and writes them to MongoDB with an idempotent upsert, then marks them published. The guarantee is at-least-once delivery with idempotent consumers, which in effect gives exactly-once results.

2.24.2 Worked example

The transaction below is the real one executed on the build machine (PostgreSQL 16, database `shop` with 500,000 orders). The WAL position was read before and after, and `pg_waldump` printed the records in between.

The order transaction SQL

```sql
BEGIN;
INSERT INTO orders VALUES (600001, 5434, 'NEW', now());
INSERT INTO order_items VALUES (600001, 1, 235, 2, 785), (600001, 2, 252, 1, 424);
UPDATE stock SET qty = qty - 2 WHERE sku = 'A-100';
COMMIT;
```

WAL generated by the transaction (real output) TERMINAL

```
$ psql shop -c "SELECT pg_wal_lsn_diff(:'after', :'before') AS wal_bytes"
 wal_bytes
-----------
     47536
$ pg_waldump --stats=record -s 0/647D5DC0 -e 0/647E1770
Type                   N  Record size  FPI size  Combined size
Transaction/COMMIT     1           34         0             34
Standby/RUNNING_XACTS  1           54         0             54
Heap/INSERT            2          130      6472           6602
Heap/INSERT+INIT       1           76         0             76
Heap/HOT_UPDATE        1           71         0             71
Heap/LOCK              4          226     16328          16554
Btree/INSERT_LEAF      6          329     23612          23941
Total                 16          920     46412          47332
```

The numbers deserve a careful reading. The transaction wrote 16 WAL records, but the record headers and payloads total only 920 bytes; 98 percent of the 47,332 bytes are full-page images (FPI). PostgreSQL writes a complete copy of a page into the WAL the first time the page is modified after a checkpoint, to protect against torn page writes during a crash (`full_page_writes`). Because these pages (index leaf pages and heap pages of a 500,000-row database) had not been touched since the last checkpoint, nearly every record carried an 8 KB image. A repeated transaction in the same checkpoint interval generated 960 bytes in a second measurement (see the hands-on section). This explains two operational observations: WAL volume spikes right after each checkpoint, and a shorter `checkpoint_timeout` increases WAL volume.

The individual records can be assigned to the statements.

No.	Record	Meaning
1	Heap/INSERT	New order row in the orders heap page
2	Standby/RUNNING_XACTS	Background snapshot of running transactions for the standby, unrelated to the order
3-4	Btree/INSERT_LEAF (two)	New entries in the two indexes of orders: primary key and customer_id
5	Heap/LOCK	Foreign key check on the referenced customer row (share-key lock, written as a WAL record)
6	Heap/INSERT+INIT	First order item row; the heap page was new, so it is initialised
7-8	Btree/INSERT_LEAF (two)	Entries in the primary key and product_id indexes of order_items
9	Heap/INSERT	Second order item row
10-11	Btree/INSERT_LEAF (two)	Entries of the second item in the same two indexes
12-14	Heap/LOCK (three)	Foreign key checks on referenced rows
15	Heap/HOT_UPDATE	Stock update; the new tuple version stays on the same page and needs no index entry (heap-only tuple)
16	Transaction/COMMIT	The commit record; the flush of this record makes the order durable

The 16 records in order (real trace, interpreted).

Three consequences follow for the Java developer. First, a plain `INSERT` of a row with two indexes and two foreign keys costs three WAL records and several buffer accesses, which is why batch inserts and fewer indexes speed up writes. Second, the stock update was a HOT update because `qty` is not indexed; indexing a frequently updated column would have added an index record per update. Third, the foreign key checks lock the referenced rows in a share mode, so a long transaction that inserts order items holds share locks on the products and blocks any `UPDATE` that changes the product key or deletes the product.

2.24.3 Hands-on

Reproducing the trace

Steps to reproduce on a local PostgreSQL 16 (commands as run on the build machine) TERMINAL

```
$ psql shop -c "SELECT pg_current_wal_lsn()"
$ psql shop -f order_txn.sql
$ psql shop -c "SELECT pg_current_wal_lsn()"
$ pg_waldump -p $PGDATA/pg_wal -s START_LSN -e END_LSN
$ pg_waldump --stats=record -p $PGDATA/pg_wal -s START_LSN -e END_LSN
```

Replace `START_LSN` and `END_LSN` with the two positions printed by the queries. To see the effect of full-page images, run `CHECKPOINT` and repeat the transaction twice: the first run after the checkpoint carries images, the second does not. Real measurement on the build machine, with new order ids 600002 and 600003:

WAL bytes of the same transaction before and after warming the pages (real output) TERMINAL

```
$ psql shop -c "SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), :'b') AS first_after_ckpt"
 first_after_ckpt
------------------
            47936
$ psql shop -c "SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), :'b2') AS second"
 second
--------
    960
```

The second execution wrote 960 bytes, about fifty times less than the first.

The service with an outbox (unverified)

OrderPlacement.java (fragment, unverified: Spring and JDBC not available on the build machine) JAVA

```java
@Service
public class OrderPlacement {

    private final JdbcTemplate jdbc;
    private final ObjectMapper json;

    public OrderPlacement(JdbcTemplate jdbc, ObjectMapper json) {
        this.jdbc = jdbc;
        this.json = json;
    }

    @Transactional
    public long place(long customerId, List<Line> lines) throws JsonProcessingException {
        Long orderId = jdbc.queryForObject(
            "INSERT INTO orders (customer_id, status, created_at) "
          + "VALUES (?, 'NEW', now()) RETURNING id",
            Long.class, customerId);
        for (int i = 0; i < lines.size(); i++) {
            Line l = lines.get(i);
            jdbc.update("INSERT INTO order_items (order_id, line_no, product_id, qty, unit_price) "
                      + "VALUES (?, ?, ?, ?, ?)",
                      orderId, i + 1, l.productId(), l.qty(), l.unitPrice());
            int changed = jdbc.update("UPDATE stock SET qty = qty - ? WHERE sku = ? AND qty >= ?",
                                      l.qty(), l.sku(), l.qty());
            if (changed == 0) {
                throw new IllegalStateException("insufficient stock for " + l.sku());
            }
        }
        String payload = json.writeValueAsString(Map.of("orderId", orderId, "lines", lines.size()));
        jdbc.update("INSERT INTO outbox (aggregate_id, type, payload) "
                  + "VALUES (?, 'OrderPlaced', ?::jsonb)", orderId, payload);
        return orderId;
    }

    public record Line(long productId, String sku, int qty, java.math.BigDecimal unitPrice) {}
}
```

What to notice: the order, the stock updates and the outbox row commit together or not at all; the checked `JsonProcessingException` is declared, and because checked exceptions do not trigger rollback by default, the only exception that must abort (insufficient stock) is a `RuntimeException`; the conditional `UPDATE` makes the stock check atomic. The relay that moves outbox rows to MongoDB and invalidates caches runs outside this transaction.

OutboxRelay.java (fragment, unverified) JAVA

```java
@Scheduled(fixedDelay = 500)
public void publish() {
    List<OutboxRow> rows = jdbc.query(
        "SELECT id, aggregate_id, payload FROM outbox WHERE published_at IS NULL "
      + "ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED", mapper);
    for (OutboxRow row : rows) {
        events.replaceOne(Filters.eq("_id", row.id()), Document.parse(row.payload())
                .append("_id", row.id()), new ReplaceOptions().upsert(true));   // idempotent
        jdbc.update("UPDATE outbox SET published_at = now() WHERE id = ?", row.id());
    }
}
```

The relay uses `FOR UPDATE SKIP LOCKED` so that several relay instances share the work without double publishing, and `replaceOne` with `upsert` keyed by the outbox id makes a redelivery after a crash between the MongoDB write and the `UPDATE` harmless. The method runs in a transaction (annotate it `@Transactional`) so that the row locks last until the published marker is written.

2.24.4 Real-world problems and solutions

Incident: order committed but event missing in MongoDB

Scenario. The service writes the order in PostgreSQL and then calls `events.insertOne(...)` directly in the same method. A deployment kills the pod between the two calls.

Symptoms. The order history view (read from MongoDB) lacks a few orders that exist in PostgreSQL; reconciliation jobs show a small, constant percentage of missing events around each deployment or MongoDB failover.

Diagnosis.

1. Compare counts per day: `SELECT date(created_at), count(*) FROM orders GROUP BY 1` against `db.events.aggregate([{ $group: { _id: { $dateTrunc: { date: '$createdAt', unit: 'day' } }, n: { $sum: 1 } } }])`.
2. Find missing ids by key range comparison and check the application log for the window around restarts and MongoDB elections (`NotWritablePrimary` errors).
3. Inspect the code path: the event write happens after `COMMIT` or inside the transaction, outside any retry.

Root cause. Two stores cannot commit atomically. A crash or an error between the two writes leaves the stores inconsistent; writing the event before the commit leaves the opposite inconsistency (an event for an order that rolled back).

Fix. Immediate: backfill the missing events from PostgreSQL with an idempotent script using `replaceOne` with upsert. Permanent: introduce the outbox table and relay shown above, with at-least-once delivery.

Prevention. A scheduled reconciliation that compares counts and checksums per day and alerts above a threshold; a deployment drill that kills the service during order placement.

Incident: stale product stock displayed after an order

Scenario. The product page reads stock through a cache-aside Redis key and, for uncached reads, from a PostgreSQL replica. After a purchase, the page keeps showing the old quantity for several seconds, and sometimes for a minute.

Symptoms. Customers see an item as available after the last unit was sold; support receives complaints about overselling messages at checkout.

Diagnosis.

1. Check the order of operations: the service invalidates the key after commit, but a concurrent reader may reload the old value from the replica and repopulate the key.
2. Measure replication lag: `SELECT now() - pg_last_xact_replay_timestamp()` on the replica (typical output: 00:00:01.8).
3. Check the cache time to live (TTL) and whether keys are deleted or overwritten.

Root cause. The reader that reloads the key after the invalidation reads from a replica that has not yet replayed the commit, and stores the stale value with a long TTL. The invalidation happened and was then overwritten.

Fix. Immediate: shorten the TTL for stock keys. Permanent: read stock for display from the primary or invalidate with a short delayed second delete; store a version or LSN with the cached value and refuse older values; keep the authoritative stock check in the conditional `UPDATE` at checkout, so the display may be stale but overselling cannot happen.

Prevention. Treat cache and replica reads as stale by design; never make a business decision on cached stock.

2.24.5 Questions to expect

Questions and model answers

1. **What happens between `COMMIT` and the acknowledgement in PostgreSQL?** The commit record is appended to the WAL and flushed to disk (`fsync`, with `synchronous_commit = on`); with synchronous replication the standby must also confirm. Only then does the client receive success.
2. **Why was most of the WAL volume in the experiment full-page images?** The first modification of a page after a checkpoint logs the entire page to protect against torn writes; later changes to the same page log only small deltas.
3. **Why do foreign keys appear in the WAL?** The check locks the referenced row in share mode, and the lock is recorded as a tuple lock record.

4. **What is a HOT update?** A heap-only tuple update: the new version fits on the same page and no indexed column changed, so no index entries are added.
5. **How do you keep PostgreSQL and MongoDB consistent?** The outbox pattern: write the event row in the business transaction, publish it afterwards with an idempotent write; this yields at-least-once delivery without two-phase commit.
6. **Why can a cache show stale data after an invalidation?** A concurrent reader can repopulate the key from a lagging replica after the delete. Shorter TTLs, versioned values and primary reads for critical views mitigate it.
7. **Where should the stock check be made?** In the database, as a conditional atomic update at the moment of purchase; displays may be stale, the decision may not.
8. **Which failures does each layer tolerate?** The pool: slow queries (bounded wait). The WAL: a crash of the database process. The replica: loss of the primary host. The outbox: a crash between systems. The majority write concern: failover of the MongoDB primary.

KEY POINTS

A request touches many structures; each has a failure mode that the design must name.

- The commit is durable when the WAL record is flushed; data pages follow later.
- Full-page images dominate WAL right after a checkpoint, and HOT updates avoid index writes.
- Separate stores need an outbox and idempotent consumers, not hope.
- Caches and replicas are stale by design; decisions are made in the primary with a conditional write.

sreeram